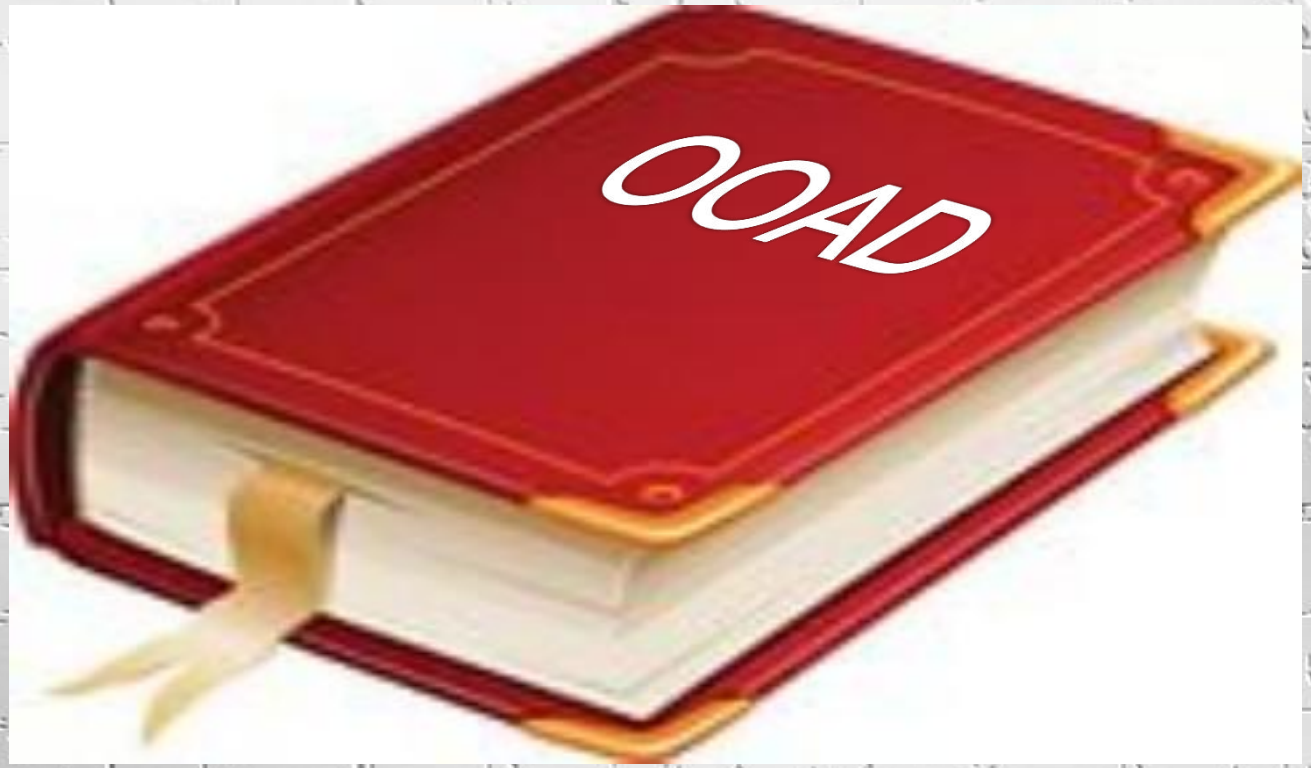
ТЕХНОЛОГИИ
РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

ТРПО

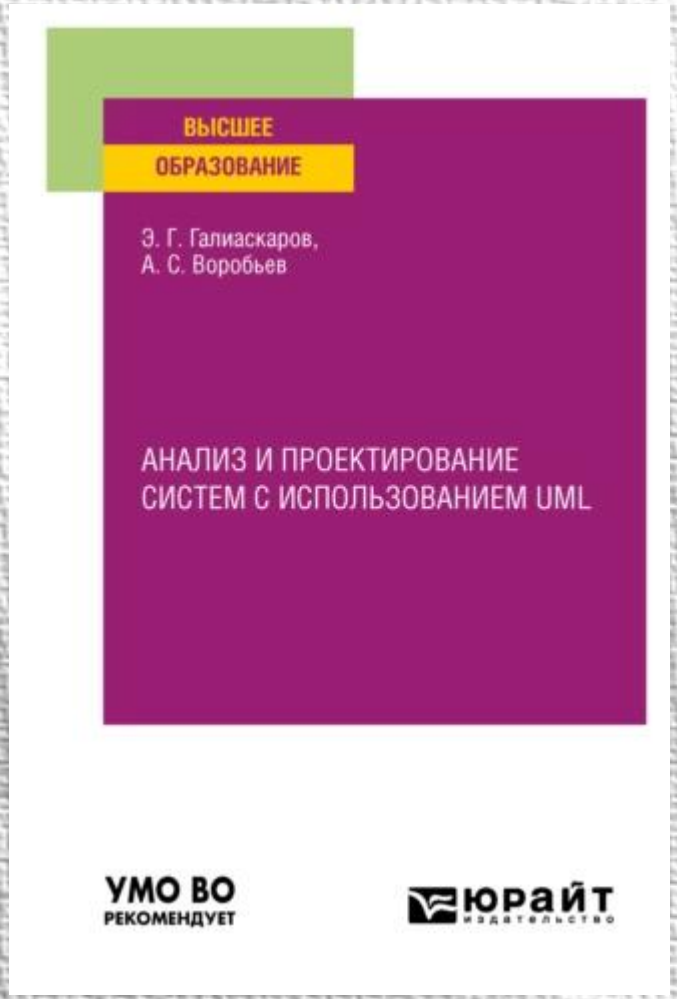
(Программная инженерия)

Часть 5. Объектно-ориентированный анализ и проектирование

© Ржавин В.В.



ЛИТЕРАТУРА

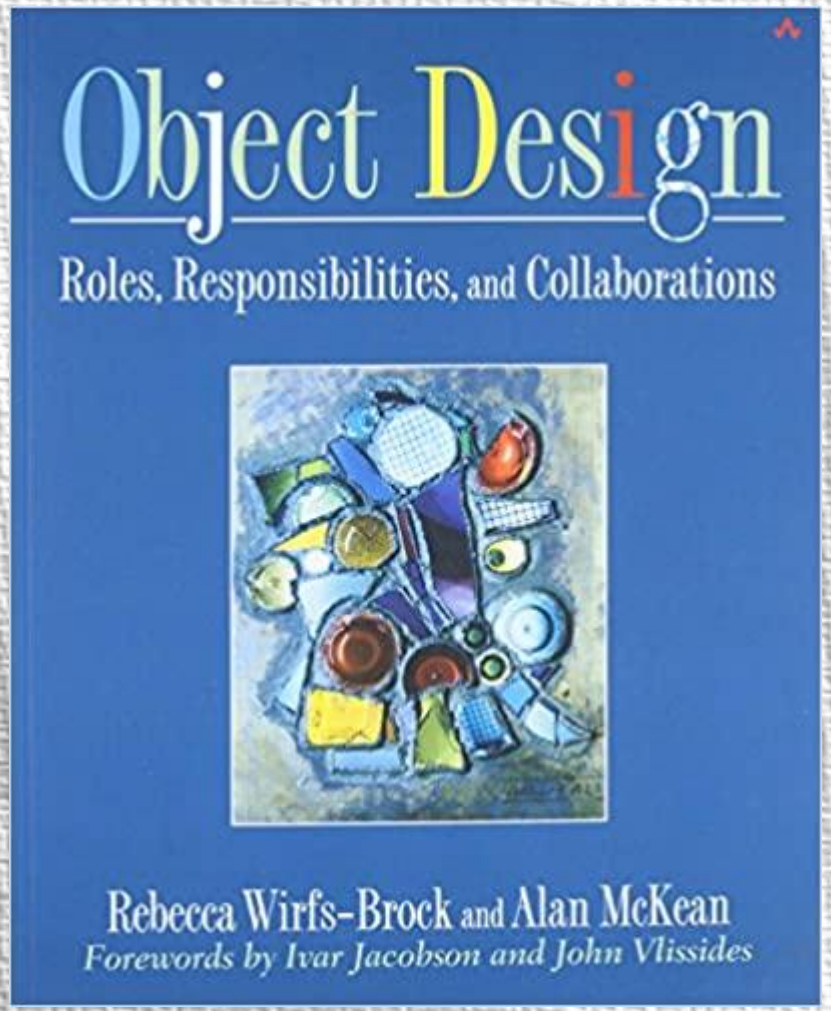


Э. Г. Галиаскаров, А. С. Воробьев. Анализ и проектирование систем с использованием UML. Учебное пособие для вузов. 2021

Данное пособие представляет собой практическое руководство по использованию UML для разработки программных систем. Оно позволит научиться выявлять основные понятия предметной области и разрабатывать красивые диаграммы классов, описывать функциональные требования в виде спецификаций вариантов использования и превращать их в правильные проектные решения. Дополнительно в пособии рассмотрена работа с замечательным CASE-средством Visual Paradigm и средой разработки MDriven. Соответствует актуальным требованиям федерального государственного стандарта высшего образования. Для студентов высших учебных заведений, обучающихся по ИТ-направлениям, а также преподавателей и всех интересующихся.

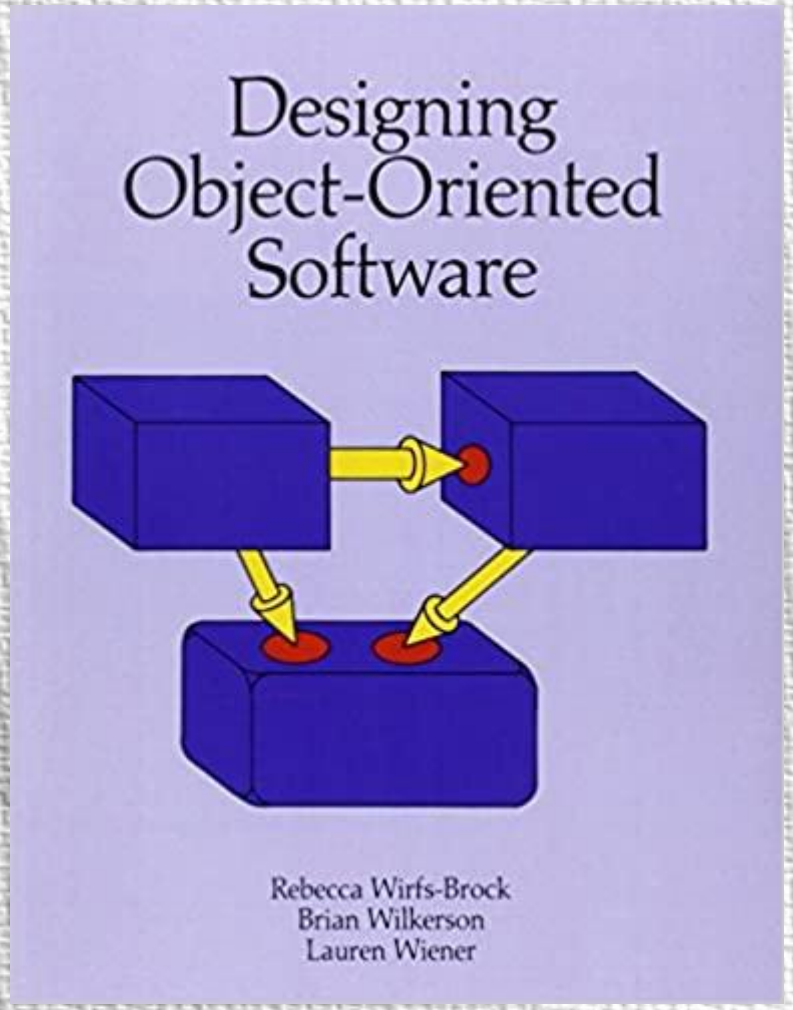
В книге «***Microsoft.NET: архитектура корпоративных приложений***», написанной ***Дино Эспозито*** и ***Андреа Салтарелло***, рассказывается от начала и до конца как применять шаблоны и лучшие методики для управления сложными проектами, как облегчить их сборку и упростить поддержку и дальнейшую модернизацию.

Разработка эффективной архитектуры — лучшая стратегия борьбы со сложностью проекта, которая приводит к улучшению результатов. Книга «***Microsoft.NET: архитектура корпоративных приложений***» поможет Вам принять правильные архитектурные решения и улучшить качество и надежность Ваших приложений на платформе ***Microsoft.NET***



Проектирование объектов: роли, обязанности и совместная работа Ребекка Вирфс-Брок, Алан Маккин

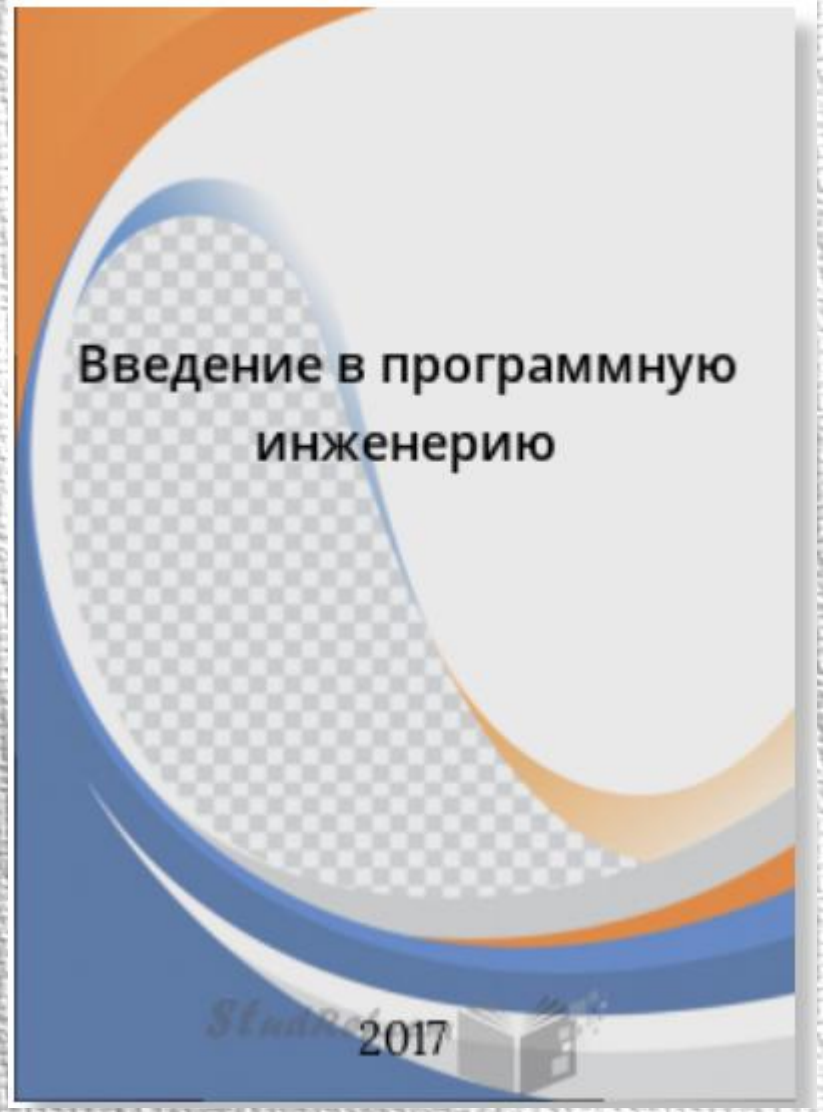
Фокусируется на практике проектирования объектов как неотъемлемых членов сообщества, где каждый объект имеет определенные роли и обязанности. Эта работа включает в себя практики и методы ответственного проектирования и показывает, как применять их при разработке современных объектно-ориентированных приложений. Он подходит для разработчиков программного обеспечения и студентов.



Проектирование объектно-ориентированного программного обеспечения

Ребекка Вирфс-Брок, Брайан Уилкерсон, Лорен Винер

Этот бестселлер исследует искусство проектирования объектно-ориентированного программного обеспечения. Он предлагает основные принципы проектирования и конкретный процесс проектирования, который может быть применен к любым усилиям по программированию программного обеспечения - даже к тем, которые не используют объектно-ориентированные языки программирования или среды. **КЛЮЧЕВЫЕ ТЕМЫ:** Охватывает концепции объектно-ориентированной технологии, представляет процесс применения этих концепций, инструменты для использования на протяжении всего процесса и примеры, чтобы собрать все это вместе.



- https://studref.com/320259/informatika/vvedenie_v_programmnuyu_inzheneriyu

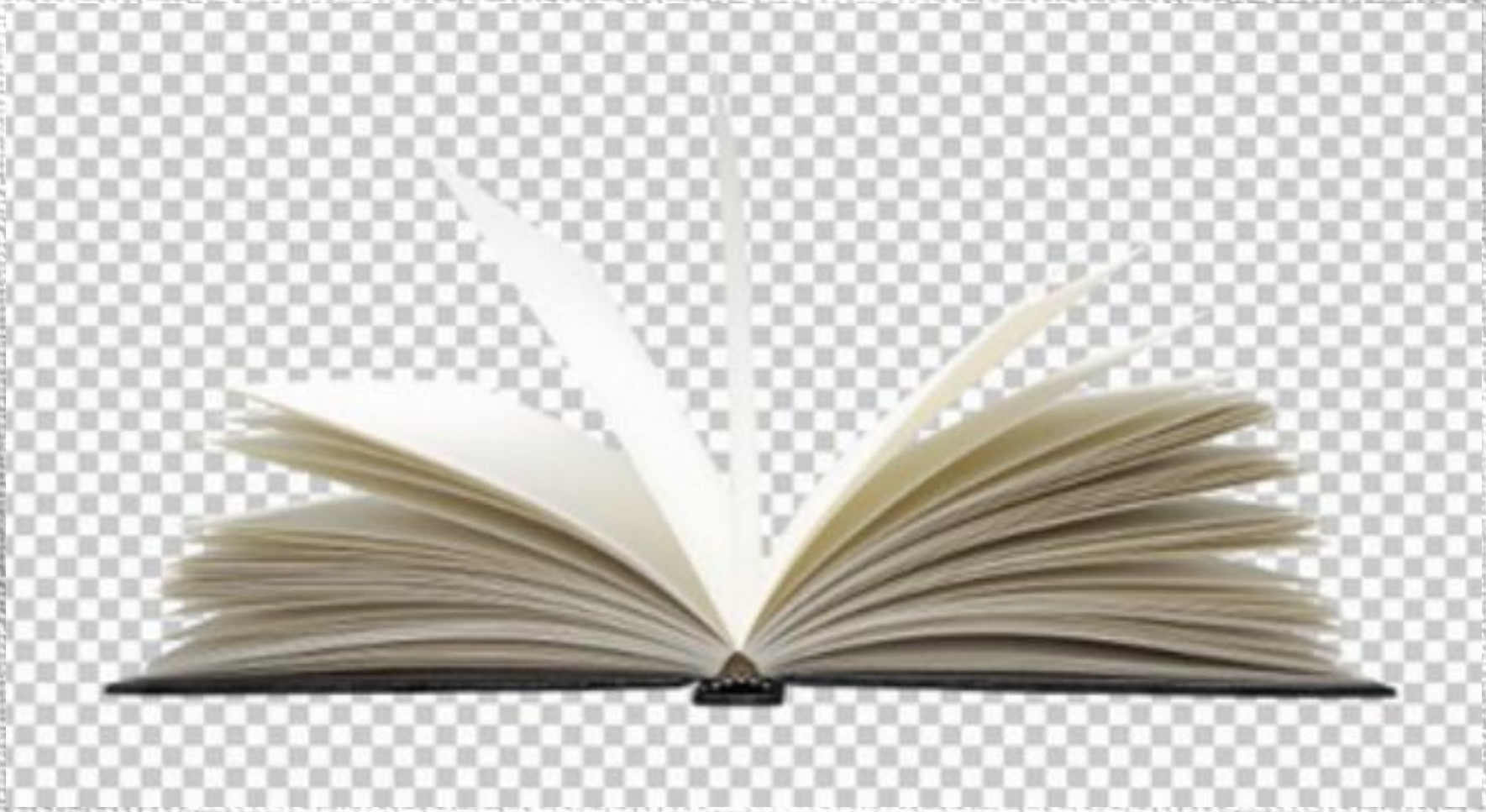
Книга посвящена методологии DDD (предметно-ориентированному проектированию), что особенно актуально в условиях дробления предметных областей и усложнения бизнес-взаимодействий. Рассказано, как оценить масштаб и сложность предметной области, измерить темпы ее развития, учесть необходимые зависимости, применять событийно-ориентированную архитектуру и структурировать создаваемое ПО, эффективно вписывая его в сеть данных (Data Mesh). Материал будет особенно интересен при развитии стартапа и разработке наукоемких отраслевых систем.

справочник

СОДЕРЖАНИЕ

- Аутентификация, авторизация и идентификация
- Управление конфигурацией
- Управление исключениями
- Модули и компоненты
- Маршалинг и сериализация

Аутентификация, авторизация и идентификация



Идентификация

Идентификация (от латинского identifico — отождествлять): присвоение субъектам и объектам идентификатора и/или сравнение идентификатора с перечнем присвоенных идентификаторов. Например, представление человека по имени отчеству - это идентификация.

Аутентификация и авторизация

Аутентификация и авторизация – две ключевые функции сервисной инфраструктуры для защиты конфиденциальных данных и операций от несанкционированного доступа со стороны злоумышленников.

Хотя эти два термина используются в одном контексте, они представляют собой принципиально разные понятия, поскольку осуществляют защиту взаимодополняющими способами.

Аутентификация

Аутентификация используется для подтверждения подлинности личности зарегистрированного пользователя. Проверка подлинности – это процесс проверки учетных данных: идентификатора пользователя (имени, адреса электронной почты, номера телефона) и пароля.

Если идентификатор и пароль совпадают с записями, хранящимися в базе данных системы, пользователю предоставляется доступ. В случае неправильного ввода данных программа вызывает предупреждение безопасности и блокирует вход. Если неудачных попыток будет несколько, система заблокирует саму учетную запись.

Авторизация

Авторизация является функцией определения прав доступа к ресурсам и управления этим доступом. Авторизация — это не то же самое что идентификация и аутентификация: идентификация — это называние лицом себя системе; аутентификация — это установление соответствия лица названному им идентификато-ру; а авторизация — предоставление этому лицу возможностей в соответствие с положенными ему правами или проверка наличия прав при попытке выполнить какое-либо действие. Например, авторизацией являются лицензии на осуществление определённой деятельности.

Происходит после того, как личность пользователя успешно аутентифицируется системой. Процесс авторизации определяет, имеет ли прошедший проверку человек доступ к определенным ресурсам: информации, файлам, базе данных. Факторы проверки подлинности, необходимые для авторизации, могут различаться в зависимости от уровня безопасности.

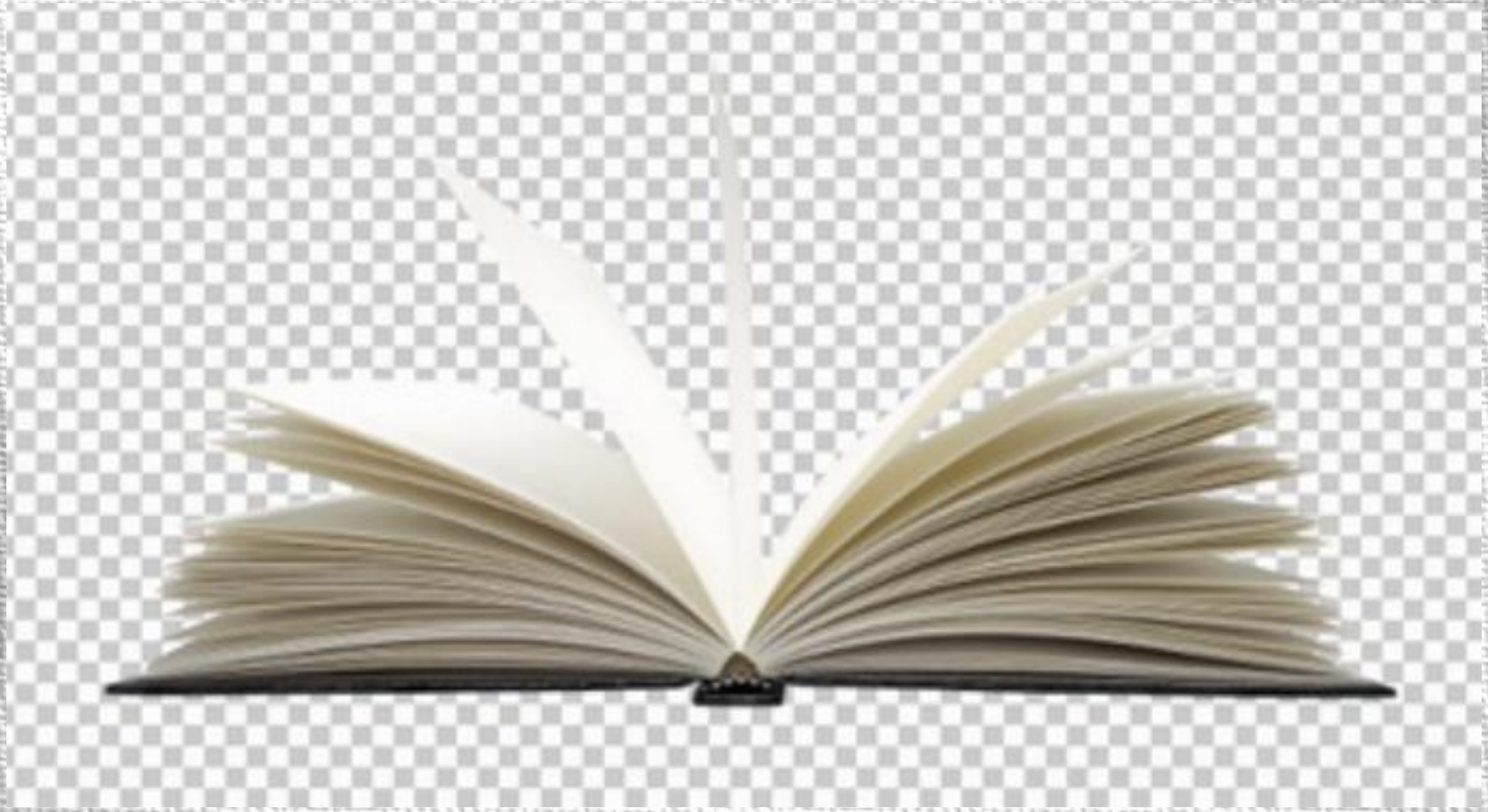
Например, процесс проверки и подтверждения идентификаторов сотрудников и паролей в организации называется аутентификацией, но определение того, какой сотрудник имеет доступ к определенным ресурсам, называется авторизацией.

- Предположим, что вы путешествуете и собираетесь сесть на самолет. Когда вы предъявляете свой билет и удостоверение личности перед регистрацией, то получаете посадочный талон, который подтверждает, что администрация аэропорта удостоверила вашу личность. Но это не все. Чтобы получить доступ к внутренней части самолета и его ресурсам, вам необходимо получить на посадку.
- Разрешение бортпроводника - это и есть авторизация.

Итак,

- Вы кто такой? Я – Сидоров
  - Это **идентификация** (И)
- А как докажете? Вот мой паспорт
  - Это **аутентификация** (А)
- В БД системы должен храниться как идентификатор, так и аутентификатор. Именно совпадение А и И дает доступ к данным.
- А что разрешено вам делать с данными?
- К каким ресурсам имеете доступ?
  - На этот вопрос ответит **авторизация**

Управление конфигурацией



Конфигурация — это знание о том, какова работающая (актуальная) система, выделенная из множества ее возможных состояний. Конфигурация находится под контролем, если конфигурация определения системы соответствует конфигурации воплощения системы. Если какие-то части этих конфигураций не соответствуют друг другу, то говорят о конфигурационных коллизиях.

Управление конфигурацией (configuration management) — техническая дисциплина системной инженерии, обеспечивающая поддержание надлежащей (задуманной, одобренной) конфигурации системы во время всего её жизненного цикла. Если говорить попроще, то управление конфигурацией — это практика, обеспечивающая на протяжении всего жизненного цикла

- совместимость версий (отсутствие коллизий!) и
- полноту частей системы (отсутствие коллизий!).

Конфигурационное управление (англ. software configuration management, SCM) в программной инженерии — комплекс методов, направленных на систематический учёт изменений, вносимых разработчиками в программный продукт в процессе его разработки и сопровождения, сохранение целостности системы после изменений, предотвращение нежелательных и непредсказуемых эффектов, формализацию процесса внесения изменений.

В целом, конфигурационное управление отвечает на вопрос:
«Кто-то уже сделал нечто, как нам это воспроизвести?»

К процедурам можно отнести создание резервных копий, контроль исходного кода, требований проекта, документации и т.д. Степень формальности выполнения данных процедур зависит от размеров проекта, и при правильной её оценке данная концепция может быть очень полезна.

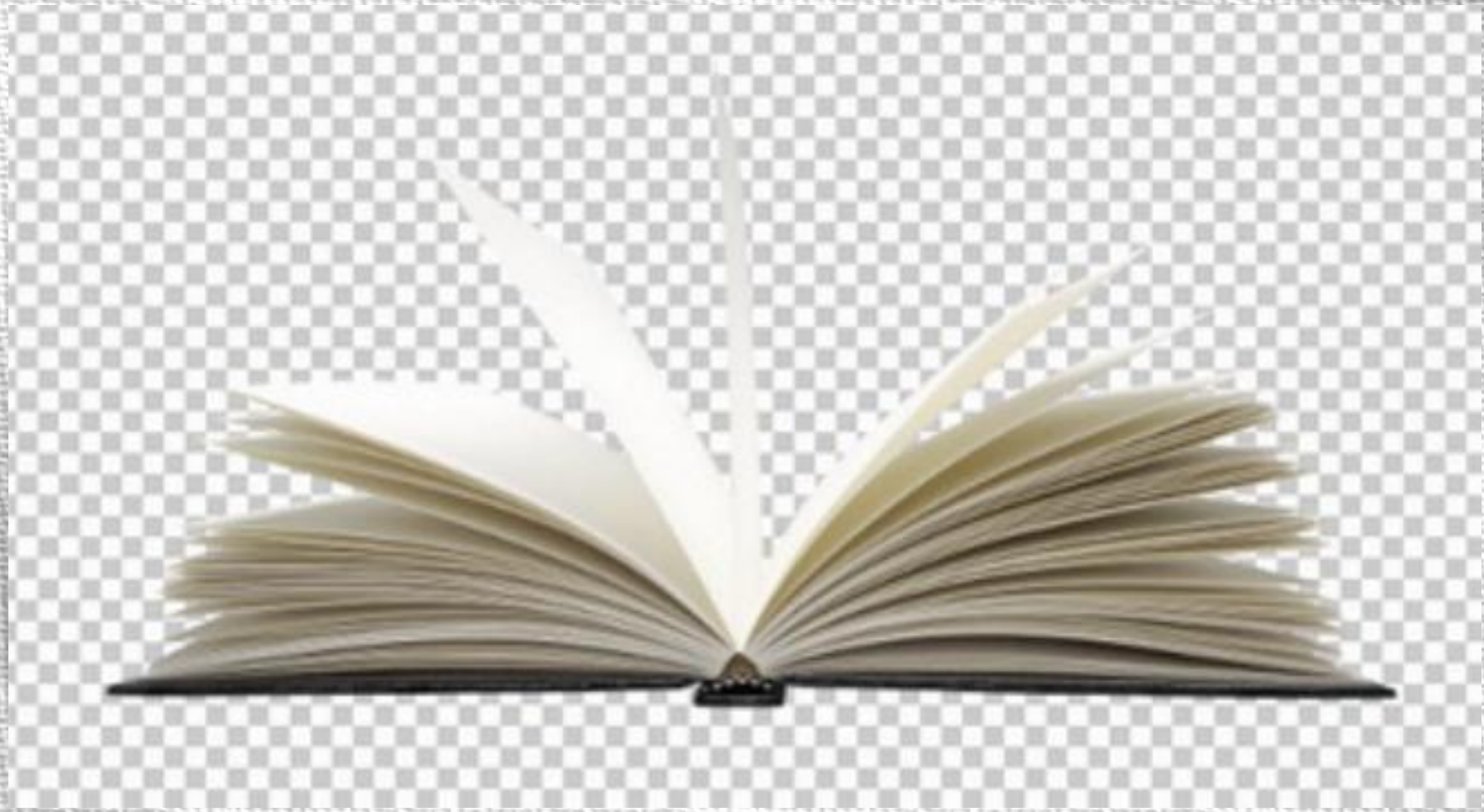
Задачи конфигурационного управления

- контроль конфигурации: контроль над изменениями материалов;
- учёт текущего состояния: состояние документов, состояние кода, состояние отдельных задач и всего проекта в целом;
- управление процессом разработки;
- управление сборкой;
- управление окружением;
- отслеживание задач и проблем (в частности, отслеживание ошибок);

Процедуры управления конфигурацией

- **Ревизия конфигурации** — процесс проверки соответствия документа нижнего уровня всем требованиям верхнего.
- **Аудит конфигурации** — процесс проверки соответствия готового продукта или его части документации.
- **Контроль конфигурации** — процесс, при котором все предлагаемые изменения продукта проходят одобрение специальной группы (или отдельного человека). Одна из функций такой группы — контроль актуальности всех имеющихся документов, а также контроль того что все изменения сначала вносятся в документацию, а уже затем в объект изменения.
- **Учет состояния конфигурации** — процесс подготовки отчетов о текущем состоянии продукта и состоянии утверждённых изменений.

Управление исключениями



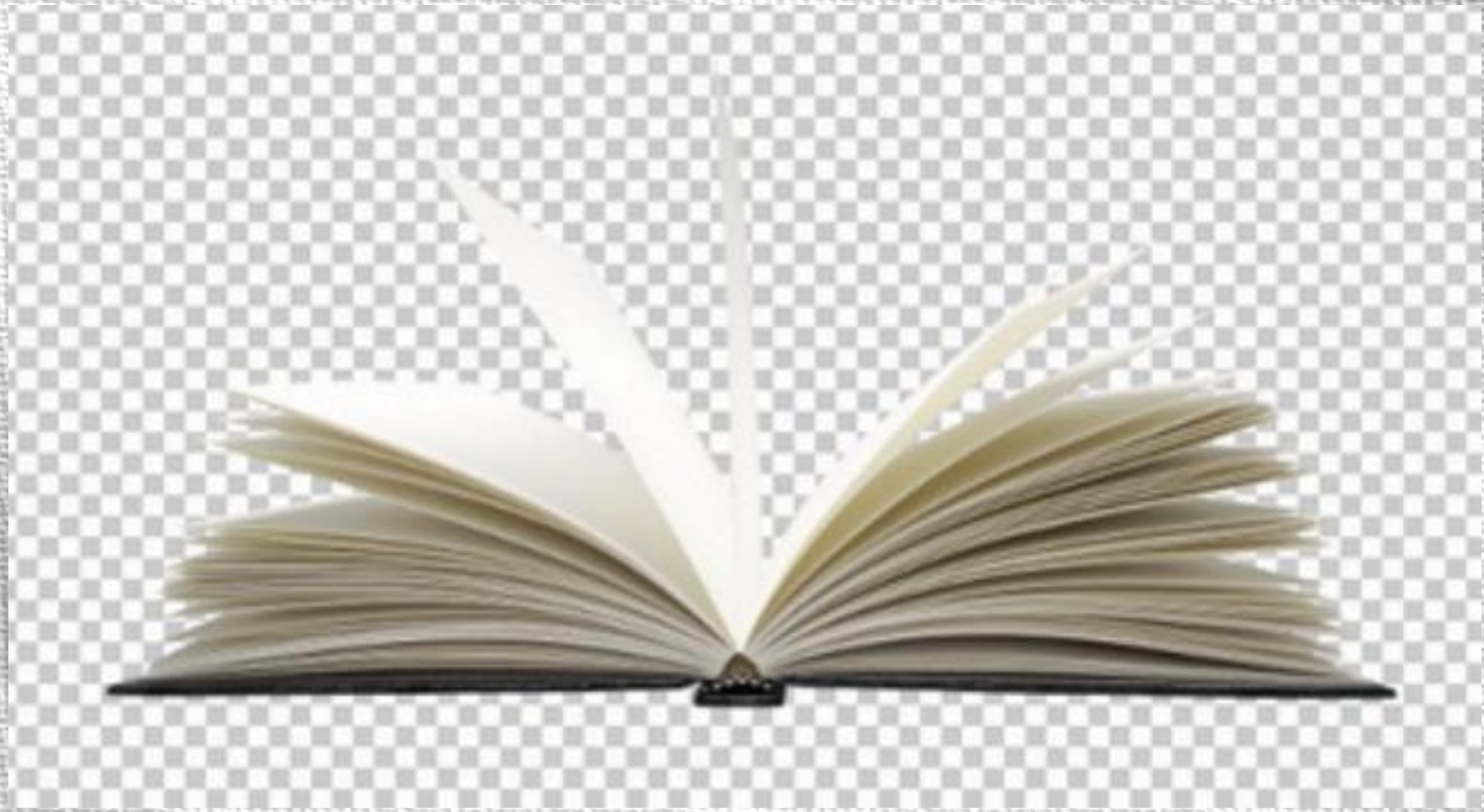
Обработка исключительных ситуаций (англ. exception handling) — механизм языков программирования, предназначенный для описания реакции программы на ошибки времени выполнения и другие возможные проблемы (исключения), которые могут возникнуть при выполнении программы и приводят к невозможности (бессмысленности) дальнейшей отработки программой её базового алгоритма. В русском языке также применяется более короткая форма термина: «*обработка исключений*».

Классические примеры подобных ситуаций приведены ниже.

- **Целочисленное деление на ноль**. Конечного результата у данной операции быть не может, поэтому ни дальнейшие вычисления, ни попытка использования результата деления не приведут к решению задачи.
- **Ошибка при попытке считать данные с внешнего устройства**. Если данные не удаётся получить, любые дальнейшие запланированные операции с ними бессмысленны.
- **Исчерпание доступной памяти**. Если в какой-то момент система оказывается не в состоянии выделить достаточный для прикладной программы объём оперативной памяти, программа не сможет работать нормально.

- **Появление сигнала аварийного отключения электропитания системы**. Прикладную задачу, по всей видимости, решить не удастся, в лучшем случае (при наличии какого-то резерва питания) прикладная программа может позаботиться о сохранении данных.

Модули и компоненты



Что такое модуль

Модуль - это небольшая автономная часть компьютерной программы, которая выполняет определенную работу. Это как строительный блок в разработке программного обеспечения, фокусирующийся на одной конкретной задаче. Модули созданы для независимой работы, что упрощает понимание кода и управление им. Они помогают организовать сложные программы, разбивая их на более мелкие части. Разработчики создают модули для использования одного и того же кода в разных частях программы или других проектах.

Что такое программный компонент?

Программный компонент - это более крупная и независимая часть компьютерной программы, которая выполняет определенные действия. Это как автономный модуль, который можно использовать в разных проектах. Программные компоненты предназначены для самостоятельной работы с четким способом взаимодействия с другими компонентами. Они объединяют несколько модулей, делая разработку программного обеспечения более продвинутой. Эти компоненты помогают создавать более сложные и гибкие программные системы, предлагая функциональные возможности, которые можно легко использовать в разных местах. По сути, программный компонент представляет собой блок многократного использования, который упрощает организацию и разработку компьютерных программ.

Разница между модулем и программным компонентом

В разработке программного обеспечения мы часто слышим о модулях и программных компонентах, но что это такое и чем они отличаются?

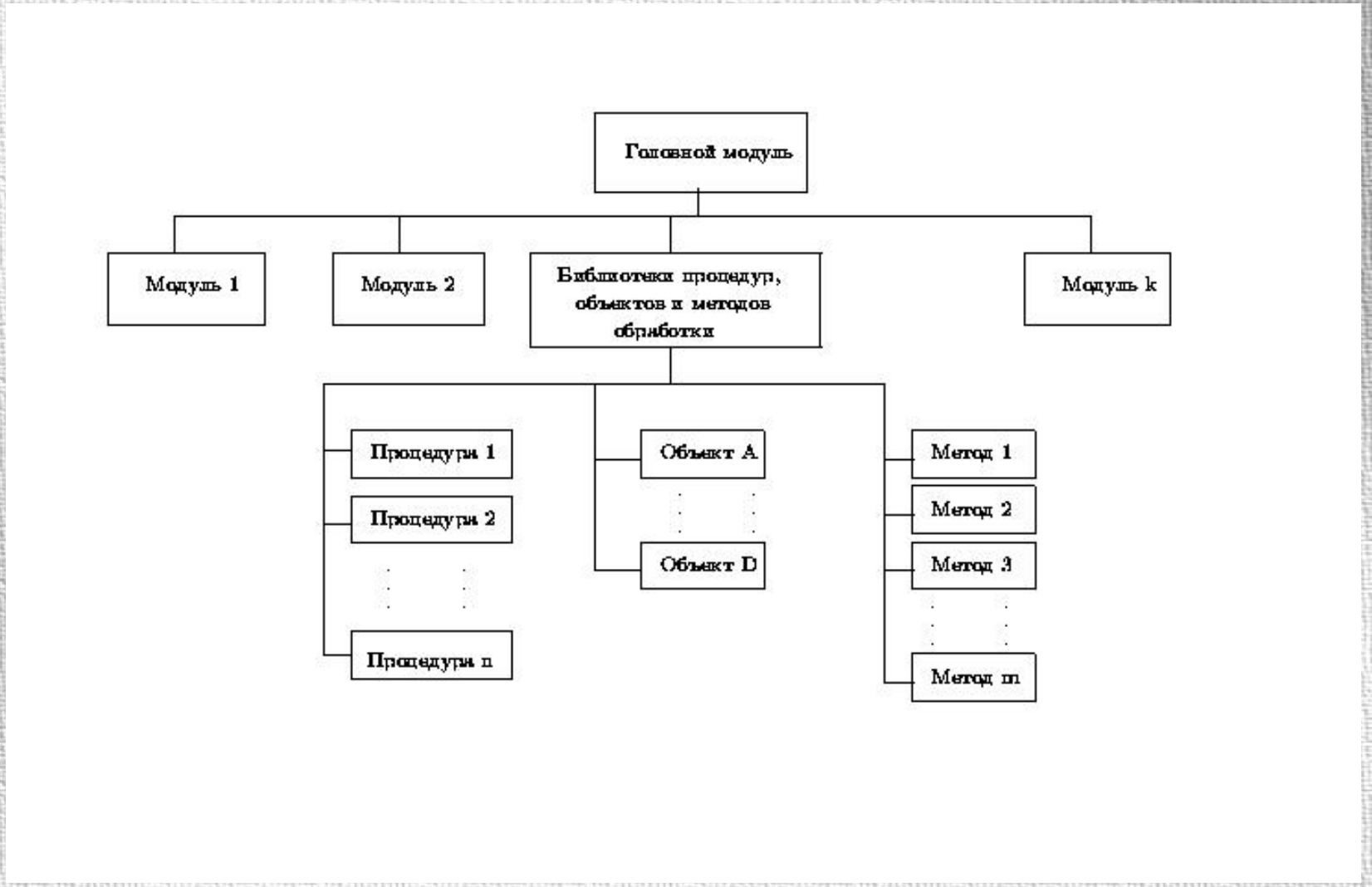
- Проще говоря, модуль подобен небольшому специализированному набору инструментов внутри программы, выполняющему конкретные задачи.
- С другой стороны, программный компонент - это более крупный автономный инструмент, который можно повторно использовать в разных программах.

Аспект	Модуль	Программный компонент
Размер и объем задач	Модуль - это меньшая часть, которая выполняет определенные задачи или функции в программе.	Программный компонент - это более крупная независимая единица, выполняющая более широкий спектр функций в программе или системе.
Детализация	Модули более тонкие, ориентированные на индивидуальные задачи.	Программные компоненты более грубые, они инкапсулируют несколько модулей и предлагают обширную функциональность.

Взаимодействие	Модули взаимодействуют с другими модулями в рамках одной программы или системы.	Программные компоненты взаимодействуют с другими компонентами, часто в разных системах или приложениях.
Уровень абстракции	Модули работают на более низком уровне, ближе к детальной реализации.	Программные компоненты работают на более высоком уровне, ориентируясь на общие общесистемные функции.

Повторное использование в рамках проекта	Модули способствуют повторному использованию кода в рамках одного проекта, позволяя разработчикам использовать одни и те же функциональные возможности в разных частях программы.	Программные компоненты также можно повторно использовать в рамках проекта, но они созданы для большей гибкости. Они могут использоваться в различных частях проекта и часто разрабатываются с возможностью адаптации для использования в различных проектах или системах.
Масштабируемость	Модули могут быть менее масштабируемыми по отдельности, что требует координации с другими.	Программные компоненты, будучи более независимыми, способствуют лучшей масштабируемости системы в целом.
Пример	Модулем может быть определенная функция или набор функций, например модуль калькулятора.	Программным компонентом может быть компонент обработки платежей, обрабатывающий платежи, транзакции и безопасность.

Модуль - это самостоятельная часть программы, имеющая определенное назначение и обеспечивающая заданные функции обработки **автономно от других программных модулей.**



Типовая структура программного продукта, состоящего из отдельных программных модулей и библиотек процедур, встроенных функций, объектов и т.п.

Среди множества модулей различают:

- **головной модуль** - управляет запуском программного продукта (существует в единственном числе);
- **управляющий модуль** - обеспечивает вызов других модулей на обработку;
- **рабочие модули** - выполняют функции обработки;
- **сервисные модули** и библиотеки, утилиты - осуществляют обслуживающие функции.

В работе программного продукта активизируются необходимые программные модули. Управляющие модули задают последовательность вызова на выполнение очередною модуля. Информационная связь модулей обеспечивается за счет использования общей базы данных либо межмодульной передачи данных через переменные обмена.

Каждый модуль может оформляться как самостоятельно хранимый файл; для функционирования программного продукта необходимо наличие программных модулей в полном составе.

Структурно-сложные программные продукты разрабатываются как пакеты программ, и чаще всего они имеют прикладной характер - пакеты прикладных программ, или ППП.

Еще одна точка зрения

Чем модуль отличается от компонента. Чем схема модулей отличается от схемы компонентов.

- **Компонент** — это функциональная единица.
- **Схема компонентов** — ее цель ответить на вопрос "Как это работает?".
- **Модуль** — это конструктивная единица.
- **Схема модулей** — ее цель, ответить на вопрос — "Из чего это состоит?".
- Схема компонентов отражает функциональное назначение системы, схема модулей отражает физическую структуру системы (конструктивную).

- Схема модулей отражает физическое воплощение системы, ее структуру. Т.е. из каких конкретных элементов состоит эта система, которая выполняет работу, описанную на схеме компонентов. Один модуль может содержать в себе несколько компонентов, и наоборот, один компонент может быть распределен по нескольким модулям.
- Если компоненты как правило, это больше абстракции, целью которых служит показать принцип и схему работы системы, то модули это как правило уже **конкретные физические объекты реального мира**, которые имеют соответствующие атрибуты: производитель, модельный номер, материал изготовления, размеры, объем и т.д.

С точки зрения программной инженерии

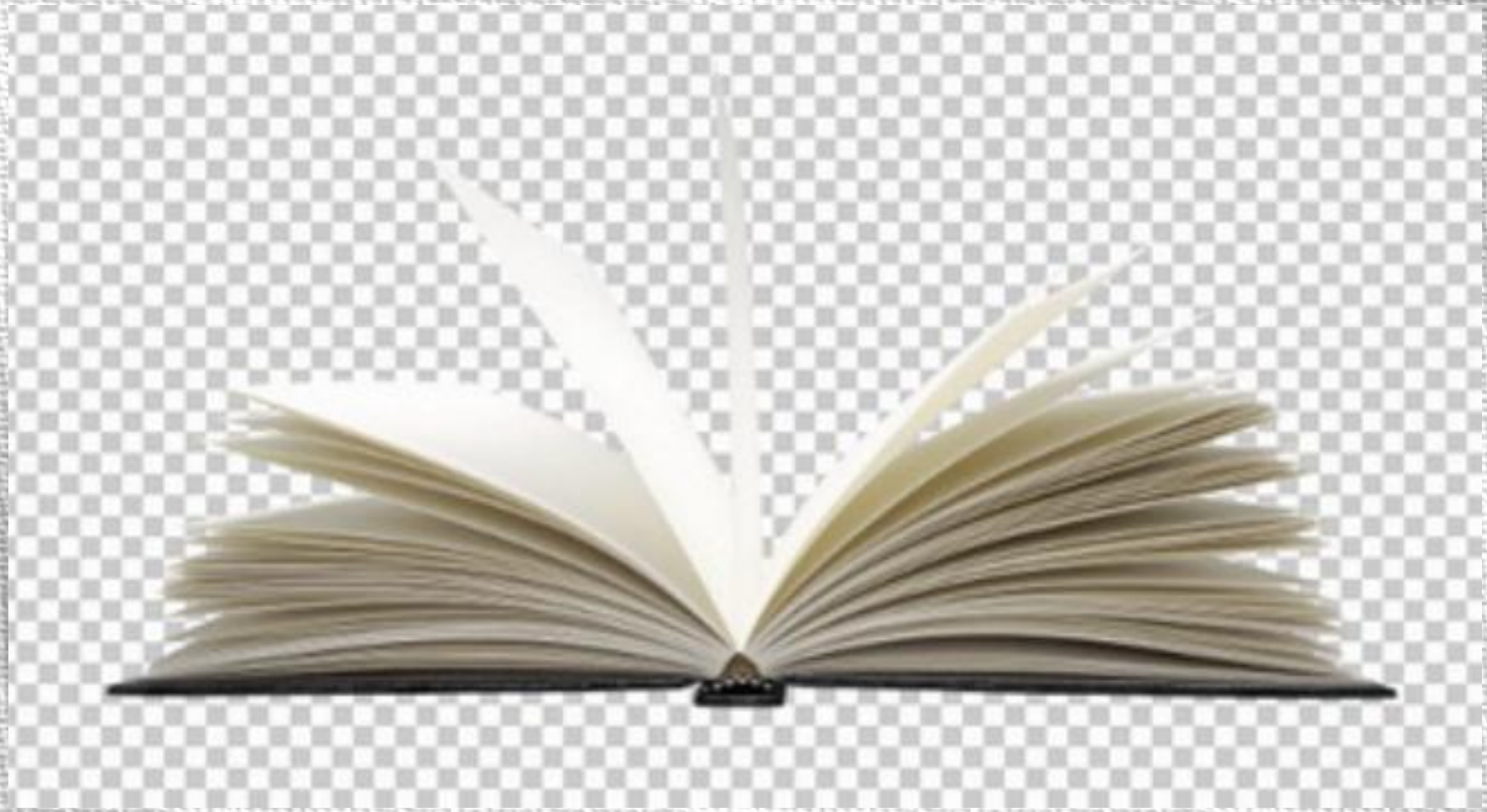
- Если это переносить в область программной инженерии, то модули в программной инженерии — это библиотеки, исполняемые файлы, пакеты (nuget, js и т.д). Также к модулям можно отнести проекты, подпроекты, файлы исходного кода (cs, cpp), но это уже модули описания системы, а не ее воплощения.
- В системной инженерии принято **разделять саму систему, и описание системы**. К описанию системы относятся исходные коды, документация и т.д. А сама система — это уже скомпилированные файлы и данные, работающие в продуктовом окружении, и делающие полезную работу, ради которой собственно и создавалась эта система.

- Так вот, компонентами в программной инженерии является те сущности, и логика предметной области, которые мы описываем в наших файлах исходных кодов. Это те абстракции, выявленные в ходе предметно-ориентированного проектирования (и объектно-ориентированного в том числе, как фундамента для предметно-ориентированного проектирования), это те классы их методы которые описывают ту работу, которая выполняет запрограммированная нами система.
- Если говорить довольно упрощенно, то модули — это те файлы и проекты, в которых мы пишем наш код. и те библиотеки, докер-контейнеры, микросервисы как выходные результаты наших исходных текстов, А компоненты, это — то логическое наполнение, которое записываем в этих исходных файлах, и которое на машинном уровне превращается уже в конкретные команды процессору.

Резюме

- **Схема компонентов** — описывает систему с функциональной точки зрения (т.е. как это работает)
- **Схема модулей** — описывает систему с конструктивной точки зрения (из чего это состоит)

Маршалинг и сериализация



Маршалинг

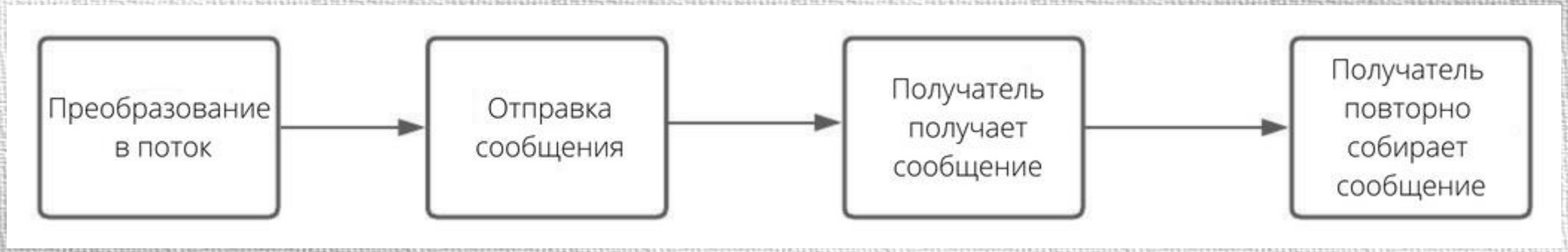
Применительно к компьютерным сетям маршалинг означает процесс преобразования данных в формат, в котором данные могут быть переданы по сети и могут пройти через сетевые ограничители. Данные преобразуются в поток байт, упаковываются, делятся на части, передаются по сети средствами сетевого протокола. Принятые данные преобразуются обратно в исходный формат.

Сериализация

Сериализация – это процесс, в котором одна служба берет структуру данных, такую как словарь в Python, упаковывает ее и передает другой службе для чтения. Это максимально простое определение.

Представьте, что мне нужно отправить кому-то сообщение. Итак, я записываю текст на уже собранный пазл. Далее я разбираю части пазла, добавляю несколько инструкций о том, как его собрать, и отправляю его.

Затем получатель сообщения, получив кусочки головоломки, собирает их вместе. И теперь у него есть мое сообщение.


Преобразование в поток
Отправка сообщения
Получатель получает сообщение
Получатель повторно собирает сообщение

Маршалинг и сериализация – в чем разница?

Маршалинг – это процесс преобразования представления объекта в памяти в форму, подходящую для передачи.

Хотя маршалинг и сериализация в общих чертах похожи, между ними все-таки есть принципиальная разница. Например, при создании программы в Golang для считывания JSON данных в структуру данных Golang вы можете использовать маршалинг для преобразования пары «ключ-значение» JSON в пару «ключ-значение» Golang.

Разница в том, что маршалинг используется для преобразования данных. А сериализация, напротив, отправляет или сохраняет данные в потоке байтов и повторно собирает их в исходную форму. Оба процесса вроде бы выполняют процесс сериализации, но с разными намерениями.

Форматы данных сериализации

В основном я использую JSON для сериализации, когда этого требует задача. Но тем не менее, вы можете использовать и другие варианты. У JSON много издержек, но для меня он идеален, потому что он читабелен. Вы также можете использовать Protobuf, YAML или XML. Это лишь некоторые из возможных.

лекции

Чтобы написать действующую программу, не нужно много знать и уметь. Написать правильную программу сложно. Для этого необходимы знания и умения, которые молодые программисты еще не успели приобрести.

А чтобы приобрести их, требуется мыслить и анализировать, на что у многих программистов просто нет времени. Это требует такой самодисциплины и организованности, которые не снились большинству программистов.

А для этого нужно испытывать страсть к профессии и желание стать профессионалом.

Мартин Чистая архитектура

ОБЪЕКТНО-ОРИЕНТИРОВАННЫЙ АНАЛИЗ

- Анализ предметной области
- Анализ приложения

Анализ — это достаточно широкое понятие. Его содержание более точно отражают термины

- **анализ требований** (requirements analysis) - исследование требований к системе и
- **объектно-ориентированный анализ** (object-oriented analysis) - исследование объектов предметной области.

Зачем нужна модель анализа, если требования уже сформулированы в Спецификации требований к ПО?

Модель анализа создается для того, чтобы обеспечить более высокий уровень понимания и взаимодействия, чем тот, что дает текстовая спецификация требований к ПО или любое другое представление требований.

Объектно-ориентированный анализ — это методология, при которой требования к системе воспринимаются с точки зрения классов и объектов, выявленных в предметной области.

Анализ предметной области и анализ приложения

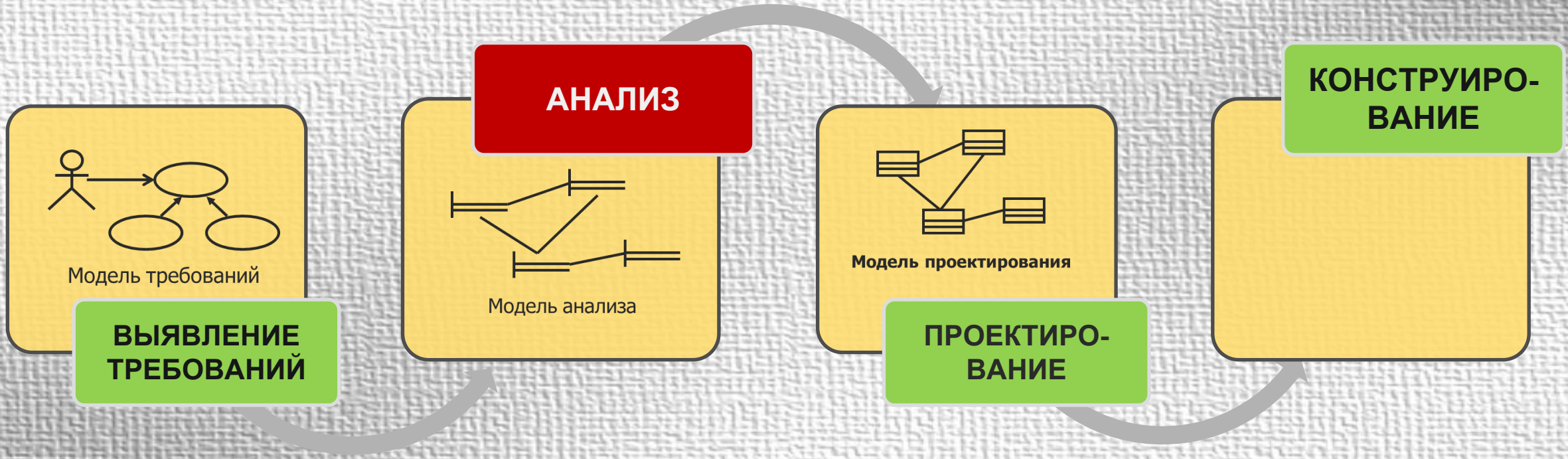
- Предметная область
  - Элементы
  - Явления
  - Отношения
  - Процессы
- Приложение
  - Функциональность
  - Структура
  - Поведение
  - ...

Этап анализа делится на анализ предметной области и анализ приложения.

- Статическая модель предметной области описывает общие знания о приложении. Такая модель **обычно включает модели классов и состояний**, но очень редко – модель взаимодействия.
- Динамическая модель приложения отражает наиболее важные артефакты, видимые пользователем и требующие их одобрения. Для приложения важнее всего **модель использования** (взаимодействия), однако нельзя забывать модель классов и состояний.

МОДЕЛИ ЭТАПОВ РАЗРАБОТКИ

Модель анализа



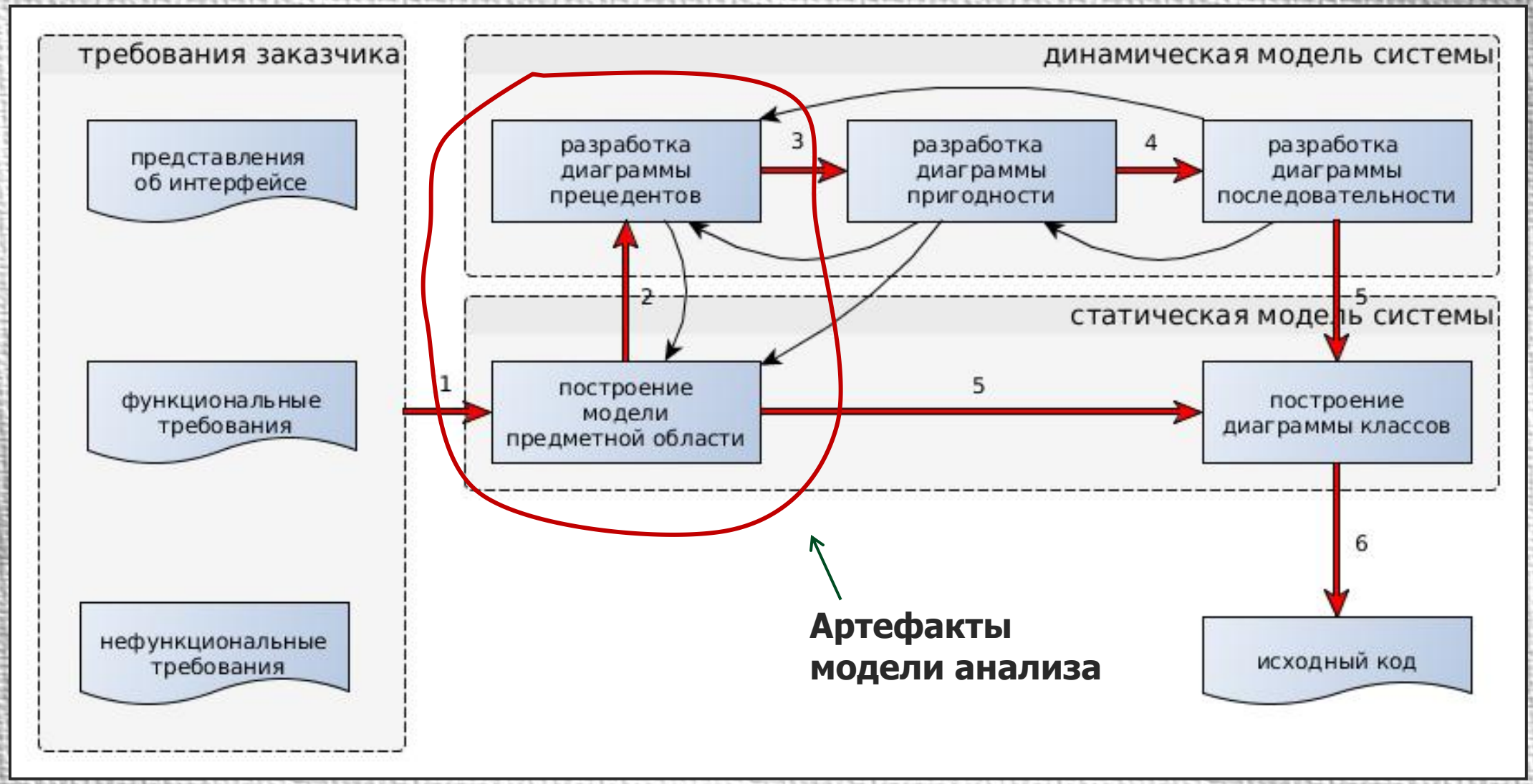
Модель требований

1. Бизнес-модель
2. Спецификация требований
3. Словарь

Модель проектирования

1. Модель данных
2. Документ Архитектура

Последовательность моделирования



МОДЕЛЬ АНАЛИЗА

Основу модели анализа составляют:

- **Модель предметной области**
- **Модель вариантов использования (прецедентов)**

Учтите, что в некоторых источниках под моделью анализа понимают только модель предметной области.

В модель прецедентов, кроме непосредственно диаграммы прецедентов могут включать реализацию прецедентов и системные диаграммы последовательностей. В системной ДП описывается как субъект и Система общаются (обмениваются сообщениями).

В описании модели анализа может быть отражено:

- **Введение**: Текстовое описание, которое служит в качестве краткого введения в модель.
- **Пакеты анализа**: Пакеты в модели, представляющие иерархию.
- **Классы**: Классы в модели, принадлежащие пакетам.
- **Взаимосвязи:** Взаимосвязи в модели, принадлежащие пакетам.
- **Реализации вариантов использования:** Реализации вариантов использования в модели, принадлежащие пакетам.
- **Диаграммы**: Диаграммы в модели, принадлежащие пакетам.

АРТЕФАКТЫ МОДЕЛИ АНАЛИЗА

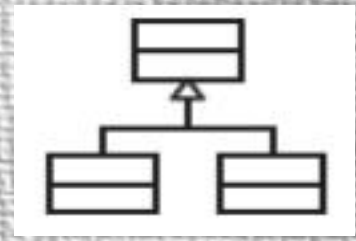
При разработке модели анализа рекомендуется построить следующие диаграммы (основные артефакты):

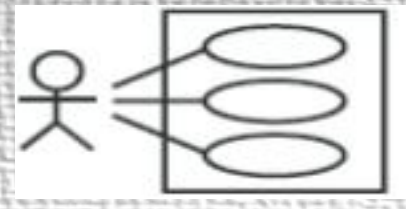
- **диаграмма классов анализа (модель предметной области);**
- **диаграмма вариантов использования Use Case;**

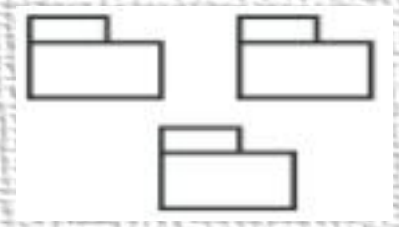
В качестве ***дополнительных артефактов*** в данную модель также входят диаграммы состояний и согласованные между заказчиком и разработчиком ***нефункциональные требования*** к системе.

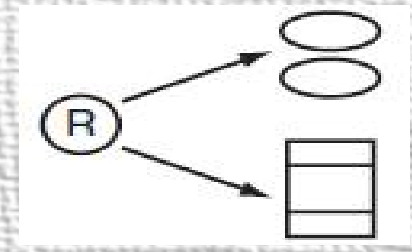
КАК ПРОВОДИТЬ АНАЛИЗ

1. Выявите объекты предметной области, а также существующие между ними отношения обобщения и агрегирования. Начните рисовать высокоуровневую диаграмму классов.
2. Если можно, создайте приблизительный прототип системы (модель пользовательского интерфейса) или соберите всю доступную информацию об унаследованной системе, которую вы хотите переработать.
3. Выявите прецеденты, используя для этой цели диаграммы прецедентов.
4. Организуйте прецеденты в группы. Изобразите группы в виде пакетов.
5. Распределите функциональные требования между прецедентами и объектами предметной области

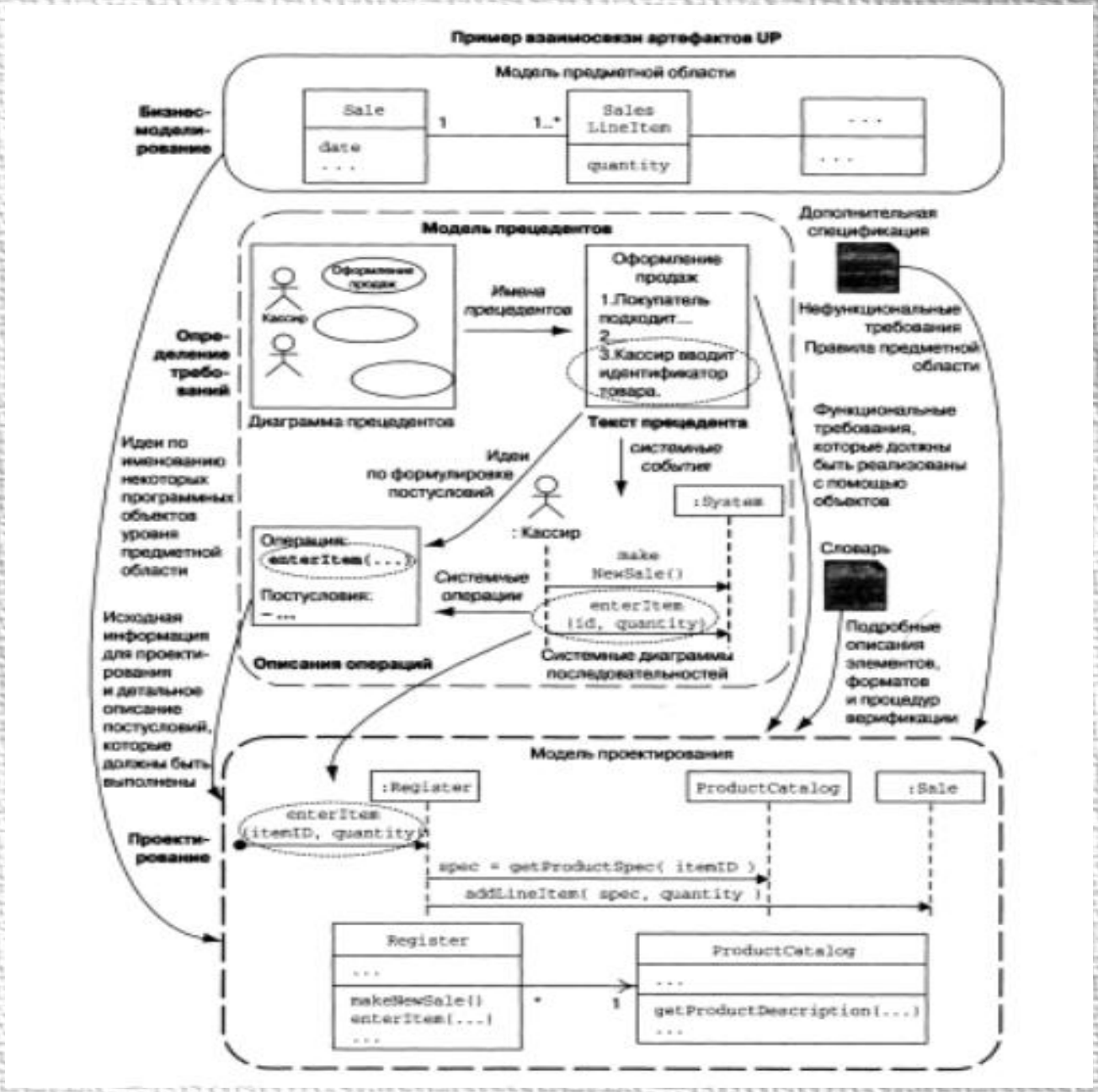




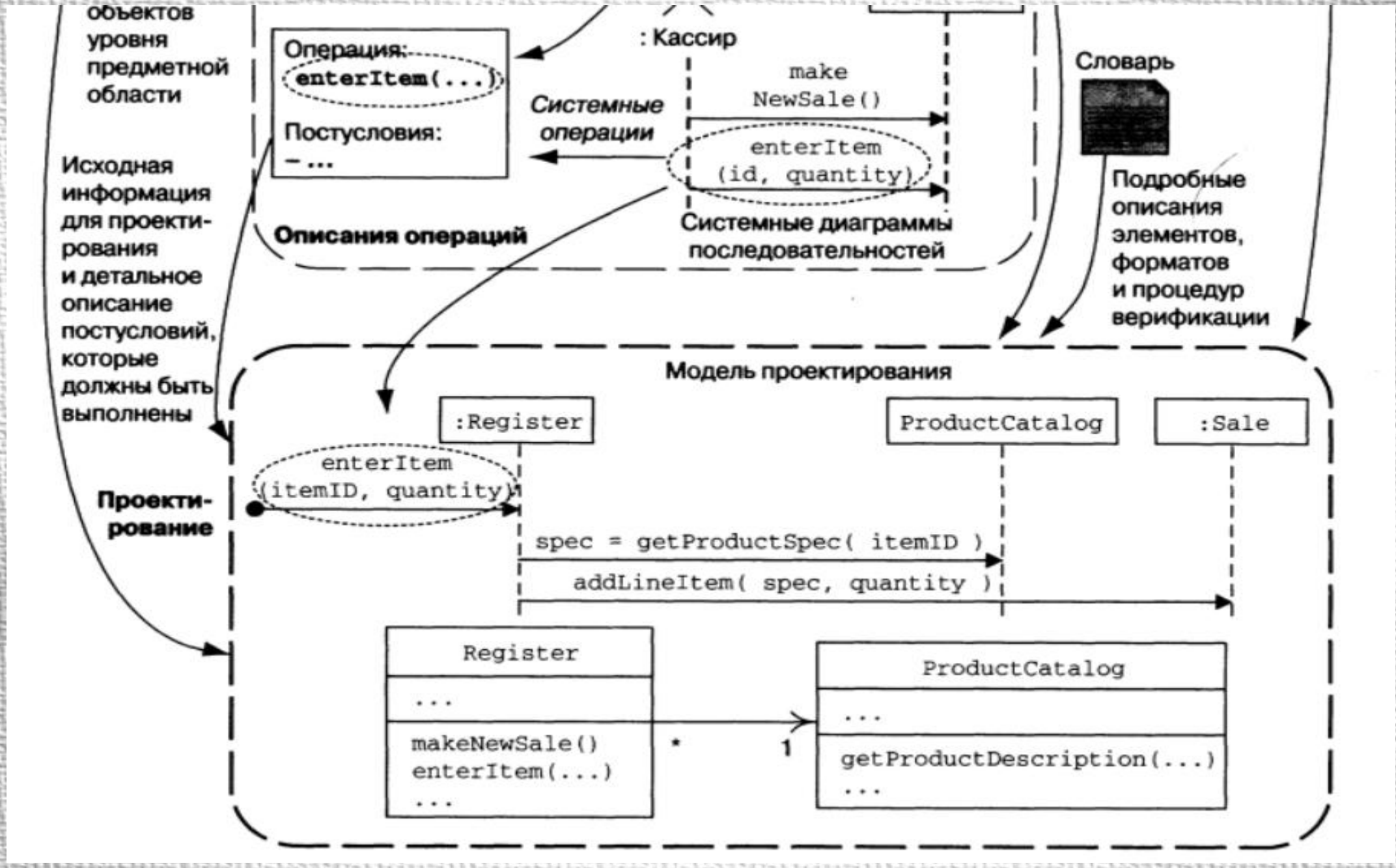




Пример взаимосвязи артефактов в процессе объектно-ориентированного проектирования



Пример взаимосвязи артефактов UP
Модель предметной области
Бизнес-моделирование
Sale
date
...
1
1..*
Sales LineItem
quantity
...
...
Дополнительная спецификация
Модель прецедентов
Оформление продаж
Кассир
Имена прецедентов
Оформление продаж
1.Покупатель подходит...
2...
3.Кассир вводит идентификатор товара.
Определение требований
Нефункциональные требования
Правила предметной области
Диаграмма прецедентов
Текст прецедента
Идеи по именованию некоторых программных объектов
Идеи по формулировке постусловий
системные события
:System
Функциональные требования, которые должны быть реализованы с помощью объектов


объектов
уровня
предметной
области
Операция:
enterItem(...)
Постусловия:
– ...
: Кассир
make
NewSale()
Системные
операции
enterItem
(id, quantity)
Словарь
Исходная
информация
для проекти-
рования
и детальное
описание
постусловий,
которые
должны быть
выполнены
Описания операций
Системные диаграммы
последовательностей
Подробные
описания
элементов,
форматов
и процедур
верификации
Модель проектирования
:Register
ProductCatalog
:Sale
enterItem
(itemID, quantity)
Проекти-
рование
spec = getProductSpec(itemID)
addLineItem(spec, quantity)
Register
...
makeNewSale()
enterItem(...)
...
*
1
ProductCatalog
...
getProductDescription(...)
...

Этап 1

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

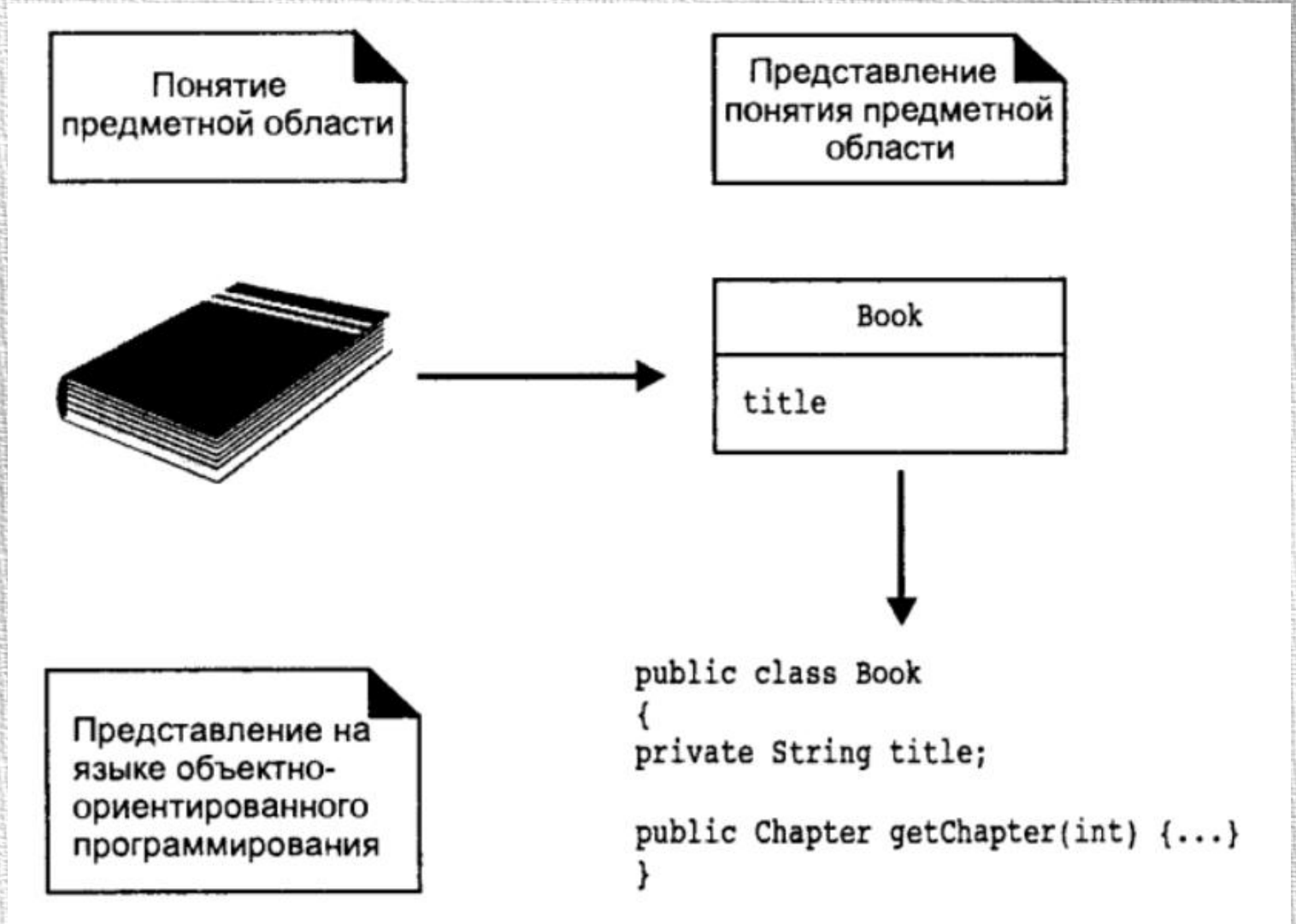
МОДЕЛЬ ПРЕДМЕТНОЙ ОБЛАСТИ (DOMAIN MODEL)

Начнем с так называемой модели предметной области. Это своего рода словарь основных абстракций, то есть самых важных существительных в пространстве задачи. Если, к примеру, предметной областью является электронная коммерция, то, вероятно, в словарь войдут понятия «каталог» и «заказ на покупку».

Что такое объекты «реального мира» (предметной области) и какие между ними ассоциации?

Существительные, которые описывают понятия из предметной области, будут называться ***доменными объектами***. **В самом начале анализа и проектирования мы создадим модель предметной области, в которой все доменные объекты будут изображены на одной большой диаграмме классов.**

Представление объектов с использованием объектно-ориентированного подхода



Моделирование предметной области

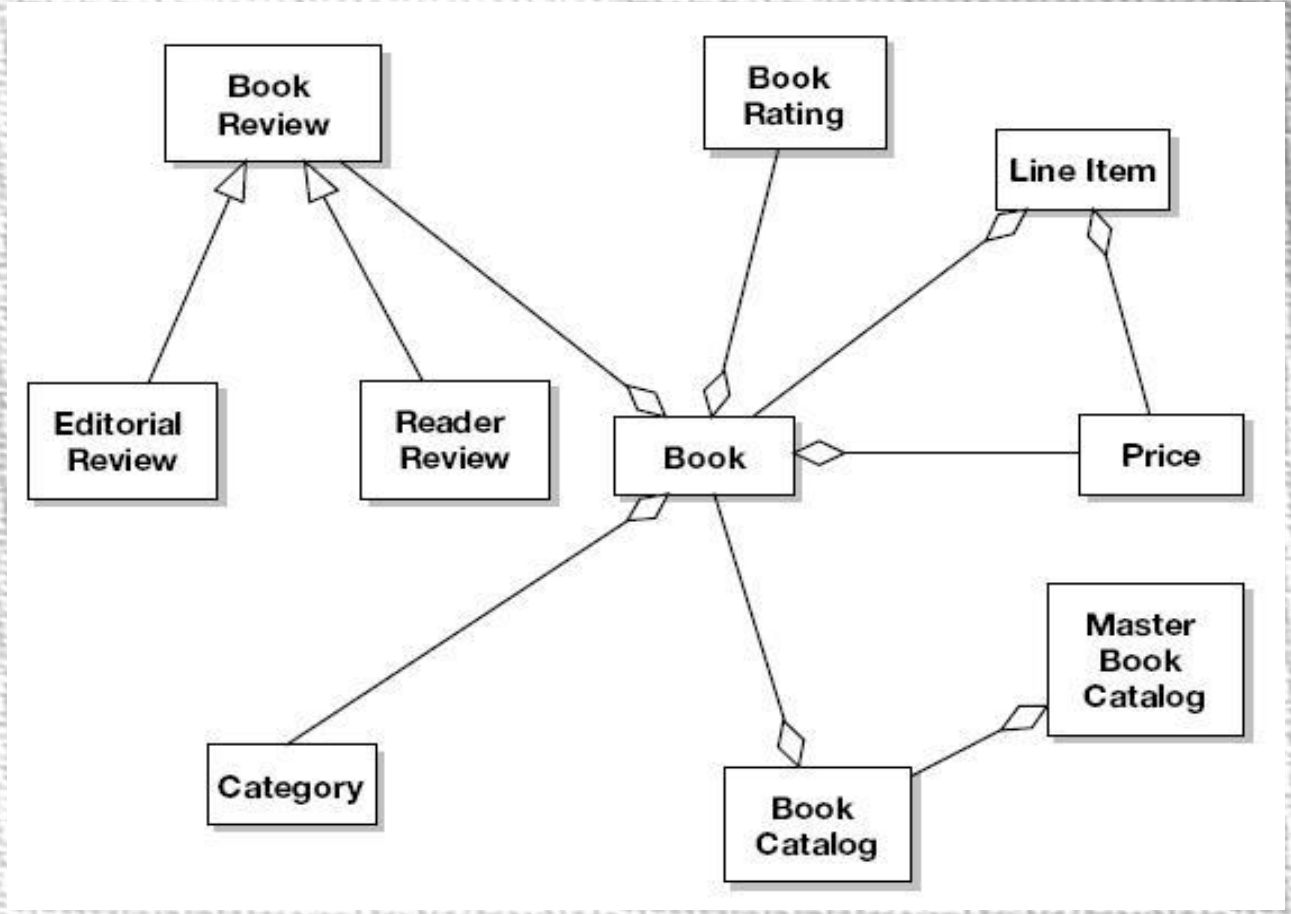
В терминологии UML концептуальная модель предметной области – это, по сути дела, диаграмма классов. Обычно в этой модели опускается большая часть деталей, в частности атрибуты и операции классов, поскольку задача модели заключается в **распределении обязанностей между классами-сущностями**. Затем в процессе работы над вариантами использования мы будем постепенно уточнять эти обязанности и, в конце концов, получим детальную статическую модель системы.

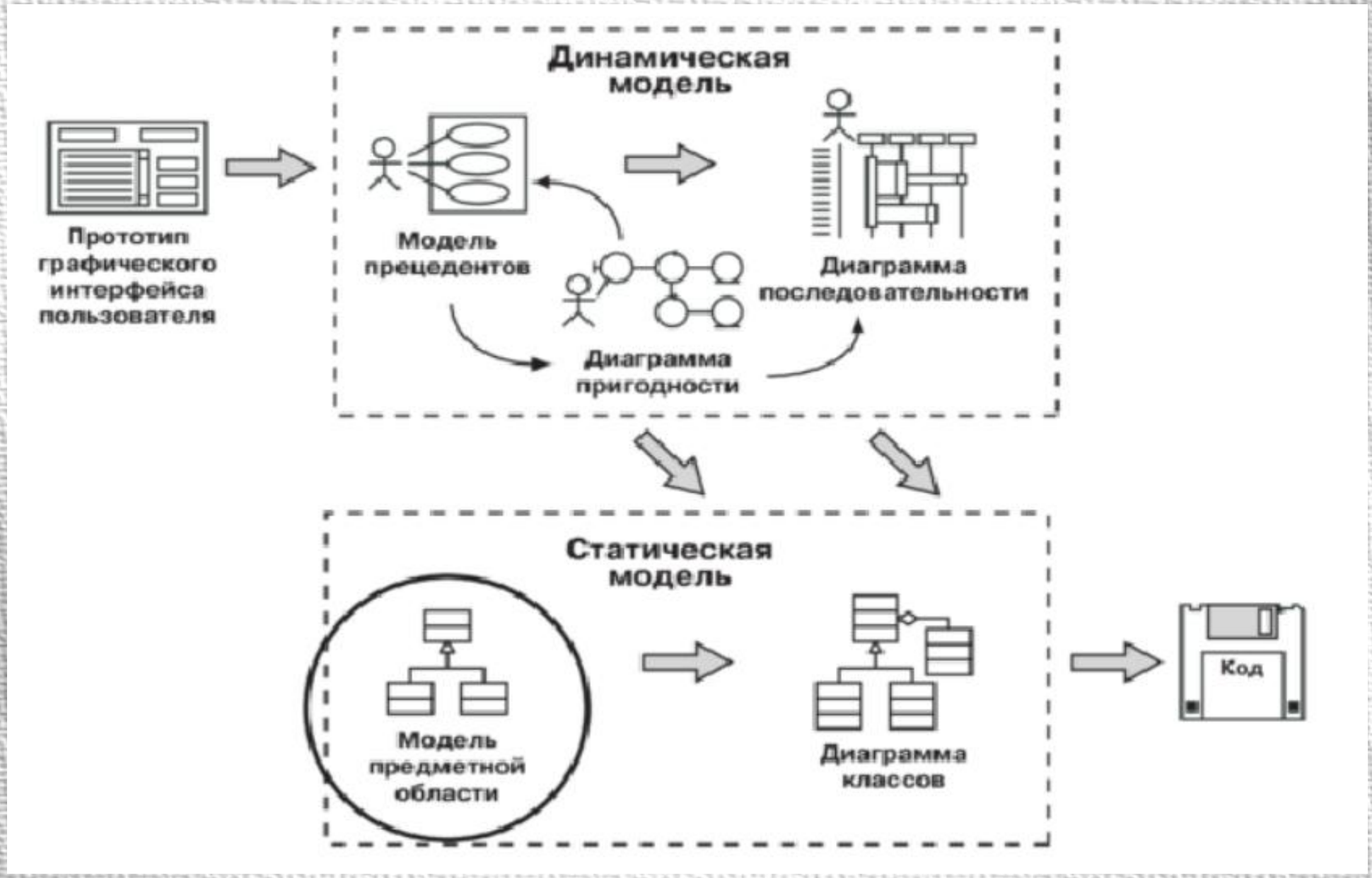
Классы анализа применяются для фиксации основных "кластеров ответственности" в системе.

Удобно начать анализ с модели предметной области, потому что варианты использования лучше писать в контексте объектной модели, т. е. в ВИ должны использоваться термины из объектной модели. Благодаря этому будут связаны статическая часть модели и динамическая, которую описывают варианты использования.

Статическая часть описывает структуру, динамическая – поведение системы.

С использование словаря предметной области должно происходить общение между участниками рабочей группы. Но модель предметной области лучше, чем словарь. Она показывает графически, как термины связаны между собой. На практике эта модель – это упрощенная диаграмма классов. На ней используются отношения агрегации (**has-a**) и наследования (**is-a**).

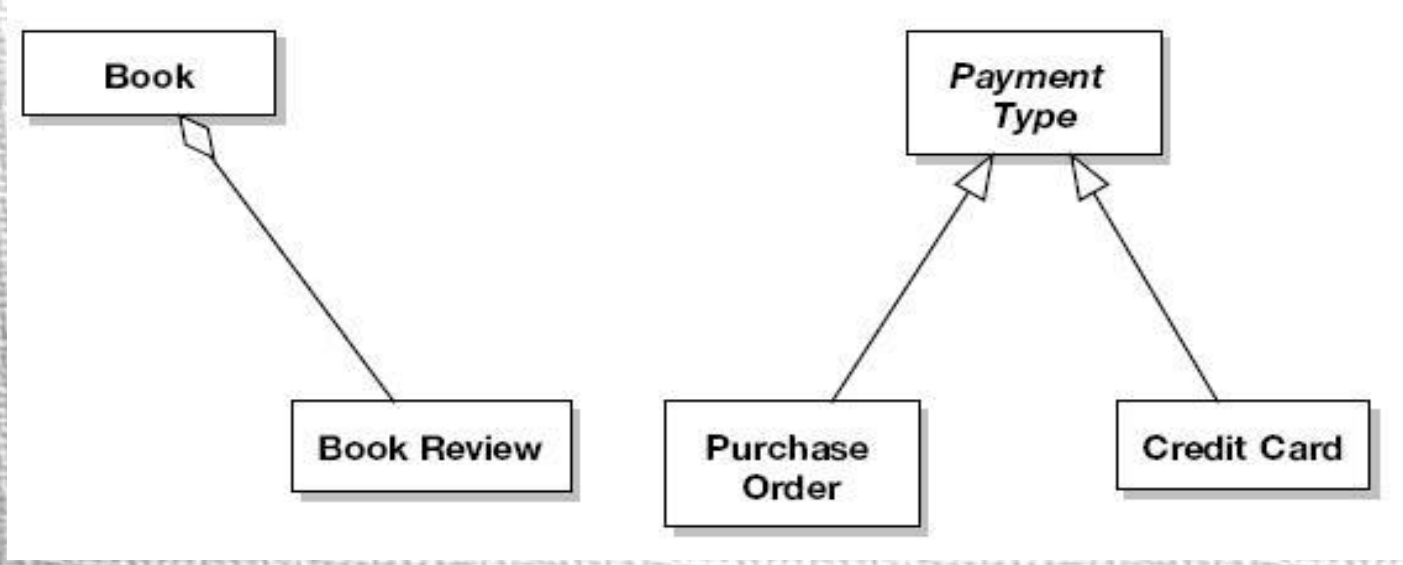



Динамическая модель
Прототип графического интерфейса пользователя
Модель прецедентов
Диаграмма последовательности
Диаграмма пригодности
Статическая модель
Модель предметной области
Диаграмма классов
Код

ОСНОВНЫЕ ПРАВИЛА ПОСТРОЕНИЯ МОДЕЛИ ПРЕДМЕТНОЙ ОБЛАСТИ

Можно сформулировать следующие правила построения модели предметной области.

1. **Используйте отношения has-a и is-a для того, чтобы показать, как объекты соотносятся друг с другом.**



Желательно размещать отношения слева направо и сверху вниз, как обычный текст. Это улучшит читаемость диаграмм.

2. **Первоначальный набросок модели должен быть построен не больше, чем за пару часов.** Этот набросок все равно не будет идеальным и будет многократно изменен на следующих этапах, поэтому много времени на этом этапе тратить не стоит. Вся рассматриваемая методология основана на том, что модель предметной области неполна и требует постоянного уточнения. Для этого существуют специальные приемы. Однако на этом этапе выявляются 80% всех понятий предметной области
3. **Нужно выделить ключевые абстракции и привязать к ним объекты предметной области.** Если ориентироваться на абстракции, то можно подготовиться к возможным изменениям в требованиях, которые обязательно произойдут, а когда это случится, модель сможет быть адаптирована к изменениям. Кроме того, организация классов вокруг абстракций позволяют инкапсулировать изменчивость в сравнительно независимых модулях/подсистемах.

4. **Нельзя путать объект (который является экземпляром сущности) с таблицей базы данных (в которой содержатся набор описаний экземпляров).** Один объект может примерно соответствовать одной строке таблицы базы данных. Объект содержит атрибуты, которые могут быть поставлены в соответствие полям таблицы, но в записи таблицы может находиться и другая информация, например, ключевые поля.

5. **Нужно использовать модель предметной области как словарь проекта.** В этом словаре необходимо найти и исключить различные названия одних и тех же понятий.

6. **Модель предметной области нужно построить до составления вариантов использования, чтобы избежать дублирования понятий.** Если похожие названия попадут в тексты вариантов использования, то найти их там гораздо сложнее, чем в словаре проекта.

7. **Окончательная диаграмма классов может быть не похожа на диаграмму модели предметной области, хотя соответствие должно быть.** Реальная диаграмма классов будет включать в себя гораздо больше деталей, чем диаграмма модели предметной области. На ней появятся всевозможные классы-помощники, фабрики классов и т. п. Диаграмму модели предметной области всегда можно разделить на несколько подробных диаграмм классов.
8. **В модель предметной области нельзя помещать название экранов пользовательского интерфейса или другие классы, относящиеся к пользовательскому интерфейсу.** Если нарушить это правило, то модель сразу же переполнится разнообразными понятиями, не относящимися непосредственно к делу.

Последовательность разработки модели предметной области

1. Выделить классы
2. Подготовить словарь данных
3. Выделить ассоциации
4. Выделить атрибуты объектов и связей
5. Организовать и упростить классы при помощи наследования
6. Проверить наличие маршрутов для наиболее вероятных запросов
7. Перейти к следующий итерации и уточнить модель
8. Пересмотреть уровень абстрагирования
9. Сгруппировать классы в пакеты

ИДЕНТИФИКАЦИЯ КЛАССОВ

Описание классов и отношений между ними является основным средством моделирования структуры в UML. Но прежде чем переходить к технике описания классов, полезно обсудить вопрос, как выделяются классы, подлежащие описанию.

По этому вопросу в литературе приводится множество соображений, советов, рекомендаций и даже принципов. Само разнообразие подходов свидетельствует о том, что среди них нет универсального и применимого во всех случаях.

ПРИЕМЫ ВЫДЕЛЕНИЯ КЛАССОВ

Ниже приведены три приема выделения классов, самых простых, а потому самых действенных и широко применимых:

1. **словарь предметной области;**
2. **реализация вариантов использования;**
3. **шаблоны проектирования.**

❶ Словарь предметной области

(domain dictionary)

Словарь предметной области **– это набор основных понятий (сущностей) данной предметной области.**

Рассмотрите внимательно текст технического задания (или иного документа, лежащего в основе проекта) и выделите в содержательной части имена существительные – все они являются кандидатами на то, чтобы быть названиями классов (или атрибутов классов) проектируемой системы. Разумеется, после этой простой операции к полученному списку нужно применить фильтр здравого смысла и опыта, отсекая ненужное или добавляя пропущенное.

Рассмотрим пример информационной системы отдела кадров. В тексте технического задания первый вводный абзац можно отбросить сразу – это общие слова. Суть заключена в нумерованных пунктах. Но в этом тексте вообще все слова являются существительными (кроме союзов). Выпишем их в том порядке, как они встречаются, но без повторений:

- прием;
- перевод;
- увольнение;
- сотрудник;
- создание;
- ликвидация;
- подразделение;
- вакансия;
- сокращение;
- должность.

Заметим, что некоторые их этих слов, по сути, являются названиями действий (и по форме являются отглагольными существительными). Фактически, это глаголы, замаскированные особенностями родного языка. Отбросим замаскированные глаголы. Таким образом, остается список из четырех слов:

- сотрудник,
- подразделение;
- вакансия;
- должность.

При анализе технического задания мы отметили (опираясь на знание предметной области), что вакансия – это должность в особом состоянии. Таким образом, это слово в списке лишнее и у нас остались три кандидата, которые мы оставляем в словаре (и заодно присваиваем им английские идентификаторы).

- **Сотрудник (Person).**
- **Подразделение (Department).**
- **Должность (Position).**

❷ РЕАЛИЗАЦИЯ ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ

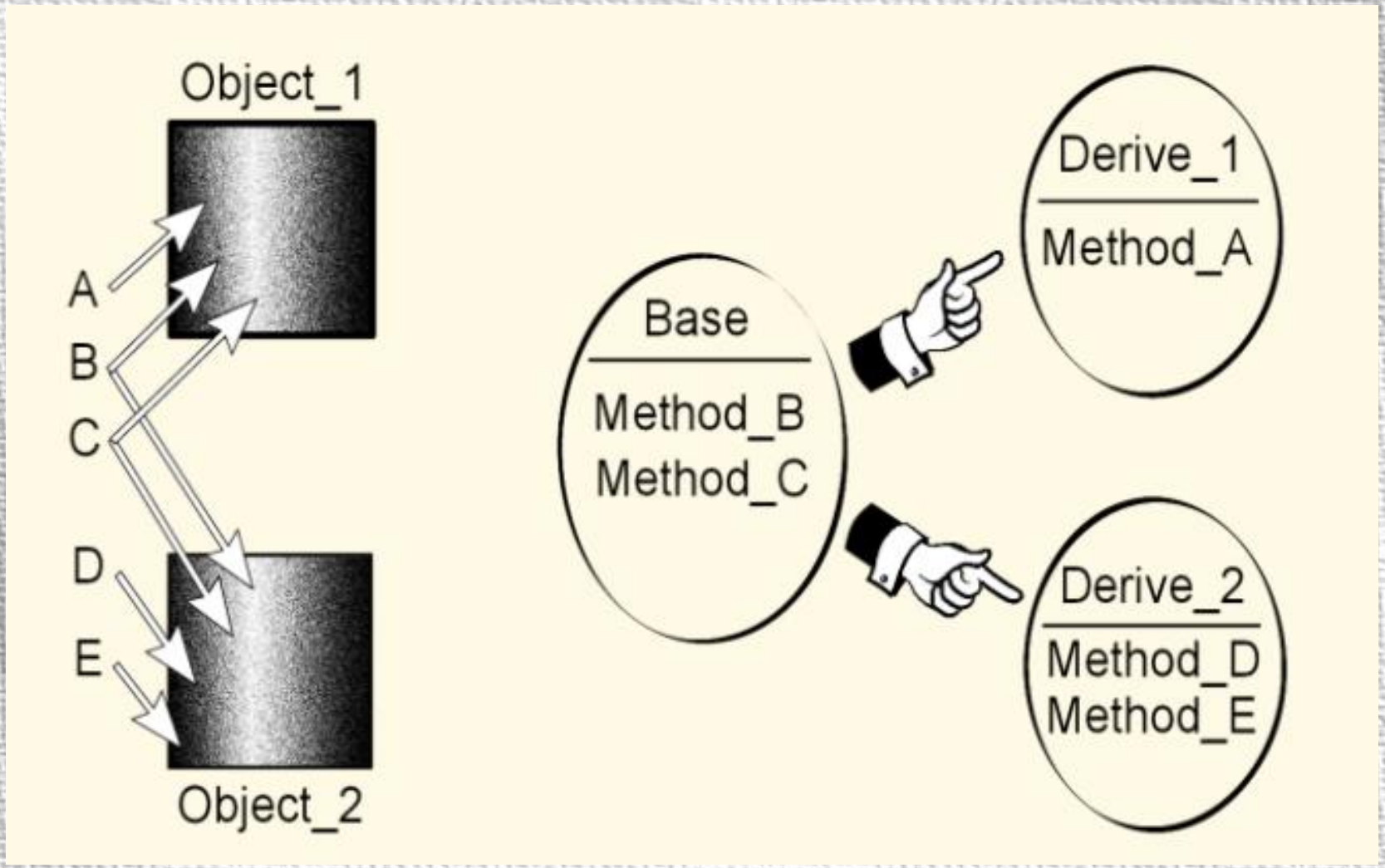
Если при реализации ВИ применяются диаграммы взаимодействия, то в этом процессе в качестве побочного эффекта выделяются некоторые классы непосредственно, поскольку на диаграммах кооперации и последовательности основными сущностями являются объекты, которые по необходимости нужно отнести к определенным классам. Использование диаграмм деятельности также может подсказать, какие классы нужно определить в системе, особенно если на диаграмме деятельности наряду с потоком управления (*control flow*) присутствует поток данных (*data flow*).

Сейчас не существует единого и универсального способа объектно-ориентированного проектирования. К настоящему времени не разработаны строгие методы классификации и нет правил, позволяющих выделять классы и объекты. Можно, однако, рекомендовать следующий порядок действий:

1. **Р**азработку программы следует начать с анализа функционирования моделируемой системы, так как её поведение обычно известно задолго до всех остальных свойств. Затем следует спроектировать объекты и сообщения, которыми они будут обмениваться. После определения того, как объекты будут «общаться» друг с другом, можно приступать к проектированию классов.

2. Проектирование иерархии классов. Обработчики всех общих сообщений объединяйте в один базовый класс. Например, если имеется объект, получающий сообщения A, B, C и другой, получающий сообщения B, C, D, E, то нужно создать базовый класс, обрабатывающий сообщения B, C и два производных от него класса, один из которых будет содержать метод, обрабатывающий A, а другой — методы, работающие с D и E. Если продолжать этот процесс, пока не будут исчерпаны все объекты, то в результате получится требуемая иерархия классов, в «вершине» которой будет располагаться самый общий класс, от которого наследуют все другие классы. Любой базовый класс должен иметь не менее двух потомков.

Процесс проектирования классов



Возможности базового класса должны использоваться всеми его производными классами. Все методы, обрабатывающие сообщения объявите открытыми (public), а все другие, по возможности, должны быть закрытыми (private, protected).

3. Данные добавляются в последнюю очередь. Для поддержки работы с данными на этом этапе также добавляются методы, которые должны быть закрытыми, как и сами данные. Обращение к данным должно быть реализовано только с помощью методов класса, т.е. интерфейс объекта с его окружением должен полностью определяться его методами, чтобы к его состоянию не было другого доступа извне, кроме как через методы.

ПРОБЛЕМЫ ИДЕНТИФИКАЦИИ

Однако, если сценарии ВИ описываются на естественном языке или псевдокоде, то выделить классы значительно труднее. Фактически, если ВИ реализуются на псевдокоде или диаграммами деятельности без всякой связи с объектами, то выявление объектной структуры системы просто откладывается "на потом". Иногда это может быть вполне оправдано — например, архитектор, моделирующий систему, прежде чем начать проектирование основной структуры классов, хочет более глубоко вникнуть в логику бизнес-процессов незнакомой ему предметной области.

Обсудим это на примере информационной системы отдела кадров.

Диаграмма последовательности для типового сценария

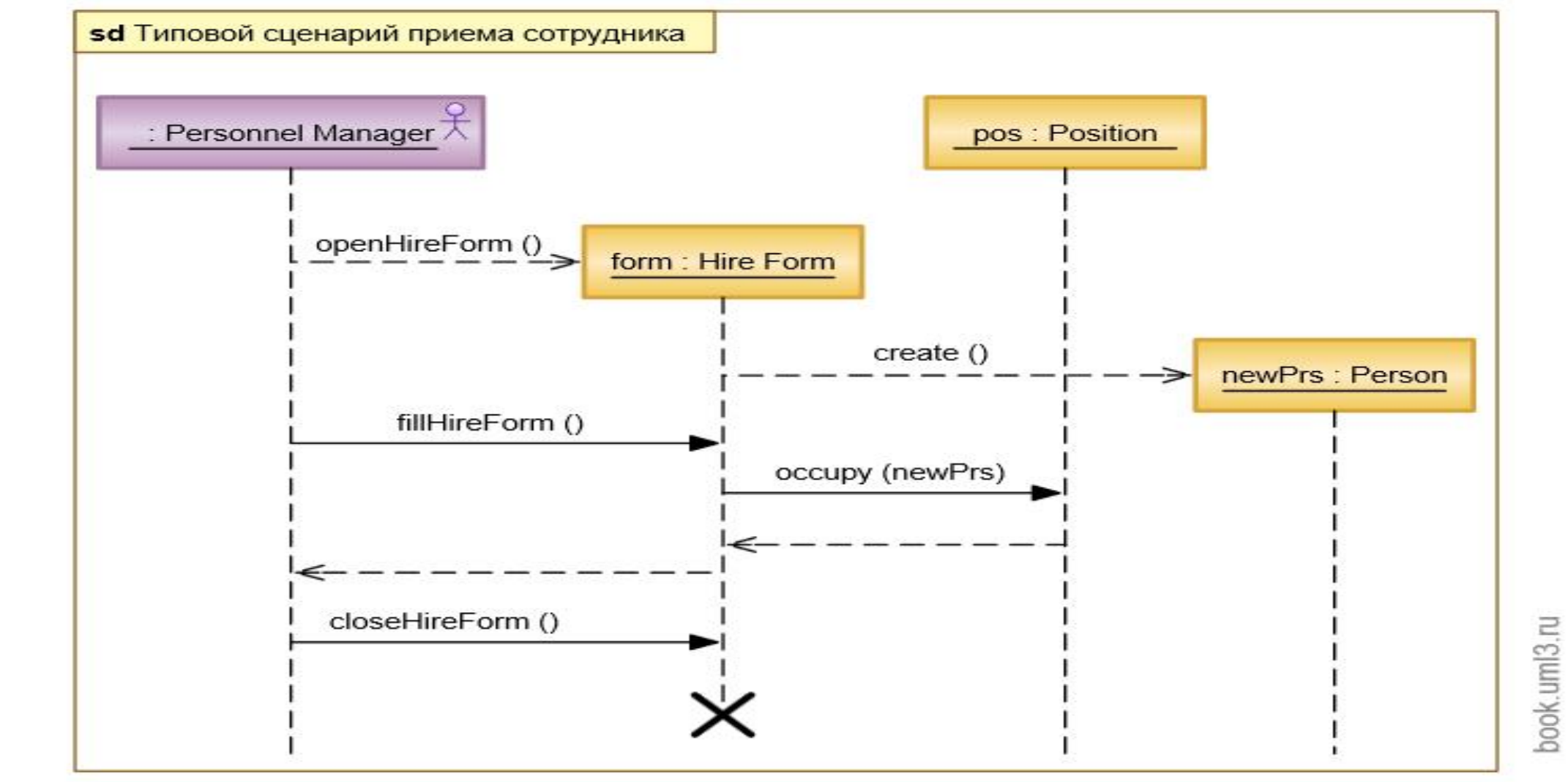
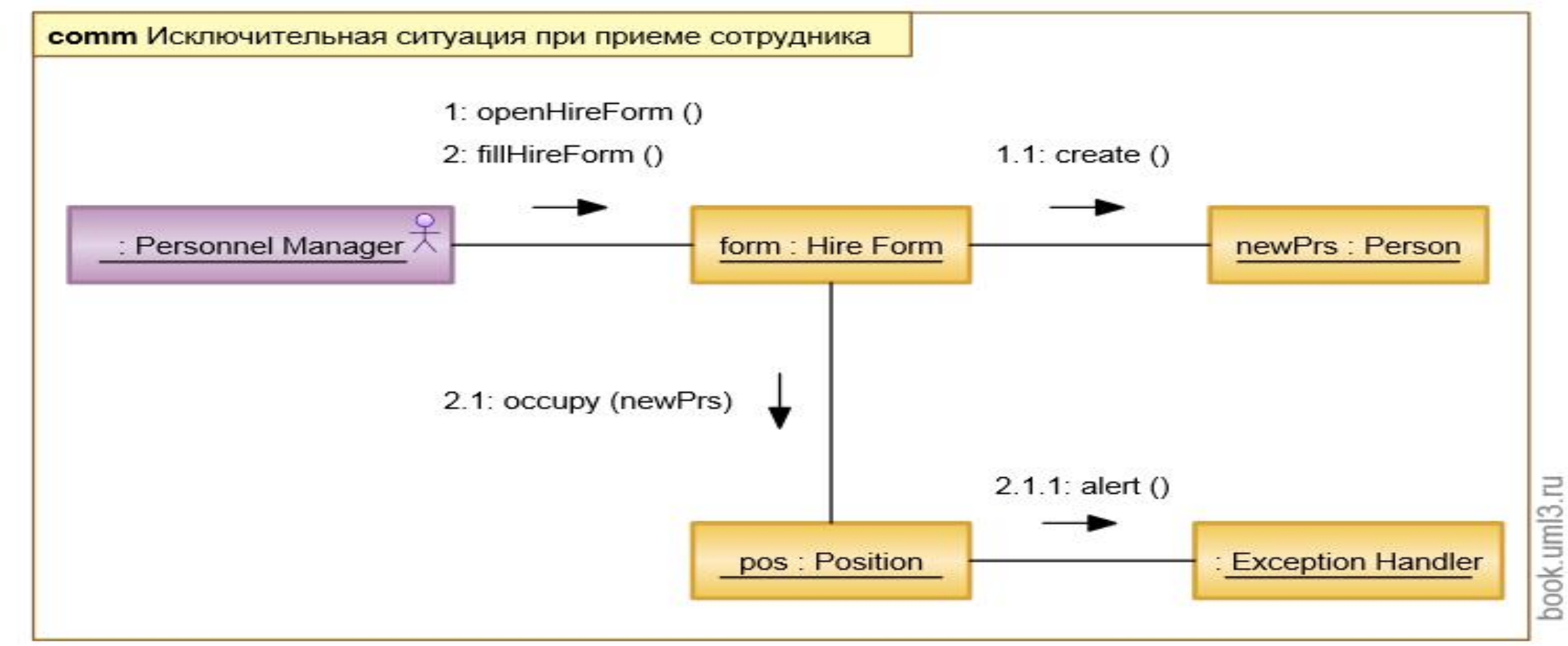


Диаграмма коммуникации для исключительной ситуации



Два класса – **Person** и **Position** – те же, что нам подсказал анализ словаря предметной области. Очевидно, что если одни и те же выводы получены разными способами, доверие к ним возрастает.

Полезный класс **Exceptions Handler**, однако, вряд ли мог появиться из словаря: описывая предметную область, люди склонны закрывать глаза на возможные ошибки, исключительные ситуации и прочие неприятности.

Особого обсуждения заслуживает появление класса **Hire Form**, но его мы пока оставим в стороне.

3 ШАБЛОНЫ ПРОЕКТИРОВАНИЯ

Шаблон проектирования или **паттерн** (*design pattern*) в разработке программного обеспечения — повторяемая архитектурная конструкция, представляющая собой решение проблемы проектирования в рамках некоторого часто возникающего контекста.

Применение шаблонов (образцы, паттерны) проектирования требуют специальных знаний и здесь рассматриваться не будет.

Обычно шаблон не является законченным образцом, который может быть прямо преобразован в код; это лишь пример решения задачи, который можно использовать в различных ситуациях. Объектно-ориентированные шаблоны показывают отношения и взаимодействия между классами или объектами, без определения того, какие конечные классы или объекты приложения будут использоваться.

«Низкоуровневые» шаблоны, учитывающие специфику конкретного языка программирования, называются **идиомами**. Это хорошие решения проектирования, характерные для конкретного языка или программной платформы, и потому не универсальные.

На наивысшем уровне существуют архитектурные шаблоны, они охватывают собой архитектуру всей программной системы. Алгоритмы по своей сути также являются шаблонами, но не проектирования, а **вычисления**, так как решают вычислительные задачи.

Подведем итоги

Мы полагаем, что классы в модели идентифицируются в результате проведения трех частично независимых процессов:

- анализа предметной области,
- согласования уже построенной модели
- применения теоретических соображений.

Ни одним из этих методов нельзя пренебрегать, но одновременно заметим, что ни один из них не является универсальным и самодостаточным: решающим является здравый смысл и опыт архитектора, составляющего модель.

ВЫДЕЛЕНИЕ АССОЦИАЦИЙ

На следующем этапе вы должны выделить ассоциации между классами. Структурные отношения между двумя и более классами являются ассоциацией. Если один класс ссылается на другой класс, это тоже ассоциация. Как уже говорилось, для ссылок на классы не следует использовать атрибуты, их правильнее представлять в модели в виде ассоциации. Например, в классе *Человек* не должно быть атрибута *работодатель*. Правильно связать класс *Человек* с классом *Компания* ассоциацией *РаботаетНа*.

Ассоциации показывают отношения между классами на том же уровне абстракции, на котором находятся сами классы, тогда как атрибуты скрывают зависимости и их двухстороннюю природу. Ассоциации могут быть реализованы множеством способом, выбор которых следует отложить до последующих этапов, чтобы сохранить относительную свободу во время проектирования.

ГРУППИРОВКА КЛАССОВ В ПАКЕТЫ

Последний этап моделирования - группировка классов в пакеты. Пакеты делают модель более удобной с точки зрения построения, печати и просмотра. Помещая определённые классы и ассоциации в пакет, вы делаете семантическое утверждение. Классы одного пакета связаны друг с другом сильнее, чем классы в разных пакетах.

Нужно стараться ограничивать ассоциации рамками одного пакета некоторые классы можно повторять в разных пакетах. Чтобы распределить классы по пакетам, ищите точки сочленения - классы, являющиеся единственным соединением двух частей модели, не имеющих между собой других связей такой класс образует мост между двумя пакетами. Например, в системе управления файлами класс *Файл* является точкой соединения сочленения между структурой каталогов и содержимым файла.

Старайтесь выбирать пакеты так, чтобы сокращать количество пересечений на диаграммах классов. Приложив некоторые усилия, можно добиться того, что большинство диаграммы классов станут плоскими графами без пересекающихся линий.

Модель состояний ПрО

Объекты предметные области за время своей жизни сменяет несколько качественно различных состояний. В этих состояниях они могут иметь разные ограничения на значение атрибутов, разные ассоциации или кратности, выполнять разные операции или иметь разное поведение и т.д. Часто бывает полезно построить диаграмму состояний для таких классов. Диаграмма состояний описывает различные состояния, в которых будет находиться объект, свойства объекта и действующие на них ограничения, события вызывающий переход объекта из одного состояния в другое.

Большинство классов предметы области не требует использования диаграмм состоянии. Для их описания достаточно списка операций. Модель состоянии может помочь понимании поведение тех классов, могут находиться существенно разных состояниях.

Резюме

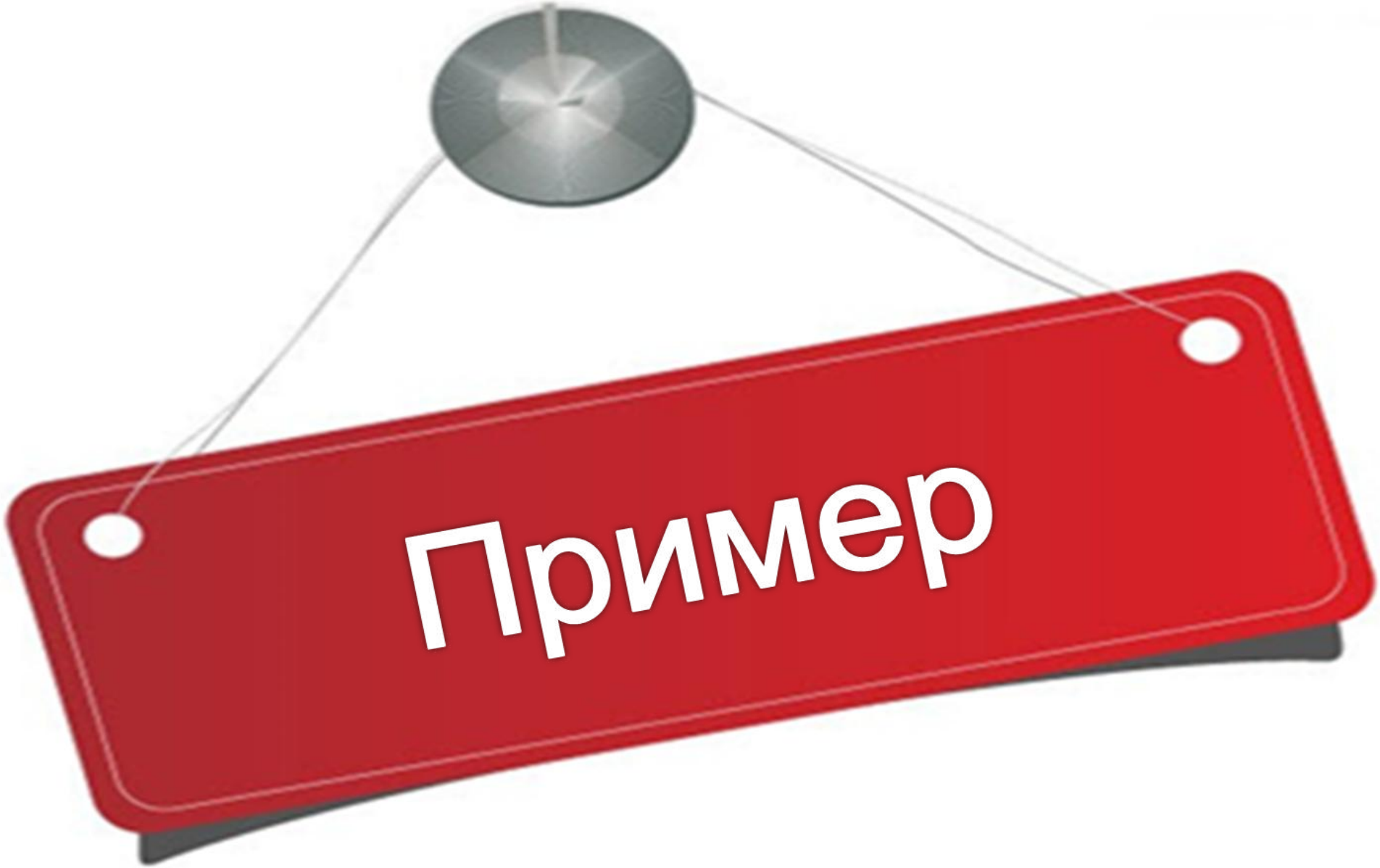
Модель предметной области содержит общие сведения о приложении: понятие и отношения, известные экспертам предметной области. Эта модель обычно включают в себя модели классов и состоянии. Реже в её состав входит модель взаимодействия. Назначение этапа анализа состоит в том, чтобы понять задачу для последующей разработки корректного проекта. Хорошая аналитическая модель описывает важнейшие черты задачи, но не вносит артефактов реализации, ограничивающих проектные решения раньше времени .

Модель классов предметной области отражает статическую структуру реального мира. Сначала нужно выделить классы, затем определить ассоциации между ними, отметить атрибуты. Упорядочения и упрощение структуры классов может быть выполнена при помощи обобщения.

Сгруппируйте родственные классы и ассоциации в пакеты. Дополните модель классов словарём данных — кратким текстовый описаниям назначения и области применения каждого элемента.

Если класс предметный области может находиться в существенно различных состояниях, нарисуйте его диаграмму состояний. Для большинства классов этого не потребуется.

Методология никогда не бывает такой же линейной, как её описание в учебнике. Любая сложная аналитическая модель строятся итерациями на нескольких уровнях. Вы не должны разрабатывать все части модели в одинаковом темпе. Результат анализа заменяет исходную формулировку задачи и служит основанием для проектирования.


Пример

ИНТЕРНЕТ-МАГАЗИН ПО ПРОДАЖЕ КНИГ.

ПОСТРОЕНИЕ МОДЕЛИ ПРЕДМЕТНОЙ ОБЛАСТИ В ПЕРВОМ ПРИБЛИЖЕНИИ НА ОСНОВЕ ТРЕБОВАНИЙ

Требования к системе обычно выглядят как фразы, построенные в виде утверждений: система должна делать то и не должна делать это. Для построения модели предметной области эти требования просматриваются, из них выделяются существительные и характеризующие их прилагательные. По ним строится словарь предметной области.

Например, выделим сущности предметной области из следующих требований.

Интернет-**магазин** должен иметь веб-интерфейс, но он должен иметь возможность подключения через другие интерфейсы (веб-сервисы и т. п.) Интернет-магазин предназначен для продажи **книг**, с оплатой **счетов** через **интернет**. Пользователь должен иметь возможность добавить книги в онлайн **корзину**, после чего произвести **оплату**. Пользователь может убрать **предметы** из корзины. Пользователь должен иметь возможность вести **списки желаемых покупок**, т. е. книг, которые он хочет купить позже. Пользователь должен иметь возможность отменить **заказ** до того, как он отправлен по почте. Пользователь должен иметь возможность оплатить заказ **кредитной картой** или по **счету на оплату**. Должна быть возможность вернуть книги. Интернет-магазин должен встраиваться на сайты партнеров в виде **миникаталога**, который составляется по **основному каталогу**, хранящемуся в центральной **базе данных**.

Мини-каталог должен быть построен на основе XML для того, чтобы была возможность передавать его между системами. **Отправка по почте** должна осуществляться через почтового оператора. Пользователь должен иметь возможность создать **учетную запись**, чтобы система запоминала данные пользователя (имя, адрес, данные банковской карты и т. д.) и восстанавливала их при входе. Система должна вести список учетных записей в центральной базе данных. При входе пользователя его **пароль** должен сверяться с паролем в основном списке паролей, сохраненным в базе данных. Пользователь должен иметь возможность искать книги различным способом – по заголовку, по автору, ключевому слову или категории и после поиска просматривать детальное описание книги. Пользователь должен иметь возможность оставлять **отзывы о книгах**. Оставленные отзывы должны появляться в **детальном описании книги**. Отзыв должен включать **рейтинг** (1-5), который должен показываться вместе с заголовком книги в **общем списке книг**.

Отзывы пользователей должны модерироваться, т. е. им должен присваиваться статус Ok кем-то из администраторов прежде, чем они появятся на сайте. Длинные отзывы должны обрезаться при выводе детального описания книги. Пользователь может щелкнуть по отзыву, чтобы просмотреть полный отзыв на отдельной странице. Должна быть возможность размещения администраторами редакторских отзывов. Они также должны появляться на странице с детальным описанием книги. Магазин должен позволять добавлять каталоги сторонних продавцов в свой основной каталог, чтобы книги этих продавцов участвовали в поиске.

Магазин должен быть масштабируем со следующими требованиями:

1. Должна быть возможность управлять до 100 тыс. пользователей за первые 6 месяцев работы и затем до 1 млн. пользователей.
2. Должна быть возможность обслуживать одновременно 1000 посетителей (до 10000 тысяч после 6 месяцев)
3. Система должна обслуживать 100 поисковых запросов в минуту (1 тыс./мин. после 6 месяцев) Система должна обслуживать 100 покупок в час (1 тыс./час после 6 мес.)

Эти требования – богатый источник для классов предметной области. Выделим существительные (в единственном числе) и связанные с ними слова.

Associate Partner	Партнер
Author	Автор
Book	Книга
Book Catalog	Каталог книг
Book Details	Детальное описание книги
Book List	Список книг
Book Review	Отзыв на книгу
Bookstore	Магазин книг
Category	Категория
Checkout	Подсчет стоимости
Credit Card	Банковская карта

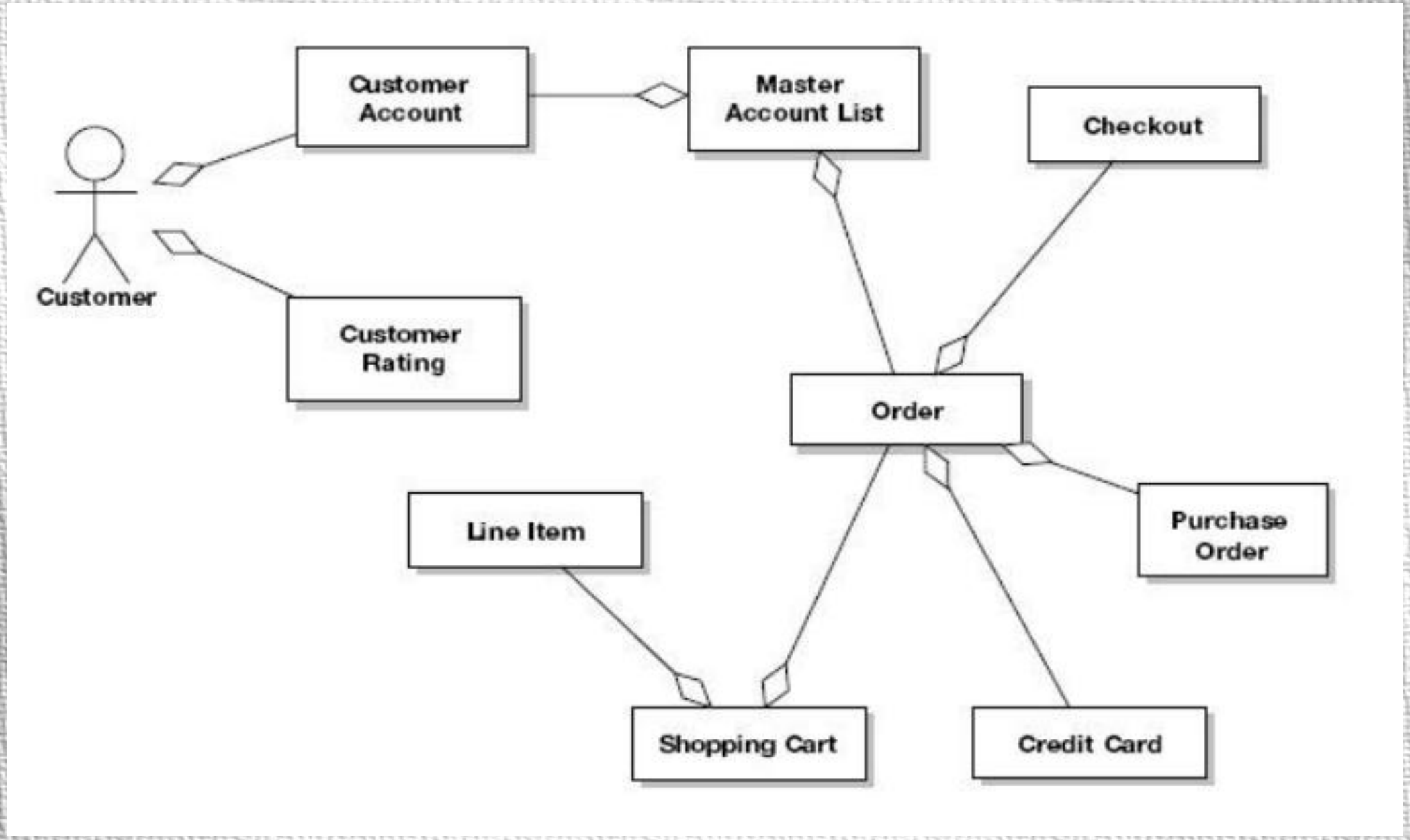
Customer	Клиент
Customer Account	Учетная запись клиента
Customer Rating	Рейтинг
Database	База данных
Editorial Review	Редакторский отзыв
Internet	Интернет
Item	Предмет
Keyword	Ключевое слово
List of Accounts	Список учетных записей
Master Account List	Основной список учетных записей
Master Book Catalog	Основной каталог книг
Master Catalog	Основной каталог
Mini-Catalog	Миникаталог
Order	Заказ

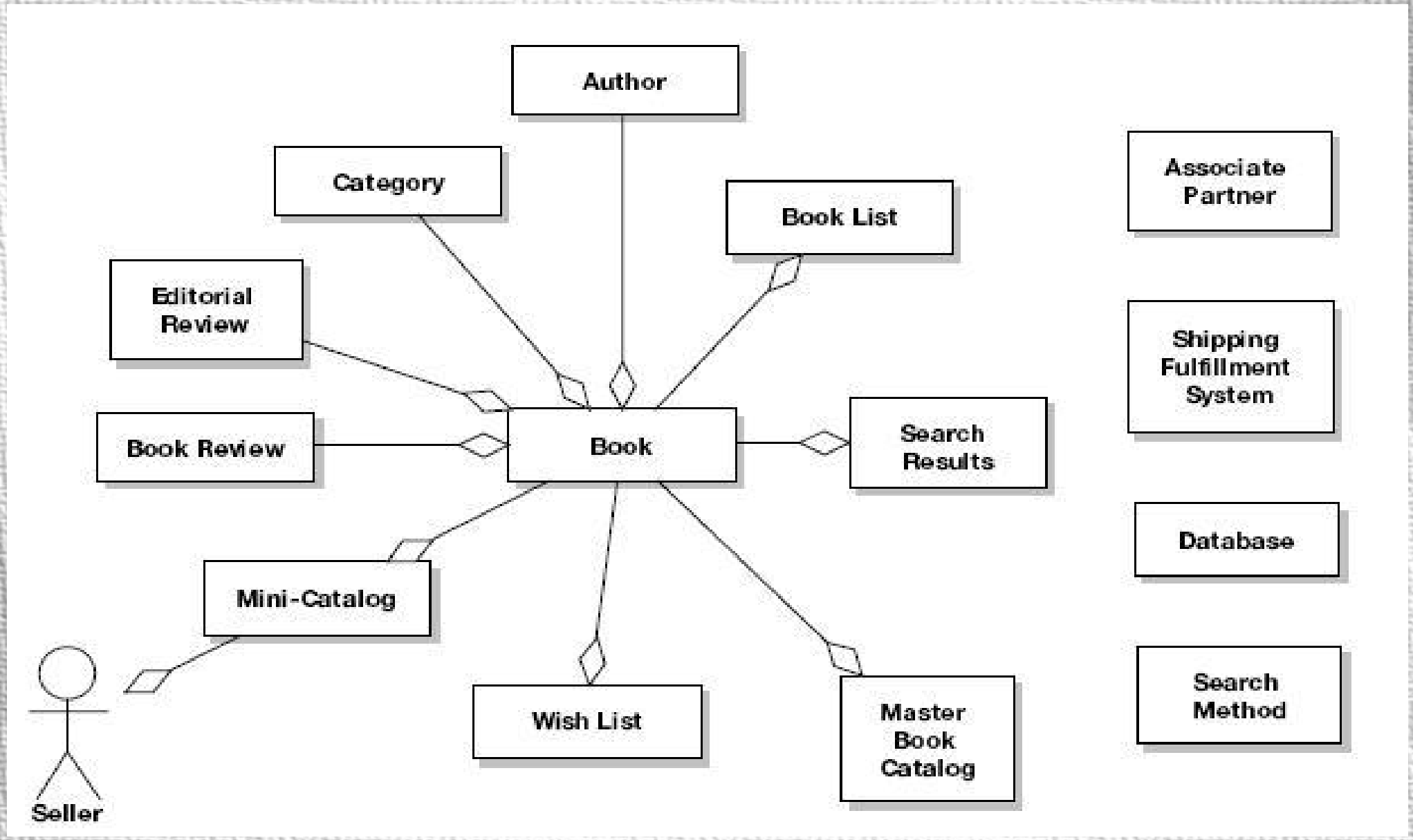
Password	Пароль
Purchase Order	Счет на оплату
Review Comment	Комментарий на книгу
Search Method	Способ поиска
Search Results	Результаты поиска
Seller	Продавец
Shipping Fulfillment	Отправка по почте
System	Система
Shopping Cart	Онлайн корзина
Title	Заголовок
User Account	Учетная запись пользователя
Wish List	Список желаемых покупок

В итоге, у нас остаются:

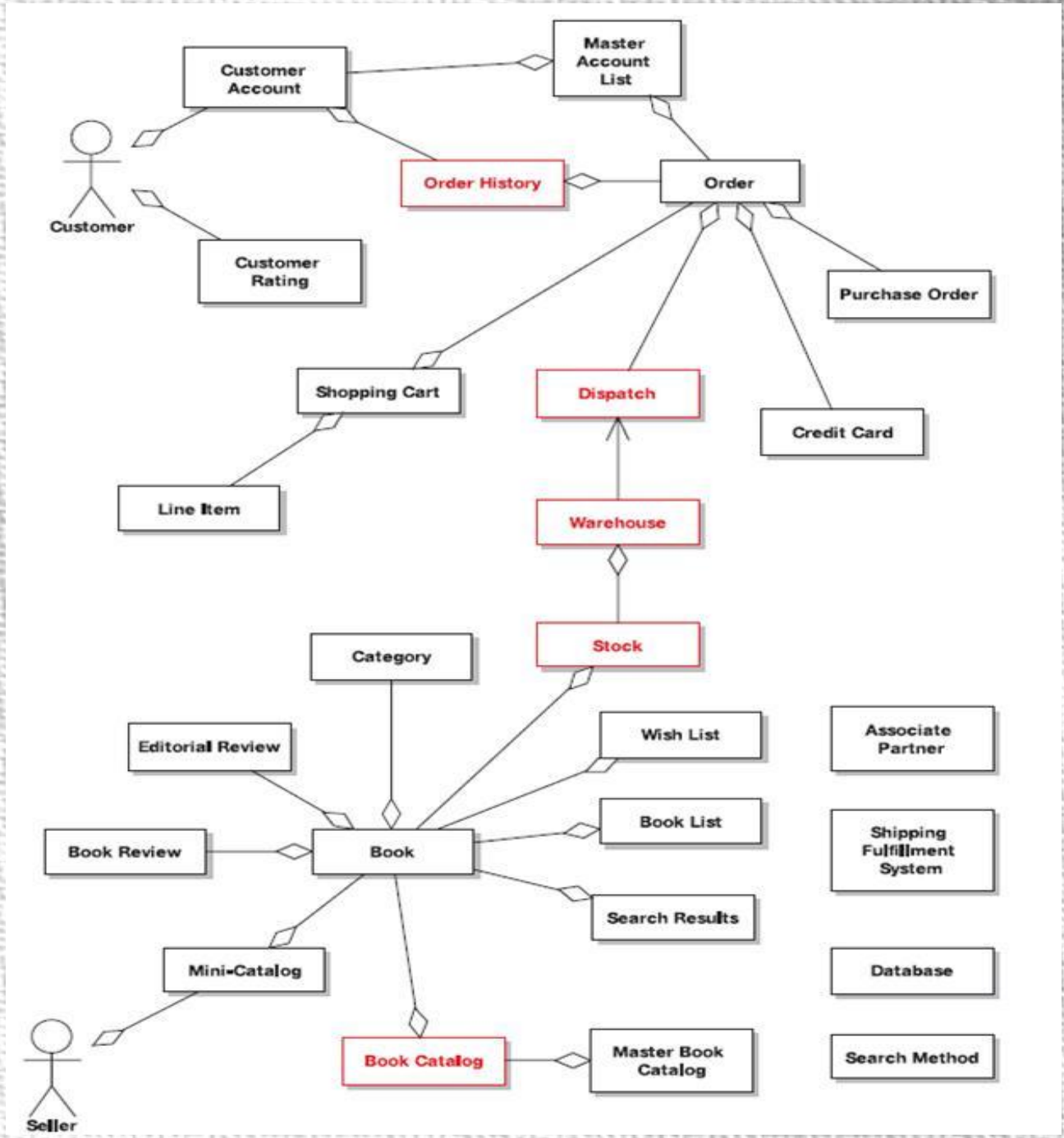
1. Associate Partner
2. Author
3. Book
4. Book List
5. Book Review
6. Category
7. Checkout
8. Credit Card
9. Customer Account
10. Customer Rating
11. Database
12. Editorial Review
13. Line Item
14. Master Account List
15. Master Book Catalog
16. Mini-Catalog
17. Order
18. Purchase Order
19. Search Method
20. Search Results
21. Shipping Fulfillment
22. System
23. Shopping Cart
24. Wish List

На следующей диаграмме мы свяжем эти понятия вместе с помощью отношения has-a (агрегации).
Понятно, что Элемент являются частью Корзины, но можно ли то же самое сказать о Подсчете стоимости (checkout), которая является частью Заказа? Т. е. между ними скорее есть отношение принадлежности (агрегации), но не композиции.
В первом приближении модель предметной области выглядит так:

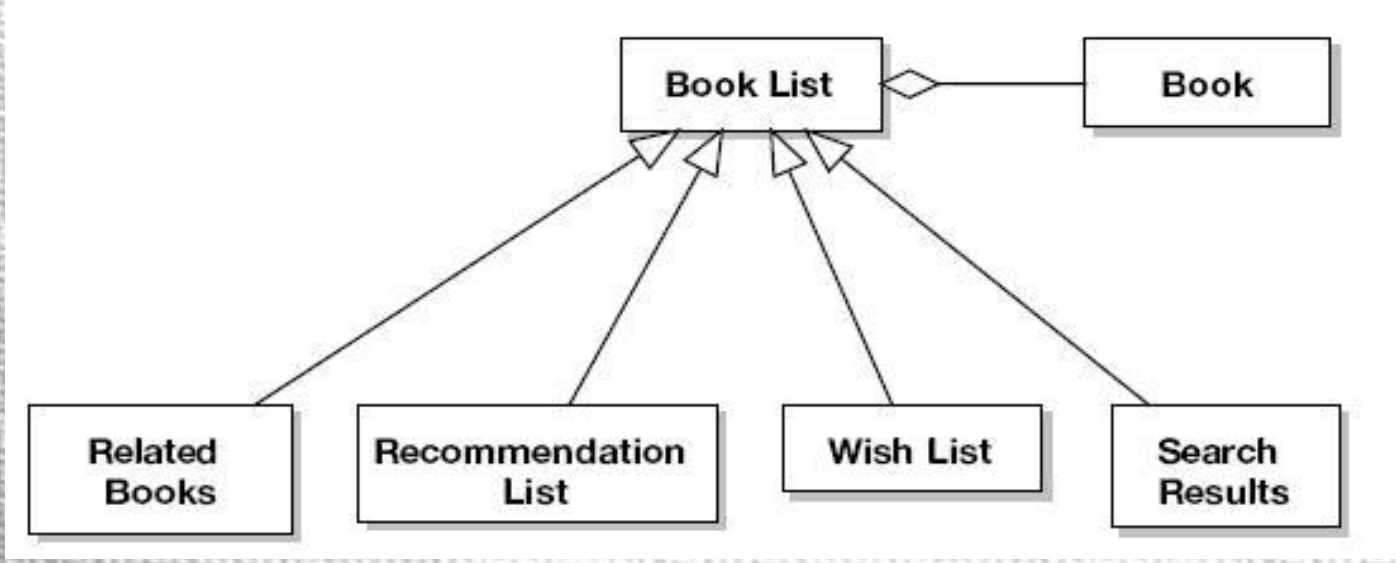

Customer Account
Master Account List
Checkout
Customer
Customer Rating
Order
Line Item
Purchase Order
Shopping Cart
Credit Card


Author
Category
Book List
Editorial Review
Book Review
Book
Search Results
Mini-Catalog
Wish List
Master Book Catalog
Seller
Associate Partner
Shipping Fulfillment System
Database
Search Method

Постепенно мы будем уточнять нашу модель. Пусть в процессе анализа мы выделили два дополнительных объекта: ***История заказа*** и ***Диспетчер заказа***. Этих понятий в требованиях не было, но из опыта можно сказать, что они должны присутствовать в хорошем интернет магазине.

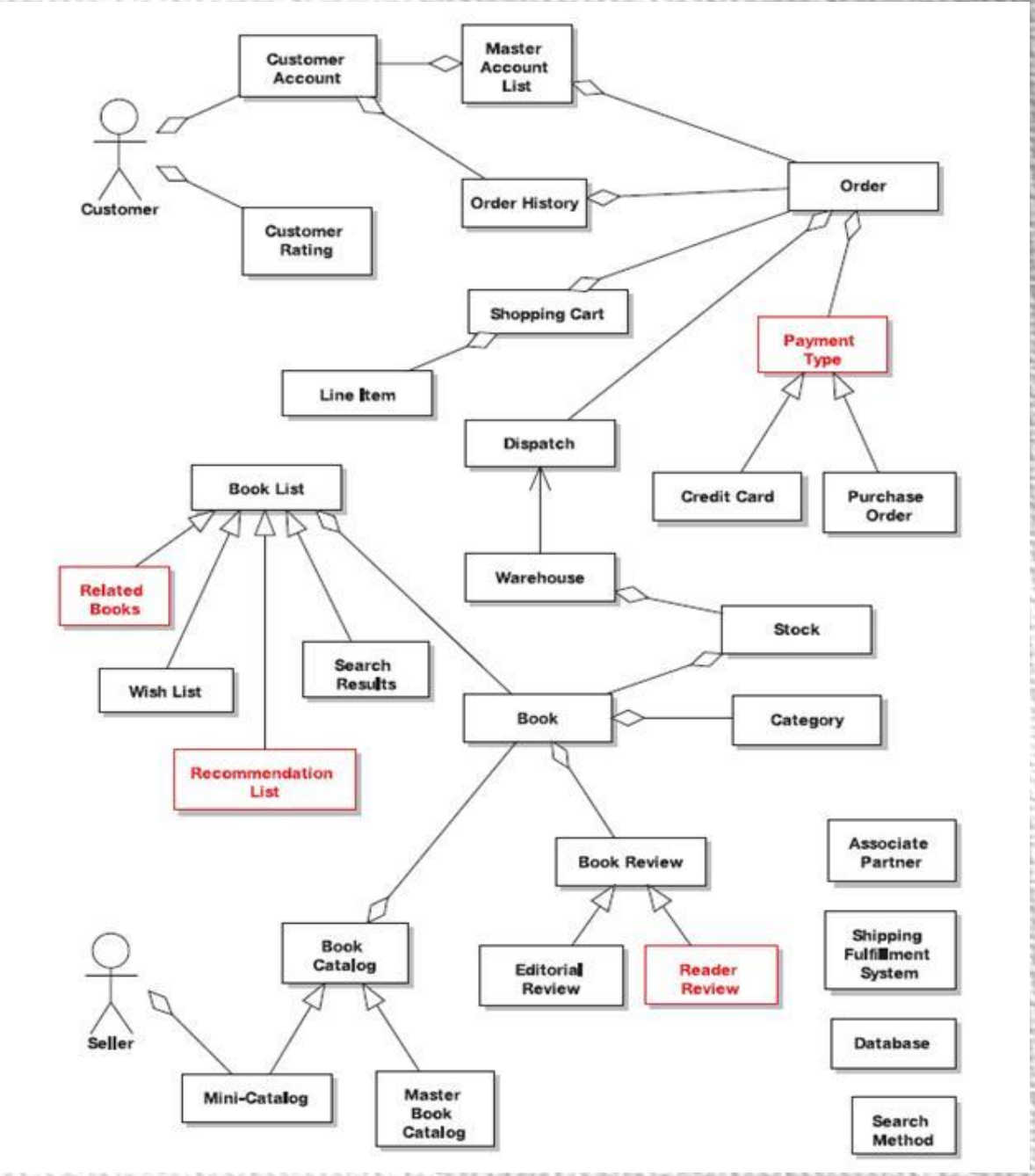


Каталог книг (Book catalog) является хорошей кандидатурой для базового класса, потому что Миникаталог и Основной каталог книг являются разновидностями Каталога книг. Список книг – также может быть базовым классом, потому что у нас достаточно много различных списков книг.

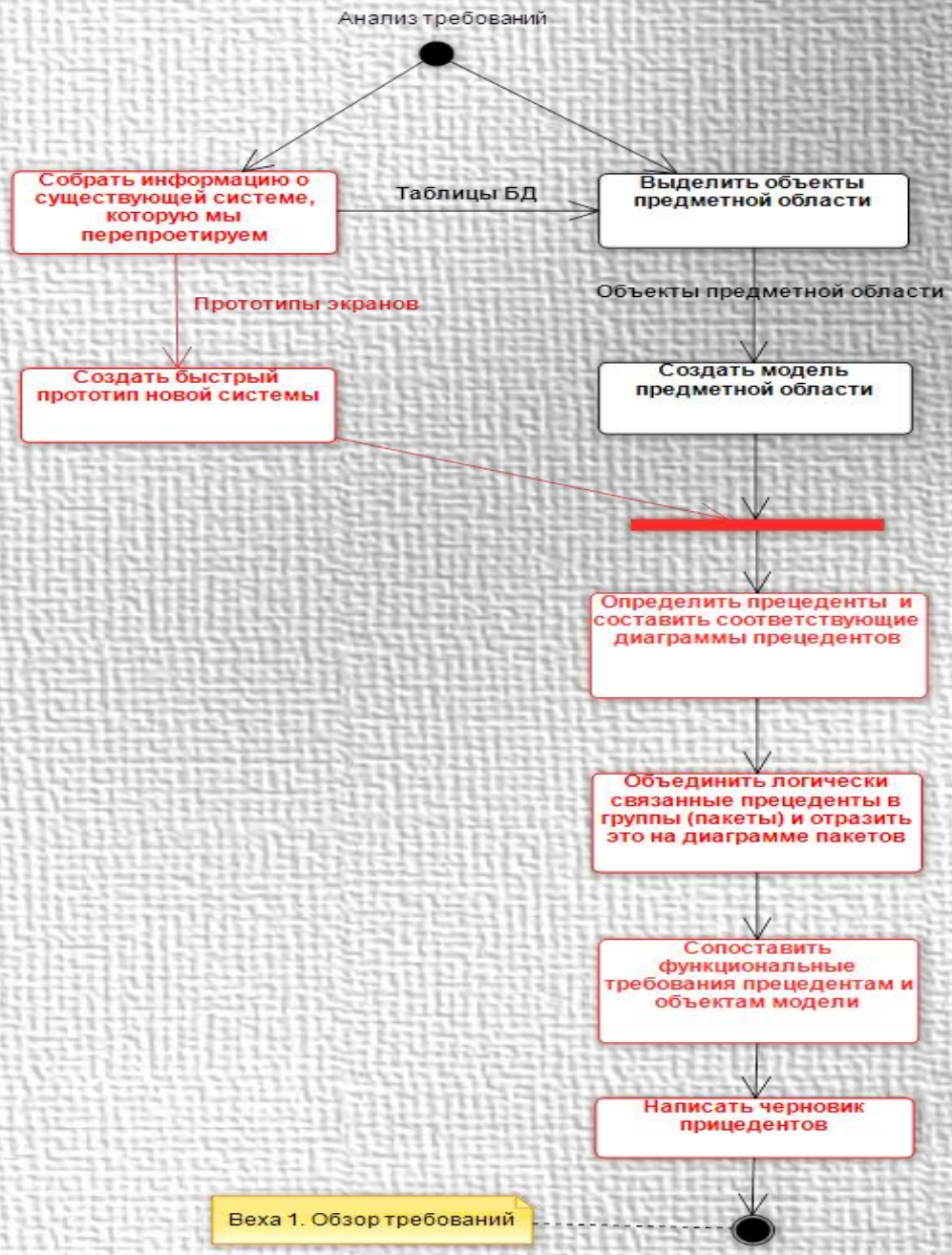


Можно сказать, что Книга принадлежит Списку книг, а Связанные по тематике книги являются разновидностью Списка книг.

Диаграмма, на которую мы ввели отношения is-a, показана ниже. При выявлении отношений мы обнаружили несколько новых классов. Они показаны красным. Кроме того, мы ввели новый класс Способ оплаты (Payment Type), который отделил Банковскую карту и Счет на оплату от Заказа.

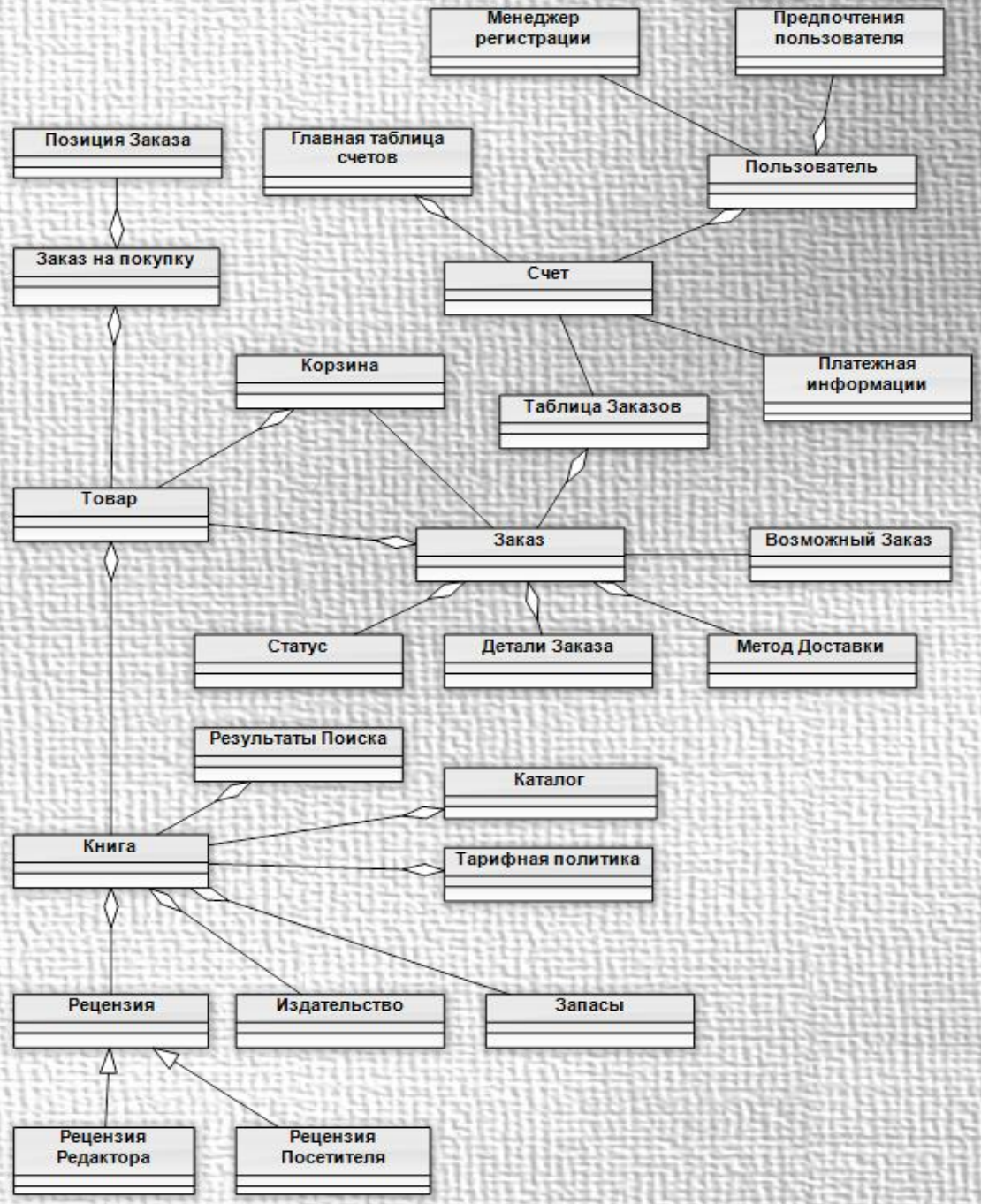


Таким образом, на данном этапе мы проделали работу, место которой в общем процессе можно показать с помощью диаграммы деятельности.



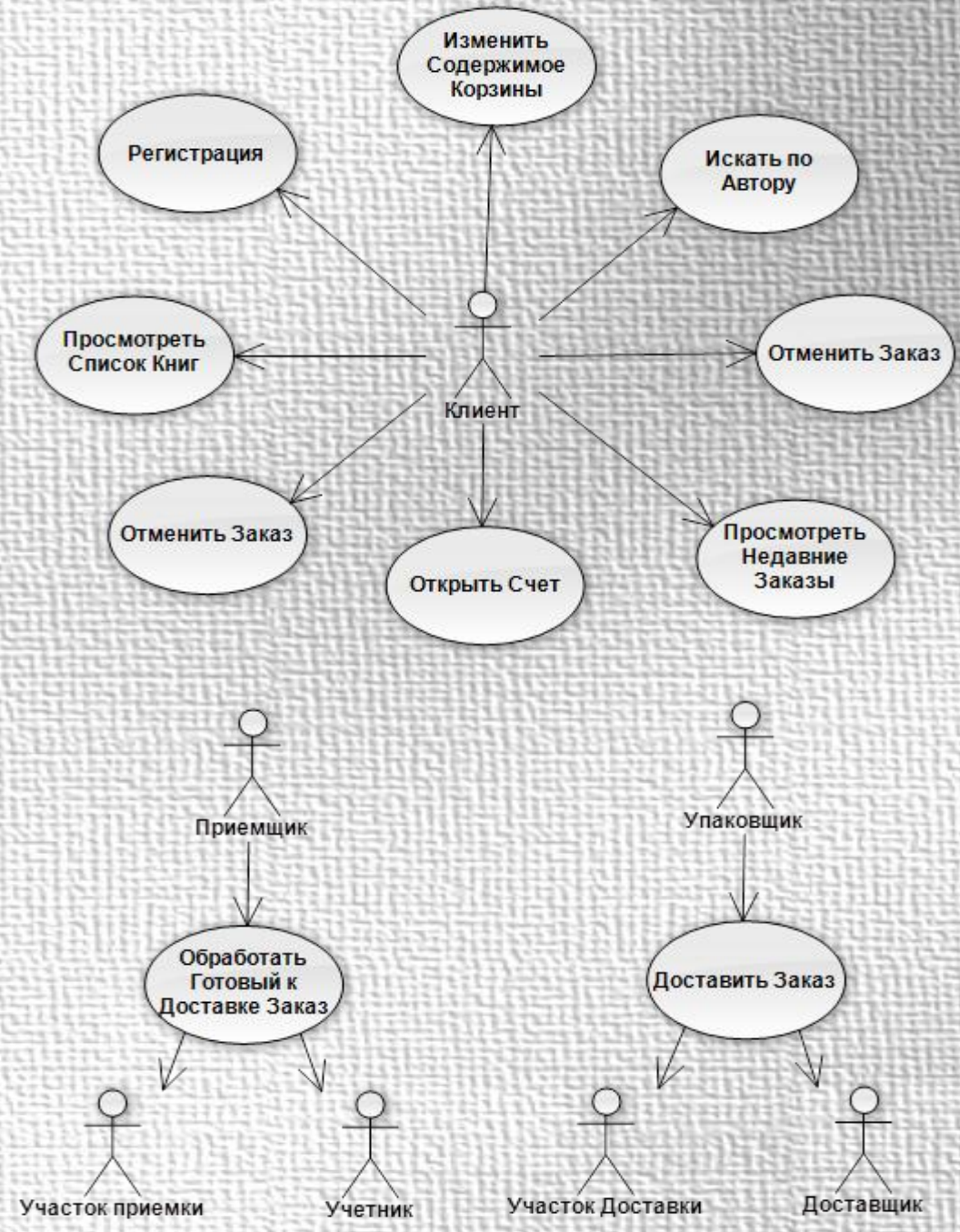
Модель предметной области Internet магазина (диаграмма классов)

На рисунке приведена альтернативная модель предметной области для книжного Internet-магазина. Диаграмма объединяет фрагменты, встречающиеся в упражнениях, а также содержит классы и ассоциации, которые будут рассматриваться позднее



Готовая диаграмма прецедентов

На рисунке показана полная диаграмма прецедентов для книжного Internet.-магазина. В частности, на ней присутствуют и прецеденты, которые упоминались в упражнениях, и задействованные в них актеры.



МОДЕЛЬ СОСТОЯНИЙ ПРЕДМЕТНОЙ ОБЛАСТИ

Некоторые объекты предметный области за время своей жизни сменяют несколько качественно различных состояний. В этих состояниях они могут иметь разные ограничения на значение атрибутов, разные ассоциации или краткости, выполнять разные операции или иметь разное поведение и т.д. Часто бывает полезно построить диаграмму состоянии для таких классов. Диаграмма состоянии описывает различные состояния, в которых может находиться объект, свойства объекта и действующие на них ограничения, а также события, вызывающий переход объекта из одного состояния в другое. Большинство классов предметной области не требуют использования диаграмм состояний. Для их описания достаточно списка операций. Модель состояний может помочь в понимании поведения всех классов, который могут находиться существенно разных расстояниях.

Модель состоянии предметной области конструируются в несколько этапов:

- выявление классов, обладающих разными состояниями;
- выделение состояний;
- выделение событий;
- построении диаграмм состояний;
- проверка диаграмм состояний.

Для модели состоянии предметной области важны только те события, который вызывает переходы. Информацию, содержащуюся в событии, следует представлять в форме списка его параметр .

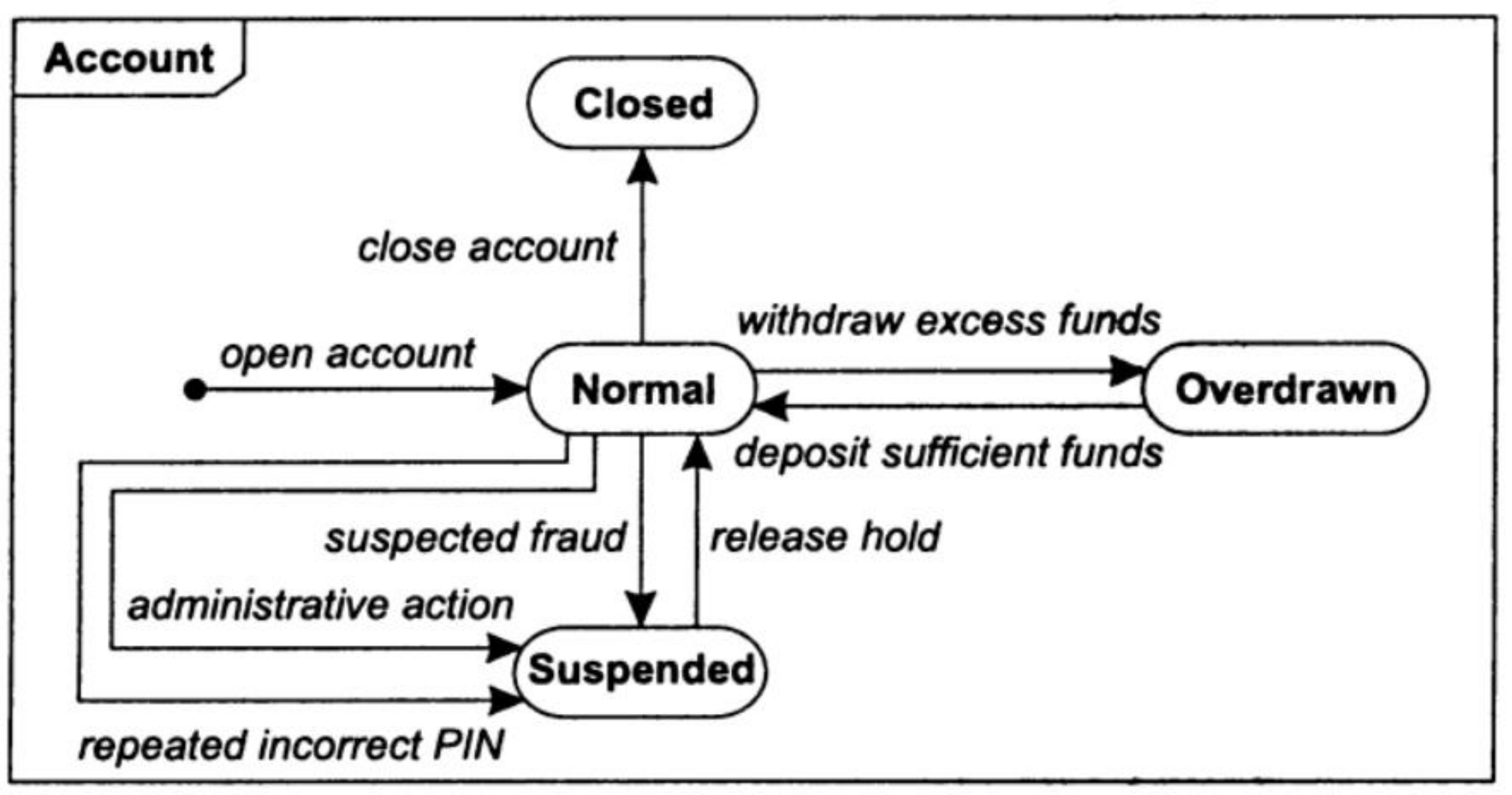
Пример с банкоматом

Класс *Account* (Счёт) может находиться в состояниях:

1. ***Normal*** (нормальное— обычный режим доступа),
2. ***Closed*** (закрытый— клиент закрыл свой счёт, но он ещё не был исключён из записей банка),
3. ***Overdrawn*** *(* с превышенным кредитным лимитом— клиент превысил кредитный лимит по своему счёту)
4. ***Suspended*** (приостановлен - доступ к счёту был заблокирован по каким либо причинам).

Перечислим важнейшие события в нашей модели:

- *close account* (закрытие счёта)
- *withdraw excess* funds (превышение кредитного лимита)
- *repeated incorrect PIN* (повторный неправильный ввод PIN кода)
- *suspected* (возможная подделка)
- *administrative action* (административные действия)


Account
Closed
close account
withdraw excess funds
open account
Normal
Overdrawn
deposit sufficient funds
suspected fraud
release hold
administrative action
Suspended
repeated incorrect PIN

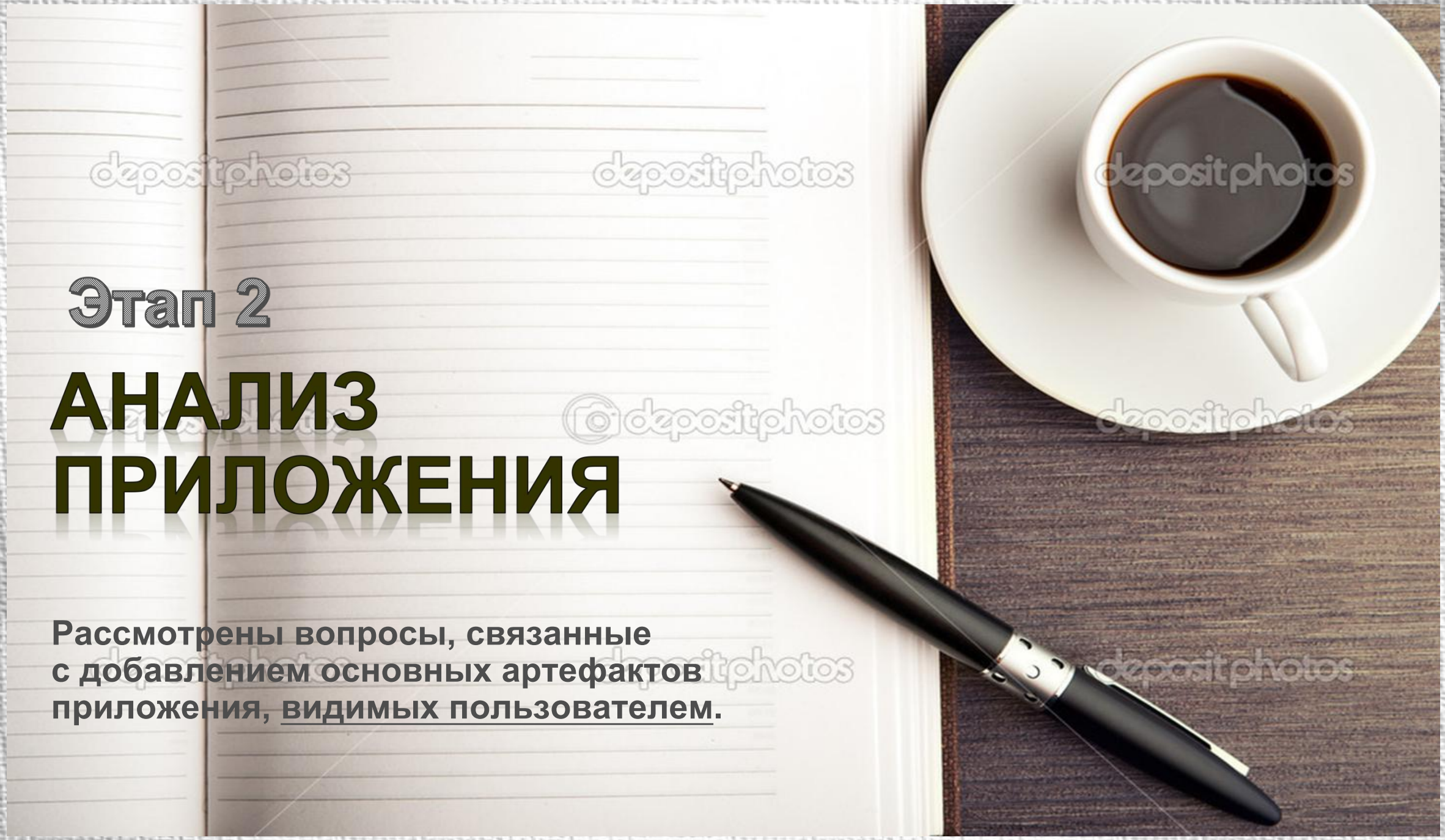
Проверка диаграмм состояний

Проверьте каждую модель состояний. Все ли состояние достижимы?

Особенное внимание нужно уделить маршрутом по диаграмме.

- Если она описывает класс с развивающимся поведением, есть ли на диаграмме маршрут, соединяющий начальное состояние с конечным?
- Присутствуют ли на ней ожидаемые отклонения от основной последовательности ?
- Если диаграмма описывает класс с циклическим поведением, есть ли на ней основная петля цикла?
- Есть ли тупиковые состояния, завершающие цикл?

:



Этап 2

АНАЛИЗ ПРИЛОЖЕНИЯ

Рассмотрены вопросы, связанные с добавлением основных артефактов приложения, видимых пользователем.

МОДЕЛИРОВАНИЕ ИСПОЛЬЗОВАНИЯ (модель взаимодействия)

На этом этапе вы можете нарисовать предварительную диаграмму вариантов использования. На ней следует показать действующие лица и варианты использования, соединив их между собой. Обычно вариант использования можно соединить с действующим лицом, которое его инициирует, но вообще говоря в нём могут участвовать и другие действующие лица. Не беспокойтесь о том, что вы пропустите их. В процессе проработки вариантов использования это станет очевидно. Кроме того, для каждого варианта использования полезно сформулировать его краткое описание в одно-два предложения.

Большинство моделей предметной области являются статическими. Операции в этих моделях не отражается, потому что предметная область как целое обычно ничего не выполняет. Суть моделирования предметной области состоит в том, чтобы охватить внутренние концепции. Завершив разработку модели предметной области, мы переходим к рассмотрению взаимодействия. Моделирование взаимодействия следует начинать с определения внешней границы системы. Затем следует выделить варианты использования и детально проработать их в сценариях и на диаграммах последовательности. Для сложных и неочевидных вариантов использования нужно построить диаграммы деятельности. После того как вы полностью разберетесь в вариантах использования, их можно упорядочить при помощи отношений. Наконец, необходимо провести проверку модели взаимодействия по модели классов предметной области.

Этапы построения

Модель взаимодействия для приложения строится в несколько этапов:

1. Определить границу системы.
2. Выделить действующие лица.
3. Выделить варианты использования.
4. Выделить начальные и конечные события.
5. Подготовить типовые сценарии.
6. Добавить сценарии, описывающие вариации и исключительные ситуации.
7. Выделить внешние события.
8. Построить диаграммы деятельности для сложных вариантов использования.
9. Структурировать действующие лица и варианты использования.
10. Выполнить проверку по модели классов предметный области.

Стадии точности

Действующие лица и цели. Перечислите действующих лиц и каждую из их целей, которые будет обеспечивать система. Проанализируйте правильность и полноту этого списка. Расставьте приоритеты и распределите цели между командами разработчиков и версиями. Теперь у вас есть функциональные требования к 1 уровню точности.

Изложение вариант использования или основной сценарий. Для вариантов использования вы выбрали следование основному сценарию в общих чертах. Рассмотрите его в форме набросков, чтобы быть уверенным, что система действительно удовлетворяет интересам участников, о которых вы заботитесь. Это 2 уровень точности функциональных требований.

Условия отказа. Завершите основной сценарий и продумайте, какие отказы могут произойти. Занесите их в список, прежде чем решать, как система должна их обрабатывать. Описание процесса обработки отказов более трудоёмкая работа, чем перечисление отказов. Те, кто начинает описывать обработку отказов немедленно, часто выдыхаются, не закончив списка условий отказов.

Обработка отказа. Напишите как система должна реагировать на каждый отказ. Это часто непростой, утомительный и непредсказуемый процесс. Непредсказуемый, потому что довольно часто вопрос о малопонятном бизнес-правиле всплывает во время работы над описанием или при обработке отказа неожиданно обнаружится новое действующее лицо или новая цель, достижения которой должна обеспечить система.

Что дальше?

После того, как построена диаграмма прецедентов встает естественный вопрос: что дальше? То есть, как далее следует продолжать моделирование средствами UML?

Вообще говоря, ответ может быть такой: ничего больше не делать средствами UML. Представление использования, если оно тщательно продумано и детально прорисовано, является формой технического задания (спецификацией функциональных требований), содержащей достаточно информации для дальнейшего проектирования и реализации любым другим методом. Может статься, что в коллективах разработчиков, в совершенстве владеющих какой-либо другой методикой проектирования и реализации, такое ограниченное использование UML окажется вполне оправданным.

Переход от моделирования использования к другим видам моделирования

UML содержит средства не только для моделирования использования, но и для поддержки всех остальных фаз процесса разработки. Какие есть возможные пути?

Действующие лица находятся вне системы – с ними ничего делать не нужно. Можно сказать, что действующие лица уже выполнили свою задачу, просто появившись в модели системы. Таким образом, переход от моделирования использования к другим видам моделирования состоит в **уточнении, детализации и конкретизации вариантов использования**. В представлении использования мы показали, что делает система, теперь нужно определить, как это делается. Назовем это реализацией вариантов использования.

РЕАЛИЗАЦИЯ ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ

Повторим еще раз: вариант использования – это описание множества последовательностей событий или действий (сценариев), доставляющих значимый для действующего лица результат. Наиболее часто используемый метод описания множества последовательностей действий состоит в указании алгоритма, выполнение которого доставляет последовательность действий из требуемого множества. Существует множество способов описания алгоритмов, более или менее формальных.

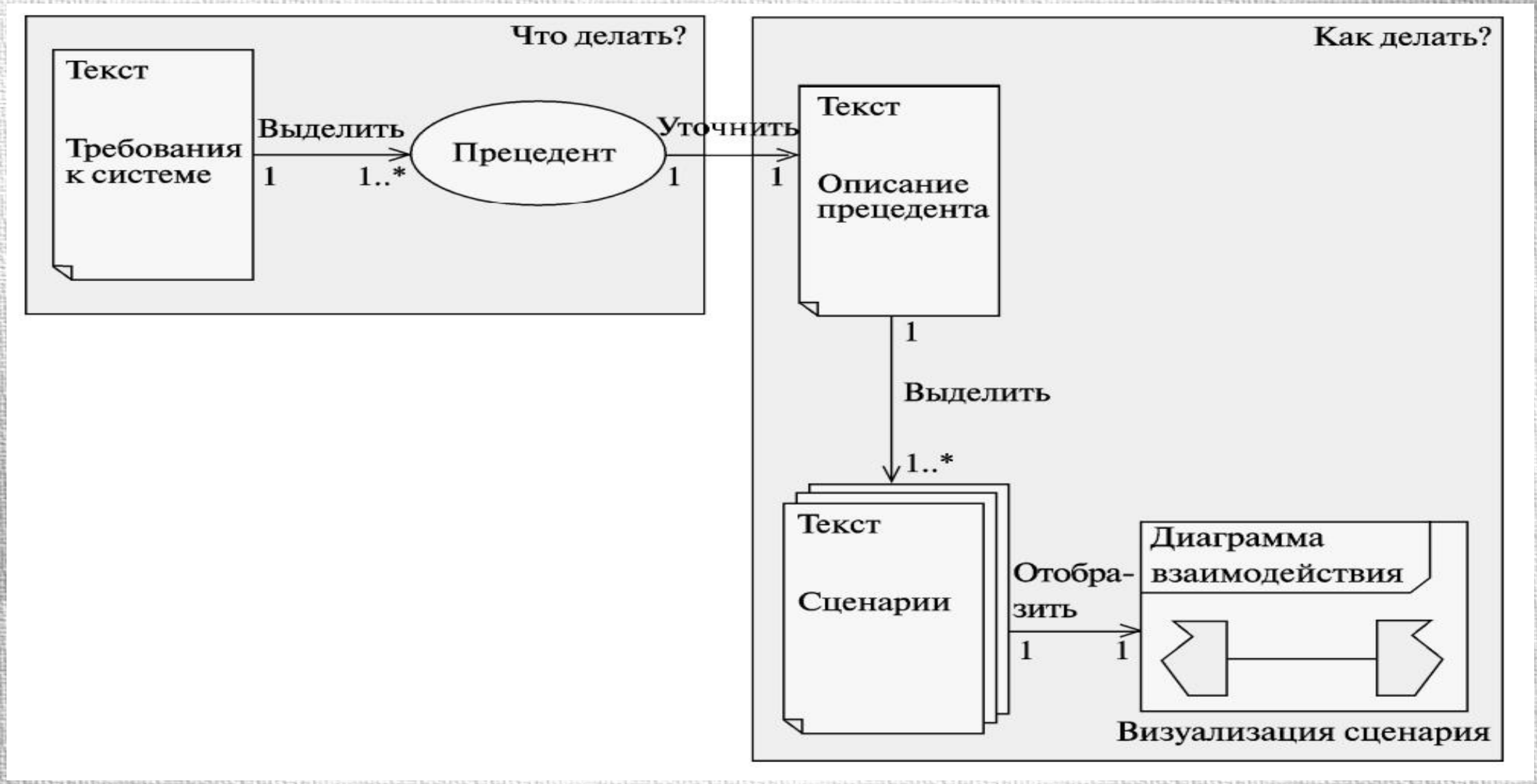
Мы рассмотрим четыре, часто применяемых именно при реализации вариантов использования.

1. Документирование потока событий
2. Реализация программой на псевдокоде
3. Реализация диаграммами деятельности
4. Реализация диаграммами взаимодействия

Указанные вопросы реализации прецедентов выделены в отдельный обзор!

Конечно, только диаграмм прецедентов, как и сценариев, ими определяемых, недостаточно, чтобы создать модель поведения системы. Как уже не раз упоминалось, прецеденты говорят, что делает система, но не говорит как. Об этом говорят сценарии, но в текстовой форме, что делает их довольно сложными для восприятия. На помощь приходят диаграммы взаимодействий, которые визуализируют сценарии.

Моделирование при помощи диаграмм прецедентов



Основные артефакты этапа анализа

- Модель предметной области (диаграмма классов – ключевых абстракций)
- Модель вариантов использования
- Сценарии вариантов использования
  - типовые сценарии
  - сценарии, описывающие вариации и исключительные ситуации.
- Диаграммы деятельности для сложных и неочевидных вариантов использования

ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОЕКТИРОВАНИЕ

Основная задача – усовершенствование модели анализа таким образом, чтобы она была реализована с использованием компонентов требуемой архитектуры.

Программирование – сердце инженерии программ, но лишь часть инженерии. Концепции инженерии направлены на разработку систем, которые могут быть большими, разрабатываться в течение долгого периода, подвержены изменениям, удовлетворяют строгим критериям качества, временных сроков и стоимости.

Бертран Мейер

Подобно строительной индустрии, над сложной системой должен поработать архитектор, прежде чем она будет построена. Она должна быть разбита на модули, используя обобщение и метод «разделяй и властвуй». Каждый модуль затем должен быть тщательно специфицирован и определены его интерфейсы к другим модулям, прежде чем его отдавать программистам для кодирования.

Проектирование — процесс определения **архитектуры, компонентов, интерфейсов** и других характеристик системы или её части (ISO 24765). Результатом проектирования является проект — целостная совокупность моделей, свойств или характеристик, **описанных в форме, пригодной для реализации системы.**

Проектирование, наряду с анализом требований, является частью большой стадии жизненного цикла системы, называемой определением системы (англ. system definition). Результаты этой стадии являются входной информацией для стадии реализации (воплощения) системы (англ. system realization).[

Среди разработчиков популярна старая шутка о том, что чтобы считаться архитектором, надо просто отвечать на все технические вопросы: «ну, зависит от… ».

– *Вопрос*: Как реализовать слой доступа к данным?

– *Ответ*: Ну, зависит от…

– *Вопрос*: Какую технологию необходимо применить для UI моего решения?

– *Ответ*: Ну, зависит от…

– *Вопрос*: Как сделать приложение масштабируемым?

– *Ответ*: Ну, зависит от…

Но основная идея в том, что ведь на самом деле зависит. В конечном счете, каждое решение уникально, и существует множество факторов, как технических, так и не технических, которые могут существенно влиять на архитектуру и дизайн решения любых масштабов. Задача разработчика и архитектора решения – найти равновесие между (зачастую противоречащими друг другу) требованиями и ограничениями, налагаемыми бизнесом, конечным пользователем, ИТ-средой и инфраструктурой управления организации, экономической ситуацией и, конечно же, технологиями и инструментальными средствами, используемыми для создания решения.

Предисловие к *Руководству Microsoft по проектированию архитектуры приложения*

ВВЕДЕНИЕ

Во время анализа требований к системе основное внимание уделялось выяснению того, **что** должно быть сделано, вне зависимости от того, как это сделать. На этапе разработки системы решается вопрос, **как** реализовать решения, принятые на этапе анализа.

Внимательно рассмотрев практику программирования компьютеров, вы заметите, что очень немногое изменилось за 50 лет. Языки стали немного лучше. Инструменты стали фантастически лучше. Но основные строитель-ные блоки компьютерных программ остались прежними.

Я написал очень много приложений, я создал очень много систем. И благо-даря накопленному опыту я пришел к потрясающему выводу:

Все архитектуры подчиняются одним и тем же правилам!

Потрясающему, потому что все системы, в создании которых я участвовал, радикально отличались друг от друга. Почему архитектуры таких разных систем подчиняются одним и тем же правилам? Я пришел к выводу, что правила создания архитектур не зависят от любых других переменных.

Внимательно рассмотрев практику программирования компьютеров, вы заметите, что очень немногое изменилось за 50 лет. Языки стали немного лучше. Инструменты стали фантастически лучше. Но основные строительные блоки компьютерных программ остались прежними.

Несмотря на появление новых языков, фреймворков и парадигм, правила остались теми же, как во времена, когда в 1946-м Алан Тьюринг написал первый программный код.

Изменилось только одно: тогда, в прошлом, мы не знали этих правил. Поэтому нарушали их снова и снова. Теперь, с полувековым опытом за плечами, мы знаем и понимаем правила

Роберт Мартин
Чистая архитектура

Пример из практики

Рис. 1.1. Рост численности инженерно-технического персонала. Воспроизводится с разрешения автора презентации Джейсона Гормана (Jason Gorman)

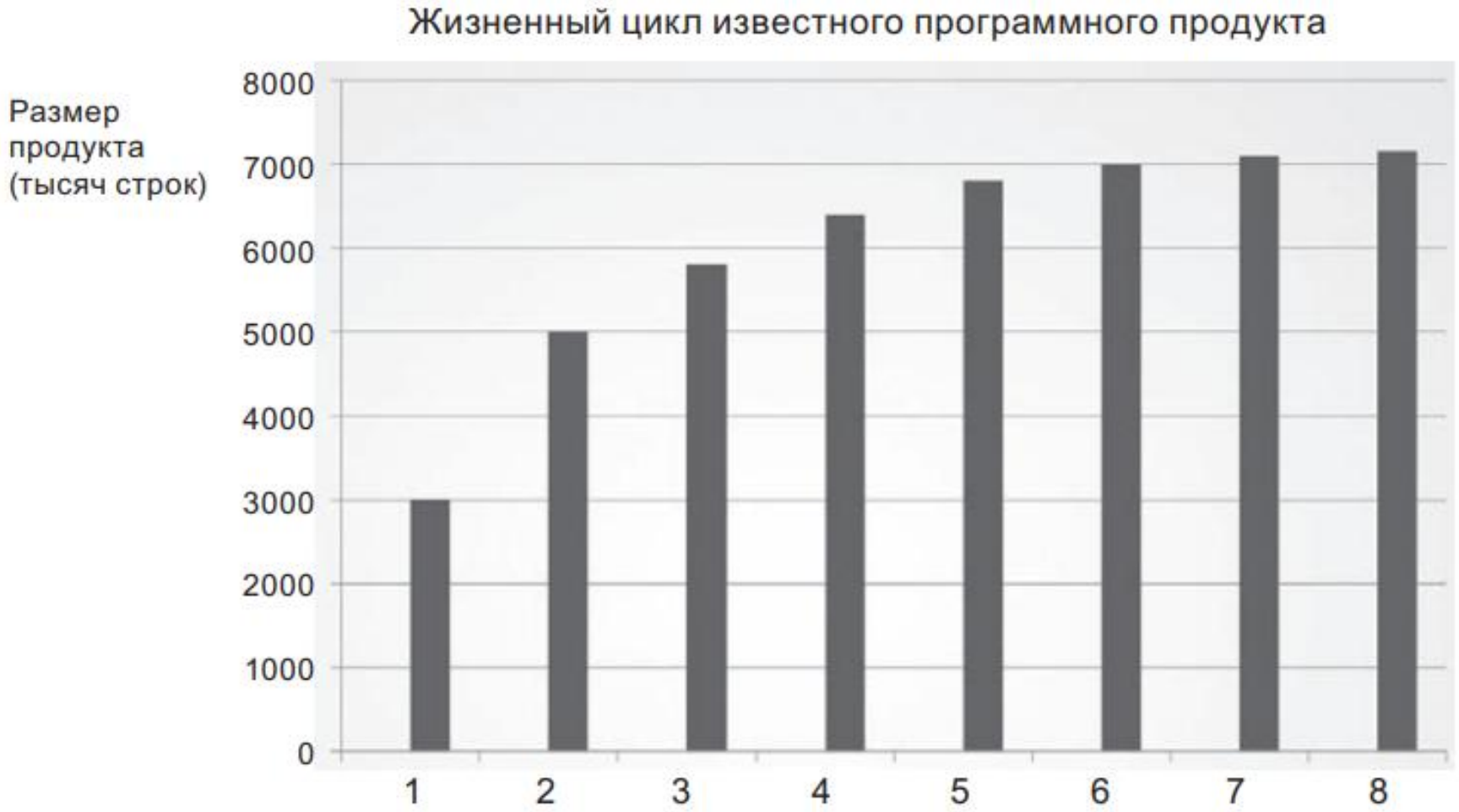


Рис. 1.2. Продуктивность за тот же период

Рис. 1.3. Изменение стоимости строки кода с течением времени

Рис. 1.4. Изменение продуктивности с выпуском новых версий

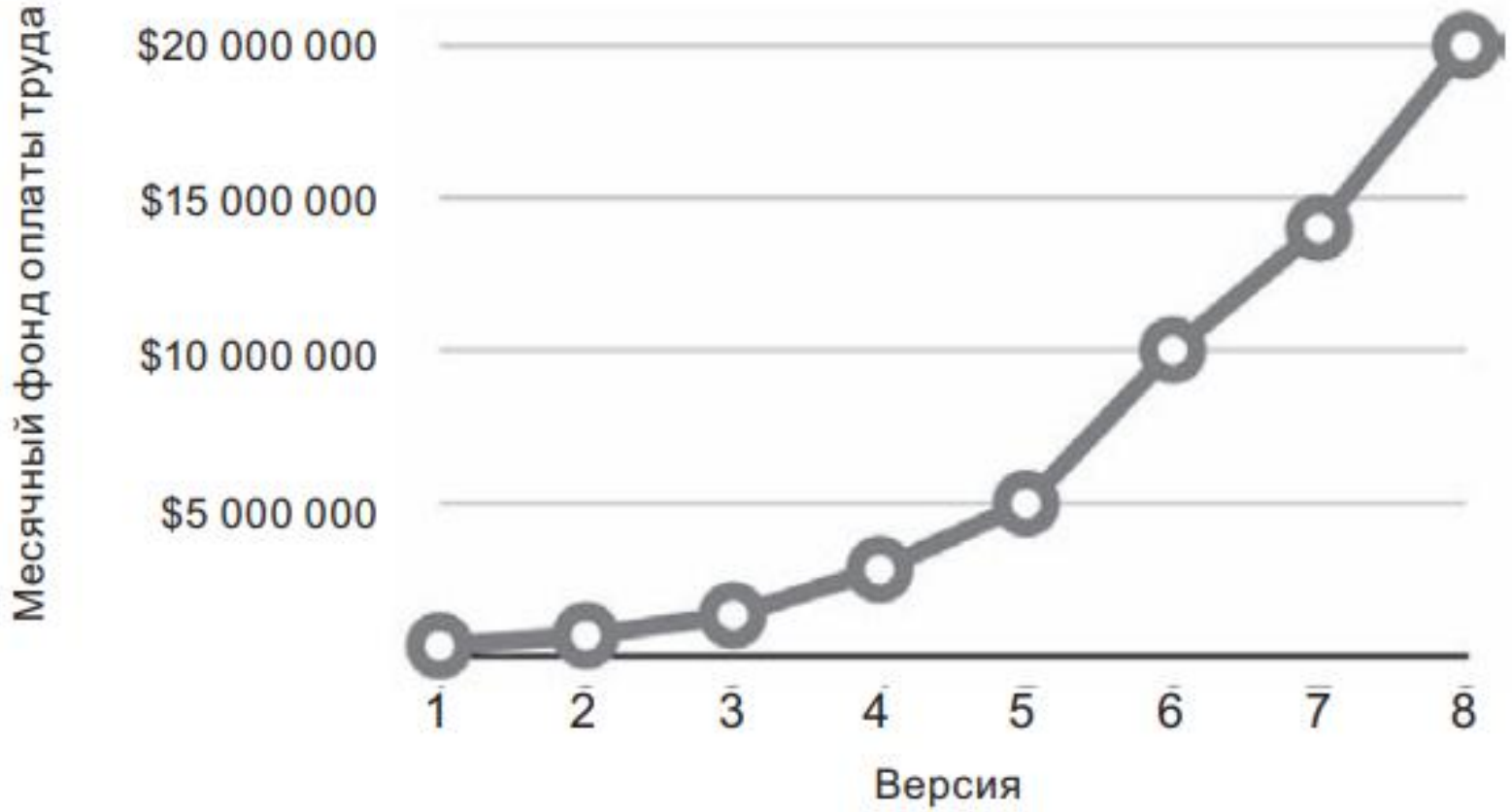
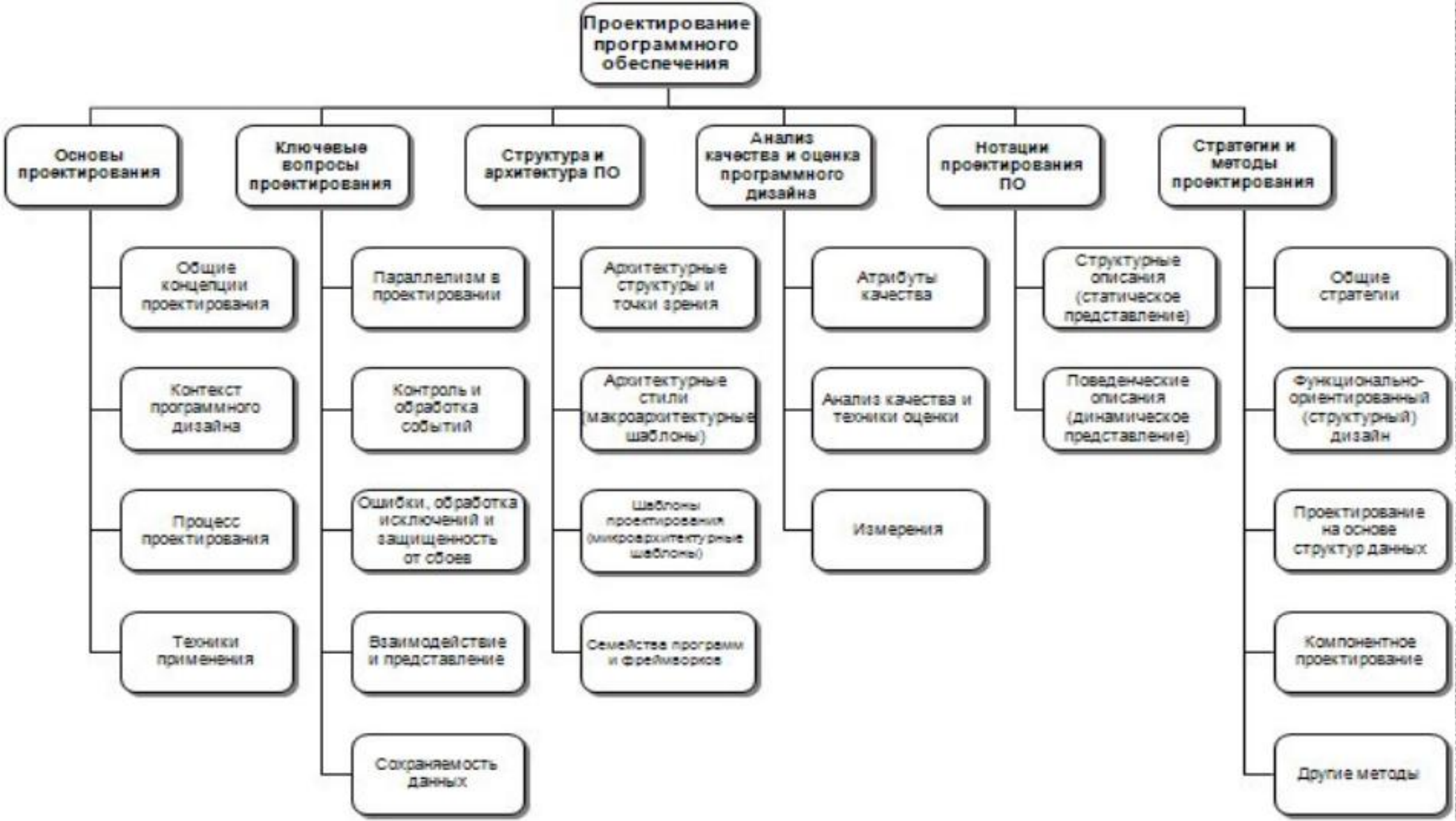


Рис. 1.5. Изменение фонда оплаты труда разработчиков с выпуском новых версий


Проектирование программного обеспечения
Основы проектирования
Общие концепции проектирования
Контекст программного дизайна
Процесс проектирования
Техники применения
Ключевые вопросы проектирования
Параллелизм в проектировании
Контроль и обработка событий
Ошибки, обработка исключений и защищенность от сбоев
Взаимодействие и представление
Сохраняемость данных
Структура и архитектура ПО
Архитектурные структуры и точки зрения
Архитектурные стили (макроархитектурные шаблоны)
Шаблоны проектирования (микроархитектурные шаблоны)
Семейства программ и фреймворков
Анализ качества и оценка программного дизайна
Атрибуты качества
Анализ качества и техники оценки
Измерения
Нотации проектирования ПО
Структурные описания (статическое представление)
Поведенческие описания (динамическое представление)
Стратегии и методы проектирования
Общие стратегии
Функционально-ориентированный (структурный) дизайн
Проектирование на основе структур данных
Компонентное проектирование
Другие методы

Цель проектирования

Основной **целью анализа** было построение логической модели системы, отражающей функциональность, которую должна предоставлять система для удовлетворения требований пользователя.

Цель проектирования – определить в полном объеме, как будет реализовываться эта функциональность. Одним из путей решения этой задачи является одновременное изучение предметной области и области решения. Требования поступают из предметной области, и анализ можно рассматривать как ее исследование с точки зрения заказчиков системы. Проектирование предполагает объединение технических решений (библиотек классов, механизмов сохранения объектов и т. д.) для создания модели системы (проектной модели), которая может быть реализована в действительности.

Основные принципы проектирования

Приступая к работе над архитектурой приложения, необходимо помнить об основных принципах проектирования. Это поможет создать архитектуру, которая будет следовать проверенным подходам, обеспечит минимизацию затрат, простоту обслуживания, удобство использования и расширяемость. Рассмотрим основные принципы:

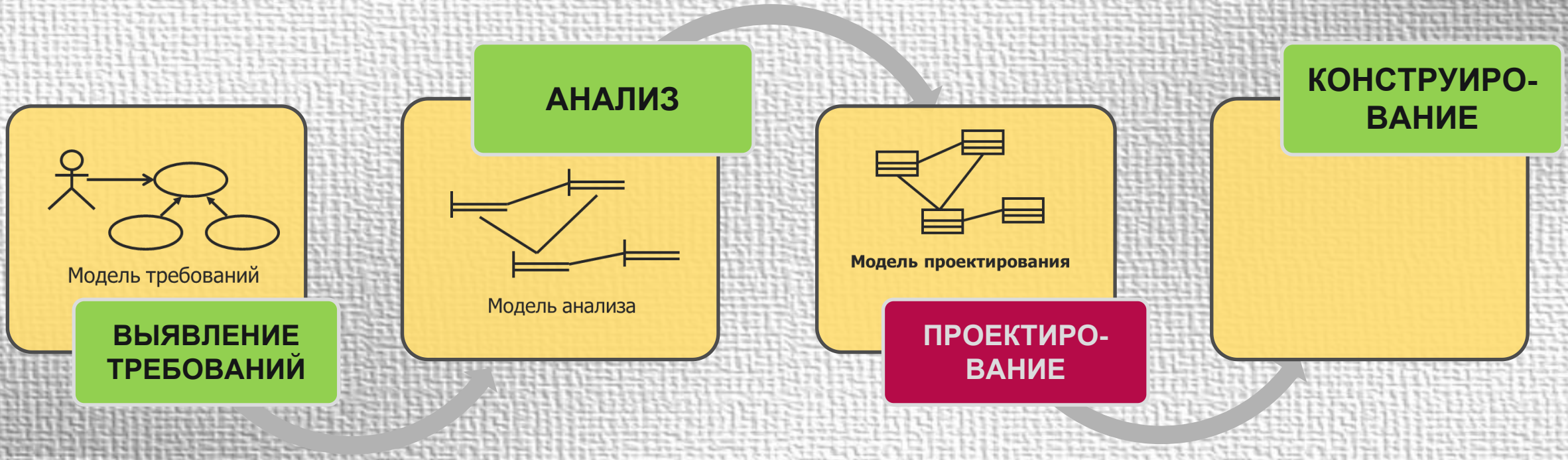
1. **Разделение функций**. Разделите приложение на отдельные компоненты, по возможности, с минимальным перекрытием функциональности. Важным фактором является предельное уменьшение количества точек соприкосновения, что обеспечит высокую связность (high cohesion) и слабую связанность (low coupling). Неверное разграничение функциональности может привести к высокой связанности и сложностям взаимодействия, даже несмотря на слабое перекрытие функциональности отдельных компонентов.

2. **Принцип единственности ответственности**. Каждый отдельно взятый компонент или модуль должен отвечать только за одно конкретное свойство/функцию или совокупность связанных функций.
3. **Принцип минимального знания** (также известный как Закон Деметера (Law of Demeter, LoD)). Компоненту или объекту не должны быть известны внутренние детали других компонентов или объектов.
4. **Не повторяйтесь (Don’t repeat yourself, DRY)**. Намерение должно быть обозначено только один раз. В применении к проектированию приложения это означает, что определенная функциональность должна быть реализована только в одном компоненте и не должна дублироваться ни в одном другом компоненте.

5. **Минимизируйте проектирование наперед.** Проектируйте только то, что необходимо. В некоторых случаях, когда стоимость разработки или издержки в случае неудачного дизайна очень высоки, может потребоваться полное предварительное проектирование и тестирование. В других случаях, особенно при гибкой разработке, можно избежать масштабного проектирования наперед (big design upfront, BDUF). Если требования к приложению четко не определены, или существует вероятность изменения дизайна со временем, старайтесь не тратить много сил на проектирование раньше времени. Этот принцип называют YAGNI («You ain’t gonna need it»).

МОДЕЛИ ЭТАПОВ РАЗРАБОТКИ

Модель анализа



Модель требований

1. Бизнес-модель
2. Спецификация требований
3. Словарь

Модель проектирования

1. Модель данных
2. Документ Архитектура

От аналитической модели к проектной

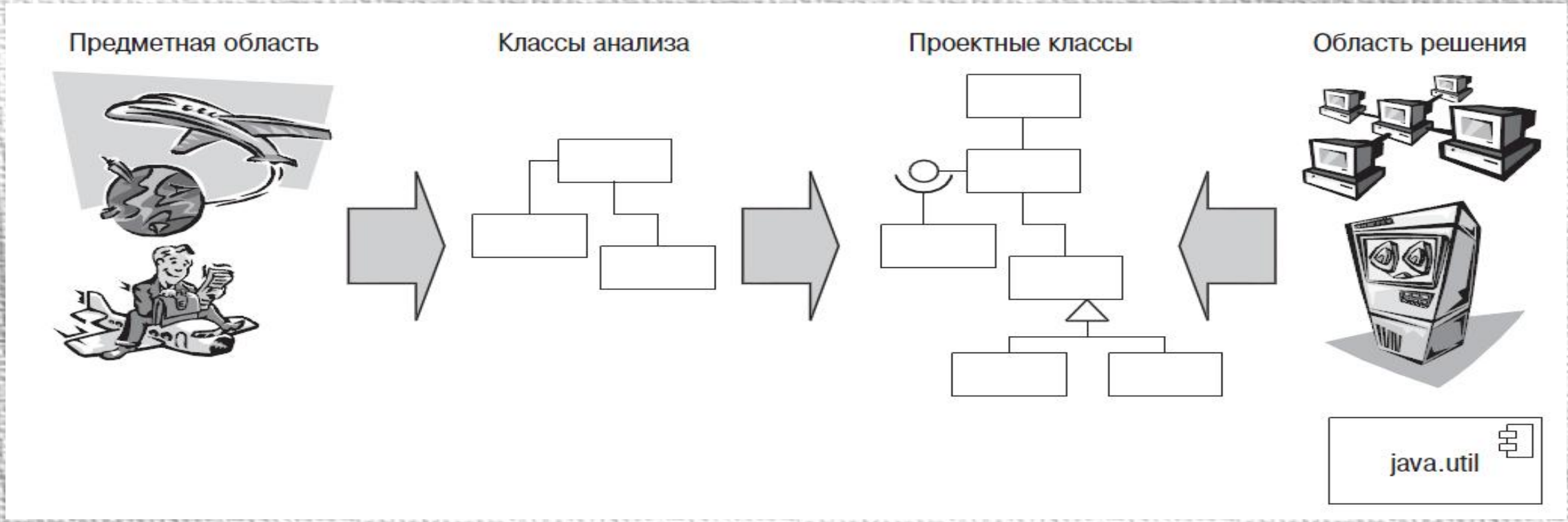
По мере завершения анализа фокус моделирования перемещается на проектирование. В значительной степени анализ и проектирование могут проводиться параллельно. Однако, как мы увидим позже, важно четко различать артефакты этих двух рабочих потоков – **аналитическую модель** и **проектную модель**.

Рекомендуют не разделять функции аналитиков и проектировщиков. За разработку артефактов (таких как прецедент) – от требований через анализ и проектирование до реализации – должна отвечать одна команда.

Модель проектирования является абстракцией реализации системы. Она применяется для представления, а также для документирования эскиза системы программного обеспечения. Это обширный, составной рабочий продукт, заключающий все классы проектирования, подсистемы, пакеты, кооперации и взаимосвязи между ними.

Проектная модель

Проектная модель базируется на **модели анализа** и может считаться просто ее улучшенной и уточненной версией.



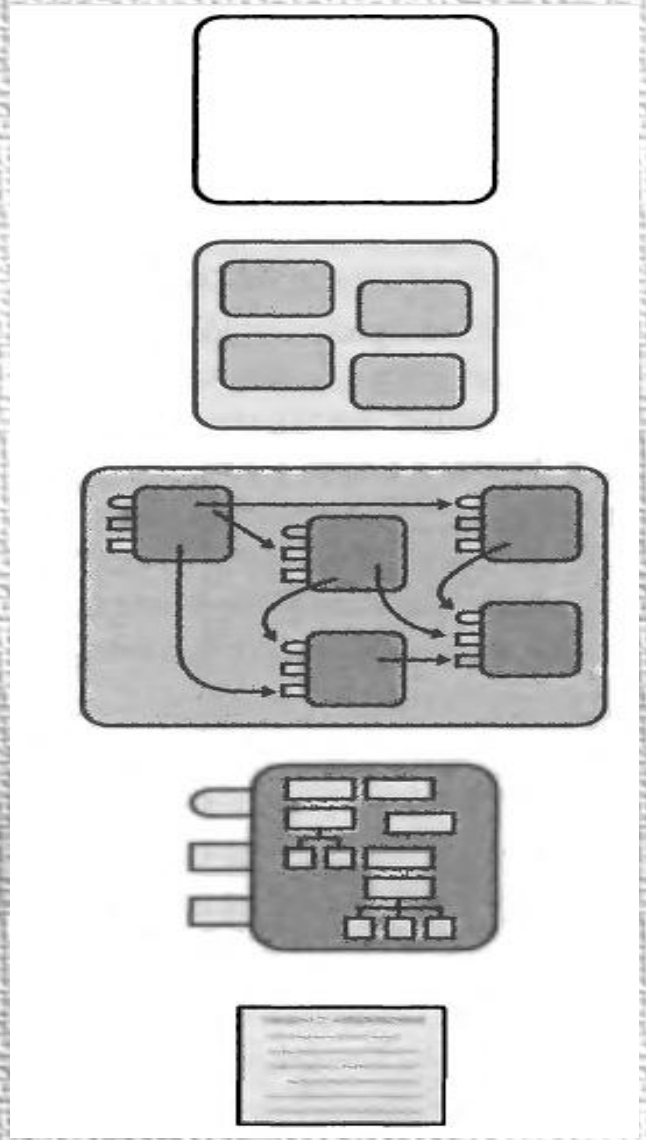
Проект предметной области

Проект ПрО – это набор документов, содержащих текст и диаграммы, которые служат основой для создания полного программного кода приложения.

Полный проект ПрО должен быть описан настолько детально, чтобы программист мог создавать на его основе код приложения без необходимости использовать какие-либо другие документы.

Проект ПрО аналогичен набору чертежей здания, которых достаточно подрядчику, для построения требуемого здания.

УРОВНИ ПРОЕКТИРОВАНИЯ



1. Программная система
2. Разделение системы на подсистемы/пакеты
3. Разделение пакетов на классы
4. Разделение классов на данные и методы
5. Проектирование методов

Уровень ①: программная система

Первому уровню проектирования соответствует вся система. Некоторые программисты с системного уровня сразу переходят к проектированию классов, но обычно целесообразно обдумать более высокоуровневые комбинации классов, такие как подсистемы или пакеты.

Уровень ②: разделение системы на подсистемы или пакеты

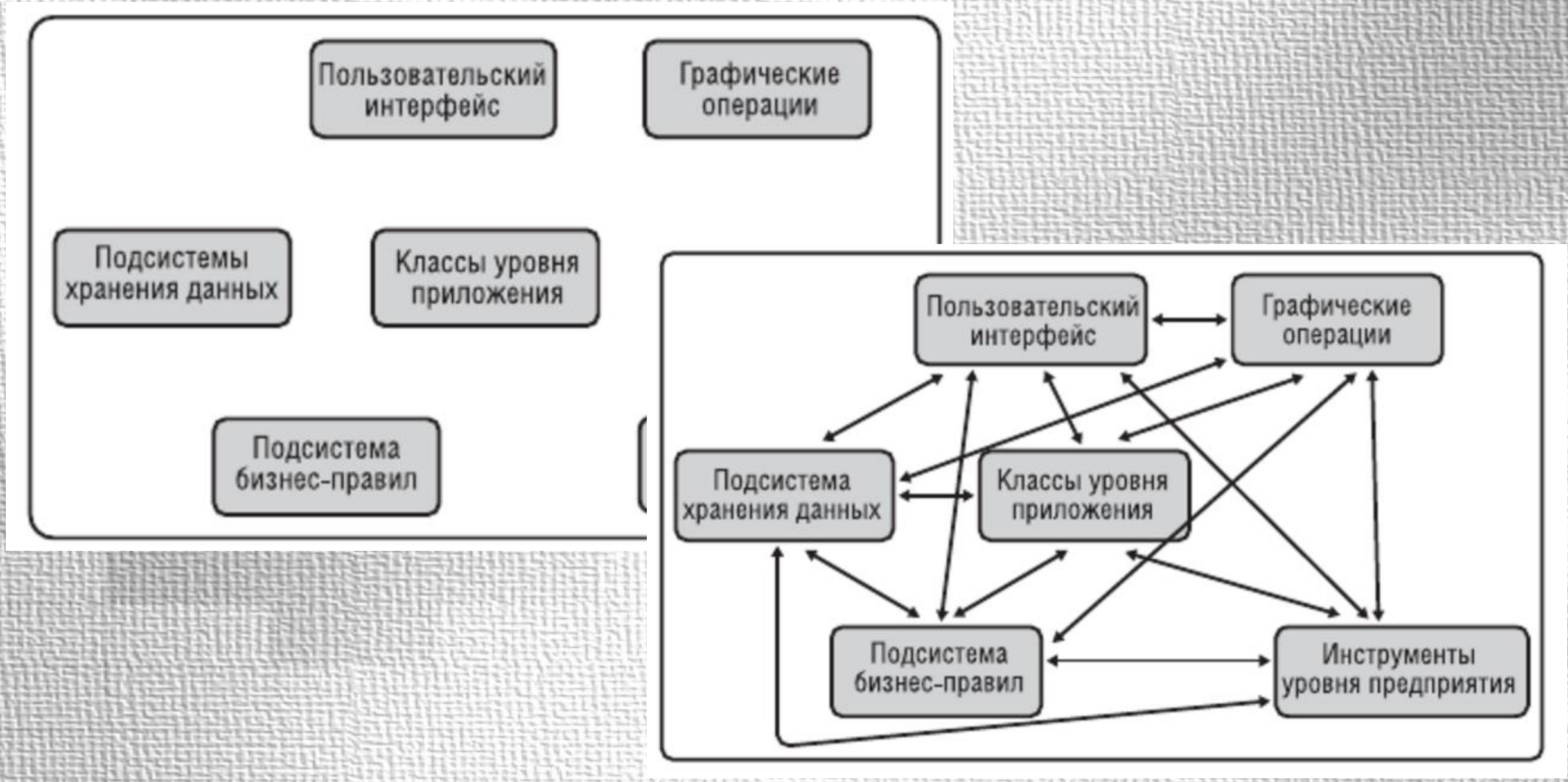
Главный результат проектирования на этом уровне — определение основных подсистем. Подсистемы могут быть довольно крупными, такими как модуль работы с базами данных, модули GUI, бизнес-правил или создания отчетов, интерпретатор команд и т. д. Суть проектирования на данном уровне заключается в разделении программы на основные подсистемы и определении взаимодействий между подсистемами. Обычно этот уровень нужен при работе над любыми проектами, требующими более нескольких недель.

Особенно важный аспект этого уровня — определение правил взаимодействия подсистем. Если все подсистемы могут взаимодействовать друг с другом, выгода их разделения исчезает.

Подчеркивайте суть подсистем, ограничивая их взаимодействие между собой.

Допустим, вы определили систему из шести подсистем. При отсутствии каких-либо ограничений в силу второго закона термодинамики энтропия любой системы должна увеличиться. Один из способов увеличения энтропии является абсолютно свободное взаимодействие между подсистемами.

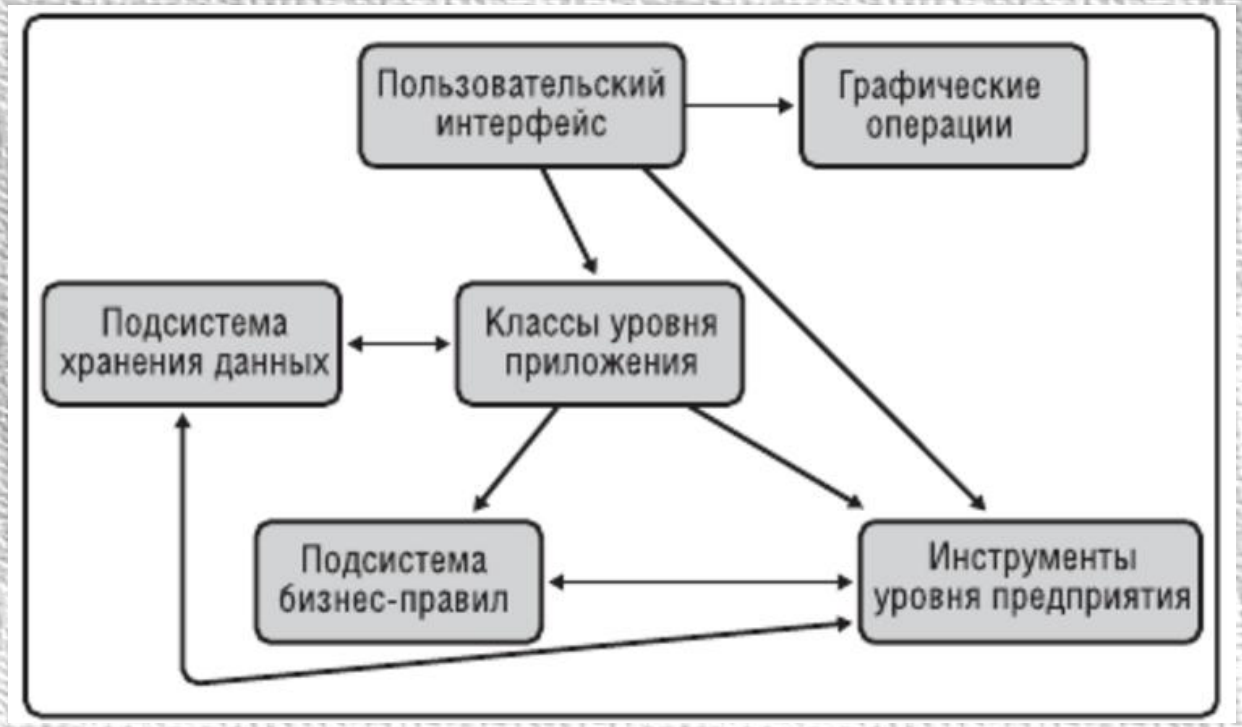
Пример системы, включающей шесть подсистем



Как видите, в итоге все подсистемы начинают напрямую взаимодействовать, что поднимает несколько важных вопросов:

- в скольких разных частях системы нужно хоть немного разбираться разработчику, желающему изменить какой-то аспект подсистемы графических операций?
- что будет, если вы попытаетесь задействовать данный модуль бизнес-правил в другой системе?
- что будет, если вы захотите включить в систему новый пользовательский интерфейс (например, интерфейс командной строки, удобный для проведения тестирования)?
- что произойдет, если вы захотите перенести модуль хранения данных на удаленный компьютер?

Стрелки между подсистемами можно рассматривать как шланги с водой. Если вам захочется «выдернуть» одну из подсистем, к ней наверняка будут подключены несколько шлангов. Чем больше шлангов вам нужно будет отсоединить и подключить заново, тем сильнее вы промокнете. Архитектура системы должна быть такой, чтобы замена подсистем требовала как можно меньше возни со шлангами.



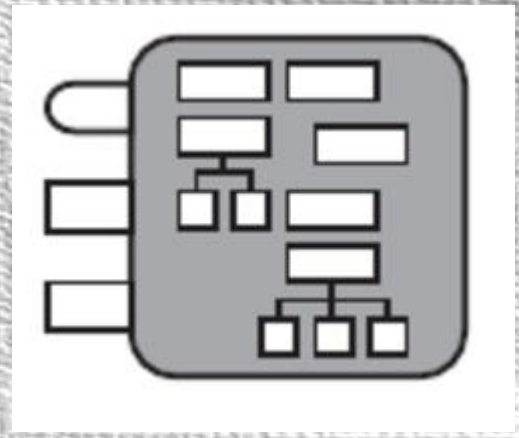
Чтобы соединения подсистем были понятными и легкими в сопровождении, старайтесь поддерживать простоту отношений между подсистемами.

- Самым простым отношением является то, при котором одна подсистема вызывает методы другой.
- Более сложное отношение имеет место, когда одна подсистема содержит классы другой.
- **Самое сложное отношение — наследование классов одной подсистемы от классов другой.**

Придерживайтесь одного разумного правила: диаграмма системного уровня вроде той, что показана на рисунке, должна быть ациклическим графом. Иначе говоря, программа не должна содержать циклических отношений, при которых класс А использует класс В, класс В использует класс С, а класс С — класс А.

Уровень ③: разделение подсистем на классы

Этот уровень проектирования предполагает определение всех классов системы.



Например, подсистема доступа к БД может быть далее разделена на классы доступа к данным и классы хранения данных, а также метаданные БД.

Кроме того, на этом уровне следует определить детали взаимодействия каждого класса с остальными элементами системы, особенно интерфейс класса. В целом сутью проектирования на данном уровне является декомпозиция подсистем до такого уровня детальности, который позволит реализовать части подсистем в форме отдельных классов.

Уровень ④: разделение классов на методы

Данный уровень проектирования заключается в разделении каждого класса на методы. Некоторые методы уже будут определены на уровне 3, при проектировании интерфейсов классов. На уровне 4 вы детально определите закрытые методы классов. При этом многие методы окажутся простыми, тогда как другие будут включать иерархии методов и потребуют дополнительного проектирования.

Полное определение методов класса часто позволяет лучше понять его интерфейс, что может подтолкнуть к соответствующему изменению интерфейса, т. е. к возвращению на уровень 3.

Уровень ⑤: проектирование методов

На этом уровне проектирование заключается в детальном определении функциональности отдельных методов, за что обычно отвечают отдельные программисты, работающие над конкретными методами. Данный уровень может включать такие действия, как написание псевдокода, поиск алгоритмов в книгах, размышление над оптимальной организацией фрагментов метода и написание кода. Этот уровень проектирования выполняется во всех случаях, но не всегда осознанно и качественно.

ПРИНЦИПЫ ПРОЕКТИРОВАНИЯ СИСТЕМ

1. **Модульность**

Разделение системы на более мелкие, управляемые модули помогает снизить ее сложность, улучшить удобство обслуживания и возможности повторного использования.

2. **Абстракция**

Чтобы упростить сложные системы, можно скрыть подробности их реализации и демонстрировать только основные функции. Такой подход также способствует модульности.

3. **Разбиение на слои**

Распределение системы по слоям, каждый из которых содержит определенный набор функций, помогает адресно работать с проблемами и улучшает удобство обслуживания.

4. **Масштабируемость**

Проектирование системы с учетом возрастания нагрузки на нее. С возросшей нагрузкой можно справляться добавлением дополнительных ресурсов (горизонтальное масштабирование) или оптимизацией мощности системы (вертикальное масштабирование).

5. **Производительность**

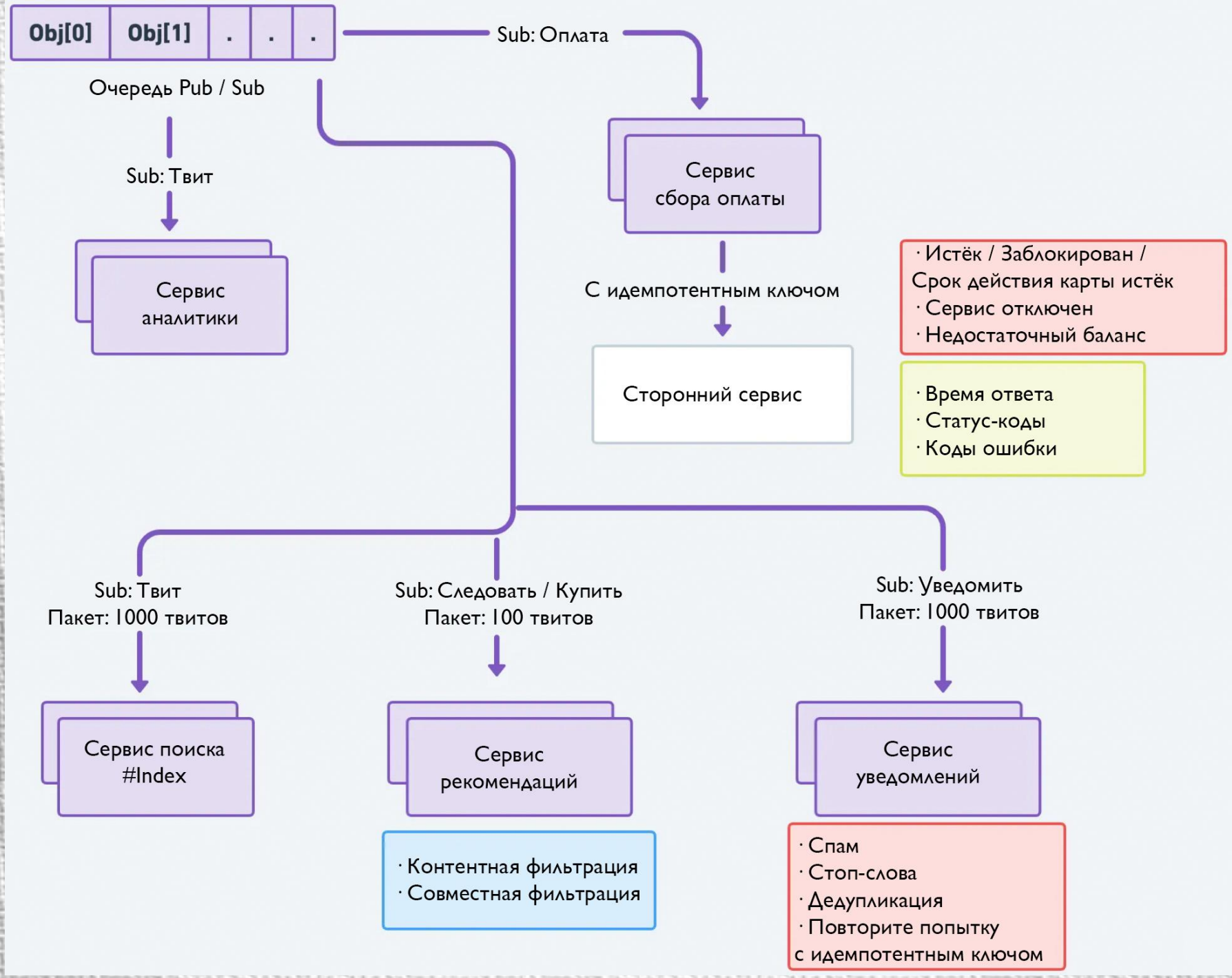
Оптимизация времени отклика, пропускной способности и использования ресурсов — все это важно учитывать для проектирования эффективной системы.

6. **Безопасность**
   - Чтобы обеспечить конфиденциальность, целостность и доступность системы, нужно внедрять соответствующие инструменты обеспечения безопасности.
7. **Отказоустойчивость и способность к восстановлению**
   - Проектирование систем, которые устойчивы к сбоям и могут восстанавливаться после ошибок.
8. **Тестируемость**

СТАНДАРТНЫЕ КОМПОНЕНТЫ ПРИ ПРОЕКТИРОВАНИИ СИСТЕМЫ

В этом разделе мы рассмотрим некоторые стандартные компоненты, часто встречающиеся в современных системах. Понимание этих компонентов поможет разработчикам легко интегрировать их в свои системы и повышать их общую функциональность.

- *Платежный сервис*
- *Сервис аналитики*
- *Сервисы уведомлений*
- *Поисковые сервисы*
- *Служба рекомендаций*


Obj[0]
Obj[1]
.
.
.
Очередь Pub / Sub
Sub: Оплата
Sub: Твит
Сервис аналитики
Сервис сбора оплаты
С идемпотентным ключом
Сторонний сервис
· Истёк / Заблокирован / Срок действия карты истёк
· Сервис отключен
· Недостаточный баланс
· Время ответа
· Статус-коды
· Коды ошибки
Sub: Твит
Пакет: 1000 твитов
Sub: Следовать / Купить
Пакет: 100 твитов
Sub: Уведомить
Пакет: 1000 твитов
Сервис поиска #Index
Сервис рекомендаций
Сервис уведомлений
· Контентная фильтрация
· Совместная фильтрация
· Спам
· Стоп-слова
· Дедупликация
· Повторите попытку с идемпотентным ключом

Платежный сервис

Платежные сервисы обрабатывают транзакции между клиентами и компаниями. Для сервисов электронной коммерции и платформ, основанных на подписке, критически важно интегрировать надежный платежный сервис. Примеры популярных сервисов: Stripe, PayPal и Square. Обычно они предоставляют API для обеспечения безопасности транзакций и управления повторяющимися платежами, возвратами и т. п.

Сервис аналитики

Сервисы аналитики позволяют собирать, обрабатывать и визуализировать данные, помогая компаниям принимать взвешенные решения. Они могут отслеживать поведение пользователей, контролировать производительность системы и анализировать тенденции. Примеры таких сервисов — это Google-Analytics, Mixpanel и Amplitude. Интеграция сервисов аналитики в систему поможет компаниям оптимизировать свои предложения и улучшить клиентский опыт.

Сервисы уведомлений

Сервисы уведомлений информируют пользователей о новостях, событиях и другой важной информации. Эти сервисы могут доставлять уведомления по почте, SMS, отправлять push-уведомления.

Примеры сервисов: Firebase Cloud Messaging (FCM), Amazon Simple Notification Service (SNS) и Twilio.

Поисковые сервисы

Мощная поисковая система необходима для систем с большим объемом данных или контента. Она должна быть масштабируемой и обеспечивать быстрый и релевантный поиск. Примеры поисковых сервисов — Elasticsearch, Apache Solr и Amazon CloudSearch. Обычно они поддерживают фильтры, полнотекстовый и фасетный поиск. Это позволяет пользователям находить нужную информацию быстро и эффективно.

Служба рекомендаций

Сервисы рекомендаций показывают пользователям персонализированные предложения на основе их предпочтений, поведения и других факторов. Они могут значительно повысить вовлеченность и удовлетворенность пользователей. Методы создания рекомендаций включают коллаборативную и контентную фильтрацию, а также гибридные подходы. Для создания более сложных рекомендаций также можно использовать алгоритмы машинного обучения, такие как матричная факторизация и глубокое обучение.

Две части этапа проектирования

Разделим этап проектирования на две части:

- **Архитектурное проектирование**
- **Дизайн или детальное проектирование**

Логическое и физическое проектирование

Принято различать также два аспекта проектирования:

- **логическое проектирование**, включающее те проектные операции, которые непосредственно не зависят от имеющихся технических и программных средств, составляющих среду функционирования будущего программного продукта;
- **физическое проектирование**, которое заключается в привязке к конкретным техническим и программным средствам среды функционирования.

АРХИТЕКТУРА & ДИЗАЙН

Многие проектировщики разделяют понятия архитектуры и дизайна.

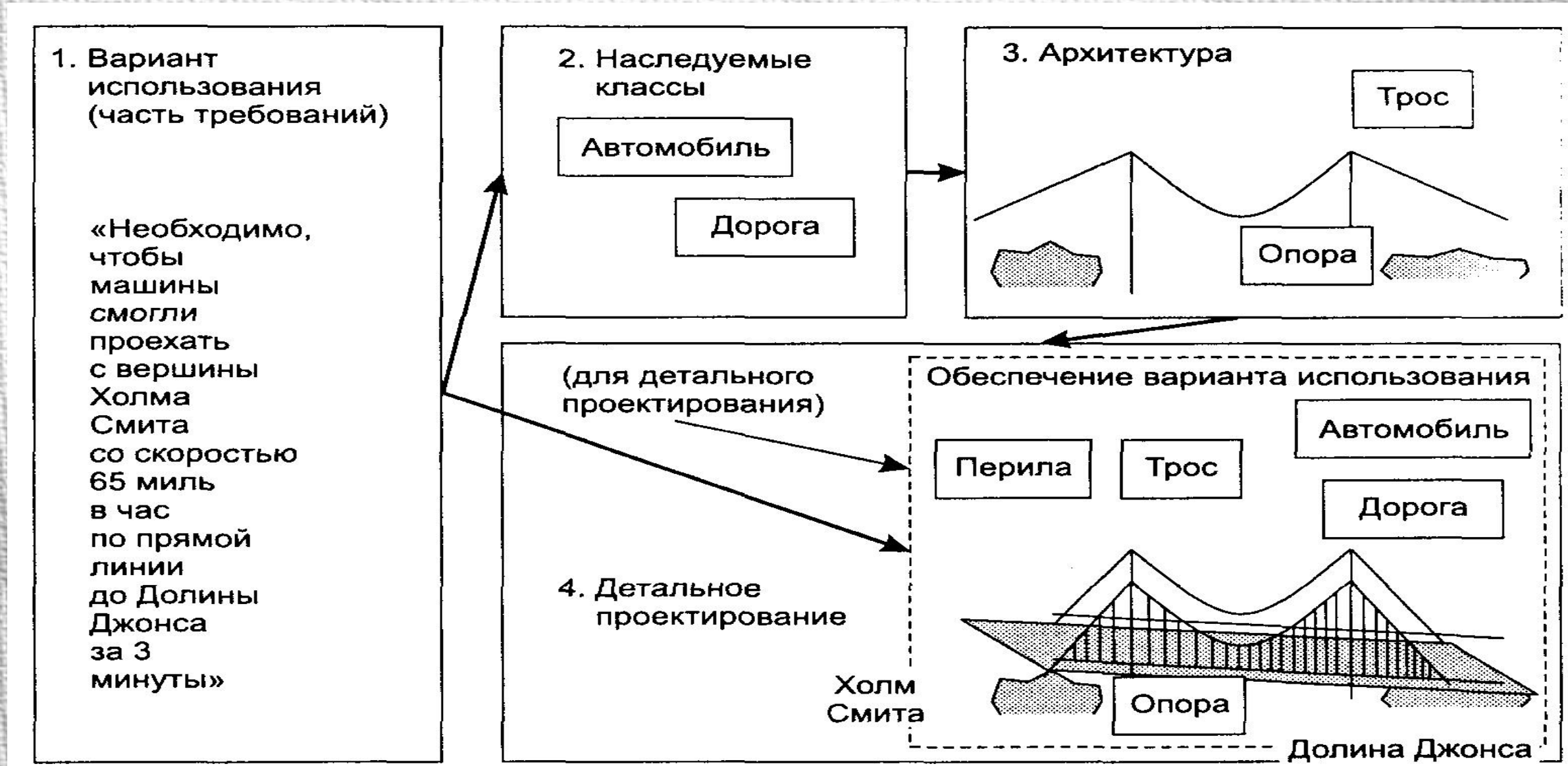
- **Первое понятие больше относится к структуре приложения и отношениям между его частями.**
- **Второе - это больше функциональный аспект взаимодействия элементов (компонентов) архитектуры.**

В настоящее время стандартом де-факто для описания архитектуры является UML.

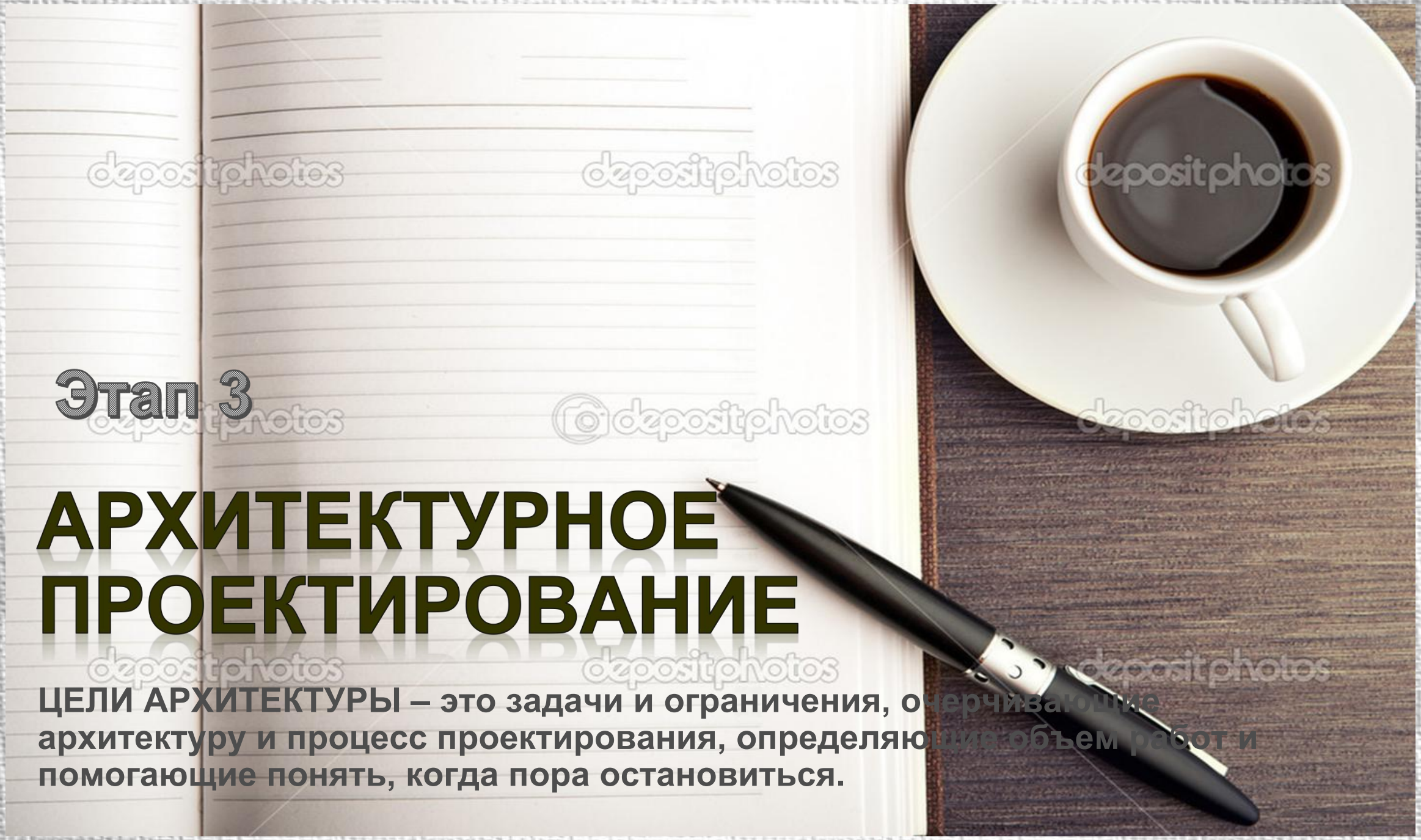
Связь между вариантами использования, архитектурой и детальным проектированием

В проектировании программного обеспечения каждый последующий уровень накапливает дополнительные классы.

- На шаге 1 варианты использования фиксируются как часть требований.
- На шаге 2 они вместе с другими источниками используются для определения классов предметной области.
- На шаге 3 мы разрабатываем программную архитектуру.
- Последний шаг заключается в проверке того факта, что архитектура и детальный проект удовлетворяют требуемым вариантам использования.


1. Вариант использования (часть требований)
«Необходимо, чтобы машины *смогли* проехать с вершины Холма Смита со скоростью 65 миль в час по прямой линии до Долины Джонса за 3 минуты»
2. Наследуемые классы
Автомобиль
Дорога
3. Архитектура
Трос
Опора
(для детального проектирования)
4. Детальное проектирование
Обеспечение варианта использования
Автомобиль
Перила
Трос
Дорога
Опора
Холм Смита
Долина Джонса

Применительно к нашему примеру с проектированием моста мы проверяем то, что автомобили действительно смогут использовать спроектированный мост при поездке с Холма Смита в Долину Джонса, как это было заявлено в вариантах использования. В случае программного проекта мы проверяем то, что классы и методы, специфицированные при детальном проектировании, способны реализовать требуемые варианты использования.



Этап 3

АРХИТЕКТУРНОЕ ПРОЕКТИРОВАНИЕ

ЦЕЛИ АРХИТЕКТУРЫ – это задачи и ограничения, очерчивающие архитектуру и процесс проектирования, определяющие объем работ и помогающие понять, когда пора остановиться.

Можно ли подготовить архитектора в вузе?
Правильный ответ — «НЕТ»
Это все равно что спросить «А где учат на директоров завода?»

Зачем нужна архитектура

Люди плохо умеют прогнозировать будущее. Нам сложно предсказать, как будет меняться программа или требования к ней. Единственное, что мы знаем точно — требования (а значит, и программа) будут меняться.

Грамотная архитектура помогает спроектировать и развивать систему так, чтобы её было проще и удобнее расширять и изменять.

- Если общение между модулями регламентировано, их реализацию проще заменить на другую.
- Если общение с внешним миром регламентировано, меньше шансов для утечки данных.
- Если код разделён грамотно, программу проще тестировать.

Надо различать…

1. **Архитектура предприятия**
2. **Системная архитектура**
3. **Архитектура ПО**

Ниже мы будем говорить только об архитектуре ПО

Архитектура ПО (разработка архитектуры ПО) - это искусство и наука строить и проектировать программное обеспечение таким образом, чтобы оно удовлетворяло всем заявленным к нему требованиям, а также обеспечивало максимальную простоту доработки, развертывания и масштабирования приложения.

Запомните!

Архитектурный анализ связан с определением и удовлетворением нефункциональных (например, качественных) требований к системе в контексте функциональных требований.

Несколько слов о важности архитектуры

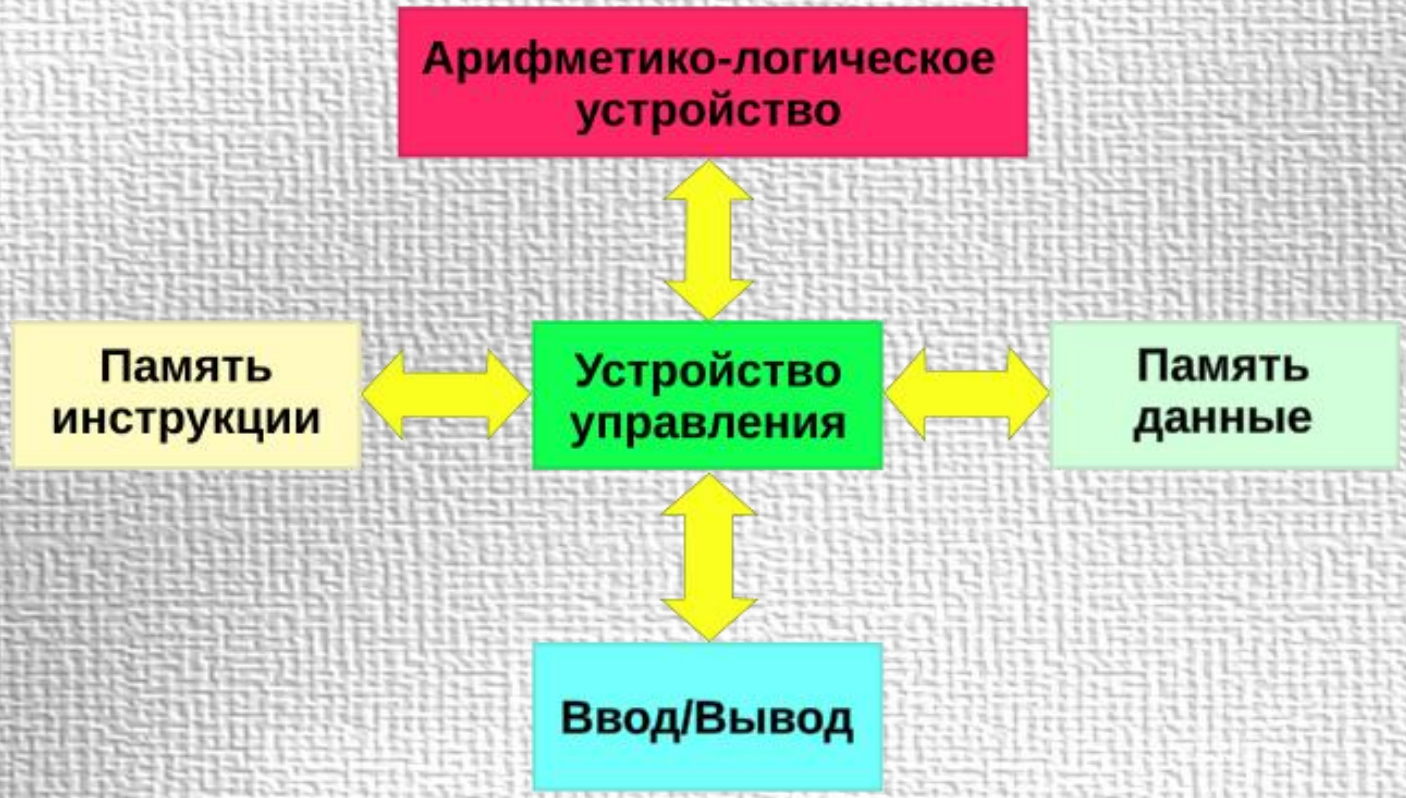
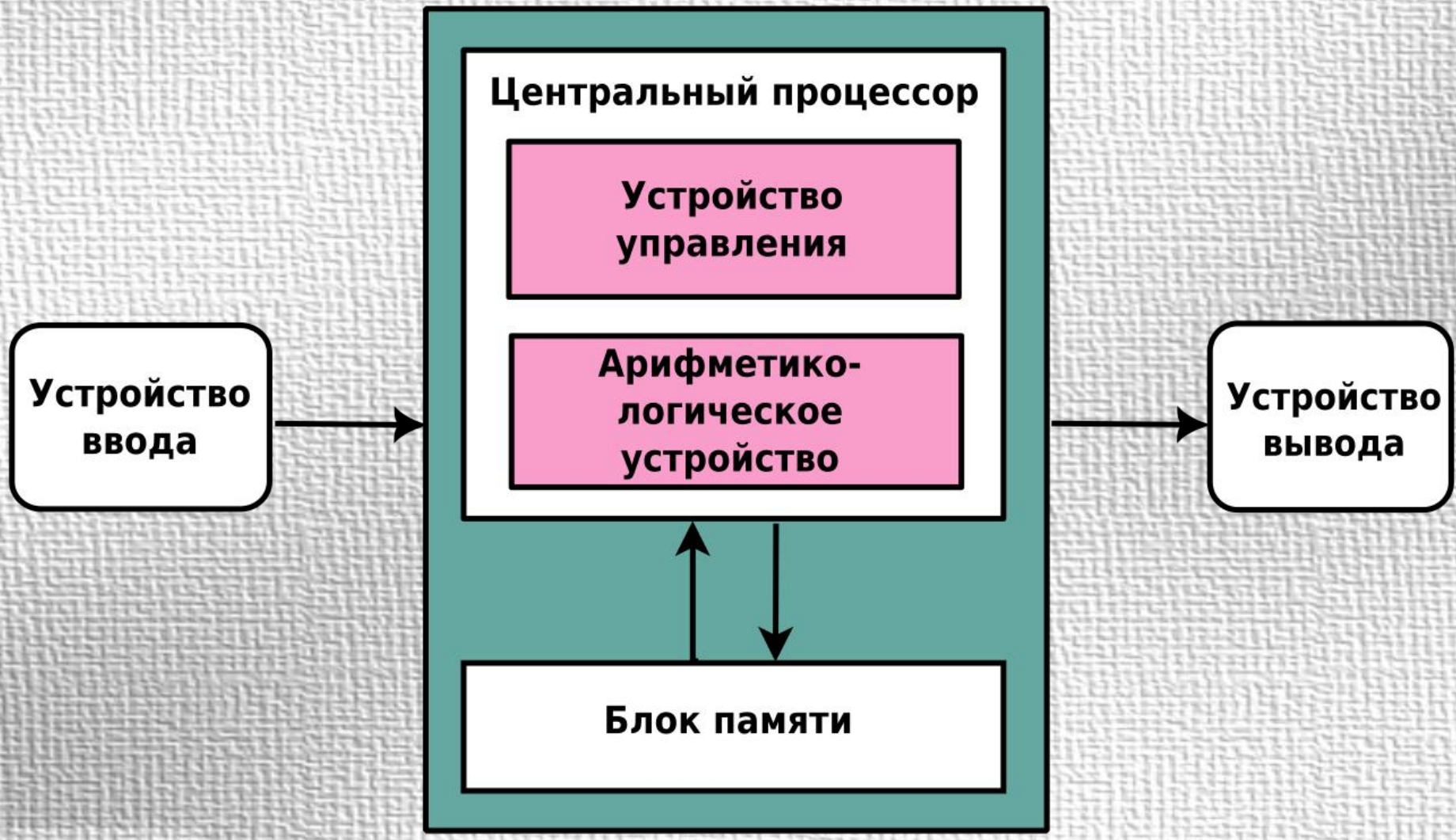


Схема Гарвардской архитектуры компьютера



Архитектура фон Неймана

Архитектура ПО vs Архитектура предприятия vs Системная архитектура

Архитектура предприятия описывает структуры бизнеса и его внутренние процессы (поток информации, взаимодействие между отделами и т.д.).

Системная архитектура описывает элементы и их взаимодействие между собой в рамках всей системы и включает как программные, так и аппаратные элементы. Системная архитектура описывает элементы с точки зрения их функциональности, но не с точки зрения их структуры.

И таким образом архитектура предприятия и системная архитектура создают среду, в которой живёт само ПО

Отличие архитектуры ПО от детального проектирования ПО

Архитектура ПО является реализацией нефункциональных требований к системе, в то время как детальное проектирование ПО является реализацией функциональных требований.

Архитектура ПО, которую также можно представить себе в виде разработки ***стратегии*** — это деятельность, связанная с определением глобальных ограничений, накладываемых на проектирование системы, такие как выбор парадигмы программирования, архитектурных стилей, стандарты разработки ПО, основанные на использовании компонентов, принципы проектирования и ограничения, накладываемые государственным законодательством.

Стандарты архитектуры

Первым стандартом программной архитектуры является стандарт IEEE 1471: ANSI / **IEEE 1471—2000**: Рекомендации по описанию преимущественно программных систем.

Он был принят в 2007 году, под названием **ISO / IEC 42010:2007**.

Следующая редакция: **ISO / IEC / IEEE 42010:2011** заменяет ISO / IEC 42010:2007 и IEEE Std 1471:2000.

В стандарте ANSI/IEEE 1471-2000 дается следующее определение архитектуры:

«**фундаментальная организация системы, реализованная в ее компонентах, связях этих компонентов друг с другом и внешней средой и принципах, определяющих структуру и развитие системы**».

ПРОЕКТНЫЕ МЕХАНИЗМЫ

Проектирование архитектуры системы выполняется **архитектором**. Определяются детали реализации архитектурных (проектных) механизмов, обозначенных в процессе анализа.

Например, уточняется способ хранения данных, вводится дублирование данных для повышения надежности системы и т. п. Поскольку **практически все механизмы – это некоторые типовые решения (образцы, шаблоны, каркасы), они документируются в проекте (модели) с помощью кооперaций со стереотипом <<mechanism>>,** при этом структурная часть каждого механизма описывается с помощью диаграмм классов, а поведение – с помощью диаграмм взаимодействия.

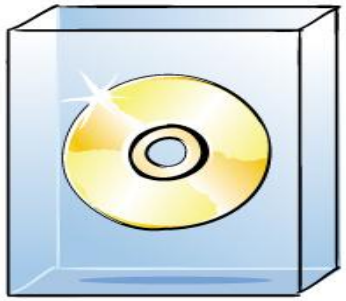
Замысел
(Идея)

Система, в которой приложение используется

Stakeholder
(Заинтересованное лицо)

Concern
(Интерес)

Система производства приложения



System
(Приложение)

Три уровня представления архитектуры

Методика описания архитектуры стандарта базируется на трех уровнях представления архитектуры:

1. **Viewpoint – точка зрения, аспект**
2. **View – вид, представление**
3. **Model - модель**

❶.Viewpoint

Viewpoint - точка зрения (аспект), относительно которой мы рассматриваем архитектуру или дизайн приложения. Разные ЗЛ заинтересованы в раскрытии различных аспектов архитектуры приложения. Эта заинтересованность выражается термином **concern**. Например, разработчикам прикладного слоя и администраторам базы данных важно знать и понимать архитектуру приложения с точки зрения хранения данных, то есть им интересен *persistent viewpoint*.

Точка зрения (аспект) (Viewpoint)

Ниже перечислены типичные аспекты проекта:

1. **Functional viewpoint**
2. **Domain viewpoint**
3. **Conceptual viewpoint**
4. **Persistence viewpoint**
5. **Logical viewpoint**
6. **Behavioral viewpoint**
7. **Implementation viewpoint**
8. **Deploymen viewpoint**

Вам необходимо составить таблицу, в которой нужно перечислить все заинтересованные лица, их интересы по отношению к архитектуре приложения и собственно те аспекты, которые им необходимо видеть и понимать. Ниже приведен пример такой таблицы, которую вы можете использовать за основу при подготовке к созданию архитектурного описания приложения.

Заинтересованное лицо	Интересы	Аспекты
Специалист поддержки	1. Каким образом, на каких серверах развернуты компоненты решения? 2. Какие есть инструменты для контроля работы компонентов?	Deployment viewpoint
Проектировщик БД	1. Какова предметная область приложения? 2. Какие компоненты решения требуют хранения в базе данных? 3. Какие есть требования к производительности и надежности подсистемы хранения данных?	Domain viewpoint, Persistent viewpoint, Deployment viewpoint

Viewpoint - пример из жизни

Вас могут оценить с разных сторон

- художник
- соседи
- врач
- полицейский
- преподаватель
- ...

❷.View

View - представление, некоторая часть архитектурного описания приложения, например, функциональная, логическая или посвященная безопасности, например *security view*. С некоторой точки зрения (**Viewpoint**) мы разрабатываем несколько представлений, например, с точки зрения развертывания решения нам интересны представления *deployment view* и *dependency view*. На первом отражено развертывание компонентов решения и связей между ними, а на втором - зависимых компонентов или способов связи с ними (в UML это диаграммы развертывания и компонентов, как **модели** представления) .

Виды (представления) (views)

Архитектура ПО обычно содержит несколько видов, которые аналогичны различным типам чертежей в строительстве зданий. В онтологии, установленной ANSI / IEEE 1471—2000, ***виды являются экземплярами точки зрения***, где точка зрения существует для описания архитектуры с точки зрения заданного множества заинтересованных лиц.

Другими словами - точка зрения на архитектуру группы ЗЛ отражается в соответствующем **представлении** или **виде**.

Архитектурные виды можно поделить на 3 основных типа:

1. **Модульные виды** (англ. module views) — показывают систему как структуру из различных программных блоков.
2. **Компоненты-и-коннекторы** (англ. component-and-connector views) — показывают систему как структуру из параллельно запущенных элементов (компонентов) и способов их взаимодействия (коннекторов).
3. **Размещение** (англ. allocation views) — показывает размещение элементов системы во внешних средах.

Примеры модульных видов:

- **Декомпозиция** (англ. decomposition view) — состоит из модулей в контексте отношения «является подмодулем»
- **Использование** (англ. uses view) — состоит из модулей в контексте отношения «использует» (т.е. один модуль использует сервисы другого модуля)
- **Вид уровней** (англ. layered view) — показывает структуру, в которой связанные по функциональности модули объединены в группы (уровни)
- **Вид классов/обобщений** (англ. class/generalization view) — состоит из классов, связанные через отношения «наследуется от» и «является экземпляром»

Примеры видов компонентов и коннекторов:

- **Процессный вид (англ. process view)** — состоит из процессов, соединённых операциями коммуникации, синхронизации и/или исключения
- **Параллельный вид (англ. concurrency view)** — состоит из компонентов и коннекторов, где коннекторы представляют собой «логические потоки»
- **Вид обмена данными (англ. shared-data (repository) view)** — состоит из компонентов и коннекторов, которые создают, сохраняют и получают постоянные данные
- **Вид клиент-сервер (англ. client-server view)** — состоит из взаимодействующих клиентов и серверов, а также коннекторов между ними (например, протоколов и общих сообщений)

Примеры видов размещения:

- **Развертывание (англ. deployment view)** — состоит из программных элементов, их размещения на физических носителях и коммуникационных элементов
- **Внедрение (англ. implementation view)** — состоит из программных элементов и их соответствия файловым структурам в различных средах (разработческой, интеграционной и т.д.)
- **Распределение работы (англ. work assignment view)** — состоит из модулей и описания того, кто ответственен за внедрение каждого из них

View - пример из жизни

Как описывают вас

- художник - **портрет**
- соседи - **донос**
- врач - **анамнез**
- полицейский - **протокол**
- преподаватель - **ведомость**
- ...

❸.Model

Model - модель, содержащая конкретные компоненты и отражающая связи между ними. В качестве моделей можно использовать диаграмму классов для отображения представления предметной области или логического представления архитектуры приложения.

Подведем итоги…

В стандарте определяются так называемые **stakeholders**, то есть заинтересованные лица, которым интересны те или иные аспекты архитектуры.

Каждый проект можно рассматривать с разных точек зрения (с разных сторон, аспектов). Это и называется **Viewpoint**. Интерес **(concern)** рассматривать проект с этой стороны исходит от заинтересованных сторон**.**

Для объяснения данной точки зрения **Viewpoint** может быть использовано несколько представлений - **View.**

КЛЮЧЕВЫЕ ПОНЯТИЯ АРХИТЕКТУРЫ ПО

КЛЮЧЕВЫЕ ПОНЯТИЯ АРХИТЕКТУРЫ ПО

1. Декомпозиция
2. Компоненты
3. Каркасы (Фреймворки)
4. Шаблоны
5. Многослойное представление
6. Уровни

ДЕКОМПОЗИЦИЯ

Разработка архитектуры системы — это процесс разбиения большой системы на более мелкие части. Для обозначения этих частей придумано множество названий: *программы, компоненты, подсистемы, модули, …*

КОМПОНЕНТЫ

Традиционно программа проектировалась из отдельных файлов, модулей или классов, которые компилировались и компоновались в единое целое.

Разработка программ из компонентов так называемых приложений **компонентной архитектуры** происходит совершенно иначе.

Программы состоят из отдельных компонент.

Компонента поставляется пользователю как двоичный код, скомпилированный, скомпонованный и готовый к использованию.

Доступ к этому коду осуществляется через точно документированный интерфейс. Во время выполнения компоненты подключаются к другим компонентам, формируя программу.

КАРКАСЫ (ФРЕЙМВОРКИ)

В самом простом смысле каркас может означать любой фрагмент разработанного и проверенного ПО, который повторно используется в нескольких проектах разработки ПО.

В более формальном смысле фреймворк представляет собой обобщенный архитектурный шаблон, который может использоваться для построения приложений в определенной предметной области.

Фреймворк -- это набор всевозможных библиотек (инструментов) для быстрой разработки повседневных (рутинных) задач. Чаще всего использует одну из распространенных архитектур приложения (к примеру MVC) для разделения проекта на логические сегменты (модули).

Главная цель фреймворка -- предоставить программисту удобную среду для проекта с большим и хорошо расширяемым функционалом.

ШАБЛОНЫ

Шаблон проектирования или паттерн в разработке программного обеспечения — повторяемая архитектурная конструкция, представляющая собой решение проблемы проектирования в рамках некоторого часто возникающего контекста.

Обычно шаблон не является законченным образцом, который может быть прямо преобразован в код; это лишь пример решения задачи, который можно использовать в различных ситуациях. Объектно-ориентированные шаблоны показывают отношения и взаимодействия между классами или объектами, без определения того, какие конечные классы или объекты приложения будут использоваться.

Примеры архитектурных шаблонов:

Многоуровневый шаблон (Layered pattern). Система разбивается на уровни, которые на диаграмме изображаются один над другим. Каждый уровень может вызывать только уровень на 1 ниже него. Таким образом разработку каждого уровня можно вести относительно независимо, что повышает модифицируемость системы.

Клиент-серверный шаблон (Client-Server pattern). Если есть ограниченное число ресурсов, к которым требуется ограниченный правами доступ большого числа потребителей, то удобно реализовать клиент-серверную архитектуру. Такой подход повышает масштабируемость и доступность системы. Но при этом сервер может стать узким местом системы, при его недоступности становится недоступна вся система.

Шаблон посредника (Broker pattern). Когда в системе присутствует большое количество модулей, их прямое взаимодействие друг с другом становится слишком сложным. Для решения проблемы вводится посредник (например, шина данных, по которой модули общаются друг с другом),.

Шаблон «Модель-Представление-Контроллер» (Model-View-Controller pattern). Т.к. требования к интерфейсу меняются чаще всего, то возникает потребность часто его модифицировать, при этом сохраняя корректное взаимодействие с данными (чтение, сохранение). Для этого в шаблоне Model-View-Controller (MVC) интерфейс отделён от данных. Это позволяет менять интерфейсы, равно как и создавать их разные варианты.

УРОВНИ И СЛОИ
(МНОГОСЛОЙНОЙ ПРЕДСТАВЛЕНИЕ)

Понятия ***слой*** (layer) и ***уровень*** (tier) очень близки.

Слой используется для логического разделения, а уровень для физического разворачивания. Но когда в контексте не важно или абсолютно понятно, какое именно разделение имеется в виду, используется слово уровень, хотя в английском языке наоборот.

Поэтому системы клиент/сервер обычно описывают как двухуровневые (в общем случае “клиент” действительно отделен от сервера физически): клиент — это приложение для настольной машины, а сервер — процесс, выполняемый сетевым компьютером сервером. Мы применяем термин слой, чтобы подчеркнуть, что слои вовсе не обязательно должны располагаться на разных машинах. Отдельный слой бизнес-логики может функционировать как на персональном компьютере “рядом” с клиентским слоем интерфейса, так и на сервере базы данных. В подобных ситуациях речь идет о двух узлах сети, но о трех слоях или уровнях. Если база данных локальна, все три слоя могут соседствовать и на одном компьютере. но даже в этом случае они должны сохранять свой суверенитет.

Горячие точки архитектуры

- *Ключевые вопросы*
- *Ключевые проблемы*

Архитектура системы обычно включает:

- Общее описание системы;
- Основные компоненты;
- Формат и способ хранения данных;
- Специфические бизнес-правила;
- Способ организации пользовательского интерфейса;
- Подход к безопасности системы;
- Оценки производительности;
- Возможности масштабирования;
- Моменты, связанные с интернациональностью, т.е. будет ли система интернациональной.

КАТЕГОРИИ

1. Аутентификация и авторизация (Authentication and Authorization)
2. Кэширование и состояние (Caching and State)
3. Взаимодействие (Communication)
4. Композиция (Composition)
5. Параллельные вычисления и транзакции (Concurrency and Transactions)
6. Управление конфигурацией (Configuration Management)
7. Связанность и сцепление (Coupling and Cohesion)

8. Доступ к данным (Data Access)
9. Работа с исключениями (Exception Management)
10. Ведение логов и мониторинг (Logging and Instrumentation)
11. Взаимодействие с пользователем (User Experience)
12. Проверка данных (Validation)
13. Поток операций (Workflow)

КЛЮЧЕВЫЕ ВОПРОСЫ

1. Аутентификация и авторизация

- Как хранить идентификационные данные зарегистрированных пользователей?
- Как аутентифицировать запросы?
- Как авторизовывать запросы?
- Как передавать идентификационные данные пользователей между слоями приложения?

2. Кэширование и состояние

- Как определить правила удаления данных из кэша из-за устаревания или переполнения?
- Как загружать данные кэша?
- Как следить за кэшем?
- Как синхронизовать между собой множественные кэши?
- Как определить какой способ кэширования даёт наибольшую производительность и масштабируемость для определённого сценария и конфигурации.
- Как определить какая технология кэшировния удовлетворяет требованиям приложения насчёт безопасности, лёгкости управления и мониторинга.

- Как выбрать эффективную стратегию кэширования?
- Как увеличить производительность с помощью кэширования?
- Как улучшить безопасность с помощью кэширования?
- Как улучшить стабильность с помощью кэширования?
- Как поддерживать кэш в актуальном состоянии?
- Как определить когда и каким образом использовать специализированный кэш?
- Как определить какие данные кэшировать?
- Как определить куда кэшировать данные?

3. Взаимодействие (Communication)

- Как передавать информацию между слоями приложения?
- Как совершать асинхронные операции?
- Как передавать секретные данные?

4. Композиция
(Composition)

- Как спроектировать структуру приложения?
- Как спроектировать слабую связанность между модулями?
- Как работать с зависимостями в слабосвязанном режиме?

6. Управление конфигурацией (Configuration Management)

- Как определить какая информация подлежит конфигурированию?
- Как определить где и каким образом сохранять конфигурационную информацию?
- Как работать с секретной информацией?
- Как работать с конфигурационной информацией в кластере?

7. Связанность и сцепление (Coupling and Cohesion)

- Как разделить функциональности?
- Как структурировать приложение?
- Как выбрать подходящее разбиение на слои?
- Как задать границы между частями системы?

8. Доступ к данным (Data Access)

- Как управлять соединениями БД?
- Как управлять исключениями?
- Как улучшить производительность?
- Как улучшить управляемость?
- Как работать с бинарными данными (BLOBs)?
- Как управлять транзакциями?

13. Поток операций

- Сильная связанность.
- Негибкие процессы.
- Проблемы с приоритетами и взаимными блокировками (race and deadlock issues).

КЛЮЧЕВЫЕ ПРОБЛЕМЫ

1. Аутентификация и авторизация

- Хранение идентификационных данных в открытом виде в файлах.
- Передача идентификационных данных в открытом виде по сети.
- Аккаунты с излишними привилегиями.
- Долгоживущие сессии.
- Смешивание персонализации и аутентификации.

- Использование единственного контроллера доступа (gatekeeper).
- Отсутствие ограничений на доступ к системным ресурсам для приложения.
- Отсутствие ограничений на доступ к БД кроме определённых хранимых процедур (stored procedures).
- Неадекватное разделение прав доступа.

КЛЮЧЕВЫЕ РЕКОМЕНДАЦИИ

1. Аутентификация и авторизация

- Подумайте над требованиям к регистрации.
- Разделите открытые всем и закрытые зоны.
- Используйте политики блокировки аккаунтов конечных пользователей (например при множественных ошибках в пароле).
- Поддерживайте настраиваемый срок действия пароля.
- Обеспечьте возможность отключения (disable) аккаунтов администратором.

- Требуйте «сильных» паролей.
- Не пересылайте паролей в открытом виде по сети.
- Защищайте cookies с идентификационными данными.
- Используйте несколько контроллеров доступа (gatekeepers).
- Ограничьте доступ пользователей к системным ресурсам.
- Подумайте над степенью детализации прав доступа.

Архитектура должна

- Раскрывать структуру системы, но скрывать детали реализации.
- Реализовывать все варианты использования и сценарии.
- По возможности отвечать всем требованиям различных заинтересованных сторон.
- Выполнять требования, как по функциональности, так и по качеству.

ОСНОВНЫЕ ВОПРОСЫ ПРОЕКТИРОВАНИЯ

- Определение типа приложения
- Выбор стратегии развертывания
- Выбор соответствующих технологий
- Выбор показателей качества
- Решение о путях реализации сквозной функциональности

Контрольный список: архитектура

1. Ясно ли описана общая организация программы?
2. Включает ли спецификация грамотный обзор архитектуры и ее обоснование?
3. Адекватно ли определены основные компоненты программы, их области ответственности и взаимодействие с другими компонентами?
4. Все ли функции, указанные в спецификации требований, реализуются разумным, не слишком большим и не слишком малым, числом компонентов?

5. Приведено ли описание самых важных классов и их обоснование?
6. Приведено ли описание организации данных и ее обоснование?
7. Приведено ли описание организации и содержания БД?
8. Определены ли все важные бизнес-правила? Описано ли их влияние на систему?
9. Описана ли стратегия проектирования GUI?
10. Сделан ли GUI модульным, чтобы его изменения не влияли на оставшуюся часть программы?
11. Приведено ли описание стратегии ввода-вывода данных и ее обоснование?

12. Указаны ли оценки степени использования ограниченных ресурсов, таких как потоки, соединения с БД, дескрипторы, пропускная способность сети?
13. Приведено ли описание стратегии управления такими ресурсами и ее обоснование?
14. Описаны ли требования к защищенности архитектуры?
15. Определяет ли архитектура требования к объему и быстродействию всех классов, подсистем и функциональных областей?
16. Описывает ли архитектура способ достижения масштабируемости системы?

17. Рассмотрены ли вопросы взаимодействия системы с другими системами?
18. Описана ли стратегия интернационализации/локализации?
19. Определена ли согласованная стратегия обработки ошибок?
20. Определен ли подход к отказоустойчивости системы (если это требуется)?
21. Подтверждена ли возможность технической реализации всех частей системы?
22. Определен ли подход к реализации избыточной функциональности?

23. Приняты ли необходимые решения относительно «покупки или создания» компонентов системы?
24. Описано ли в спецификации, как повторно используемый код будет адаптирован к другим аспектам архитектуры?
25. Сможет ли архитектура адаптироваться к вероятным изменениям?

Проблемы, связанные с проектированием ПО
(по Макконнелу)

1. Проектирование — «грязная» проблема
2. Проектирование — неряшливый процесс
3. Проектирование связано с определением компромиссов и приоритетов
4. Проектирование подразумевает ограничение возможностей
5. Проектирование — недетерминированный процесс
6. Проектирование — эвристический процесс
7. Проектирование — постепенный процесс

Проектирование — «грязная» проблема

Хорст Риттел и Мелвин Веббер определили «грязную» проблему как проблему, которую можно ясно определить только путем полного или частичного решения (Rittel and Webber, 1973). По сути данный парадокс подразумевает, что проблему нужно «решить» один раз, чтобы получить ее ясное определение, а затем еще раз для создания работоспособного решения.

Проектирование — эвристический процесс

Так как проектирование не детерминировано, методы проектирования чаще всего являются эвристическими методами, т. е. «практическими правилами» или «способами, которые могут сработать», а не воспроизводимыми процессами, которые всегда приводят к предсказуемым результатам. Проектирование — метод проб и ошибок. Инструменты или методы проектирования, оказавшиеся эффективными в одном случае, в другой ситуации могут оказаться куда менее эффективными. Универсальных методик проектирования не существует.

Образ разработчика, проектирующего программу рациональным безошибочным способом на основе ясно сформулированных требований, совершенно нереалистичен. Никакая система так никогда не разрабатывалась и, наверное, не будет разрабатываться. Даже примеры разработки небольших программ, встречающиеся в учебниках, нереалистичны. Авторы перепроверяют и улучшают их до тех пор, пока не продемонстрируют нам то, что они хотели бы получить, а не то, что получается на самом деле.

Дэвид Парнас и Пол Клементс (David Parnas and Paul Clements)

Желательные характеристики проекта

- Минимальная сложность
- Простота сопровождения
- Слабое сопряжение
- Расширяемость
- Возможность повторного использования
- Высокий коэффициент объединения по входу
- Низкий или средний коэффициент разветвления по выходу
- Портируемость
- Минимальная, но полная функциональность
- Стратификация
- Соответствие стандартным методикам

ЭЛЕМЕНТЫ АРХИТЕКТУРНЫХ РЕШЕНИЙ

(АРХИТЕКТУРНЫЕ МЕХАНИЗМЫ)

- Кэширование
- Масштабирование
- Брокеры сообщений
- Микросервисы
- Мониторинг, сбор логов и метрик

БРОКЕРЫ СООБЩЕНИЙ

- Apache Kafka
- RabbitMQ
- Redis

В первую очередь брокер сообщений упрощает обмен информацией между системами и приложениями, даже если они работают на разных языках и платформах. При небольшом количестве участников обмениваться данными можно напрямую, однако с увеличением количества участников возникает потребность в большей интерактивности, и этот способ становится неэффективным. В таких случаях лучше использовать брокер сообщений, который управляет процессом обмена информацией и является посредником между отправителем и получателем.

Особенно он будет полезен для асинхронного взаимодействия микросервисов. Асинхронная связь не требует ожидания ответа в режиме реального времени. В качестве примера можно привести электронную почту, где пользователи могут отправить письмо и продолжить выполнять другие задачи.

Брокер сообщений (англ. message broker, integration broker, interface engine) — архитектурный шаблон в распределённых системах; приложение, которое преобразует сообщение по одному протоколу от приложения-источника в сообщение протокола приложения-приёмника, тем самым выступая между ними посредником. Кроме преобразования сообщений из одного формата в другой, в задачи брокера сообщений также входит:

- проверка сообщения на ошибки;
- маршрутизация конкретному приемнику(ам);
- разбиение сообщения на несколько маленьких, а затем агрегирование ответов приёмников и отправка результата источнику;
- сохранение сообщений в базе данных;

- вызов веб-сервисов;
- распространение сообщений подписчикам, если используются шаблоны типа «издатель — подписчик».

Использование брокеров сообщений позволяет разгрузить веб-сервисы в распределённой системе, так как при отправке сообщений им не нужно тратить время на некоторые ресурсоёмкие операции типа маршрутизации и поиска приёмников. Кроме того, брокер сообщений для повышения эффективности может реализовывать стратегии упорядоченной рассылки и определение приоритетности, балансировать нагрузку и прочее..

Брокер сообщений представляет собой тип построения архитектуры, при котором элементы системы «общаются» друг с другом с помощью посредника. Благодаря его работе происходит снятие нагрузки с веб-сервисов, так как им не приходится заниматься пересылкой сообщений: всю сопутствующую этому процессу работу он берёт на себя.

Можно сказать, что в работе любого брокера сообщений используются две основные сущности: **producer** (**издатель** сообщений) и **consumer** (потребитель/**подписчик**).

Одна сущность занимается созданием сообщений и отправкой их другой сущности-потребителю. В процессе отправки есть ещё серединная точка, которая представляет собой папку файловой системы, где хранятся сообщения, полученные от продюсера.

Основные компоненты брокера сообщений

Producer (издатель) отвечает за отправку сообщений в брокер, который затем доставляет их подписчикам.

Consumer (подписчик) получает и обрабатывает сообщения из брокера сообщений.

Queue (очередь) представляет собой модель отправки и получения сообщений, где отправители помещают сообщения в очередь, а получатели извлекают их из очереди. Каждое сообщение обрабатывается только одним получателем.

Topic (тема) — это модель отправки и получения сообщений, где producer отправляет сообщения на конкретную тему, а получатели, подписанные на эту тему, получают эти сообщения.

Здесь возможны несколько вариантов:

- *Сообщение отправляется напрямую от отправителя к получателю.*

В этом случае каждое сообщение используется только однократно;

- *Схема публикации/подписки.*

В рамках этой схемы обмена сообщениями отправитель не знает своих получателей и просто публикует сообщения в определённую тему. Потребители, которые подписаны на эту тему, получают сообщение. Далее на базе этой системы может быть построена работа с распределением задач между подписчиками. То есть выстроена логика работы, когда в одну и ту же тему публикуются сообщения для разных потребителей. Каждый «видит» уникальный маркер своего сообщения и забирает его для исполнения.

Модели обмена сообщениями при помощи брокера сообщений

Реализовать обмен сообщениями можно несколькими способами. Мы рассмотрим три основных.

1. **Обмен сообщениями «Point-to-Point»**
2. **Опубликовать/подписаться**
3. **Гибридная модель обмена сообщениями**

Обмен сообщениями «Point-to-Point»

В рамках модели P2P (точка-точка) каждое сообщение отправляется в определенную очередь и может быть обработано только одним получателем. При этом брокер сообщений гарантирует, что недоставленные сообщения будут сохранены в очереди до тех пор, пока они не будут обработаны получателем.

Преимущество модели P2P заключается в том, что каждая транзакция обрабатывается всего один раз. Один из примеров использования P2P для обмена сообщениями — это обработка транзакций, где надежность и безопасность являются ключевыми факторами.

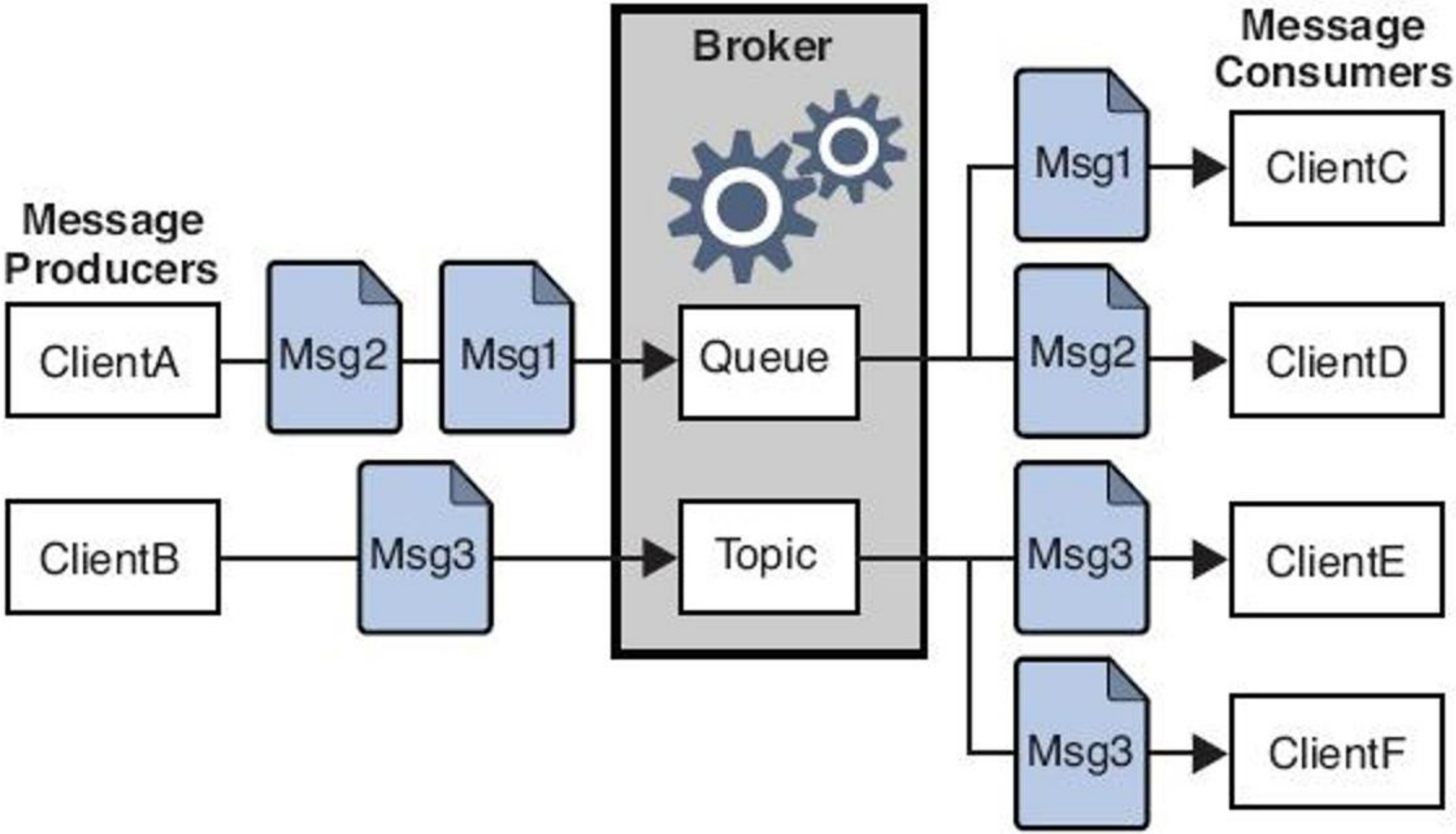
БРОКЕР СООБЩЕНИЙ
Очередь
Очередь
Продюсер
Потребитель

Схема публикации/подписки

При использовании шаблона pub/sub (широковещательная рассылка) сообщения распространяются между всеми пользователями, подписанными на общую тему. Это полезно для реализации механизмов уведомлений или распределения независимых задач. Причем могут поддерживаться различные типы сообщений, включая строки, хэши, списки и множества, что позволяет использовать его для передачи различных типов данных.

Самые простые примеры, где реализуется данная модель: телеграм-каналы и онлайн-чаты. Кроме того, брокер можно использовать для передачи информации о событиях между разными инстансами приложений. Этот механизм может помочь отслеживать изменения в базе данных и активность пользователей.

БРОКЕР СООБЩЕНИЙ
Тема
Тема
Издатель
Подписчик
Подписчик


Broker
Message Consumers
Message Producers
ClientA
Msg2
Msg1
Queue
ClientB
Msg3
Topic
Msg1
ClientC
Msg2
ClientD
Msg3
ClientE
Msg3
ClientF

Для чего нужны брокеры сообщений?

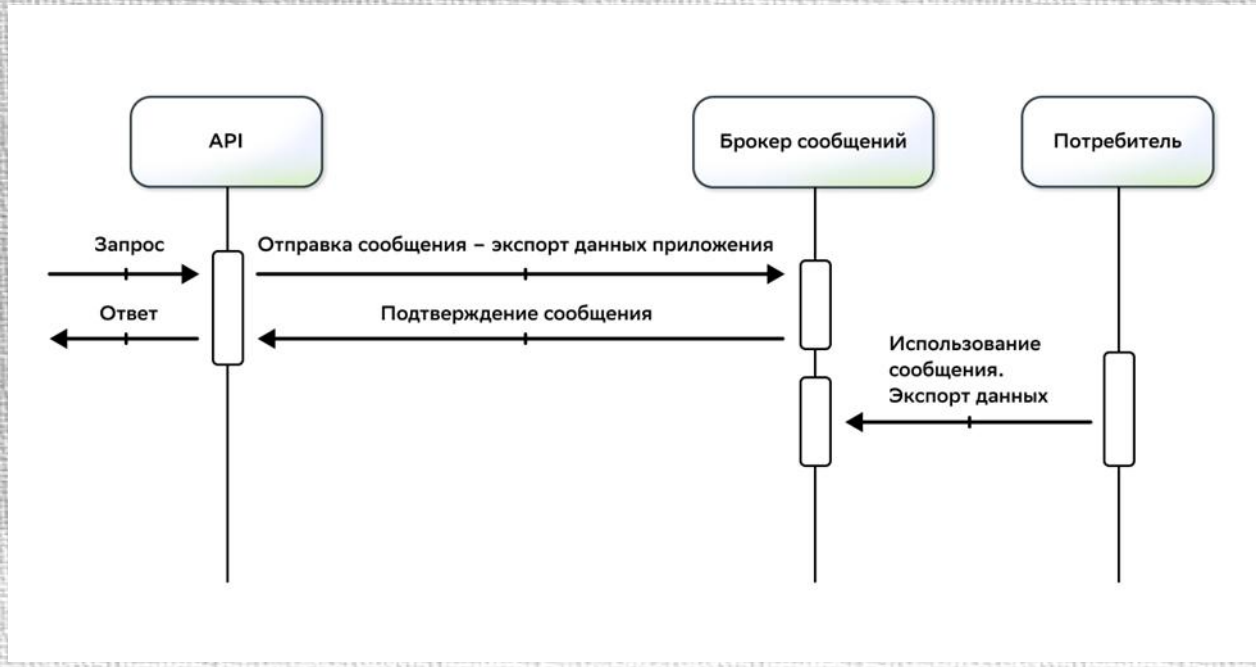
- Для организации связи между отдельными службами, даже если какая-то из них не работает в данный момент. То есть продюсер может отправлять сообщения, несмотря на то, проявляет ли активность потребитель в настоящее время.
- За счёт асинхронной обработки задач можно увеличить производительность системы в целом.
- Для обеспечения надёжности доставки сообщений: как правило, брокеры обеспечивают механизмы многократной отправки сообщений в тот же момент или через определённое время. Кроме того, обеспечивается соответствующая маршрутизация сообщений, которые не были доставлены.

Недостатки брокеров сообщений

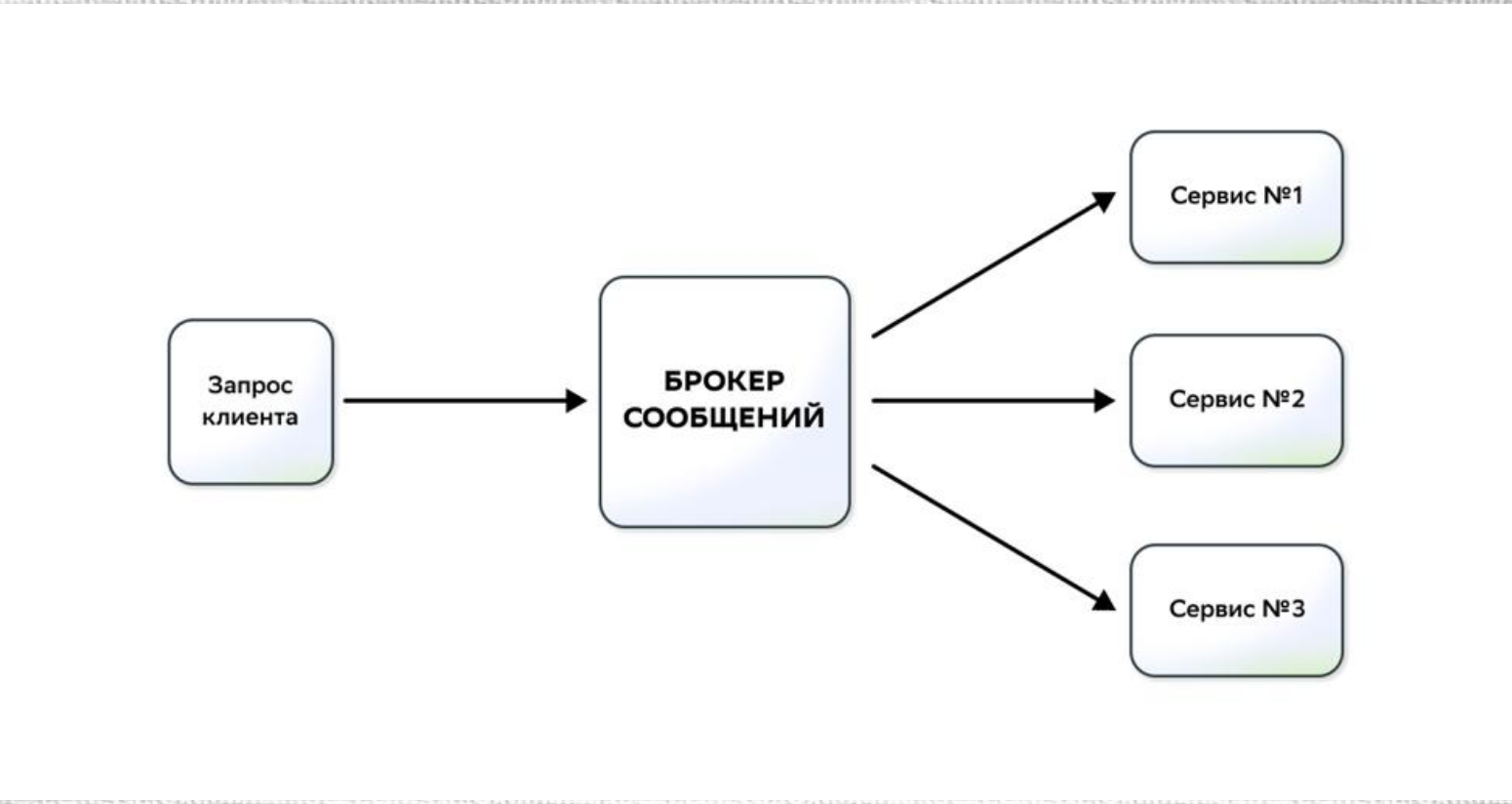
- Усложнение системы в целом как таковой, так как в ней появляется ещё один элемент. Кроме того, возникает зависимость от надёжности распределённой сети, а также потенциальная возможность возникновения проблем из-за потребности в непротиворечивости данных, так как некоторые элементы системы могут обладать неактуальными данными.
- Из-за асинхронной работы всей системы, а также её распределённого характера могут возникать ошибки, выяснение сути которых может стать непростой задачей.
- Освоение подобных систем является не самым простым вопросом и может занять существенное время.

Когда брокеры сообщений могут быть полезны

- Если в рамках вашей системы есть действия, которые требуют для своего выполнения много времени и потребляют много ресурсов, при этом они не требуют немедленного результата.



- **Микросервисы:** если ваша система достаточно сложна и состоит из отдельных сервисов, то для их координации можно использовать брокер сообщений, который в этом случае будет выступать в роли как бы центрального роутера. Каждый сервис подписывается только на свой тип сообщений, выстраивается определённая логика их обработки.



- **Мобильные приложения**: здесь возможен вариант с задействованием push-уведомлений, когда множество смартфонов с установленным приложением подписаны на определённую тему. Если в ней публикуется какая-либо новость, то подписанный смартфон выводит уведомление.
- **Транзакционные системы**: если какое-то действие системы состоит из множества отдельных этапов, каждый из которых выполняется отдельным элементом системы, то в этом случае брокер сообщений может выступить в роли своеобразной «доски уведомлений». Каждый сервис отписывается после того, как его этап общей задачи был выполнен. После этого в работу вступает следующий сервис, обрабатывающий, соответственно, следующий этап общей задачи.

Подтверждение видео
Сжатие видео
Генерация иконки
Публикация видео
Загрузка запроса на видео
БРОКЕР СООБЩЕНИЙ

❶ RabbitMQ

Какие есть брокеры сообщений?

Брокеров сообщений существует великое множество.

Каждый из них обладает определёнными «фишками» и хорошо решает возложенные на него задачи. Например, если одни предназначены для создания инфраструктуры связи между распределёнными частями приложения, другие предназначены для достаточно специфических задач, например «интернета вещей», и функционируют на основе легковесного протокола MQTT. Подобные брокеры служат для сбора статистики, температуры и других показателей с распределённых датчиков, установленных на определённых машинах, элементах конструкций, географически разных точках территории.

Apache Kafka и RabbitMQ: что выбрать?

В корпоративных инфраструктурных системах популярностью пользуются Apache Kafka и RabbitMQ. В связи с чем часто возникает вопрос, какую из них выбрать в конкретном случае?

Говоря о RabbitMQ, можно сказать, что он представляет собой классический брокер, в котором присутствуют две описанные выше сущности – продюсер (система, генерирующая сообщения о разнообразных событиях) и подписчик, являющийся получателем этих сообщений.

Обе эти сущности в процессе работы взаимодействуют с очередью сообщений, которая представляет собой хранилище, где накапливаются отправляемые сообщения.

В рамках этого брокера инициатором информационного обмена является продюсер, только он отправляет сообщение в сеть, в то время как подписчик не может запросить его сам (так называемая «push-доставка сообщений»).

Apache Kafka и RabbitMQ это распределённые программные брокеры сообщений. Их использование лежит в основе событийно ориентированной (event driven) архитектуры взаимодействия сервисов.

Такие системы обычно состоят из трёх базовых компонентов:

1. **сервер** (брокер), обрабатывающий сообщения;
2. **продюсеры**, которые отправляют сообщения в некую именованную очередь, заранее сконфигурированную администратором на сервере;
3. **консюмеры**, которые считывают те же самые сообщения по мере их появления.

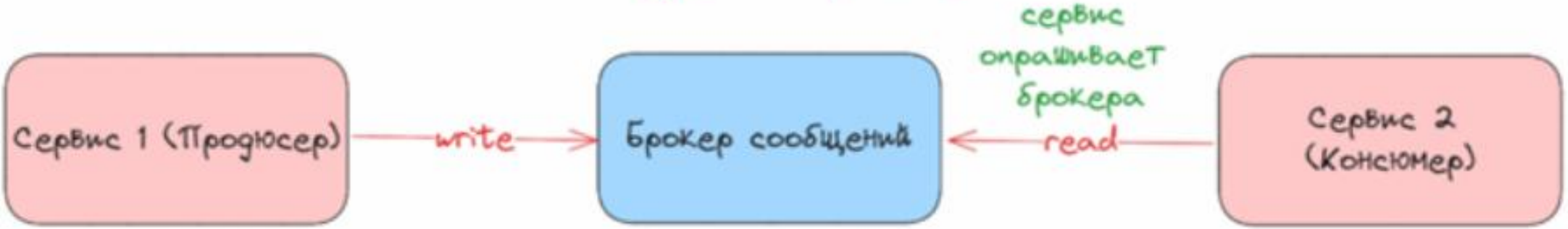
Продюсеры ничего не знают о консюмерах при записи данных в брокер, а консюмеры ничего не знают о продюсерах данных.

Существует две модели работы брокеров

- **pull модель** — консюмеры сами отправляют запрос на сервер для получения новой порции сообщений. При таком подходе консюмеры могут эффективно контролировать собственную нагрузку. Кроме того, pull модель позволяет группировать сообщения в батчи, таким образом достигая лучшей пропускной способности. По такой модели, например, работает Kafka.
- **push модель** — продюсеры самостоятельно посылает потребителю новую порцию данных. По такой модели, например, работает RabbitMQ. Она снижает задержку обработки сообщений и позволяет балансировать распределение сообщений по консюмерам.

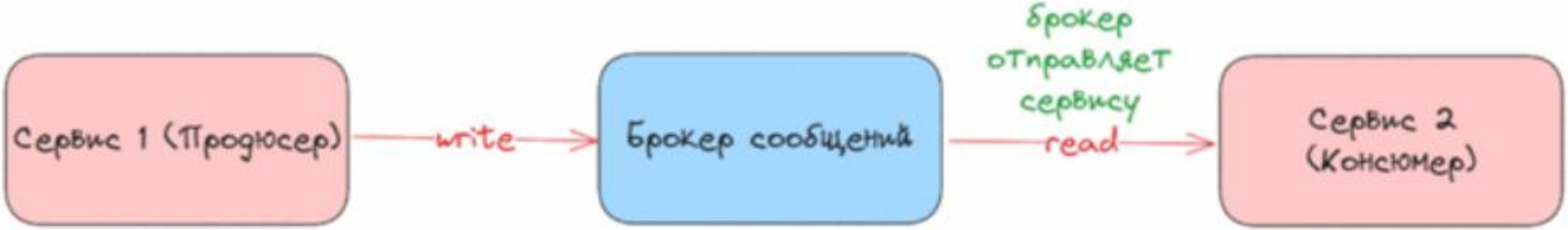
Брокеры сообщений:

PULL-модель (Apache Kafka)



tg: @sysdes_journal

PUSH-модель (RabbitMQ)



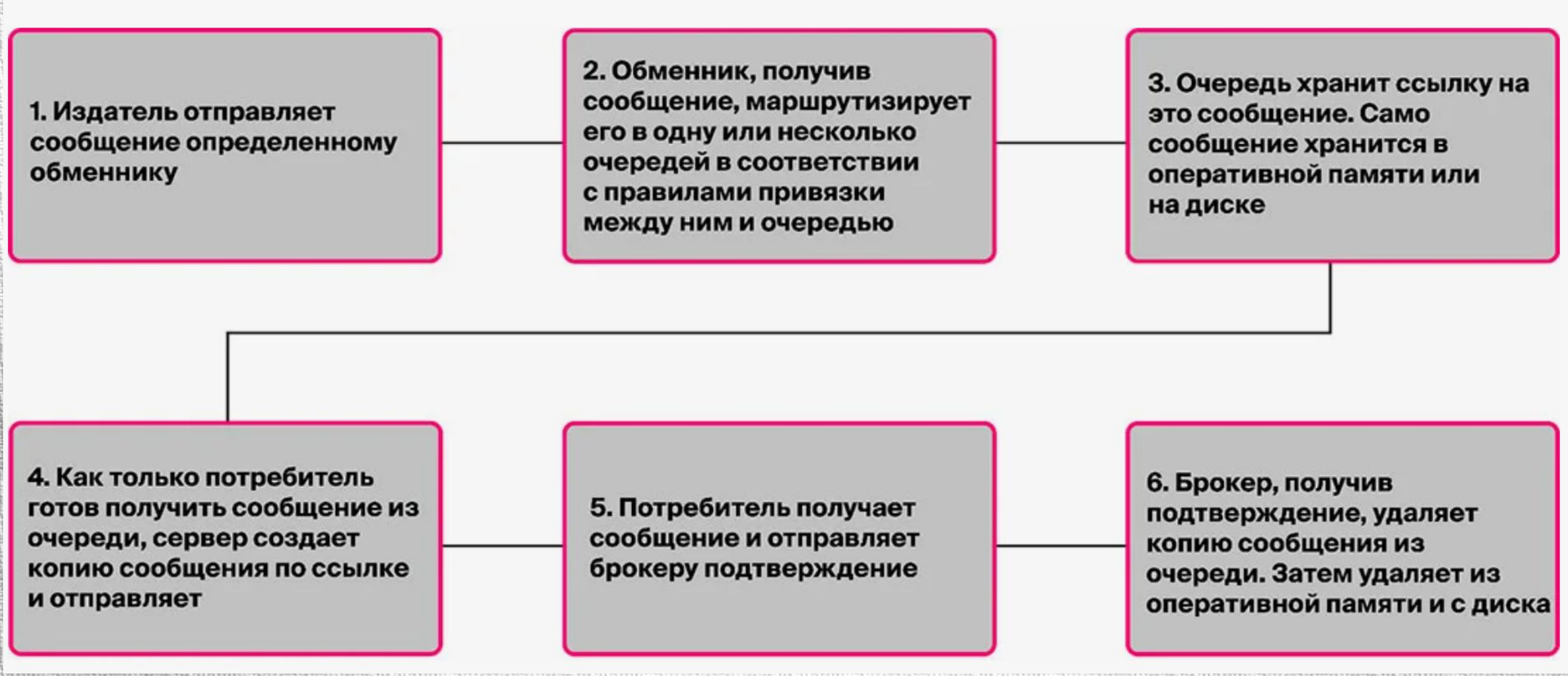
Модели доставки сообщений

- **At least once** (хотя бы один раз — допускаются дубликаты сообщений): продюсер отправляет сообщение в очередь и ждет подтверждения от брокера. Консюмер извлекает сообщение из очереди и отправляет подтверждение о его обработке. Если продюсер не получает подтверждения от брокера, он повторно отправляет сообщение. Сообщение будет обработано хотя бы один раз, но при этом возможны ситуации, когда одно и то же сообщение будет обработано несколько раз. Для предотвращения дублирования сообщений необходимо, чтобы компоненты были идемпотентными, то есть повторная обработка одного и того же сообщения не приводила бы к изменению состояния системы или ее данных. Также можно использовать уникальные идентификаторы для сообщений и хранить список уже обработанных сообщений.

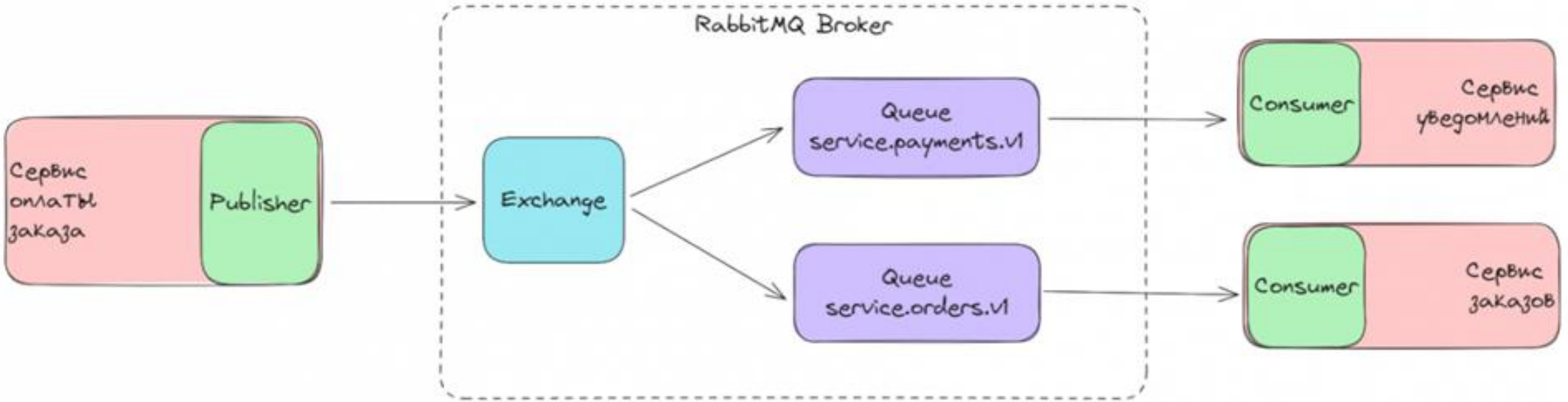
- **At most once** (не более чем один раз, но может не быть доставлено вообще — допускаются потери сообщений): продюсер отправляет сообщение в очередь и не ждёт подтверждения от брокера. Консюмер извлекает сообщение из очереди и не отправляет подтверждение о его обработке. Исключается возможность дублирования сообщений, но при этом сообщение может быть потеряно.
- **Exactly once** (строго только один раз — строго одно сообщение): брокер гарантирует, что каждое сообщение будет доставлено и обработано ровно один раз. Продюсер отправляет сообщение в очередь и ждет подтверждения от брокера о его приеме. Консюмер извлекает сообщение из очереди и отправляет подтверждение об обработке.

Producer
Producer
Producer
Брокер сообщений
RabbitMQ
Consumer
Consumer
Consumer

Как работает брокер RabbitMQ



RabbitMQ



Формат сообщения

- **Headers** — Заголовки сообщения. Нужны для работы Exchange типа headers, а также для указания атрибутов использования дополнительных возможностей RabbitMQ
- **Routing Key** — Ключ маршрутизации, может быть только один для одного сообщения
- **Delivery Mode** — Признак персистентности сообщения (как хранится на диске)
- **Payload** — Тело сообщения. Обычно используются сериализуемые форматы типа JSON, Protobuf итд

Дополнительные параметры сообщения

Content type - для сериализации Payload

Content encoding - для сериализации Payload

Message priority - приоритет сообщения

Expiration period - период после которого сообщение удаляется

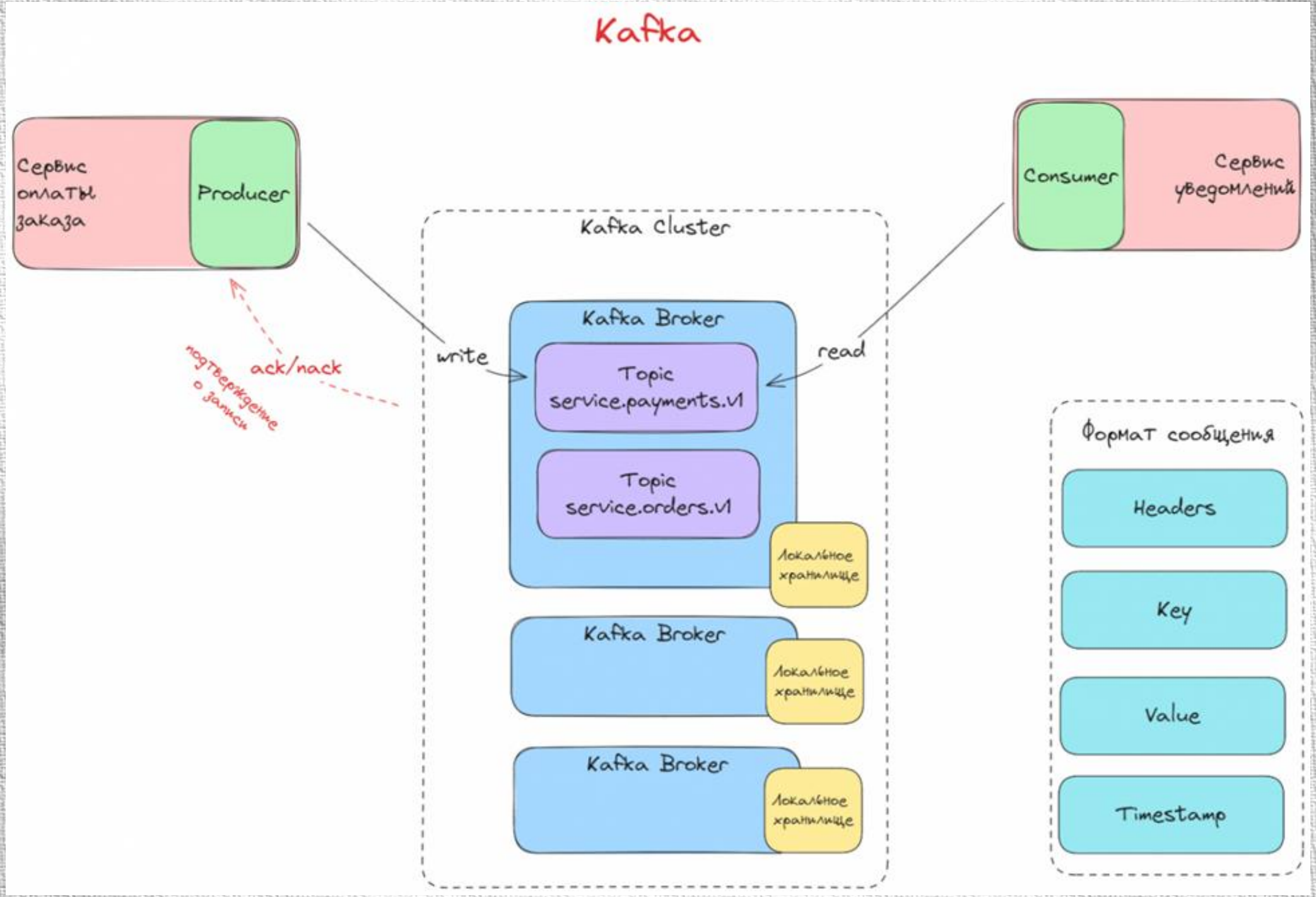
Система устроена таким образом, что поддерживает обоюдное уведомление об успешности доставки с двух сторон: после того как продюсером было отправлено целевое сообщение и оно получено, система отправляет продюсеру уведомление об успешном приёме. В свою очередь потребитель, если сообщение им успешно получено, также отправляет уведомление в систему. Если же получение прошло неуспешно, отправляется информационное сообщение, а сообщение от продюсера остаётся в очереди, пока не будет получено подписчиком.

Основной особенностью этого брокера является возможность настройки гибкого роутинга: при отправке сообщение необязательно должно проходить только прямолинейный путь от продюсера к подписчику. В процессе оно может проходить через ряд промежуточных узлов обмена, которые могут перенаправлять его в различные очереди.

❷ Apache Kafka

Apache Kafka представляет собой брокер, который, в отличие от RabbitMQ, хранит все сообщения в виде распределённого лога, причём гарантируется, что порядок сообщений отражает последовательность их поступления в систему. Сообщение в этом логе хранится в течение определённого времени, и работа построена таким образом, что продюсеры пишут новые сообщения в систему, а подписчики сами их запрашивают. При надобности организуется хранение сообщений в рамках тем. То есть можно сказать, что происходит определённого рода группировка сообщений в рамках одной темы.

Kafka


Сервис оплаты заказа
Producer
Consumer
Сервис уведомлений
Kafka Cluster
Kafka Broker
write
read
Topic service.payments.v1
Topic service.orders.v1
локальное хранилище
Kafka Broker
локальное хранилище
Kafka Broker
локальное хранилище
ack/nack
подтверждение о записи
Формат сообщения
Headers
Key
Value
Timestamp

Если попытаться некоторым образом обосновать выбор в пользу той или иной системы, то следует учесть, что RabbitMQ позволяет сконфигурировать даже весьма сложные сценарии доставки сообщений, что даёт разработчикам гибкость в построении нужного сценария информирования о событиях. При этом следует учитывать, что порядок доставки сообщений не гарантируется.

В свою очередь, брокер Apache Kafka больше предназначен для построения высоконагруженных систем сферы bigdata, так как сама его парадигма параллельной обработки, репликации позволяет создавать достаточно надёжные системы и обеспечивать неограниченные возможности по масштабированию.

Высокая пропускная способность, а также возможности извлечения сообщений из очереди за определённый период времени (так как они хранятся в очереди, как мы сказали ранее, именно в том порядке, в каком были отправлены) являются мощным инструментом для анализа происходящего в историческом разрезе.

МАСШТАБИРОВАНИЕ

- Сервисов
- Баз данных

Масштабируемость и расширяемость

Масштабируемость означает, что сеть позволяет наращивать количество узлов и протяженность связей в очень широких пределах, при этом производительность сети не ухудшается.

Расширяемость означает возможность добавления отдельных компонентов сети (пользователей, компьютеров, приложений, служб), наращивания длины сегментов кабелей и замены существующей аппаратуры более мощной.

Зачем нужна масштабируемость?

Веб-сайты передовых компаний должны выдерживать огромную нагрузку. К тому же, для успеха компании крайне важны высокая производительность и удобство эксплуатации приложений, web-сервисов и сайтов.

Новички, которые приступают к разработке своих первых web-сервисов и приложений, зачастую не знают о некоторых проблемах, с которыми придётся столкнуться впоследствии:

- **увеличение количества посетителей** снижает производительность web-сервиса и увеличивает процент отказов;
- **расширение ассортимента товаров** негативно сказывается на времени загрузки страницы; обновление каталога интернет-магазина становится проблематичным;

- **изменение структуры кода** становится опасным и сложным;
- **добавление нового продукта или услуги** потребует много времени и финансовых затрат, а возможности проведения A/B-тестирования сокращаются.

В результате, если проблемы не решаются, предприниматель или владелец web-сервиса теряет возможности, а ограничения сохраняются.

Масштабирование сервисов

Масштабирование сервисов

Масштабируемость (Scalability) — возможность приложения расти и обслуживать больше пользователей, транзакций, сетевого трафика и других сервисов без снижения его производительности и с сохранением корректности его работы.

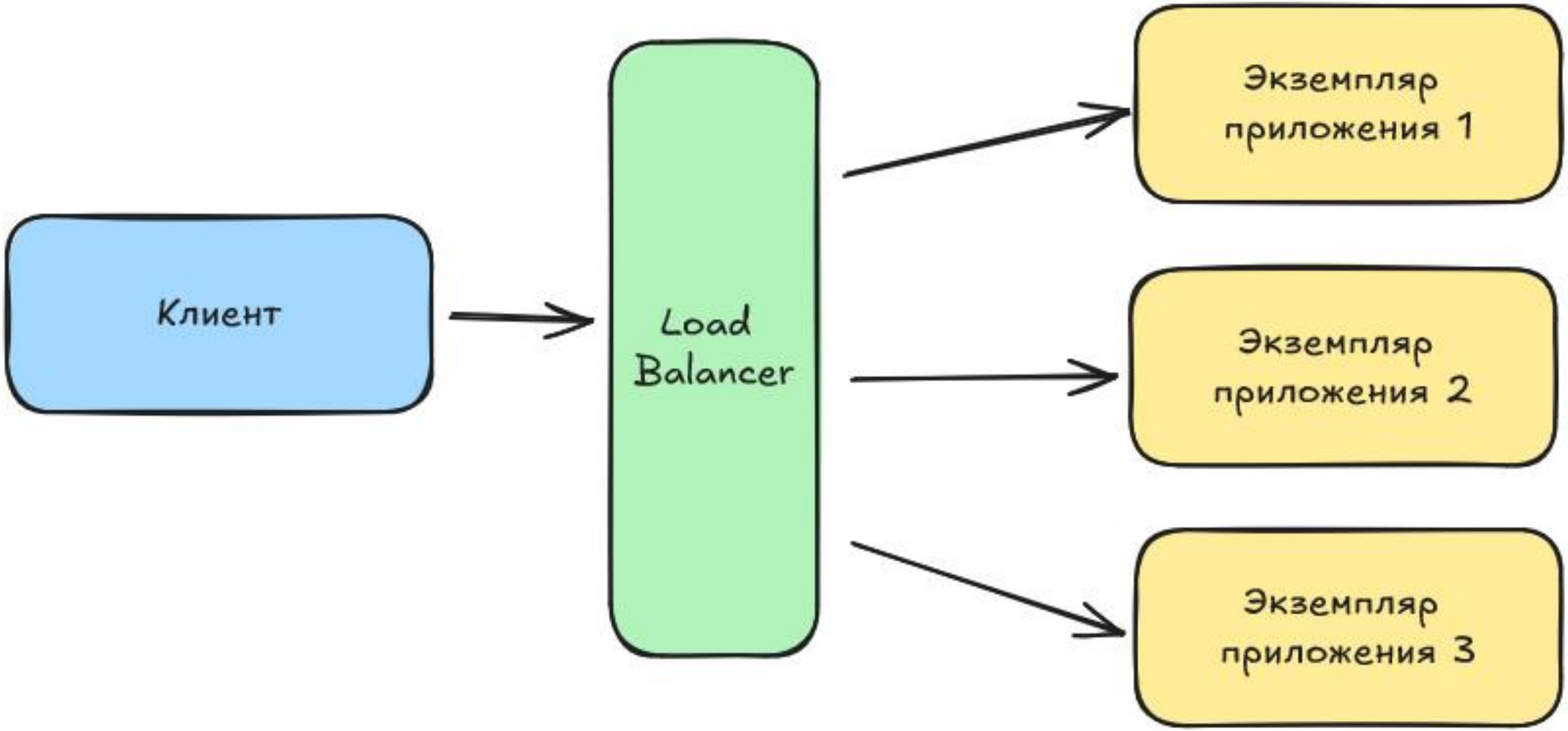
Масштабируют сервисы с помощью одного, двух или сразу трёх подходов:

1. **Копирование приложения** — создание копий приложения и равномерное распределение запросов между ними с помощью балансировщика.

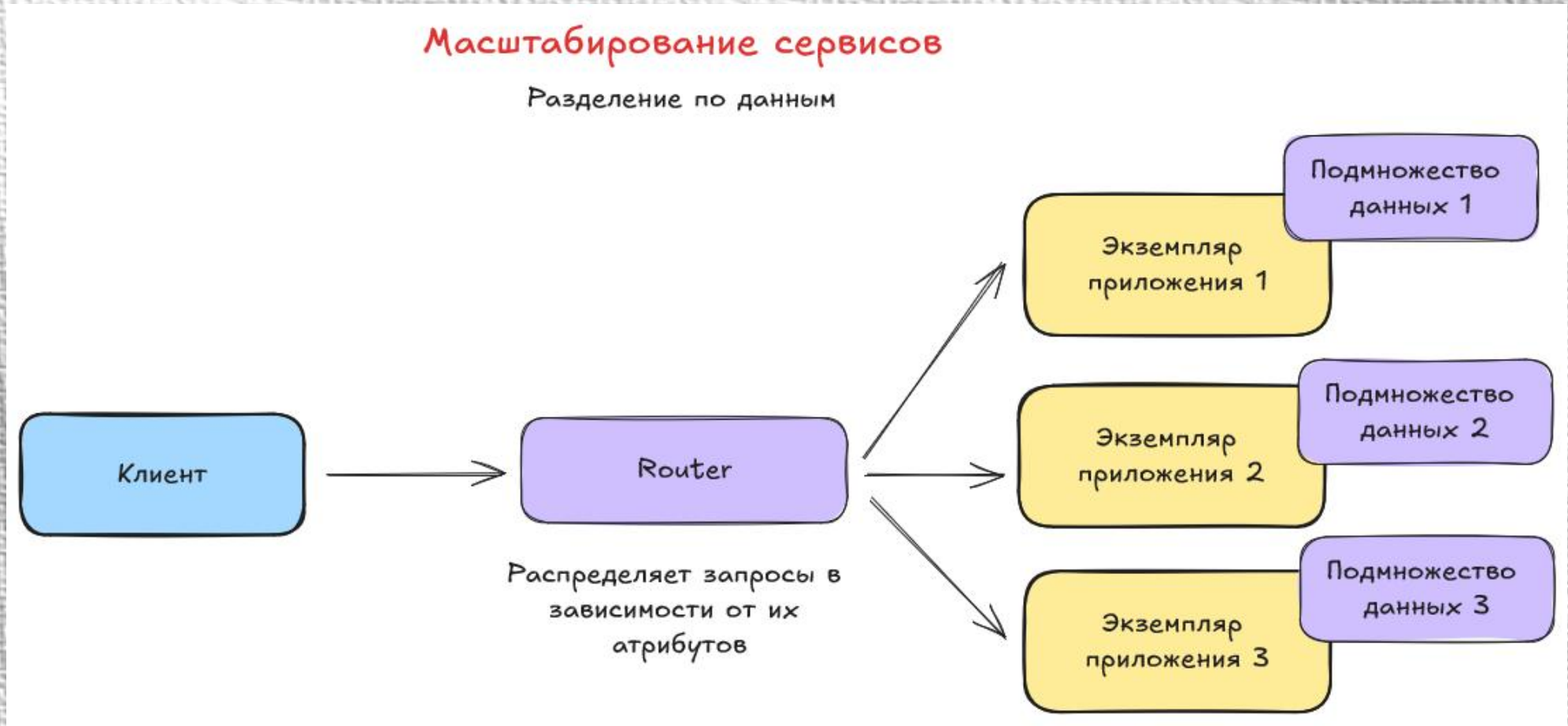
2. **Разделение по данным** — создание копий приложения, каждая из которых хранит только определённое подмножество данных (партицию), и распределение запросов между ними на основании значений их атрибутов с помощью роутера.
3. **Разделение по функциям** — разделение приложения на сервисы, каждый из которых выполняет только часть функций. Сервис, который масштабируется по функциям, можно в дальнейшем копировать и/или масштабировать по данным.
4. **Смешанный подход к масштабированию.**

Масштабирование сервисов

Копирование приложения

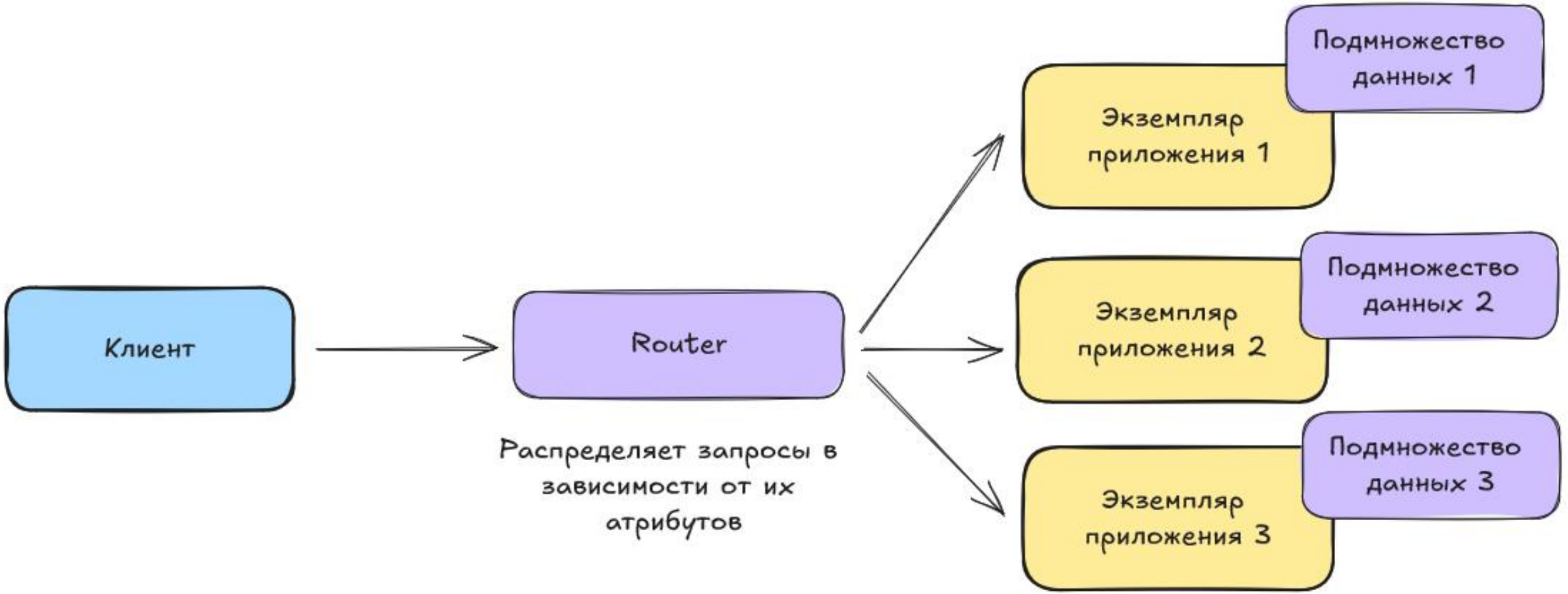


2. **Разделение по данным** — создание копий приложения, каждая из которых хранит только определённое подмножество данных (партицию), и распределение запросов между ними на основании значений их атрибутов с помощью роутера.



Масштабирование сервисов

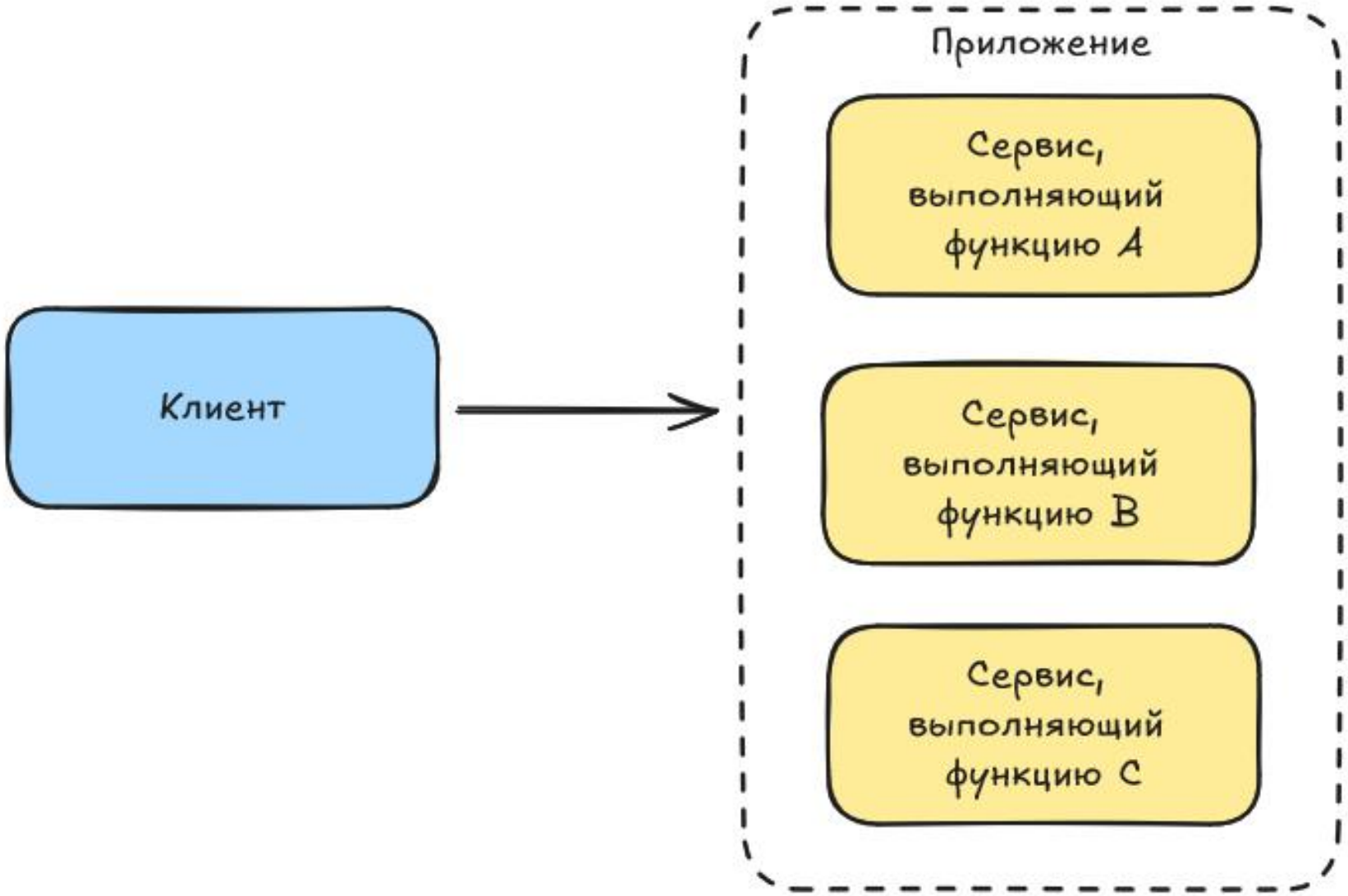
Разделение по данным



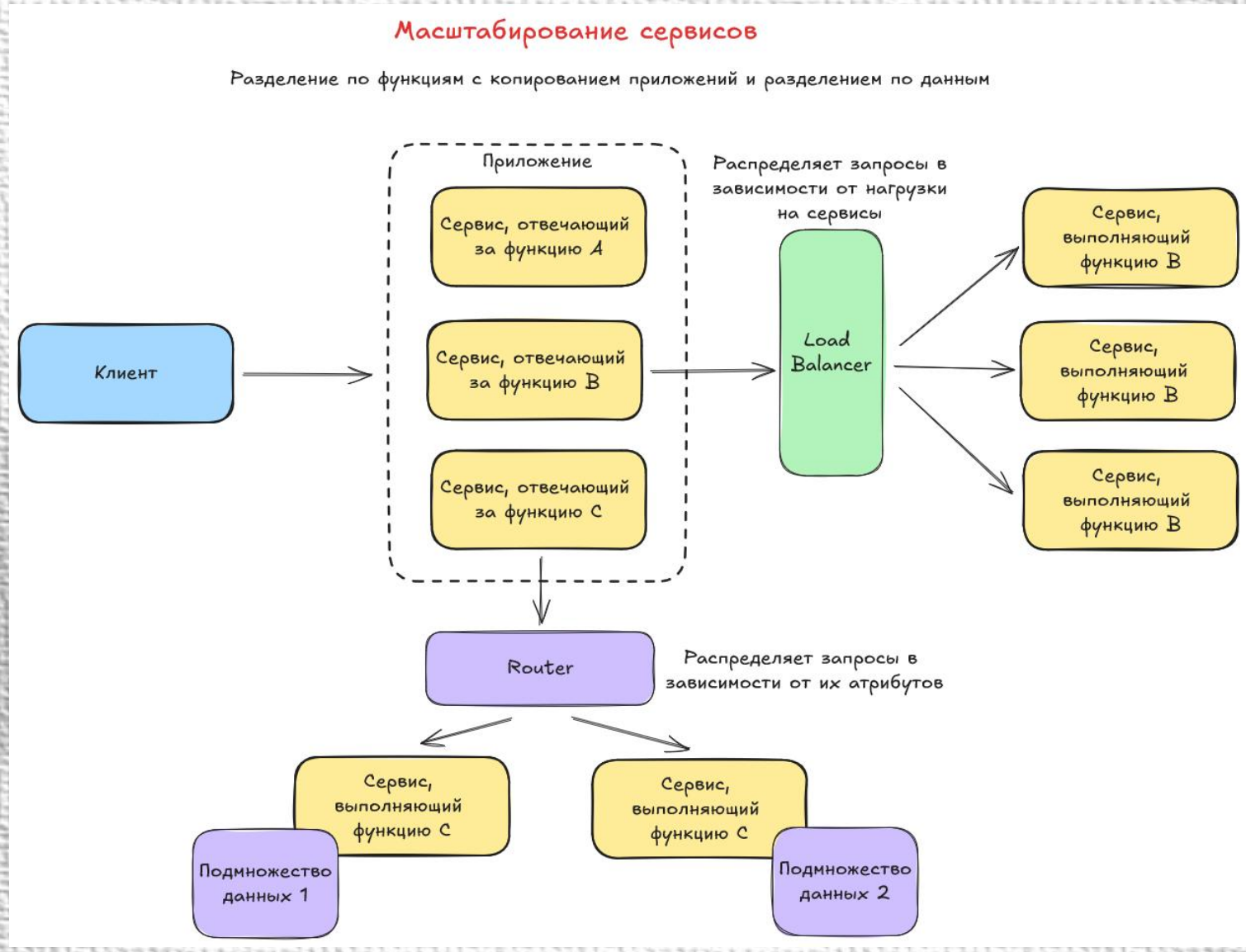
3. **Разделение по функциям** — разделение приложения на сервисы, каждый из которых выполняет только часть функций. Сервис, который масштабируется по функциям, можно в дальнейшем копировать и/или масштабировать по данным.

Масштабирование сервисов

Разделение по функциям



4. Смешанный подход к масштабированию.



Масштабирование баз данных

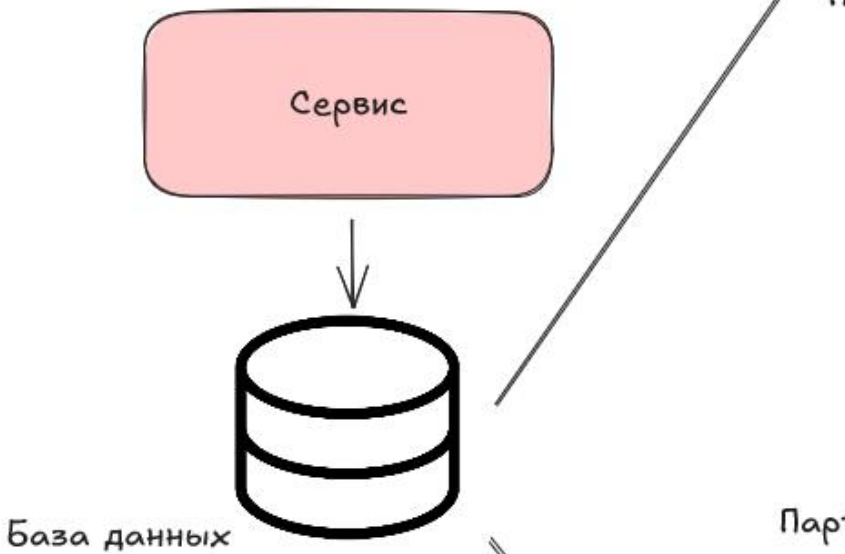
Масштабирование баз данных

Выделяют три метода:

1. **Партиционирование** — разделение БД на части в рамках одного сервера. Может быть вертикальным (по столбцам) или горизонтальным (по строкам);
2. **Шардирование** — разделение БД на части по разным серверам, может быть только горизонтальным (по строкам);
3. **Репликация** — копирование одних и тех же данных между разными серверами. Создаются два экземпляра БД — ведущий (Master) и ведомый (Slave). В нормальном режиме функционирования данные пишутся в Master, и реплицируются в Slave. В случае выхода из строя Master-ноды, взаимодействие будет моментально переключено на Slave до восстановления.

Масштабирование баз данных

Партиционирование БД



Горизонтальное партиционирование

Партиция 1

id	name	city	...
1	Nick	Moscow	
2	Anna	Los Angeles	
...	...	...	
1678	John	Sydney	

Партиция 2

id	name	city	...
1679	Mike	New York	
1680	Mary	Toronto	
...	...	...	
3124	Brad	Las Vegas	

Вертикальное партиционирование

Партиция 1

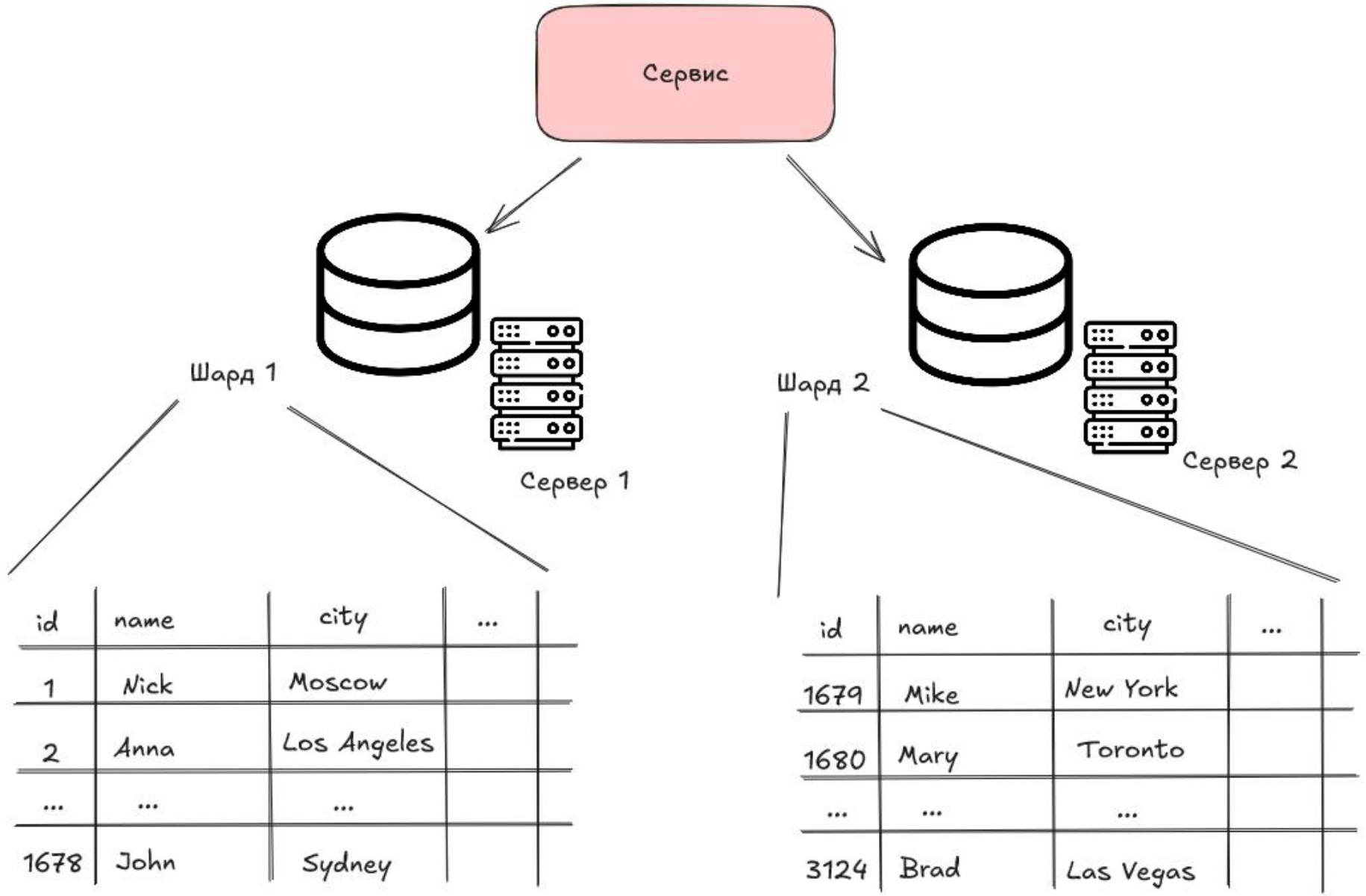
id	name	...
1	Nick	
2	Anna	
...	...	
3124	Brad	

Партиция 2

id	city	...
1	Moscow	
2	Los Angeles	
...	...	
3124	Las Vegas	

Масштабирование баз данных

Шардирование БД

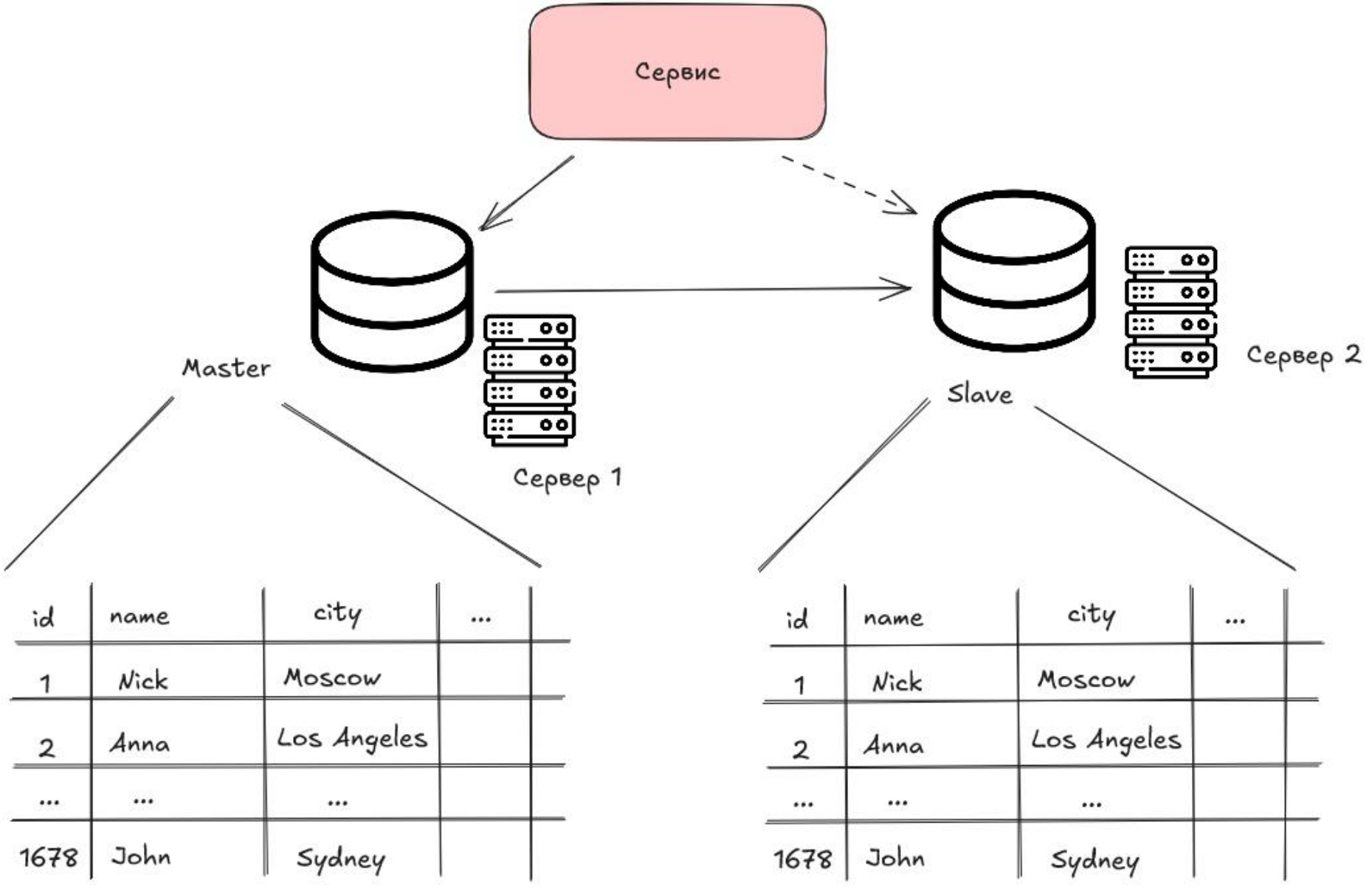


id	name	city	...
1	Nick	Moscow	
2	Anna	Los Angeles	
...	...	...	
1678	John	Sydney	

id	name	city	...
1679	Mike	New York	
1680	Mary	Toronto	
...	...	...	
3124	Brad	Las Vegas	

Масштабирование баз данных

Репликация БД



id	name	city	...
1	Nick	Moscow	
2	Anna	Los Angeles	
...	...	...	
1678	John	Sydney	

id	name	city	...
1	Nick	Moscow	
2	Anna	Los Angeles	
...	...	...	
1678	John	Sydney	

КЭШИРОВАНИЕ

Отдельно необходимо сказать про кэш, как подход к масштабированию и возможность снизить нагрузку на базу данных.

Уровень кэша - это слой временного хранилища данных, который по своей скорости работы намного опережает БД. К преимуществам отдельного уровня кэша можно отнести улучшение производительности системы.

Один из популярных и самых распространённых подходов к кешированию - **сквозное чтение**. Получив запрос, сервер сначала проверяет наличие данных в кэше. Если данные есть, данные возвращаются клиенту. Если нет, то сервер обращается к базе данных, сохраняет ответ в кэше и пересылает его обратно клиенту.

При проектировании кэша помимо выбора подхода к кэшированию, необходимо помнить ещё о нескольких аспектах:

- Предпочтительное использование кэша — для чтения редко изменяемых, но часто запрашиваемых данных.
- Согласованность — периодическое обновление данных в кэше данными из БД.
- Выбор срока действия кэша. Рекомендуется реализовать механизм, ограничивающий срок действия кэша. Устаревшие данные должны немедленно удаляться.

- Выбор политики вытеснения — когда кэш полностью заполнен, любой запрос на добавление новых элементов может привести к удалению существующих. Самой популярной политикой считается вытеснение давно неиспользуемых данных (least- recently- used, **LRU**). Для разных ситуаций могут также подойти вытеснение наименее часто используемых данных (least-frequently-used, **LFU**) или метод «первым пришел, первым ушел» (**FIFO**, first-in-first-out).

Этап 3

ОСНОВНЫЕ АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ

- Многослойная архитектура (Layered Architecture).
- Многоуровневая архитектура (Tiered Architecture)
- Сервис-ориентированная архитектура (Service Oriented Architecture—SOA)
- Микросервисная архитекура (Microservice Architecture).

Какие есть архитектурные подходы

Мы можем условно разделить архитектурные подходы по их целям и зоне действия.

1. Часть подходов распределяют ответственность между модулями. Они определяют, какие модули и за что будут отвечать.

Самый известный из таких подходов — Model-View-Controller. Он выделяет 3 «типа» задач, в соответствии с которыми назначает модулям роли.

2. Другие определяют, насколько каждый из модулей близок к бизнес-логике. Таким подходам важно, какая часть кода занимается непосредственно задачей приложения, а какая — инфраструктурными задачами.

Например, в приложении для обработки фотографий бизнес-логикой были бы функции фильтров, а инфраструктурными задачами — обращение к API камеры телефона.

В зависимости от степени близости к бизнес-логике такие подходы делят код на «слои». Самый распространённый подход среди таких — это трёхслойная архитектура.

3. Третьи управляют потоками данных в приложении. Они определяют, как модули общаются друг с другом: напрямую, опосредованно или с помощью специальных сервисов типа шины событий.

Во фронтенде самый, наверное, известный подход — это Flux и его самая распространённая реализация — Redux. Это пример однонаправленного потока данных. Кроме него используют двунаправленный поток, например, реактивные обновления данных.

4, Другие подходы определяют компоновку приложения. Будет это одна большая программа (монолит) или набор нескольких программ поменьше (микросервисов).

Архитектурный стиль (reference architecture, шаблонная архитектура) – это абстрактная архитектура, применимая к некоторому множеству программных продуктов.

За годы развития ПО разработчикам удалось придумать надежные подходы, чтобы устранить недостатки проектирования без архитектуры. Ниже представлены некоторые из самых известных.

1. **Многослойная архитектура (Layered Architecture).**
2. **Многоуровневая архитектура (Tiered Architecture).**
3. **Сервис-ориентированная архитектура (Service Oriented Architecture—SOA).**
4. **Микросервисная архитектура (Microservice Architecture).**

Слои и уровни архитектуры

Слои в архитектуре — это механизмы **логического** структурирования компонентов, из которых состоит программное решение. Наиболее часто встречаются четыре слоя:

1. **Слой представления** (слой UI, UIL, пользовательский интерфейс).
2. **Слой приложения** (сервисный слой, уровень управления).
3. **Слой бизнес-логики** (слой предметной области, доменный слой).
4. **Слой доступа к данным** (слой хранения данных, слой инфраструктуры; логирование, сетевые взаимодействия и другие сервисы).

Уровни в архитектуре — это механизмы **физического** структурирования инфраструктуры системы. Многоуровневая архитектура делит приложение на уровни, каждый из которых выполняет определённую функцию. Чаще всего встречаются четыре стандартных уровня:

1. **Уровень представления**. Представляет собой UI, то есть представление контента конечному пользователю через графический интерфейс.
2. **Уровень приложения**. Сервисный слой или уровень управления.
3. Также на этом уровне работает **бизнес-логика приложения**.
4. **Уровень данных**. Это ядро архитектуры. В основном этот слой связан с хранением и извлечением данных приложения.

Количество и типы уровней могут варьироваться.

Типы программной архитектуры

Архитектурные стили можно разделить на два основных типа:

1. **монолитные** (с единым развертываемым блоком для всего кода)
2. **распределенные** (с несколькими развертываемыми блоками, взаимодействующими друг с другом по различным протоколам).

Типы архитектуры приложений

Монолитная

- многоуровневая архитектура
- конвейерная архитектура
- микроядерная архитектура

Распределенная

- архитектура на основе сервисов
- архитектура, управляемая событиями
- архитектура на основе пространства
- сервис-ориентированная архитектура
- микросервисная архитектура

Фреймворки и подходы проектирования архитектуры ПО

Существует **множество фреймворков и подходов, которые п**омогают при проектировании архитектуры программного обеспечения. Вот несколько популярных фреймворков, которые широко используются в индустрии:

1. **Модель-Представление-Контроллер (Model-View-Controller, MVC):** MVC — один из самых распространенных фреймворков для разработки веб-приложений. Он разделяет приложение на три основных компонента: **модель** (отвечает за данные и бизнес-логику), **представление** (отображает данные пользователю) и **контроллер** (управляет взаимодействием между моделью и представлением). MVC позволяет легко поддерживать разделение ответственности и повышает модульность приложения.

2. **Model-View-ViewModel (MVVM):** MVVM — фреймворк, особенно популярный в разработке мобильных и десктопных приложений. Он расширяет концепцию MVC, добавляя «ViewModel» — промежуточный слой, который связывает модель и представление. ViewModel отвечает за предоставление данных представлению и обработку пользовательского ввода. MVVM способствует более тесной связи между представлением и моделью данных и упрощает тестирование пользовательского интерфейса.

3. **Domain-Driven Design (DDD):** DDD — подход к проектированию архитектуры, который сосредотачивается на бизнес-логике и предметной области. Он помогает разработчикам понять бизнес-требования и организовать систему вокруг концептуальных моделей предметной области. DDD предлагает использовать язык бизнеса в коде и выделять важные аспекты предметной области в отдельные слои (агрегаты, сервисы, репозитории и т.д.). Это позволяет создавать более гибкие и выразительные системы.

4. **Clean Architecture:** Clean Architecture (Чистая архитектура), разработанная Робертом Мартином (Uncle Bob), предлагает модульную и независимую от внешних фреймворков архитектуру. Она основана на принципах разделения на уровни, зависимости от абстракций и принципе единственной ответственности. Clean Architecture ставит акцент на выделение ядра системы и разделении ее на слои, что обеспечивает гибкость, тестируемость и поддерживаемость.
5. **Event-Driven Architecture (EDA):** EDA — фреймворк, ориентированный на обработку событий. В нем система строится вокруг отправки, приема и обработки событий, что позволяет реагировать на изменения в реальном времени и строить асинхронные и распределенные системы. EDA может использоваться в сочетании с другими архитектурными стилями, такими как микросервисная архитектура или серверно-ориентированная архитектура (SOA).

Важно отметить, что каждый фреймворк имеет свои особенности и подходит для различных типов проектов. Выбор фреймворка зависит от требований проекта, команды разработчиков и особенностей предметной области. Часто разработчики комбинируют несколько фреймворков и подходов, чтобы создать оптимальную архитектуру для своего проекта.

МНОГОСЛОЙНАЯ ТИПОВАЯ АРХИТЕКТУРА ПРИЛОЖЕНИЯ

Многослойная архитектура

Этот подход работает по принципу разделения ответственностей. ПО разделено на слои, лежащие друг на друге, и каждый из них выполняет определенную обязанность.

Архитектура делит ПО на следующие слои.

Слой представления (Presentation Layer) содержит пользовательский интерфейс и отвечает за обеспечение хорошего пользовательского опыта.

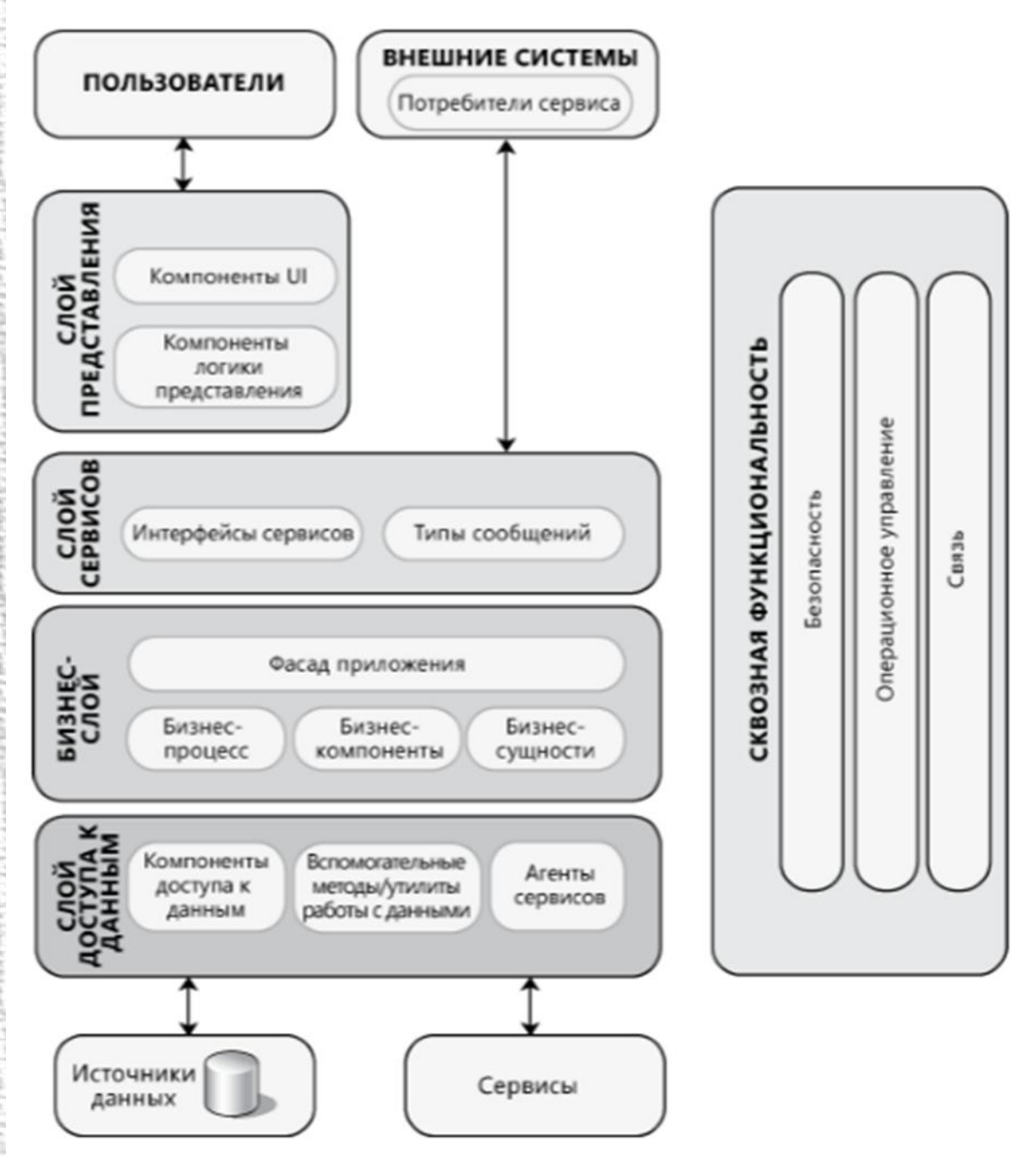
Слой бизнес-логики (Business Logic Layer), как следует из названия, содержит бизнес-логику приложения. Он отделяет UI/UX от вычислений, связанных с бизнесом. Это позволяет с легкостью изменять логику в зависимости от постоянно меняющихся бизнес-требований, никак не влияя на другие слои.

Слой передачи данных (Data Link Layer) отвечает за взаимодействие с постоянными хранилищами, такими как базы данных, и прочую обработку информации, которая не связана с бизнесом.

Данные и элементы управления проходят через каждый слой в дизайне и передаются от одного к другому. Эта система также повышает уровень абстракции и в некоторой степени даже стабильность ПО.

ТИПОВАЯ АРХИТЕКТУРА ПРИЛОЖЕНИЯ

На рисунке представлена типовая архитектура приложения, компоненты которого сгруппированы по функциональным областям.



Хотя три основных слоя — представление, бизнес-логика и источник данных — можно обнаружить в любом корпоративном приложении, способ их разделения зависит от степени сложности этого приложения.

Простой сценарий извлечения порции информации из базы данных и отображения ее в контексте Web-страницы можно описать одной процедурой. Но я все равно попытался бы выделить в нем три слоя — пусть даже, как в этом случае, распределив функции каждого слоя по разным подпрограммам. При усложнении приложения это дало бы возможность разнести код слоев по отдельным классам, а позже разбить множество классов на пакеты. Форма расслоения может быть произвольной, но в любом корпоративном приложении слои должны быть идентифицированы

Помимо необходимости разделения на слои, существует правило, касающееся взаимоотношения слоев: **зависимость бизнес-логики и источника данных от уровня представления не допускается**. Другими словами, в тексте приложения не должно быть вызовов функций представления из кода бизнес-логики или источника данных. Правило позволяет упростить возможность адаптации слоя представления или замены его альтернативным вариантом с сохранением основы приложения. Связь между бизнес-логикой и источником данных, однако, не столь однозначна и во многом определяется выбором типовых решений для архитектуры источника данных.

Слой пользовательского интерфейса

(UIL – User Interface Layer)

В самом верху пирамиды находится слой пользовательского интерфейса (UIL – User Interface Layer). Какой технологией он будет представлен – это уже вопрос второй, и это зависит от типа приложения. Это могут быть элементы Windows Forms, это могут быть страницы WPF приложения, это могут быть Web Pages технологии ASP.NET и многое другое.

Основная задача этого слоя – предоставлять эргономичный интерфейс пользователю в соответствии с функционалом, описанном в техническом задании. Этот слой просто должен быть красивым, удобным и интуитивно понятным. При этом он должен знать, каким образом он взаимодействует со слоем, находящимся на уровень ниже, как он передаёт туда данные и как он их оттуда принимает для отображения.

Что же касается логики слоя находящегося под ним, UIL о нём ничего не знает. Какие алгоритма выполняются в тех методах, которые он вызывает по нажатию кнопки или по изменению текста в поле ввода, или при выборе пункта из списка, UIL остаётся в счастливом неведении.

К главным функциям слоя представления относятся отображение информации и интерпретация вводимых пользователем команд с преобразованием их в соответствующие операции в контексте домена (бизнес-логики) и источника данных.

Слой сервисов
(Service layer - SL)

Сервисный слой инкапсулирует логику приложения от его представления и предоставляет набор доступных операций. Таким образом, удобно представить его с использованием паттерна **Фасад**. Для слоя бизнес-логики данный слой фактически перенаправляет все запросы в слой бизнес-логики. Помимо этого в сервисном слое осуществляется предварительная проверка корректности введенных данных.

Слой сервисов мы сейчас подробно обсуждать не будем, лучше сделаем это в главе, посвящённой SOA (сервисно-ориентированной архитектуре). Этот слой может не присутствовать и не быть задействованным в 3-слойной архитектуре.

Слой бизнес-логики
(BLL – Business Logic Layer)

Ниже находится слой бизнес-логики классы, описывающие сущности и основную логику работы приложения (основные алгоритмы работы приложения, вычисления, обработка и т.д.). Например, если мы пишем сетевые “крестики – нолики”, то на этом слое могут быть описаны классы, описывающие игровое поле, выполненный ход, отправку данных по сети и т.д.

Бизнес-логика — это реализация правил и ограничений автоматизируемых операций.

Является синонимом термина «логика предметной области» (*domain logic*).

Логика домена (бизнес-логика или логика предметной области) описывает основные функции приложения, предназначенные для достижения поставленной перед ним цели. К таким функциям относятся вычисления на основе вводимых и хранимых данных, проверка всех элементов данных и обработка команд, поступающих от слоя представления, а также передача информации слою источника данных.

Проще говоря, бизнес-логика — это реализация предметной области в информационной системе. К ней относятся, например, формулы расчёта ежемесячных выплат по ссудам (в финансовой индустрии), автоматизированная отправка сообщений электронной почты руководителю проекта по окончании выполнения частей задания всеми подчиненными (в системах управления проектами), отказ от отеля при отмене рейса авиакомпанией (в туристическом бизнесе) и т. д.

В фазе анализа и проектирования системы бизнес-логика воплощается в различных диаграммах языка UML или ему подобных. В фазе программирования бизнес-логика воплощается в коде классов и их методов в случае использования объектно-ориентированных языков программирования, или процедур и функций, в случае применения процедурных языков.

В добавок к бизнес логике (ядро предметной области или core domain), в слое предметной области часто можно встретить дополнительную логику, например \\

- события предметной области (domain events, события, которые выбрасываются в ключевых точках бизнес логики) и
- “сценарии использования” или use-cases (определение того, что должно делать наше приложение).

Где бы ни находился код бизнес-логики, его следует сохранять в отдельных модулях, не связанных со слоем представления, используя одно из типовых решений — **сценарий транзакции** (Transaction Script) или **модель предметной области** (Domain Model)

Иногда слои организуют таким образом, чтобы бизнес-логика полностью скрывала источник данных от представления. Чаще, однако, код представления может обращаться к источнику данных непосредственно. Хотя такой вариант менее безупречен с теоретической точки зрения, в практическом отношении он нередко более удобен и целесообразен: код представления может интерпретировать команду пользователя, активизировать функции источника данных для извлечения подходящих порций информации из базы данных, обратиться к средствам бизнес-логики для анализа этой информации и осуществления необходимых расчетов и только затем отобразить соответствующую картинку на экране.

СЛОЙ ДОСТУПА К ДАННЫМ
(DAL – DATA ACCESS LAYER)

Ниже находится слой доступа к данным (DAL – Data Access Layer). Это работа с источником данных (базами данных разных поставщиков MSSQL, MySql, Oracle…), или XML файлами, или бинарными данными, или текстовыми файлами, короче говоря, всем тем, в чём можно данные хранить постоянно. Здесь тоже описана логика, но направленная именно на взаимодействие с источником и передачу полученных результатов на уровень выше.

Могут быть ещё и другие слои, но какие и в какой последовательности они расположены в пирамиде архитектуры – зависит от приложения

СКВОЗНАЯ ФУНКЦИОНАЛЬНОСТЬ

Сквозная функциональность – это аспекты дизайна, которые могут применяться ко всем слоям, компонентам и уровням. Также это те области, в которых чаще всего делаются ошибки, имеющие большое влияние на дизайн.

Приведем примеры сквозной функциональности:

- **Аутентификация и авторизация**. Как правильно выбрать стратегию аутентификации и авторизации, передачи идентификационных данных между слоями и уровнями и хранения удостоверений пользователей.
- **Кэширование**. Как правильно выбрать технику кэширования, определить данные, подлежащие кэшированию, где кэшировать данные и как выбрать подходящую политику истечения срока действия.

- **Связь.** Как правильно выбрать протоколы для связи между слоями и уровнями, обеспечения слабого связывания между слоями, осуществления асинхронного обмена данными и передачи конфиденциальных данных.
- **Управление конфигурацией**. Как выявить данные, которые должны быть настраиваемыми, где и как хранить данные конфигурации, как защищать конфиденциальные данные конфигурации и как обрабатывать их в серверной ферме или кластере.
- **Управление исключениями**. Как обрабатывать и протоколировать исключения и обеспечивать уведомления в случае необходимости.

- **Протоколирование и инструментирование**. Как выбрать данные, подлежащие протоколированию, как сделать протоколирование настраиваемым, и как определить необходимый уровень инструментирования.
- **Валидация**. Как определить, где и как проводить валидацию; как выбрать методики для проверки длины, диапазона, формата и типа; как предотвратить и отклонить ввод недопустимых значений; как очистить потенциально злонамеренный и опасный ввод; как определить и повторно использовать логику валидации на разных слоях и уровнях приложения.

Набор правил

- *каждый слой занимается конкретной задачей.*
- *логика разных слоёв не повторяется и не пересекается*
- *способ обращения к нижестоящему слою чётко определён*
- *способ поставки информации вышестоящему слою чётко определён*
- *слои слабо связаны*
- *слои расположены вертикально, хотя есть сквозная функциональность, которая может пронизывать пирамиду сверху вниз*
- *слои могут размещаться физически на одном компьютере (в пределах одного уровня), а могут быть на разных машинах, например, в распределённых приложениях.*

- *логика разных слоёв инкапсулирована, соответственно, разным слоям не нужно делать никаких предположений о том, как реализован код других слоёв приложения.*
- *возможность использования в других сценариях (за счёт слабого связывания и чётко определённой задачи).*

Слои приложения могут размещаться физически на одном компьютере (на одном уровне) или быть распределены по разным компьютерам (n-уровней), и связь между компонентами разных уровней осуществляется через строго определенные интерфейсы. Например, типовое Веб-приложение состоит из слоя представления (функциональность, связанная с UI), бизнес-слоя (обработка бизнес-правил) и слоя данных (функциональность, связанная с доступом к данным, часто практически полностью реализуемая с помощью высокоуровневых инфраструктур доступа к данным).

Подробно n-уровневая архитектура приложения рассматривается ниже в разделе N-уровневая / 3-уровненая архитектура.

Общие принципы проектирования с использованием многослойной архитектуры

Рассмотрим общие принципы проектирования с использованием многослойной архитектуры:

Абстракция. Многослойная архитектура представляет систему как единое целое, обеспечивая при этом достаточно деталей для понимания ролей и ответственностей отдельных слоев и отношений между ними

Инкапсуляция. Во время проектирования нет необходимости делать какие-либо предположения о типах данных, методах и свойствах или реализации, поскольку все эти детали скрыты в рамках слоя.

Четко определенные функциональные слои. Разделение функциональности между слоями очень четкая. Верхние слои, такие как слой представления, посылают команды нижним слоям, таким как бизнес-слой и слой данных, и могут реагировать на события, возникающие в этих слоях, обеспечивая возможность передачи данных между слоями вверх и вниз.

Высокая связность. Четко определенные границы ответственности для каждого слоя и гарантированное включение в слой только функциональности, напрямую связанной с его задачами, поможет обеспечить максимальную связность в рамках слоя.

Возможность повторного использования. Отсутствие зависимостей между нижними и верхними слоями обеспечивает потенциальную возможность их повторного использования в других сценариях.

Слабое связывание. Для обеспечения слабого связывания между слоями связь между ними основывается на абстракции и событиях.

Преимущества

- Более простая реализация по сравнению с другими подходами.
- Предлагает абстракцию благодаря разделению ответственностей между уровнями.
- Изолирование защищает одни слои от изменений других.
- Повышает управляемость программного обеспечения за счет слабой связанности.

Недостатки

- Не предлагает большой масштабируемости.
- ПО, созданное с таким подходом, будет иметь монолитную структуру, усложняющую внесение модификаций.
- Данные должны проходить по каждому слою, даже если нет необходимости передавать их с определенных слоев.

МНОГОУРОВНЕВАЯ АРХИТЕКТУРА (TIERED ARCHITECTURE)

Этот архитектурный подход разделяет комплекс ПО на уровни по принципу взаимодействия “клиент-сервер”. Архитектура может иметь один, два и больше уровней, разделяющих ответственности между поставщиком данных и потребителем.

Этот подход использует шаблон *Request Response* для связи между уровнями. В отличие от многослойной архитектуры, он предлагает масштабируемость, которая может быть как горизонтальной (масштабирование сети с помощью высокопроизводительных узлов), так и вертикальной (масштабирование каждого узла путем повышения его производительности).

Одноуровневая система

Одноуровневая архитектура, монолитная архитектура — это подход к организации программного обеспечения, при котором все компоненты системы работают на одном уровне абстракции. **Это означает, что клиентская часть, бизнес-логика и база данных находятся в едином пространстве. Один уровень = на одном компьютере.**

В данном подходе единая система может работать как на стороне сервера, так и клиента. Это обеспечивает простоту развертывания и отличную скорость связи, а также устраняет необходимость межсистемного взаимодействия (Inter-system communication — ISC).

Такая система подходит только для небольших однопользовательских приложений.

Принцип работы

- Прямая коммуникация между компонентами без промежуточных уровней или слоёв.
- **Разделение ответственности:** клиент отвечает за представление данных и взаимодействие с пользователем, сервер — за обработку запросов и хранение данных.
- **Централизация данных**: все данные сосредоточены на сервере, что облегчает контроль доступа и обеспечение целостности.
- **Автономность клиента:** клиентская часть может функционировать независимо от сервера в периоды отсутствия необходимости в обмене данными.

Важно понимать, что **понятие «одноуровневости» относится именно к количеству логических слоёв в архитектуре, а не к физическому распределению компонентов**. Клиент и сервер могут быть расположены как на одном физическом устройстве, так и на разных, соединённых сетью.

Плюсы

Некоторые преимущества одноуровневой архитектуры:

Простота реализации и минимальные требования к ресурсам.

Высокая производительность — отсутствие необходимости передачи данных между уровнями снижает время отклика системы. Особенно это важно для приложений реального времени и высоконагруженных систем.

Безопасность данных — поскольку все компоненты находятся в едином пространстве, легче контролировать доступ к информации и предотвращать несанкционированное проникновение. Это особенно актуально для корпоративных приложений, работающих с конфиденциальными данными.

Минусы

Проблемы с масштабированием — поскольку сервер напрямую обрабатывает запросы клиентов, увеличение нагрузки может привести к снижению производительности. Это можно компенсировать вертикальным масштабированием (увеличение мощности сервера) или горизонтальным (распределение нагрузки между несколькими серверами с балансировщиком), но последнее частично выводит систему за рамки чистой одноуровневой модели.

Сложности в синхронизации — данные на разных рабочих машинах часто сохраняются в собственных вариантах, что может вызывать проблемы.

Проблемы безопасности — поскольку всё находится на одном компьютере, если кто-то получит доступ к системе, он сможет легко получить доступ как к данным, так и к приложению.

Примеры

Некоторые примеры систем с одноуровневой архитектурой:

Автономные приложения — программы для работы с документами, графические редакторы.

Встраиваемые системы — программное обеспечение для бытовой техники, автомобилей.

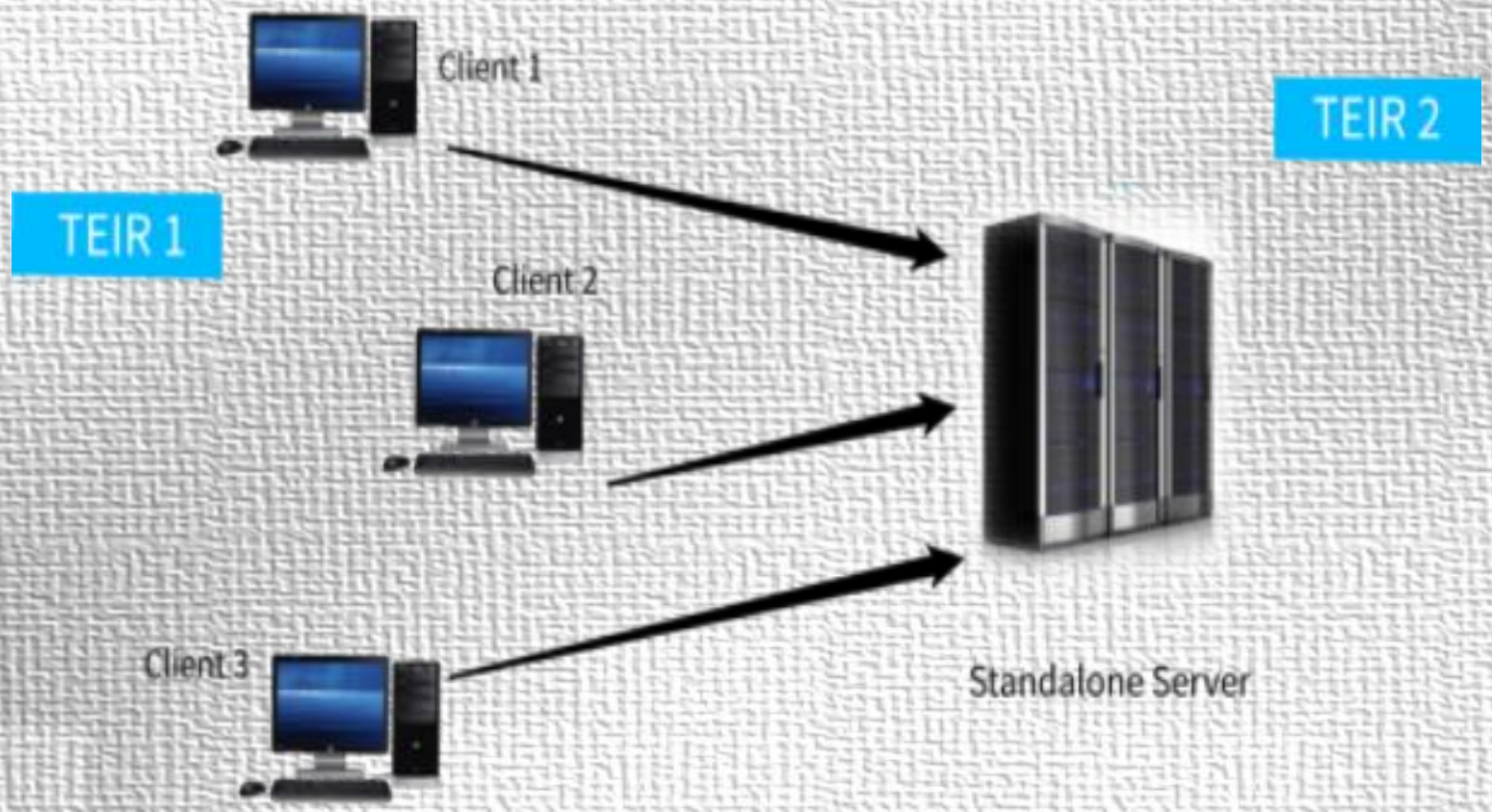
Системы реального времени — промышленная автоматизация, медицинское оборудование.

Мобильные приложения — офлайн-функционал различных сервисов.

Двухуровневая система

Эта система состоит из двух физических машин в качестве сервера и клиента. Все файлы и прикладные программы, необходимые для работы системы, располагаются на сервере, а на рабочие станции устанавливаются клиентские приложения, обеспечивающие доступ к общему серверу и возможность работы с данными. Этими системами проще управлять, они обеспечивают высокую производительность и легко масштаби-руются.

Двухуровневая архитектура

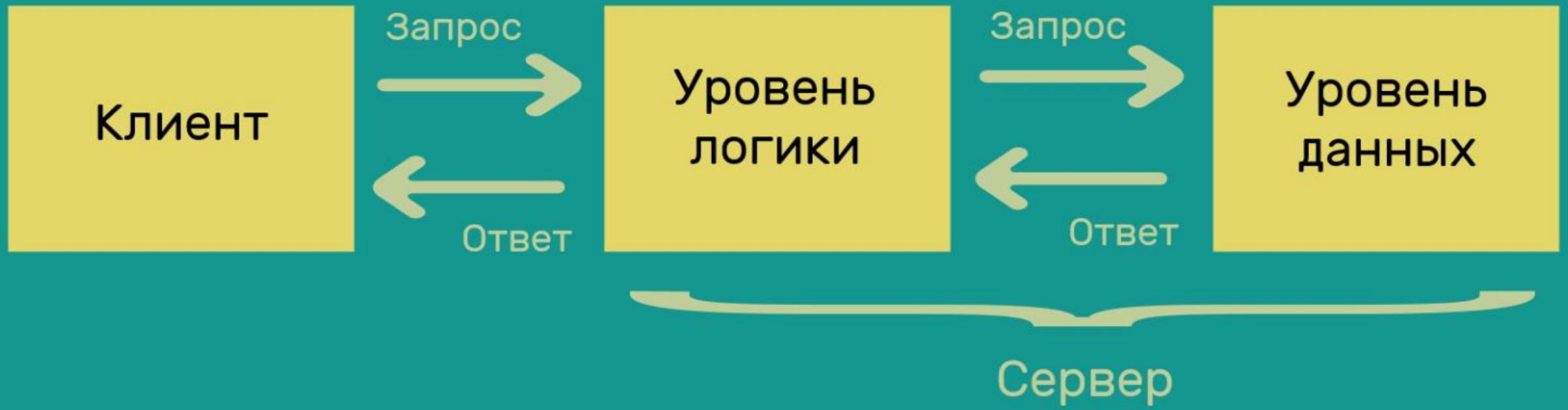


Трехуровневая и n-уровневая системы

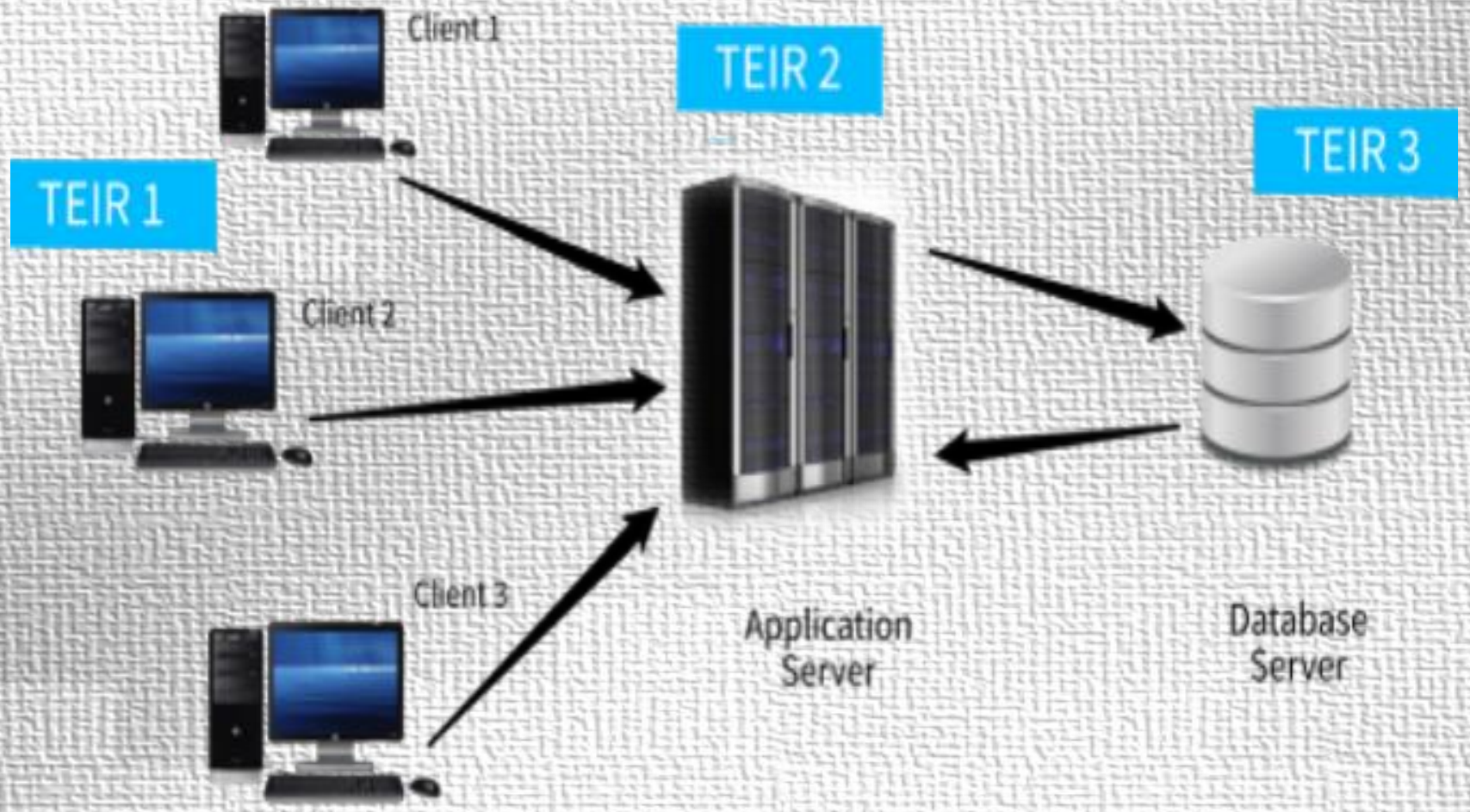
Клиент напрямую не взаимодействует с сервером. Вместо этого он взаимодействует с сервером приложений, который в дальнейшем взаимодействует с системой баз данных, а затем происходит обработка запросов и управление транзакциями.

Такие архитектуры обладают высокой масштабируемостью как по горизонтали, так и по вертикали. Реализация n-уровневой системы, как правило, обходится дороже, но обеспечивает высокую производительность. Поэтому она обычно применяется в крупных и комплексных программных решениях.

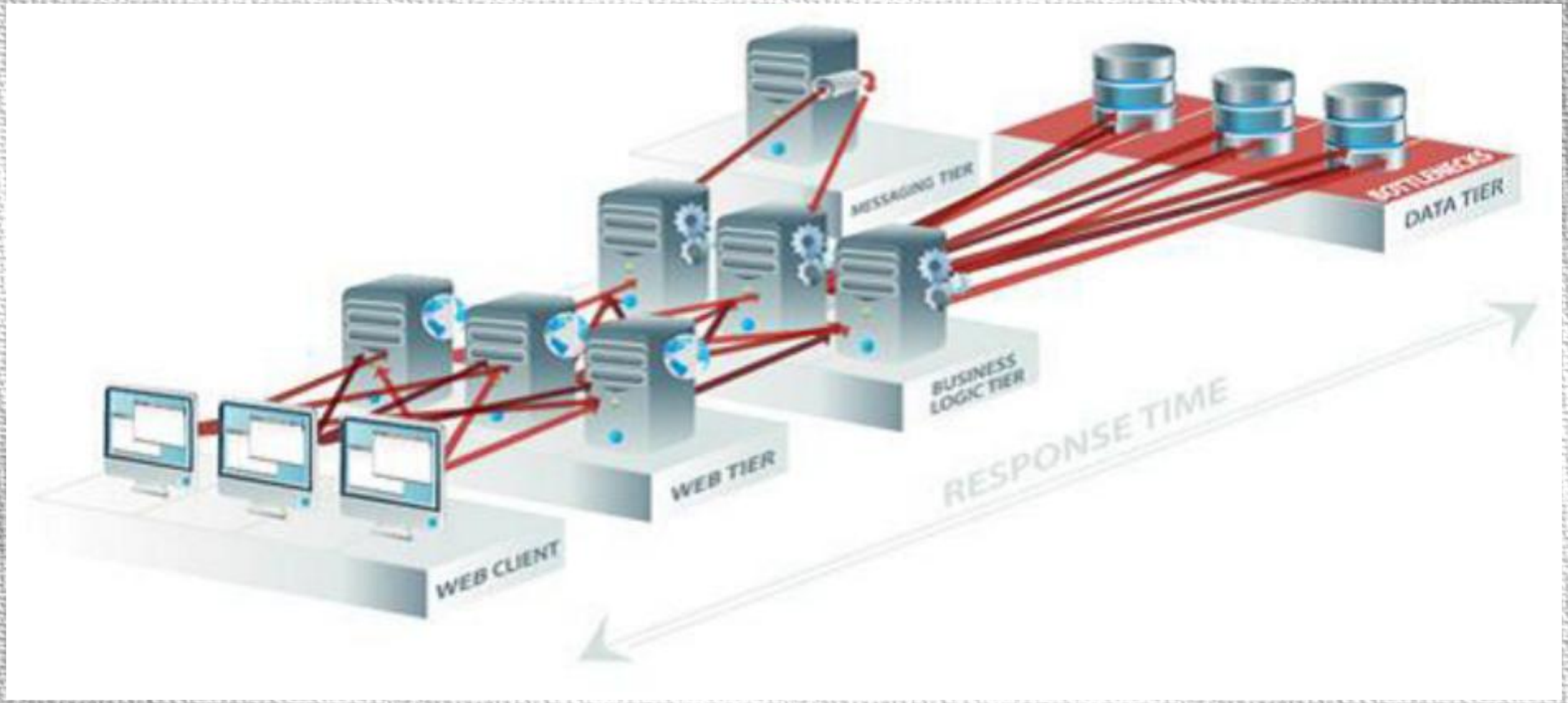
Клиент-серверная архитектура: трехуровневая



Трехуровневая архитектура

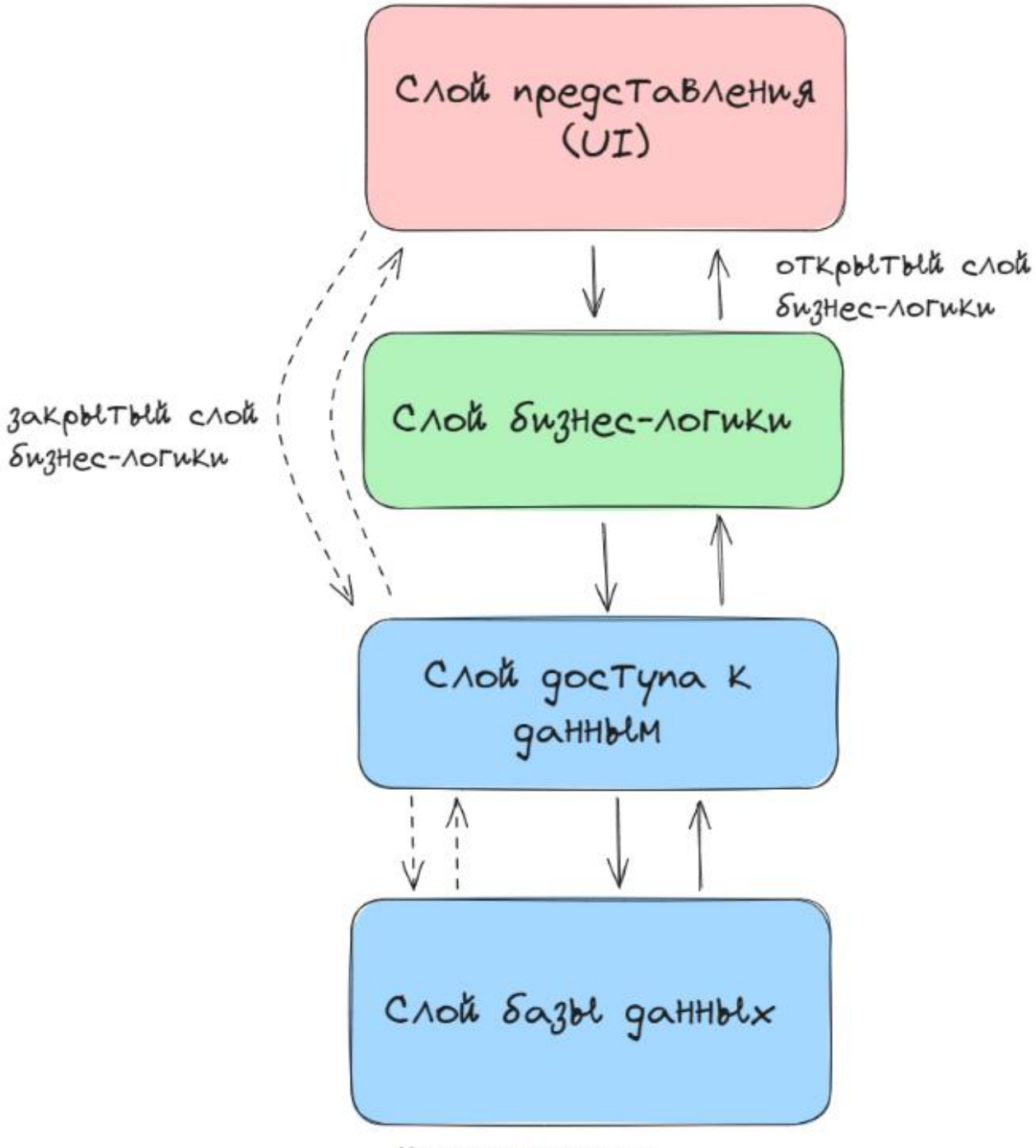


Этот подход можно сочетать с современной сервис-ориентированной архитектурой, чтобы создавать сложнейшие модели. Поскольку реализация может оказаться дорогостоящей с точки зрения времени и ресурсов, рекомендуется использовать его для сложных ПО, требующих производительности и масштабируемости.


MESSAGING TIER
DATA TIER
BUSINESS LOGIC TIER
WEB TIER
WEB CLIENT
RESPONSE TIME

Количество и типы уровней могут варьироваться, но чаще всего встречаются четыре стандартных уровня:

1. ***слой представления,***
2. ***слой бизнес- логики,***
3. ***слой доступа к данным***
4. ***и слой базы данных.***



Архитектура каждого слоя состоит из множества элементов. Эти элементы представляют собой части программного кода. Такие элементы называют **компонентами**.

Компоненты слоя представления реализуют функции, необходимые для того, чтобы пользователи могли взаимодействовать с приложением. Это компоненты пользовательского интерфейса и компоненты, обрабатывающие действия пользователя.

Компоненты слоя бизнес-логики реализуют ключевые функции программы и выражают соответствующую бизнес-логику. Это компоненты бизнес-процессов, обрабатывающих данные сущностей предметной области.

Компоненты слоя доступа к данным реализуют логику извлечения, записи, редактирования и удаления данных независимо от особенностей их хранения.

Компоненты слоя базы данных реализуют функцию хранения данных и представляют собой различные СУБД, платформы управления облачными хранилищами и другие источники данных.

Для того, чтобы выполнить ожидаемую пользователем функцию запрос может последовательно проходить все слои ПО сверху вниз (от слоя представления до слоя хранилища данных).

После выполнения запроса формируется ответ. Ответ может проходить все слои ПО снизу вверх (от слоя хранилища данных до слоя представления), чтобы передать пользователю результат выполнения функции.

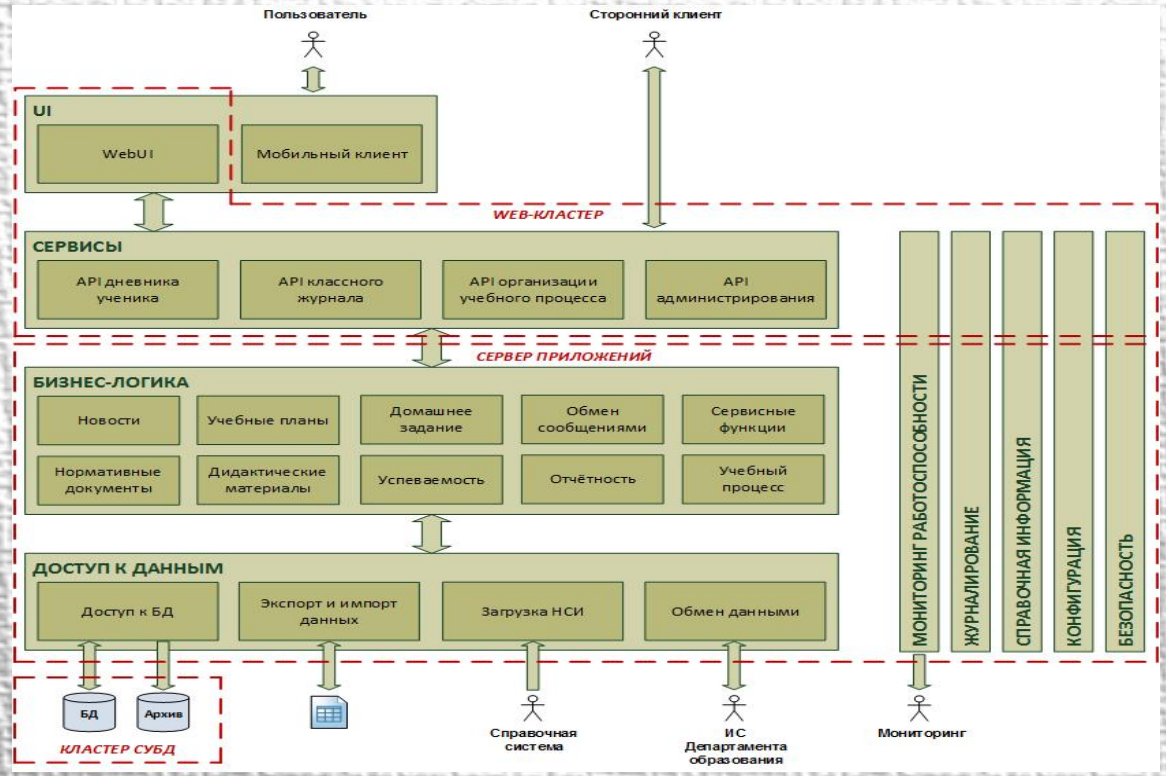
Вместе с тем, запрос и ответ могут пропускать некоторые слои ПО. Обязательность прохождения через слой ПО определяется для каждого слоя по отдельности. Любой слой может быть открытым или закрытым.

Этот стиль подходит для небольших и простых приложений или веб-сайтов, особенно, когда бюджет и время ограничены. **Благодаря своей простоте и доступности для разработчиков многоуровневая архитектура является одной из наименее затратных.**

Однако, по мере роста в домене числа приложений, построенных по этому принципу, возникают проблемы с сопровождением, гибкостью, тестированием и развертыванием. Крупные приложения и системы, использующие многоуровневую архитектуру, могут нуждаться в более модульных подходах.

➕ Плюсы: простота и скорость разработки и тестирования.

➖ Минусы: низкая производительность под высокой нагрузкой, слабая масштабируемость, сложность модификации.



ПРИМЕР РАЗРАБОТКИ ТРЕХСЛОЙНОЙ АРХИТЕКТУРЫ

Алгоритм проектирования

- Возьмите лист бумаги и карандаш. Если не сильно уверены в своих силах, то ещё и резинку, чтобы править схему. Более продвинутые читатели могут обратиться к профессиональным инструментам для проектирования архитектуры в электронном виде.
- Теперь выясните, кто будет обращаться к вашей системе, чтобы передать или забрать данные, а к чему будет обращаться Ваша программа.
  - Те системы или пользователи, которые обращаются к программе сами, нарисуйте схематически на листе бумаги вверху.
  - Те, к которым будет обращаться программа (включая БД), — снизу.

- Теперь нарисуйте под каждым нарисованным сверху субъектом прямоугольник с названием **UI** или **API** — это интерфейсы, к которым будет обращаться пользователь или внешняя управляющая система. Иногда UI тоже может обращаться к API. Объедините все прямоугольники с UI одним контуром и обзовите слоем представления. Объедините все прямоугольники с API и обзовите **слоем сервисов**.
- Для систем, нарисованных снизу, укажите компоненты, которые будут отвечать за доступ к этим системам. Объедините все эти компоненты одним контуром и обзовите **слоем доступа к данным**.

- Между слоем сервисов и слоем доступа к данным нарисуйте большой контур и назовите его **слоем бизнес-логики**. В маленьких прямоугольниках внутри этого контура перечислите основные бизнес-задачи. Один компонент Вашей системы будет решать одну бизнес-задачу.
- Теперь справа нарисуйте несколько длинных прямоугольников снизу доверху и напишите в них: *журналирование, конфигурация, мониторинг производительности, обработка исключений* и что-то ещё, что является **общей инфраструктурой** (**или сквозной функциональностью**) для всех слоёв вашей программы.

- Вы получили логическую архитектуру приложения.
- Разбросайте слои по серверам — получите физическую архитектуру.
- Теперь вам остаётся детально проработать каждый маленький квадратик.

Система контроля успеваемости для средней школы

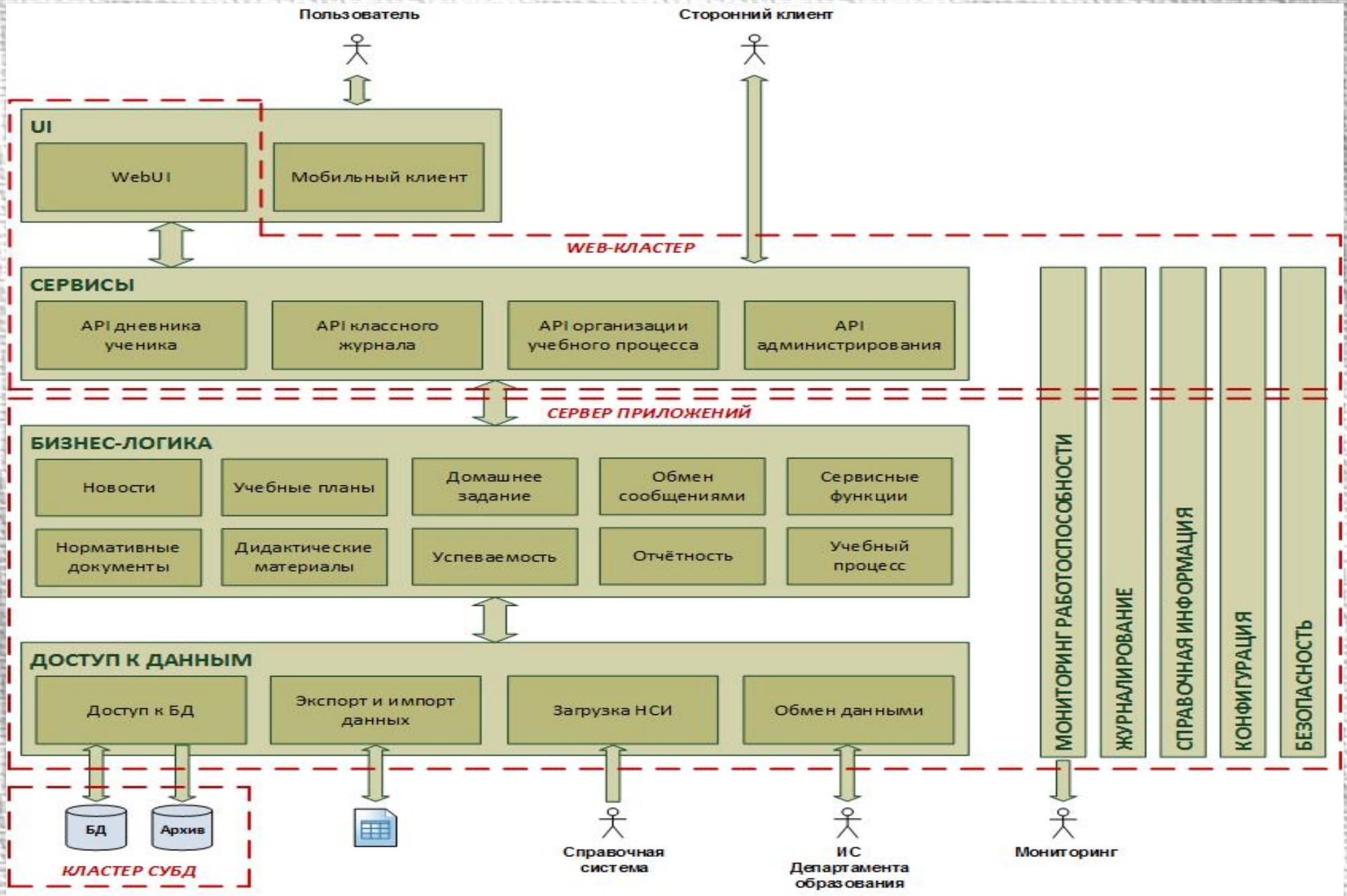
Для примера возьмем совершенно виртуальный проект системы контроля успеваемости для средней школы.

- Итак, наша система будет предназначена для ведения электронных дневников учащихся и электронных классных журналов.
- Еще пусть система имеет некоторый дополнительный набор функций, позволяющий учащимся, родителям и преподавателям обмениваться сообщениями, а руководству школы — контролировать учебный процесс.

- Пользователями нашей системы будут учащиеся, их родители, учителя и руководство школы. Кроме того, пользователями будут являться администраторы системы, которым важно получать информацию о событиях в системе и выполнять некоторые сервисные операции.
- Пользователи будут взаимодействовать с нашей системой либо посредством web-интерфейса, либо с помощью мобильного приложения. Оба пользовательских интерфейса предназначены для школ, которые не могут себе позволить разработку своего собственного сайта. Остальные смогут обращаться к нашей системе через web-сервисы.
- Сервисов будет четыре: электронный дневник, электронный журнал, организация учебного процесса и администрирование.

- Под слоем сервисов будет слой бизнес-логики, состоящий из десятка компонентов, решающих бизнес-задачи пользователей.
- Данные будут храниться в базе данных. Причём базы будет две: боевая и архивная.
- Система будет экспортировать/импортировать данные из файлов. Кроме того, будет две внешние системы, к которым наша система будет подключаться удалённо: система справочной информации и информационная система Департамента образования.
- Сквозную функциональность будут составлять механизмы журналирования, мониторинга, управления конфигурацией, обеспечения безопасности.

В итоге, получаем вот такую простую схему:



Теперь можно выделить физические узлы.

- Web-интерфейс пользователя и сервисы будут развёрнуты в web-кластере.
- Слои бизнес-логики и доступа к данным будут реализованы на сервере приложений.
- Базы данных разместятся в отдельном отказоустойчивом кластере.

Позже можно будет все узлы изобразить в виде красивой схемы физической архитектуры.

Задача архитектуры ПО — минимизация человеческих ресурсов при разработке и последующем сопровождении системы

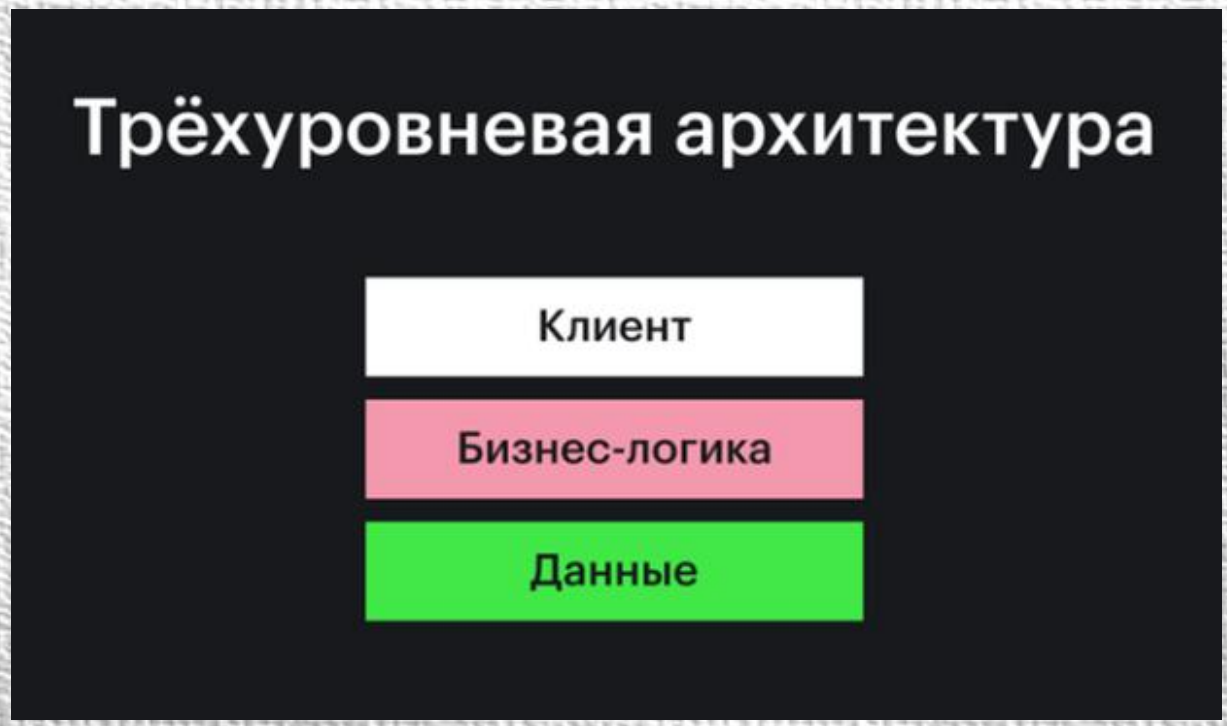
Роберт С. Мартин

ЧИСТАЯ АРХИТЕКТУРА

Трёхуровневая (трёхзвенная) архитектура

Серверное приложение обычно построено на слоях и связях между ними. Есть несколько подходов для реализации слоёв.

Трёхуровневая (трёхзвенная) архитектура — исторически один из первых подходов, который построен на совместной работе трёх слоёв: слоя клиента, слоя бизнес-логики и слоя данных. Нижний слой — слой данных. Он взаимодействует со слоем бизнес-логики, который в свою очередь взаимодействует с клиентским слоем. В трёхуровневой архитектуре клиентский слой изолирован от слоя данных.



Проблемы такого подхода очевидны. Всё приложение построено в привязке к данным. Например, структура в базе данных будет полностью определять все верхние слои, что в корне неправильно. Это не позволяет обеспечивать должный уровень абстракции. Будут проблемы, если изменится какой-то внешний API или способ получения данных от микросервиса. Развивать и поддерживать такое приложение будет крайне сложно.

Чистая архитектура — это рекомендации по организации системной архитектуры. Они были предложены Робертом С. Мартином (известным также как Дядя Боб) и основаны на ряде прежних архитектурных построений, таких как гексагональная архитектура, луковая архитектура и т. д.

Это одно из основных правил для создания адаптируемого программного обеспечения (ПО), удобного в тестировании и поддержке.

Правильно выбранная архитектура упрощает тестирование, поддержку, модификацию, разработку и развертывание, а также обеспечивает независимость.

Почему важно следовать принципам Clean Architecture

Первое, что необходимо понимать — приложения, созданные с помощью принципов чистой архитектуры, обладают высокой устойчивостью к изменениям. Они быстро адаптируются к новым требованиям и возможностям, сохраняя при этом высокое качество кода и производительность. Кроме того, система становится гораздо проще для понимания и сопровождения, что делает ее более доступной для других разработчиков.

Принципы чистой архитектуры также помогают избежать проблем с зависимостями и разделить приложение на логические блоки. Это повышает удобство добавления новых функций и изменения существующих. В результате время разработки сокращается, а приложение становится более гибким и масштабируемым.

Основная цель чистой архитектуры

Создавать системы, которые обеспечивают:

- **Удобство обслуживания**: Простота понимания и модификации, позволяющая разработчикам вносить изменения с минимальным риском внесения ошибок.
- **Тестируемость**: Разработан таким образом, чтобы тестирование было простым, что позволяет создавать автоматические тесты для различных частей системы.
- **Независимость от фреймворков**: фреймворки могут быть заменены с минимальным воздействием на систему.

- **Независимость от пользовательского интерфейса**: пользовательский интерфейс может изменяться, не затрагивая базовую логику или бизнес-правила.
- **Независимость от базы данных**: механизмы хранения и извлечения данных могут изменяться, не затрагивая систему.
- **Независимость от каких-либо внешних агентств**: бизнес-правила не привязаны к каким-либо конкретным деталям реализации, что делает систему легко адаптируемой к изменениям.

Чистая архитектура (clean architecture) подразумевает организацию проекта таким образом, чтобы в нем можно было легко разобраться, и чтобы его можно было легко изменять по мере роста проекта.

Принципы внедрения чистой архитектуры

За основу внедрения чистой архитектуры берется набор принципов SOLID. Они являются принципами на уровне класса, но имеют схожие аналоги, применимые к компонентам (группам связанных классов).

1. **Р**азделение на уровни
2. **З**ависимость от абстракций
3. **Г**раничные интерфейсы
4. **П**ринцип "Чистой зависимости" (Pure Dependency Rule)

1. Разделение на уровни

Система разделяется на различные уровни, каждый из которых отвечает за определенные аспекты функциональности. Типичная архитектура может включать уровни, такие как пользовательский интерфейс, бизнес-логика и доступ к данным. Это позволяет создавать слабосвязанные компоненты, которые могут изменять-ся независимо друг от друга.

2. Зависимость от абстракций

Компоненты системы должны зависеть от абстракций, а не от конкретных реализаций. Это достигается путем определения интерфейсов, которые описывают контракты между компонентами. Зависимость от абстракций обеспечивает гибкость и возможность замены компонентов без необходимости изменения остальной системы.

3. Граничные интерфейсы (Boundaries)

Граничные интерфейсы (*Boundary Interfaces*) — это интерфейсы, которые разделяют используемые элементы на две области: внутри приложения и вне его. Они служат для определения, какие элементы способны перейти за границу приложения и какие нет.

Граничные интерфейсы играют важную роль в Clean Architecture, так как они определяют, как пользовательский интерфейс должен общаться с приложением. Пользовательский интерфейс является внешней частью приложения, которая взаимодействует с пользователем и передает запросы в приложение. Граничные интерфейсы определяют, какие запросы пользовательский интерфейс может отправлять в приложение, и какие данные он может получать в ответ.

Примером граничного интерфейса является REST API веб-сервера, который позволяет клиентской стороне общаться с серверной стороной приложения. Если слой приложения использует граничные интерфейсы, то он был бы независим от клиентской стороны и мог бы быть более гибким при изменениях в клиентской стороне.

Интерфейс — это контракт на поведение модуля. Он описывает, какие поля и методы этого модуля могут использовать другие модули.

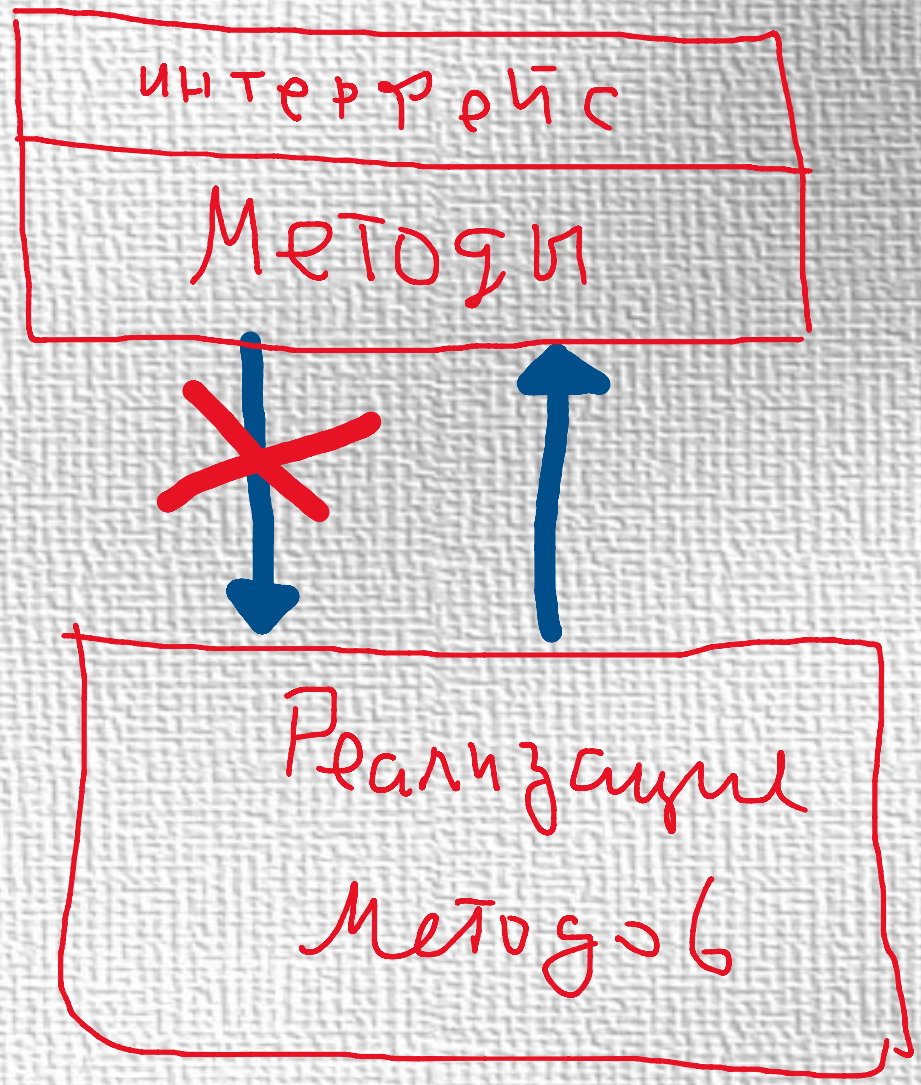
4. Принцип "Чистой зависимости" (Pure Dependency Rule)

Принцип "Чистой зависимости" (*Pure Dependency Rule*) гласит, что более высокоуровневые модули не должны зависеть от более низкоуровневых модулей. Оба модуля должны зависеть от абстракций. Абстракции не должны зависеть от деталей, а детали должны зависеть от абстракций.

Таким образом, при разработке уровней приложения, каждый уровень использует интерфейсы и абстракции, что позволяет изолировать изменения сверху. Это обеспечивает большую гибкость системы при изменениях.

- Абстракции не должны зависеть от деталей,
- а детали должны зависеть от абстракций

- **База данных** — это деталь реализации
- **Клиенты (Web, Mobile, etc)** — детали реализации
- **Фреймворки** — это деталь реализации



Например, если нам нужно изменить базу данных, то мы можем заменить конкретную базу данных на новую, не меняя остальную структуру приложения. Это было бы невозможно, если бы мы использовали конкретную реализацию базы данных во всех частях приложения.

Use case — это принцип, который определяет входы и выходы пользовательских сценариев. В основе этого принципа лежит модель, где каждый пользовательский сценарий описывается согласно интересующим его элементам.

Основные характеристики чистой архитектуры

При разработке большого проекта нам необходимо, чтобы его можно было легко поддерживать, при необходимости изменяя отдельные компоненты. Для этого нам необходимо разделять файлы и классы на компоненты, которые могут изменяться независимо друг от друга.

Clean Architecture объединила в себе идеи нескольких других архитектурных подходов, которые сходятся в том, что архитектура должна:

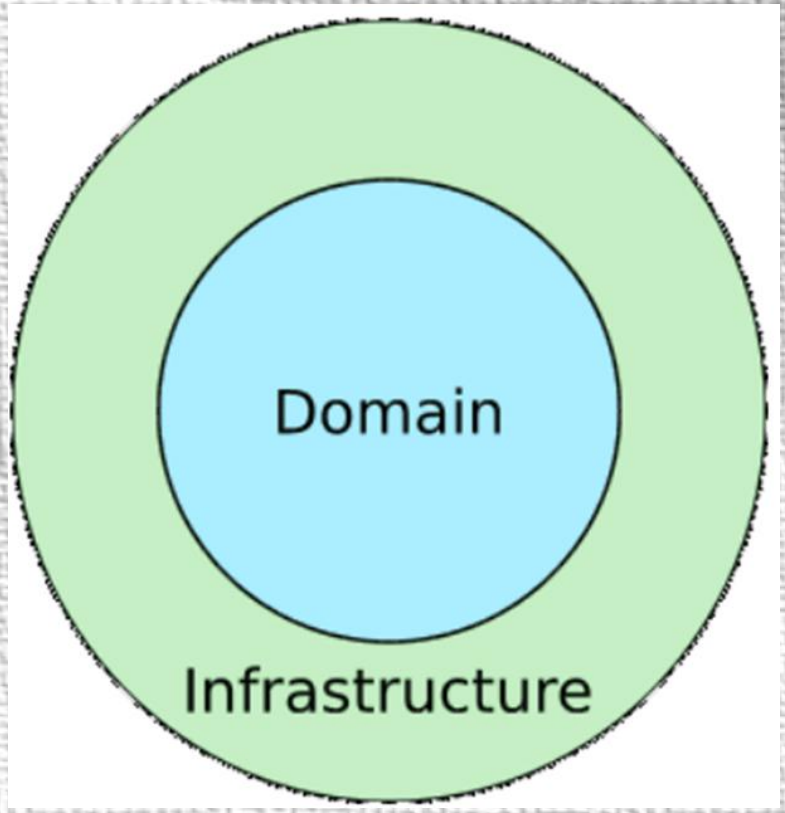
- быть тестируемой;
- не зависеть от UI;
- не зависеть от БД, внешних фреймворков и библиотек.

Это достигается разделением на слои и следованием Dependency Rule (правилу зависимостей)!

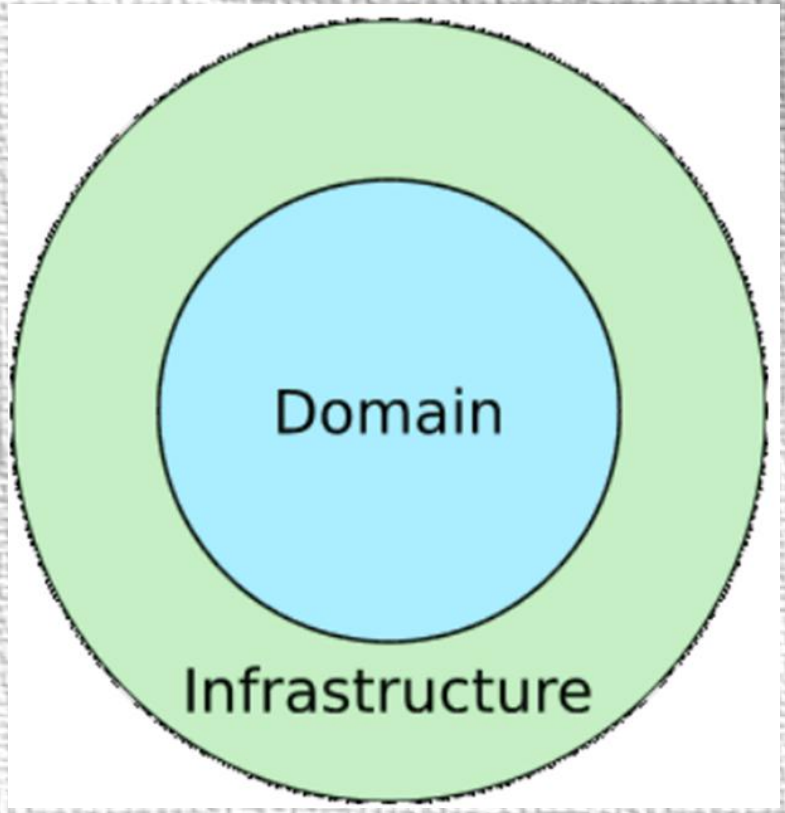
Внутренний круг - это уровень предметной области вашего приложения. Здесь вы размещаете бизнес-правила. Под "бизнесом" мы не обязательно подразумеваем компанию. **Это просто означает суть того, что делает ваше приложение, основную функциональность кода**.

- Приложение для перевода выполняет перевод.
- В интернет-магазине есть товары для продажи.

Эти бизнес-правила довольно стабильны, поскольку вы вряд ли будете часто менять суть того, что делает ваше приложение.



Внешний круг - это инфраструктура, которая обслуживает внутренний круг. Он включает в себя такие элементы, как пользовательский интерфейс, база данных, веб-API и фреймворки. Эти элементы подвержены изменениям с большей вероятностью, чем домен. Например, у вас больше шансов изменить внешний вид кнопки пользовательского интерфейса, чем способ расчета кредита.



Информационная инфраструктура

Инфраструктура - это, по сути, все то, без чего наша повседневная жизнь была бы куда менее комфортной. Представьте себе дороги, мосты, аэропорты, больницы, электрические сети и водопроводы. Это все является частью инфраструктуры.

Слово "инфраструктура" произошло от латинского "infra", что означает "ниже", и "structura", что переводится как "строение". Таким образом, это некий "основной слой" для функционирования общества. На него опирается вся наша жизнедеятельность.

Информационная инфраструктура: сети связи, цифровые платформы, базы данных и другие объекты, обеспечивающие доступ к информации и коммуникациям.

Граница между доменом и инфраструктурой устанавливается таким образом, что **домен ничего не знает об инфраструктуре**.

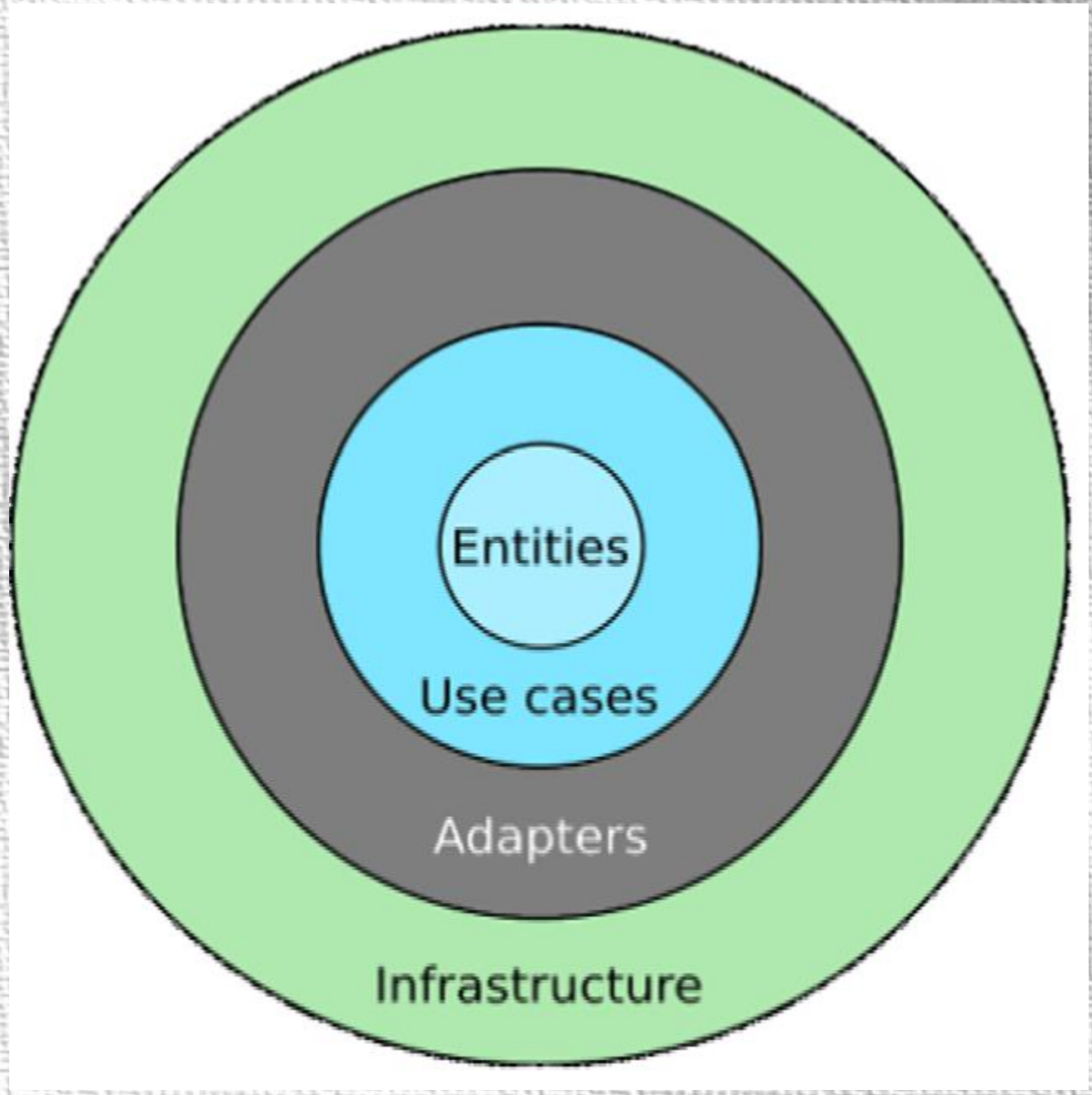
Это означает, что пользовательский интерфейс и база данных зависят от бизнес-правил, но эти правила не зависят от пользовательского интерфейса или базы данных.

Это делает его архитектурой подключаемого модуля. Не имеет значения, является ли пользовательский интерфейс веб-интерфейсом, настольным приложением или мобильным приложением. Также не имеет значения, хранятся ли данные с использованием SQL, NoSQL или в облаке. Домену это все равно. Это упрощает изменение инфраструктуры.

На изображении выше мы представили Clean architecture в виде двух кругов. Однако, для лучшего понимания принципов чистой архитектуры лучше выполнить небольшую декомпозицию.

Здесь мы подразделяем уровень домена на сущности и различные варианты использования, а уровень адаптера образует границу между доменом и уровнем инфраструктуры.

Так как эти термины могут немного сбивать с толку, давайте рассмотрим их по отдельности.



Сущности и варианты использования

Любое приложение существует в особом предметном мире, который создаёт разработчик. Ключевые элементы этого мира мы получаем от бизнес-аналитиков. Их задача - дать наиболее полную информацию об этом мире. Наша задача - спроектировать этот мир через **бизнес-сущности**.

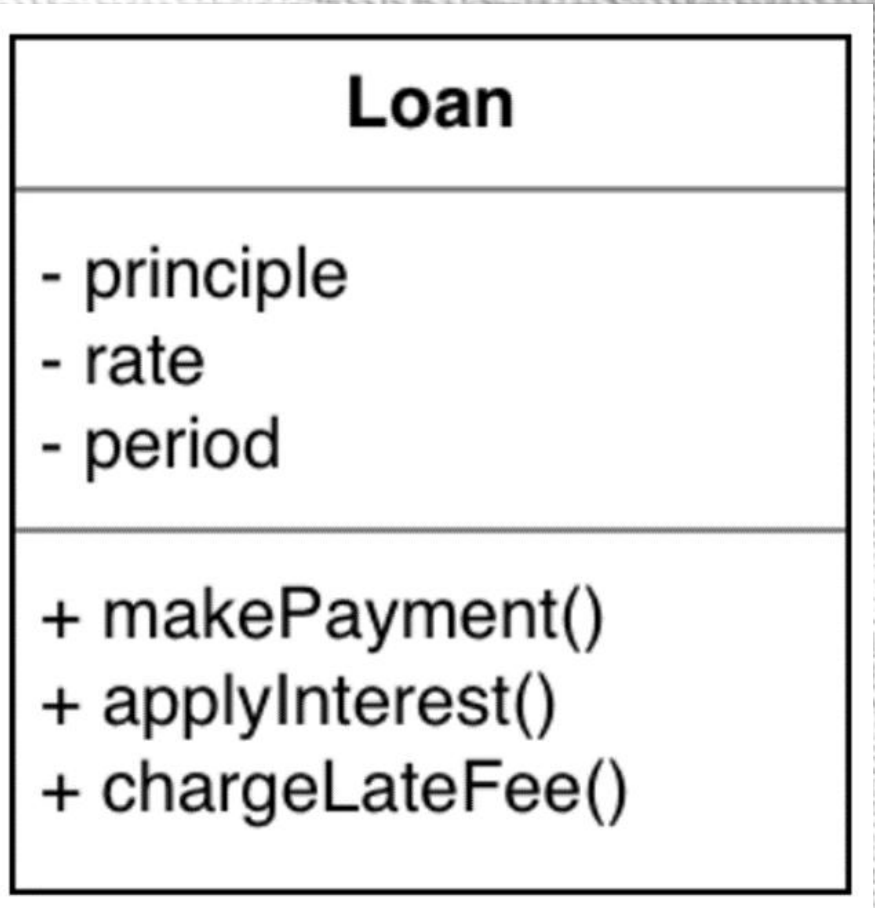
① Сущности

Сущность (entity) - это набор атрибутов и связанных бизнес-правил, которые имеют решающее значение для функционирования приложения. В объектно-ориентированном языке программирования правила для сущности были бы сгруппированы как методы в классе.

Эти правила логики функционирования проекта являются **общими** для всех приложений, создаваемых на основе этой предметной области.

Вальс танцуют одинаково во всех странах. Правила общие.

Даже если бы не было приложения, эти правила все равно существовали бы. Например, взимание процентов по кредиту в размере 10% - это правило, которое может быть введено банком. Это было бы справедливо независимо от того, рассчитывались ли проценты на бумаге или с помощью компьютера.



Бизнес-сущности ничего не умеют делать. Их поведение ограничивается обслуживанием самих себя. Они не знают ни про что, кроме как про себя и про другие такие же бизнес-сущности. Они полностью беспомощны. Они просто существуют и всё. Их единственная задача - наполнять своим существованием предметную область приложения.

При этом, важно понимать, что сущности ничего не знают о других слоях и они ни от чего не зависят. **То есть, они не используют имена каких-либо других классов или компонентов, которые находятся во внешних слоях**.

② Варианты использования

Варианты использования (use cases) - это бизнес-правила для **конкретного** приложения. Они рассказывают, как автоматизировать систему, и с их помощью определяется поведение приложения. Они взаимодействуют с объектами и зависят от них, но они ничего не знают о других уровнях. Им все равно, веб-страница это или приложение для iPhone, и также, им все равно, хранятся ли данные в облаке или в локальной базе данных SQLite.

Этот уровень определяет интерфейсы или содержит абстрактные классы, которые могут использовать внешние уровни.

Бизнес-правила = бизнес-логика.

③ Адаптеры

Адаптеры, также называемые интерфейсными адаптерами, являются трансляторами между доменом и инфраструктурой.

- Например, они берут входные данные из графического интерфейса пользователя и переупаковывают их в форму, удобную для вариантов использования и сущностей.
- Затем они берут выходные данные и переупаковывают их в форму, удобную для отображения в графическом интерфейсе или сохранения в базе данных.

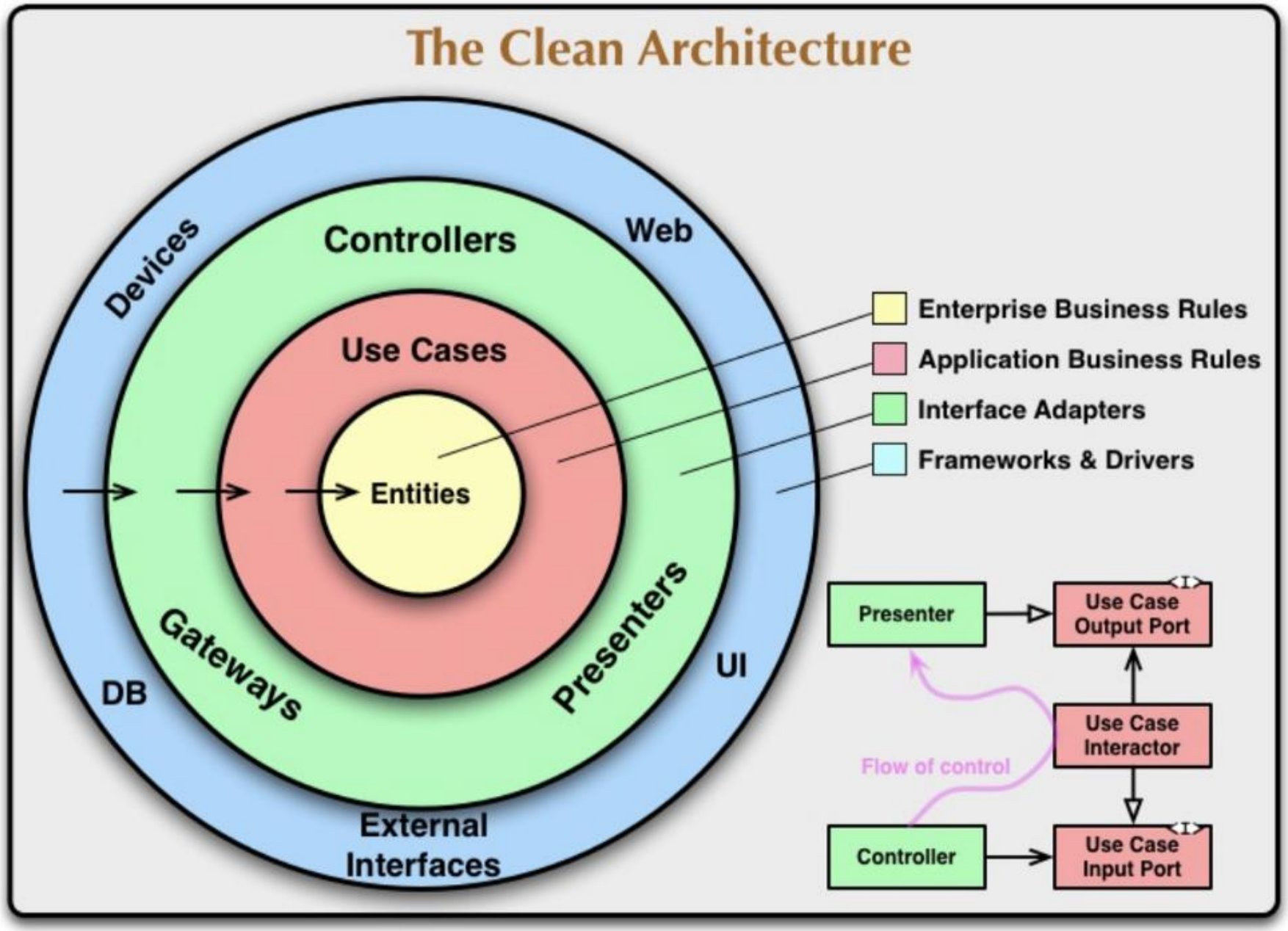
④ Инфраструктура

На этом уровне находятся все компоненты ввода-вывода: пользовательский интерфейс, база данных, фреймворки, устройства и т.д. Это самый нестабильный уровень. Поскольку на этом уровне очень часто происходят изменения, они находятся как можно дальше от более стабильных уровней домена. Поскольку они хранятся отдельно, относительно легко вносить изменения или заменять один компонент другим.

Разберем подробнее состав и работу всех слоев

Схематично стек архитектуры представлен четырьмя уровнями: **синий**, **зеленый**, **красный** и **желтый**.

Каждая окружность соответствует различным составляющим ПО. Внешний уровень ПО является самым низким. К центру уровень повышается. В целом, чем ближе слой к центру, тем он менее подвержен изменениям.



Слои

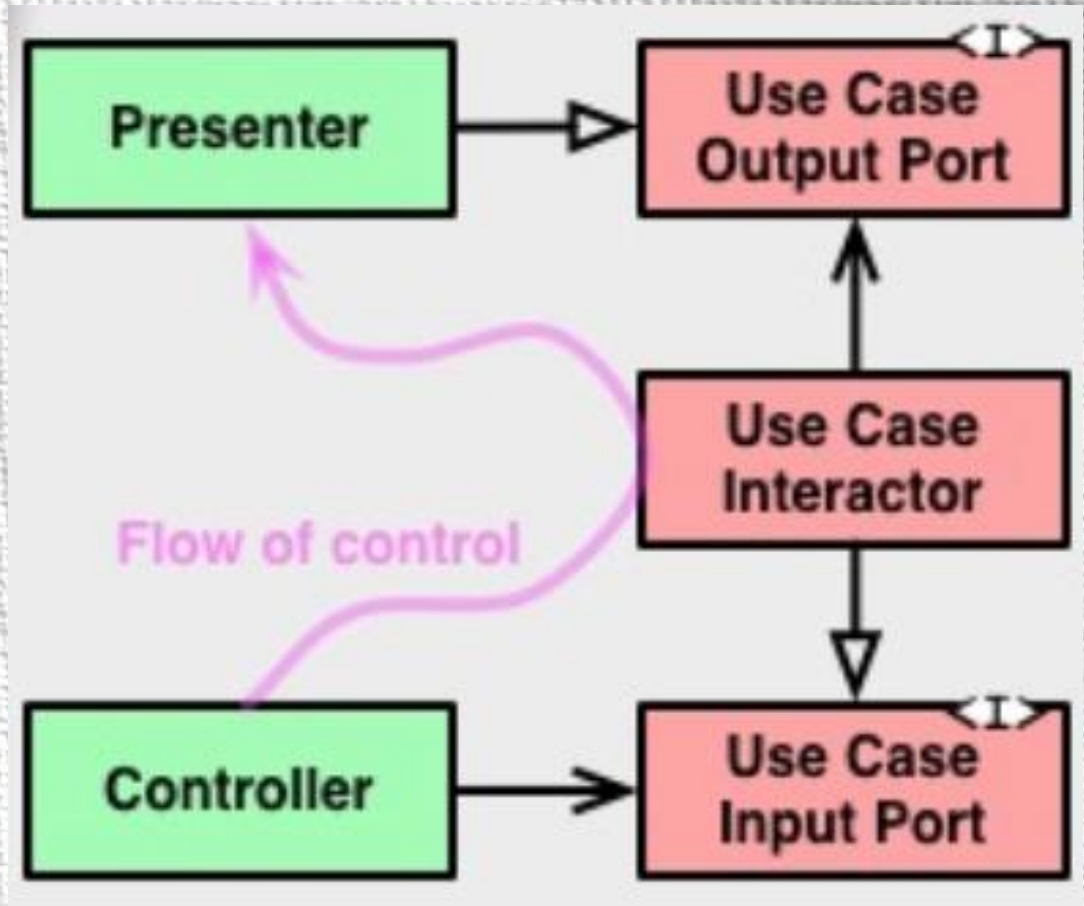
Uncle Bob выделяет 4 слоя:

1) **Entities**. Бизнес-логика общая для многих приложений.
2) **Use Cases** (Interactors). Логика приложения.
3) **Interface Adapters**. Адаптеры между Use Cases и внешним миром. Сюда попадают Presenter'ы из MVP, а также Gateways (более популярное название репозитория).
4) **Frameworks**. Самый внешний слой, тут лежит все остальное: UI, база данных, http-клиент, и т.п.

Переходы между слоями

Переходы между слоями осуществляются через ***Boundaries***, то есть через два интерфейса: один для запроса и один для ответа. Их можно увидеть справа на оригинальной схеме (Input/OutputPort).

Они нужны, чтобы внутренний слой не зависел от внешнего (следуя *Dependency Rule*), но при этом мог передать ему данные.



Оба интерфейса относятся к внутреннему слою (обратите внимание на их цвет на картинке).

Смотрите, *Controller* вызывает метод у *InputPort*, его реализует *UseCase*, а затем *UseCase* отдает ответ интерфейсу *OutputPort*, который реализует *Presenter*. То есть данные пересекли границу между слоями, но при этом все зависимости указывают внутрь на слой *UseCase*'ов.

Чтобы зависимость была направлена в сторону обратную потоку данных, применяется принцип инверсии зависимостей (буква D из аббревиатуры SOLID). То есть, вместо того чтобы *UseCase* напрямую зависел от *Presenter'a* (что нарушало бы **Dependency Rule**), он зависит от интерфейса в своём слое, а *Presenter* должен этот интерфейс реализовать.

Точно та же схема работает и в других местах, например, при обращении *UseCase* к *Gateway/Repository*. Чтобы не зависеть от репозитория, выделяется интерфейс и кладется в слой *UseCases*.

Что же касается данных, которые пересекают границы, то это должны быть простые структуры. Они могут передаваться как DTO или быть завернуты в *HashMap*, или просто быть аргументами при вызове метода. Но они обязательно должны быть в форме более удобной для внутреннего слоя (лежать во внутреннем слое).

❶ **Entities**. Правила логики функционирования проекта (**Enterprise Business Rules**)

Внутренний круг - это уровень предметной области вашего приложения. Здесь вы размещаете бизнес-правила. Под "бизнесом" мы необязательно подразумеваем компанию. Это просто означает суть того, что делает ваше приложение, основную функциональность кода. Приложение для перевода выполняет перевод. В интернет-магазине есть товары для продажи. Эти бизнес-правила, довольно стабильны, поскольку вы вряд ли будете часто менять суть того, что делает ваше приложение.

Это наименее подверженный изменениям уровень.

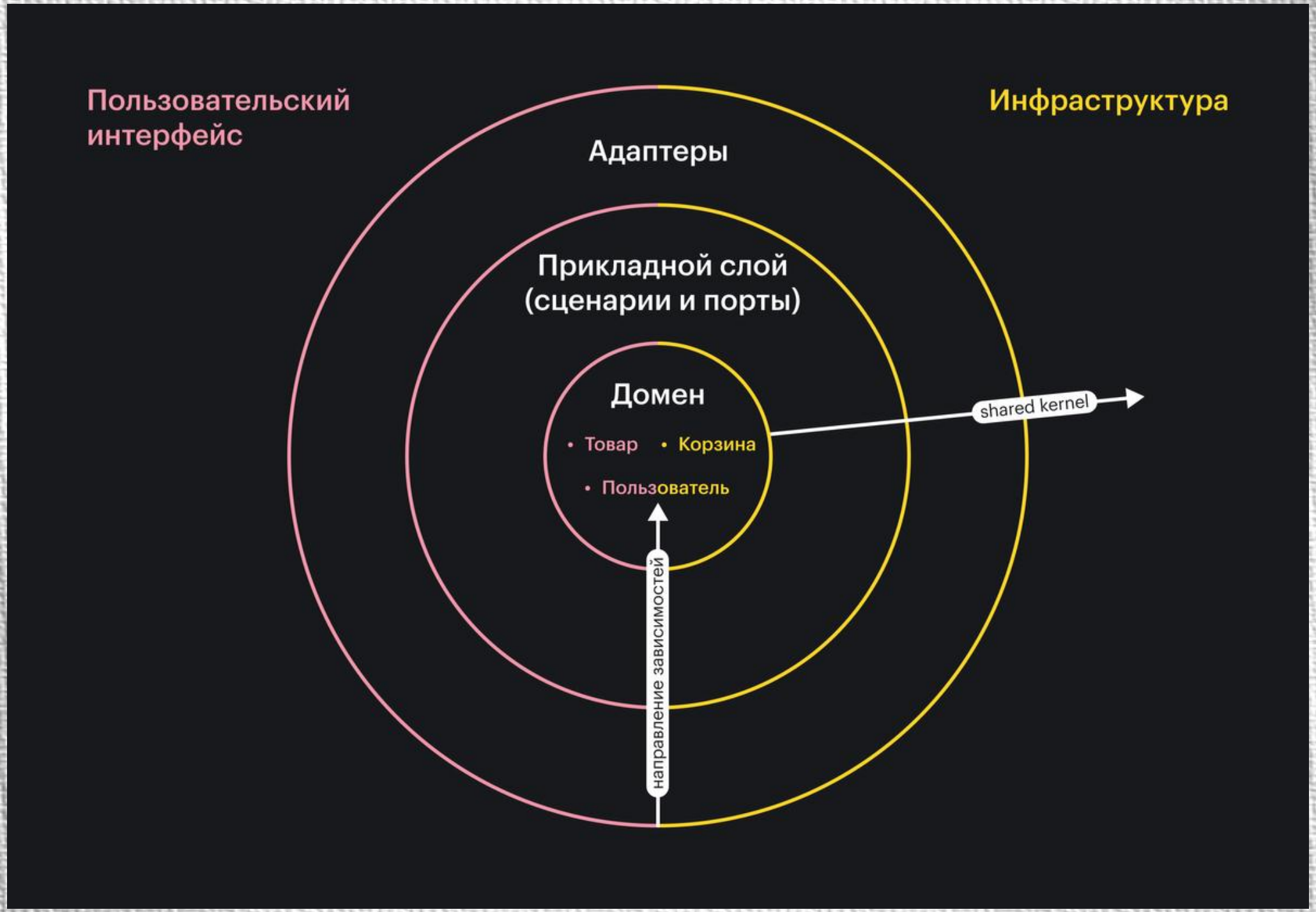
На него не влияют изменения на любом внешнем (нижнем) уровне. Поскольку логика функционирования (*Business Rules*) меняться будет не часто, изменения на этом уровне происходят очень редко. Этот уровень включает *Entities*.

***Entity* (логический объект) может быть либо основной структурой данных, необходимой для правил логики функционирования, либо объектом с методами, содержащими логику функционирования.**

Например, модуль *Interest* (вычисления процентов) в банковском приложении — это основная логика функционирования, которая должна находиться на этом уровне.

Например, в интернет-магазине в домен могут попасть сущности пользователя, товара, корзины и заказа, код для создания нового пользователя или заказа, функции подсчёта итоговой цены корзины и скидки на товары, функция для добавления и удаления товара из корзины — это всё домен.

А вот код для общения с базой данных или для отрисовки интерфейса, например, к домену не относится. Он не попадает в предметную область приложения, он скорее её «обслуживает», чтобы преобразованные доменом данные можно было сохранить в БД или чтобы их увидели пользователи.


Пользовательский интерфейс
Инфраструктура
Адаптеры
Прикладной слой (сценарии и порты)
Домен
• Товар
• Корзина
• Пользователь
shared kernel
направление зависимостей

На схеме домен расположен в центре, потому что именно он определяет, как будет организован остальной код приложения.

Домен определяет:

- **В каком виде ему необходимо передать данные из внешнего мира.**
- **С помощью каких методов и функций к нему можно обращаться.**
- **В каком виде он предоставит результат.**

Можно сказать, что весь остальной код *служит* домену и удовлетворяет его потребности.

В JavaScript данные домена, как правило, представляются в виде структур: **объектов, массивов, встроенных примитивов.**

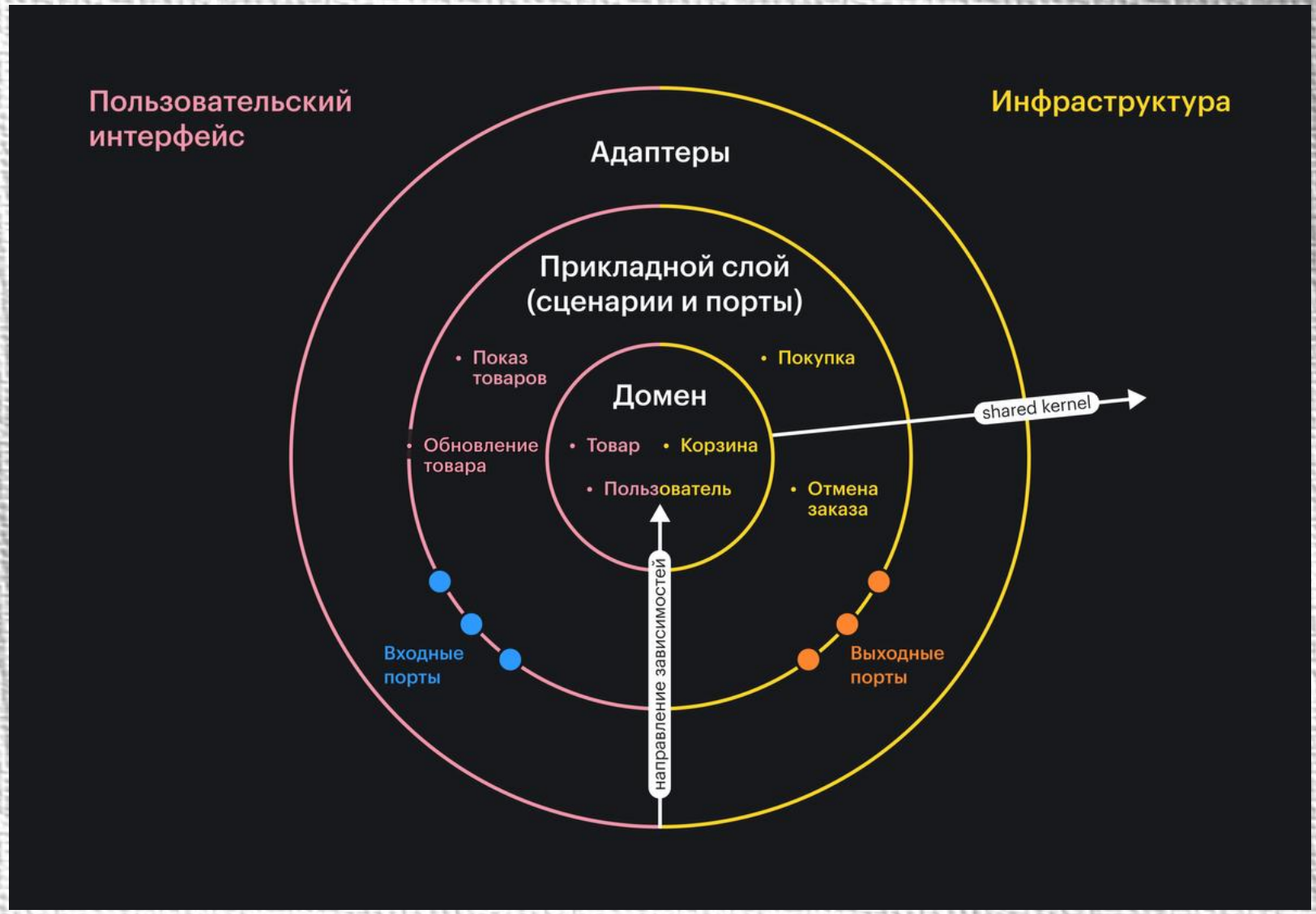
❷ UC- Application Business Rules
(Правила логики функционирования приложения)

Вокруг домена находится **прикладной слой**. Он содержит код сценариев вариантов использования приложения.

Обычно это обработчики команд, которые выполняют пользовательский сценарий. Ещё слой содержит интерфейсы портов и адаптеров.

Интерфейс — это контракт на поведение модуля. Он описывает, какие поля и методы этого модуля могут использовать другие модули.

В случае с нашим магазином в прикладном слое было бы описано, как проходит процесс покупки и оплаты, какие заказы может видеть пользователь, что может видеть продавец или администратор магазина. А также здесь находились бы интерфейсы модулей, которые получают данные от внешних сервисов (но не сами эти модули).



Как правило, код прикладного слоя по большей части состоит из:

- получения нужных данных;
- вызова доменного кода, чтобы преобразовать данные в соответствии со сценарием;
- сохранения или отправки результата.

Прикладной слой реализует функциональную архитектуру, где получением и отправкой данных занимаются функции с сайд-эффектами, а преобразованием данных занимаются чистые функции.

Чем больше логики написано на чистых функциях, тем проще будет тестировать код и отлаживать его.

Разделение на слои побуждает оставлять в домене функции, которые занимаются только преобразованием данных. Они ничего не хранят, ничего сами не запрашивают — они зависят лишь от аргументов. Из-за этого их проще тестировать и изменять.

Прикладной слой может выполнять роль «нечистого» контекста, который заботится о том, чтобы передать доменному слою данные в нужном ему виде и обработать результат.

Важно отметить, что прикладной слой никогда не вызывает внешние сервисы напрямую. Также он не содержит кода портов и адаптеров, а только их интерфейсы — то есть прикладной слой диктует спецификации, контракты, по которым с нашим приложением должны работать внешние сервисы.

Реализация этих интерфейсов находится в наружном слое — слое портов и адаптеров.

❸ Слой портов и адаптеров (Interface Adapters)

Этот слой содержит код для связи приложения с внешним миром.

Порт — это спецификация, как сторонний сервис может общаться с нашим приложением, или как наше приложение хочет, чтобы с ним общались сторонние сервисы.

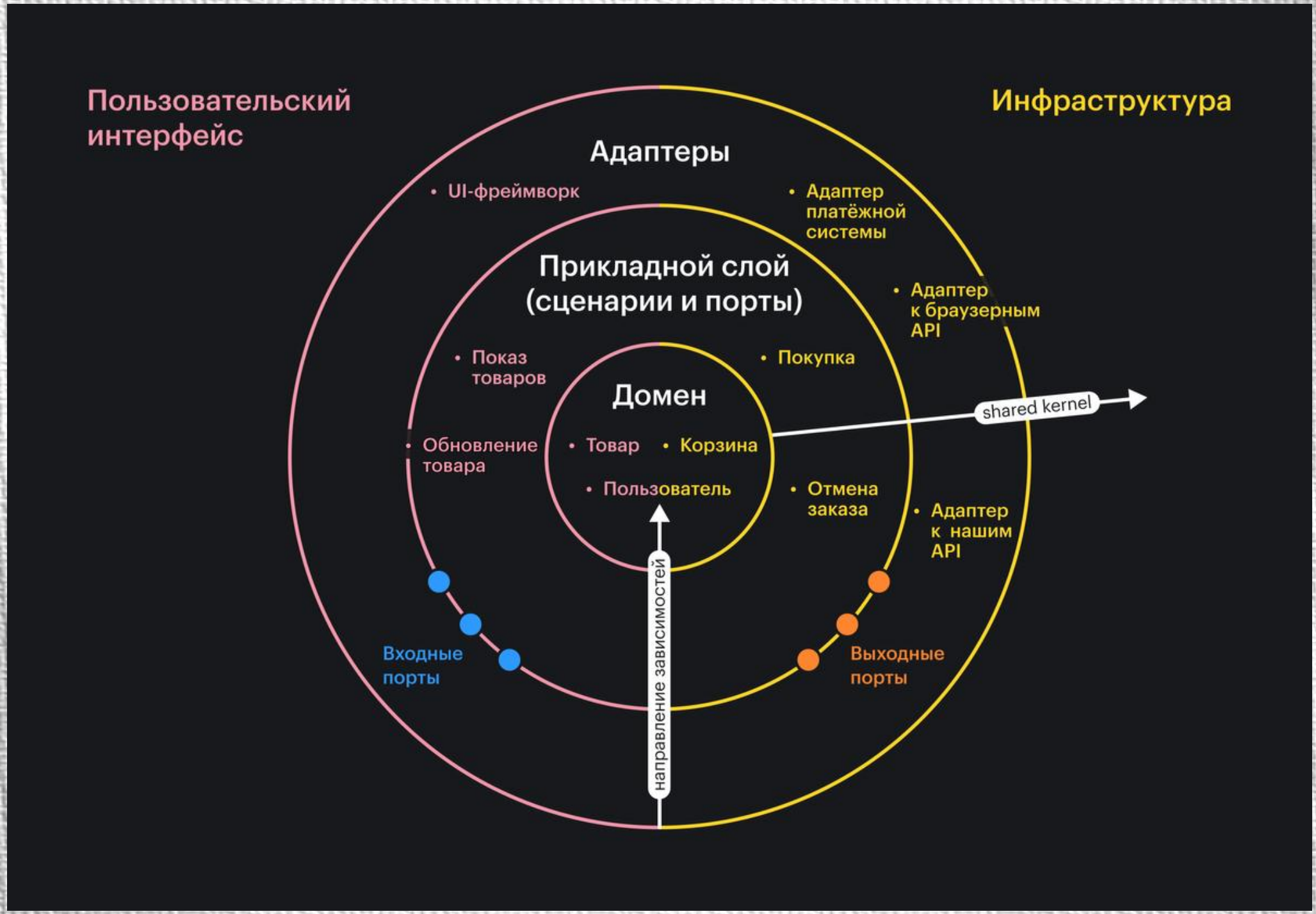
Адаптер — переходник, который делает несовместимый интерфейс внешнего сервиса совместимым с тем, который требует наше приложение.

Если мы пишем какой-то консольный сервис или сервер API, то входными портами в этих случаях будут соответственно консоль и эндпоинты API.

✏️ Адаптеры делают внешний мир совместимым с нашим приложением, порты делают наше приложение доступным внешнему миру.

Во фронтенде **входной** порт — чаще всего интерфейс пользователя. Обработка пользовательских событий, перерисовка экрана — это всё задачи как раз для слоя адаптеров и портов.

Выходными портами можно назвать те интерфейсы, которые общаются, например, с бэкенд-сервером.


Пользовательский интерфейс
Инфраструктура
Адаптеры
• UI-фреймворк
• Адаптер платёжной системы
Прикладной слой (сценарии и порты)
• Адаптер к браузерным API
• Показ товаров
• Покупка
Домен
shared kernel
• Обновление товара
• Товар
• Корзина
• Пользователь
• Отмена заказа
• Адаптер к нашим API
направление зависимостей
Входные порты
Выходные порты

Слой интерфейсных адаптеров включает:

- **Presenters** (логика и состояния пользовательского интерфейса);
- **Controllers** (интерфейс с необходимыми приложению методами, которые реализованы через
  - интернет (Web),
  - устройства (Devices) или
  - внешние интерфейсы (External Interfaces);
- **Gateways** (интерфейс, включающий все операции типа CRUD (create, read, update, delete), выполняемые приложением с базой данных).

❹ Инфраструктура и Shared Kernel (Frameworks and Drivers)

Кроме слоёв в концепции трёхслойной архитектуры ещё можно встретить понятия инфраструктуры и «общего ядра» (shared kernel).

Под инфраструктурой понимают код, который соединяет приложение с базой данных, внешними сервисами типа отправки SMS, рассылки писем и т. п. Во фронтенде чаще всего инфраструктура — это бэкенд-сервер и сторонние API типа систем оплаты.

Shared Kernel — это код, который доступен любому модулю, но зависимость от которого не повышает их зацепление. В фронтенде к shared kernel можно отнести, например, глобально доступные объекты типа window. В каких-то случаях к нему относят и сам язык программирования.

Иногда shared kernel и слои могут переходить друг в друга — это зависит от глубины проектирования. Если код, например, запускается и в браузере, и на сервере, то от глобальных объектов зависеть напрямую уже нельзя, и надо использовать адаптеры.

В этом слое размещены программные компоненты:

- User Interface (пользовательский интерфейс);
- Database (база данных);
- External Interfaces (внешние интерфейсы, например платформа API);
- Web (например, сетевой запрос);
- Devices (устройства, например принтеры и сканеры).

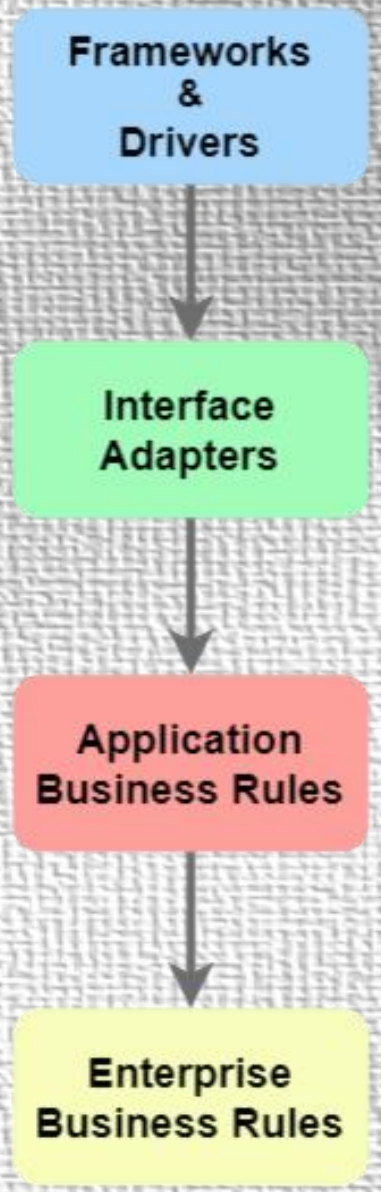
Правило зависимостей

Согласно этому правилу, **зависимости исходного кода могут быть направлены только внутрь круговой схемы**.

Это означает, что компоненты из внутренней окружности могут вообще ничего не знать о компонентах из внешней, то есть внутренняя окружность никак не зависит от внешней.

Направление черных стрелок на схеме соответствует этому принципу.

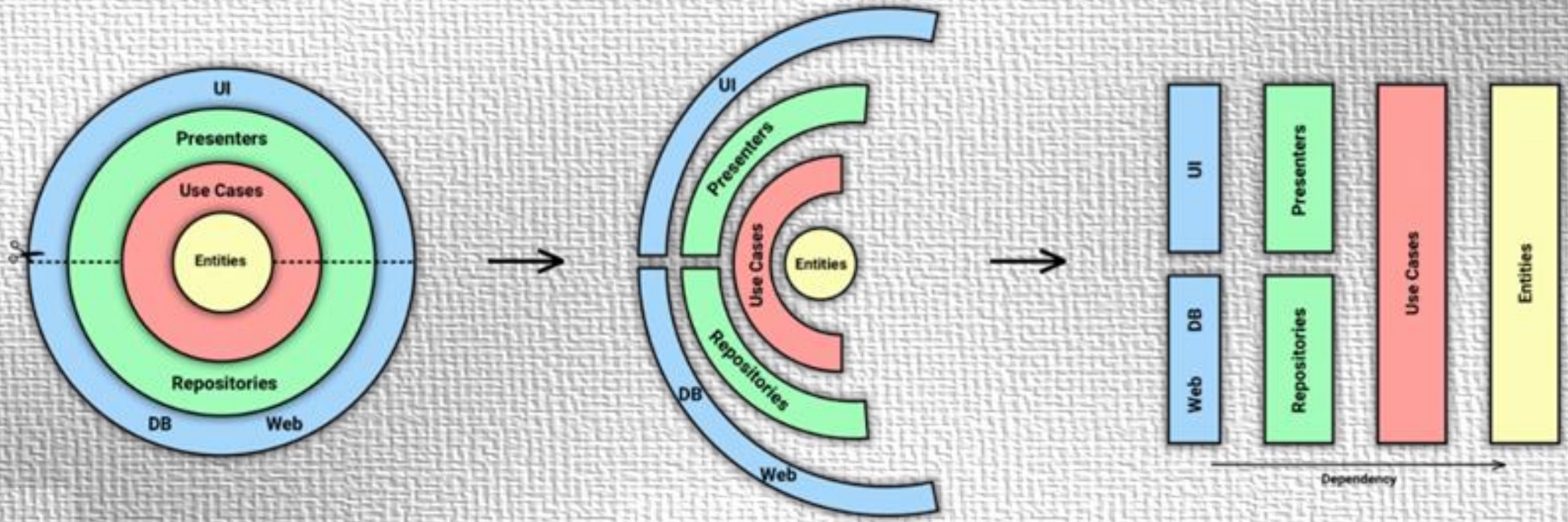
Это важное, но не всегда понятное правило архитектуры. Чтобы лучше понять имеющиеся проблемы, сначала нарушим правило, а затем разберемся как нужно его соблюдать.

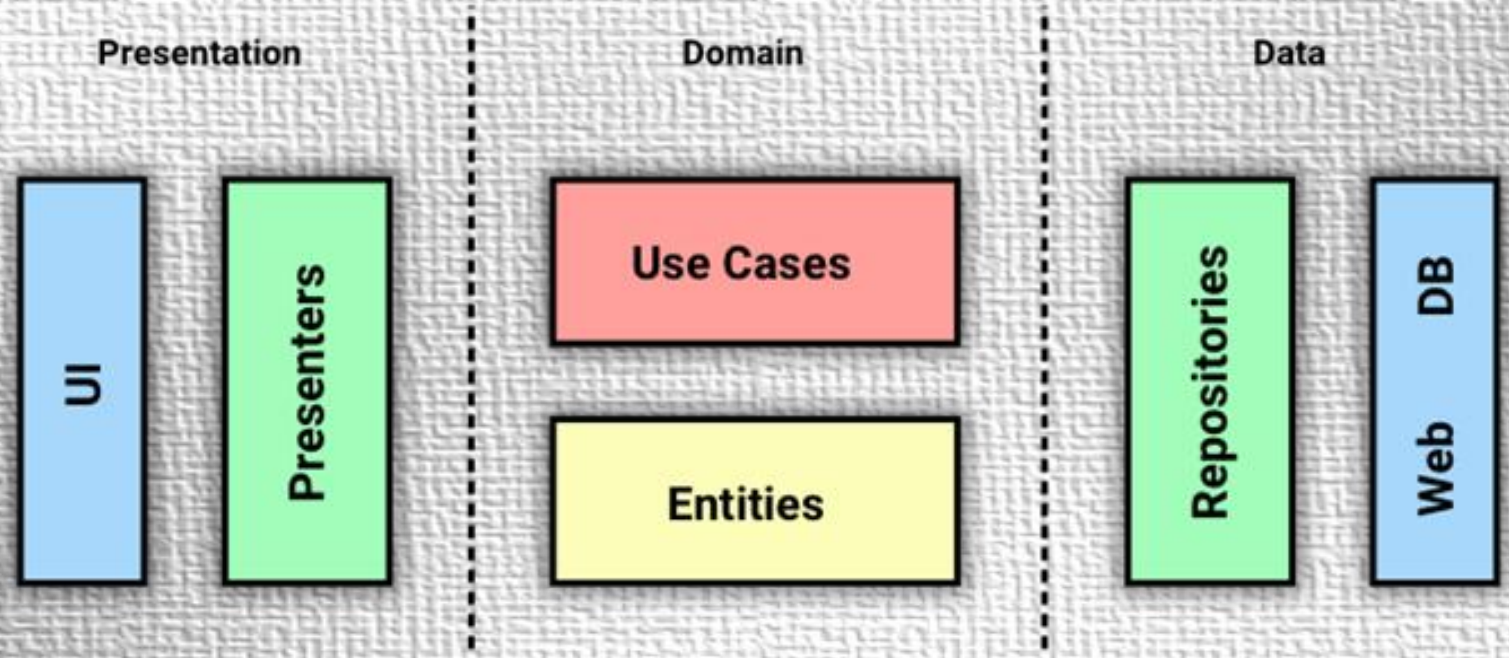


Помните, что стрелка должна читаться как “зависит от”!!!

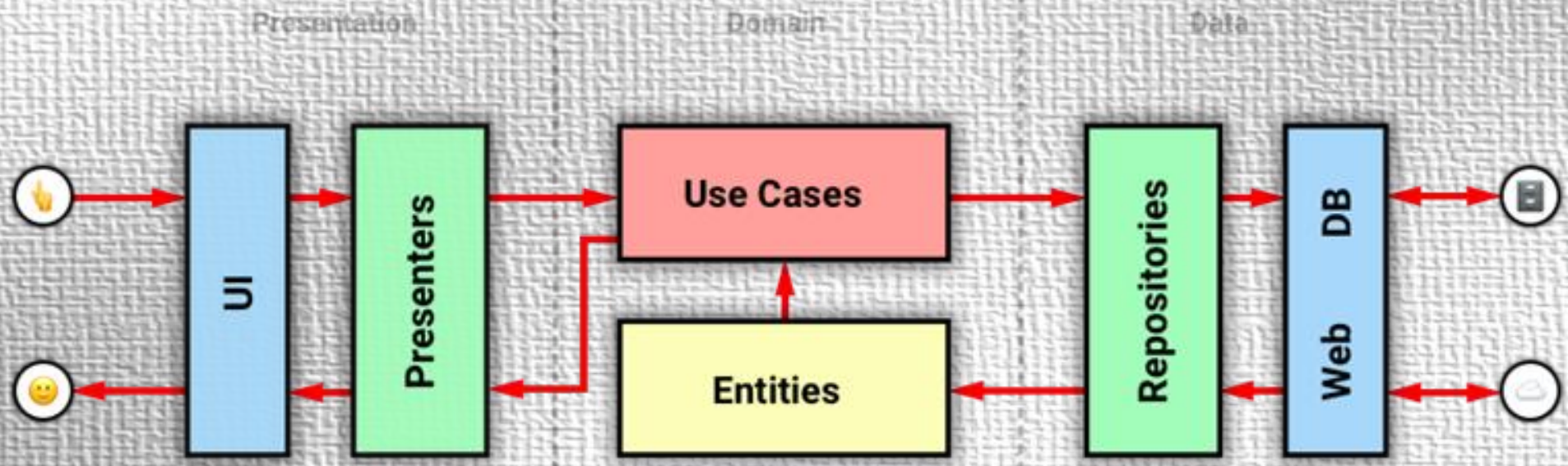
То есть Frameworks and Drivers должны зависеть от Interface Adapters, те в свою очередь зависят от Application Business Rules, а они зависят от Enterprise Business Rules.

Ни один нижний слой не должен зависеть от верхнего.


UI
Presenters
Use Cases
Entities
Repositories
DB
Web
UI
Presenters
Use Cases
Entities
Repositories
DB
Web
UI
DB
Web
Presenters
Repositories
Use Cases
Entities
Dependency

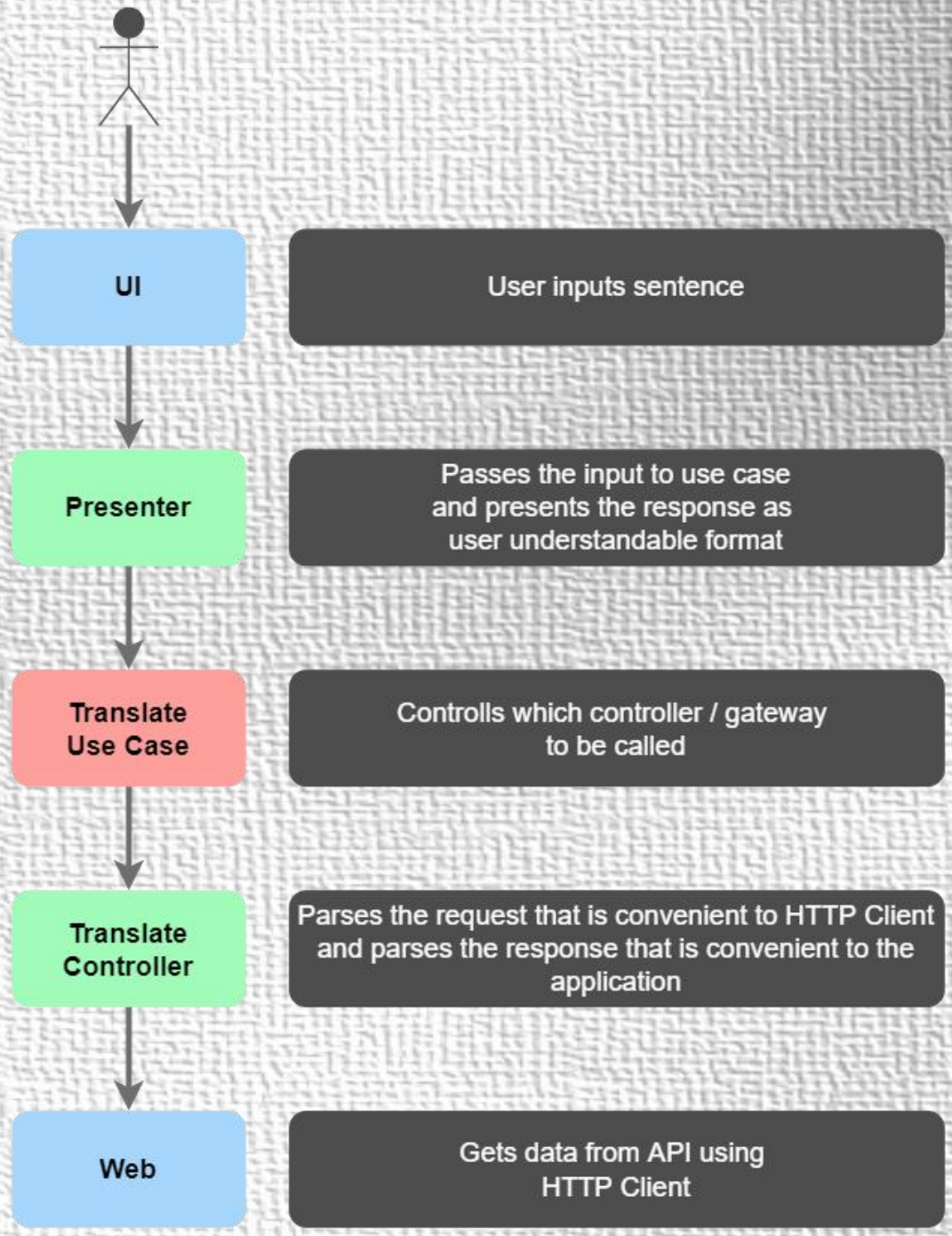

Presentation
Domain
Data
UI
Presenters
Use Cases
Entities
Repositories
Web
DB

Такая схема воспринимается проще. Ведь обычно события и данные в наших приложениях ходят от UI к backend'у или базе данных и обратно. Давайте изобразим этот процесс:



- Событие пользователя идет в Presenter, тот передает в Use Case. Use Case делает запрос в Repository. Repository получает данные где-то, создает Entity, передает его в UseCase. Так Use Case получает все нужные ему Entity. Затем, применив их и свою логику, получает результат, который передает обратно в Presenter. А тот, в свою очередь, отображает результат в UI.
- На переходах между слоями (не категориями, а слоями, отмеченными разным цветом) используются Boundaries, описанные ранее.

Разберемся на примере. Для этого создадим простое приложение с одним сетевым запросом. Оно выполняет перевод заданного пользователем предложения с помощью специального API . Посмотрим, как это можно сделать.



На каждом уровне выполняется определенная задача. Но все ли здесь логично? Проконтролируем направление зависимостей для этой архитектуры (структуры) в соответствии с правилом: “Зависимости в исходном коде должны иметь только внутренние направления”.

Какие преимущества могут дать принципы Clean Architecture

- Улучшение качества кода;
- Быстрая адаптация к изменениям и новым функциям;
- Удобство сопровождения и расширения приложений;
- Минимизация проблем с зависимостями;
- Повышение гибкости и масштабируемости приложений;
- Сокращение времени разработки и улучшение производительности.

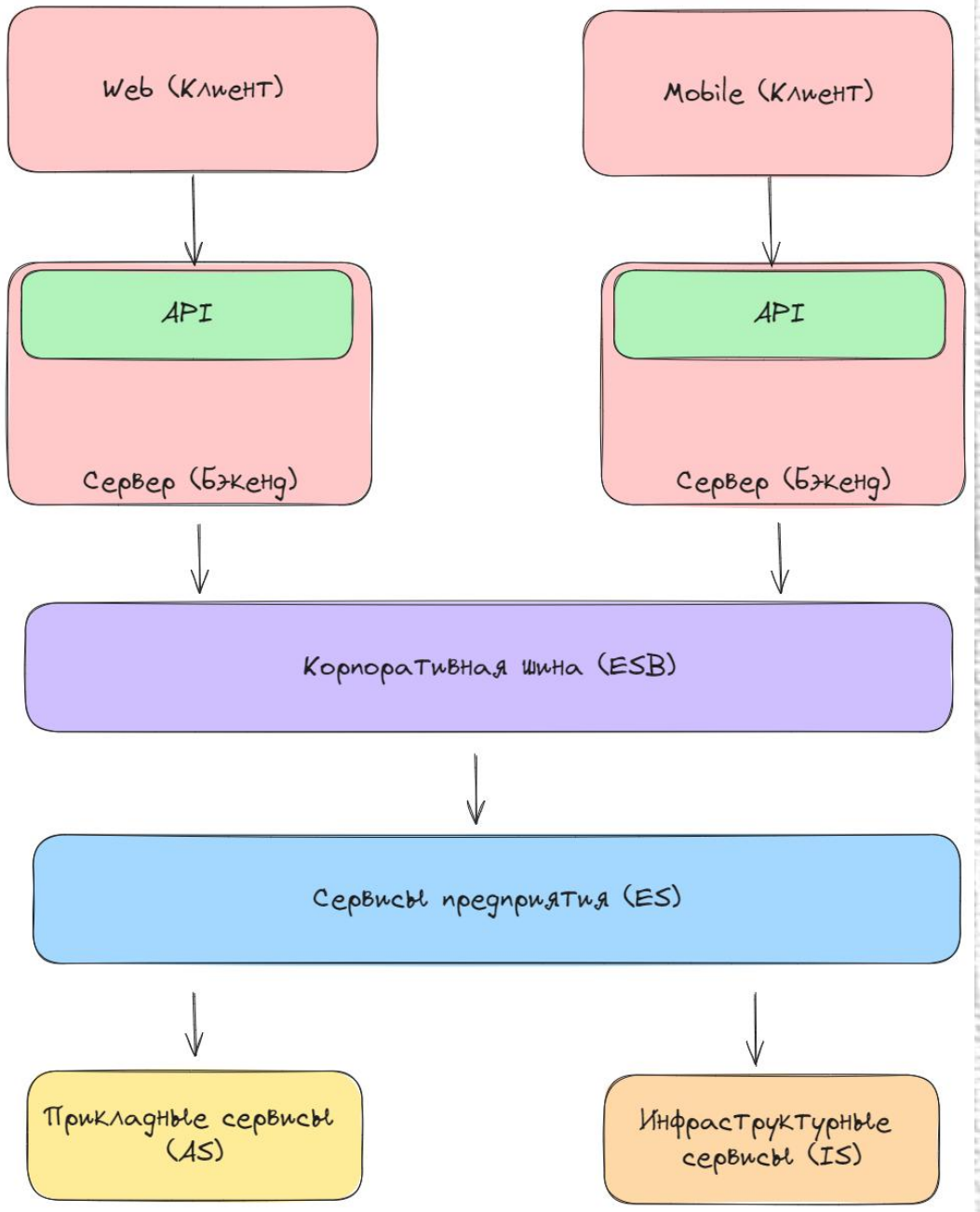
Все эти пункты дают принципам чистой архитектуры важное преимущество в сравнении с другими методиками разработки.

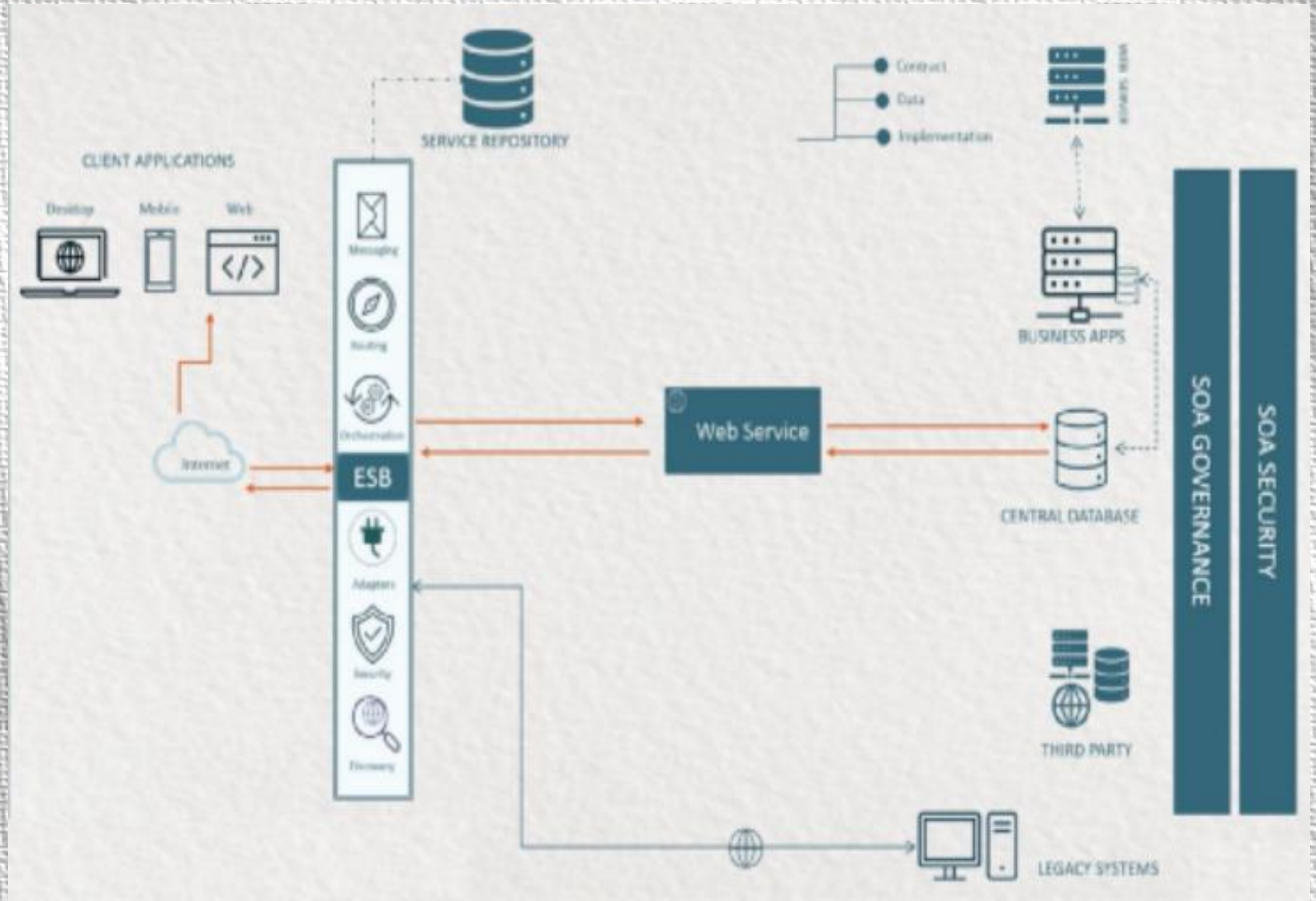
СЕРВИС-ОРИЕНТИРОВАННАЯ АРХИТЕКТУРА (SOA)

Сервис-ориентированная архитектура (SOA) является архитектурой уровня предприятия, а не отдельной системы (приложения), и базируется на двух составляющих:

1. **Сервисы**
2. **Интеграционная шина** (ESB — Enterprise Service Bus) для обмена сообщениями между ними.

Сервис-ориентированная архитектура (SOA)




SERVICE REPOSITORY
Contract
Data
Implementation
CLIENT APPLICATIONS
Desktop
Mobile
Web
Messaging
Routing
Orchestration
Internet
ESB
Adapters
Security
Discovery
Web Service
BUSINESS APPS
CENTRAL DATABASE
THIRD PARTY
LEGACY SYSTEMS
SOA GOVERNANCE
SOA SECURITY

Се́рвис-ориенти́рованная архитекту́ра (СОА, англ. service-oriented architecture- SOA) — модульный подход к разработке программного обеспечения, базирующийся на обеспечении удаленного по стандартизированным протоколам использования распределённых, слабо связанных, легко заменяемых компонентов (сервисов) со стандартизированными интерфейсами.

Программные комплексы, разработанные в соответствии с СОА, обычно реализуются как набор веб-служб, взаимодействующих по протоколу SOAP, но существуют и другие реализации (например, на базе jini, CORBA, на основе REST).

Главное, что отличает SOA — это использование независимых сервисов, выполняющих точно определённые задачи, вызываемые стандартным способом через точно определённые интерфейсы, при отсутствии у сервисов знаний о вызывающих их приложениях, а у приложений — способов, которыми сервисы выполняют свою задачу.

SOA также может рассматриваться как стиль архитектуры информационных систем, который позволяет создавать приложения, построенные путём комбинации слабосвязанных взаимодействующих сервисов. Эти сервисы взаимодействуют на основе строго определённого платформенно- и языково-независимого интерфейса (например, WSDL). Определение интерфейса скрывает языковую реализацию сервиса.

Таким образом, системы, основанные на SOA, могут быть независимы от технологий разработки и платформ (таких как Java, .NET и т. д.). К примеру, сервисы, написанные на C#, работающие на платформах .Net и сервисы на Java, работающие на платформах Java EE, могут быть одинаковым образом вызваны общим составным приложением. Обеспеченная таким образом возможность приложениям, работающим на одних платформах, вызывать сервисы, работающие на других платформах, облегчает повторное использование компонентов.

Элементы SOA

Эта архитектурная модель состоит из компонентов и приложений, которые связываются друг с другом с помощью четко определенных сервисов.

Она состоит из 5 элементов:

- сервисы (Services);
- сервисная шина (Service Bus);
- сервисный репозиторий (Service Repository catalogue of services);
- безопасность SOA (SOA Security);
- управление SOA (SOA Governance).

Сервисы — это независимые элементы системы, которые реализуют некоторую бизнес- функцию и/или соответствуют бизнес- процессу предприятия.

Шина — это независимый элемент, предназначенный для параллельного получения, обработки и перенаправления запросов между сервисами.

Сервисы предприятия — высокоуровневые сервисы, соответствующие бизнес- доменам или процессам, использующим сервисы нижнего уровня.

Прикладные сервисы — чаще всего не переиспользуемые под разные задачи сервисы, ограниченные тем или иным прикладным контекстом. Встраиваются в сервисы предприятия или взаимодействуют между собой и/или инфраструктурными сервисами.

Инфраструктурные сервисы — отличаются от прикладных лишь тем, то переиспользуются под разные задачи.

В парадигме SOA сервисы зачастую являются крупными монолитными приложениями, общающимися с интеграционной шиной, которая передает сообщения от одной системы к другой.

Главное преимущество SOA — это масштабируемость, так как она не накладывает ограничений на количество сервисов, которые можно использовать для обмена сообщениями с шиной.

Клиент отправляет запрос с использованием стандартного протокола и формата данных по сети. Этот запрос обрабатывается ESB (enterprise service bus — сервисная шина предприятия), которая считается сердцем сервис-ориентированной архитектуры и отвечает за оркестровку и маршрутизацию. С помощью сервисного репозитория ESB направляет запрос в специальный сервис, который может взаимодействовать с другими сервисами и базами данных, чтобы составить полезную нагрузку (данные) ответа.

Полный вызов ответа на запрос согласуется с правилами управления и безопасности SOA для выполнения безопасной и корректной транзакции.

Как правило, сервисы делятся на два вида.

1. **Атомарные сервисы** (Atomic services) предоставляют функциональности, которые не подлежат дальнейшей декомпозиции.
2. **Композиционные сервисы** (Composite services) сочетают в себе несколько атомарных сервисов, чтобы предоставлять сложную составную функциональность.

Типы сервисов

- Организационные сервисы (Entity service).
- Доменные сервисы (Domain Service).
- Вспомогательные сервисы (Utility Service).
- Интегрированный сервис (Integrated Service).
- Сервис приложений (Application Service).
- Сервис безопасности (Security Service).

REST веб-сервисы

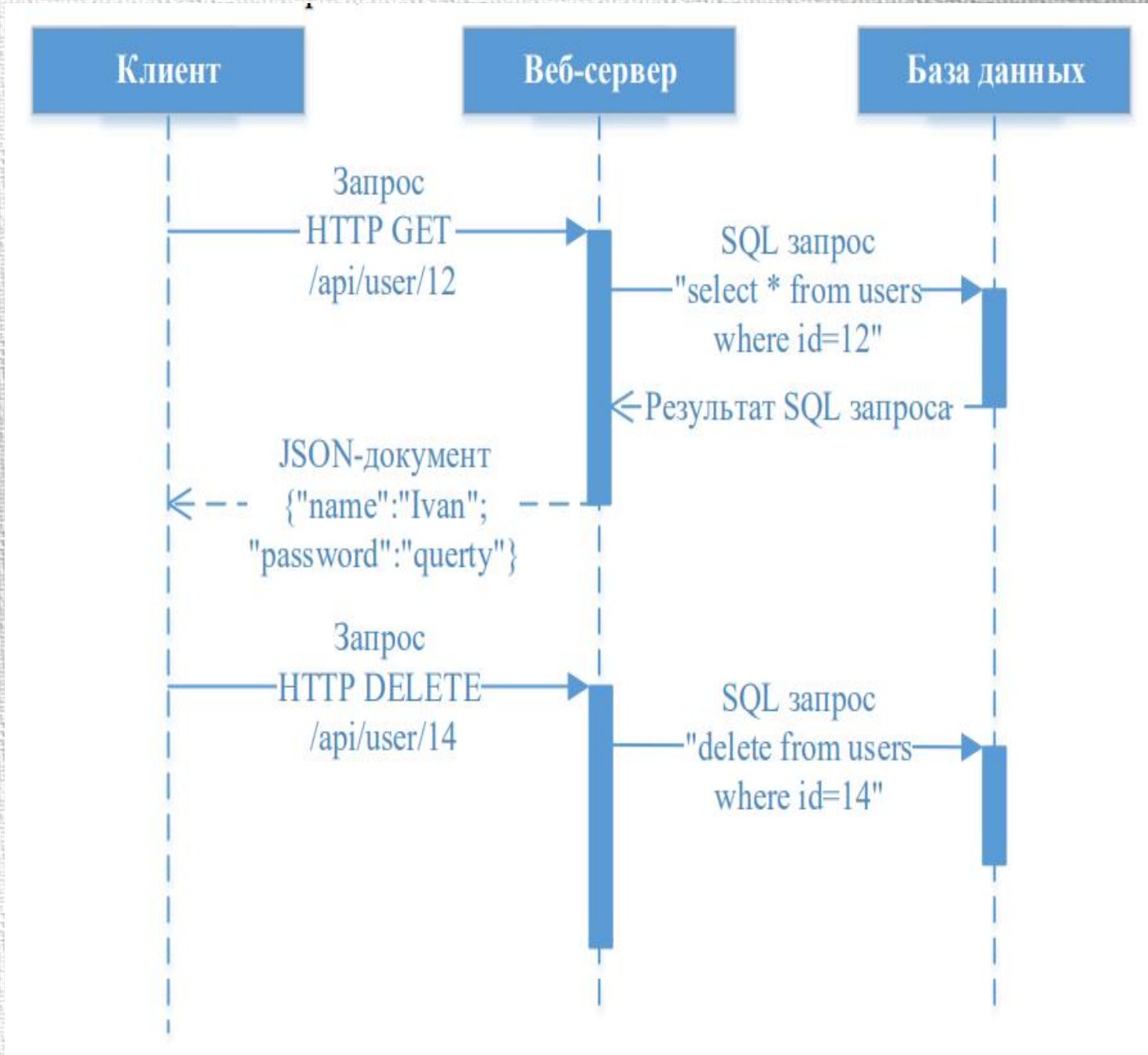
REST — метод взаимодействия компонентов приложения с использованием протокола HTTP для вызова процедуры. При этом необходимые данные передаются в качестве параметров запроса. Этот способ является альтернативой более сложным методам, таким как SOAP, CORBA и RPC.

Веб-сервисы REST являются веб-сервисами, реализуемые с использованием HTTP и принципов REST. Как правило, Web-сервис RESTful определяет URI основного ресурса, поддерживаемые MIME-типы представления/ответа и операции.

REST веб-сервис

Листинг . Пример запроса к REST веб-сервису

```
GET /StockPrice/IBM
HTTP/1.1
Host: example.org
Accept: text/xml
Accept-Charset: utf-8
```



Листинг. Пример ответа на запрос к REST веб-сервису

```
HTTP/1.1 200 OK
Content-Type: text/xml;
charset=utf-8;
Content-Length:nnn;
<?xmlversion="1.0"?>
<s:Quote xmlns:s="http://example.org/stock-service">
<s:TickerSymbol>IBM</s:TickerSymbol>
<s:StockPrice>45.25</s:StockPrice>
</s:Quote>
```

Ресурс может являться практически любым понятным и значимым адресуемым объектом. Представление ресурса — это обычно документ, отражающий текущее или требуемое состояние ресурса. Ресурсы обычно представляются документами в форматах XML или JSON.

REST и горизонтальный подход

Стратегия, опирающаяся на горизонтальный подход к протоколам, наиболее радикальна. Слово «горизонтальный» означает в данном случае сохранение существующего уровня, без выстраивания уровней поверх него. Предполагается отказаться от разработки новых протоколов, а использовать несколько хорошо проверенных, считая, что для работы с объектами вполне достаточно уметь выполнять четыре типа действий: создание (Creation), восстановление (Retrieval), изменение (Update) и уничтожение (Destruction). Из этих действий получается так называемый «шаблон проектирования» CRUD. Протокол Hypertext Transfer Protocol определяет методы GET/PUT/POST/DELETE, которые и реализуют шаблон CRUD.

HTTP-методы в REST веб-сервисах

HTTP-метод	CRUD операция
GET	Read
POST	Create (иногда используется для операций update, delete)
PUT	Create (иногда используется для операции update)
DELETE	Delete

Разработчики SOAP веб-сервисов создают свой собственных перечень имен существительных и глаголов (например, getUsers(), savePurchaseOrder(...)) для обозначения операций веб-сервиса. В этом смысле SOAP реализуют сервисный принцип работы, в соответствии с которым главную роль при взаимодействии со службами играют их методы. В основе архитекторы REST веб-сервисов лежит ресурсный (объектный) подход.

REST веб-сервисы разрабатываются согласно следующим принципам:

1. Возвращайте любые данные по их идентификатору.

2. Используйте стандарты, Имеется в виду, экономьте свои силы и деньги заказчика, используйте стандартные методы HTTP.

3. Одни и те же данные можно вернуть различных форматах. Например, в XML или JSON для последующей программной обработки.

4. **Передача данных без сохранения состояния**. При обращении к REST-сервису не учитываются результаты ранее выполненных операций. REST приложение не сохраняет никакого состояния сессии на стороне сервера. Вся информация, необходимая для выполнения запроса, передается в самом запросе.

Преимущества REST-сервисов

- Независимы от языка и платформы (транспорт-HTTP)
- Проще чем SOAP
- Быстрее изучение, меньше требуется инструментов
- Краток, нет промежуточного протокола сообщений (SOAP)
- По дизайну и философии ближе к WWW

Недостатки REST-сервисов

- Предполагает взаимодействие только клиент-сервер, неприменим для разработки распределенных систем
- Не стандартизированы механизмы безопасности, гарантированной доставки сообщений и т.п. - больше приходится реализовать самостоятельно прикладной системе
- Жестко привязан к модели протокола HTTP

Сети веб-сервисов (сервисная шина предприятия)

Автономные веб-сервисы могут быть объединены в сеть взаимодействующих друг с другом сервисом.

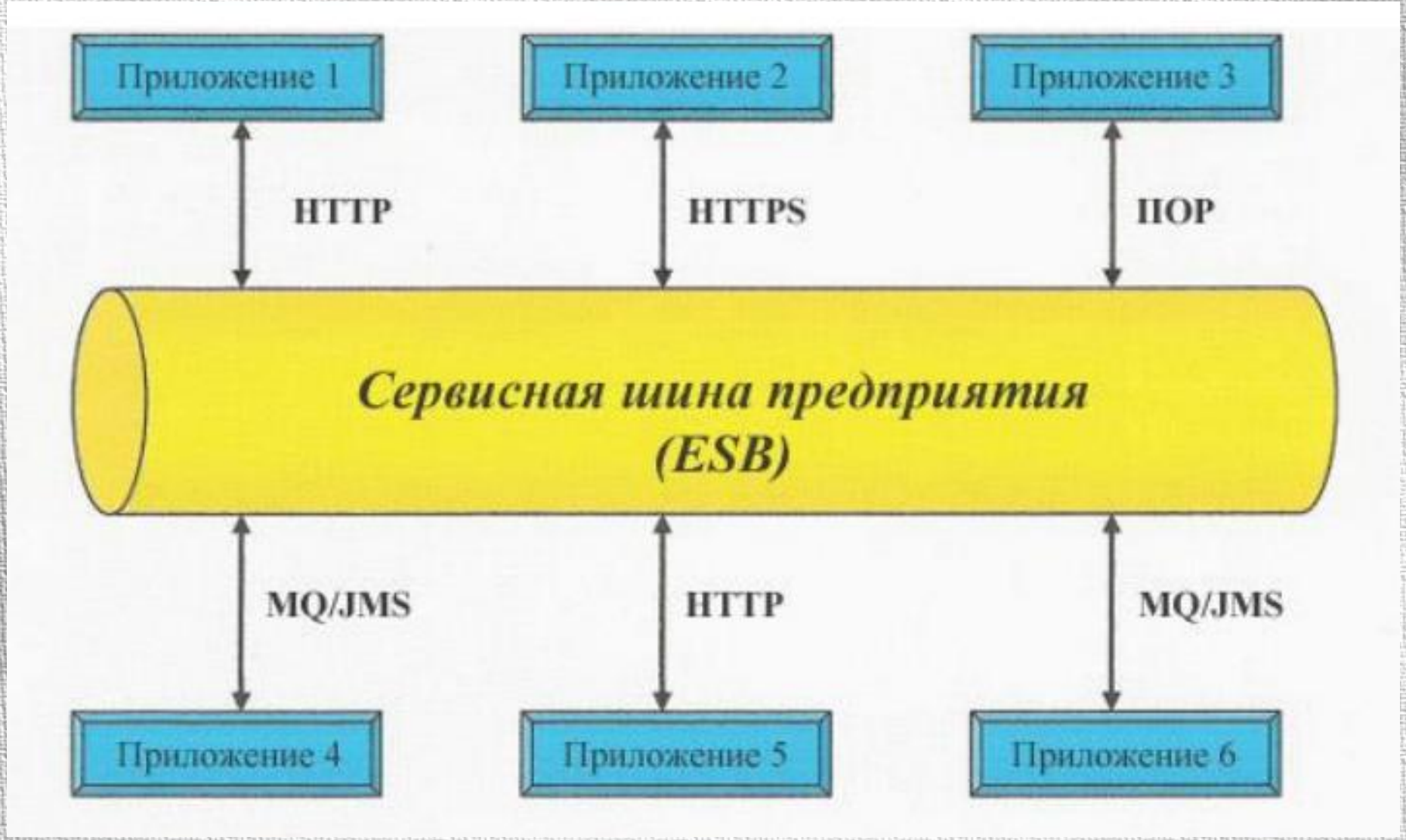
Сервисная шина предприятия (Enterprise service bus) – архитектурное решение для построения распределённых корпоративных информационных систем. Обычно включает в себя промежуточное ПО, которое обеспечивает взаимосвязь между различными приложениями по различным протоколам взаимодействия. Основной принцип сервисной шины — концентрация обмена сообщениями между различными системами через единую точку, в которой, при необходимости, обеспечивается транзакционный контроль, преобразование данных, сохранность сообщений.

Все настройки обработки и передачи сообщений предполагаются также сконцентрированными в единой точке, и формируются в терминах служб, таким образом, при замене какой-либо информационной системы, подключённой к шине, нет необходимости в перенастройке остальных систем.

Решения ESB поставляются компаниями Oracle (Oracle Service Bus), IBM (IBM WebSphere ESB), Microsoft (Microsoft BizTalk Server). Также существуют бесплатные шины сервисов с открытым исходным кодом: Open ESB, Ultra ESB, Apache Service Mix и др .

Использование веб-сервисов без ESB

Сервисная шина предприятия



Основные функции ESB

- поддержка синхронного и асинхронного способа вызова служб;
- использование защищённого транспорта, с гарантированной доставкой сообщений, поддерживающего транзакционную модель;
- статическая и алгоритмическая маршрутизация сообщений;
- доступ к данным из сторонних информационных систем с помощью готовых или специально разработанных адаптеров;
- обработка и преобразование сообщений;
- оркестровка и хореография служб;
- протоколирование

Достоинства и недостатки SOA

Плюсы:

- Независимость набора технологий, развёртывания и масштабируемости сервисов.
- Стандартный, простой и надёжный канал связи (передача текста по HTTP через порт 80).
- Оптимизированный обмен сообщениями.
- Стабильная спецификация обмена сообщениями.
- Изолированность контекстов доменов (Domain contexts).

Минусы:

- Разные веб-сервисы тяжело интегрировать из-за различий в языках передачи сообщений. Например, два веб-сервиса, использующих разные JSON-представления одной и той же концепции.
- Синхронный обмен сообщениями может перегрузить системы.

С точки зрения технологий SOA не просто сервисная архитектура, а набор политик, методик и фреймворков, благодаря которым мы предоставляем и получаем нужные сервисы.

Получила распространение в конце 1990-х — начале 2000-х годов. С середины 2010-х годов обрела популярность микросервисная архитектура — вариант COA, базирующийся на применении, насколько это возможно, минимальных по размеру сервисов.

МИКРОСЕРВИСНАЯ АРХИТЕКТУРА

- Микроархитектурные

Микросервисы - это архитектурный стиль, который структурирует приложение как набор небольших автономных сервисов, смоделированных на основе микросервисной архитектуры, или микросервисов. В микросервисной архитектуре каждая служба является автономной и реализует единую бизнес-функцию.

Разработчики используют эту технологию для масштабирования своих приложений в соответствии со своими потребностями. Монолитное приложение разлагается на небольшие крошечные микроприложения, которые упаковываются и развертываются независимо.

Микросервисная архитектура — это такой способ проектирования системы, при котором бизнес-логика поделена на несколько частей и реализована в виде независимых приложений. Эти приложения взаимодействуют между собой по определённому протоколу для достижения конкретного результата.

Если у вас растущий бизнес, и вы планируете масштабирование в перспективе, использование микросервисной архитектуры позволит вам создать устойчивую к нагрузкам систему. Вы сможете выбирать актуальные технологии под ваши нужды и оперативно доставлять новые фичи.

Микросервисная архитектура – одна из самых полезных архитектур в индустрии программного обеспечения. При правильном подходе она может помочь в создании лучших программных приложений.

Архитектурный паттерн, способ или стиль архитектуры, если следовать ему правильно, приведет к созданию программного приложения, состоящего из нескольких сервисов, вместо одного большого монолита.

Сервисы в этом паттерне легко разрабатывать, тестировать, деплоить и поддерживать по отдельности. Достаточно небольших команд, которые отвечают за каждый сервис, что сокращает объем коммуникаций, а также облегчает управление.

Это позволяет командам внедрять различные технологические стеки, модернизировать технологии в существующих сервисах, масштабировать, а также изменять или деплоить каждый сервис независимо.

Типы архитектуры приложений

Монолитная

многоуровневая архитектура
конвейерная архитектура
микроядерная архитектура

Распределенная

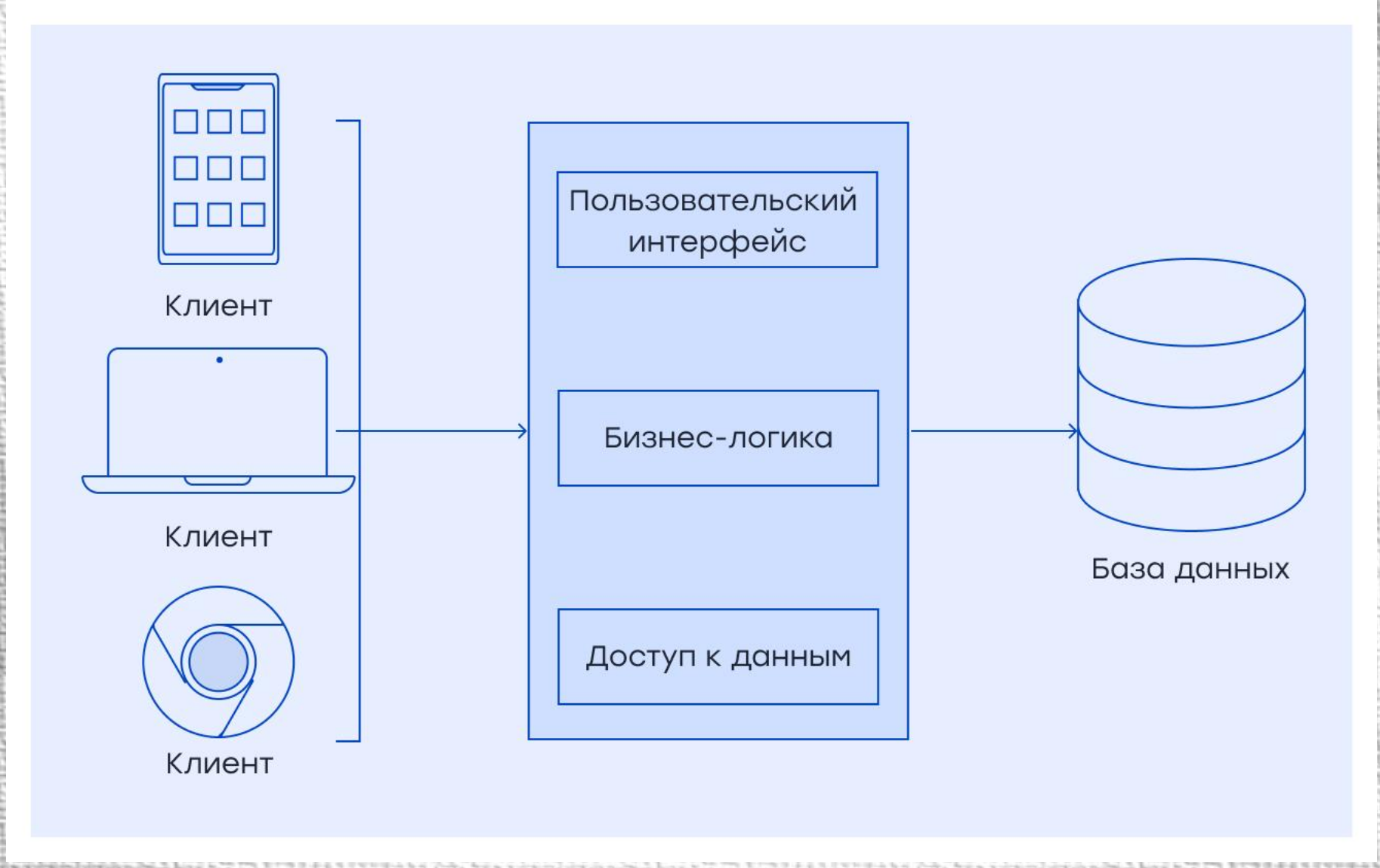
архитектура на основе сервисов
архитектура, управляемая событиями
архитектура на основе пространства
сервис-ориентированная архитектура
микросервисная архитектура

Микросервисы против Монолитов

И микросервисы, и монолитные сервисы – это архитектурные паттерны, которые используются при разработке программных приложений для обслуживания бизнес-требований. У каждого из них есть свои преимущества, недостатки и проблемы.

Монолитные архитектуры могут быть громоздкими в работе, когда нужно добавить новые функции, внести изменения или даже удалить некоторые ненужные функции.

Монолитная архитектура



Монолитные архитектуры по-прежнему являются хорошим вариантом для приведенных ниже случаев:

- малогабаритные приложения;
- нет необходимости в обновлении технологии;
- вам требуется меньше времени для вывода на рынок;
- команды знакомы с Монолитными подходами.

С другой стороны, архитектура микросервисов не очень хорошо подходит для небольших приложений, требующих единого технологического стека, платформы деплоя, доменной зоны и т. д. **Но для крупномасштабных приложений пока нет ничего лучше этой архитектуры**.

Проблемы монолитной архитектуры

- **Негибкий** – монолитные приложения не могут быть построены с использованием различных технологий.
- **Ненадежный** – даже если одна часть системы не работает, то вся система не работает.
- **Немасштабируемый** – приложения не могут быть легко масштабированы, так как каждый раз, когда приложение должно быть обновлено, вся система должна быть перестроена.
- **Блоки непрерывного развития** – многие функции приложений не могут быть созданы и развернуты одновременно.

- **Медленное развитие** – разработка в монолитных приложениях занимает много времени, так как каждая функция должна быть построена одна за другой.
- **Не подходит для сложных применений** – особенности сложных приложений имеют тесно связанные зависимости.

Вышеуказанные проблемы были основными причинами, которые привели к эволюции микросервисов. Итак, теперь давайте разберемся, что такое микросервисы простыми словами?

Что такое микросервисы?

Микросервисы — это архитектурный стиль, который структурирует приложение как набор небольших автономных служб, смоделированных вокруг бизнес-домена.

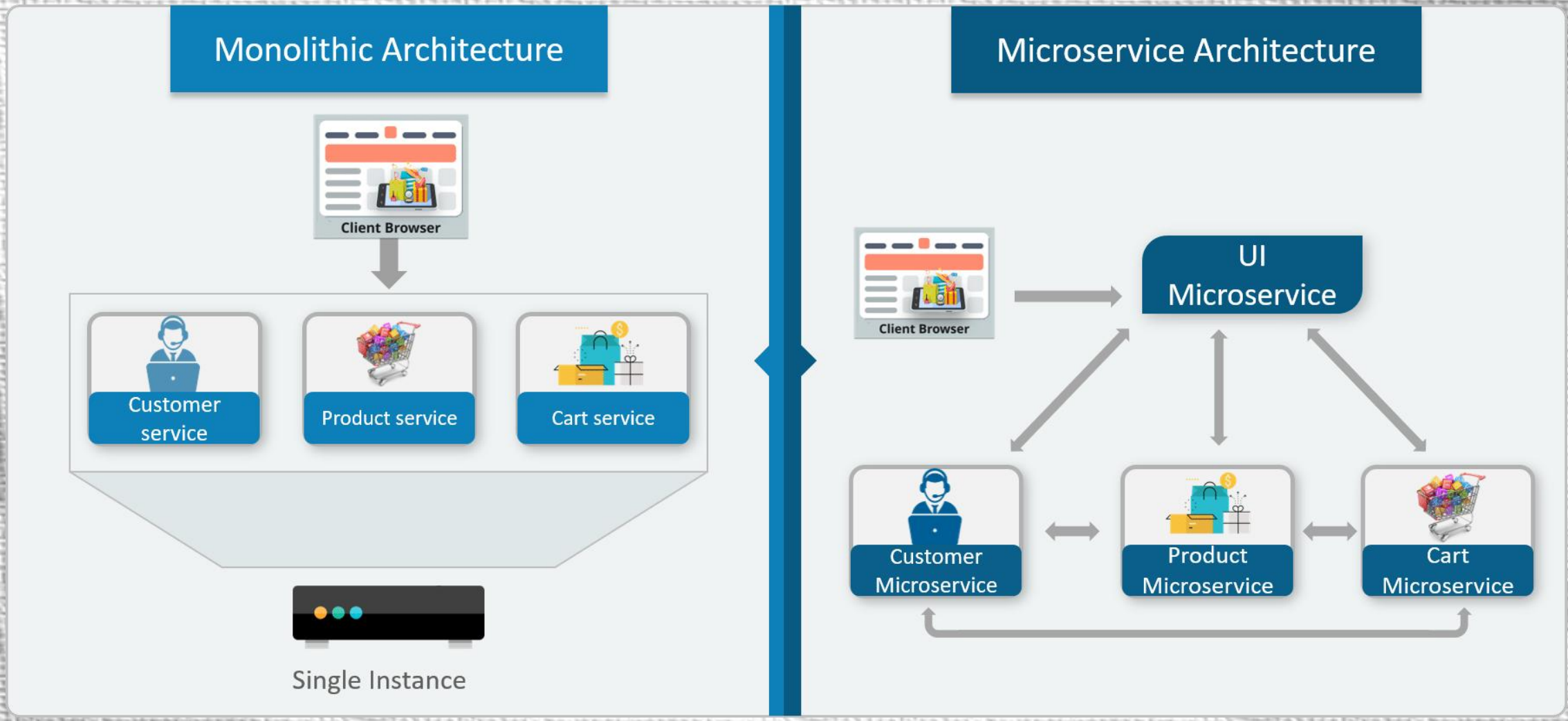
*В данной архитектуре каждая служба является **автономной** и реализует **единую бизнес -возможность.***

Вот вам простая аналогия.

Вы, должно быть, видели, как пчелы строят свои соты, выравнивая шестиугольные восковые ячейки. Сначала они начинают с небольшого участка, используя различные материалы, и продолжают строить из него большой улей. Эти ячейки образуют узор, в результате чего получается прочная структура, которая удерживает вместе определенную секцию улья. Здесь каждая ячейка независима от другой, но она также коррелирует с другими ячейками. Это означает, что повреждение одной ячейки не повреждает другие ячейки, поэтому пчелы могут восстанавливать эти ячейки, не влияя на весь улей.

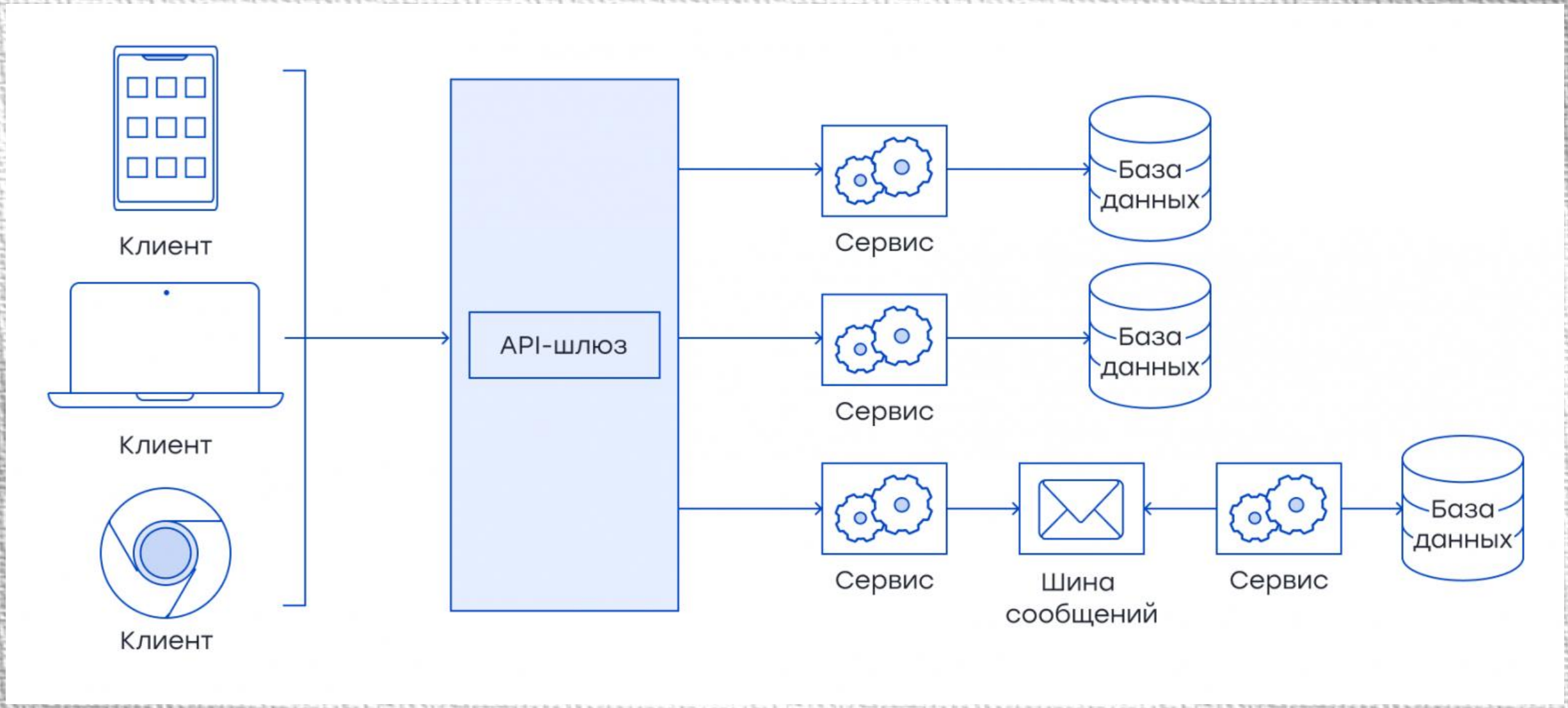
Здесь каждая шестиугольная форма представляет отдельный компонент сервиса. Подобно работе с пчелами, каждая команда agile создает индивидуальный компонент сервиса с доступными фреймворками и выбранным стеком технологий. Как и в улье, каждый компонент сервиса формирует надежную архитектуру микросервиса для обеспечения лучшей масштабируемости. Кроме того, проблемы с каждым компонентом сервиса могут решаться индивидуально командой agile без какого-либо или минимального воздействия на все приложение.

Различия между монолитной архитектурой и микросервисами

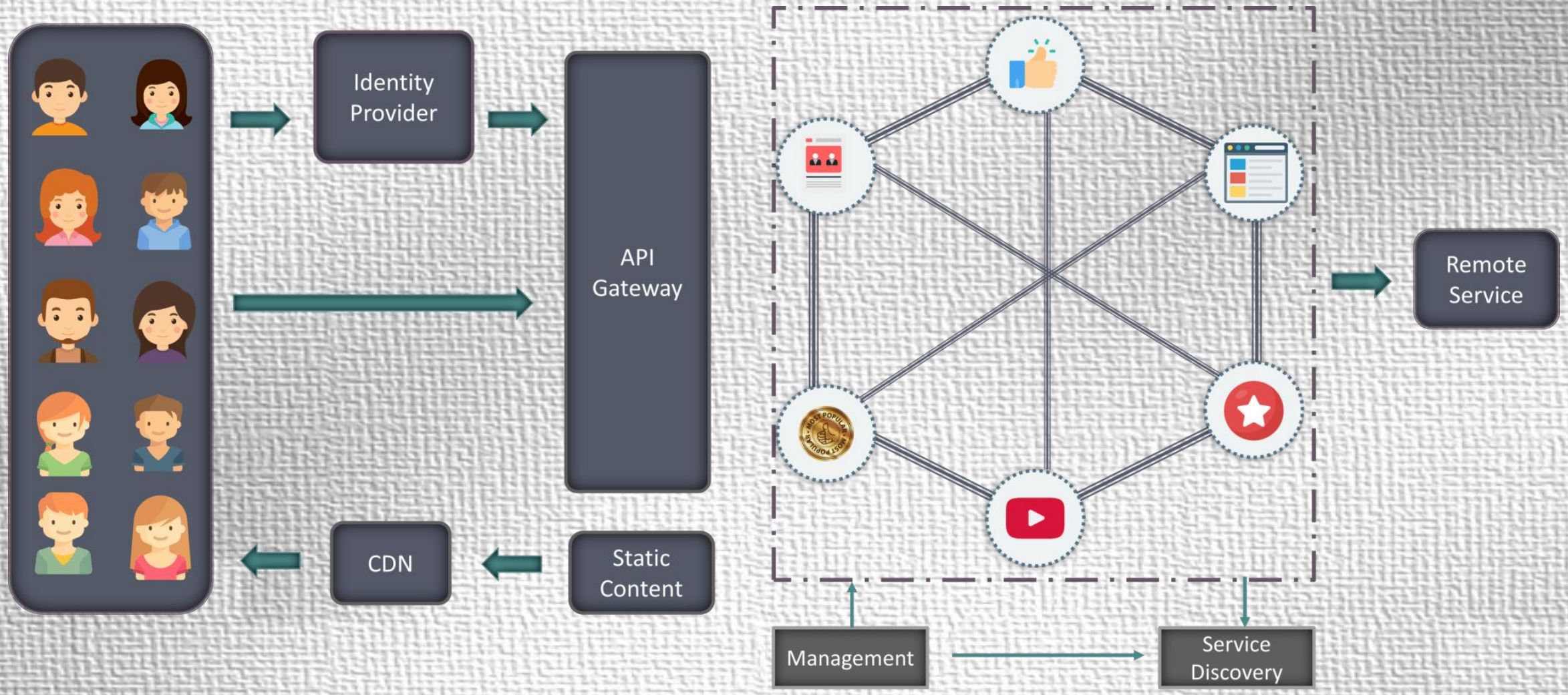


Основное различие, которое мы наблюдаем на приведенной выше диаграмме, заключается в том, что все функции изначально находились под одним экземпляром, совместно использующим одну базу данных. Но затем, с микрослужбами, каждой функции была выделена отдельная микрослужба, обрабатывающая свои собственные данные и выполняющая различные функции.

Микросервисная архитектура



Микросервисная архитектура



Клиенты – разные пользователи с разных устройств отправляют запросы.

Поставщики удостоверений (Identity Providers) – проверяет подлинность удостоверений пользователей или клиентов и выдает токены безопасности.

API Gateway – Обрабатывает запросы клиентов.

Статическое содержимое (Static Content) – содержит все содержимое системы.

Управление (Management) – балансирует службы на узлах и выявляет сбои.

Обнаружение служб (Service Discovery) – руководство по поиску маршрута связи между микросервисами.

Сети доставки контента (Content Delivery Networks) – распределенная сеть прокси-серверов и их центров обработки данных.

Удаленное обслуживание (Remote Service) – позволяет получить удаленный доступ к информации, которая находится в сети ИТ-устройств.

Зачем вообще нужны API Gateway

При работе с микросервисной архитектурой рано или поздно приходится столкнуться с проблемой, которой нет у монолитных систем, — с необходимостью получать и обрабатывать данные из нескольких источников для обслуживания одного-единственного запроса.

Представьте себе: у вас есть интернет-магазин по продаже реплик молота Тора. Для удобства пользователя имеется как сайт под десктоп и мобильные устройства, так и приложения для Android и iPhone, которые взаимодействуют с сервером через REST API.

Чтобы на странице товара отображались верные данные, нам нужно обратиться к нескольким службам: в одной учитывается наличие молота, в другой записаны материал, вес и длина ручки, в третьей сохраняются отзывы клиентов, а цена вообще указана в четвёртой. API Gateway позволяет обойтись одним запросом.

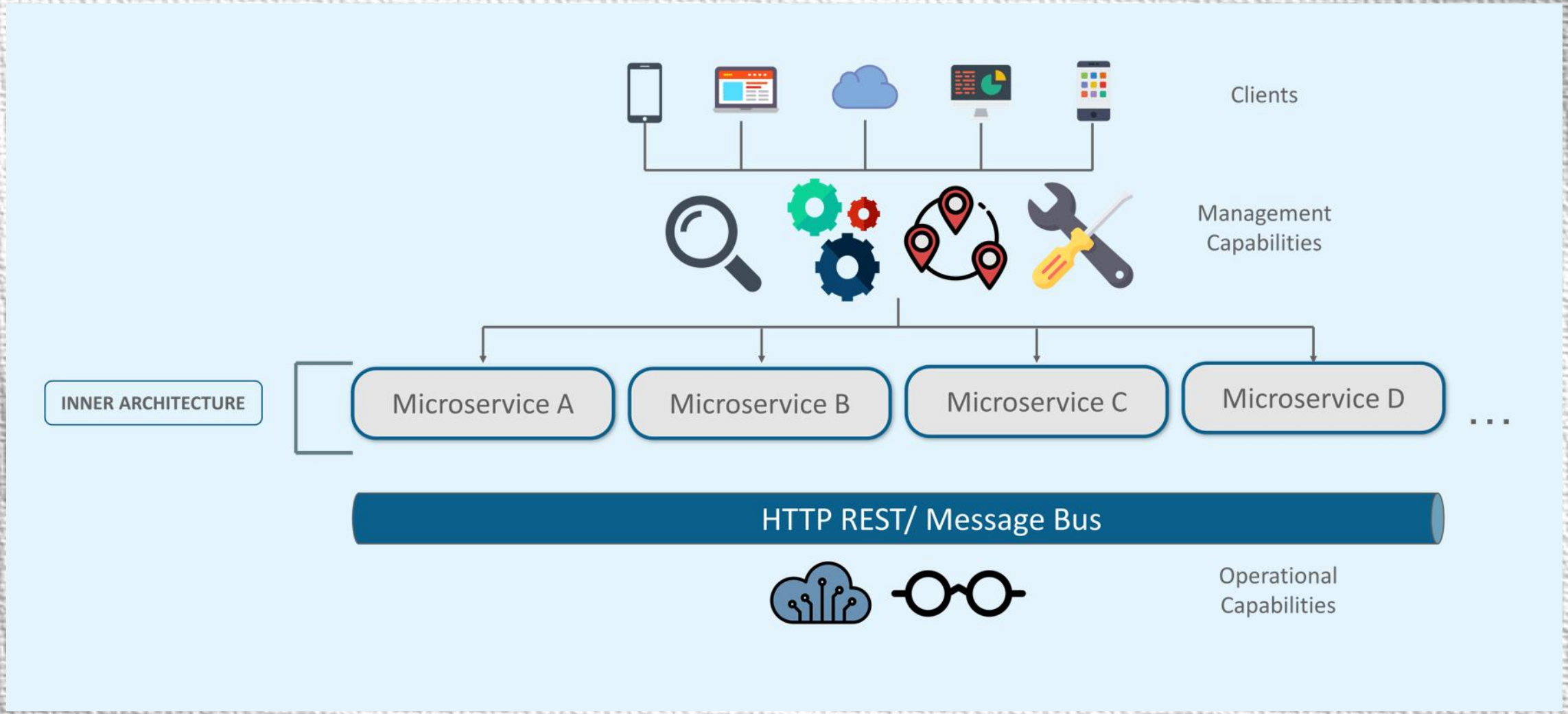
API Gateway выполняет множество задач:

- принимает, обрабатывает и распределяет запросы,
- контролирует трафик,
- осуществляет мониторинг и контроль доступа.

В микросервисной архитектуре паттерн API Gateway появился в качестве службы, обеспечивающей единую точку входа для веб-приложений и API, эдакой «серверной части для клиентской части». В чём польза именно для микросервисов?

Например — возможность повторного использования компонентов, упрощение бэкенда приложения, обеспечение доступа к статическим веб-страницам и документам, удобная проверка авторизации и подбор оптимального для каждого типа клиента API — как это делает Netflix API Gateway.

Архитектура микросервисов



- Разные клиенты с разных устройств пытаются использовать разные сервисы, такие как поиск, сборка, настройка и другие возможности управления.
- Все службы разделены в зависимости от их доменов и функциональных возможностей и далее распределяются по отдельным микрослужбам.
- Эти микрослужбы имеют свой собственный балансировщик нагрузки и среду выполнения для выполнения своих функций и в то же время захватывают данные в своих собственных базах данных.
- Все микрослужбы взаимодействуют друг с другом через сервер без отслеживания состояния, который является либо REST, либо Message Bus.

- Микросервисы знают свой путь связи с помощью Service Discovery и выполняют операционные возможности, такие как автоматизация, мониторинг.
- Затем все функциональные возможности, выполняемые микрослужбами, передаются клиентам через шлюз API.
- Все внутренние точки подключаются из шлюза API. Таким образом, любой, кто подключается к шлюзу API, автоматически подключается ко всей системе.

Теперь давайте узнаем больше о микросервисах, взглянув на их функции.

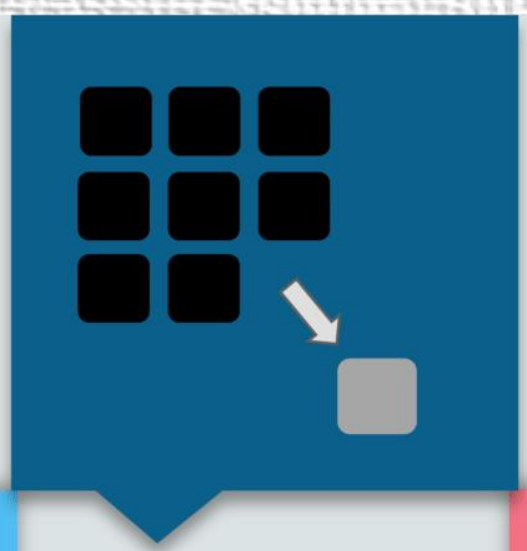
Возможности микрослужб

1. **Разъединение** — службы в системе в значительной степени разъединены. Таким образом, приложение в целом может быть легко построено, изменено и масштабировано.
2. **Компонентизация** — микрослужбы рассматриваются как независимые компоненты, которые можно легко заменить и обновить.
3. **Бизнес-возможности** — микросервисы очень просты и ориентированы на одну возможность
4. **Автономия** — разработчики и команды могут работать независимо друг от друга, тем самым увеличивая скорость
5. **Непрерывная поставка** — позволяет часто выпускать программное обеспечение благодаря систематической автоматизации создания, тестирования и утверждения программного обеспечения

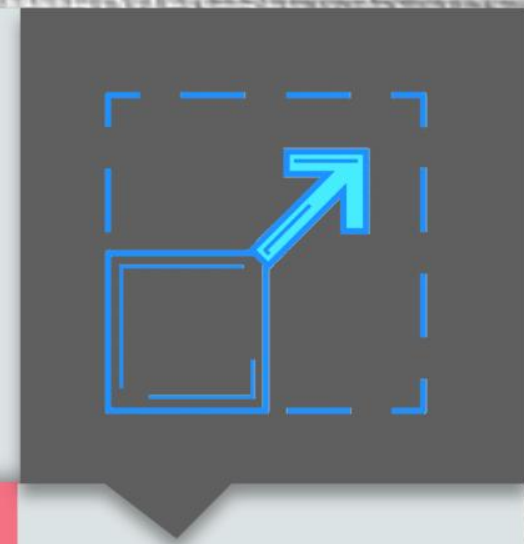
6. **Ответственность** — микросервисы не фокусируются на приложениях как проектах. Вместо этого они рассматривают приложения как продукты, за которые они несут ответственность.
7. **Децентрализованное управление** - Основное внимание уделяется использованию правильного инструмента для правильной работы. Это означает, что не существует стандартизированного шаблона или какого-либо технологического шаблона. Разработчики имеют свободу выбора лучших полезных инструментов для решения своих проблем
8. **Гибкость** — любая новая функция может быть быстро разработана и отброшена снова

Преимущества микросервисов

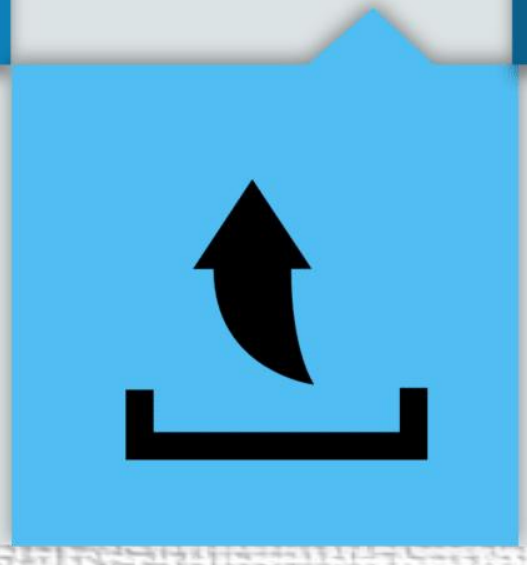
Independent Deployment



Mixed Technology Stack



Independent Development



Fault Isolation

Granular Scaling

1. **Независимая разработка** — все микросервисы могут быть легко разработаны на основе их индивидуальной функциональности.
2. **Независимое развертывание** — на основе их услуг они могут быть индивидуально развернуты в любом приложении.
3. **Изоляция неисправностей** — даже если одна служба приложения не работает, система все равно продолжает функционировать.
4. **Смешанный стек технологий** — различные языки и технологии могут использоваться для создания различных сервисов одного и того же приложения.
5. **Гранулярное масштабирование** — отдельные компоненты могут масштабироваться в соответствии с потребностями, нет необходимости масштабировать все компоненты вместе.

Микросервисная архитектура является хорошим вариантом для следующих случаев:

- широкомасштабные приложения;
- вам необходимо обновление технологии;
- разные команды работают с разными техническими скилами (языки программирования, фреймворки и другое);
- если у вас есть различные домены, которые необходимо обработать;
- И многие другие.

Сравнение

Сравним некоторые характеристики микросервисов и монолитной архитектуры, чтобы лучше понять их.

Характеристика	Микросервисы	Монолит
Скорость	Сравнительно медленнее в понятиях разработки, поскольку мелкие сервисы разрабатываются отдельно.	Быстрее, когда речь идет о разработке одного приложения, содержащего почти все необходимое с самого начала.
Независимость	Каждый сервис имеет собственное хранилище данных и отвечает за конкретный домен, и может разрабатываться изменяться или деплоиться независимо.	Благодаря тесному взаимодействию все приложение деплоится при любом изменении, поскольку вся бизнес-логика находится в одном месте.
Гибкость	Сервисы имеют меньший размер, что делает их легко адаптируемыми к новым технологическим изменениям.	Используется одна и та же технология, которую нельзя изменить, поскольку все приложение было разработано с ее использованием.
Масштабирование	Легко масштабировать сервисы без ущерба для других.	Приходится масштабировать все приложение, даже если нужно масштабировать только одну фичу внутри.

Характеристика	Микросервисы	Монолит
Надежность	Нет необходимости останавливать приложение, если требуется внести изменения в сервис.	Приходится заново деплоить все приложение, если что-то требует изменений.
Тестируемость	Сервисы, специфичные для конкретного домена, легко тестировать.	Сложно протестировать всю систему сразу.
Удобство обслуживания	Отдельные сервисы могут быть обновлены и изменены в соответствии с рыночными условиями.	Трудно поддерживать при увеличении размера приложения, поскольку изменения в одном месте могут иметь регрессивные последствия для других.
Команды	Можно задействовать несколько небольших команд с различными наборами технологических скиллов.	Требуется одна большая команда с почти одинаковыми навыками и пониманием технологий.
Управление	Управление различными небольшими командами, каждая из которых работает над определенной областью бизнеса, позволяет легко анализировать, кто за что отвечает.	Управление одной большой командой, работающей над полным доменом, требует внутреннего разделения.
Коммуникация	Коммуникация медленнее, но легче.	Есть сложность общения с руководством по поводу возникающих проблем.
Инфраструктура	Экспоненциальная стоимость, поскольку разные команды с разными навыками работают с несколькими репозиториями, средами и пайплайнами.	Экономическая эффективность, поскольку команды работают над одним и тем же репозиторием, и все команды обладают практически одинаковыми навыками в области технологий. Таким образом, нет необходимости в нескольких различных средах разработки и деплоя.

Проектирование микросервисов

При проектировании микросервисов необходимо отталкиваться от их основных характеристик. Каждый микросервис должен быть небольшим, сфокусированным, и независимым настолько, насколько это возможно.

В фокусе одного микросервиса должна быть одна задача. Это не только позволит сделать его максимально компактным, но и обеспечит «самостоятельность» микросервиса, исключив лишние связи.

Если вы спроектировали систему, где имеются микросервисы с плотным взаимодействием и один не может существовать без другого, это повод задуматься над тем, чтобы объединить эти приложения в одно.

Также стоит определиться с тем в каком формате будут взаимодействовать ваши микросервисы (JSON, XML, GraphQL, gRPC). Данный шаг очень важен, так как впоследствии с ростом микросервисов будет сложно перестроиться на новый формат.

Все микросервисы могут и должны взаимодействовать друг с другом, осуществляя синхронные и асинхронные операции.

Представим, что пользователь изменил фамилию у себя в личном кабинете. Эта информация должна также меняться в email-рассылке, в чате, словом, везде, где используются контактные данные пользователя. Это и будет асинхронной операцией, для осуществления которой создается сообщение, которое затем отправляется в сервер очередей. Информацию об изменении получают те микросервисы, которые подписаны на обновления. При получении этой информации происходит событие — в нашем случае изменение данных пользователя.

Важные паттерны проектирования

В микросервисной архитектуре существует несколько паттернов, рассмотрим ниже некоторые из них.

- Strangler (Шаблон душителя или шаблон виноградной лозы)
- API-шлюз
- Агрегатор
- Сага (Saga)
- Асинхронный обмен сообщениями
- Circuit Breaker

Шаблоны проектирования микросервисов

- Агрегатор
- API Gateway
- Цепочка или цепочка ответственности
- Асинхронный обмен сообщениями
- База данных или общие данные
- Поиск событий
- Филиал
- Разделитель ответственности за выполнение командных запросов
- Автоматический выключатель
- Декомпозиция

Преимущества

- Меньшие размеры команды с различными наборами навыков позволяют использовать лучшее из каждого языка и фреймворка.
- Более быстрый и независимый деплой, когда в действие вступает реальное использование непрерывной интеграции и деплоя.
- Тестирование каждой службы является простым и менее трудоемким, поскольку службы специфичны для своего хранилища данных.
- Различные технологии могут способствовать созданию комплексного решения. Переход от одной технологии к другой также прост.
- Изменение в сервисе не повлияет на всю систему, что обеспечивает высокую надежность.

Недостатки

- Инфраструктура требует больших затрат, поскольку различные технологии имеют различные методологии тестирования, среды для каждого сервиса и деплоя.
- Дебаг может стать громоздким из-за различных логов каждого сервиса и выявления ошибок в реальном сервисе.
- Полиглот микросервисов имеет и побочные эффекты, когда компании необходимо поддерживать ресурсы для каждой технологии, используемой для разработки сервисов.

Инструменты для работы с МС: создание, логирование, мониторинг

- **Docker** – платформа для разработки и эксплуатации программных продуктов. Docker стал технологией, которая пришла на замену виртуализации серверов и дала буст микросервисной архитектуре. Docker позволяет завернуть каждый микросервис в отдельный контейнер и поднимать, когда это необходимо.
- Когда приложение обрастет достаточным количеством микросервисов, потребуется приложение-менеджер для оркестрации. Самый популярный такой менеджер на сегодняшний день — **Kubernetes**, который позволяет автоматизировать их развёртывание, масштабирование и координацию.

- При этом мы не можем оставить микросервисы без должного наблюдения, поэтому мы подключаем процессы логирования и мониторинга. Логировать необходимо следующие моменты: критические и фатальные ошибки, входящие и исходящие сообщения для поиска места бага, а также недоступность микросервиса, чтобы затем системы мониторинга смогли оповестить, если один из элементов вышел из строя.

  Как можно догадаться, объём логов может выходить за границы чувственного восприятия человека. На помощь приходит **ELK-схема** — набор из трёх инструментов для генерации, обработки и индексации лога.

- Помимо логирования, необходимо вести мониторинг определенных параметров, чтобы быть в курсе текущего состояния микросервисов. Для мониторинга можно использовать популярные **Zabbix** или **Prometheus**.
- Напоследок упомянем **CI/CD** — практику continuous integration и continuous delivery. Так как микросервисов достаточно много, процесс интеграции вручную не представляется возможным. CI/CD позволяет автоматизировать выкладку на разные ландшафты и обладает следующими преимуществами: автоматизация сборки проекта, автоматическое тестирование, экономия времени при доставке фич на разные ландшафты. Для реализации CI/CD используют: Gitlab CI, Bitbucket Pipelines, Jenkins, Gitlab Actions.

Чем микросервисная архитектура (MSA) отличается от SOA?

Одним словом, SOA – это шаблон проектирования, а Microservice – это методология реализации для реализации SOA, или мы можем сказать, что Microservice – это тип SOA.

Микросервис через SOA

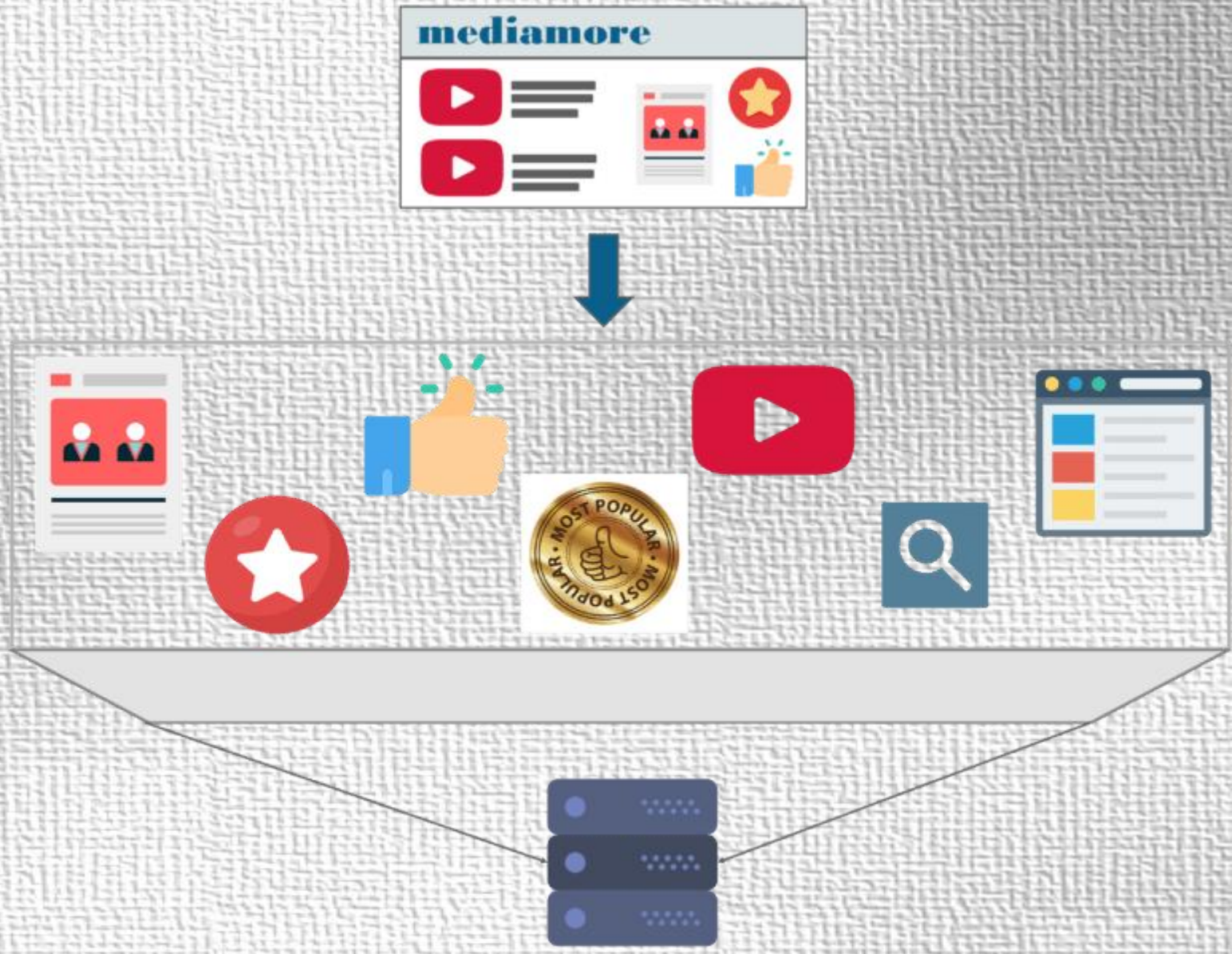
В следующей таблице перечислены некоторые функции SOA и Microservice, подчеркивая важность использования microservice поверх SOA.

Составная часть	SOA	Microservice
Дизайн шаблона	SOA – это парадигма проектирования компьютерного программного обеспечения, где компоненты программного обеспечения открыты для внешнего мира для использования в форме сервисов.	**Микро Сервис является частью SOA. Это специализированная реализация SOA.** В более широком смысле, мы можем рассматривать его как подмножество одного SOA-приложения.
Зависимость	Бизнес-единицы зависят друг от друга.	Все бизнес-единицы независимы друг от друга.
Размер	Размер программного обеспечения больше, чем у обычного программного обеспечения.	Размер программного обеспечения невелик.
Технология	Технологический стек меньше, чем у Микросервиса.	Микросервис по своей природе неоднороден, поскольку для выполнения конкретной задачи используются точные технологии. Микросервисы можно рассматривать как конгломерат многих технологий.

Составная часть	SOA	Microservice
Автономный и Фокус	SOA-приложения созданы для выполнения нескольких бизнес-задач.	Микросервисные приложения созданы для выполнения одной бизнес-задачи.
Природа	Монолитный по своей природе.	Полный стек в природе.
Развертывание	Развертывание занимает много времени.	Развертывание очень просто. Следовательно, это будет менее трудоемким.
Экономическая эффективность	Более экономически эффективным.	Менее рентабельно.
Масштабируемость	Меньше по сравнению с микросервисами.	Полностью масштабируется.
Пример	Давайте рассмотрим одну заявку на бронирование CAB онлайн. Если мы хотим создать это приложение с использованием SOA, то его программные модули будут: •GetPayments и DriverInformation и MappingDataAPI •Аутентификация пользователей и драйверов API	Если то же приложение построено с использованием микросервисной архитектуры, то его API будут: •SubmitPaymentsService •GetDriverInfoService •GetMappingDataService •AuthenticateUserService •AuthenticateDriverService

Как это все работает вместе (пример)

Mediamore (СМИ больше) - это развлекательная компания, которая предоставляет потоковое мультимедиа и видео онлайн. Состоит из различных жанров телешоу на разных языках. Как и многие другие компании, mediamore начала свой путь с монолитной архитектуры.



Обратитесь к приведенной выше схеме. Мы можем сделать вывод, что все функции, такие как поиск, информация о пользователе, рекомендации, список воспроизведения видео и другие, помещаются в единую базу данных с использованием единого кода.

Сценарий

Алиса - заядлый пользователь mediamore. Она регулярно использует mediamore, чтобы смотреть свои любимые сериалы онлайн. Недавно она пропустила просмотр эпизода своего любимого телешоу.

Когда Алиса входит в приложение, она видит наиболее рекомендуемый контент на своей домашней странице. После недолгих поисков она наконец находит свое телешоу.

Но что, если Алиса хочет получить свое телешоу одним щелчком мыши.

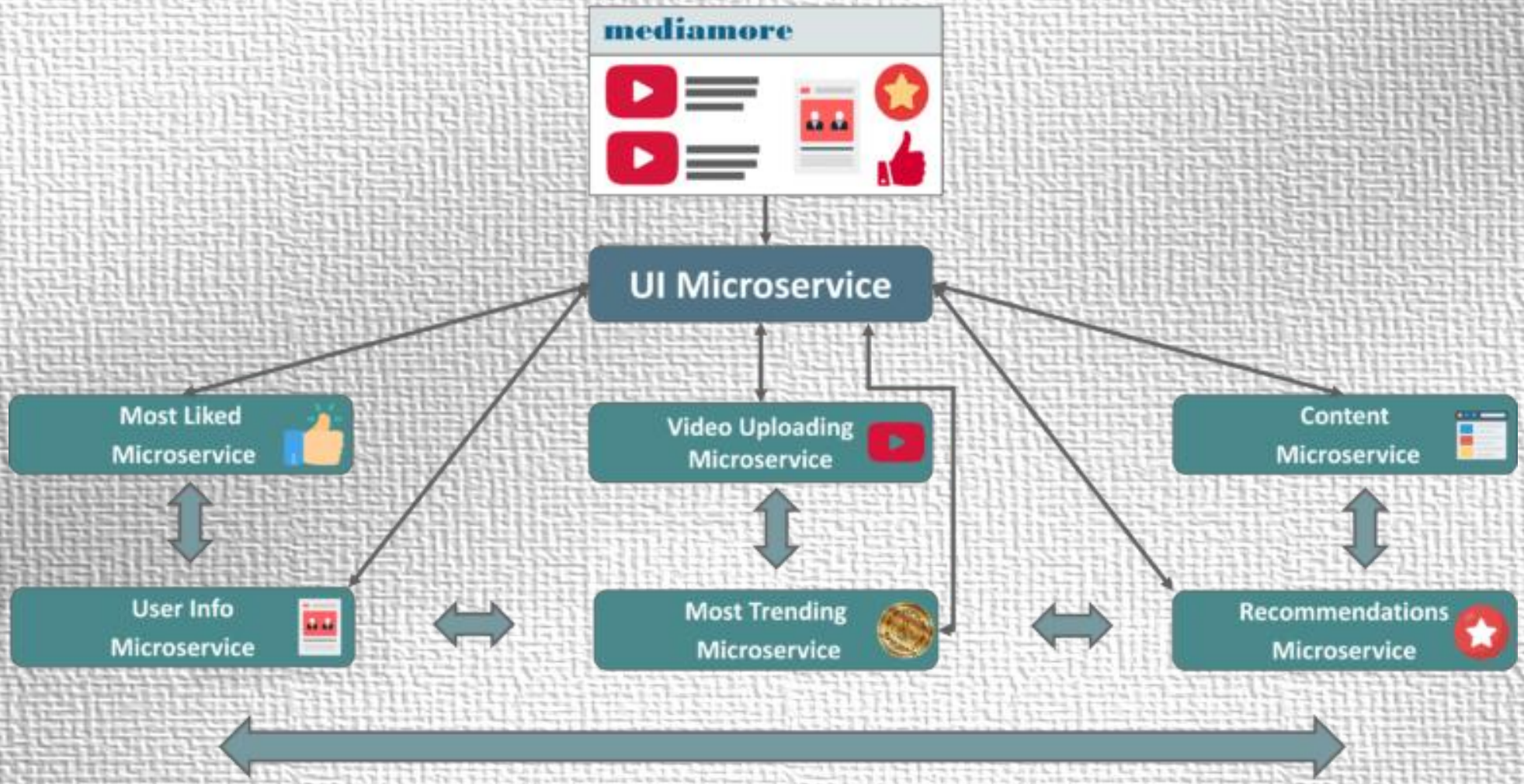
Как разработчики будут работать вместе, чтобы выполнить просьбу Алисы?

Запрос Алисы передается поставщику удостоверений. Таким образом, поставщик удостоверений проверяет подлинность запроса Алисы, идентифицируя ее как обычного пользователя mediamore.

Эти запросы передаются шлюзу API, который действует как точка входа для Алисы, чтобы пересылать ее запросы соответствующим микросервисам.

Каждая функция имеет свой собственный рабочий микросервис, обрабатывающий свои собственные данные. Эти микросервисы также имеют свои собственные балансировщики нагрузки и среды выполнения для правильной работы.

Архитектура микросервисов Mediamore

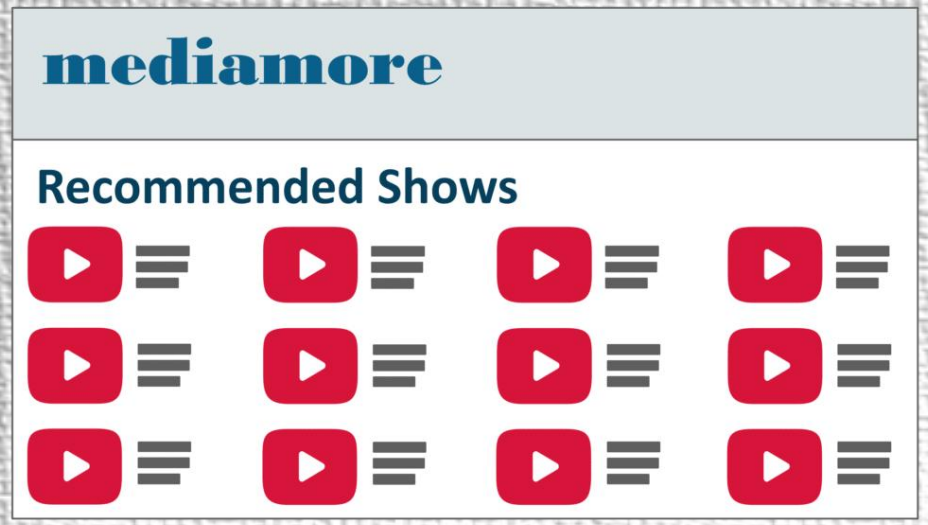


Обратитесь к схеме ниже. Каждый микросервис обрабатывается небольшой гибкой командой, такой как команда контента, команда загрузки видео, самая популярная команда, поисковая команда и т. д.

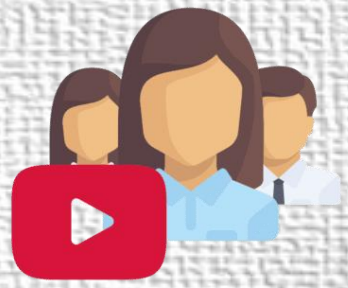
Разделение команд Mediamore

Content Team

Most Trending Team



User Info Team



Video Uploading Team

1. Команда контента состоит из миллионов телешоу, которые предоставляет приложение.
2. Команда загрузки видео несет ответственность за загрузку всего контента в приложение
3. Самая трендовая команда размещает самые трендовые шоу в соответствии с географическим расположением пользователей и так далее.

Эти небольшие команды разработчиков связывают каждый фрагмент контента с метаданными, описывающими искомый контент. Затем метаданные передаются в другую микрослужбу, т.е. функцию поиска, которая обеспечивает захват результатов поиска Алисы в каталог контента.

Затем третья микрослужба, т.е. наиболее популярная микрослужба, захватывает трендовый контент среди всех пользователей mediamore в соответствии с их географическим местоположением.

Содержимое этой микрослужбы — это то, что Алиса видит, когда она впервые входит в mediamore.

Эти индивидуально развертываемые микрослужбы помещаются в определенные контейнеры для присоединения к приложению. Контейнеры используются для доставки кода в сектор, где требуется развертывание.

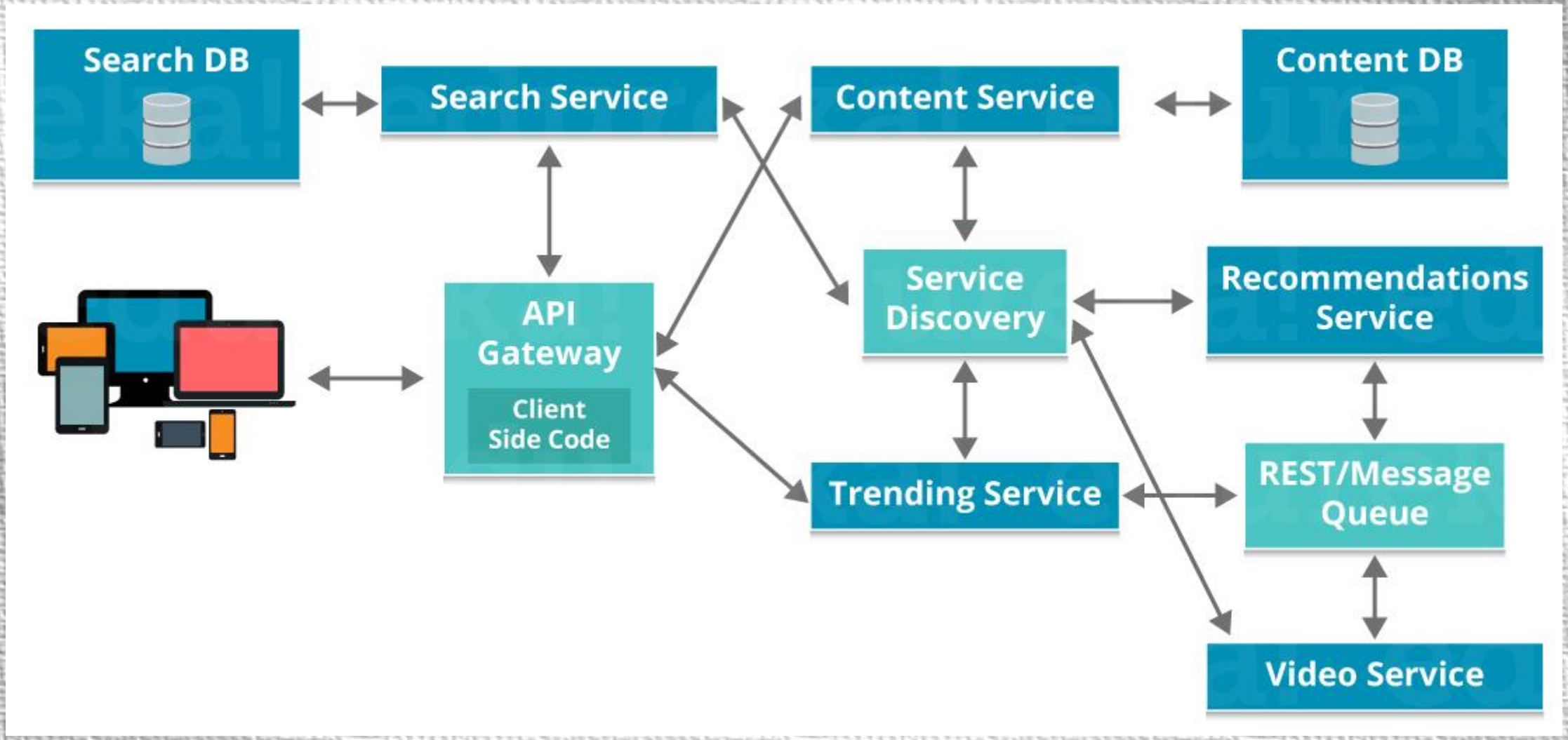
Но прежде чем они присоединятся к приложению для совместной работы, они должны найти друг друга, чтобы выполнить просьбу Алисы.

Как эти микрослужбы находят друг друга?

Микросервисы используют обнаружение сервисов, которое выступает в качестве руководства для поиска маршрута связи между каждым из них. Затем микрослужбы взаимодействуют друг с другом через сервер без отслеживания состояния, то есть либо через HTTP Request/Message Bus.

Эти микрослужбы взаимодействуют друг с другом с помощью **интерфейса прикладной программы (API).** После того, как микрослужбы взаимодействуют внутри себя, они развертывают **статический контент** в облачной службе хранения, которая может доставлять его непосредственно клиентам через **сети доставки контента (CDN).**

Взаимодействие между микрослужбами



Таким образом, когда Алиса ищет свое ТВ-шоу, микрослужба поиска связывается со службой каталога контента в API о том, что ищет Алиса, а затем эти микрослужбы сравнивают введенные слова с метаданными, которые у них уже есть.

После того, как команды разработчиков захватывают наиболее набранные слова Алисы, команда аналитиков обновляет код в микросервисе рекомендаций и сравнивает наиболее просматриваемый контент и предпочтения Алисы с популярным контентом среди других пользователей в том же географическом регионе.

Это означает, что в следующий раз, когда Алиса войдет в приложение, она не только увидит самый популярный контент, но и найдет персонализированный плейлист, который содержит шоу, которые она ранее просматривала.

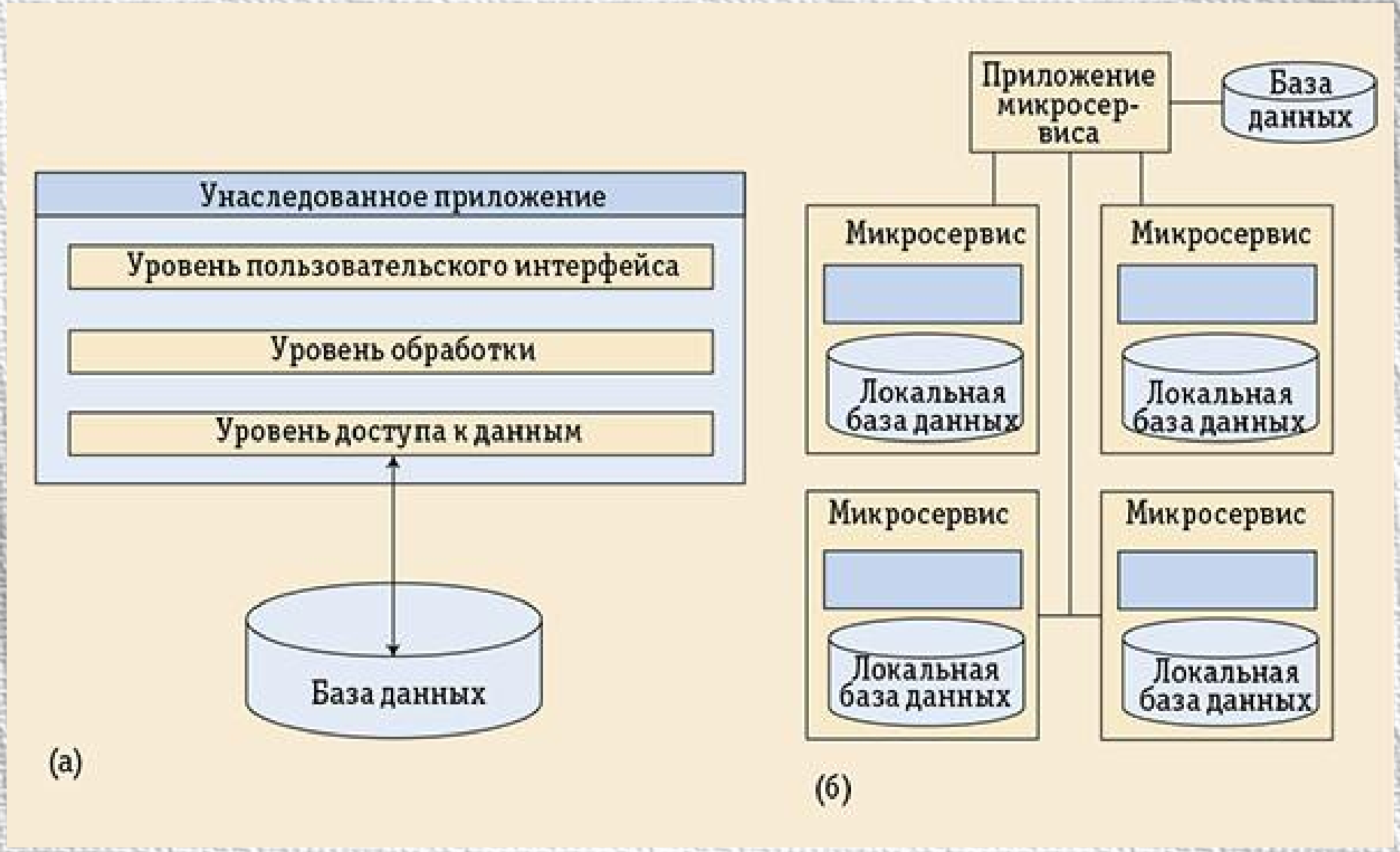
Таким образом, запрос Алисы выполняется командой разработчиков быстро, поскольку им не нужно было снова создавать полное приложение, а просто нужно было обновить код для развертывания этой новой функциональности.

Таким образом, микрослужбы вызывают параллельные среды, чтобы удовлетворить миллионы клиентов с различными интересами.

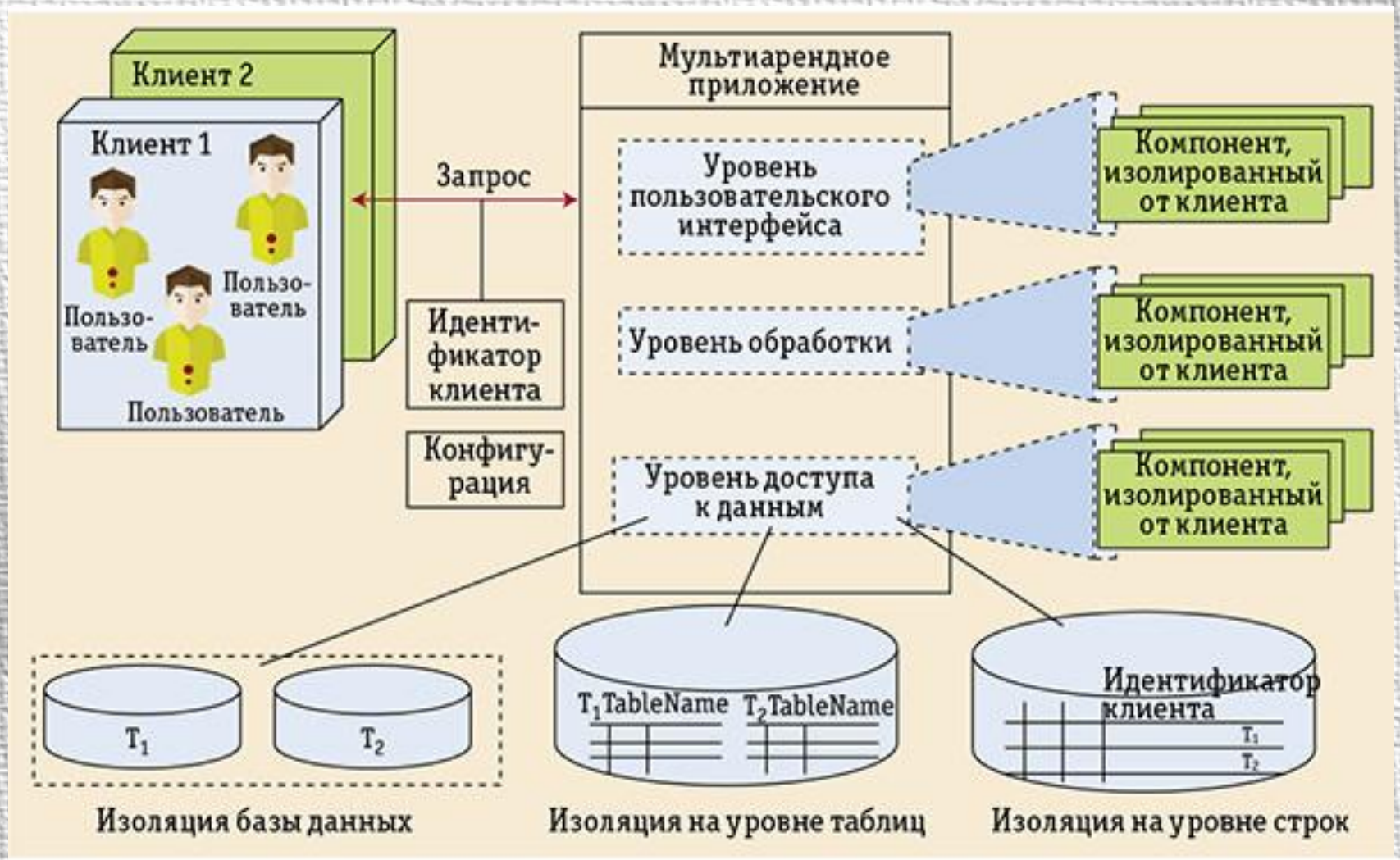
МИКРОСЕРВИСЫ

Модернизация унаследованных корпоративных систем — вопрос, с которым сталкиваются многие организации, планирующие использовать новые облачные технологии для обеспечения высокого уровня масштабируемости и готовности. Если исходный код унаследованной системы доступен и допускает внесение изменений, то микросервисы становятся многообещающим решением, при котором централизованные сервисы заменяются несколькими независимыми

Сравнение (а) унаследованной (монолитной) архитектуры и (б) архитектуры микросервисов



Архитектура мультиарендного корпоративного программного обеспечения, предоставляемого в качестве сервиса



Микросервисы-1

Автоматизацию периода окрестили «лоскутной» или «островной». Пришло понимание, что маленькие автономные программы пригодны для отдельных функций, но последующая интеграция их в единый организм и сопровождение превращаются в настоящий ад даже при согласованных интерфейсах информационного обмена.

Архитектура корпоративных информационных систем быстро сместилась к интегрированным решениям, вначале создававшимся на основе простого разделения общих файлов данных (файл-сервер), затем на базе клиент-серверной технологии, вскоре эволюционировавшей к многозвенной.

Микросервисы-2

1990-е годы – период эволюции двухзвенных клиент-серверных систем к многозвенным. Потому что прикладную логику надо размещать в таком месте, где её можно повторно использовать с минимальными затратами. Архитектура, превращающая СУБД в сервер приложений из хранимых процедур до сих пор успешно используется, но имеет ряд существенных ограничений, не всегда приемлемых. Поэтому более общий подход – использование промежуточных слоев служб.

Микросервисы-3

Подъем из небытия темы микросервисов совпал с увеличением потока проектов миграции инфраструктур служб ИТ в «облака». В самом деле, идеальный компонент для использования в облаке должен быть полностью автономным и не иметь состояния. В таком случае микросервис можно неограниченно масштабировать по горизонтали для обеспечения постоянной готовности и балансировки нагрузки. Почему бы вместе с миграцией не продать клиенту еще и переделку части системы?

Цикл хайпа

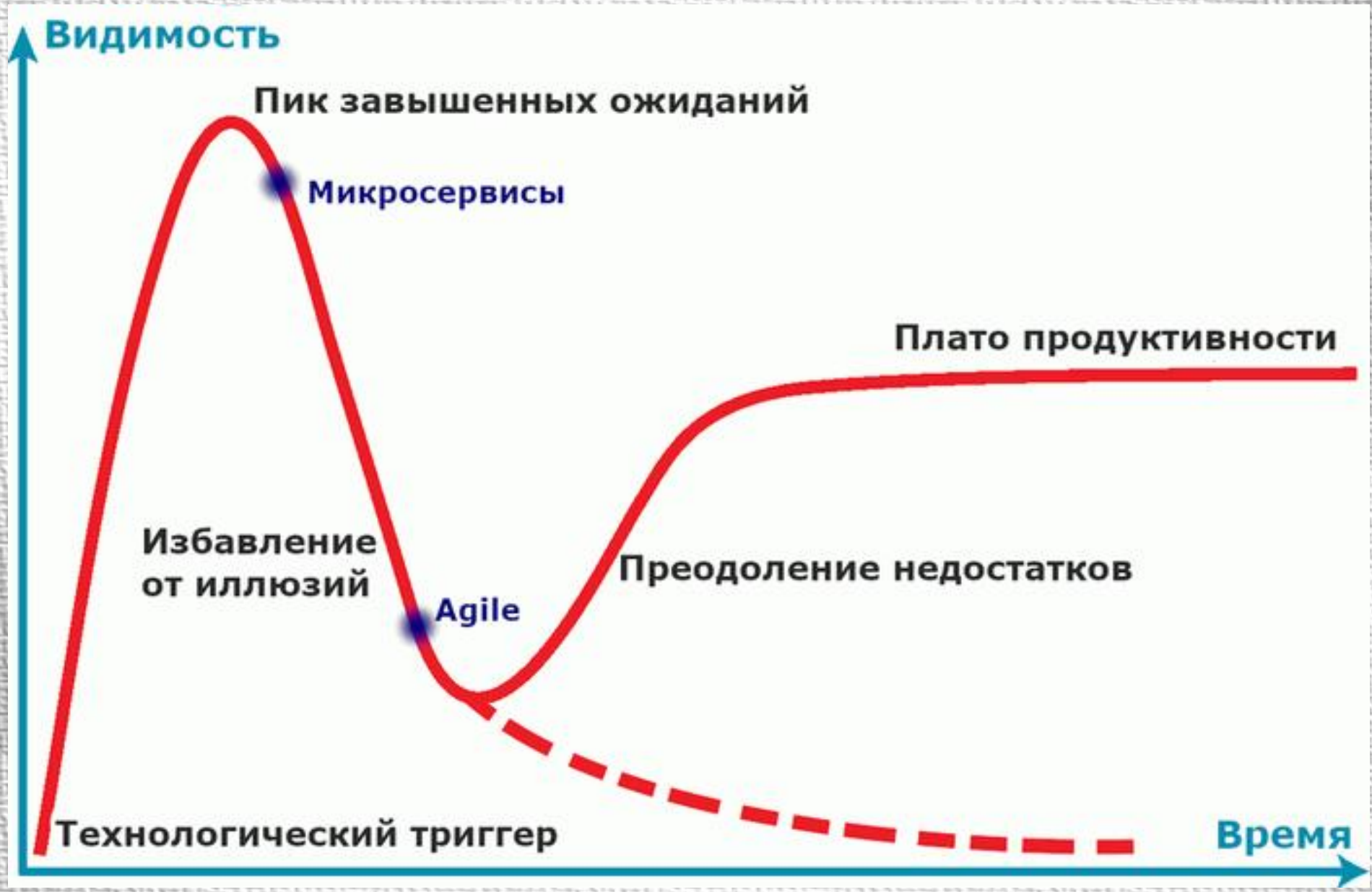
25 лет назад консалтинговая компания Gartner сформулировала «Hype cycle» – график известности, зрелости технологий. На нем представлены этапы, которые проходят технологии от своего появления до устойчивого применения.

Избавление от иллюзий вовсе не гарантирует преодоления недостатков: множество технологий остались забытыми, так и не выйдя на плато продуктивности.

Согласно этому циклу аджайлы опускаются в район точки «избавление от иллюзий», тогда как микросервисы пока еще вблизи пика ожиданий чудес.

Пять этапов

Gartner's «Hype cycle»
Ожидания
Пик чрезмерных ожиданий
Плато продуктивности
Склон просветления
Дно разочарований
Триггер инновации
Время
Сергей Дубовик
sdubovik.ru


Видимость
Пик завышенных ожиданий
Микросервисы
Плато продуктивности
Избавление
от иллюзий
Agile
Преодоление недостатков
Технологический триггер
Время

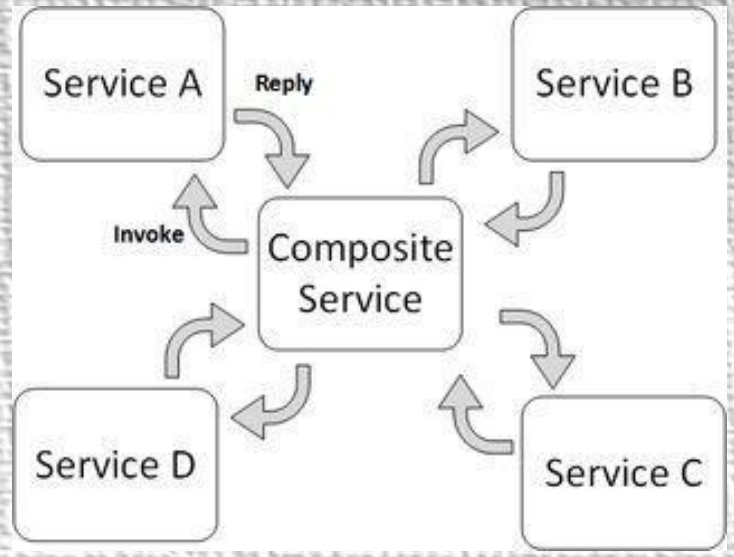
Способы организации Микросервисной Архитектуры

Существует 2 способа организации МСА:

1. Оркестрация
2. Хореография

Оркестрация сервисов

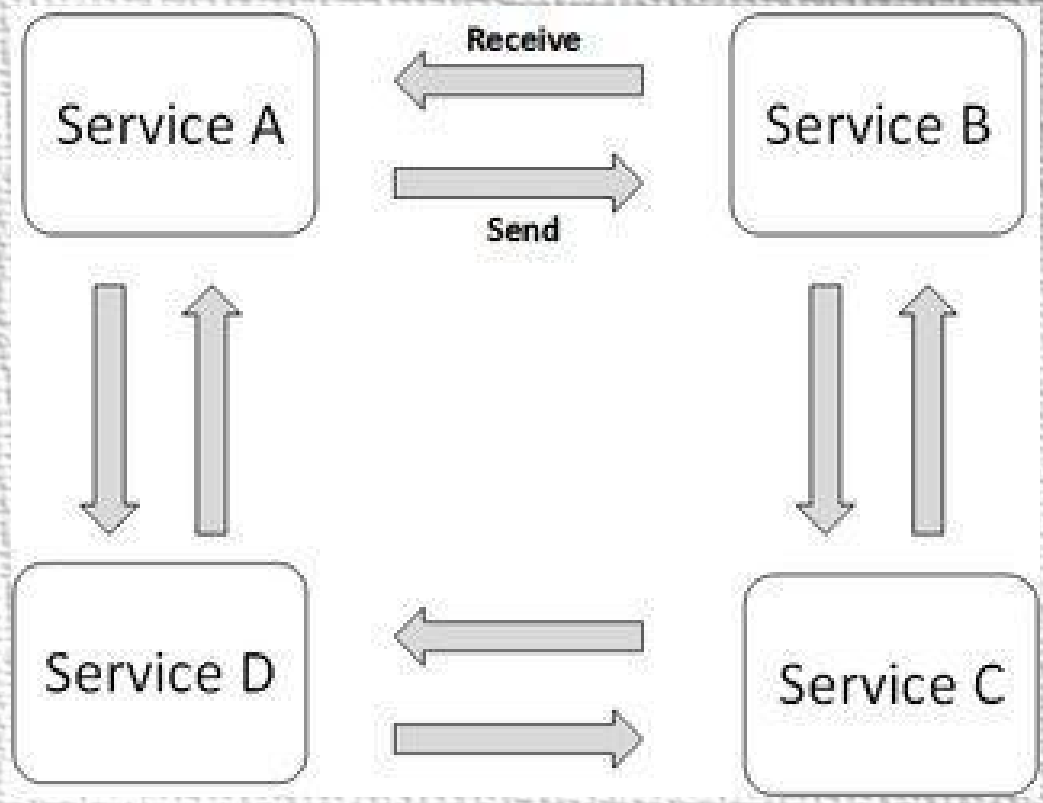
Оркестрация служб представляет собой единый централизованный исполняемый бизнес-процесс (оркестратор), который координирует взаимодействие между различными службами (например, ***Camunda***). Оркестратор отвечает за вызов и объединение служб.



Отношения между всеми участвующими службами описываются одной конечной точкой (т. е. составной службой). Оркестрация включает в себя управление транзакциями между отдельными службами и использует централизованный подход к составлению сервисов.

Хореография сервисов

Хореография служб - это глобальное описание участвующих служб, которое определяется обменом сообщениями, правилами взаимодействия и соглашениями между двумя или более конечными точками. Хореография использует децентрализованный подход к составлению сервисов.



Хореография описывает взаимодействие между несколькими службами, а оркестровка представляет контроль с точки зрения одной стороны. Это означает, что ***хореография*** *отличается* от ***оркестрации*** в отношении того, где должна находиться логика, управляющая взаимодействиями между задействованными сервисами.

✚Преимущества

- Предлагает слабую связанность благодаря высокой степени изоляции.
- Повышает модульность.
- Сбой в одном сервисе не затронет всю систему, поскольку они изолированы.
- Предлагает высокую гибкость и масштабируемость.
- Простота модификации может ускорить итерации.
- Позволяет реализовать улучшенную систему обработки ошибок.
- Решает проблемы с потоками данных, которые бывают у многослойной архитектуры.

– Недостатки

- Повышенный риск сбоя при обмене данными между сервисами.
- Большим количеством сервисов трудно управлять.
- Требует решения таких проблем, как задержки в сети, балансировка нагрузки и прочих трудностей, свойственных распределенной архитектуре.
- Нуждается в комплексном тестировании в распределенной среде.
- На реализацию потребуется гораздо больше времени.

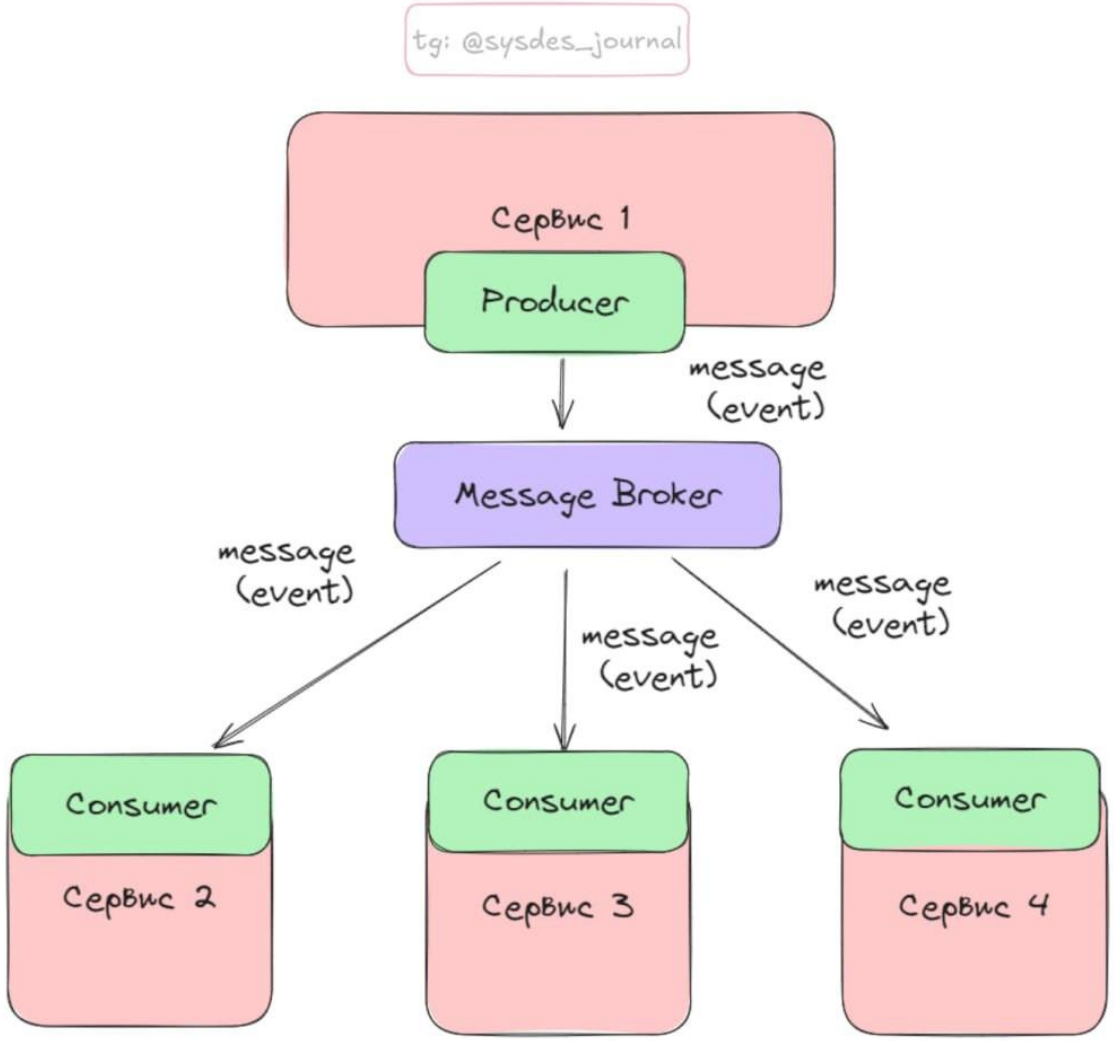
СОБЫТИЙНО-ОРИЕНТИРОВАННАЯ АРХИТЕКТУРА (EVENT-DRIVEN ARCHITECTURE, EDA)

Событийно-ориентированная архитектура (Event-driven architecture, EDA), является популярным стилем распределенной асинхронной архитектуры, которая используется для создания высокопроизводительных масштабируемых приложений.

Асинхронный обмен данными может быть весьма эффективным приемом сокращения времени отклика системы.

Событийно-ориентированная архитектура (архитектура, управляемая событиями) (EDA)

tg: @sysdes_journal



Компоненты EDA

Событийно- ориентированная архитектура состоит из отдельных компонентов, которые асинхронно получают и обрабатывают события. Она может использоваться как сама по себе, так и встраиваться в другие архитектурные стили.

Компоненты событийно- ориентированной архитектуры:

1. **Событие** (event) — создание новой сущности или изменение состояния существующей.
2. **Производитель события** (producer) — сервис, который создаёт событие.

.

3. **Потребитель события** (consumer) — сервис, который получает событие и обрабатывает его, после чего порождается новое событие — результат обработки события.
4. **Брокер сообщений** (message broker) — система, обеспечивающая доставку события от производителя до потребителя.

Любой сервис может выполнять как роль производителя, так и роль потребителя событий.

Существуют две модели доставки событий:

1. **Производитель/Подписчик** (Publisher/Subscriber): производитель отправляет сообщения брокеру, брокер рассылает сообщения подписчикам, после отправки сообщения удаляются. Самый часто используемый пример такого брокера – RabbitMQ.
2. **Потоковая передача**: производитель отправляет сообщения брокеру, брокер сохраняет сообщения, потребители сами по готовности считывают сообщения из брокера. События не удаляются брокером, а хранятся в течение настраиваемого промежутка времени. Самый часто используемый пример такого брокера – Apache Kafka.

В событийно-ориентированной архитектуре, используются две основные топологии: **медиатор** (mediator) и **брокер** (broker).

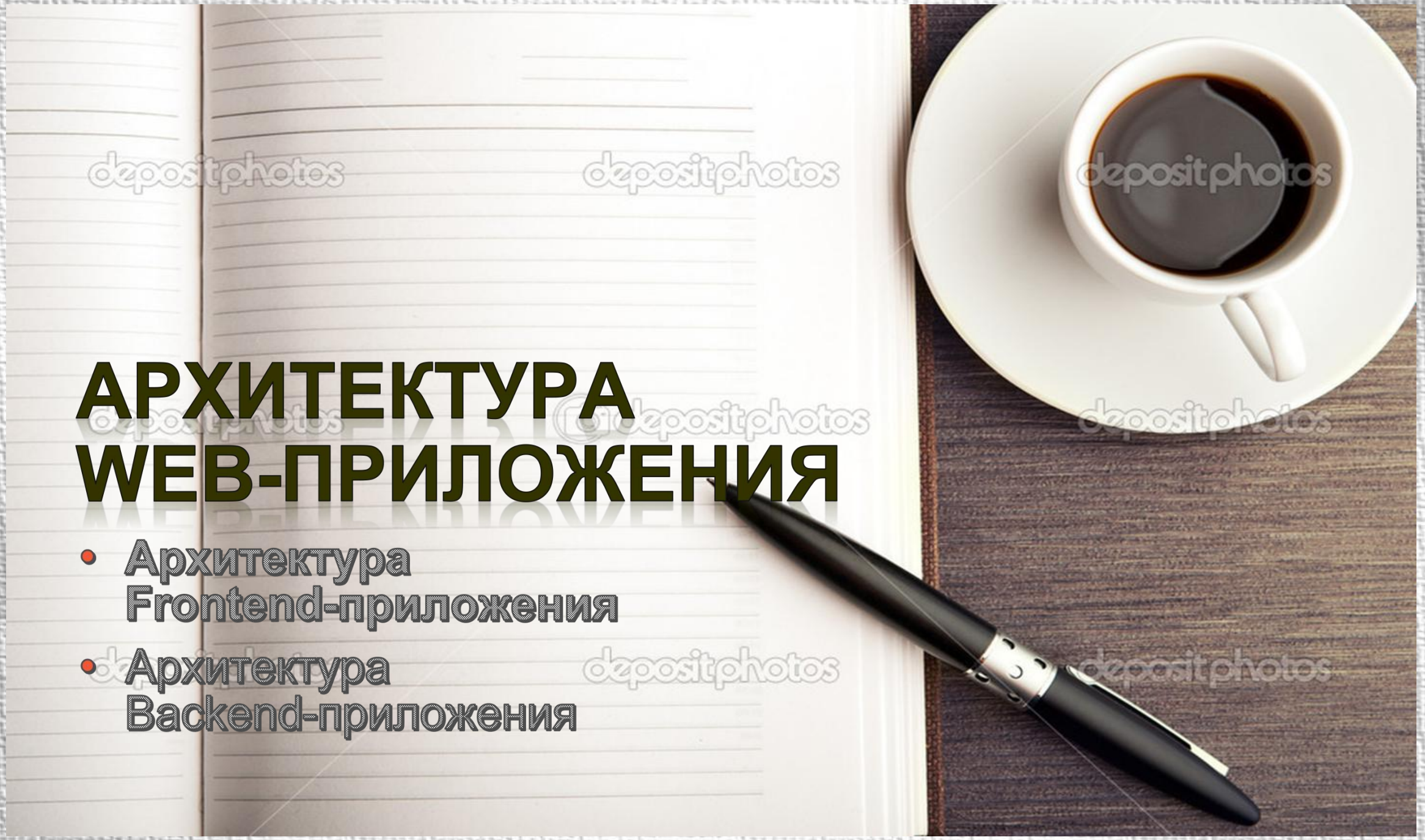
Отличие топологии брокера от топологии медиатора в том, что обработчики событий в топологии медиатора не объявляют всей системе о своих действиях, а рассылают сообщения в режиме «точка- точка», в то время, как в топологии брокера рассылку сообщений можно назвать «широковещательной» — все подписчики получат сообщение, но прочитает только тот, кому оно предназначалось.

+ Преимущества EDA

- слабая связанность сервисов, взаимодействующих исключительно через события;
- высокая скорость и параллельная обработка событий;
- высокая доступность и отказоустойчивость в виду возможности реплицирования/резервирования сервисов.

— Недостатки EDA

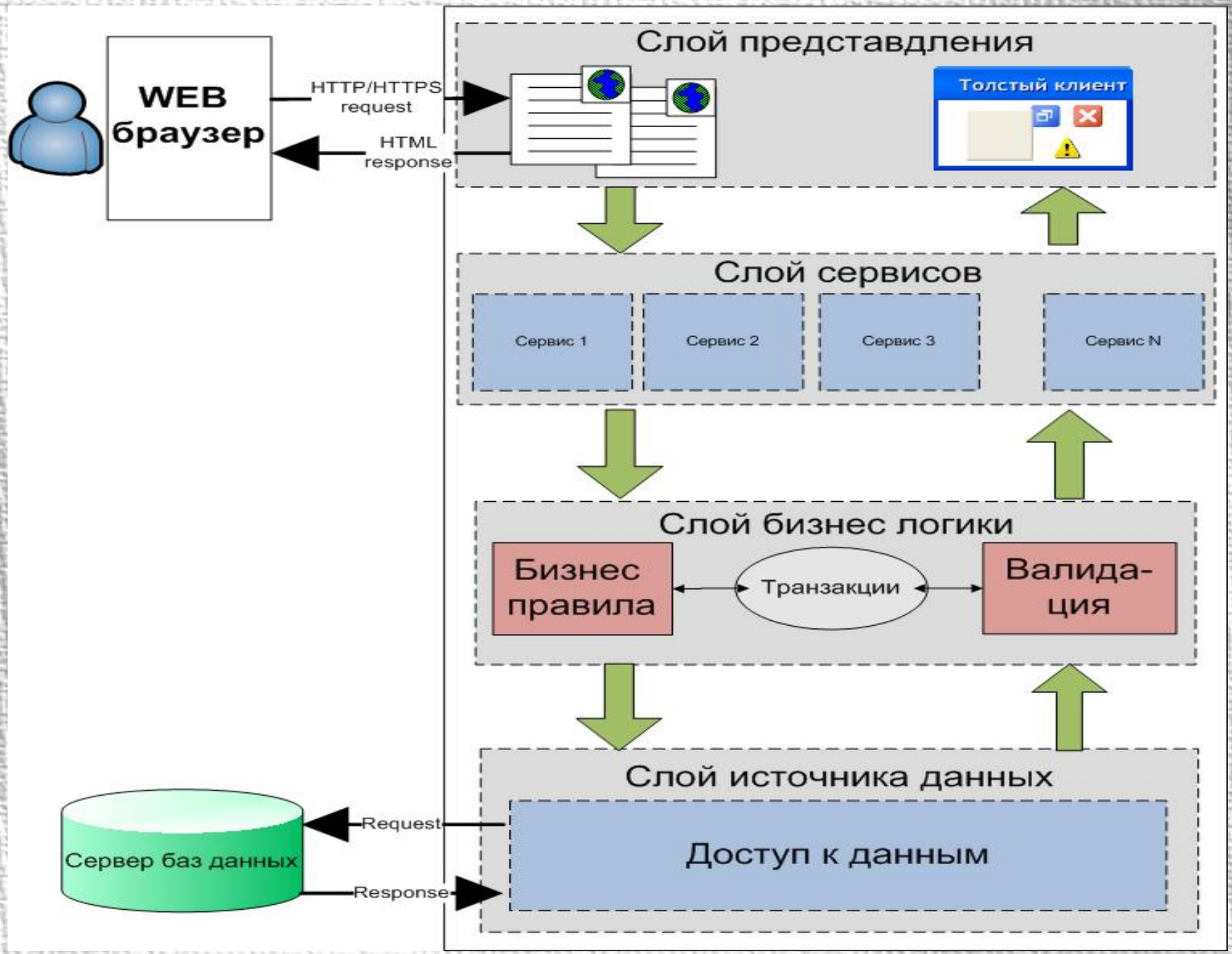
- сложность разработки, развертывания и тестирования;
- единая точка отказа в виде брокера сообщений;
- высокие затраты на инфраструктуру в виду высокой производительности решения.

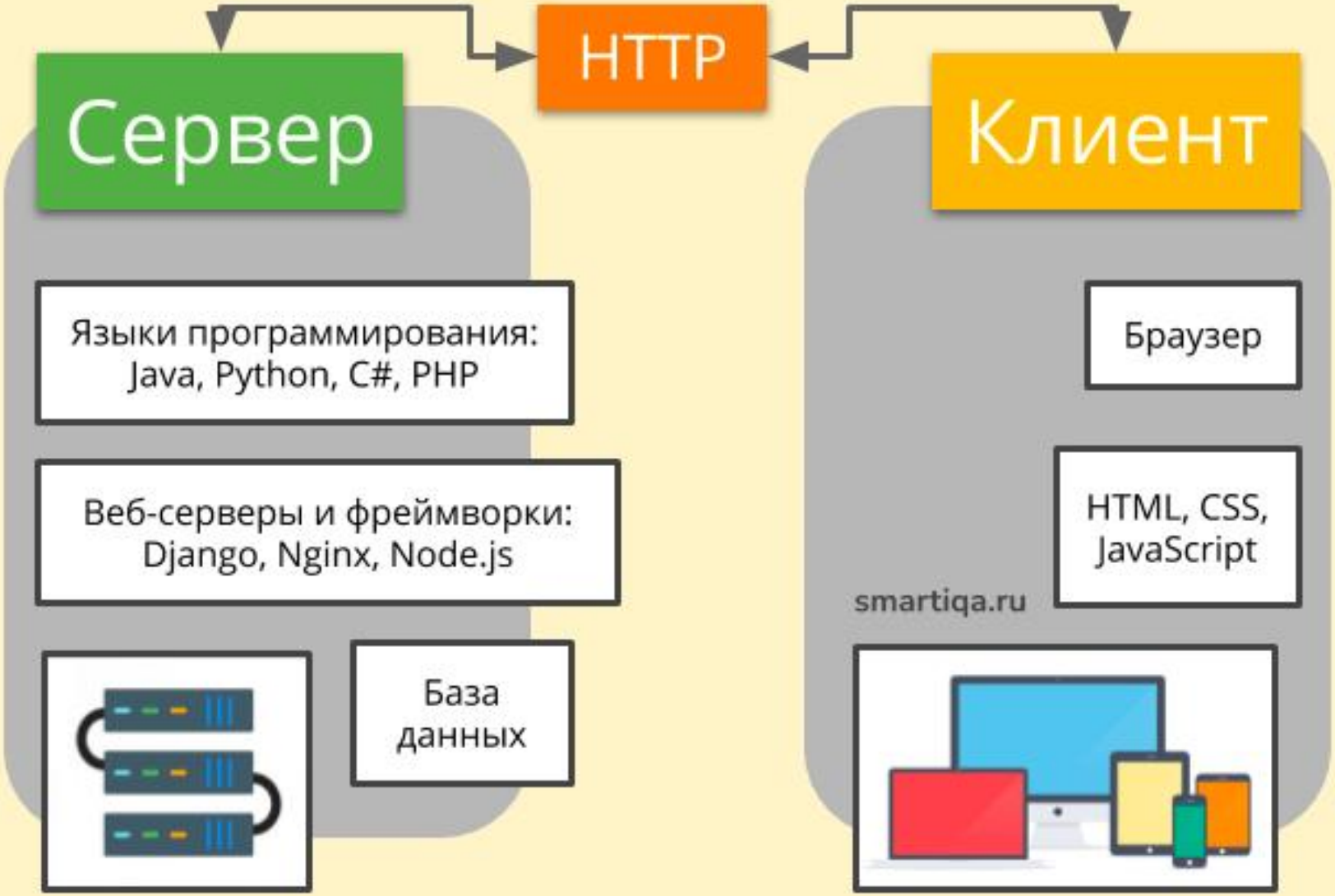


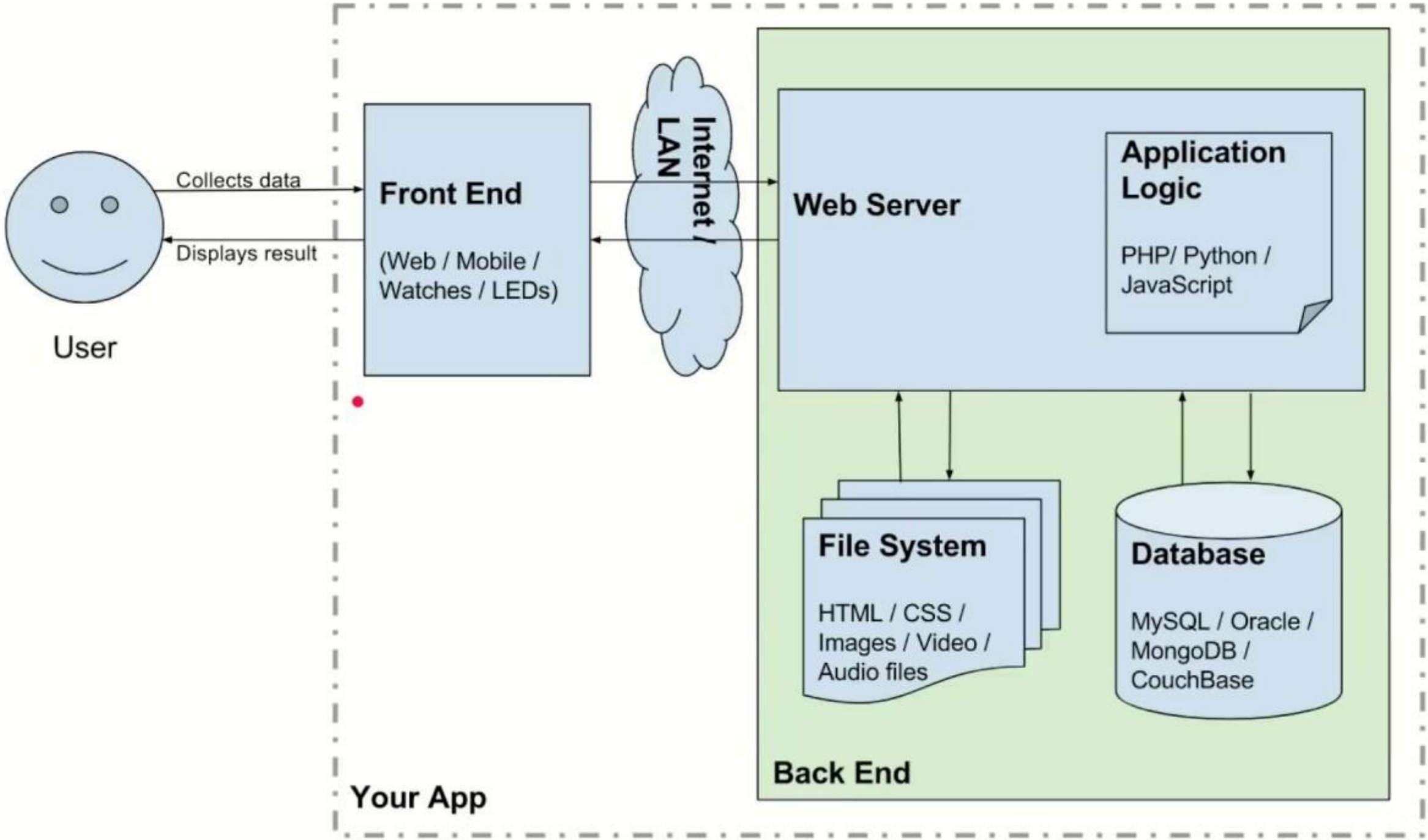
АРХИТЕКТУРА WEB-ПРИЛОЖЕНИЯ

- Архитектура Frontend-приложения
- Архитектура Backend-приложения

Типовая архитектура веб-приложения




HTTP
Сервер
Клиент
Языки программирования:
Java, Python, C#, PHP
Веб-серверы и фреймворки:
Django, Nginx, Node.js
База
данных
Браузер
HTML, CSS,
JavaScript
smartiqa.ru

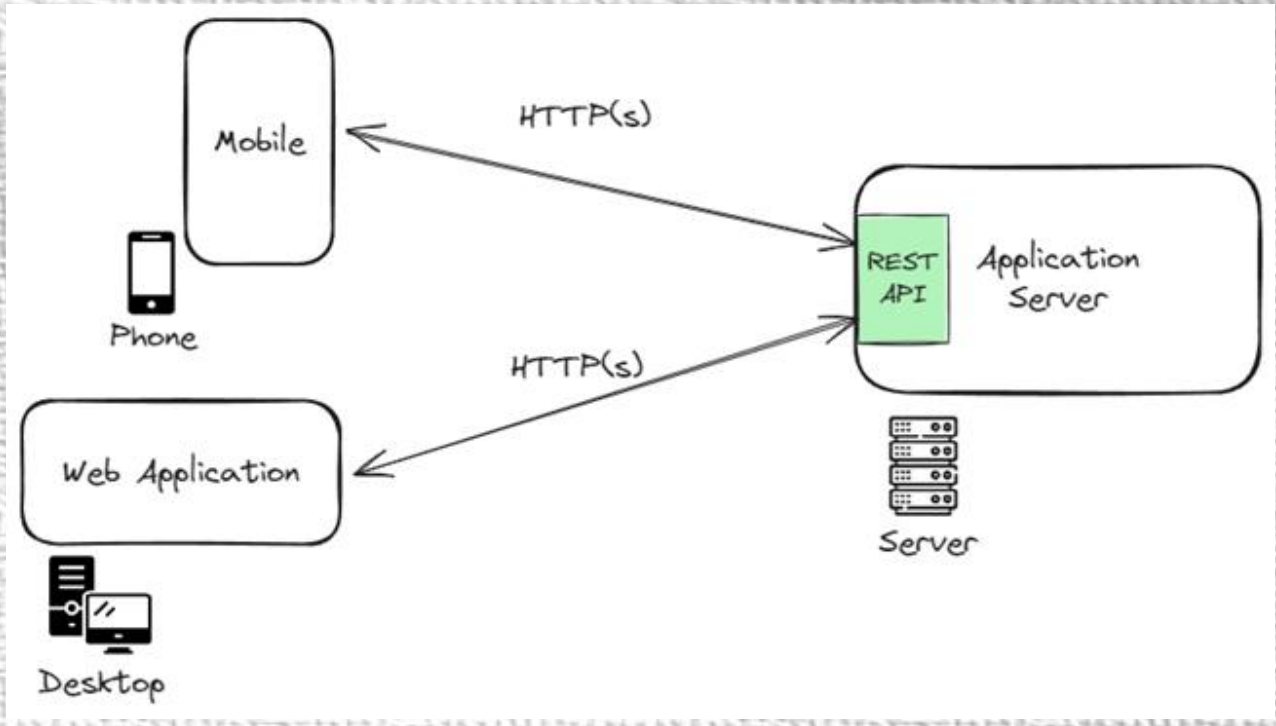

Collects data
Displays result
User
Front End
(Web / Mobile / Watches / LEDs)
Internet / LAN
Web Server
Application Logic
PHP/ Python / JavaScript
File System
HTML / CSS / Images / Video / Audio files
Database
MySQL / Oracle / MongoDB / CouchBase
Back End
Your App

АРХИТЕКТУРА BACKEND-ПРИЛОЖЕНИЯ

В современном мире, где объемы данных растут с каждым днем, масштабирование высоконагруженных приложений становится необходимостью. Приложения, которые обрабатывают большие нагрузки, должны быть способными эффективно масштабироваться, чтобы обеспечить непрерывную работу и высокую производительность.

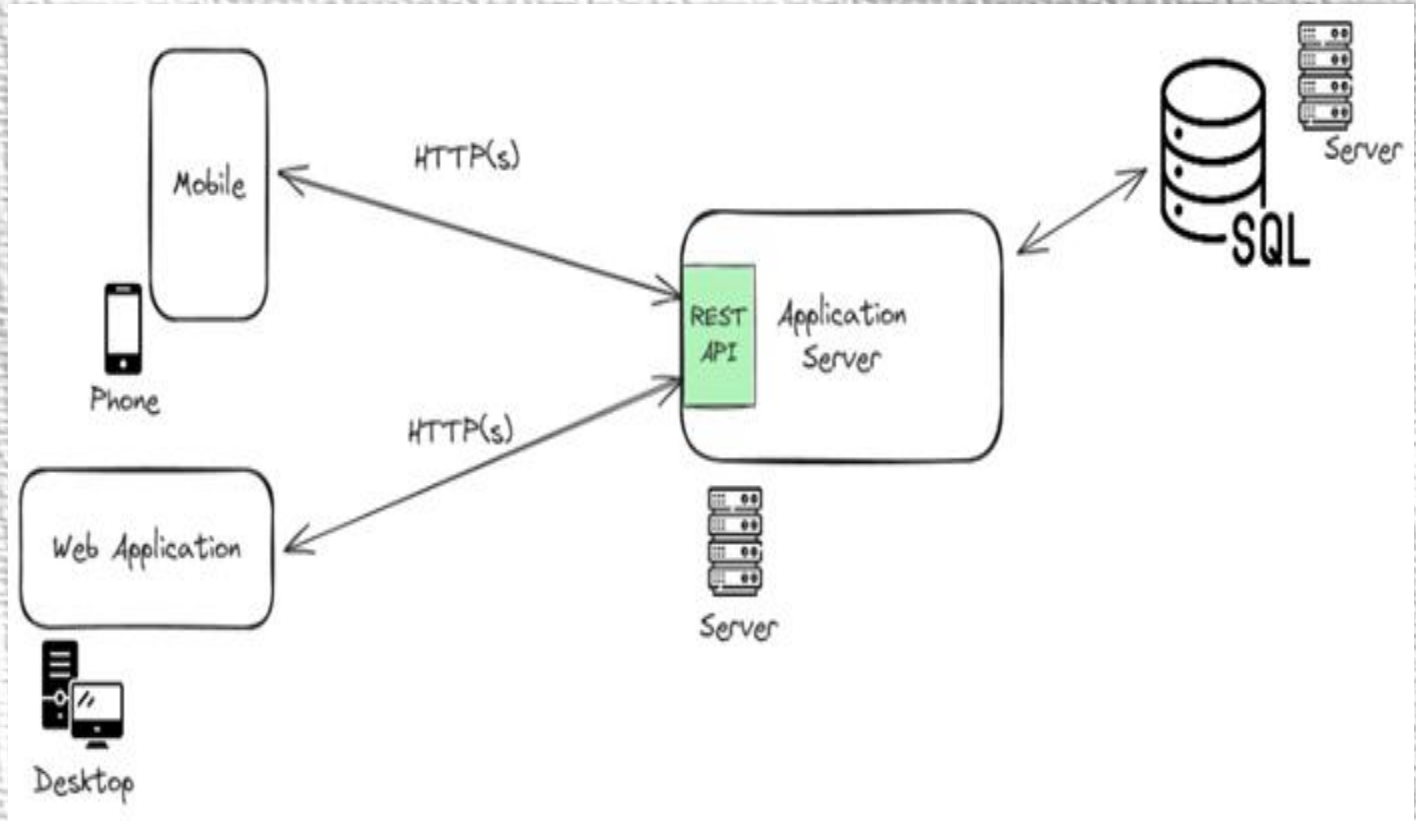
1. Простейший вариант клиент-серверной архитектуры

Рассмотрим вариант клиент-серверной архитектуры, в которой у нас есть сервер (Application Server) с монолитным приложением и два клиента, общающиеся с приложением по REST API.



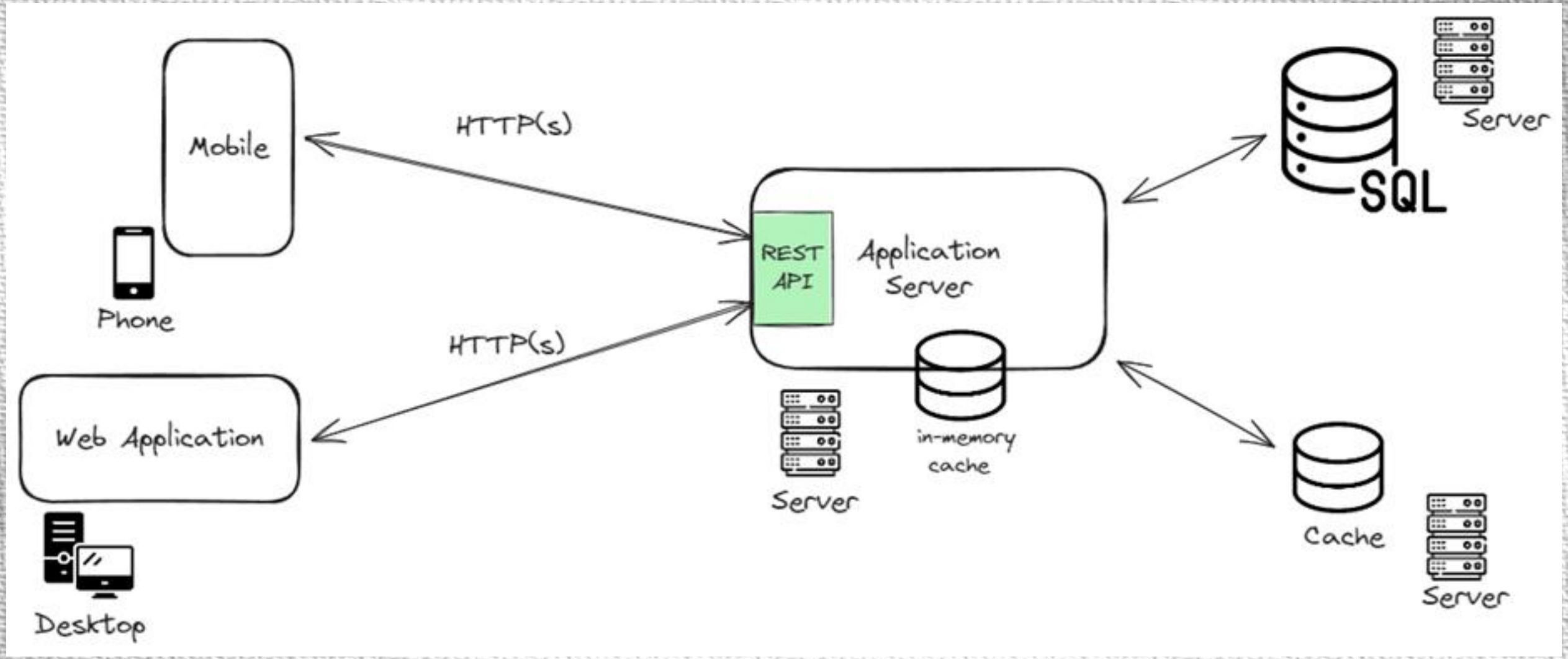
2. Добавляем БД для хранения данных

Для того, чтобы хранить данные приложению необходимо иметь подключение к базе данных. В данном случае чаще всего для хранения структурированных данных используют реляционные БД.



3. Добавляем кэш для хранения часто запрашиваемых и редко изменяемых данных

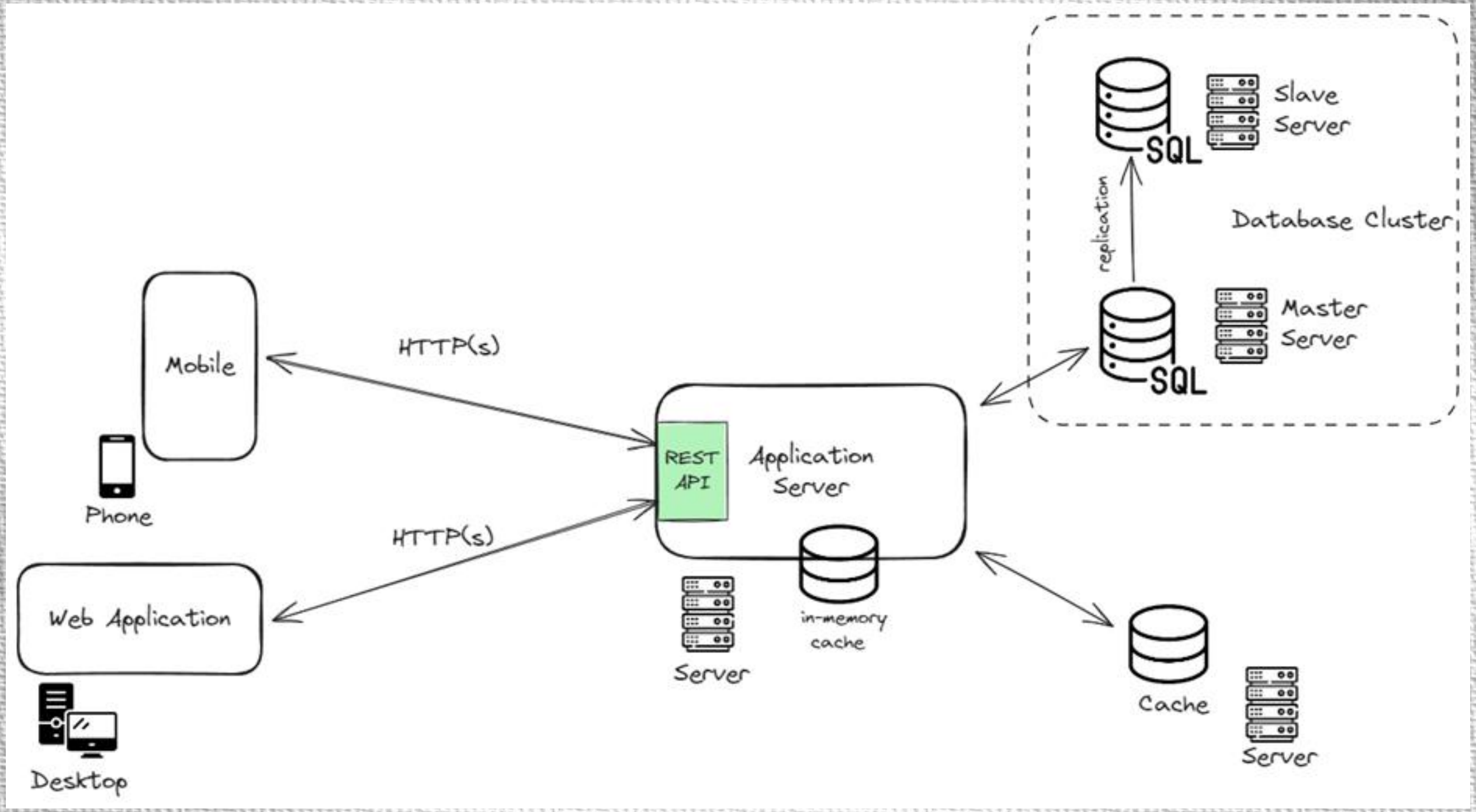
С ростом количества запросов к приложению, необходимо предусмотреть кэш, чтобы сократить время отклика приложения при частых запросах одних и тех же редко изменяемых данных. Так как БД хранит данные на жестком диске, то для ускорения доступа к ним логично хранить данные в оперативной памяти на локальном сервере (in-memory cache) или на отдельном сервере с помощью in-memory database.


Mobile
Phone
HTTP(s)
HTTP(s)
Web Application
Desktop
REST API
Application Server
Server
in-memory cache
SQL
Server
Cache
Server

4. Масштабируем БД

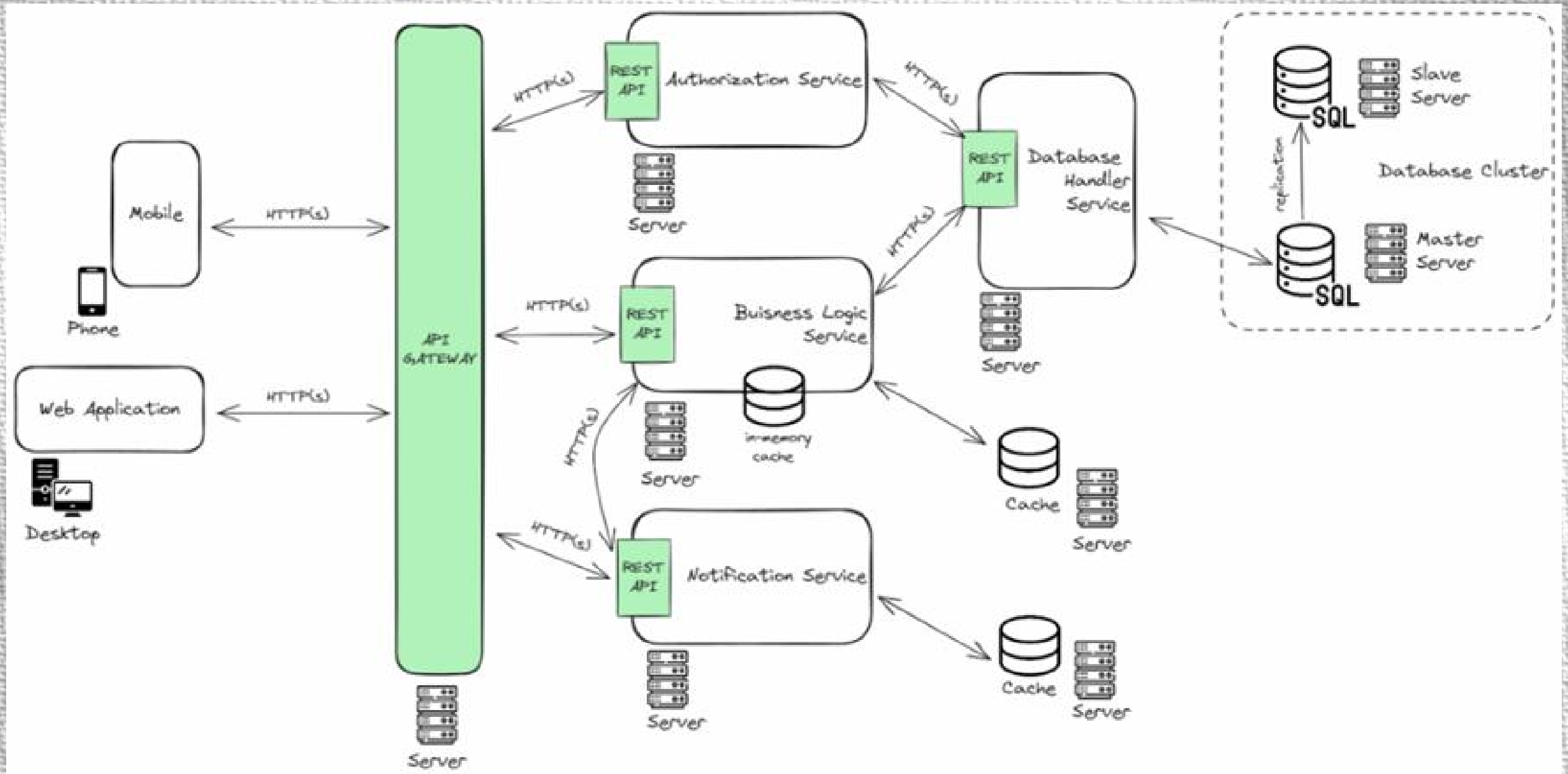
Для обеспечения отказоустойчивости приложения, хранения и доступа к наиболее критичным данным, необходимо использовать подходы к масштабированию БД, один из которых – репликация.

С этой целью БД объединяются в кластер (Database Cluster). Объявляется Master Server, в случае выхода из строя которого основную нагрузку до его восстановления принимает Slave сервер. Данные реплицируются из Master-ноды по всем Slave-нодам.


Slave
Server
SQL
replication
Database Cluster
Master
Server
SQL
Mobile
HTTP(s)
Phone
REST
API
Application
Server
HTTP(s)
Web Application
Server
in-memory
cache
Cache
Server
Desktop

5. Переходим к микросервисной архитектуре

С ростом нагрузки возникает необходимость перейти к микросервисной архитектуре. Монолитное приложение разбивается на сервисы (каждый из которых выполняет определенный кусок логики внутри бизнес-контекста). Для маршрутизации запросов от клиентских устройств выделяется отдельный слой (приложение) API Gateway, который помимо маршрутизации запросов между сервисами выполняет набор дополнительных функций (о которых будет сказано далее). Если к БД обращаются несколько микросервисов, то рекомендуется выделение отдельного сервиса (handler) для работы с этой БД.

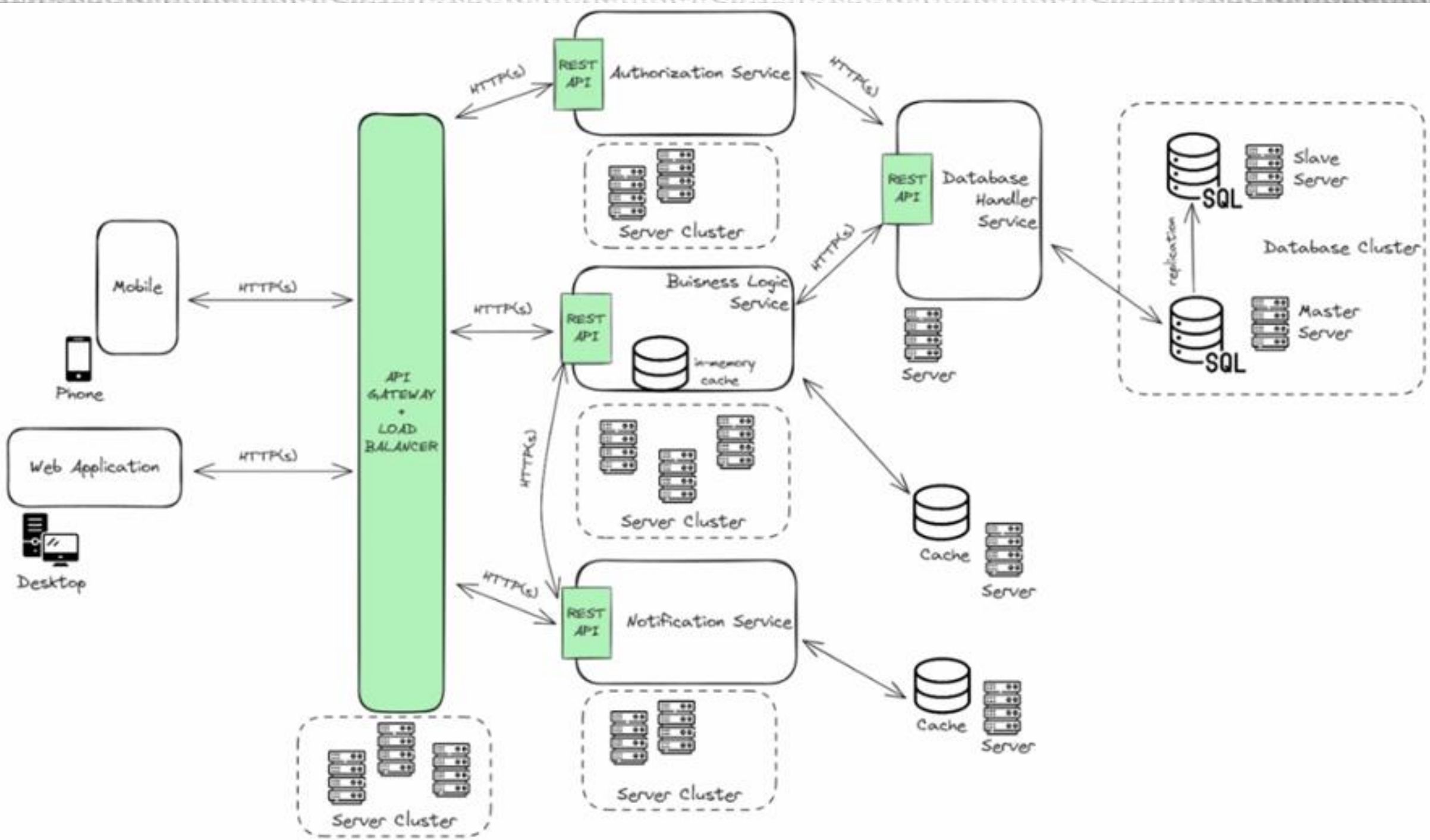

Mobile
Phone
Web Application
Desktop
HTTP(s)
HTTP(s)
API GATEWAY
Server
HTTP(s)
REST API
Authorization Service
Server
HTTP(s)
REST API
Buisness Logic Service
in-memory cache
Server
HTTP(s)
REST API
Notification Service
Server
HTTP(s)
REST API
Database Handler Service
Server
Cache
Server
Cache
Server
SQL
Slave Server
replication
Database Cluster
SQL
Master Server

6. Масштабируем приложение

Рассмотрим **масштабирование приложений**.

Для того, чтобы обрабатывать возросшее количество запросов можно добавить **реплики** (экземпляров) каждого приложения. Таким образом вырастет и количество серверов, которые объединяются в кластера (Server Cluster).

Для **балансировки трафика** между кластерами серверов и экземплярами сервисов используется API Gateway с его функционалом load balancing.


Mobile
Phone
Web Application
Desktop
HTTP(s)
HTTP(s)
API GATEWAY + LOAD BALANCER
Server Cluster
HTTP(s)
REST API
Authorization Service
Server Cluster
HTTP(s)
REST API
Buisness Logic Service
in-memory cache
Server Cluster
HTTP(s)
REST API
Notification Service
Server Cluster
HTTP(s)
HTTP(s)
REST API
Database Handler Service
Server
Cache
Server
Cache
Server
SQL
Slave Server
replication
Database Cluster
SQL
Master Server

7. Добавляем асинхронное взаимодействие между сервисами

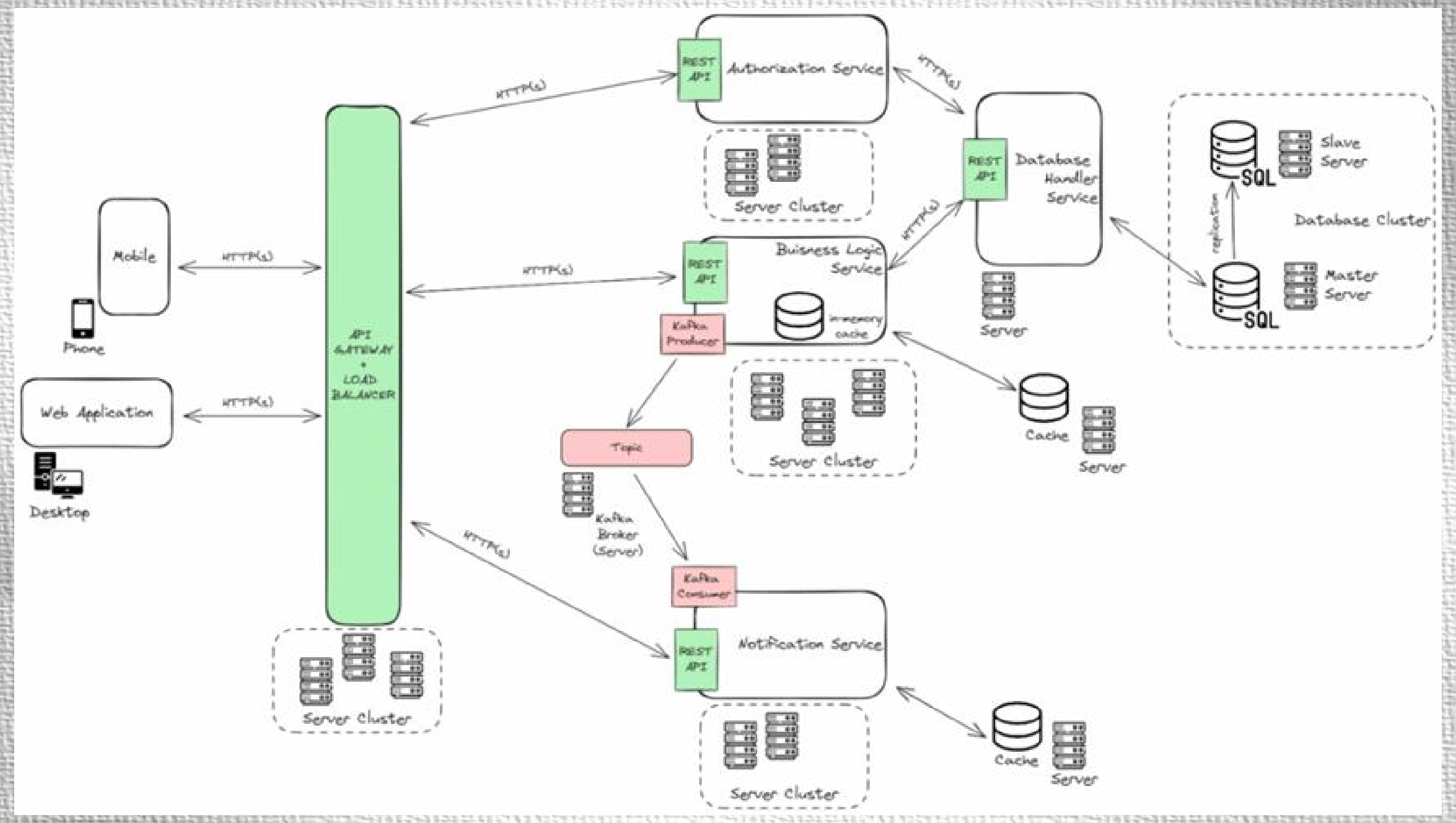
При росте нагрузки синхронный подход (REST API) к обмену сообщениями начнет давать все большую задержку, вызванную тем, что на время установления соединения все ресурсы сервиса будут заняты.

К тому же в случае разрыва соединения или отказа сервиса часть сообщений может быть потеряна.

Для избегания этих ситуаций переходят к асинхронному взаимодействию. Одним из самых распространенных подходов является использование брокеров сообщений (Kafka, RabbitMQ).

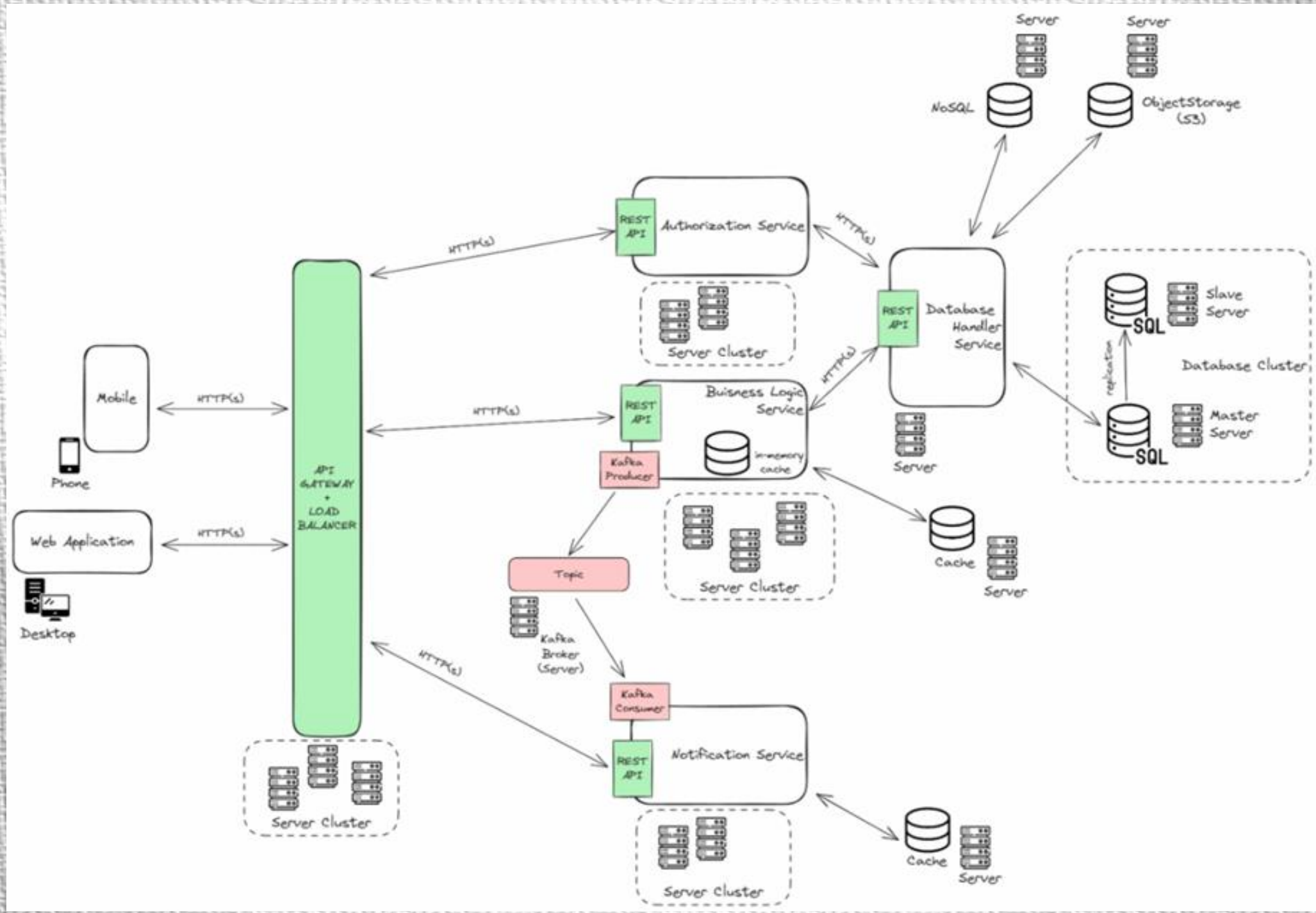
В случае с Kafka, сервис, передающий сообщение, записывает его в соответствующий то-пик (очередь сообщений), из которого это сообщение вычитывается другим сервисом.

Таким образом обеспечивается слабая связанность сервисов и гарантия доставки сообщений.


REST API
Authorization Service
HTTP(s)
Server Cluster
Mobile
Phone
Web Application
Desktop
API GATEWAY + LOAD BALANCER
Server Cluster
REST API
Database Handler Service
Server
SQL
Slave Server
Database Cluster
replication
Master Server
SQL
REST API
Buisness Logic Service
Kafka Producer
in-memory cache
Server Cluster
Cache
Server
Topic
Kafka Broker (Server)
Kafka Consumer
REST API
Notification Service
Server Cluster
Cache
Server

8. Храним неструктурированные данные

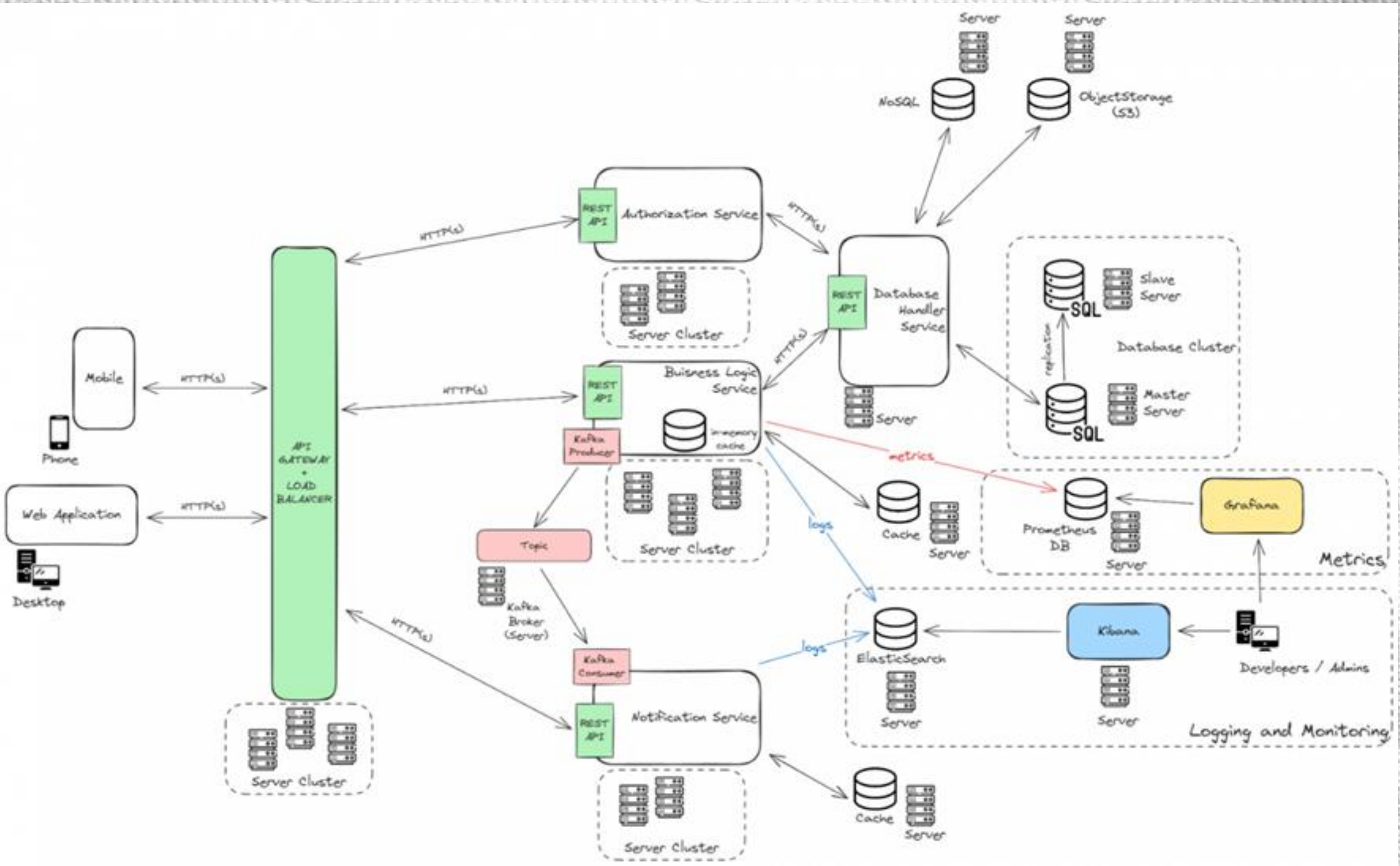
Для хранения неструктурированных данных (временных рядов, документов JSON или файлов) более производительным решением по сравнению с реляционными базами данных будут noSQL БД и объектные хранилища S3.


Server
Server
NoSQL
ObjectStorage
(S3)
REST API
Authorization Service
HTTP(s)
Server Cluster
REST API
Database Handler Service
Server
SQL
Slave Server
replication
Database Cluster
SQL
Master Server
Mobile
Phone
HTTP(s)
API GATEWAY + LOAD BALANCER
REST API
Buisness Logic Service
Kafka Producer
in-memory cache
Server Cluster
Cache
Server
Web Application
Desktop
Topic
Kafka Broker (Server)
Kafka Consumer
REST API
Notification Service
Server Cluster
Cache
Server
Server Cluster

9. Обеспечиваем наблюдаемость

Для обеспечения наблюдаемости сервисов необходимо использовать системы мониторинга, сбора логов и метрик.

Самыми популярными решениями является использование связки ElasticSearch (БД) + Kibana (графический интерфейс) и/или PrometheusDB (БД) + Grafana (графический интерфейс).


Server
Server
NoSQL
ObjectStorage (S3)
REST API
Authorization Service
Server Cluster
HTTP(s)
Mobile
Phone
Web Application
Desktop
API GATEWAY + LOAD BALANCER
Server Cluster
REST API
Database Handler Service
Server
Slave Server
SQL
Database Cluster
replication
Master Server
SQL
REST API
Buisness Logic Service
in-memory cache
Kafka Producer
Server Cluster
metrics
Cache
Server
Prometheus DB
Server
Grafana
Metrics
Topic
Kafka Broker (Server)
logs
ElasticSearch
Server
Kibana
Server
Developers / Admins
Logging and Monitoring
Kafka Consumer
REST API
Notification Service
Server Cluster
Cache
Server

- Шаблоны организации бизнес-логики
- Шаблоны организации источников данных
- Шаблоны представления

ТИПОВЫЕ РЕШЕНИЯ (по Фаулеру)

Типовые решения можно сравнить с полуфабрикатом: чтобы получить удовольствие от еды, вам придется довершить приготовление блюда по собственному рецепту. Используя типовое решение, я “поджариваю” его всякий раз по-другому и наслаждаюсь отменным и своеобразным вкусом.

Фаулер

Основа типового решения — подход, достаточно общий и эффективный для того, чтобы обеспечить преодоление периодически возникающих проблем определенной природы. Типовое решение можно воспринимать и в виде некоторой рекомендации: искусство создания типовых решений состоит в вычленении и кристаллизации таких относительно независимых рекомендаций, которые допускают возможность более или менее обособленного применения.

Структура типовых решений

- **Название решения**
- **Аннотация и эскиз.**

  Аннотация состоит из одного-двух предложений и содержит краткое описание типового решения; эскиз служит визуальной иллюстрацией решения и часто (хотя не всегда) представляет собой UML диаграмму.
- **Проблема**, которая стимулировала появление типового решения. Если таких проблем несколько, выбирается более характерная.

- **Принцип действия.** Освещаются особенности реализации решения.
- **Назначение.** Содержит сведения о том, при каких обстоятельствах следует применять типовое решение, а также о его сравнительных преимуществах и недостатках.

ТИПОВЫЕ РЕШЕНИЯ (Шаблоны)

1. Шаблоны организации бизнес-логики
2. Шаблоны организации источников данных
3. Шаблоны представления

ШАБЛОНЫ ОРГАНИЗАЦИИ БИЗНЕС-ЛОГИКИ

Рассмотрим вопрос рационального выбора варианта организации бизнес-логики. Наша основная цель — научиться избегать грубых ошибок за счет применения простой схемы выбора варианта.

Существует множество разных подходов к описанию и реализации бизнес-логики. Можно сказать, что у каждого коллектива разработчиков так или иначе формируется свой подход, что выражается в методах проектирования структуры приложения, правилах кодирования и т.д. Однако все чаще предпринимаются попытки систематизации и классификации подходов к проектированию для описания профессионального опыта в этой области. Один из способов систематизации приемов и методов проектирования структуры программных систем — шаблоны проектирования или шаблоны архитектуры (также называемые в русском переводе «паттернами»)

Существует три типовых подхода к организации бизнес-логики:

1. **Сценарий транзакции (Transaction Script),**
2. **Модель предметной области (Domain Model)**
3. **Модуль таблицы (Table Module).**

❶ Сценарий транзакции (функциональный подход) Transaction Script

❶ Сценарий транзакции (функциональный подход)

Простейший подход к описанию бизнес-логики связан с использованием *сценария транзакции* - процедуры, которая получает на вход информацию от слоя представления, обрабатывает ее, проводя необходимые проверки и вычисления, сохраняет в базе данных и активизирует операции других систем. Затем процедура возвращает слою представления определенные данные, возможно, осуществляя вспомогательные операции для форматирования содержимого результата. Бизнес-логика в этом случае описывается **набором процедур, по одной на каждую (составную) операцию**, которую способно выполнять приложение.

При таком подходе программная система представляет собой набор функций, в котором каждая функция соответствует операции, которую приложение выполняет для пользователя.

Это старая добрая процедурная модель, хорошо известная разработчикам. Проектируется иерархия функций, возможно повторное использование функций нижних уровней. Существуют давно и хорошо отработанные методологии, а также стандарты функционального моделирования, с помощью которых можно проектировать функциональную структуру программной системы в сложных случаях.

Таким образом, типовое решение "*сценарий транзакции*" можно трактовать как сценарий действия, или бизнес-транзакцию. Оно не обязательно должно представлять собой единый фрагмент кода. Код делится на подпрограммы, которые распределяются между различными сценариями транзакции.

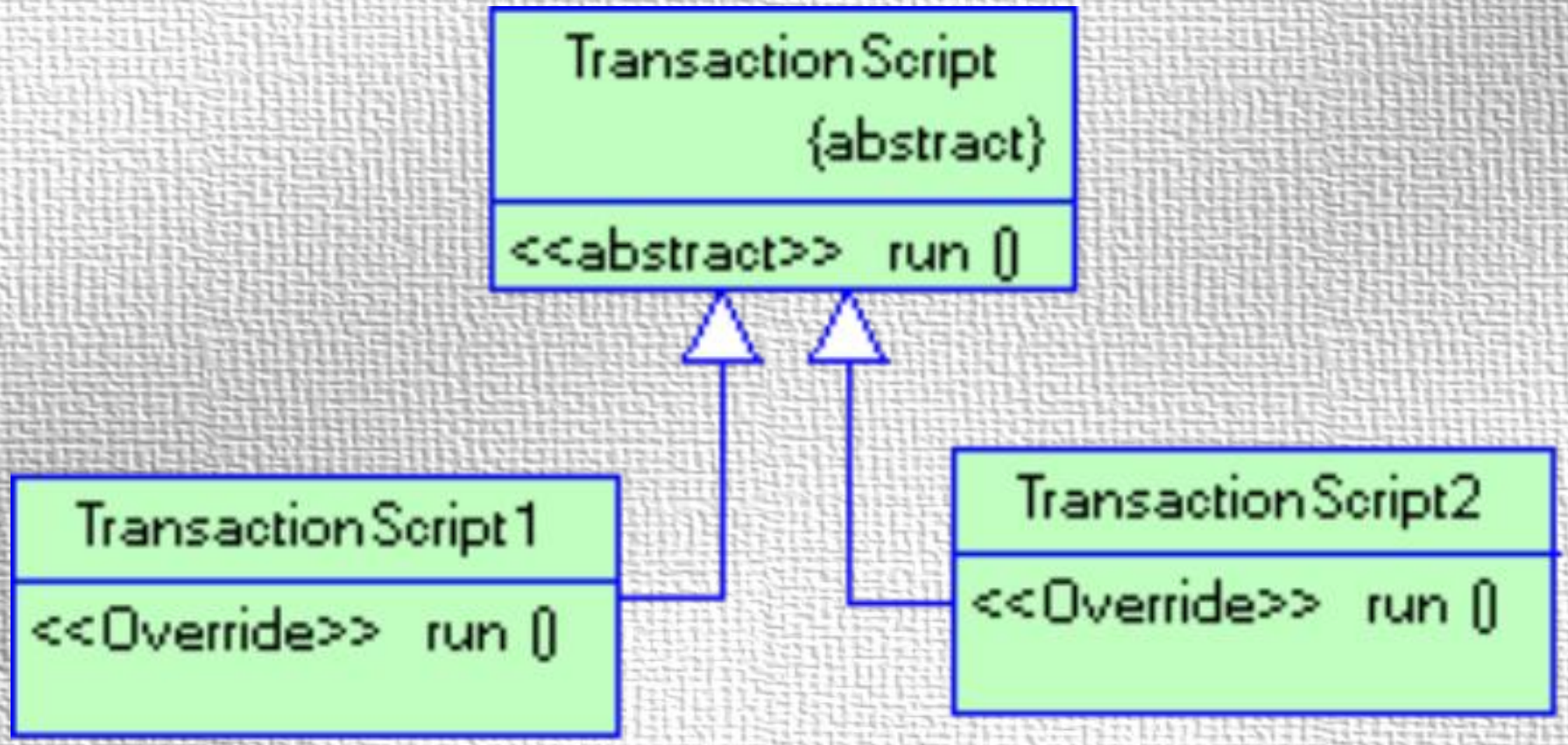
При использовании типового решения *сценарий транзакции* логика предметной области распределяется по транзакциям, выполняемым в системе. Если, например, пользователю необходим заказать номер в гостинице, соответствующая процедура должна предусматривать действия по проверке наличия подходящего номера, вычислению суммы оплаты и фиксации заказа в базе данных.

Существует два способа разнесения кода сценариев транзакции по классам.

1. Наиболее общий, прямолинейный и удобный во многих ситуациях — использование **одного** класса для реализации нескольких сценариев транзакции.
2. Второй, следующий схеме типового решения **Команда** (Command), связан с разработкой собственного класса для каждого сценария транзакции: определяется тип, базовый по отношению ко всем командам, в котором предусматривается некий метод выполнения, удовлетворяющий логике сценария транзакции. Преимущество такого подхода - возможность манипулировать экземплярами сценариев как объектами в период выполнения, хотя в системах, где бизнес-логика организована с помощью сценариев транзакции, подобная потребность возникает сравнительно редко.

Разумеется, во многих языках **модель классов можно полностью игнорировать**, полагаясь, скажем, только на глобальные функции. Однако вполне очевидно, что аппарат создания объектов помогает преодолевать проблемы потоков вычислений и облегчает изоляцию данных.

Применение паттерна Command в паттерне Transaction Script



Типовое решение *сценарий транзакции* отличается следующими преимуществами:

- представляет собой удобную процедурную модель, легко воспринимаемую всеми разработчиками;
- удачно сочетается с простыми схемами организации слоя источника данных на основе типовых решений **шлюз записи данных** (Row Data Gateway) и **шлюз таблицы данных** (Table Data Gateway);
- определяет четкие границы транзакции.

С возрастанием уровня сложности бизнес-логики типовое решение сценарий транзакции демонстрирует и ряд недостатков. Если нескольким транзакциям необходимо осуществлять схожие функции, возникает опасность дублирования фрагментов кода. Поскольку каждый сценарий транзакции призван обслуживать одну транзакцию, все общие порции кода неизбежно приходится воспроизводить вновь и вновь.

С этим явлением удается частично справиться за счет вынесения общих подпрограмм "за скобки", но даже в этом случае большая часть дубликатов остается на месте. В итоге приложение может выглядеть как беспорядочная мешанина без отчетливой структуры.

Однако каким бы ярым приверженцем объектных технологий вы ни становились, не отбрасывайте сценарий транзакции: существует множество простых проблем, и их решения также должны быть простыми.

❷ Модель предметной области (объектный подход)

❷ Модель предметной области
(объектный подход)

Этот подход наиболее сложен в реализации, но и наиболее продуктивен. Разрабатывается объектная модель предметной области — по крайней мере, для основных понятий. Например, для интернет-магазина могут быть созданы объекты «клиент», «товар», «корзина», «заказ» и т.д. Вместо того, чтобы помещать бизнес-логику расчета стоимости в одну или несколько процедур, для каждого объекта реализуется функциональность исходя из зоны ответственности этого объекта.

Например, объект «заказ» содержит алгоритм вычисления стоимости заказа на основе цен входящих в него товаров, при этом алгоритмы определения цен на товары реализуются отдельно, в объектах «товар». Если для разных товаров применяются разные алгоритмы вычисления цены, это достаточно легко реализовать в модели за счет создания разных типов товаров.

Объектно-ориентированная модель предметной области часто напоминает схему соответствующей базы данных, хотя между ними все еще остается множество различий. В модели предметной области смешиваются данные и функции, допускаются многозначные атрибуты, создаются сложные сети ассоциаций и используются связи наследования.

В сфере корпоративных программных приложений можно выделить две разновидности моделей предметной области.

1. "**Простая**" во многом походит на схему базы данных и содержит, как правило, по одному объекту домена в расчете на каждую таблицу.
2. "**Сложная**" модель может отличаться от структуры базы данных и содержать иерархии наследования, стратегии и иные шаблоны, а также сложные сети мелких взаимосвязанных объектов. Сложная модель более адекватно представляет запутанную бизнес-логику, но труднее поддается отображению в реляционную схему базы данных.

Вместо использования одной подпрограммы, несущей в себе **всю** логику, которая соответствует некоторому действию пользователя, каждый объект наделяется только функциями, **отвечающими его природе**. Если прежде вы не пользовались моделью предметной области, процесс обучения может принести немало огорчений, когда в поисках нужных функций вам придется метаться от одного класса к другому.

Чтобы добиться успехов в применении модели, новичкам придется затратить немало времени: некоторым требуется несколько месяцев работы над соответствующим проектом, прежде чем их стиль мышления перестроится в нужном направлении. После приобретения опыта работать становится намного проще - в вас даже просыпается энтузиазм. Через это прошли все, кто так одержим объектной парадигмой. Впрочем, сбросить груз привычек и сделать шаг вперед многим, к сожалению, так и не удается.

Другая особенность модели *предметной области* — необходимость создания сложных программных механизмов взаимодействия объектных структур с реляционными базами данных. Разработка такого механизма обычно обходится дорого (хотя на этот счет тоже имеются типовые решения), и это, конечно, не упрощает жизнь в случае применения объектной модели.

❸ Модуль таблицы
(смешанный подход)

❸ Модуль таблицы (смешанный подход)

Перечисленные выше методы организации бизнес-логики представляют собой два «крайних» случая. Третий случай — смешанный, гибридный подход, сочетающий в себе определенные достоинства (и недостатки) функционального и объектного подходов.

Типовое решение «модуль таблицы» предусматривает, как и в модели предметной области, отдельные объекты для товаров, заказов и т.д. Однако, в отличие от «настоящей» модели предметной области, в «модуле таблицы» для работы со всеми (к примеру) заказами, содержащимися в базе данных, применяется только один объект. **Именно этот единственный объект содержит логику обработки заказов**. А чтобы работать с отдельным заказом, следует указывать его уникальный идентификатор.

С точки зрения прозрачности, читабельности кода модуль таблицы заметно проигрывает «настоящей» модели предметной области (конечно, предполагается, что разработчик в каждом случае использует максимум возможностей сделать код простым и понятным). При несложной логике предметной области модуль таблицы проигрывает в простоте реализации и функциональному подходу.

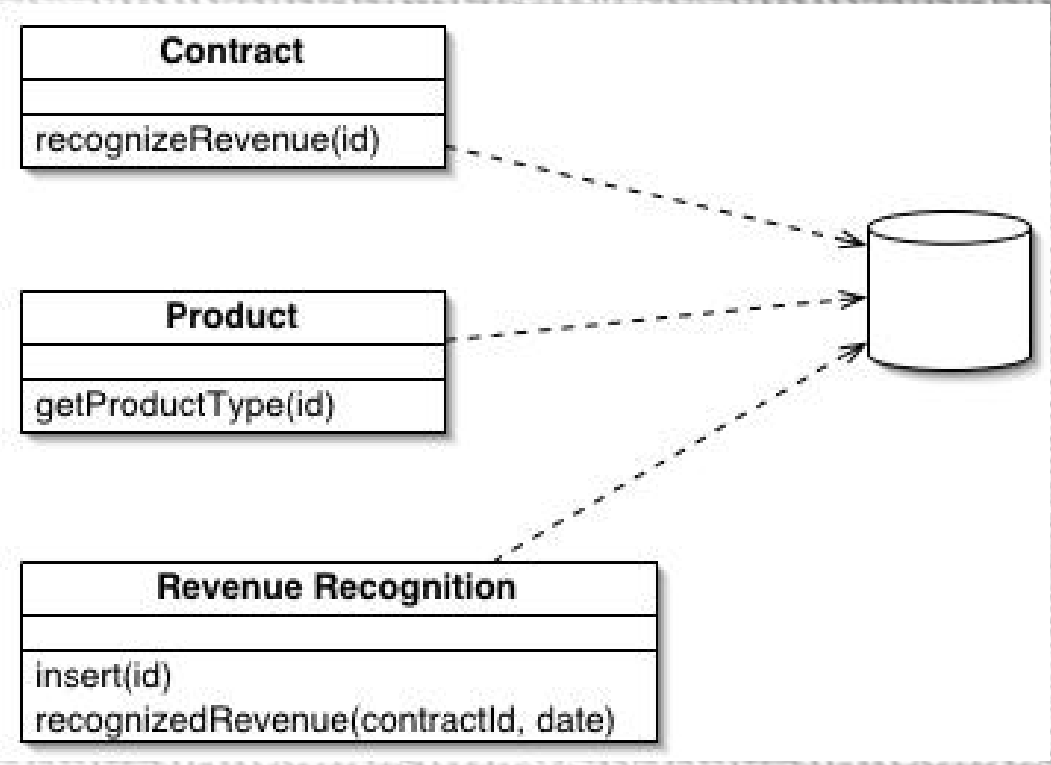
Основные преимущества модуля таблицы — простота взаимодействия с базой данных и гибкость структурирования бизнес-логики по сравнению с функциональным подходом. Возможно также применение шаблонов проектирования. Однако по сравнению с моделью предметной области гибкость и возможности применения шаблонов в модуле таблицы существенно ограничены.

Целесообразность применения модуля таблицы также зависит от уровня поддержки структуры множества записей в конкретной среде разработки (включая удобство отображения множества записей на базу данных и на элементы графического интерфейса). Развитая поддержка набора данных (DataSet) в среде **Microsoft.Net** во многих случаях оправдывает применение этой платформы разработки.

Table Module
(Обработник таблицы)

Одна сущность обрабатывает всю бизнес-логику для всех строк таблице БД или виде.

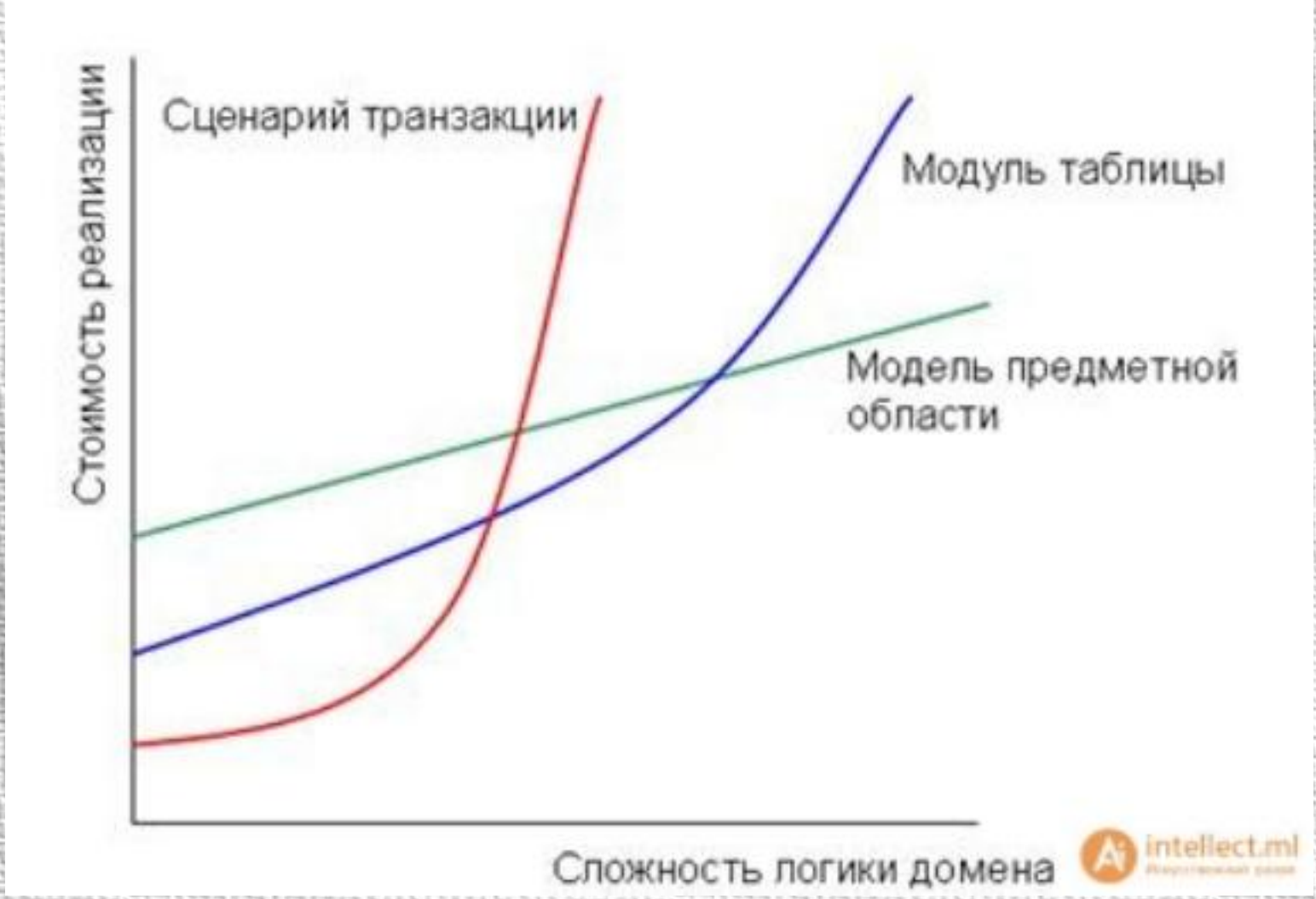
Один из основополагающих принципов в ООП - сочетание данных и методов обработки этих данных. Традиционный объектно-ориентированный подход основан на объектах с соответствием, как, например, в паттерне Domain Model.



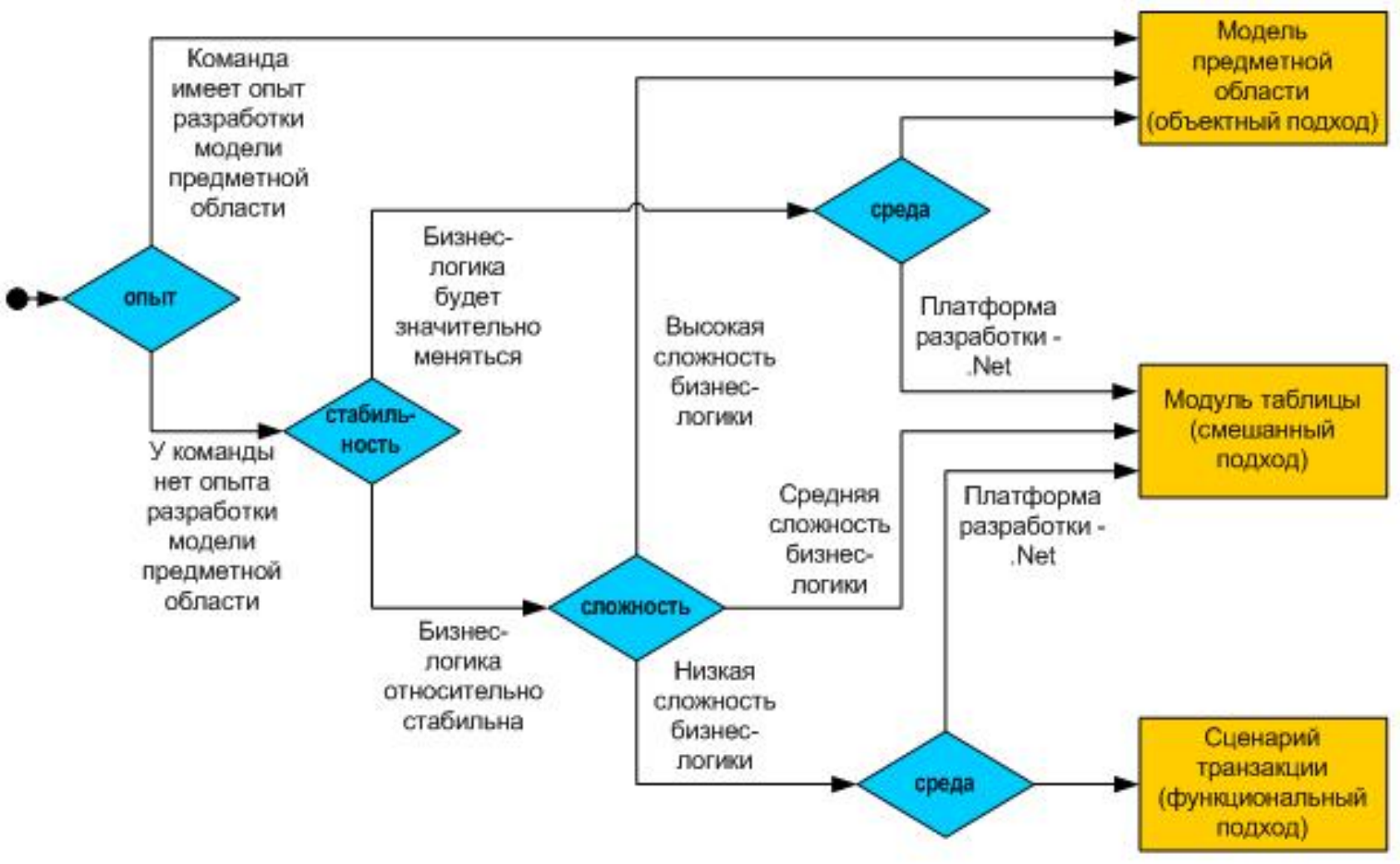
Особенности подходов

- **Сценарий транзакции** (**функциональный подход**): предельно прост, плохо работает при сложной логике, затрудняет развитие системы.
- **Модель предметной области** (**объектный подход**): наиболее сложный и затратный, позволяет реализовать сложную бизнес-логику, дает возможность «цивилизованно» развивать систему; для простых систем подход целесообразен, только если уже «обкатан» разработчиком.
- **Модуль таблицы** (**смешанный подход**): обеспечивает определенное сочетание простоты и гибкости, однако гибкость ограничена, а эффективность подхода зависит от среды разработки.

Зависимость стоимости реализации различных схем организации бизнес-логики от ее сложности



Упрощенная схема выбора типового решения для реализации бизнес-логики



Переезд на центральный узел

При переходе на центральный узел всегда есть искус «реализовать эту часть в хранимой процедуре». Потом «ту» и «вот эту». И скоро вы окажетесь в той же ситуации, что и были, без существенных изменений.

Хранимые процедуры должны использоваться для выполнения SQL и получения наборов данных в базах данных, которые оптимизируют хранимые процедуры лучше, чем представления. **Но хранимые процедуры не должны быть использованы ни для чего другого, нежели объединения и выдачи данных**. При обновлении данных она должна именно и только обновлять, но не интерпретировать данные каким-либо образом.

Есть задачи, где для повышения производительности некоторые компоненты должны быть помещены в хранимую процедуру. Но такие задачи на самом деле достаточно редки и они должны быть исключением, а не правилом. Каждое исключение должно быть проверено и одобрено, а не просто реализовано по воле разработчика или администратора базы данных.

Дешевле

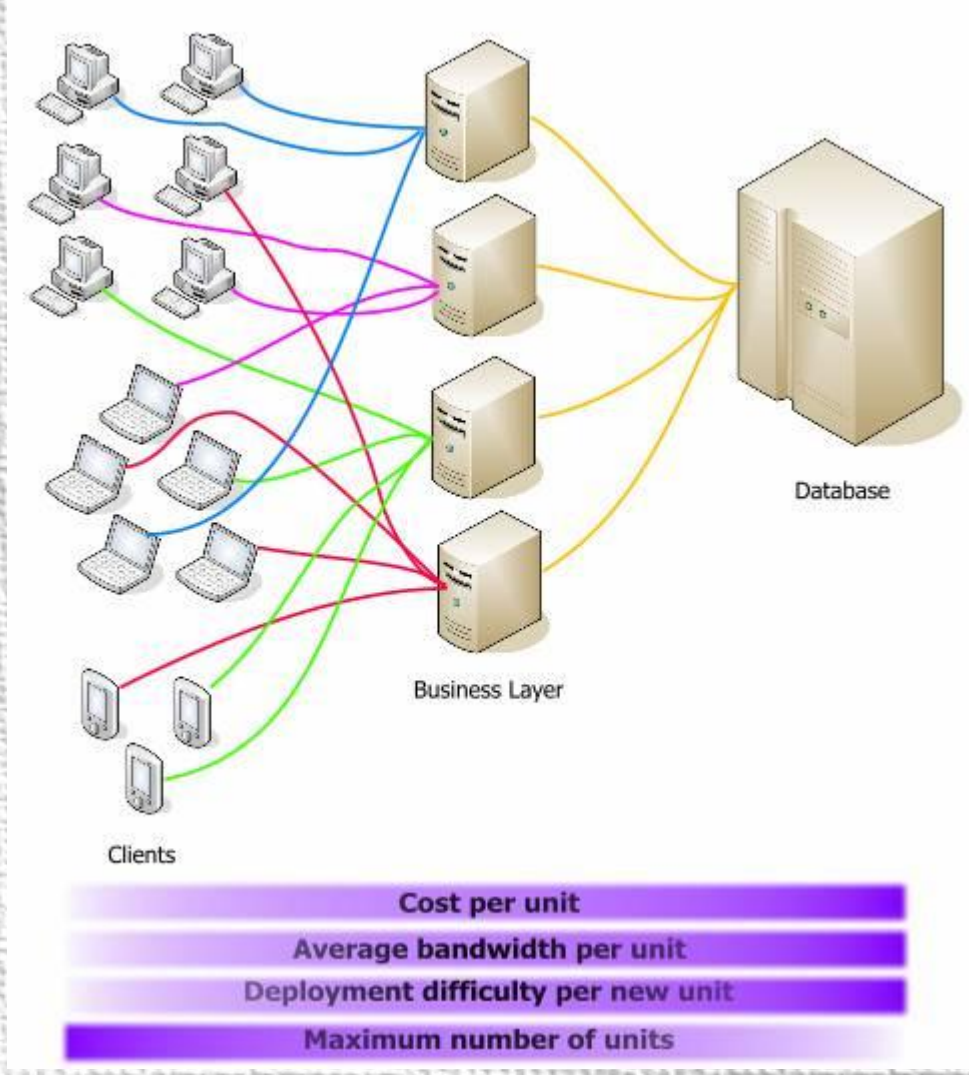
Звучит несколько странно, что покупка железа может сделать дешевле. Но при внедрении серверов среднего звена, практически никакого дополнительного ПО, кроме ОС, не требуется. А стоимость наращивания мощности сервера базы данных существенна по следующим причинам:

- Сервера баз данных, как правило, более высокого класса, чем сервера среднего звена, и стоят дороже.
- Базы данных зачастую лицензируются на процессор и добавление процессора – дорогостоящая процедура в терминах лицензий. Лицензионные сборы могут составлять от 5000 до 40000 долларов на процессор.
- При переносе логики на среднее звено, вы можете существенно сократить нагрузку на базу данных и предотвратить преждевременное наращивание ее мощностей

Проще

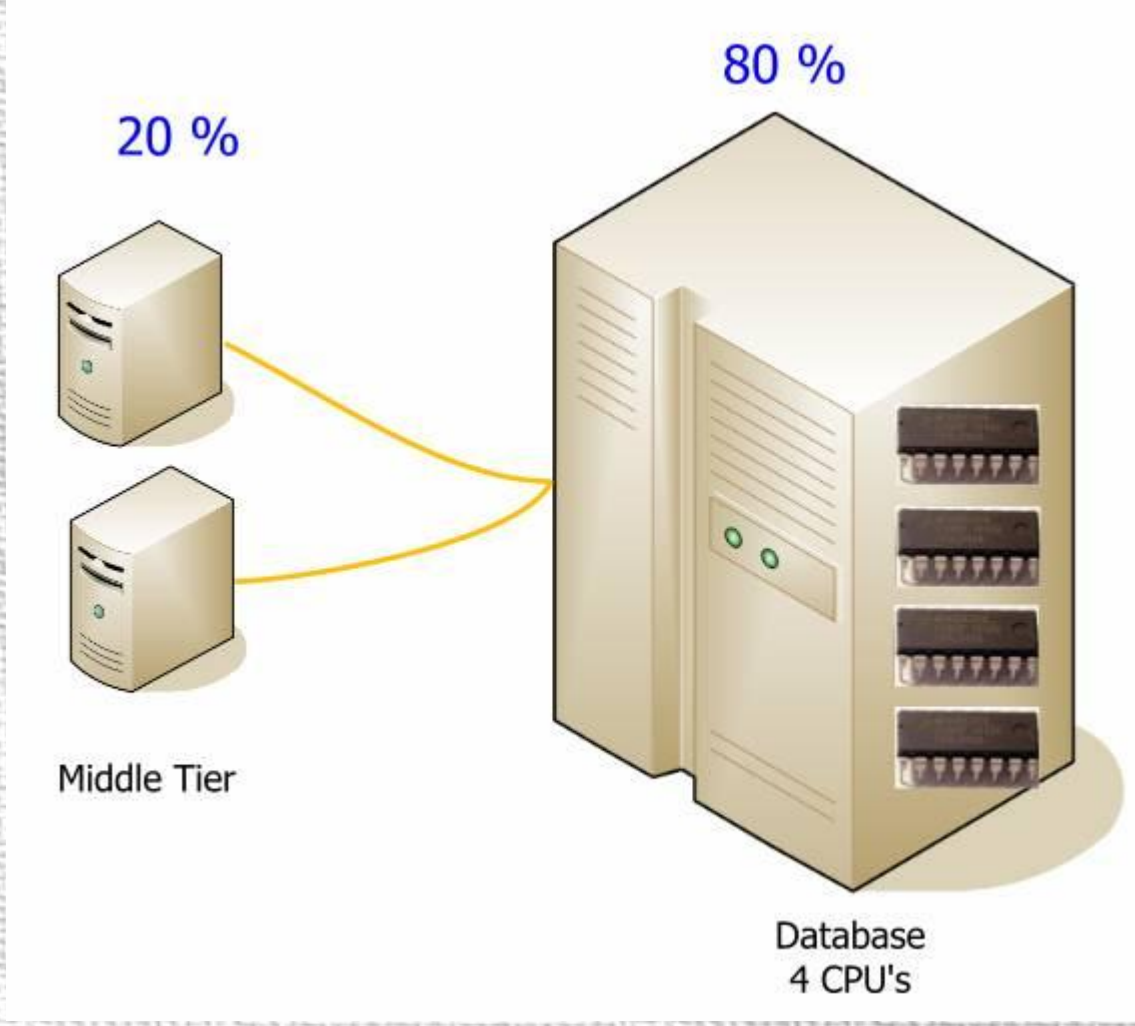
Обновление среднего звена обычно проще чем обновление базы данных.

У баз данных есть врожденный предел того, на сколько они могут быть увеличены простым добавлением железа. В какой-то момент нужно начинать использовать другие технологии вроде деления, кластеризации, репликации и т.п. Но ни одна из этих технологий не является простой, и все требуют существенных вложений в железо, миграцию и сильно влияют на существующие системы. Наращивать же сервера среднего звена гораздо проще. Как только запущен механизм распределения нагрузки, все сводится к задаче добавить новый сервер.

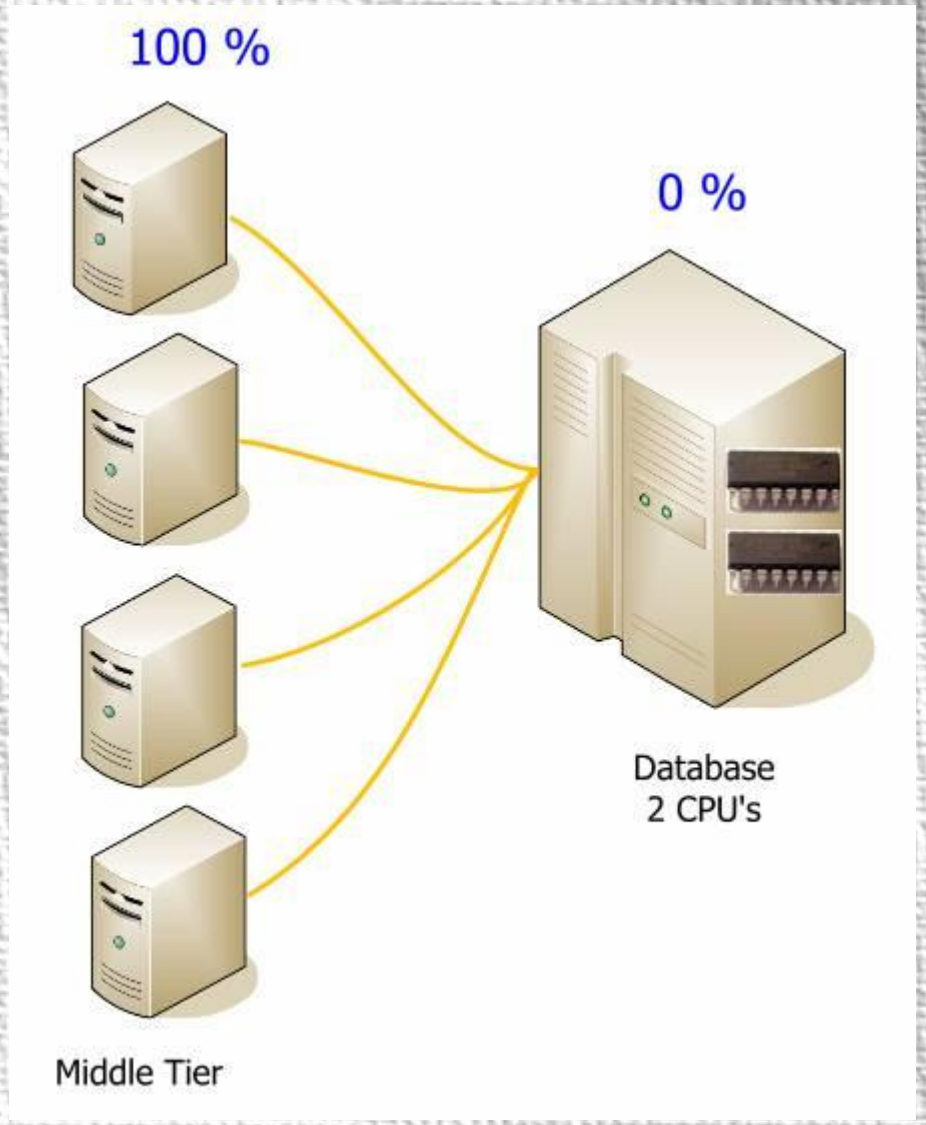


Вырасти середину

Если большая часть бизнес логики реализована в базе данных (хранимые процедуры), вам будет нужна более мощная база данных.



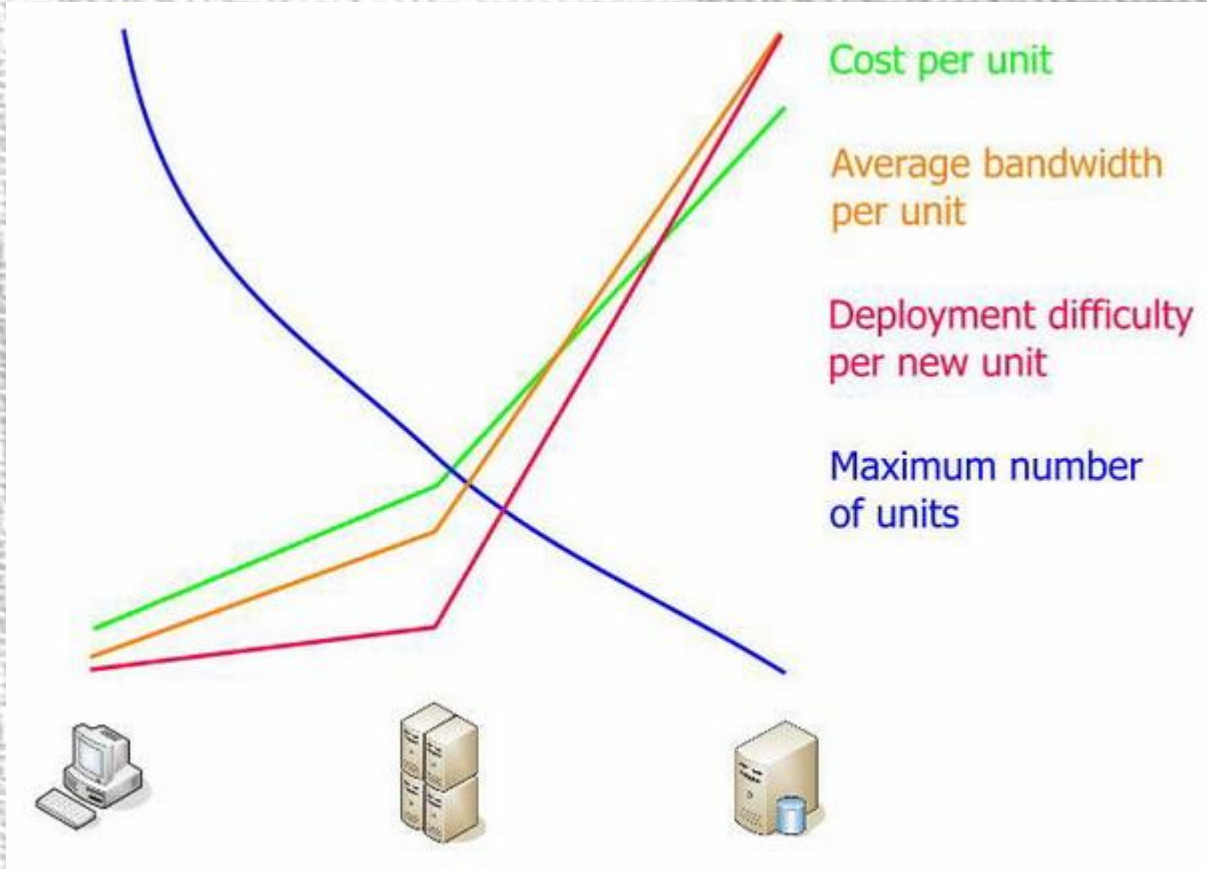
При переносе логики в среднее звено, вы можете серьезно снизить нагрузку на базу данных. Цифры представленные здесь, приведены только для демонстрации и будут меняться от системы к системе, но они могут помочь уловить идею. Хотя на следующей диаграмме и больше аппаратуры, суммарная стоимость системы будет меньше, и ее будет проще развернуть. Гораздо дешевле и проще наращивать среднее звено.

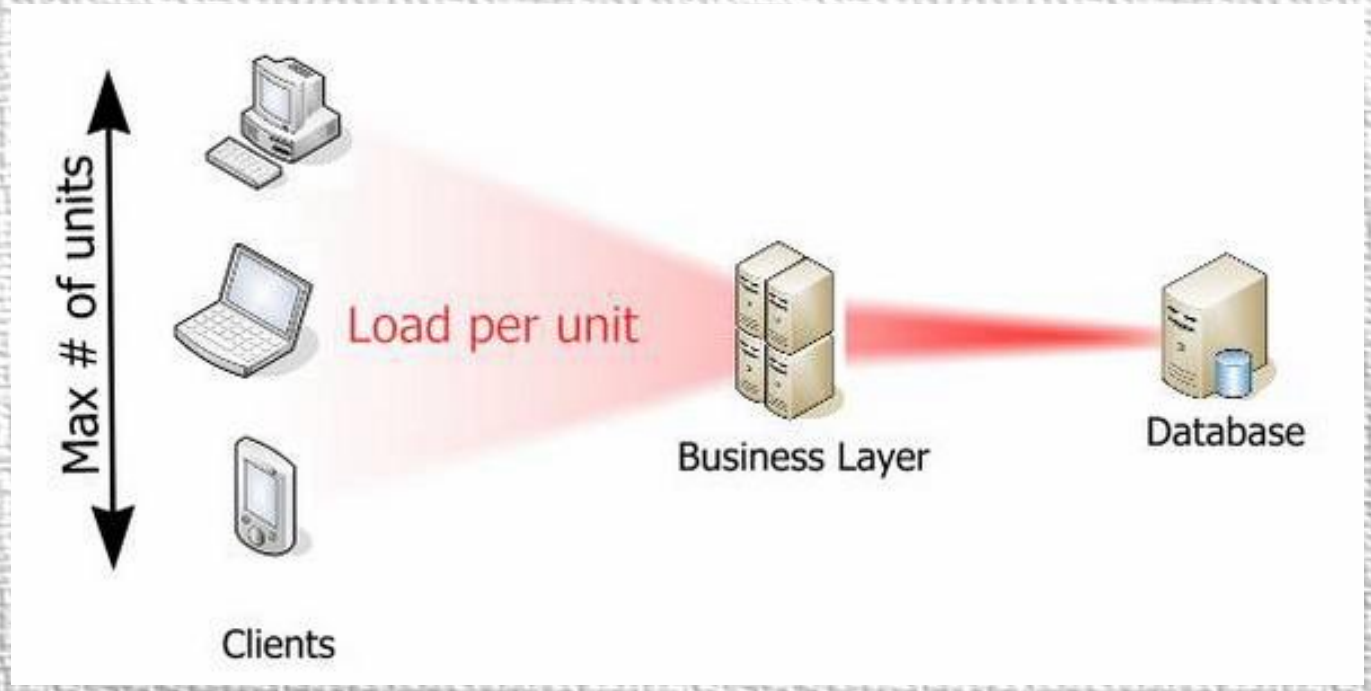


Слово единица используется для обозначения процессора или сервера, в зависимости от конфигурации.

(сверху вниз: цена единицы, средняя полоса пропускания, сложность развертывания, количество)

Цена единицы возрастает, когда мы движемся от клиента, к среднему звену, к базе данных.





Какое единственное узкое место в системе? Какое из звеньев имеет выраженный предел наращивания? Это однозначно база данных. Все упирается в базу данных.

Потому перемещая вычисления в среднее звено, мы может отойти от границ слоя данных.

ШАБЛОНЫ ОРГАНИЗАЦИИ ИСТОЧНИКОВ ДАННЫХ

То, насколько оптимально построена связь между уровнем прикладного кода и уровнем БД во многом зависит быстродействие системы в целом. Поэтому важно правильно построить взаимодействие с СУБД. Рассмотрим шаблоны взаимодействия с базами данных. Правильно выбранный шаблон взаимодействия позволит избежать многих проблем при разработке и получить качественное приложение.

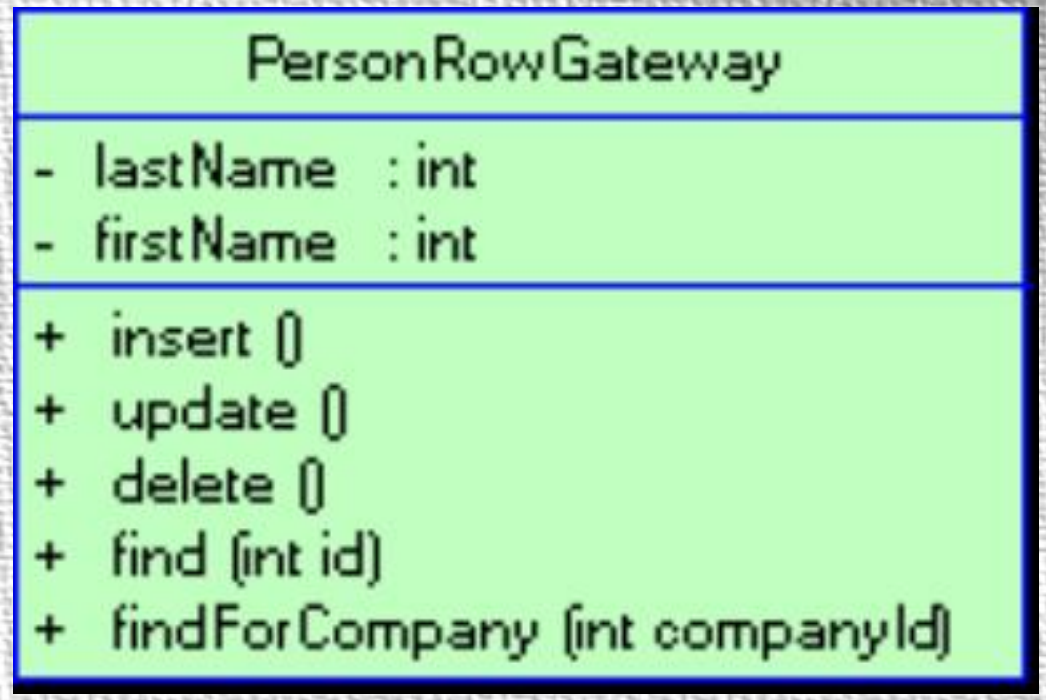
Роль слоя источника данных состоит в том, чтобы обеспечить возможность взаимодействия приложения с различными компонентами инфраструктуры для выполнения необходимых функций. Главная составляющая подобной проблемы связана с поддержкой диалога с базой данных - в большинстве случаев реляционной.

Одна из основных задач - обособить код SQL от бизнес-логики, разместив его в специальных классах.

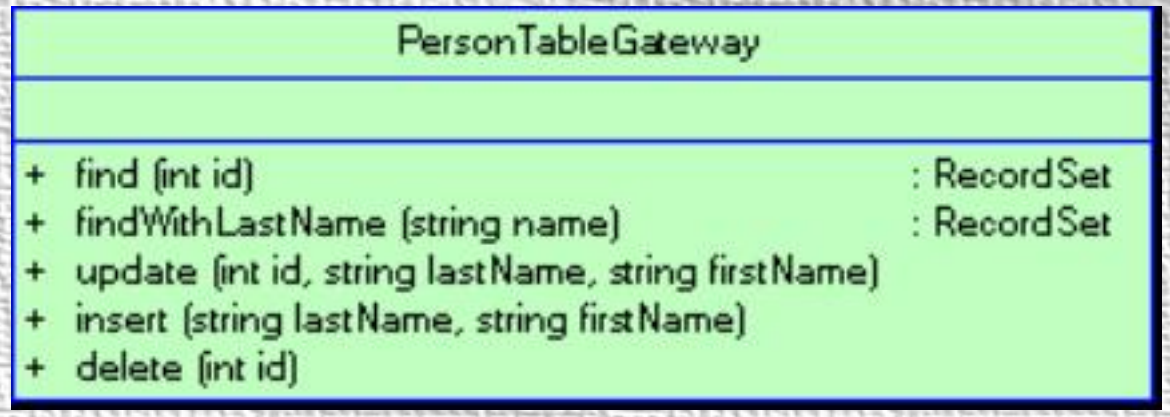
Удачный способ организации подобных классов состоит в "копировании" структуры каждой таблицы базы данных в отдельном классе, который формирует **шлюз** (Gateway,), поддерживающий возможности обращения к таблице. **Теперь основному коду приложения нет необходимости что-либо "знать" о SQL, а все SQL-операции сосредоточиваются в компактной группе классов**.

Существует два основных варианта практического использования типового решения шлюз.

1. **Н**аиболее очевидный - создание экземпляра шлюза для каждой записи, возвращаемой в результате обработки запроса к базе данных (рисунок). Подобный **шлюз записи данных** (Row Data Gateway,) представляет собой модель, естественным образом отображающую объектно-ориентированный стиль восприятия реляционных данных.



2. **В**о многих средах поддерживается **модель множества записей** (Record Set) - одна из основополагающих структур данных, имитирующая табличную форму представления содержимого базы данных. С каждой таблицей базы данных следует сопоставить соответствующий объект типа **шлюз таблицы данных** (Table Data Gateway), который содержит методы активизации запросов, возвращающих множество записей.

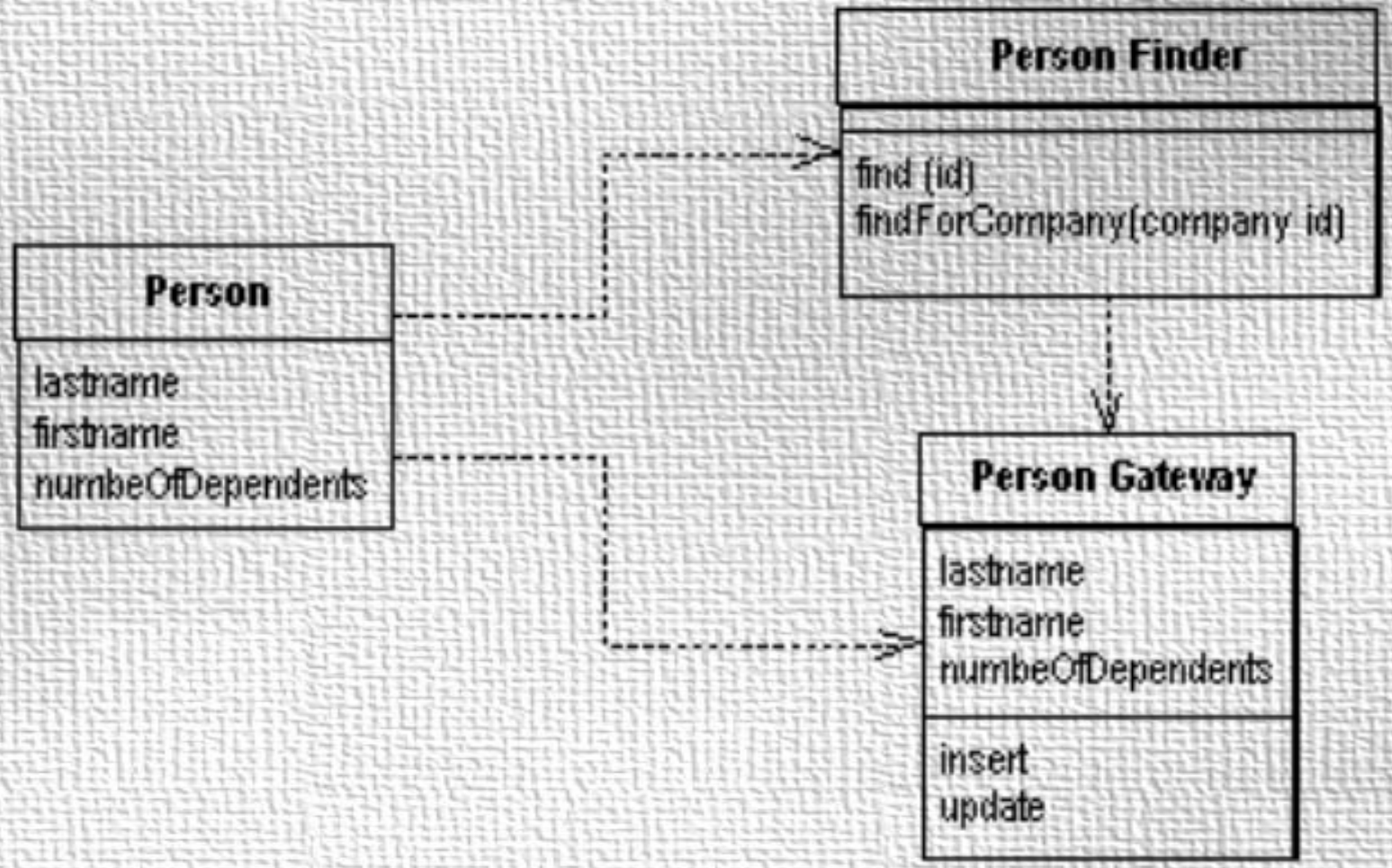


Инструментальными системами предлагаются даже графические интерфейсные элементы, реализующие схему множества записей.

Row Data Gateway (Шлюз к данным записи)

Объект выступает в роли шлюза к отдельной записи в источнике данных.

Один экземпляр на одну запись.



Встраивание кода доступа к БД в объекты, хранящиеся в памяти, может привести в некоторым неудобствам. Во-первых, если ваши объекты содержат бизнес-логику, добавление взаимодействия с БД увеличит сложность. Вскоре тестирование станет неудобным, если объекты, хранящиеся в памяти, завязаны на БД. Тесты станут медленнее из-за доступа к БД. Объект шлюза к записи представляется в точности, как запись в БД, но при этом даёт возможность доступа посредством штатных средств языка программирования. Все подробности доступа к БД скрыты за этим интерфейсом.Пример: объект шлюза PersonGateway, данные о записи из таблицы person и методы insert и update, которые позволяют прозрачно работать с записью.Качественная реализация этого паттерна на PHP существует в частности в Zend Framework в классе Zend_Db_Table_Row.

ORM

Начнем с рассмотрения Object Relation Mapping – ORM, механизма взаимодействия, который представляет собой вспомогательную прослойку между приложением и базой данных, позволяющую реализовать механизмы представления и взаимодействия между БД и собственно кодом. Суть работы ORM заключается в следующем: мы представляем ряды из таблиц в качестве объектов, свойства которых будут соответствовать именам полей из таблиц, а значения этих свойств – значениям из базы данных. Таким образом мы получаем соответствие одна строка в БД – один объект.

На практике реализация ORM осуществляется с помощью шаблонов DAO, Active Record или DataMapper. Также возможно построение гибридов из нескольких шаблонов.

DAO

Как можно понять из названия, этот шаблон предназначен для доступа к данным. Он предоставляет абстрактный интерфейс для обращений к БД. Основная суть работы данного шаблона заключается в том, чтобы можно было выполнять определенные операции не сильно вдаваясь в детали реализации базы данных. При использовании шаблона DAO функции для работы с конкретной таблицей хранятся в файле модели. Соответственно, данная модель наследует абстрактный класс, реализующий DAO. Когда будет получен ряд данных в DAO, в результирующем объекте или массиве будут содержаться все поля из БД.

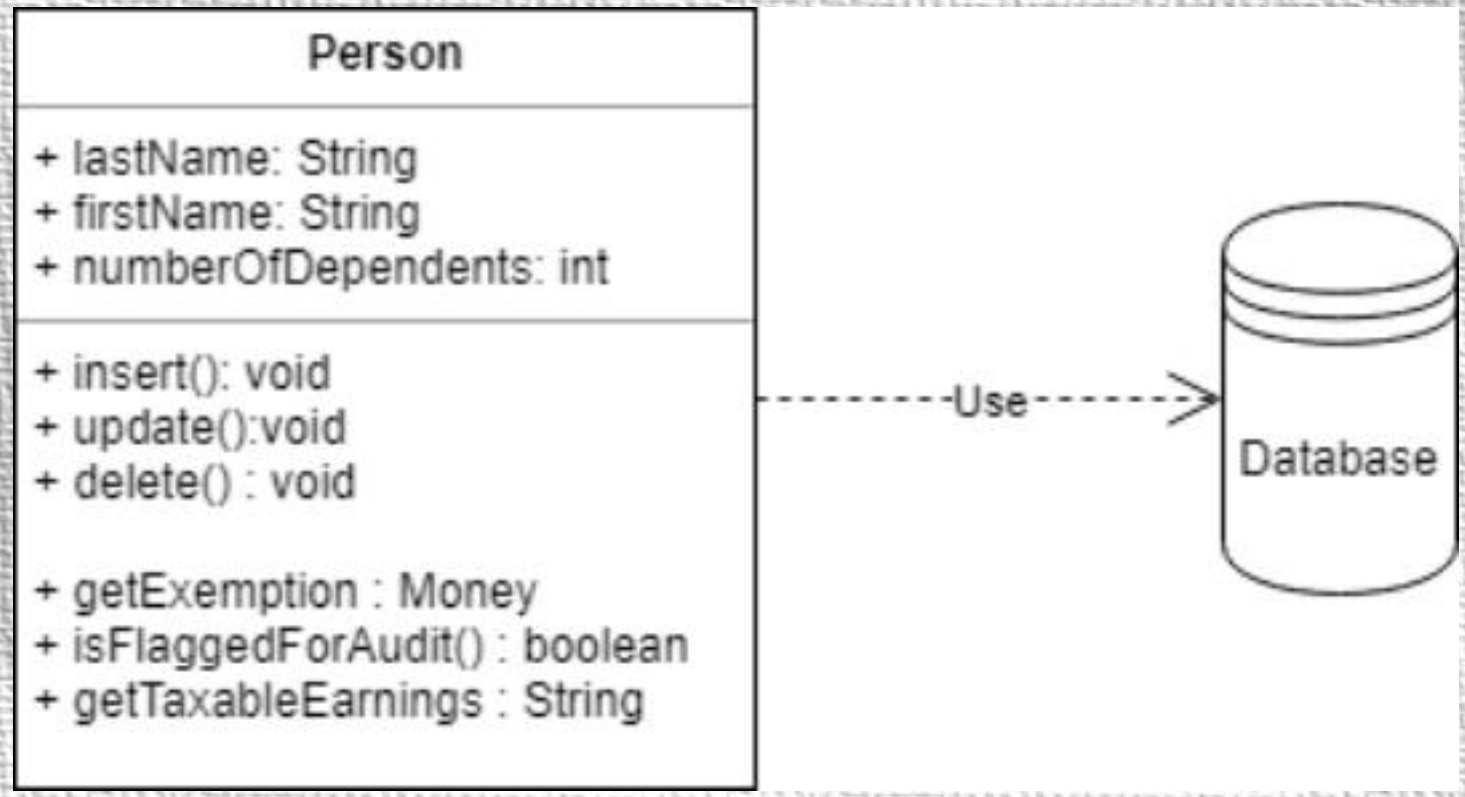
Active Record (активная запись)

Данный шаблон также отвечает за представление бизнес-логики и данных. Если у нас имеются объекты, требующие постоянного хранения в базе данных, то с помощью Active Record можно достаточно просто создавать и использовать эти объекты.

Основная идея шаблона Active Record заключается в том, чтобы позволить объекту инкапсулировать данные и операции с базой данных, которые вы можете выполнять с ним. То есть, мы передаем классу некоторый набор значений, который затем внутри этого класса будет преобразован в запрос SQL и выполнен. Такой подход позволяет нам не использовать запросы SQL напрямую. И кроме того, позволяет сделать код более безопасным, так как если нельзя выполнить любой запрос напрямую, то и выполнение SQL инъекций становится более затруднительным или даже невозможным.

Пример: объект "Person" содержит данные об одной персоне и методы: добавить, обновить или удалить. По сути паттерн продвигает идею когда модель и логика сосредоточены в 1 классе.

рассмотрим диаграмму класса "Person":



Data Mapper

Шаблон Data Mapper - это слой доступа к данным, который предоставляет двунаправленную работу с данными между БД и хранением данных в памяти (например, на время выполнения кода. В его обязанности входит передача данных между объектами и базой данных и изоляция их друг от друга. Если мы получаем средство отображения данных, объекту в памяти совершенно не обязательно знать, существует база данных или нет.

В шаблоне Data Mapper имеется еще один слой или тип сущности такой как entityManager. Именно этот слой будет отвечать за перенос состояния модели в базу данных и обратно.

Список ORM-библиотек

Для всякого языка существуют десятки различных библиотек, реализующих ORM различными способами.

Список ORM-библиотек — Википедия (wikipedia.org)

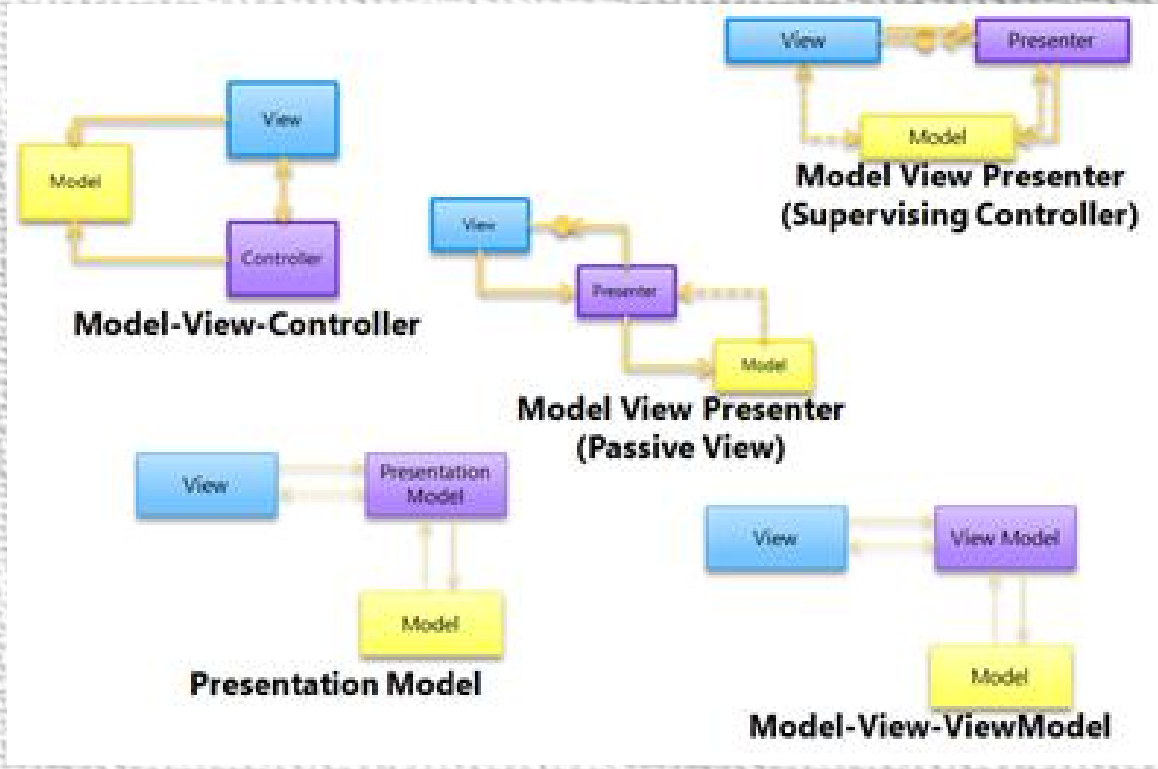
ШАБЛОНЫ ПРЕДСТАВЛЕНИЯ

Одно из самых впечатляющих изменений, происшедших с корпоративными приложениями за последние несколько лет, связано с появлением пользовательских интерфейсов в стиле Web-обозревателей. Они принесли с собой ряд преимуществ:

- исключили потребность в применении специального клиентского программного слоя,
- обобщили и унифицировали процедуры доступа и
- упростили проблему конструирования Web-приложений.

Функции Web-сервера состоят в интерпретации адреса URL-запроса и передаче управления соответствующей программе (вариант, когда WEB-сервер считывает с диска и отправляет клиенту обычный файл не рассматривается).

Существует довольно много шаблонов представления в стиле MV*-паттернов: ***Model-View-Controller, Model-View-Presenter, Presentation Model, Passive View, Supervising Controller, Model-View-ViewModel*** и многие другие:



Построение UI без использования MV*-паттернов

Как бы вы построили пользовательский интерфейс (UI), не используя вышеперечисленные паттерны? Взяли бы форму, добавили на нее виджеты, а логику написали бы в коде. Такой код, описывающий логику представления (View), жёстко связан с пользовательским интерфейсом, так как он напрямую взаимодействует с элементами на экране. Это хороший, но прямолинейный подход. Он применим только для очень простых интерфейсов. Когда логика становится более сложной, поддержка такого UI может превратиться в кошмар!

Корень проблемы заключается в том, что построение UI таким способом нарушает принцип единственной ответственности, который гласит: «***У класса должна быть только одна причина для изменения***». Если UI-компонент содержит код для отображения, логики и данных, то у него есть несколько причин для изменения. Например, если вы хотите поменять тип пользовательского элемента, который используется для отображения данных, то изменения не должны повлиять на логику. Однако поскольку логика так тесно связана с элементами управления, ее тоже придется менять. Это так называемый «код с душком» (code smell), который сигнализирует, что принцип единственной ответственности нарушен.

Таким образом, если форма содержит код для отображения элементов управления, логику интерфейса (что происходит при нажатии кнопки) и данные для отображения на экране, вы столкнетесь со следующими проблемами:

Усложнение поддержки
Изменения в UI, логике или данных, скорее всего, повлекут за собой изменения в остальных частях. Поэтому вносить правки гораздо сложнее, что затрудняет поддержку.

Ухудшение тестируемости
Логика и данные приложения могут быть написаны таким образом, чтобы каждый компонент мог быть протестирован отдельно. Однако код, связанный с пользовательским интерфейсом, плохо поддается модульному тестированию, потому что для этого часто требуется участие пользователя для запуска логики в UI.

Кроме того, любая визуализация часто требует оценки со стороны человека, что всё «выглядит правильно». *Отметим, что существуют решения для автоматизации тестирования пользовательского интерфейса. Однако они только имитируют взаимодействие с пользователем. Как правило, они сложнее в настройке и обслуживании, чем unit-тесты, и чаще всего используются при интеграционном тестировании, так как для этого требуется запуск всего приложения*.

Уменьшение возможности переиспользования

Если ваш UI-код смешан с кодом логики и данных, то его становится гораздо сложнее переиспользовать (reuse).

Цели MV*-паттернов

Хотя каждый из паттернов имеет довольно много отличий, их цели похожи: отделить UI-код (View) от кода логики (Presenter, Controller, ViewModel и т. д.) и кода обработки данных (Model). Это позволяет каждому из них развиваться самостоятельно. Например, вы сможете изменить внешний вид и стиль приложения, не затрагивая логику и данные.

Кроме того, так как логика и данные отделены от отображения, то они могут быть протестированы отдельно. Для простых приложений это может быть не так важно. Например, если ваше приложение является простым редактором данных. Однако, если у вас более сложная логика интерфейса, то возможность автоматически проверить, что она работает правильно, будет очень ценной.

АРХИТЕКТУРНЫЙ ПАТТЕРН MVC

Когда мы пишем сложные приложения, нам нужно выполнять различные операции, иногда совершенно друг на друга не похожие:

- обновить данные на сервере;
- показать всплывающее окно после клика пользователя;
- валидировать данные из формы;
- загрузить дополнительные ресурсы, картинки, скрипты;
- вызвать стороннее API и обработать ответ.

Считается хорошим тоном делить отличающийся код на модули, которые отвечают за свои конкретные задачи. Как именно разделить код на модули, по каким критериям и принципам — на эти вопросы старается ответить паттерн MVC.

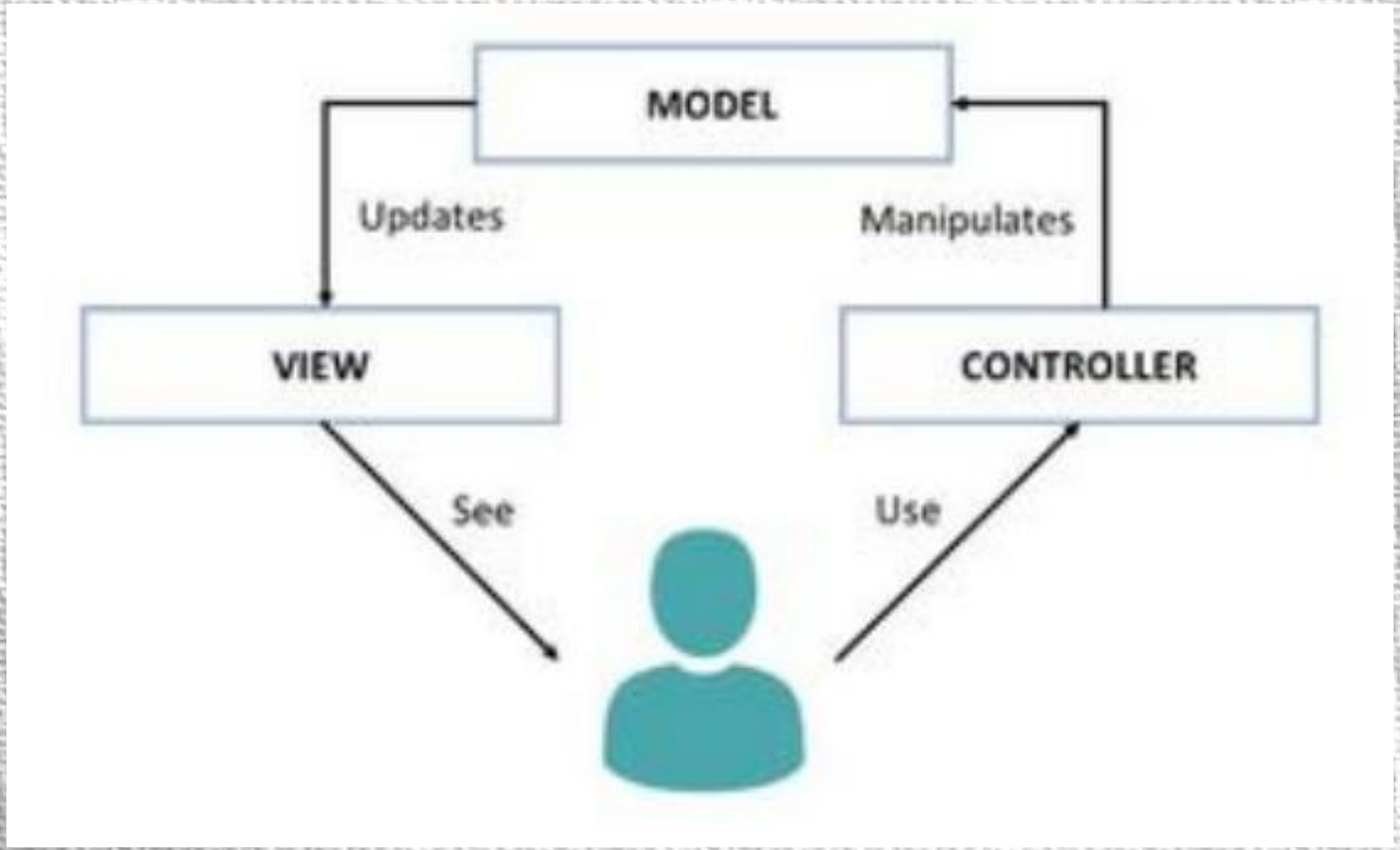
- MVC (сокращение от Model—View—Controller) — это архитектурный паттерн, который делит модули на три группы:
  - **модель (model),**
  - **представление (view),**
  - **контроллер (controller).**
- Модель содержит данные приложения, за которыми приходит пользователь. Например, список своих заказов в интернет-магазине.
- Представление показывает эти данные в понятном для пользователя виде. Например, на свёрстанной странице сайта или в приложении на телефоне.
- Контроллеры принимают пользовательские команды и преобразуют данные по этим командам. Например, если пользователь нажимает кнопку «Удалить заказ», то контроллер отмечает этот заказ в модели удалённым.

Компоненты архитектуры

В архитектуре MVC пользователь взаимодействует только с представлением — чаще всего это UI.

Есть приложения, где представлением может быть голосовой или жестовый интерфейс, но в веб-разработке чаще всего представление — это графический пользовательский интерфейс, GUI.

Пользователь подаёт команды программе. Контроллер получает эти команды и преобразует данные в модели. Модель обновляется и уведомляет представление о том, что нужно перерисовать интерфейс, чтобы отобразить изменения в данных.



В классическом MVC события пользователя вызывают контроллер. Иногда события обрабатываются сперва представлением, которое затем вызывает контроллер. Оба подхода возможны и используются.

Взаимодействие компонентов

При использовании архитектуры MVC мы определяем, как компоненты будут общаться друг с другом — то есть определяем потоки данных.

Поток данных

В классическом MVC стандартом считается, когда данные:

- от пользователя передаются представлению;
- от представления — контроллеру;
- через контроллер обновляется модель;
- модель уведомляет представление о том, что что-то изменилось.

Это называется однонаправленным потоком данных.

Иногда допускается, что компоненты могут общаться напрямую с другими компонентами не по этой схеме, но мы всё же рекомендуем не отходить от канона.

Как много должен знать контроллер?

Другой подобный вопрос, который возникает при работе с MVC — какая зона ответственности у контроллера, где она заканчивается.

Контроллер, который умеет больше, называется толстым. Тот, который умеет меньше — тонким.

При тонком контроллере знание о том, как преобразовывать данные, находится в модели. Это не всегда полезно, потому что может замусорить код модели, но иногда оправданно. Например, если мы не хотим «размазывать» это знание по нескольким компонентам.

Такая проблема называется проблемой тонкого и толстого контроллера. Разные команды решают её по-своему, исходя из договорённостей и выгод и издержек каждого варианта для своего проекта.

Похожие паттерны

Кроме MVC вы могли слышать о других вариациях этой аббревиатуры. Каждая из вариаций — это паттерн, слегка отличающийся от MVC спецификой или ответственностью компонентов.

Как выбрать архитектуру системы?

Выбор архитектуры зависит от многих факторов, включая требования к системе, масштабируемость, безопасность, производительность и бюджет.

С чем часто придется работать:

- **Необходимость интеграций с другими системами**. Если система должна интегрироваться с другими системами, аналитик должен учесть совместимость и выбрать архитектуру, которая позволит это сделать.
- **Оценка масштабируемости и производительности**. Аналитику необходимо рассчитать нагрузку на систему, а затем оценить, насколько система должна быть масштабируемой и готовой к ее росту.

- **Безопасность**. Аналитик должен учитывать требования безопасности и определять, насколько критична информация, хранимая/обрабатываемая системой и какие меры защиты должны быть приняты.
- **Технологический стек**. Аналитик должен учитывать, какие технологии доступны и какие из них лучше всего подходят для данной системы, и все это — с учетом трех пунктов выше

С чем придется сталкиваться реже:

- **Бюджет**. Аналитик должен учитывать бюджетные ограничения и выбирать архитектуру, которая соответствует этим ограничениям. Скорее всего вы не будете жестко ограничены в бюджете, и не будете обязаны рассчитывать косты проектов (от англ. cost — стоимость), это больше прерогатива сотрудников бизнес-звена.
- **Ограничения в организации и опыт команды**. Думаю, тут и так все понятно.

В любом случае — не существует правильного или универсального решения. Любое архитектурное решение это компромисс, зависящий от большого количества обстоятельств.

Паттерн Model-View-Controller

Паттерн распределяет обработку взаимодействия с пользовательским интерфейсом между тремя участниками

Типовое решение модель-представление-контроллер - одно из наиболее часто цитируемых (и, к сожалению, неверно истолковываемых). Первоначально оно появилось в виде инфраструктуры, разработанной Тригве Реенскаугом (Trigve Reenskaug) для платформы Smalltalk в конце 70-х годов прошлого столетия. С тех пор оно сыграло значительную роль в разработке множества инфраструктур и легло в основу целого ряда концепций проектирования пользовательского интерфейса.

Этот паттерн также известен как MVC-шаблон. Он разделяет интерактивные прикладные программы на 3 части:

1. **модель** — содержит ключевые данные и функционал;
2. **представление (вид)** — показывает информацию пользователю (можно задавать более одного представления);
3. **контроллер** — занимается обработкой данных от пользователя.

Это делается с целью разграничения внутреннего представления информации от способов ее представления и принятия от пользователя. Данная схема изолирует компоненты и позволяет эффективно реализовать повторное использование кода.

Model

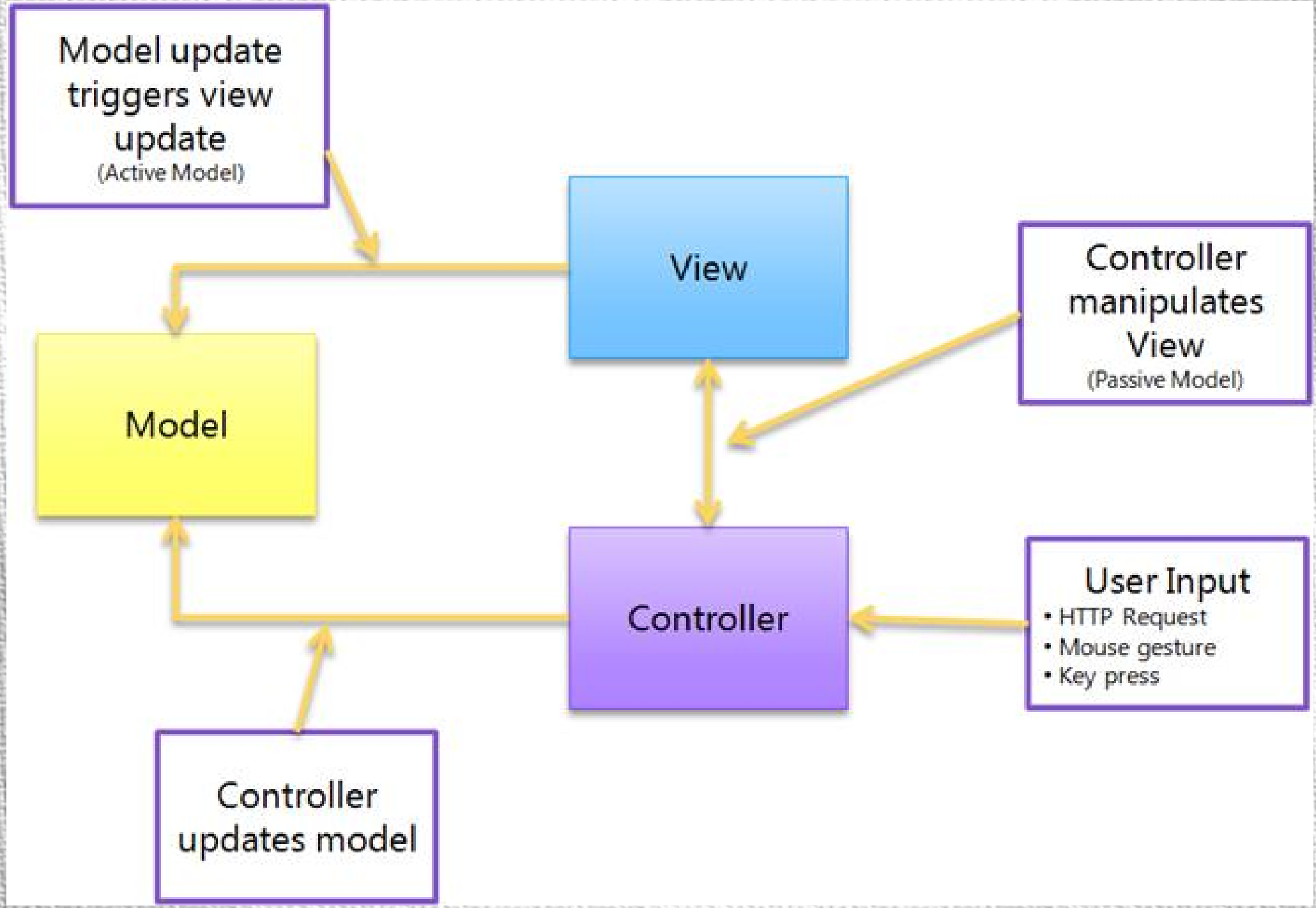
Модель – это данные вашего приложения, логика их получения и сохранения. Зачастую это модель предметной области (*domain model*), основанная на базе данных или на результатах от веб-сервисов. В некоторых случаях *domain model* хорошо проецируется на то, что вы видите на экране. Но иногда перед использованием ее необходимо адаптировать, изменить или расширить.

View

View отвечает за отображение UI на экране. Без библиотек виджетов, это означало самостоятельную отрисовку блоков, кнопок, полей ввода и т. п. View также может наблюдать за моделью и отображать данные из неё.

Controller

Controller обрабатывает действия пользователя и затем обновляет Model или View. Если пользователь взаимодействует с приложением (нажимает кнопки на клавиатуре, передвигает курсор мыши), контроллер получает уведомление об этих действиях и решает, что с ними делать.

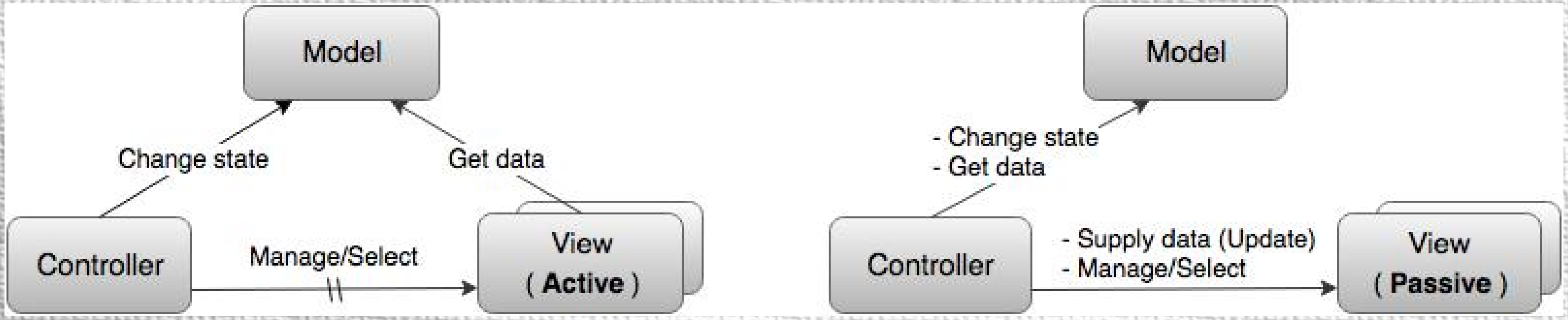

Model update triggers view update
(Active Model)
View
Controller manipulates View
(Passive Model)
Model
Controller
User Input
• HTTP Request
• Mouse gesture
• Key press
Controller updates model

1. **Модель** ничего не знает ни о **Виде**, ни о **Контроллере**, что делает возможным ее разработку и тестирование как независимого компонента. И это является главным моментом MVC.
2. **Вид** отображает **Модель.** И значит, он каким-то образом должен получать из нее нужные для отображения данные. Наиболее распространены следующие два варианта:
    1) **Активный Вид**, который знает о Модели и сам берет из нее нужные данные.
    2) **Пассивный Вид**, которому данные поставляет Контроллер. В этом случае Вид с Моделью никак не связан.

3. Видов может быть несколько — они могут по разному отображать одни и те же данные, например в виде таблицы или графика, или же отвечать за отображение разных частей данных из **Модели**.

Контроллер является пожалуй самым неоднозначным компонентом. Тем не менее общим является то, что **Контроллер** всегда знает о **Модели** и может ее изменять (как правило в результате действий пользователя).
А также он может осуществлять управление **Видом/Видами** (особенно если их несколько) и соответственно знать о **Видах**, но это не обязательно.

Отсюда мы получаем базовые (максимально упрощенные) схемы двух наиболее часто встречающихся разновидностей MVC. Перечеркнутой линией обозначена необязательная связь ***Контроллера*** с ***Видом***.

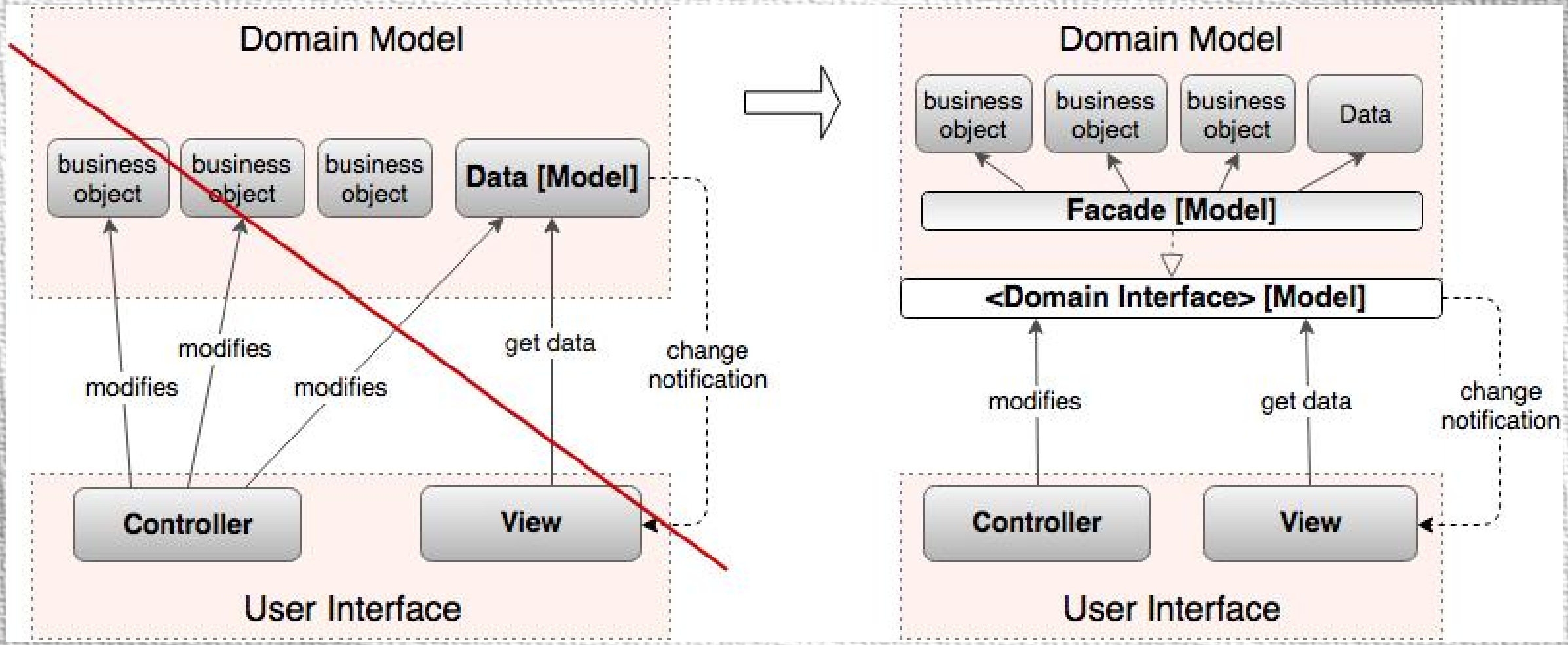


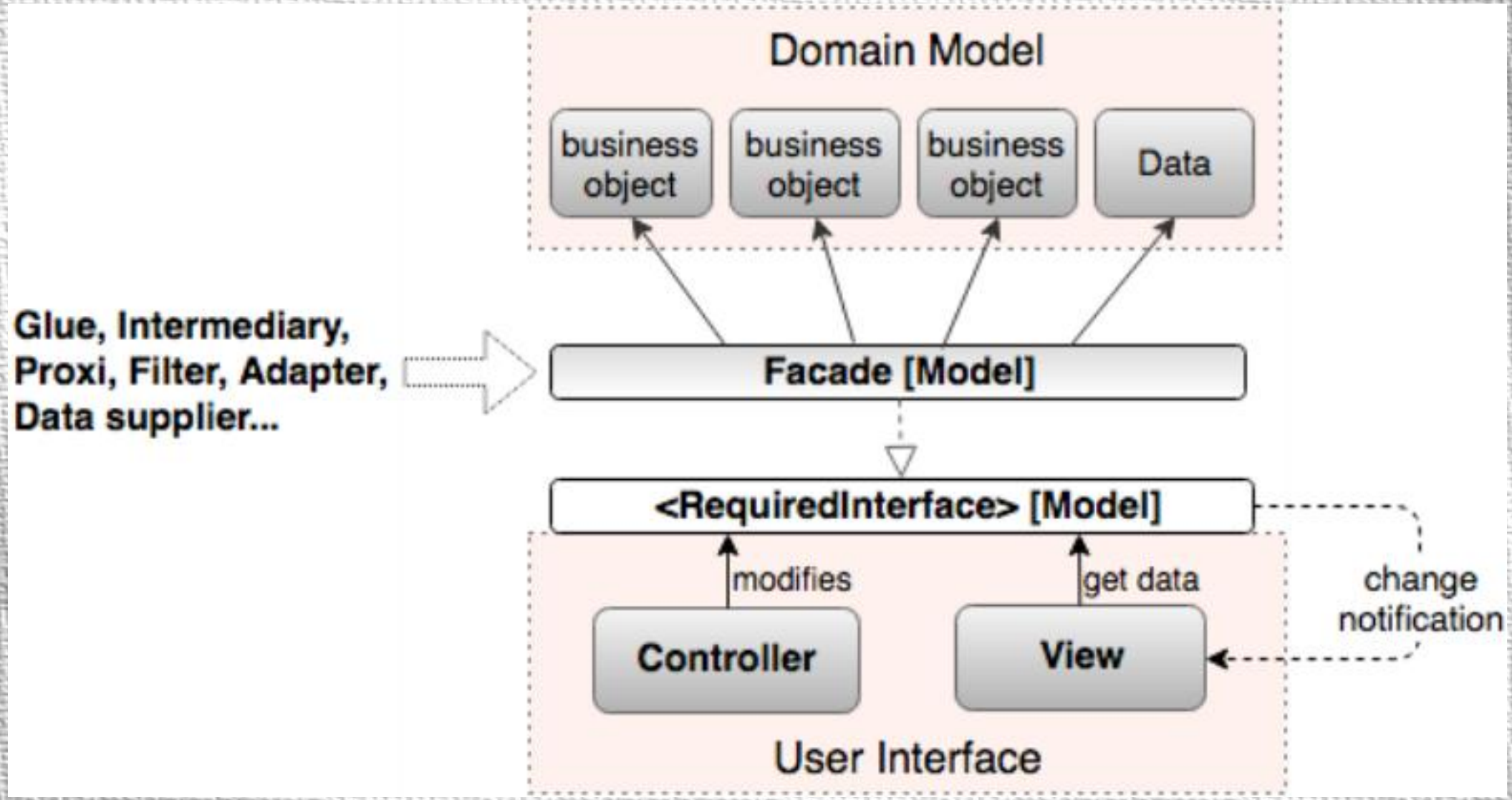
Ключевая идея MVC состоит в том, что пользовательское приложение делится на два модуля – один из которых моделирует предметную область и реализует бизнес логику (***доменная модель***), а второй отвечает за взаимодействие с пользователем (***пользовательский интерфейс***).

Но при этом Модель в «MVC схеме» вовсе не тождественна доменной модели (которая может быть сколь угодно сложной и состоять из множества объектов), а является всего лишь ее **интерфейсом** и **фасадом**.

Так что ни **Вид** ни Контроллер разумеется не должны знать о том, как устроен модуль предметной области (доменная модель), где и в каком формате там хранятся данные, и как именно осуществляется управление.

Они взаимодействуют лишь с интерфейсом и реализующим его объектом-фасадом, который предоставляет все нужные данные в нужном формате и удобный набор высокоуровневых команд для управления подсистемой, а также реализует шаблон **Наблюдатель**, для извещения о значимых изменениях в подсистеме. И если мы захотим поменять базу данных, использовать облако, или вообще собирать нужные нам данные из различных источников в сети… если внесем какие угодно изменения в бизнес логику приложения, но при этом оставим неизменным интерфейс-фасад, то ни Вид, ни Контроллер это никак не затронет. Мы имеем архитектуру устойчивую к изменениям.

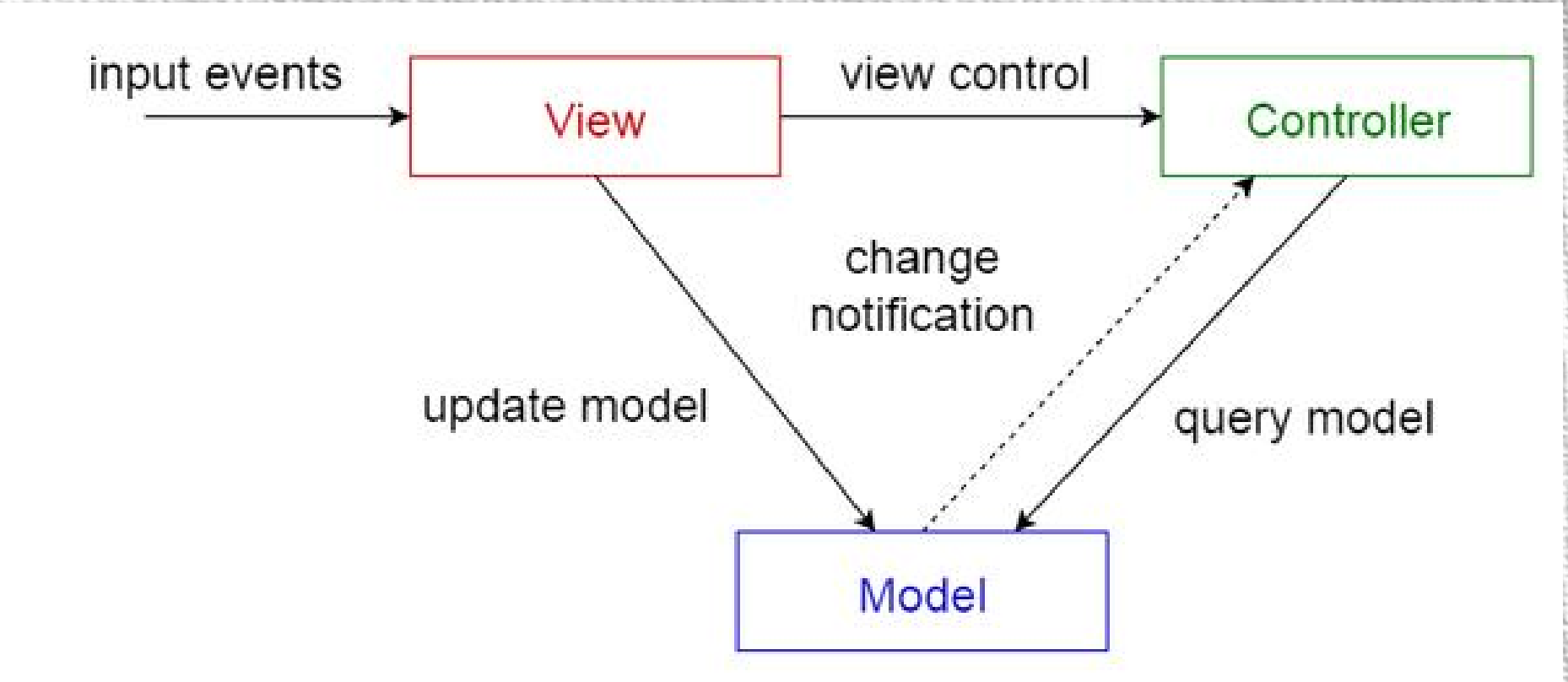

Domain Model
business object
business object
business object
Data [Model]
modifies
modifies
modifies
get data
change notification
Controller
View
User Interface
Domain Model
business object
business object
business object
Data
Facade [Model]
<Domain Interface> [Model]
modifies
get data
change notification
Controller
View
User Interface


Domain Model
business object
business object
business object
Data
Glue, Intermediary,
Proxi, Filter, Adapter,
Data supplier...
Facade [Model]
<RequiredInterface> [Model]
modifies
get data
change notification
Controller
View
User Interface

Модели это не данные, а исключительно интерфейсы/объекты-посредники/фильтры (Models as Proxies, Models as Filters), обеспечивающие удобный доступ к данным, которые могут находится где угодно – на разных машинах, в разных форматах: “О чем большинство программистов не думает, так это о том, что модели являются всего лишь интерфейсами. Они не должны содержать никаких данных! Модели-посредники расширяют охват и позволяют использовать уже существующие данные где бы они не находились”.

Использование

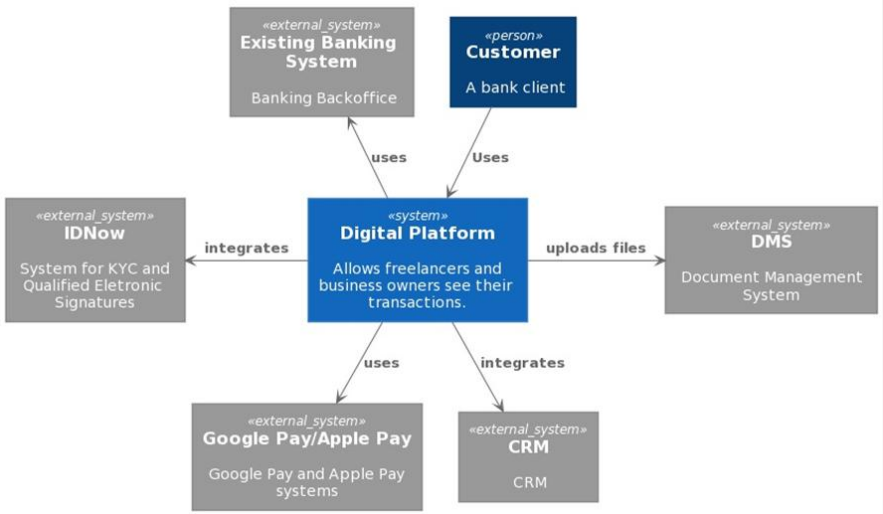
- Архитектура WWW-приложений, написанных на основных языках программирования.
- Веб-фреймворки (например, Django и Rails)



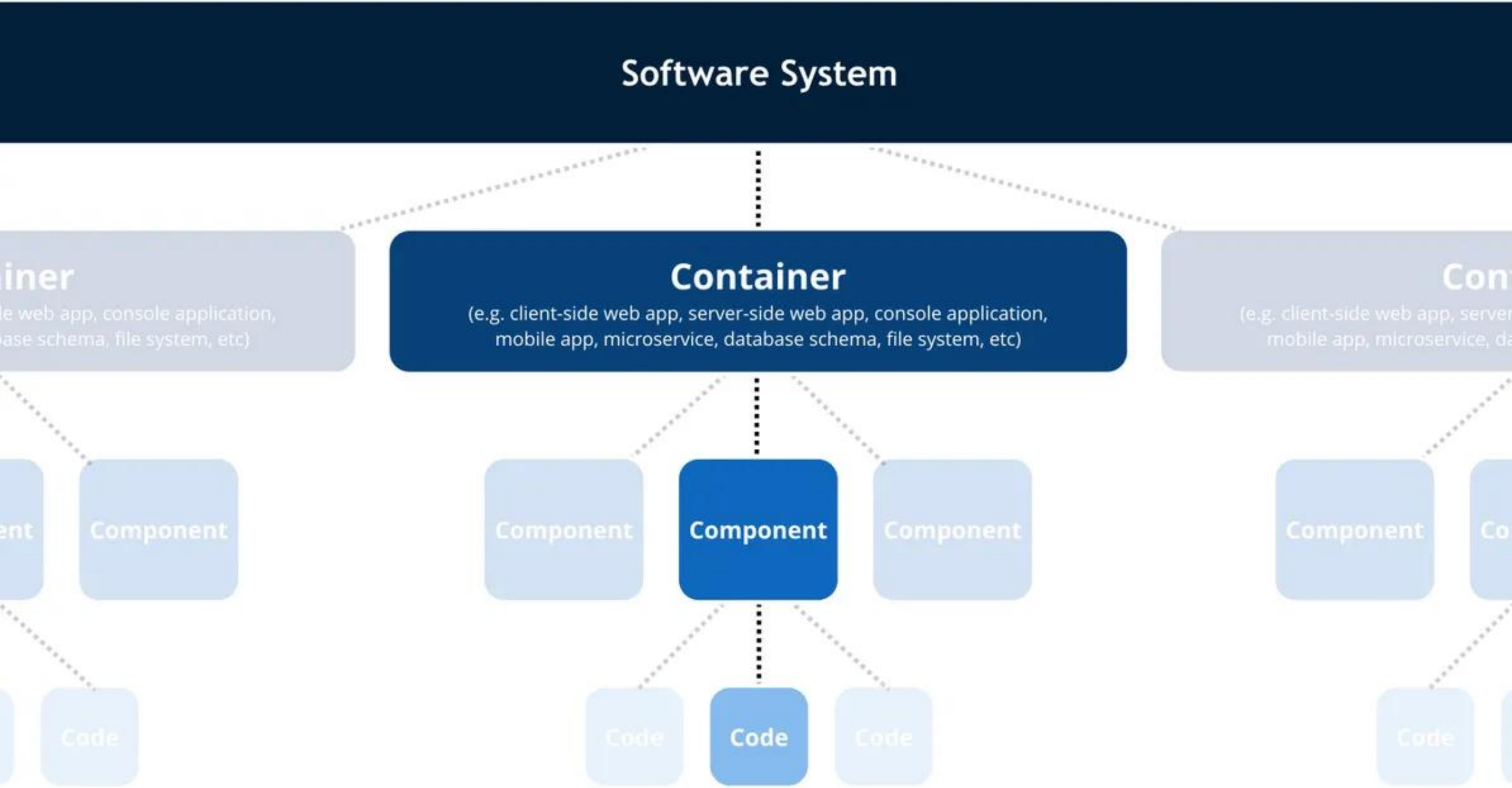
После многих лет парадигма программирования несколько изменилась – появились пользовательские элементы управления (виджеты). Виджеты как отрисовывают самих себя, так и интерпретируют пользовательский ввод. Кнопка знает, что делать, если вы кликните по ней. Поле ввода знает, что делать, если вы вводите текст в нём. Это уменьшает потребность в контроллере, и паттерн MVC стал менее актуальным.

Большинство примеров паттерна MVC фокусируются на очень небольших компонентах, таких как реализация текстового окна или реализация кнопки. При использовании более современных технологий пользовательского интерфейса, как правило, нет необходимости в этом паттерне. Но он может помочь, если вы разрабатываете свой виджет, используя очень низкий уровень API (например, Direct X).

Однако так как по-прежнему существует необходимость отделения логики приложения от представления и от данных, набрал популярность другой паттерн под названием Model-View-Presenter (MVP).

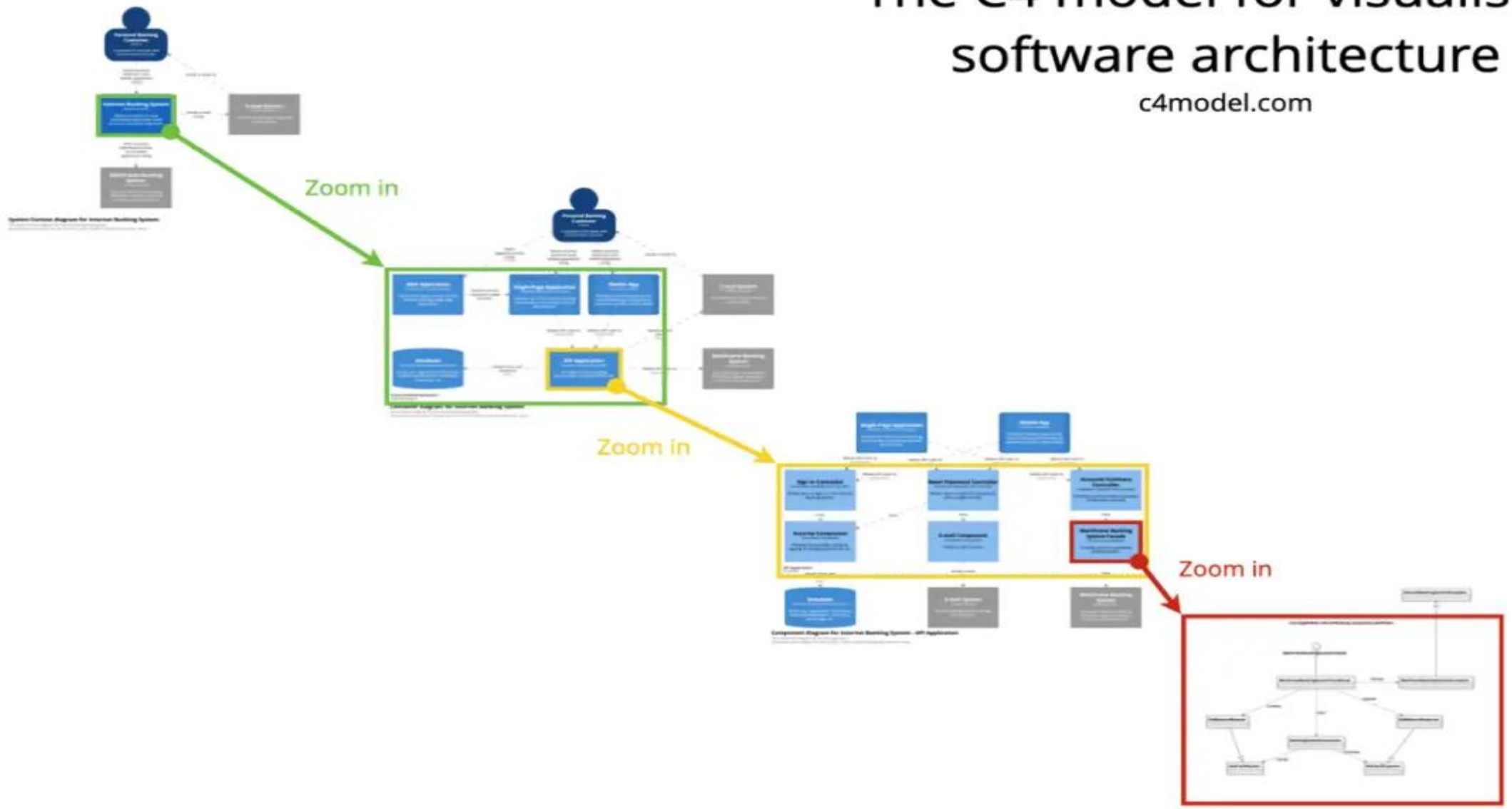


Диаграммы для архитектора


Software System
Container
(e.g. client-side web app, server-side web app, console application, mobile app, microservice, database schema, file system, etc)
Component
Component
Component
Component
Code
Code
Code

The C4 model for visualising software architecture

c4model.com



Level 1
Context

Level 2
Containers

Level 3
Components

Level 4
Code

Задача архитектора решений — четко донести проект системы до бизнеса, руководителей проектов и разработчиков. Нельзя просто нарисовать одно изображение, это невозможно и не принесет никому пользы. Вместо этого лучше сгруппировать различные проблемы и создать набор диаграмм, описывающих каждое представление. Конечно, есть миллиард способов сделать это. Как выбрать подходящий? Наиболее популярны 5 диаграмм:

1. *контекстная диаграмма C4,*
2. *диаграмма контейнеров,*
3. *диаграмма развертывания,*
4. *диаграмма последовательности и*
5. *диаграмма вариантов использования.*

1. КОНТЕКСТНАЯ ДИАГРАММА

Веб-сайт, посвященный модели C4 (**C**ontext, **C**ontainer, **C**omponent and **C**ode – назовем *сифо*), довольно хорошо объясняет свои диаграммы, я же поделюсь своим представлением, как эта модель работает.

Любая система работает в некотором контексте — окружении. В первую очередь, это пользователи и другие системы. Пользователи могут иметь разные роли, такие как создатель контента, читатель, администратор. Также они могут быть внутренними или внешними. Другие программы могут быть источником данных для вашей системы или получать информацию от нее. Важно понимать этот контекст, чтобы правильно спроектировать систему и напомнить себе о необходимости интеграции с внешними системами.

По русски ЧетыреКо

1. **ко**нтент
2. **ко**нтейнер
3. **ко**мпонент
4. **ко**д

Как рисовать

- Определите пользователей.
- Определите внешние системы.
- Создайте единый прямоугольник, изображающий вашу систему.
- Добавьте связи между системой, пользователями и внешними системами.
- Напишите содержательные комментарии по каждому компоненту.

Инструменты

Существуют различные инструменты, которые можно использовать для создания контекстной диаграммы. Существуют трафареты C4 для Omni Graffle, примеры C4 для LucidChart, шаблоны есть также в draw.io.

Чтобы использовать диаграммы как код, попробуйте PlantUML.

Можно рекомендовать SaaS-продукт icepanel.io. Это веб-сервис, который предоставляет возможность построения архитектурных диаграмм в нотации C4, в т.ч. на бесплатном тарифе. Пользоваться сервисом довольно удобно. Например, можно сразу привязать технологию к проектируемой модели, выбрать протокол связи, СУБД или фреймворк для разработки приложения. Также можно поделиться своей моделью с коллегами, расшарив публичную ссылку.

Важно!

Контекстная диаграмма — это первое, что вы создаете при работе с системой. Если этого не сделать, это может стоить вам пропущенных интеграций и ошибок при проектировании системы. Контекст может включать некоторые детали более низкого уровня, это нормально.

Контекстная диаграмма — это важнейшее представление, обеспечивающее высокий уровень понимания людей и систем, с которыми будет взаимодействовать ваша будущая система.

2. ДИАГРАММА КОНТЕЙНЕРОВ

(приложения и хранилища данных)

Контейнеры здесь не означают обязательно докер-контейнеры. Контейнер — это любой развертываемый объект или хранилище данных с точки зрения C4. Это может быть мобильное приложение, веб-сайт, виртуальная машина, докер-контейнер, база данных или хранилище объектов; **все, что вы можете развернуть**. По опыту эта диаграмма – самая сложная, а потому привлекает к себе особое внимание. Это, можно сказать, главная диаграмма, над которой нужно работать!

Диаграмма контейнера показывает высокоуровневую форму архитектуры программного обеспечения и то, как обязанности распределены по нему. В нем также показаны основные технологические решения и способы взаимодействия контейнеров друг с другом.

Эта диаграмма в разы масштабнее и более нагруженная, чем предыдущая. То, что было одним прямоугольником, теперь состоит из нескольких прямоугольников и стрелок. Эти прямоугольники теперь и являются контейнерами.

Только не Docker! В модели C4 контейнер представляет собой приложение или хранилище данных. **Контейнер — это то, что должно быть запущенным для того, чтобы вся программная система работала**. В реальном выражении контейнер — это что-то вроде:

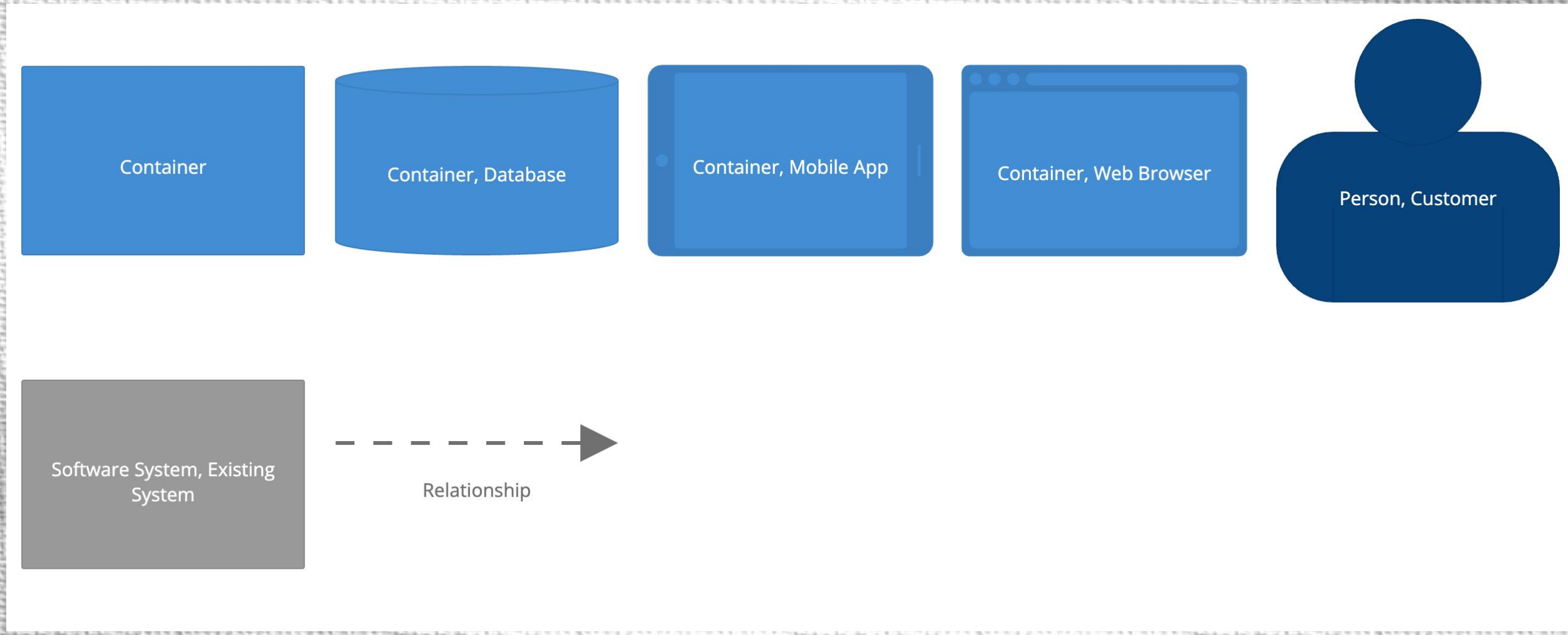
- **Серверное веб-приложение**: веб-приложение Java EE, работающее на Apache Tomcat, приложение ASP.NET MVC, работающее на Microsoft IIS, приложение Ruby on Rails, работающее на WEBrick, приложение Node.js и т. д.
- **Клиентское веб-приложение**: JavaScript-приложение, работающее в веб-браузере с использованием Angular, Backbone.JS, jQuery и т. д.

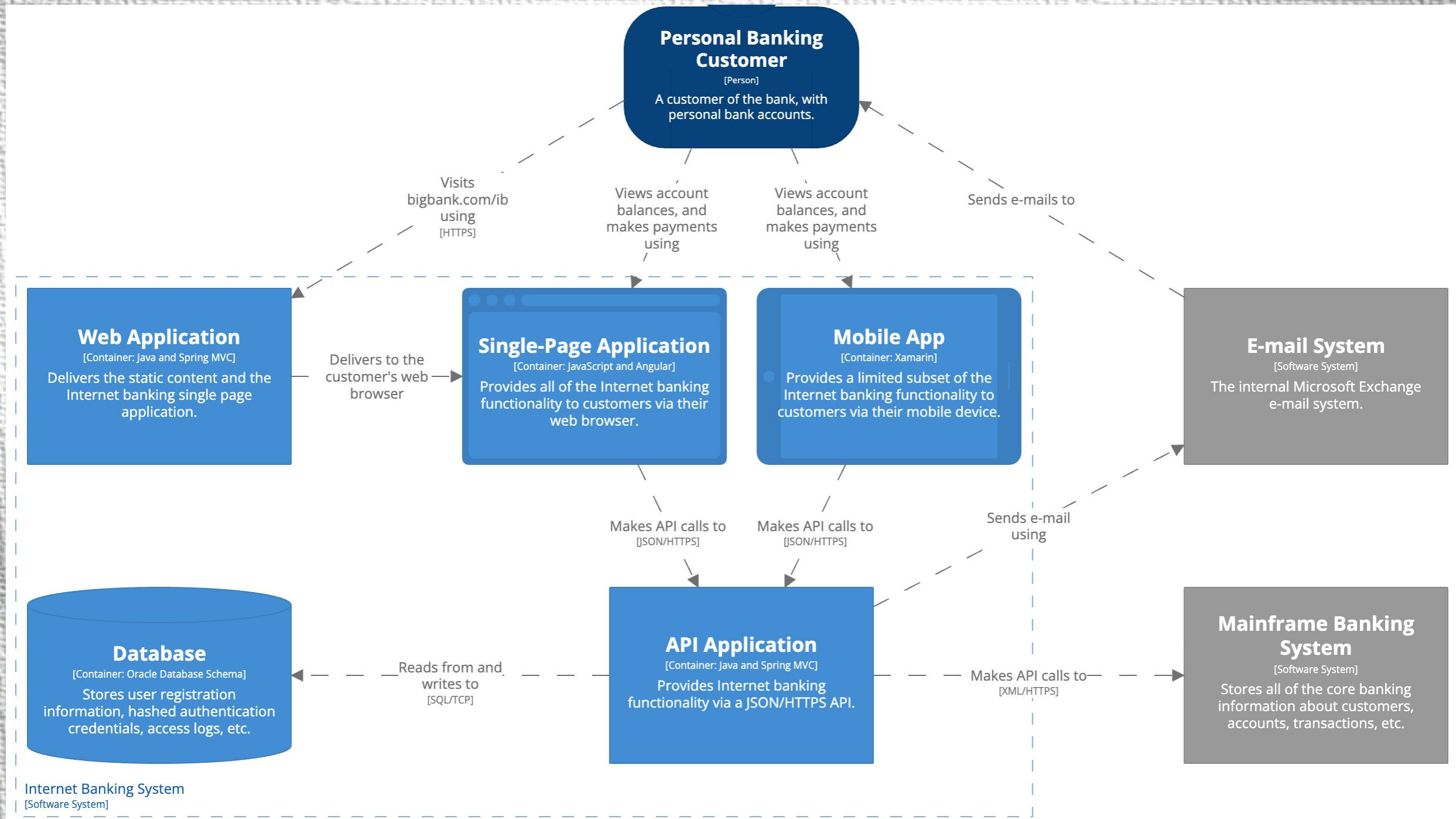
- **Клиентское настольное приложение**: настольное приложение Windows, написанное с помощью WPF, настольное приложение OS X, написанное с помощью Objective-C, кроссплатформенное настольное приложение, написанное с использованием JavaFX и т. д.
- **Мобильное приложение**: приложение для Apple iOS, приложение для Android, приложение для Microsoft Windows Phone и т. д.
- **Консольное приложение** на стороне сервера: автономное (например, "public static void main") приложение, пакетный процесс и т. д.
- **Бессерверная функция**: одна бессерверная функция (например, Amazon Lambda, функция Azure и т. д.).

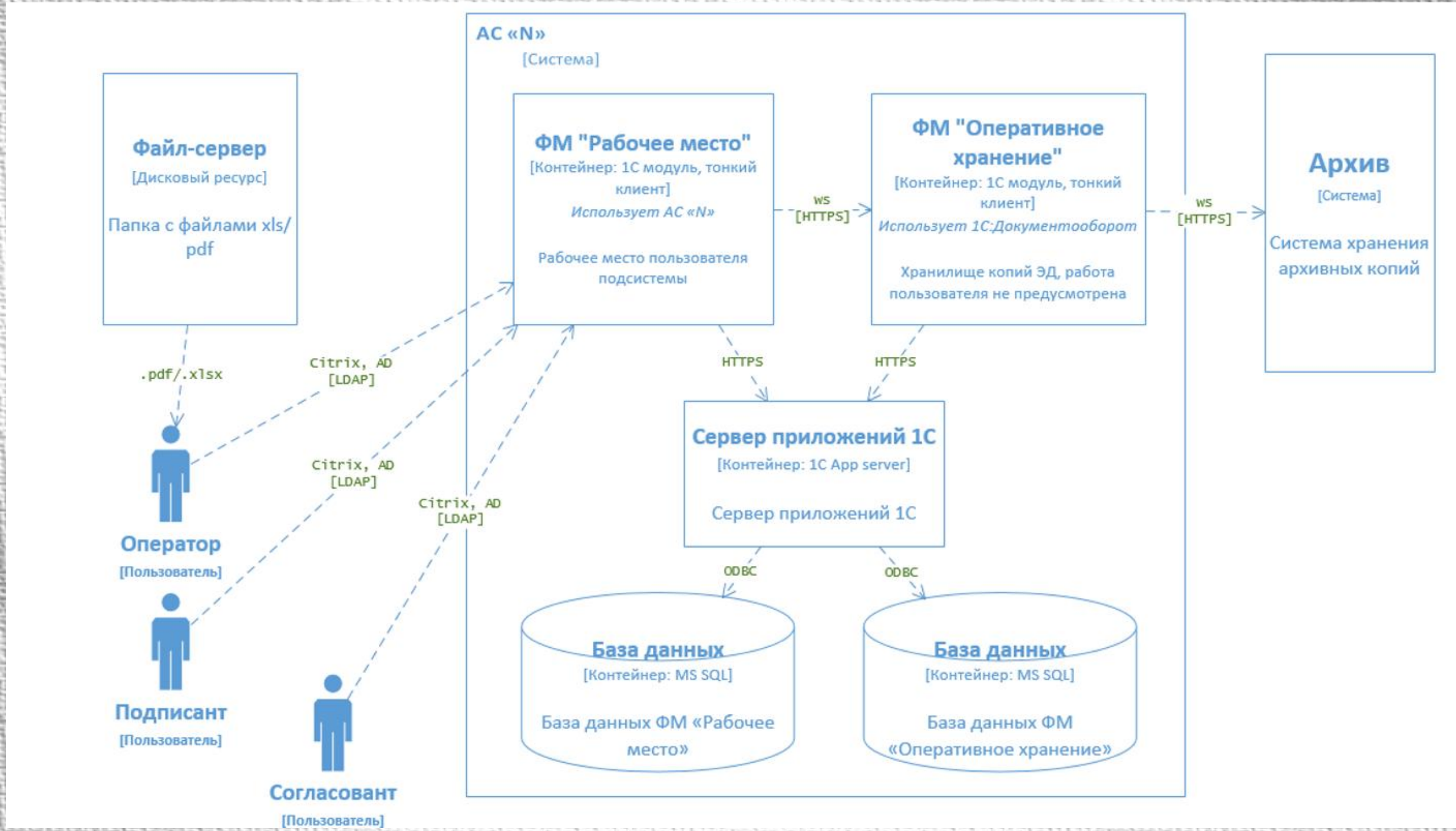
- **База данных**: схема или база данных в системе управления реляционными базами данных, хранилище документов, графовая база данных и т. д., такие как MySQL, Microsoft SQL Server, Oracle Database, MongoDB, Riak, Cassandra, Neo4j и т. д.
- **Хранилище BLOB-объектов или хранилище контента**: хранилище BLOB-объектов (например, Amazon S3, Microsoft Azure Blob Storage и т. д.) или сеть доставки контента (например, Akamai, Amazon CloudFront и т. д.).
- **Файловая система**: Полная локальная файловая система или часть более крупной сетевой файловой системы (например, SAN, NAS и т. д.).
- **Сценарий оболочки**: Один сценарий оболочки, написанный на Bash и т.д.

Контейнер — это, по сути, контекст или граница, внутри которой выполняется какой-то код или хранятся какие-то данные.

Кроме того, каждый контейнер представляет собой отдельно развертываемую/запускаемую вещь или среду выполнения, обычно (но не всегда) работающую в собственном пространстве процессов. Из-за этого взаимодействие между контейнерами обычно принимает форму межпроцессного взаимодействия.


Container
Container, Database
Container, Mobile App
Container, Web Browser
Person, Customer
Software System, Existing System
Relationship


Personal Banking Customer
[Person]
A customer of the bank, with personal bank accounts.
Visits bigbank.com/ib using
[HTTPS]
Views account balances, and makes payments using
Views account balances, and makes payments using
Sends e-mails to
Web Application
[Container: Java and Spring MVC]
Delivers the static content and the Internet banking single page application.
Delivers to the customer's web browser
Single-Page Application
[Container: JavaScript and Angular]
Provides all of the Internet banking functionality to customers via their web browser.
Mobile App
[Container: Xamarin]
Provides a limited subset of the Internet banking functionality to customers via their mobile device.
E-mail System
[Software System]
The internal Microsoft Exchange e-mail system.
Makes API calls to
[JSON/HTTPS]
Makes API calls to
[JSON/HTTPS]
Sends e-mail using
Database
[Container: Oracle Database Schema]
Stores user registration information, hashed authentication credentials, access logs, etc.
Reads from and writes to
[SQL/TCP]
API Application
[Container: Java and Spring MVC]
Provides Internet banking functionality via a JSON/HTTPS API.
Makes API calls to
[XML/HTTPS]
Mainframe Banking System
[Software System]
Stores all of the core banking information about customers, accounts, transactions, etc.
Internet Banking System
[Software System]


АС «N»
[Система]
Файл-сервер
[Дисковый ресурс]
Папка с файлами xls/ pdf
ФМ "Рабочее место"
[Контейнер: 1С модуль, тонкий клиент]
Использует АС «N»
Рабочее место пользователя подсистемы
WS [HTTPS]
ФМ "Оперативное хранение"
[Контейнер: 1С модуль, тонкий клиент]
Использует 1С:Документооборот
Хранилище копий ЭД, работа пользователя не предусмотрена
WS [HTTPS]
Архив
[Система]
Система хранения архивных копий
.pdf/.xlsx
Citrix, AD [LDAP]
Citrix, AD [LDAP]
Citrix, AD [LDAP]
HTTPS
HTTPS
Сервер приложений 1С
[Контейнер: 1С App server]
Сервер приложений 1С
ODBC
ODBC
Оператор
[Пользователь]
Подписант
[Пользователь]
Согласовант
[Пользователь]
База данных
[Контейнер: MS SQL]
База данных ФМ «Рабочее место»
База данных
[Контейнер: MS SQL]
База данных ФМ «Оперативное хранение»

Обычно проектирование на этом уровне начинается с классической слоистой трехзвенной архитектуры клиент-сервер, где пользователи взаимодействуют с клиентским приложением (Frontend), за обработку бизнес-логики отвечает серверное приложение (Backend), а за хранение данных – базы и файловые хранилища. Однако, в случае микросервисной архитектуры хранилище данных может быть не одно, а несколько, т.к. Backend состоит из нескольких компонентов – микросервисов, каждый из которых может иметь свою собственную базу данных.

Как рисовать

- Определите список сущностей: микросервисы, хранилища, внешние сервисы.
- Поместите их на диаграмму.
- Добавьте комментарии о назначении каждого компонента и технологии, которую он реализует.
- Добавьте соединения со стрелками.
- Добавьте значимые метки к каждой стрелке.
- Подберите цвет схемы.
- Создайте легенду.

Резюме

Диаграмма контейнера даёт представление о том, из каких развертываемых элементов состоит серверная часть, и как эти компоненты взаимодействуют друг с другом.

Примечания: На этой схеме ничего не говорится о кластеризации, балансировщиках нагрузки, репликации, отработке отказа и т. д., поскольку она, скорее всего, будет различаться в разных средах (например, в рабочей среде, промежуточной среде, разработке и т. д.). Эту информацию лучше всего фиксировать с помощью одной или нескольких схем развертывания.

3. ДИАГРАММА КОМПОНЕНТОВ

Затем вы можете увеличить масштаб и декомпозировать каждый контейнер, чтобы определить основные структурные строительные блоки и их взаимодействия.

Компоненты – это то из чего состоят контейнеры (наборы классов, отдельные пакеты (jar-, dll-библиотеки) и т.п.). В рамках идеологии модели C4, компоненты – это такие структурные элементы, которые не разворачиваются и не запускаются в виде отдельных сущностей.

Диаграмма Component показывает, как контейнер состоит из ряда «компонентов», их обязанности и детали технологии/реализации.

Слово «компонент» является чрезвычайно перегруженным термином в индустрии разработки программного обеспечения, но **в данном контексте компонент — это группа связанных функций, инкапсулированных за четко определенным интерфейсом**. Если вы используете такие языки, как Java или C#, самый простой способ представить компонент — это набор классов реализации, лежащих в основе интерфейса. Такие аспекты, как то, как эти компоненты упакованы (например, один компонент против нескольких компонентов в JAR-файле, DLL, разделяемой библиотеке и т. д.), являются отдельной и ортогональной проблемой.

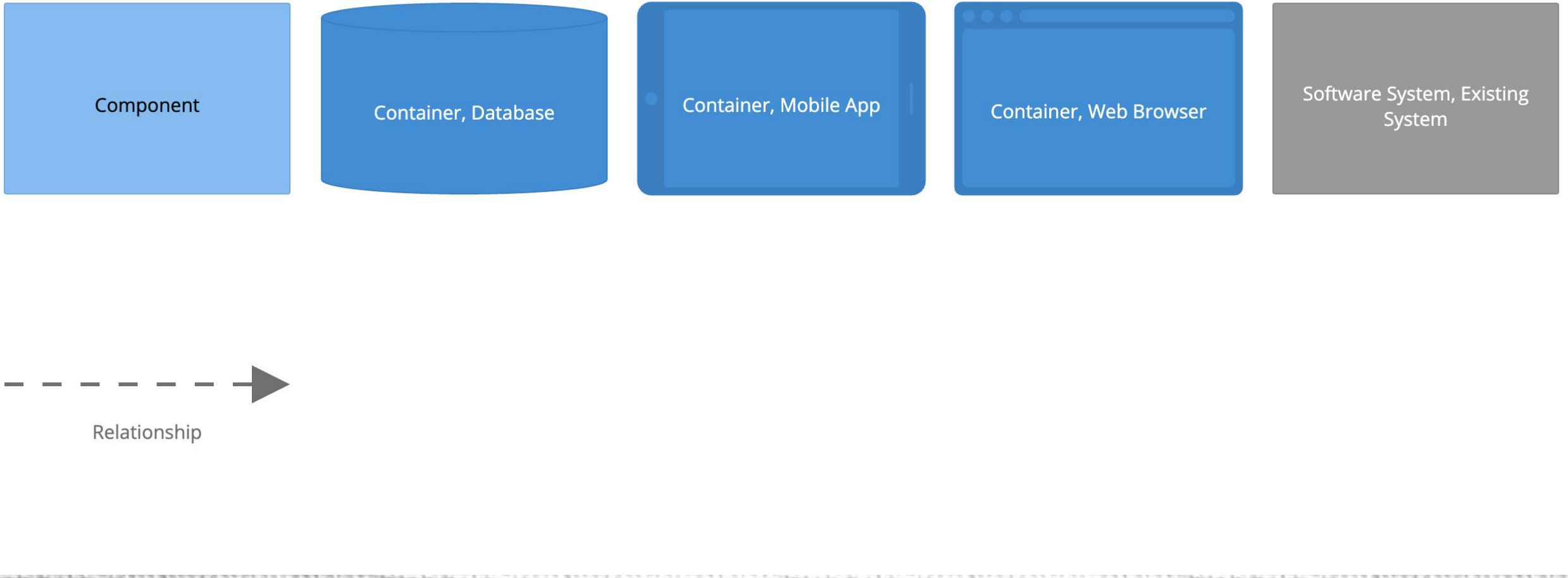
Здесь важно отметить, что все компоненты внутри контейнера обычно выполняются в одном и том же пространстве процесса. В модели C4 компоненты не являются отдельно развертываемыми единицами.

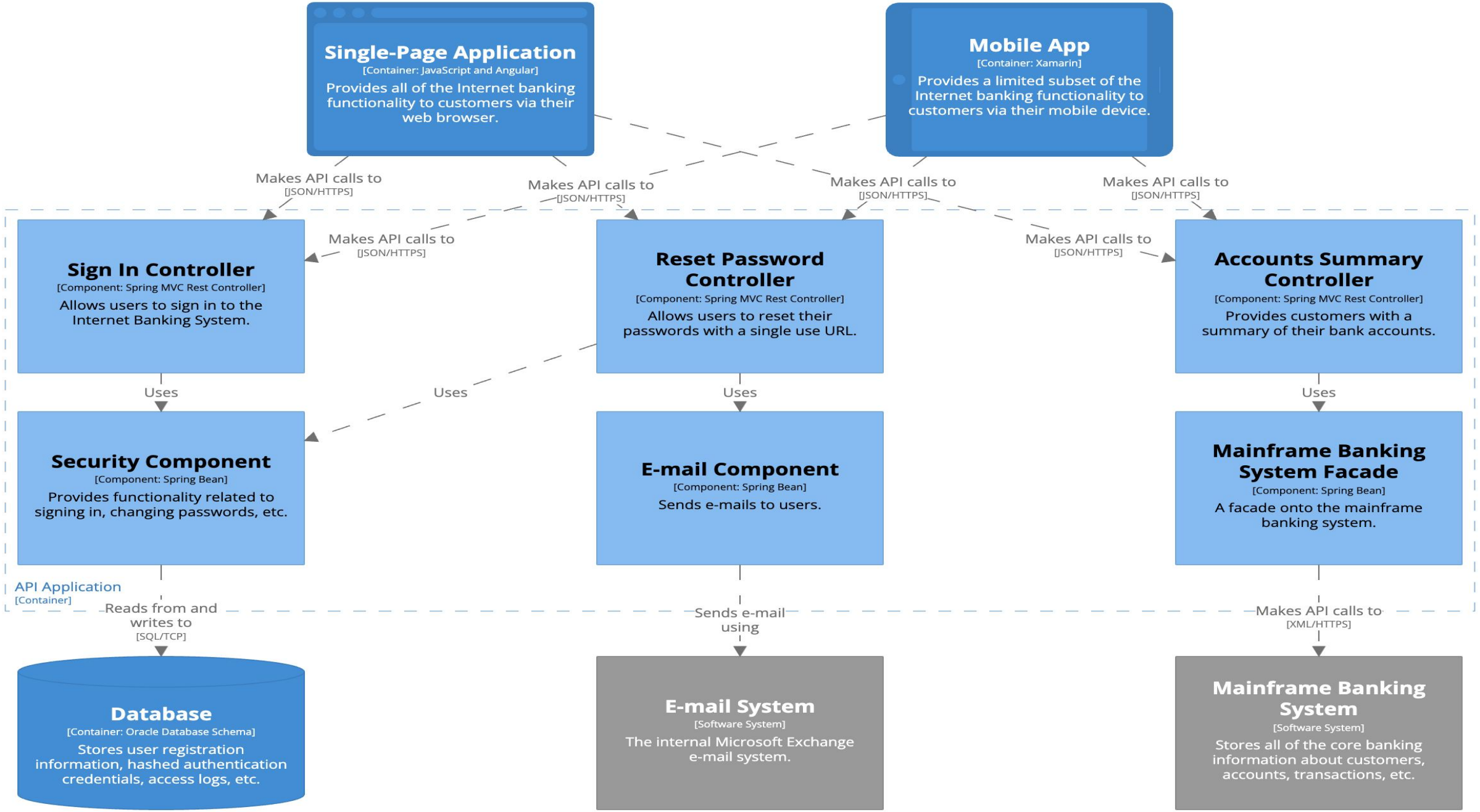
Область действия: один контейнер.

Первичные элементы: компоненты в контейнере в области видимости.
Вспомогательные элементы: контейнеры (в рамках программной системы), а также люди и программные системы, непосредственно связанные с компонентами.

Целевая аудитория: архитекторы и разработчики программного обеспечения.

Рекомендуется для большинства команд: Нет, создавайте схемы компонентов только в том случае, если вы считаете, что они приносят пользу, и рассмотрите возможность автоматизации их создания для долговременной документации.


Component
Container, Database
Container, Mobile App
Container, Web Browser
Software System, Existing System
Relationship



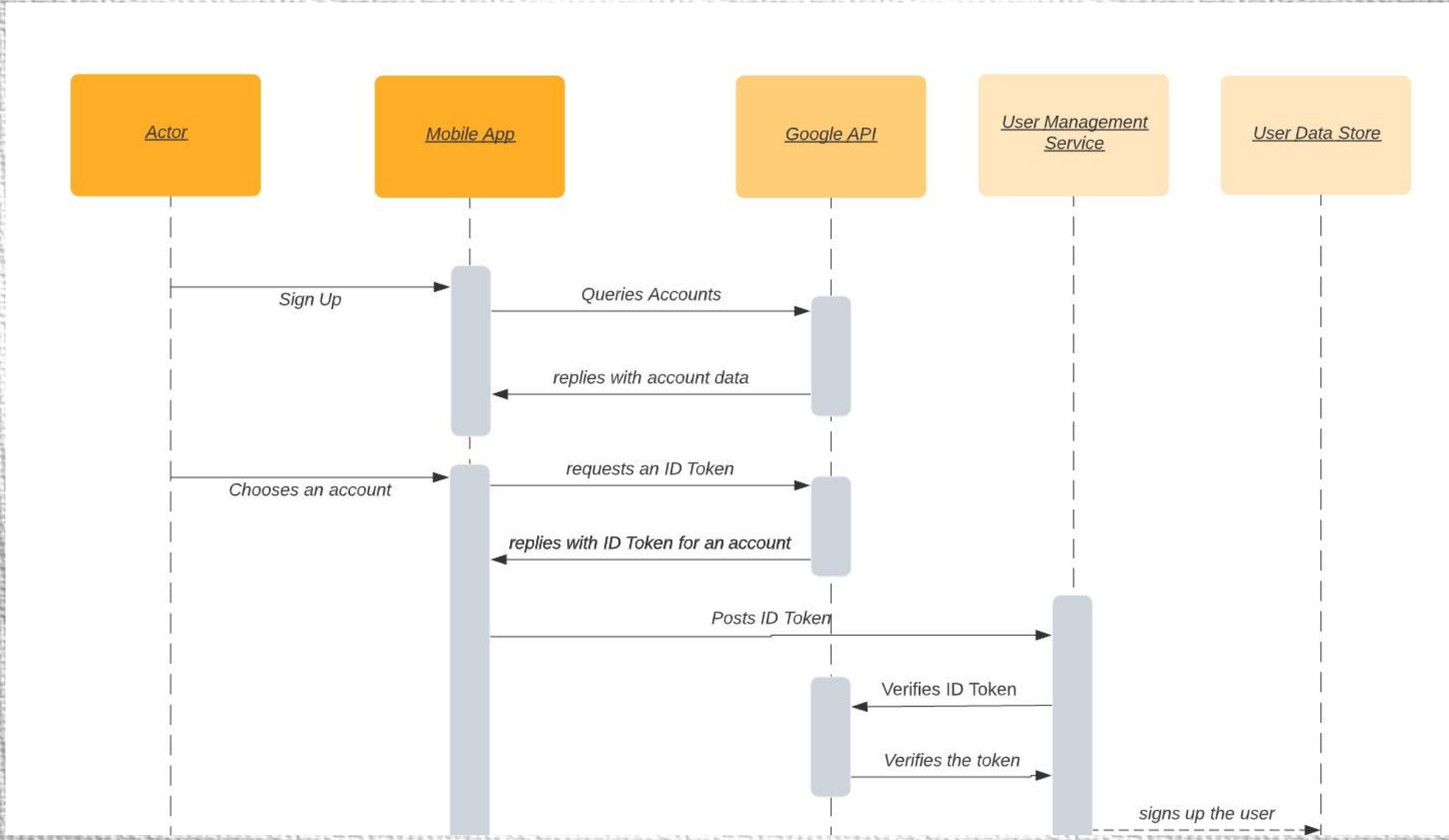
[Component] Internet Banking System - API Application

The component diagram for the API Application - diagram created with Structurizr.
Saturday, 11 November 2023 at 09:04 Greenwich Mean Time

4. ДИАГРАММА ПОСЛЕДОВАТЕЛЬНОСТЕЙ

Первые две диаграммы показывают, как элементы системы соотносятся друг с другом. Однако они не могут продемонстрировать, что происходит внутри системы.

Например, пользователь регистрируется в вашей системе. Какие компоненты задействованы? Какие действия срабатывают? Как компоненты взаимодействуют друг с другом? Диаграмма последовательностей может ответить на эти вопросы.


Actor
Mobile App
Google API
User Management Service
User Data Store
Sign Up
Queries Accounts
replies with account data
Chooses an account
requests an ID Token
replies with ID Token for an account
Posts ID Token
Verifies ID Token
Verifies the token
signs up the user

Вверху мы видим взаимодействующие объекты: люди, веб-приложения и мобильные приложения, внешние системы, сервисы и хранилища данных. У каждого объекта есть вертикальная линия внизу. Взаимодействие между сервисами обозначено горизонтальными стрелками между вертикальными линиями. Эти стрелки могут быть разных типов в зависимости от того, синхронная это операция или асинхронная. Серые прямоугольники показывают, что процесс занимает некоторое время, и длина должна указывать на продолжительность: чем длиннее прямоугольник, тем больше время.

Как рисовать

- Выберите функцию (вход, покупка и т. д.).
- Определите сущности, участвующие в этом процессе.
- Поместите их на диаграмму.
- Добавьте взаимодействие (стрелки).
- Добавьте ценные комментарии к каждой стрелке.

Инструменты

Для создания этой диаграммы можно использовать draw.io, LucidChart, PlantUML.

6. ДИАГРАММА РАЗВЕРТЫВАНИЯ

С помощью диаграмм контекста, контейнеров и диаграммы последовательностей, можно понять, из каких частей состоит система, как они связаны и взаимодействуют друг с другом. Но вряд ли они могут ответить на вопросы доступности, масштабируемости и безопасности. В этом поможет схема развертывания. **Есть несколько разных вещей, которые необходимо отобразить.**

Вычислительные ресурсы. Это могут быть виртуальные машины, докеры, кластеры Kubernetes и облачные функции. Мобильные устройства и настольные компьютеры также можно рассматривать как вычислительные ресурсы.

Хранилища. Постоянные хранилища данных, такие как реляционные базы данных и базы данных nosql, хранилища двоичных файлов, таких как изображения, музыка и видео, хранилища больших данных и так далее.

Ресурсы обмена сообщениями. Установки Kafka / RabbitMQ, Google Cloud pub / sub, AWS SQS и другие.

Сети. Ваши компьютеры используют сети для связи друг с другом. Должны отображаться как физические, так и виртуальные сети.

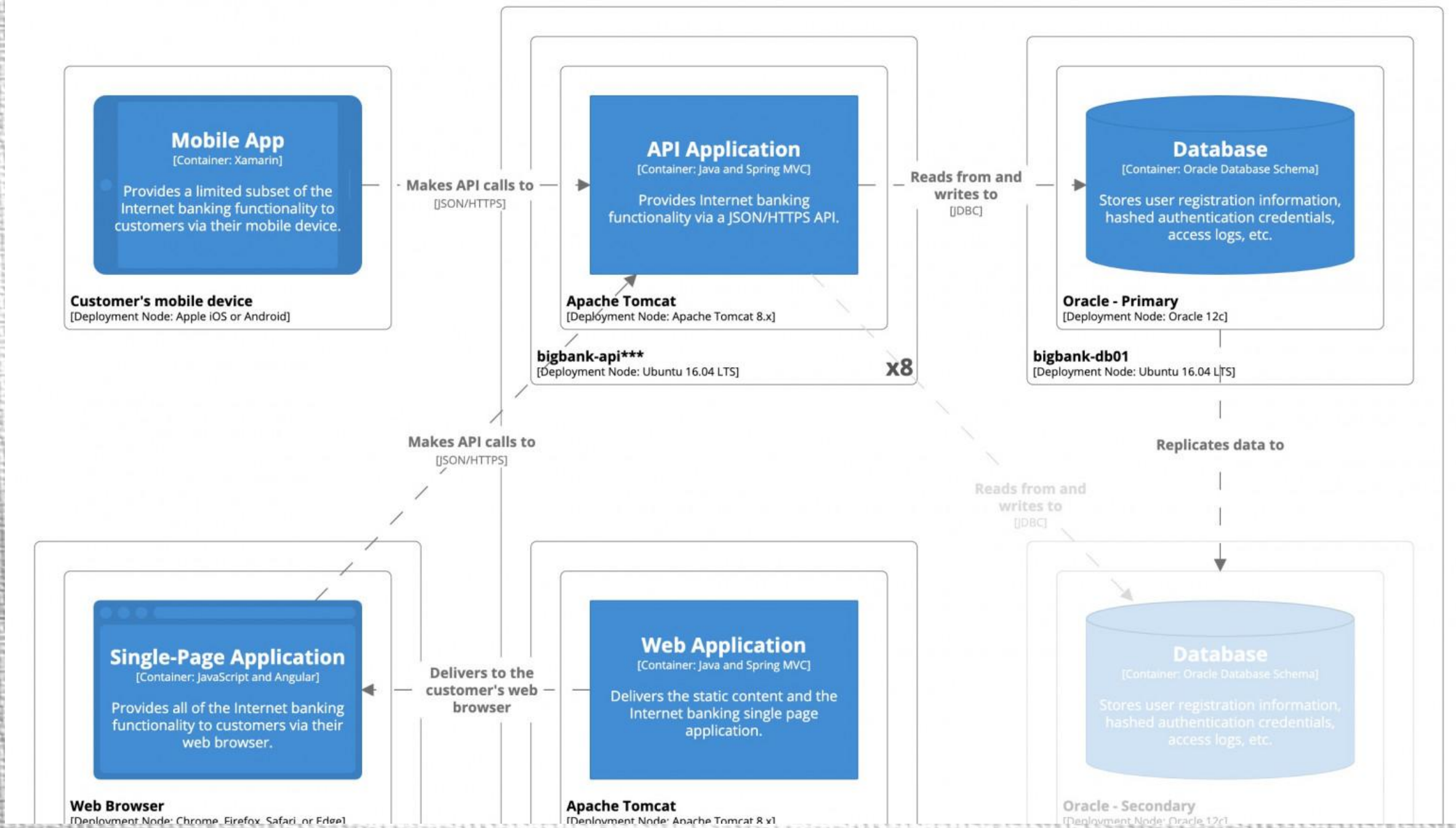
Зоны доступности. Вы можете думать о них как о центрах обработки данных.

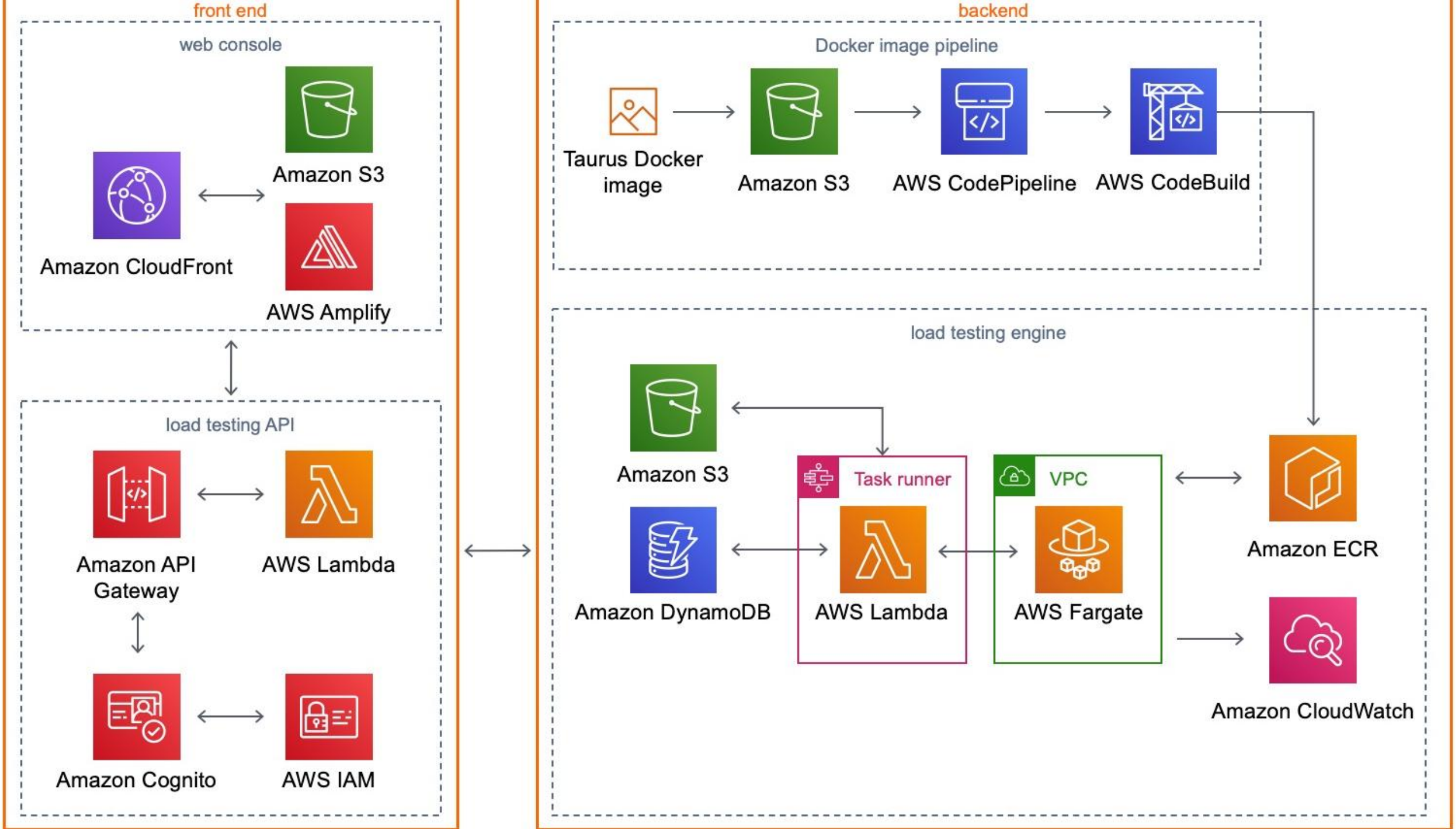
Узлы инфраструктуры. DNS-серверы, балансировщики нагрузки, брандмауэры, сети CDN

На схеме отображаются вычислительные ресурсы, ресурсы хранения и обмена сообщениями, а также сети и зоны доступности. Он также включает узлы инфраструктуры, которые не выполняют никаких функциональных требований, но вместо этого обращаются к нефункциональным.

Как нарисовать

- Разместите основные блоки: браузеры, мобильные устройства, общедоступное облако, центры обработки данных.
- Разместите вычислительные ресурсы и ресурсы хранения.
- Добавьте узлы инфраструктуры.
- Добавьте сети.
- Добавьте сетевые вызовы между узлами.
- Добавьте ресурсы мониторинга.


Mobile App
[Container: Xamarin]
Provides a limited subset of the Internet banking functionality to customers via their mobile device.
Customer's mobile device
[Deployment Node: Apple iOS or Android]
Makes API calls to
[JSON/HTTPS]
API Application
[Container: Java and Spring MVC]
Provides Internet banking functionality via a JSON/HTTPS API.
Apache Tomcat
[Deployment Node: Apache Tomcat 8.x]
bigbank-api***
[Deployment Node: Ubuntu 16.04 LTS]
x8
Reads from and writes to
[JDBC]
Database
[Container: Oracle Database Schema]
Stores user registration information, hashed authentication credentials, access logs, etc.
Oracle - Primary
[Deployment Node: Oracle 12c]
bigbank-db01
[Deployment Node: Ubuntu 16.04 LTS]
Replicates data to
Makes API calls to
[JSON/HTTPS]
Reads from and writes to
[JDBC]
Single-Page Application
[Container: JavaScript and Angular]
Provides all of the Internet banking functionality to customers via their web browser.
Web Browser
[Deployment Node: Chrome, Firefox, Safari, or Edge]
Delivers to the customer's web browser
Web Application
[Container: Java and Spring MVC]
Delivers the static content and the Internet banking single page application.
Apache Tomcat
[Deployment Node: Apache Tomcat 8.x]
Database
[Container: Oracle Database Schema]
Stores user registration information, hashed authentication credentials, access logs, etc.
Oracle - Secondary
[Deployment Node: Oracle 12c]


front end
web console
Amazon S3
Amazon CloudFront
AWS Amplify
load testing API
Amazon API Gateway
AWS Lambda
Amazon Cognito
AWS IAM
backend
Docker image pipeline
Taurus Docker image
Amazon S3
AWS CodePipeline
AWS CodeBuild
load testing engine
Amazon S3
Task runner
VPC
Amazon ECR
Amazon DynamoDB
AWS Lambda
AWS Fargate
Amazon CloudWatch

Важно

Каждая система имеет требования к безопасности, производительности, доступности и другим возможностям. Диаграмма развертывания помогает удовлетворить эти требования. В большинстве случаев диаграмма развертывания единственная, отображающая сетевые аспекты вашего решения. Диаграмма развертывания должна ясно и понятно показывать, как запросы проходят через систему.

Резюме

Диаграмма развертывания дополняет понимание системы с точки зрения внешнего вида.

7. ДИАГРАММА ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ

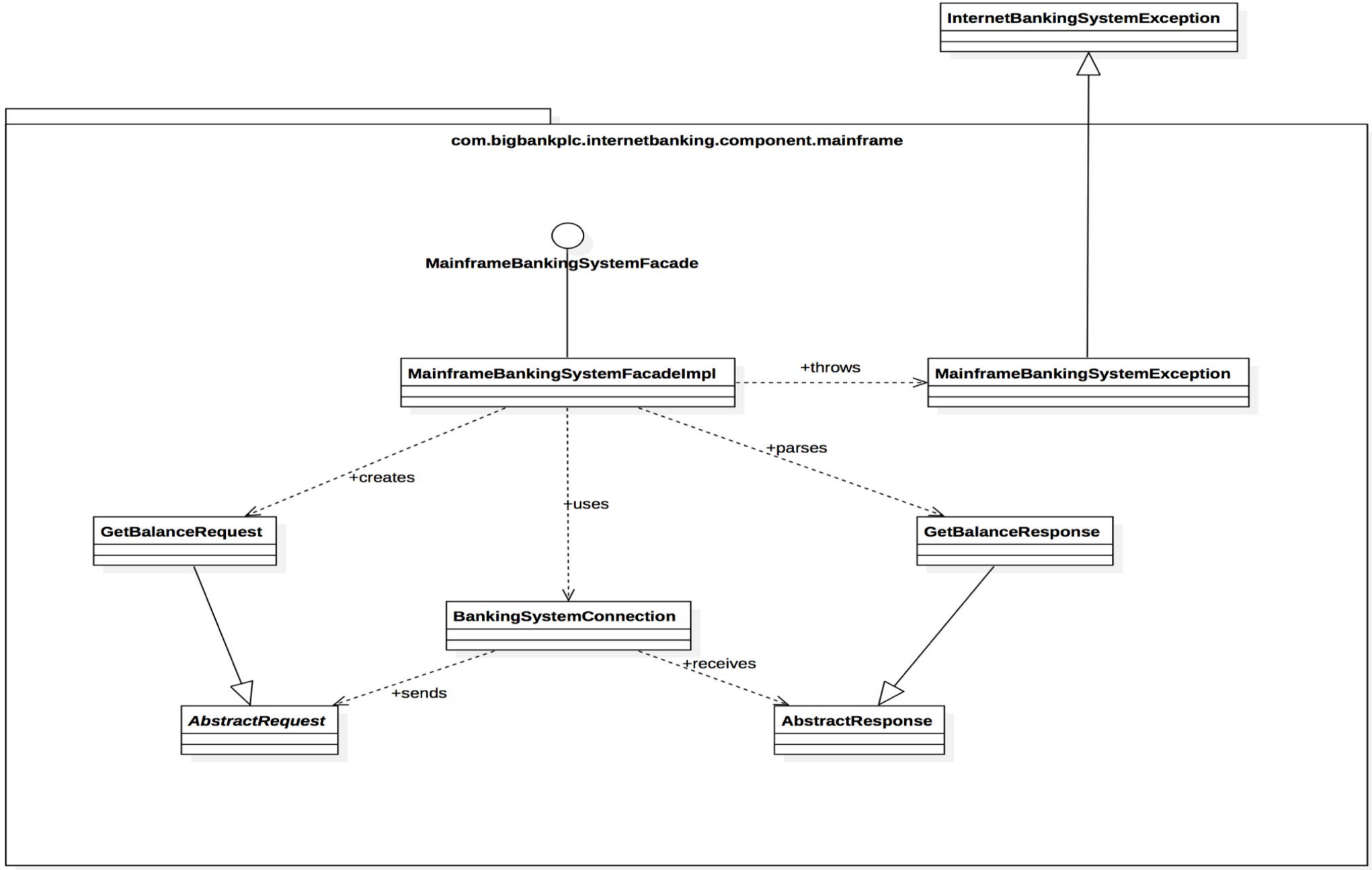
Предыдущие диаграммы были в основном техническими, в то время как диаграмма вариантов использования больше ориентирована на бизнес. Он показывает, как люди взаимодействуют с вашей системой на очень высоком уровне.

Эти диаграммы вам уже хорошо знакомы.

8. ДИАГРАММА КОДА

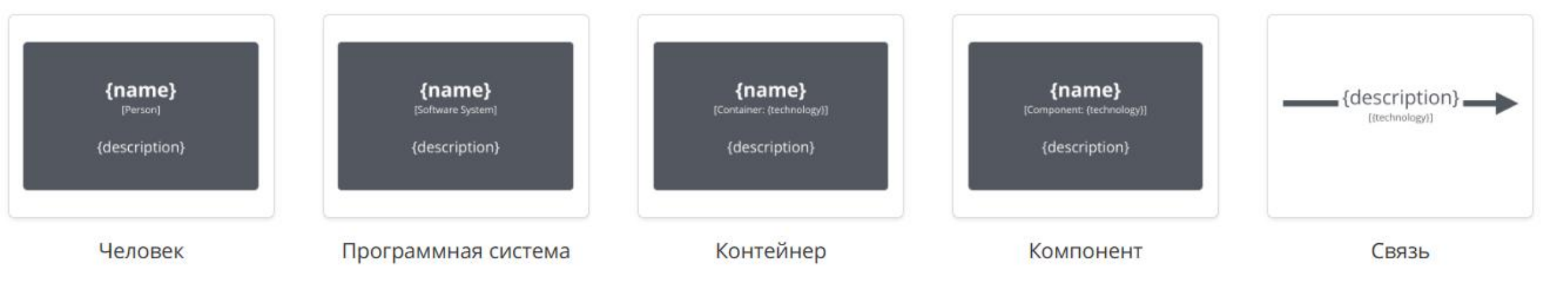
Наконец, вы можете увеличить масштаб каждого компонента, чтобы показать, как он реализован в виде кода; с использованием UML-диаграмм классов, ER- диаграмм и т.д.

Это дополнительный уровень детализации, который часто доступен по требованию с помощью таких инструментов, как интегрированные среды разработки. В идеале эта диаграмма должна быть автоматически сгенерирована с помощью инструментария (например, IDE или инструмента моделирования UML), и вы должны рассмотреть возможность отображения только тех атрибутов и методов, которые позволяют вам рассказать историю, которую вы хотите рассказать. Этот уровень детализации не рекомендуется ни для чего, кроме самых важных или сложных компонентов.


InternetBankingSystemException
com.bigbankplc.internetbanking.component.mainframe
MainframeBankingSystemFacade
MainframeBankingSystemFacadeImpl
+throws
MainframeBankingSystemException
+creates
+uses
+parses
GetBalanceRequest
GetBalanceResponse
BankingSystemConnection
+receives
+sends
AbstractRequest
AbstractResponse

Нотация

Модель C4 не зависит от нотации и не предписывает какой-либо конкретной нотации. В качестве отправной точки, однако, простая нотация, которая хорошо работает на досках, бумаге, стикерах, каталожных карточках и различных инструментах для создания диаграмм, выглядит следующим образом.



Топ 30 терминов и определений в проектировании архитектуры ПО

1. **Архитектура программного обеспечения**: организация компонентов, модулей и интерфейсов системы, которая определяет ее структуру, поведение и взаимодействие с внешними компонентами.
2. **Компонент**: отдельная часть системы, которая имеет четко определенные функции и ответственность.
3. **Модуль**: логически связанная группа компонентов, которая выполняет определенные функции внутри системы.
4. **Уровень (Layer)**: группировка компонентов по сходству их функций или ответственности.

5. **Зависимость**: отношение между компонентами, когда изменения в одном компоненте могут повлиять на другие компоненты.
6. **Интерфейс**: определение контракта или спецификации, определяющей, как взаимодействовать с компонентом или модулем.
7. **Абстракция**: упрощенное представление компонента или системы, скрывающее детали реализации и предоставляющее только необходимый функционал.
8. **Принцип единственной ответственности** (Single Responsibility Principle): принцип, согласно которому каждый компонент или модуль должен быть ответственным только за одну четко определенную задачу.

9. **Принцип открытости/закрытости** (Open/Closed Principle): принцип, утверждающий, что компоненты должны быть открытыми для расширения, но закрытыми для модификации.
10. **Принцип подстановки Барбары Лисков** (Liskov Substitution Principle): принцип, утверждающий, что объекты должны быть заменяемыми на свои подтипы без изменения корректности системы.
11. **Принцип инверсии зависимостей** (Dependency Inversion Principle): принцип, предлагающий зависеть от абстракций, а не от конкретных реализаций.
12. **Модель предметной области** (Domain Model): абстрактное представление реальной предметной области, которое описывает ее ключевые понятия, правила и взаимодействия.

13. **Агрегат**: Группа объектов, которые вместе образуют одну единицу и имеют строго определенные правила доступа и изменения.
14. **Слой представления** (Presentation Layer): отвечает за отображение данных пользователю и обработку пользовательского ввода.
15. **Бизнес-логика** (Business Logic): часть системы, отвечающая за обработку бизнес-правил и логику операций.
16. **Слой доступа к данным** (Data Access Layer): обеспечивает доступ к данным из различных источников, таких как базы данных или внешние сервисы.
17. **Слой сервисов** (Service Layer): содержит бизнес-сервисы, которые предоставляют высокоуровневые операции и функциональность для других компонентов системы.

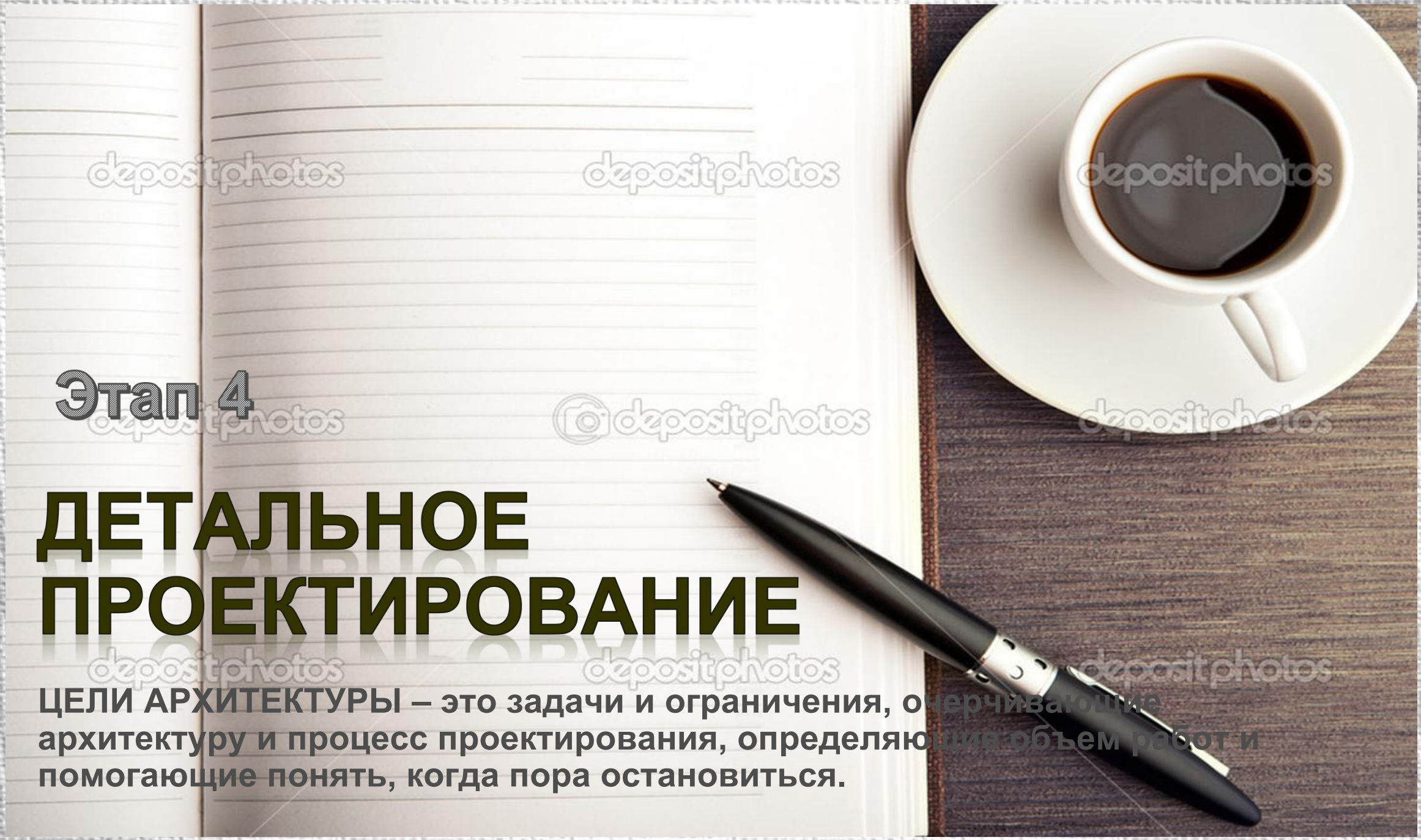
18. **Шаблон проектирования (**Design Pattern): повторяемое решение для типичной проблемы в проектировании, обеспечивающее эффективность, гибкость и повторное использование кода.
19. **Микросервисная архитектура** (Microservices Architecture): стиль архитектуры, в котором система разделена на мелкие, автономные и независимо развертываемые сервисы.
20. **Событийно-ориентированная архитектура** (Event-Driven Architecture): Стиль архитектуры, где система реагирует на события, которые возникают внутри или вне ее и инициируют обработку или взаимодействие с другими компонентами.
21. **Инверсия управления** (Inversion of Control): принцип, согласно которому создание и управление объектами делегируется контейнеру или фреймворку.

22. **Внедрение зависимостей** (Dependency Injection): подход, при котором объекту внедряются его зависимости извне, что упрощает тестирование и уменьшает связность.
23. **Миграция базы данных** (Database Migration): процесс изменения схемы базы данных и переноса данных в соответствии с новыми требованиями или версией приложения.
24. **Шина сообщений** (Message Bus): инфраструктурная компонента, которая обеспечивает асинхронную коммуникацию и передачу сообщений между компонентами системы.
25. **RESTful API:** Архитектурный стиль для создания веб-сервисов, основанный на принципах REST (Representational State Transfer), который обеспечивает унифицированный интерфейс для взаимодействия между клиентом и сервером.

26. **Серверно-ориентированная архитектура** (Service-Oriented Architecture, SOA): стиль архитектуры, в котором функциональность системы предоставляется через сервисы, которые могут быть независимо развернутыми и используемыми другими компонентами.
27. **Хранилище данных** (Data Store): механизм для хранения и организации данных, таких как база данных, файловая система или кеш.
28. **Кэширование** (Caching): механизм сохранения временных копий данных в быстром доступе для улучшения производительности и снижения нагрузки на источник данных.
29. **Сквозная функциональность** (Cross-Cutting Concerns): функциональность, которая пересекает несколько компонентов системы и не относится к их основным функциям, например, журналирование, безопасность или аудит.

30. **Тестирование архитектуры** (Architecture Testing): процесс проверки архитектуры программного обеспечения на соответствие требованиям, качеству и принципам проектирования.

Эти термины и определения являются основой для понимания и применения принципов и практик проектирования архитектуры программного обеспечения. Они помогут вам разобраться в концепциях и терминологии, используемой в этой области.



Этап 4

ДЕТАЛЬНОЕ ПРОЕКТИРОВАНИЕ

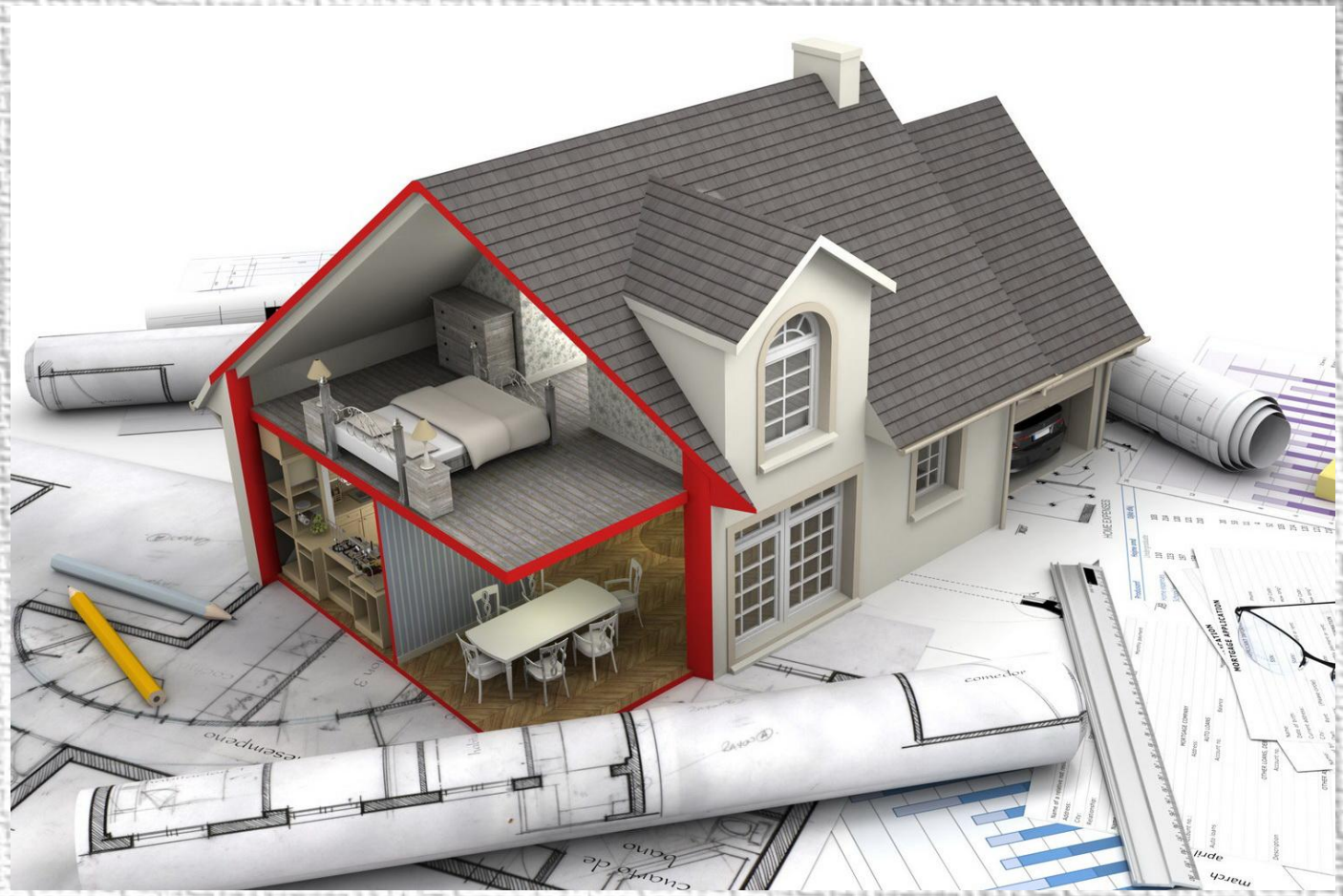
ЦЕЛИ АРХИТЕКТУРЫ – это задачи и ограничения, очерчивающие архитектуру и процесс проектирования, определяющие объем работ и помогающие понять, когда пора остановиться.

Что такое детальное проектирование

Детальное проектирование — это техническая деятельность, которая следует за выбором архитектуры. Основной целью этой деятельности является как можно более полная подготовка проекта к его реализации. Другими словами, программисты должны иметь возможность реализовать детальный проект, сконцентрировавшись только на проблемах, связанных с созданием программного кода.

Детальное проектирование
(проектирование элементов системы)

1. Проектирование прецедентов
2. Проектирование подсистем
3. Проектирование классов
4. Проектирование баз данных
5. Проектирование интерфейса



1 ПРОЕКТИРОВАНИЕ ПРЕЦЕДЕНТОВ

(РЕАЛИЗАЦИЯ ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ)

Основная задача реального проектирования состоит в поиске путей эффективной реализации модели ВИ в условиях существующей среды разработки. Базой реального проектирования являются помимо модели прецедентов модель предметной области (ПО-модель) и спецификация требований. Процесс реального проектирования можно описать следующим образом.

- После того как **модель предметной области** информационной системы создана, **проектировщик принимает ее в качестве отправной точки для создания реальной модели классов**. Помимо нее, он использует спецификацию требований и модель ВИ.

Проектировщик реальной модели классов начинает с адаптации ПО-модели к условиям реального окружения. Например, если выбранный язык программирования не поддерживает механизм наследо-вания, нельзя напрямую использовать этот механизм в реальной модели. В этом случае необходимо найти способ реализовать его с помощью доступных языковых средств. Адаптация подразумевает также решение проблем стыковки информационной системы с СУБД и другим существующим программным обеспечением.

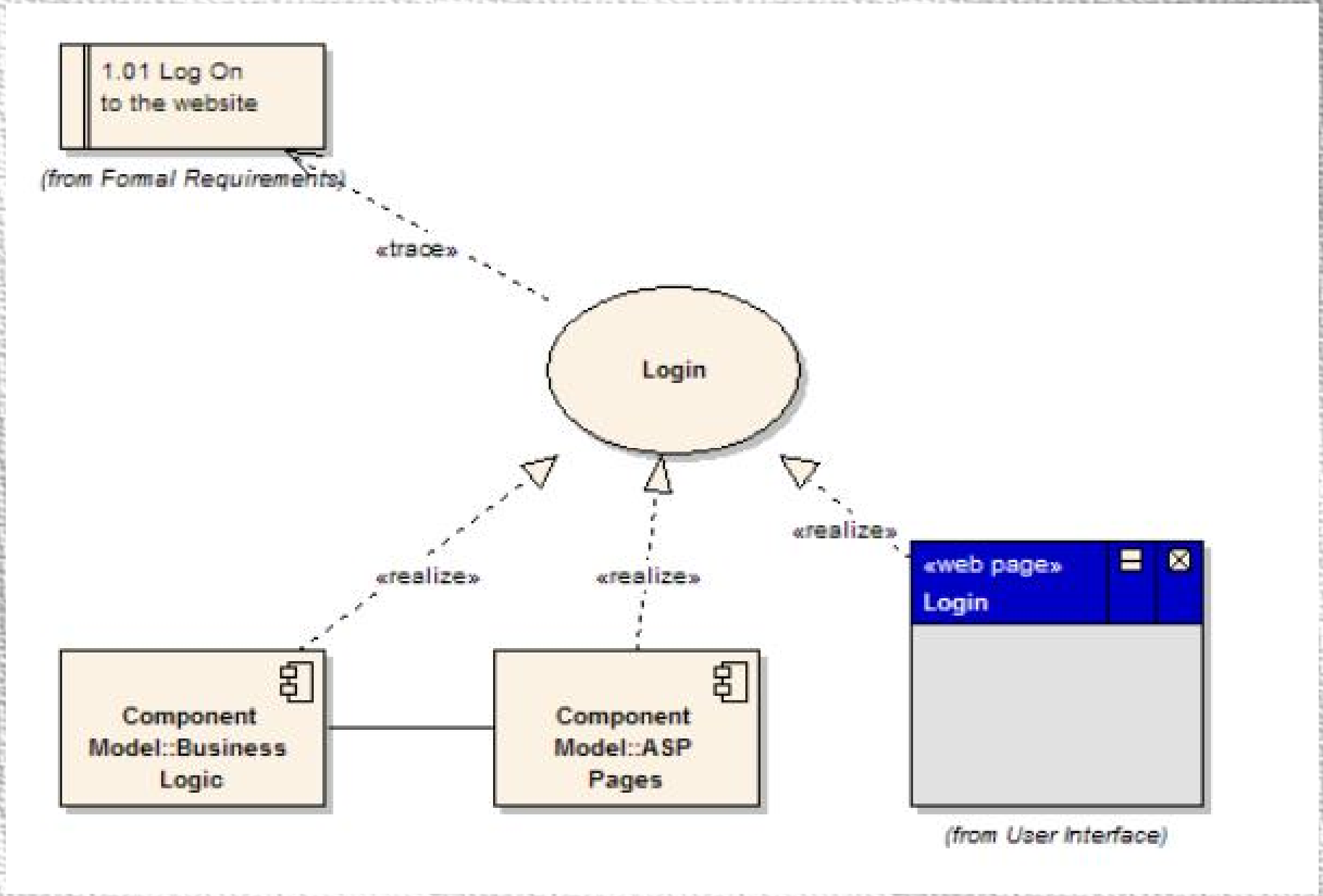
Для каждого прецедента проектировщик описывает с помощью диаграмм взаимодействия поведение взаимодействующих объектов в ходе выпол-нения прецедента.

Эту работу называют проектированием прецедентов.

Схема реализации

Вариант использования - это формальное описание функциональных возможностей, которые система будет иметь при ее создании. **Схема реализации обычно связана с вариантом использования, чтобы задокументировать, какие элементы дизайна (например, компоненты и классы) реализуют функциональность варианта использования в новой системе**. Это обеспечивает высокий уровень прослеживаемости для разработчика системы, заказчика и команды, которая фактически создаст систему. Список вариантов использования, с которыми компонент или класс связан с документами, минимальная функциональность которых должна быть реализована компонентом.

Реализация варианта использования «Логин»



В приведенном выше примере показано, что сценарий использования «Логин» реализует формальное требование «1.01 Вход на сайт». Это также показывает, что компонент «Бизнес-логика» и компонент «Страницы ASP» реализуют некоторые или все функции «Вход в систему». Еще одним уточнением является отображение экрана «Вход в систему» (веб-страницы) как варианта использования «Вход в систему». Эти ссылки на реализацию или реализацию определяют прослеживаемость от формальных требований через варианты использования до компонентов и экранов.

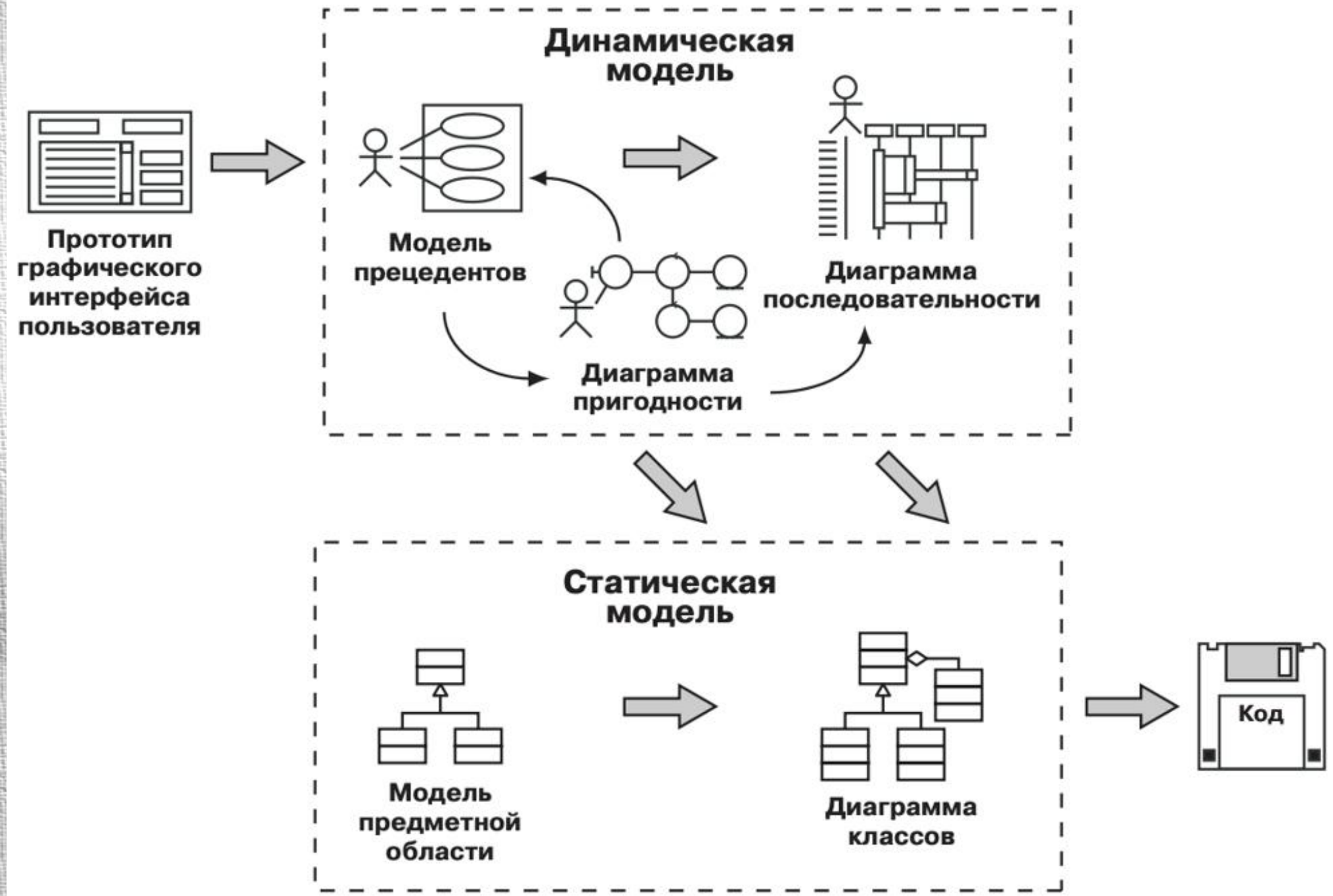
Предварительное проектирование
(диаграмма пригодности)

Предварительное проектирование

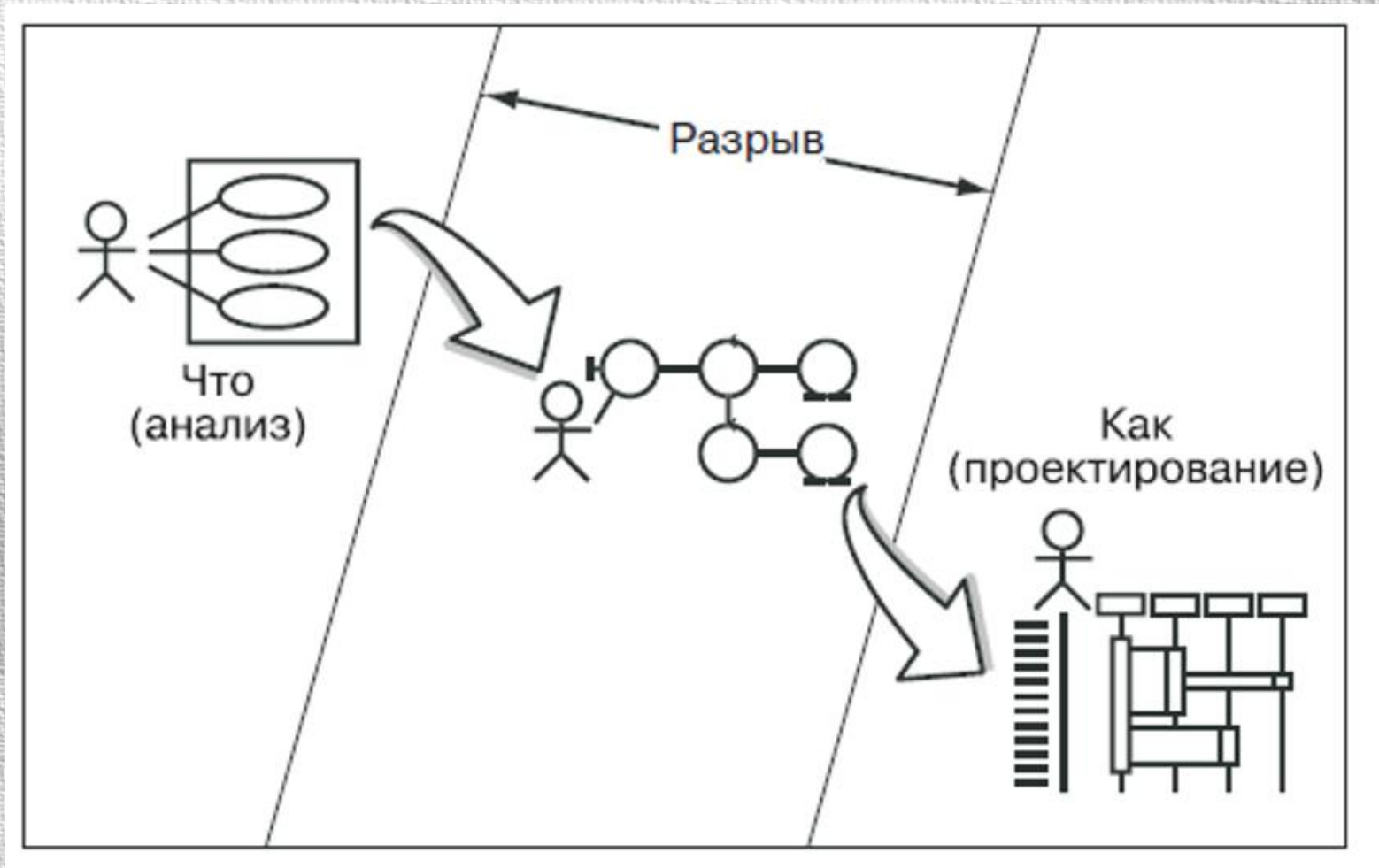
После построения диаграмм вариантов использования и их согласования с заказчиком, процесс ICONIX предлагает приступить к разработке диаграмм пригодности (робастности, robustness diagrams). Гради Буч не использует этот вид диаграмм, однако он все равно просматривает каждый вариант использования и пытается выделить объекты, после чего переходит сразу к построению диаграмм взаимодействия. **Назначение диаграмм робастности:**

- идентификация потенциальных объектов или (хотя бы) обязанностей объектов, необходимых для реализации прецедентов и использования в других диаграммах;

- уточнение и частичная проверка логики варианта использования и модели предметной области;
  - в тексте прецедента общие термины (например, «система») должны быть заменены на имена конкретных объектов;
  - помогает выявить альтернативные последовательности прецедента, которые не были прописаны в тексте;
- обеспечение согласованности терминологии, используемой в различных прецедентах (дополняется модель предметной области);
- могут являться частью технической документации. На диаграмме в наглядной форме отображаются объекты и их обязанности, при этом она намного проще для восприятия, чем диаграмма последовательностей.


Динамическая модель
Прототип графического интерфейса пользователя
Модель прецедентов
Диаграмма последовательности
Диаграмма пригодности
Статическая модель
Модель предметной области
Диаграмма классов
Код

Связь между анализом и проектированием



Для построения диаграммы пригодности нужно ответить на два вопроса:

1) ***К**акие объекты нужны для каждого прецедента?*
2) ***К**акие функции будут выполнять объекты?*

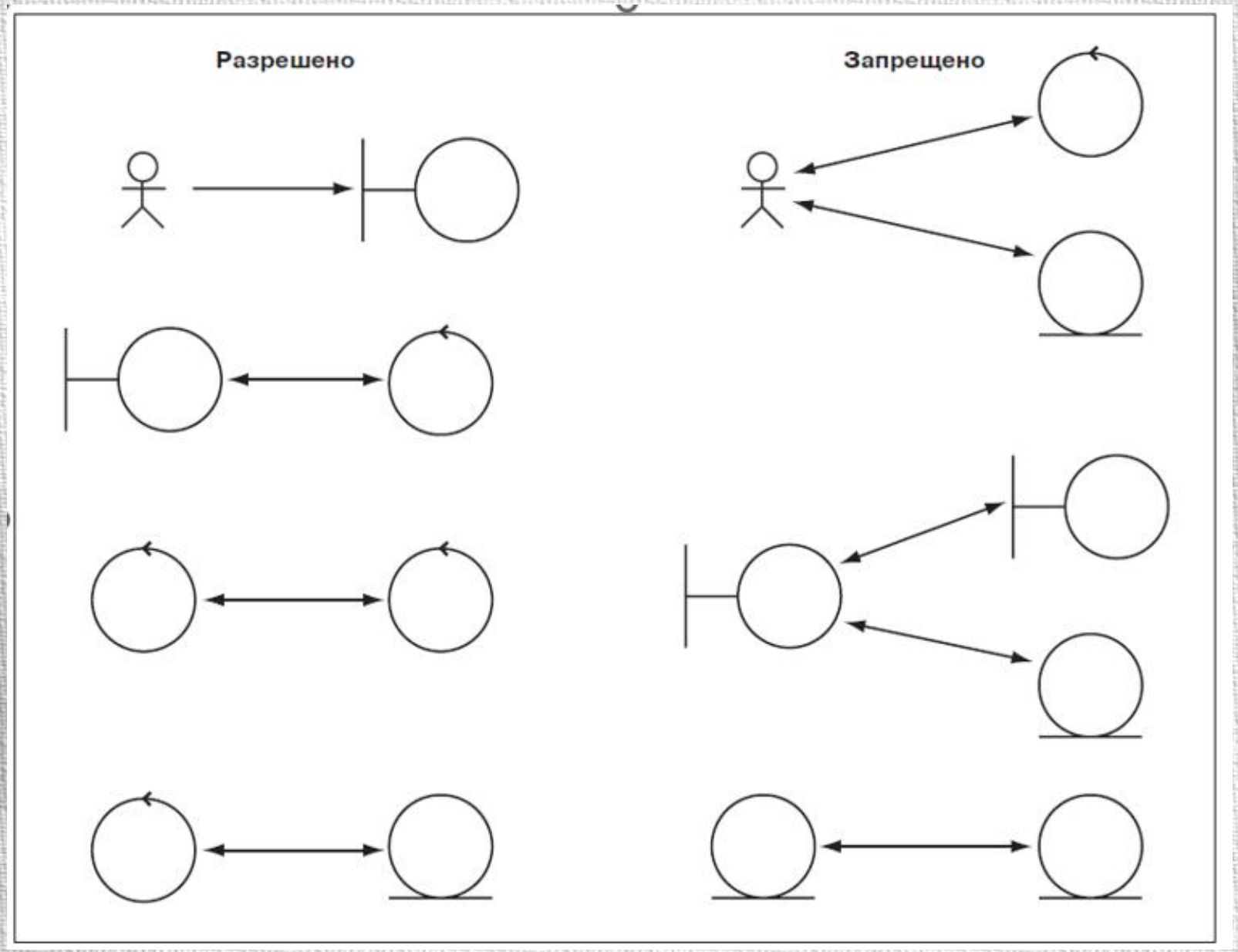
По существу, диаграмма пригодности - это диаграмма классов, на которой изображаются пиктограммы трех видов:

1. **Граничный объект** - объекты, которыми пользуются актеры (примечание: это цитата из книги, сам я предпочитаю слово “актОр”) для взаимодействия с системой
2. **Сущностный объект** - обычно, это объект из модели предметной области
3. **Управляющий объект** - контроллеры, выполняющие функцию “клея” между сущностными и граничными объектами

Диаграмма пригодности помогает находить недостающие объекты, атрибуты и взаимодействия с помощью проверки простых правил.

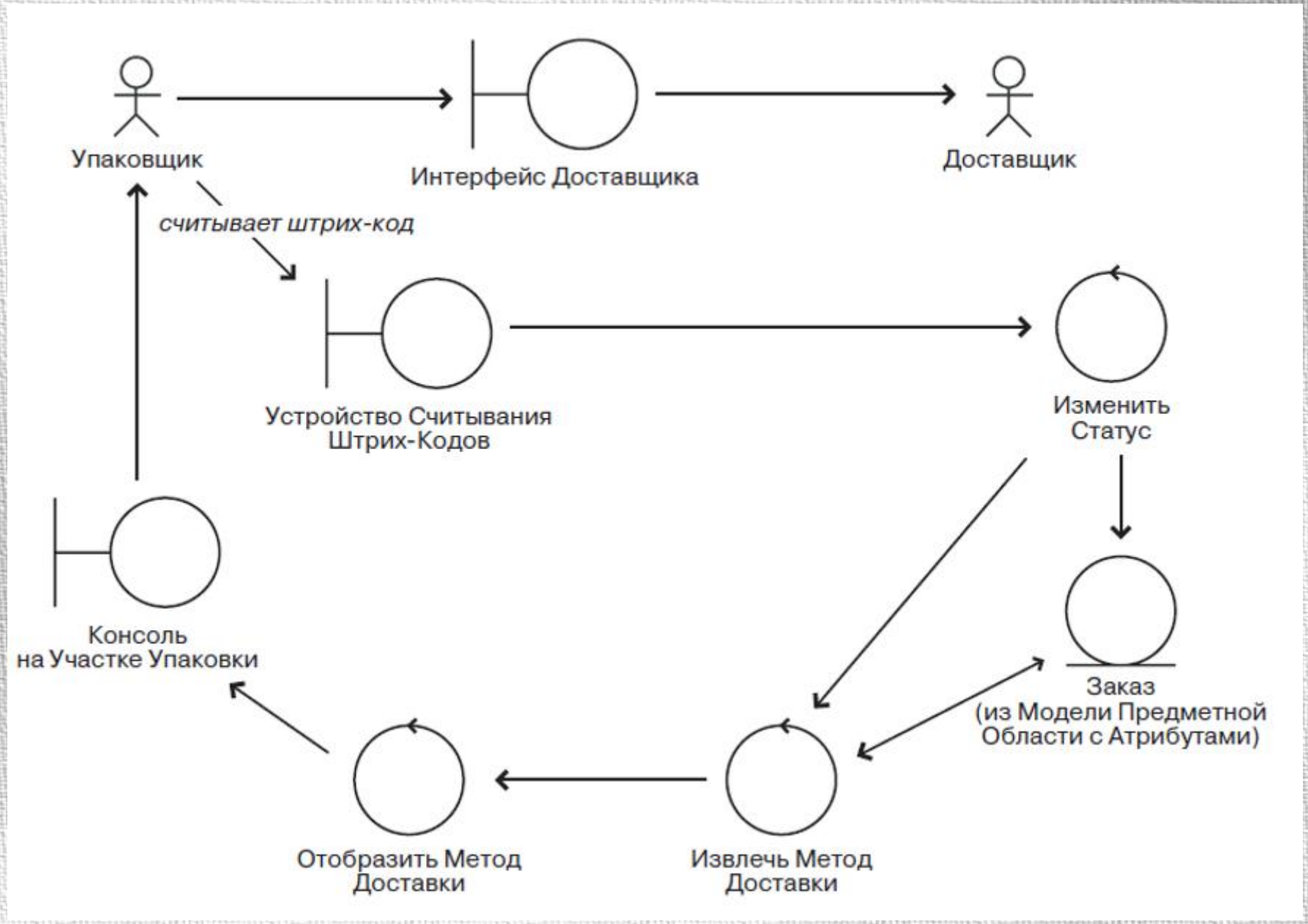
В книге вводятся четыре основных правила:

1. **А**ктеры могут общаться только с граничными объектами.
2. **Г**раничные объекты могут общаться только с контроллерами и актерами.
3. **С**ущностные объекты могут общаться только с контроллерами.
4. **К**онтроллеры могут общаться с граничными объектами, сущностными объектами и другими контроллерами, но не с актерами.


Разрешено
Запрещено

Авторы предлагают проводить анализ пригодности для каждого прецедента, включая его основной и все альтернативные потоки.

Приведем полноценный пример диаграммы из книги для книжного магазина:



Как строить диаграмму

1. **Д**иаграмму можно строить путем рассмотрения каждого предложения сценария и добавления недостающих элементов на диаграмму.
2. **О**днако существует и иной подход:
   - добавить граничные объекты для каждого важного элемента интерфейса;
   - добавить контроллеры для управления сценарием (действия пользователя с граничным объектом);
   - добавить контроллеры для бизнес-правил (правила поведения объектов, ограничения);
   - добавить контроллеры для активностей, включающих несколько разных элементов (включающих сущности и бизнес правила, например);
   - добавить сущности;
   - добавить варианты использования (как контроллеры).

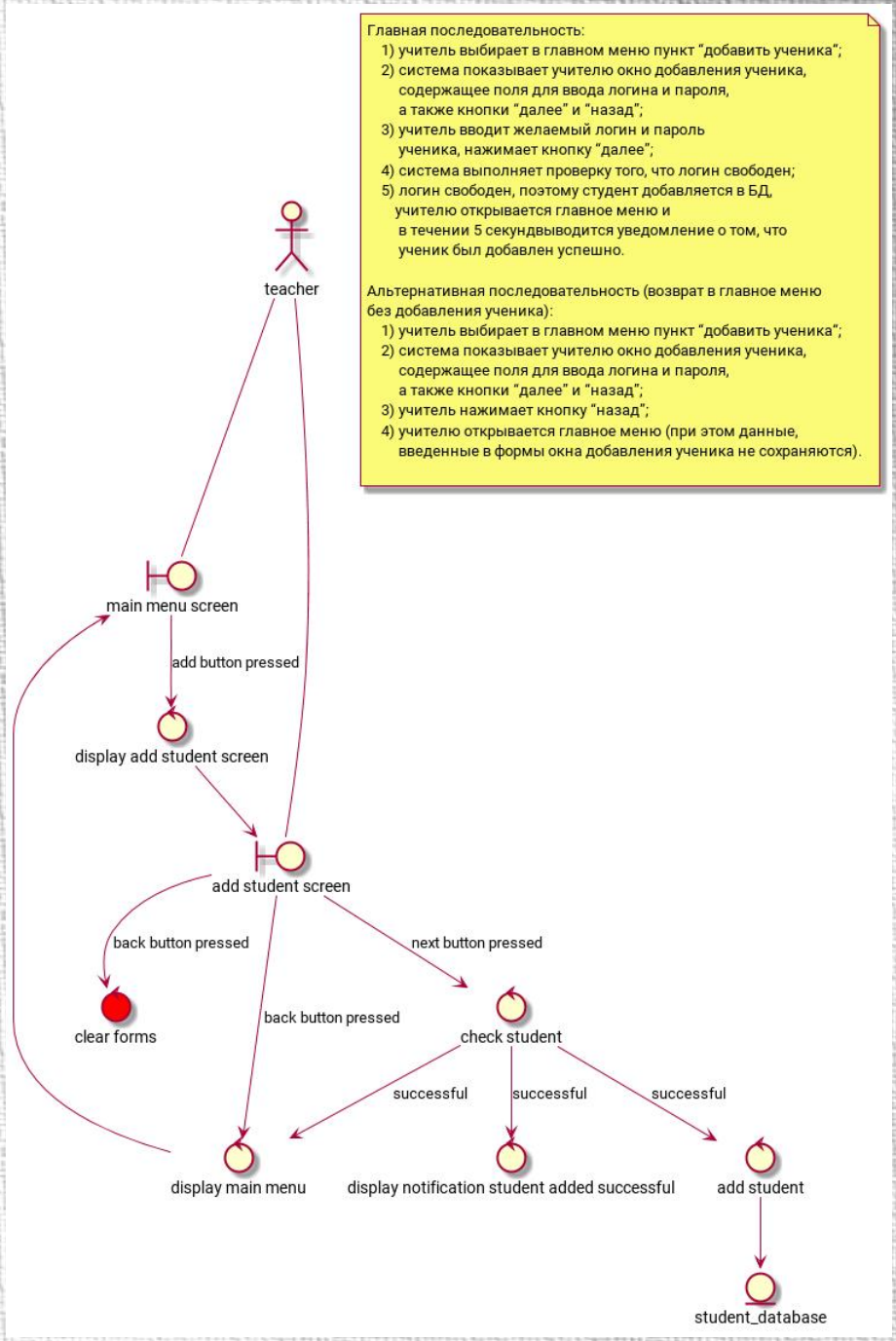
В любом случае при построении выделяются новые объекты, которые используются вместо общих понятий (например «система»), которые применялись в сценариях вариантов использования на этапе согласования технического задания с заказчиком. В связи с этим, становится возможным уточнение сценариев (обновление *use-case диаграммы*). По этой причине анализ пригодности, проводимый вне *процесса ICONIX*, часто не выделяется в отдельный шаг, а является составляющей построения диаграмма прецедентов.

Ниже приведен пример из Процесс ICONIX. Диаграммы пригодности — Блог программиста (pro-prof.com)

Диаграмма пригодности сценария добавления ученика в систему

На приведенной диаграмме контроллер *add student* выполняет фактическое добавление только в случае успешной проверки логина, поэтому при добавлении альтернативных последовательностей этот контроллер целесообразно разделить на *add student* и *check student*. Цвет объектов, относящихся только к альтернативным сценариям может отличаться от объектов главного сценария — это улучшает восприятие диаграммы. Добавим на диаграмму последовательность действий «возврат в главное меню без добавления ученика»:

Для построения диаграмм использовался PlantUM.



На последующих этапах проектирования граничные объекты превратятся в классы, обеспечивающие взаимодействие системы с клиентами, сущностные объекты — в классы управления базами данных, а контроллеры — в функции какого-либо класса. Отталкиваясь от диаграммы можно *попробовать различные варианты распределения обязанностей*, но **делать это нужно при построении диаграмм последовательностей**. Например, из нашей диаграммы видно, что контроллеры *add student* и *check student* скорее всего должны быть функциями базы данных учеников, а *clear forms* — приватной функцией *add student screen*. Однако непонятно к чему должны относиться уведомления (они могут быть как частью главного окна программы, так и самостоятельными визуальными элементами), кроме того, возможно, контроллер *display notification ... successful* должен быть логически связан не с проверкой студента, а с его фактическим добавлением (после добавления записи, база данных может инициировать показ уведомлений — паттерн Publish-Subscriber).

Советы по построению диаграмм пригодности и типичные ошибки:

- использование большого числа контроллеров резко осложняет восприятие диаграммы. Если на диаграмме размещается более 10 контроллеров, вероятно прецедент нужно разделить на несколько;
- нет смысла использовать анализ пригодности только для главной последовательности — обязательно рассматривайте альтернативные последовательности;
- при использовании диаграмм в процессе ICONIX на них не нужно тратить много времени, т.к. процесс итеративный. На дальнейших шагах вы сможете выявить ошибки и их исправление не будет очень дорого стоить;
- выполняйте визуальную сверку для каждого варианта использования — просто пройдитесь еще раз по каждому предложению прецедента и проследите этот путь на диаграмме. Это не занимает много времени, но часто позволяет найти ошибки;
- диаграмма пригодности позволяет не только дополнить прецеденты, но и дополнить модель предметной области — не забудьте обновить ее.

Сильные стороны диаграмм пригодности

- подталкивает к использованию подхода «модель-вид-контроллер», т.к. это фактически тоже самое что граничный объект-контроллер-сущностный объект.
- позволяет выделить ряд объектов и процессов, которые будут использоваться на диаграмме последовательностей, уточнить модель предметной области. При этом объекты оказываются явно выделенными на диаграмме — это помогает согласовать терминологию в различных прецедентах;
- наглядно изображает логические связи между участниками прецедента, может использоваться как часть документации;
- позволяет улучшить текст прецедента:
- конкретизацией участвующих в нем объектов;

- проверить что прецедент построен в форме диалога (описаны на только действия пользователя, но и ответ системы);
- проверить, что в прецеденте проработаны альтернативные сценарии;
- разделить прецеденты на более мелкие, ограничив количество контроллеров в них;
- может использоваться вместо диаграммы потока экранов.

ПРИМЕРЫ

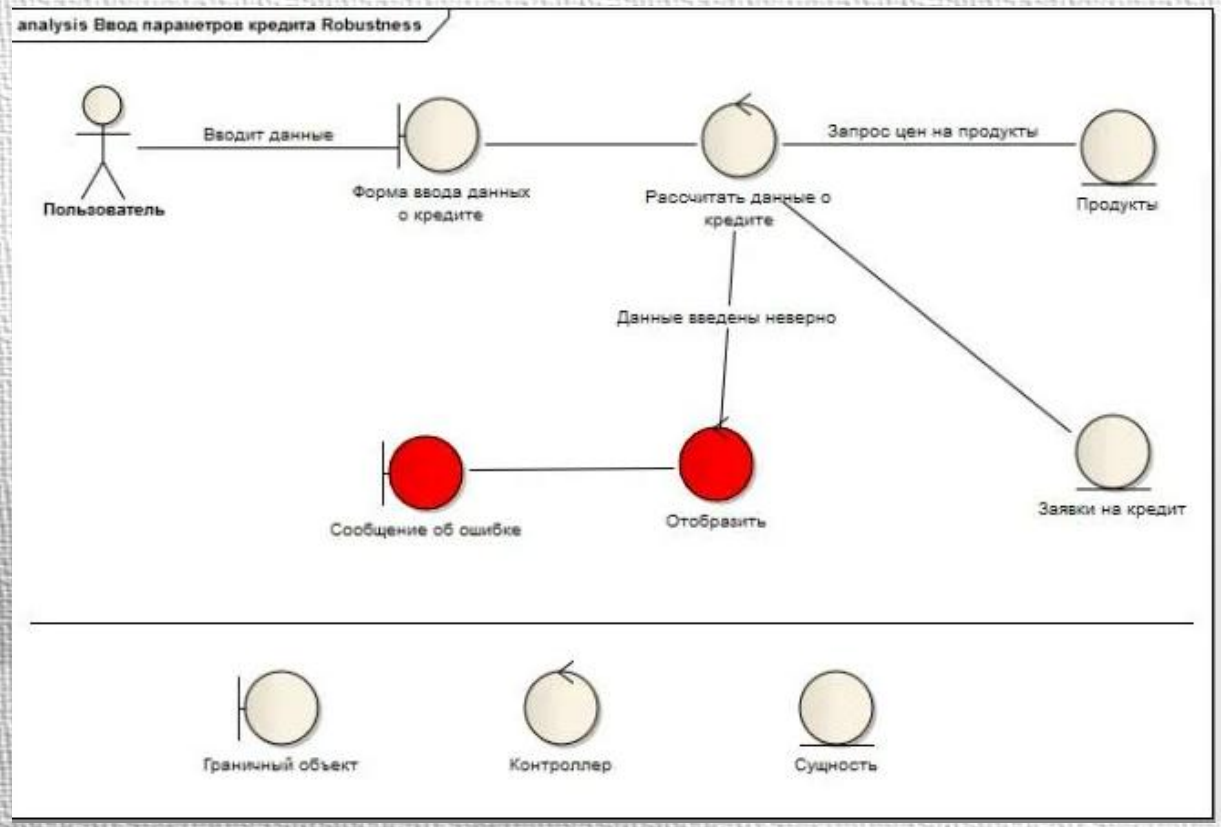
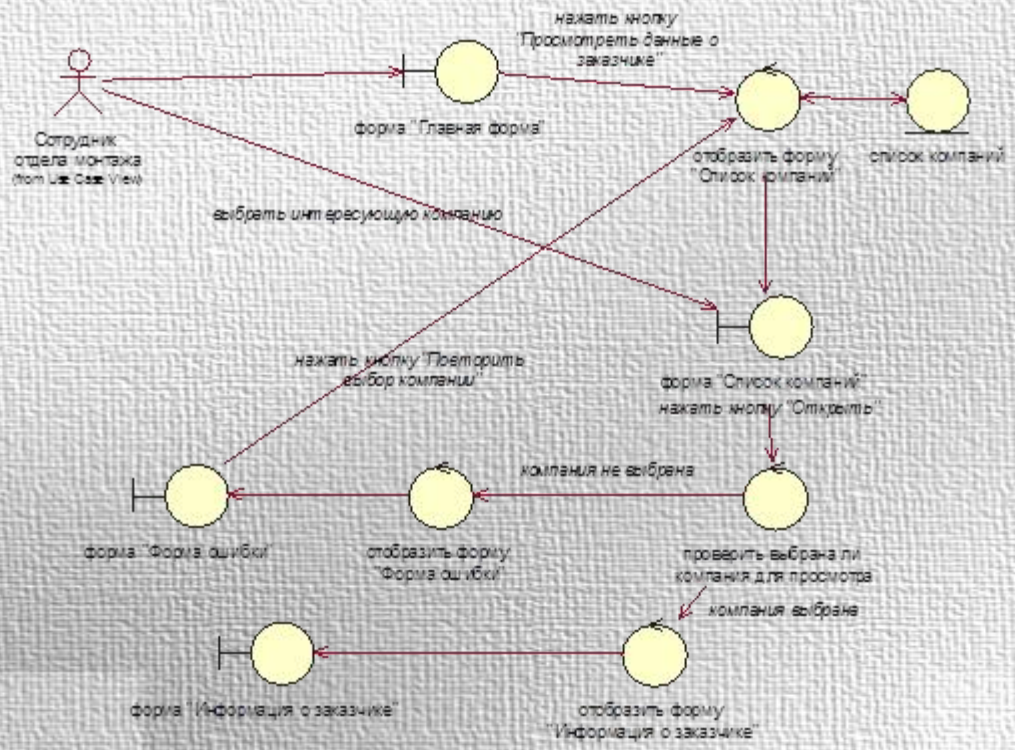
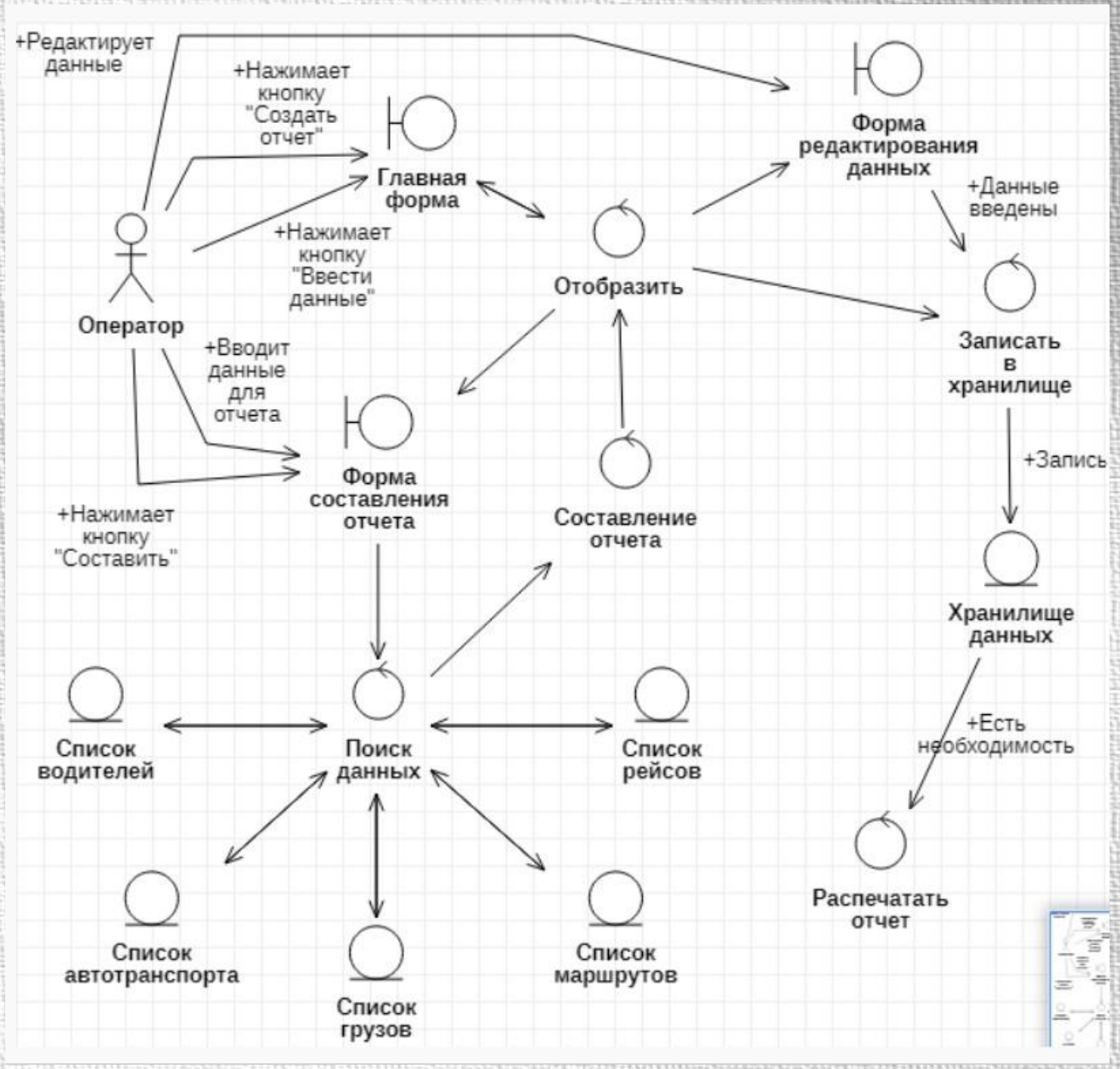

analysis Ввод параметров кредита Robustness
Вводит данные
Запрос цен на продукты
Пользователь
Форма ввода данных о кредите
Рассчитать данные о кредите
Продукты
Данные введены неверно
Сообщение об ошибке
Отобразить
Заявки на кредит
Граничный объект
Контроллер
Сущность

Диаграмма робастности для варианта использования
РЕГИСТРАЦИЯ
часть 4
Покупатель
Начальная страница
(from Logical View)
Отображает
(from Logical View)
Регистрация завершена
Проверить информацию о регистрации
(from Logical View)
Кнопка ОК
(from Logical View)
Учетная запись
(from Logical View)


Сотрудник отдела монтажа
(from Use Case View)
форма "Главная форма"
нажать кнопку "Просмотреть данные о заказчике"
отобразить форму "Список компаний"
список компаний
выбрать интересующую компанию
форма "Список компаний"
нажать кнопку "Открыть"
нажать кнопку "Повторить выбор компании"
компания не выбрана
форма "Форма ошибки"
отобразить форму "Форма ошибки"
проверить выбрана ли компания для просмотра
компания выбрана
форма "Информация о заказчике"
отобразить форму "Информация о заказчике"


+Редактирует данные
+Нажимает кнопку "Создать отчет"
Главная форма
Форма редактирования данных
+Данные введены
Оператор
+Нажимает кнопку "Ввести данные"
Отобразить
Записать в хранилище
+Вводит данные для отчета
Форма составления отчета
Составление отчета
+Нажимает кнопку "Составить"
+Запись
Хранилище данных
Список водителей
Поиск данных
Список рейсов
+Есть необходимость
Список автотранспорта
Список грузов
Список маршрутов
Распечатать отчет

Моделирование взаимодействия

(диаграмма последовательности)

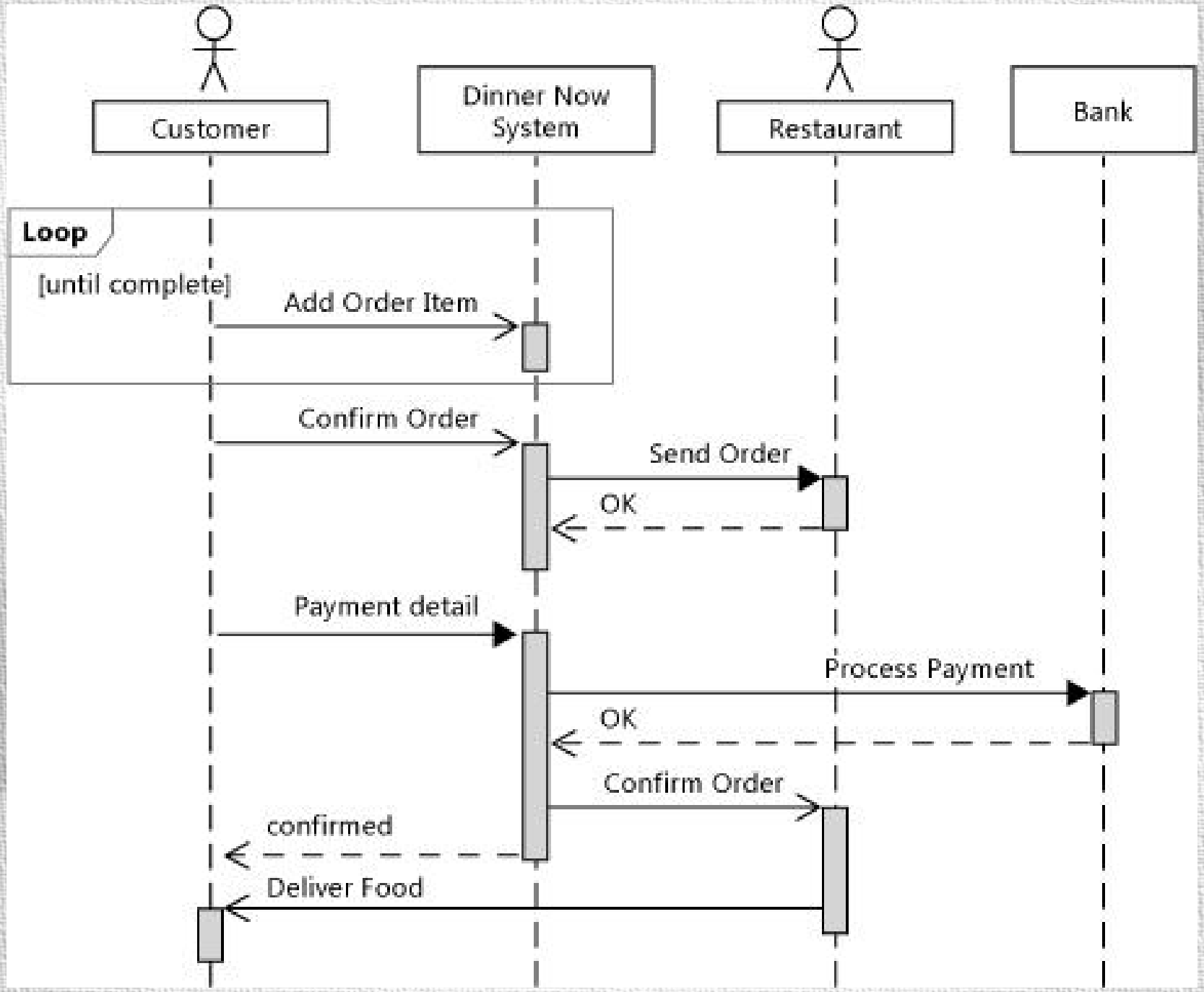
Диаграммы последовательности – *это основной рабочий продукт проектирования*. Для каждого прецедента создается диаграмма, описывающая как главную, так и альтернативные последовательности действий. При необходимости можно расположить ее на нескольких страницах.

В результате получается ядро динамической модели, в котором очень подробно определено поведение системы во время выполнения и то, как реализуется это поведение.

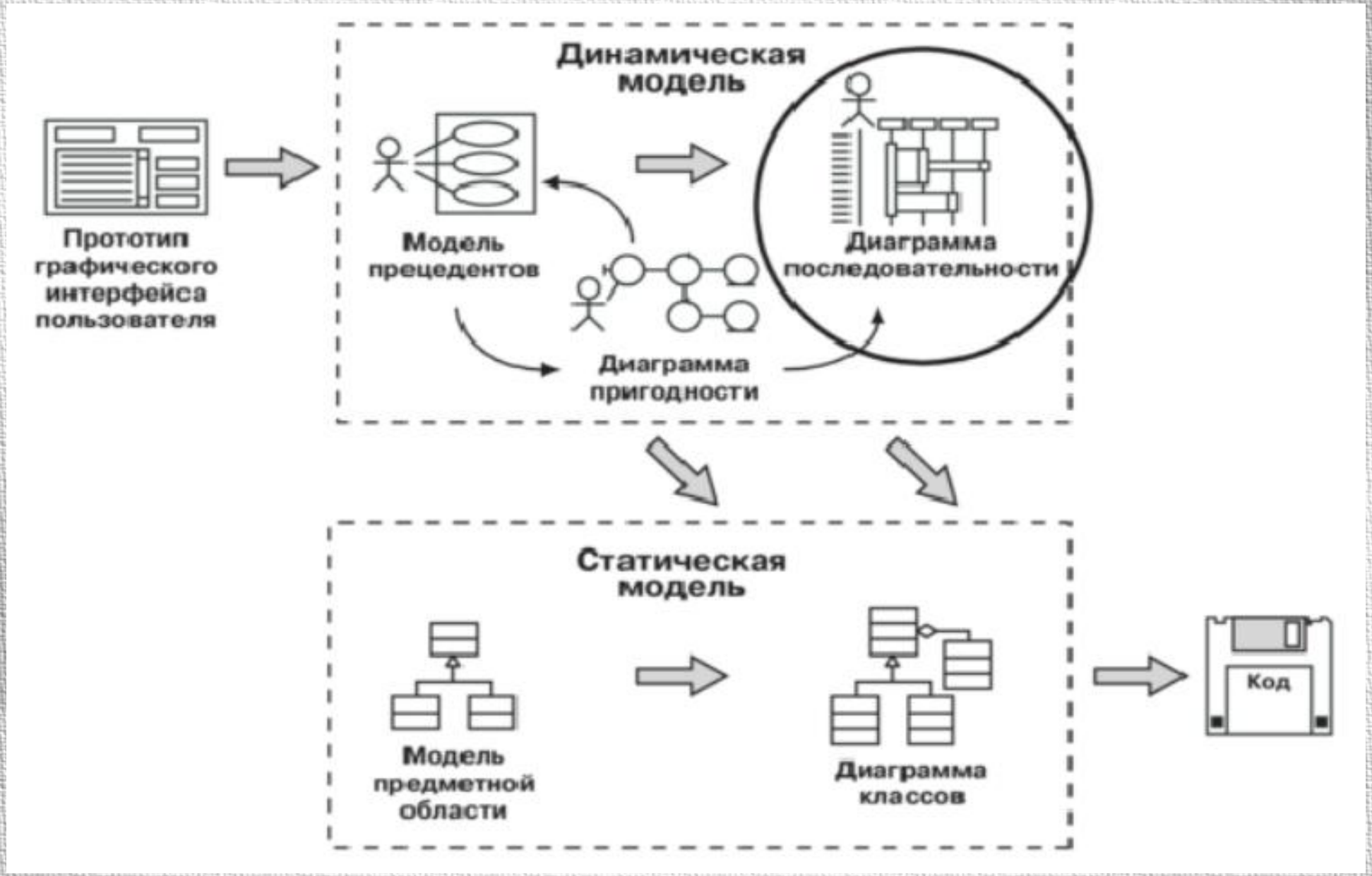
Отображение взаимодействия между пользователями и вашей системой

Диаграмму последовательностей можно использовать для отображения обмена сообщениями между системой и внешними субъектами или между частями системы. Это обеспечивает представление шагов в варианте использования, которое очень четко показывает последовательность взаимодействий. Диаграммы последовательностей особенно полезны там, где в сценарии использования есть несколько взаимодействующих сторон, а также там, где в вашей системе есть API.

Например:


Customer
Dinner Now System
Restaurant
Bank
Loop
[until complete]
Add Order Item
Confirm Order
Send Order
OK
Payment detail
Process Payment
OK
Confirm Order
confirmed
Deliver Food

Одно из преимуществ диаграмм последовательностей заключается в том, что на них легко увидеть, какие сообщения поступают в создаваемую систему. Чтобы спроектировать систему, можно заменить одну линию жизни системы отдельной линией жизни для каждого из ее компонентов, а затем отобразить взаимодействия между ними в ответ на каждое входящее сообщение.


Динамическая модель
Прототип графического интерфейса пользователя
Модель прецедентов
Диаграмма последовательности
Диаграмма пригодности
Статическая модель
Модель предметной области
Диаграмма классов
Код

Цели предварительного проектирования – выявить объекты. На этапе же детального проектирования мы занимаемся **распределением функций программы между этими объектами**.

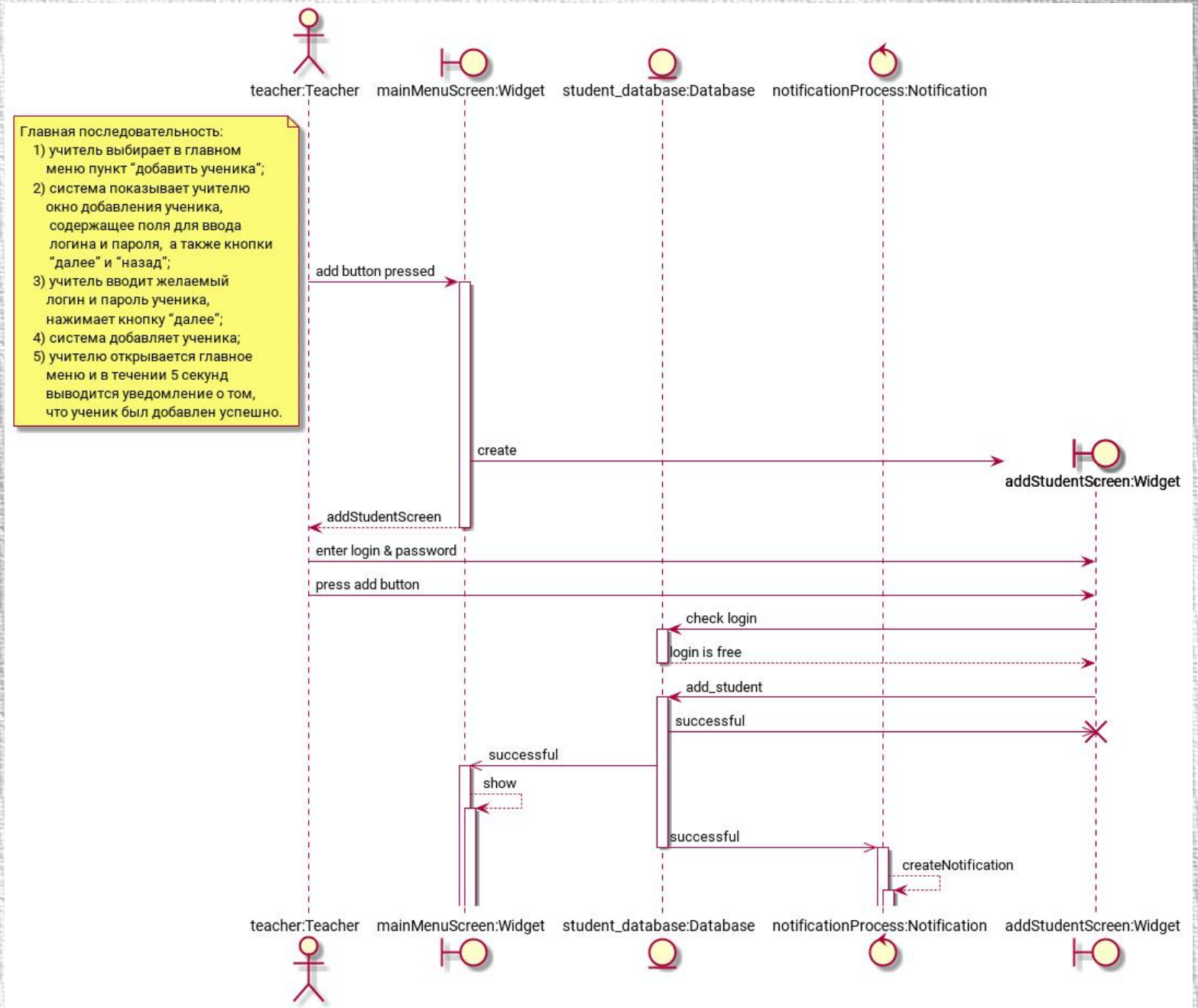
Диаграмму последовательностей можно использовать для отображения обмена сообщениями между объектами, участвующих в реализации варианта использования. **Именно диаграмма последовательностей в процессе её анализа позволит ответить на вопрос за какие операции должен отвечать каждый класс.**

Чаще всего невозможно описать прецедент с помощью только одной диаграммы последовательности действий.

Например, для прецедента *Заказ товаров* возможно оформление заказа без изменения корзины, с изменением состава корзины, или покупатель, просмотрев корзину, захочет вернуться в каталог и что-то в нее добавить, возможно, вернувшись в каталог, покупатель не станет ничего больше добавлять, а снова вернется в корзину и оформит заказ. Каждый такой вариант мы можем описать своей последовательностью действий, своим сценарием. И, таким образом, один прецедент описывает несколько последовательностей – сценариев, каждый из которых описывает один из вариантов возможного потока событий.

Обычно для основного потока событий большинства прецедентов строится одна диаграмма последовательности, для альтернативных потоков - дополнительные диаграммы, описывающие все остальные сценарии. Так поступают потому, что на диаграмме последовательности действий сложно отобразить логику ЕСЛИ-ТО-ИНАЧЕ. Однако, если это необходимо и не загромождает диаграмму, то это можно сделать с помощью условий.

Пример: диаграмма последователь ности сценария добавления ученика в систему тестирования



Четыре основных элемента ДП

1. **текста последовательности действий в прецеденте**, который записывается сверху вниз по левой стороне. Разумно разбить текст на фрагменты, отделив их пустыми строками, чтобы было видно, каким элементам диаграммы (в правой части) соответствует каждое предложение;
2. **объектов, перенесенных прямо с диаграммы пригодности** и представленных в виде прямоугольников, в которых в формате «*объект:класс*» записывается имя или номер экземпляра объекта и имя класса объекта. Любое имя может быть опущено. Объекты можно также изображать в виде стереотипов, заимствованных из диаграмм пригодности; часто это помогает проследить сообщения, которыми обмениваются актеры, граничные и сущностные объекты;

3. **сообщений, изображаемых стрелками**, которые направлены от одного объекта к другому. Стрелка может соединять две пунктирные линии, линию и прямоугольник, представляющий метод, или два таких прямоугольника;
4. **методов (операций),** представляемых в виде прямоугольников. Они расположены на пунктирных линиях, соответствующих тем объектам, которым методы принадлежат.

- Длину прямоугольника можно использовать для того, чтобы показать фокус управления в последовательности: метод владеет управлением вплоть до точки, в которой прямоугольник кончается. Увы, понятие фокуса управления представляется полезным лишь с теоретической точки зрения, так как на практике большинство средств визуального моделирования не слишком хорошо поддерживает эту возможность. Если не удается показать фокус управления на диаграмме, просто отключите его отображение, чтобы ничто не отвлекало вас от действительно важной работы.

Диаграмма классов строится на базе трех классов: **пограничный класс, класс сущности и управляющий класс**. Это способствует разделению предметной области на три составляющие части:

1. *объекты,*
2. *совершаемые над ними действия и*
3. *точки доступа к этим действиям.*

Это разделение очень напоминает расслоение, присутствующее в архитектуре трехуровневого приложения. Где приложения представляется в виде трех уровней: представление, логика и данные.

ТРИ ОСНОВНЫЕ ЗАДАЧИ МОДЕЛИРОВАНИЯ ПОВЕДЕНИЯ

1. **Р**аспределить поведение между граничными, сущностными и управляющими классами.
2. **Д**етально показать взаимодействия между объектами, участвующими в каждом прецеденте.
3. **З**акончить распределение операций по классам.

❶ Распределение поведения между граничными, сущностными и управляющими классами

Во время анализа пригодности вы выявили набор объектов, которые в совокупности способны обеспечить требуемое прецедентами поведение, а также разбили это поведение на дискретные блоки и создали управляющие объекты-контейнеры для каждого такого блока. Теперь нужно решить, какие объекты отвечают за каждый конкретный аспект поведения.

❷ Детально показать взаимодействия между объектами, участвующими в каждом прецеденте

Во время выполнения программы объекты взаимодействуют, посылая друг другу сообщения. Можно назвать их стимулами, имея в виду, что сообщение побуждает объект выполнить то или иное действие. Для каждого аспекта поведения в прецеденте вы должны идентифицировать необходимые сообщения/методы;

❸ Распределение операций по классам

Нужно стремиться к тому, чтобы значительная часть атрибутов (до 75–80%) была включена в статическую модель по завершении анализа пригодности. Что же касается определения операций в ходе моделирования предметной области и анализа пригодности, мы рекомендуем не усердствовать. Более того, лучше вообще не определять никаких методов во время предварительного проектирования. Это связано с тем, что на данных этапах у вас еще нет информации, достаточной для принятия грамотных проектных решений. (Подумайте сами: пока не закончено составление диаграмм пригодности, вы еще даже не выявили всех объектов, а попытка распределить поведение по неполному набору объектов может быть ошибочной).

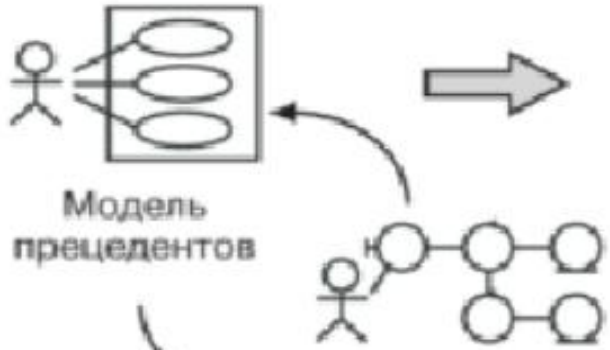
Однако при переходе к моделированию взаимодействий информации уже достаточно – по крайней мере, хочется на это надеяться. Изображая детали поведения объектов на диаграммах последовательности, вы должны окончательно определиться с тем, куда отнести все атрибуты и операции. Выполняя такое динамическое моделирование, вы продолжаете модифицировать и расширять статическую модель, улучшая понимание того, как должна работать система.

ЭТАПЫ РАЗРАБОТКИ

1. **С**копируйте текст прецедента из спецификации. Вставьте его в левое поле страницы.
2. **Д**обавьте сущностные объекты из диаграммы пригодности.
3. **Д**обавьте граничные объекты и актеров из диаграммы пригодности.

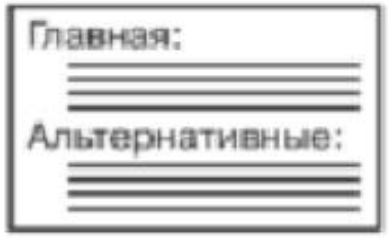
   Если вы придерживаетесь методологии ICONIX, то первые три шага должны выполняться механически.
4. **О**пределите какие методы и в какие классы поместить – это суть моделирования взаимодействия.

Текст прецедента уточняется в ходе анализа пригодности и согласуется во время рецензирования предварительного проекта.



Модель прецедентов

Диаграмма пригодности



Главная и альтернативные последовательности действий

1. Скопируйте текст прецедента в левое поле диаграммы последовательности.

2. Добавьте сущностные объекты.
3. Добавьте граничные объекты.

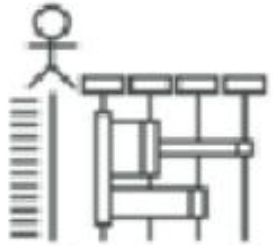


Диаграмма последовательности

4. Рассмотрите контроллеры по очереди и решите, как распределить поведение между кооперирующимися объектами.

❹ РАСПРЕДЕЛЕНИЕ МЕТОДОВ ПО КЛАССАМ

Распределение методов по классам предполагает, в частности, преобразование контроллеров, изображенных на диаграммах пригодности, в методы и сообщения, реализующие нужное поведение. Заметим, что контроллер можно превратить и в настоящий управляющий объект.

В связи с этим мы рекомендуем использовать диаграмму пригодности как контрольный список, чтобы убедиться, что на диаграммах последовательности учтены все требования к поведению системы.

Вы просто вычеркиваете управляющий объект по мере того, как соответствующему сообщения наносятся на диаграммы последовательности. Таким образом, вы избавите себя от ошибок типа «ой, про эту функцию я и позабыл», которые проявляются в самый неподходящий момент. Заметим, что **один контроллер на диаграмме пригодности может транслироваться в несколько методов** на диаграмме последовательности.

Существует два метода преобразования контроллеров:

1. ***у****правление на экране*
2. ***к****онтроллер прецедента.*

Если вы последовательно придерживаетесь того или иного подхода при составлении диаграмм последовательности для всех прецедентов, то можно сказать, что вы пользуетесь паттернами. Идея в том, чтобы все члены коллектива, отвечающие за диаграммы, определили еще на ранних стадиях работы некие стандарты и следовали им при рассмотрении всех прецедентов.

Использование шаблонов

Использование шаблонов

При моделировании взаимодействий между различными объектами большую пользу могут оказать стандартные паттерны:

- **Command** (Команда) – в системах обработки сообщений каждое сообщение может представлять и обрабатывать отдельный объект Command.
- **Facade** (Фасад) – выбор объекта, представляющего всю систему или орга-низацию в качестве внешнего контроллера.
- **Layers (**Слои) – логика реализации размещается в слое реализации, а не в слое представления.
- **PureFabrication (**Чистая синтетика) – подразумевает создание искусственного класса, не имеющего аналога в предметной области. Элементом шаблона PureFabrication является контроллер прецедента.

Распределение обязанностей

В модели системы могут быть определены десятки или сотни программных классов, а в приложении может потребоваться выполнение сотен или тысяч обязанностей. Во время объектно-ориентированного проектирования при формулировке принципов взаимодействия объектов необходимо распределить обязанности между классами. При правильном выполнении этой задачи система становится гораздо проще для понимания, поддержки и расширения. Кроме того, появляется возможность повторного использования уже разработанных компонентов в последующих приложениях.

Роли объекта

Фактически, при объектно-ориентированном проектировании проектировщику необходимо иметь антропоморфное воображение и относиться к каждому объекту, который вам нужно создать, как к персонажу со способностью мыслить с интеллектуальным суждением. В человеческой жизни, не является ли само собой разумеющимся делегировать профессиональную работу профессиональным людям?

Распределяя обязанности, мы должны сначала прояснить роль объекта, а затем можем внести свои мысли в эту роль, поставить себя на место и сделать вывод, какие обязанности эта роль может или должна взять на себя.

Первый шаг распределения обязанностей – выявление роли объекта.

Роли объектов можно разделить на пять типов:

1. **Владелец информации** (информационный эксперт)
2. **Поставщик услуг**
3. **Конструктор**
4. **Координатор**
5. **Контроллер**

Это абстрактное подразделение в дизайне, которое сильно отличается от роли, которую вы определяете для конкретного бизнеса. Однако при разграничении обязанностей и попытке их распределения действительно существует определенное контрольное значение. Исходя из этого, при разработке программного обеспечения вы можете подумать об объекте, который хотите создать, и о том, к какой роли он принадлежит.

❶. Владелец информации (эксперт)

Любой объект хранит информацию. В чем же особенность держателя информации? Основная цель разграничения ролей здесь – это дать понять разработчику, что поведение объекта связанно с информацией, которой обладает этот объект. Сначала надо рассмотреть вопрос о том, должен ли владелец информации взять на себя ответственность.

В человеческой жизни, не является ли само собой разумеющимся делегировать профессиональную работу профессиональным людям?

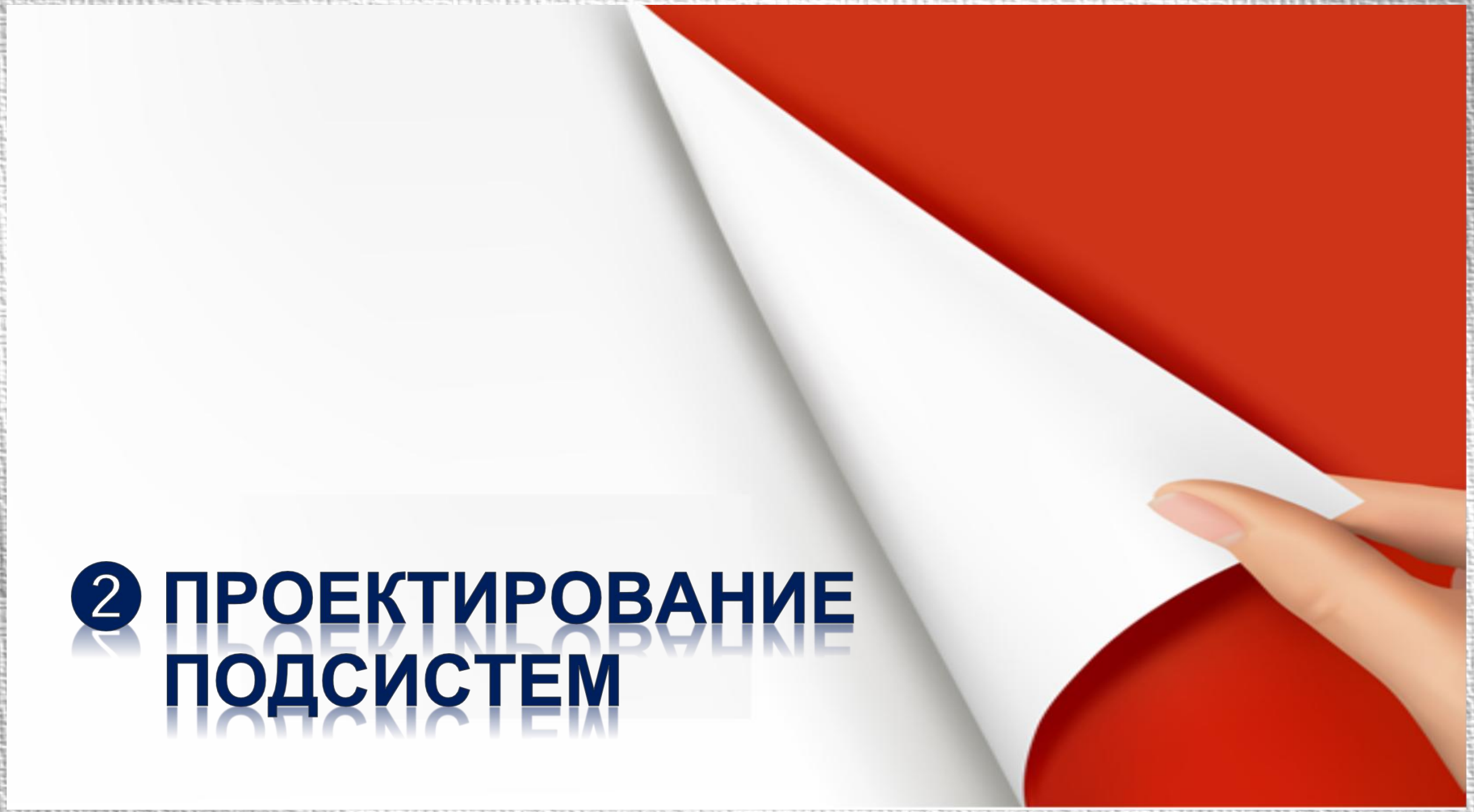
Поэтому первым результатом проектирования является объединение поведения рабочей информации с объектом, которому эта информация принадлежит. Это так называемый принцип «Данные и поведение должны быть объединены вместе».

Проектирование объектов и карты CRC

Для распределения обязанностей и выявления взаимодействия между объектами иногда используется еще одно средство, формально не являющееся частью языка UML–карты CRC (Class-Responsibility-Collaboratorcards).

Карты CRC– это индексные карты (по одной на каждый класс), на которых кратко описаны обязанности классов и перечислены объекты, взаимодействующие с данным классом при выполнении этих обязанностей.

Карты CRC– это лишь один из возможных подходов к записи результатов распределения обязанностей. Следующим шагом на этом пути является создание диаграмм классов и взаимодействий. Реальное значение имеют не сами карты как таковые, а пристальное внимание к процессу распределения обязанностей.



❷ ПРОЕКТИРОВАНИЕ ПОДСИСТЕМ

Детальное проектирование
(проектирование элементов системы)

1. Проектирование прецедентов (уточнение реализаций вариантов использования).
2. Проектирование подсистем.
3. Проектирование классов.
4. Проектирование баз данных.
5. Проектирование интерфейса.

После уточнения реализаций вариантов использования осуществляется проектирование подсистем. Оно включает в себя следующие действия:

- Распределение поведения подсистемы по ее элементам
- документирование взаимодействия элементов подсистемы в виде кооперaций («реализаций интерфейса»); построение одной или более диаграмм взаимодействия для каждой операции интерфейса подсистемы
- Документирование элементов подсистемы (в виде диаграмм классов)
- Описание зависимостей между подсистемами.

Подсистемы можно независимо

- заказывать, настраивать или доставлять;
- разрабатывать, пока интерфейсы остаются без изменений;
- развертывать в наборе распределенных вычислительных узлов;
- изменять без ущерба для других частей системы.

Таким образом, подсистемы идеальны для моделирования компонентов - сменных блоков, предназначенных для сборки во время разработки на базе компонентов.

Примечание: в UML 2.0 определен стереотип компонента, который называется <<subsystem>>; это означает, что его можно использовать, например, для представления крупномасштабных структур.

При определении подсистемы, которая будет представлять продукт, определите также один или несколько интерфейсов для представления интерфейсов продукта.

Подсистемы проектирования (моделируемые как компоненты UML) отличаются от пакетов по своей семантике: подсистема предоставляет поведение через один или несколько интерфейсов, которые она реализует. Пакеты не предоставляют поведение; они просто служат контейнерами объектов, предоставляющих поведение.

Причина, по которой следует применять подсистему, а не пакет, заключается в том, что подсистемы инкапсулируют свое содержимое, предоставляя поведение только через свои интерфейсы. Преимущество такого подхода в том, что, в отличие от пакета, содержимое и внутреннее поведение подсистемы можно беспрепятственно изменять, пока ее интерфейсы остаются без изменений. Подсистемы предоставляют также 'сменный элемент проектирования': любые два компонента <<realization>>, реализующие одинаковые интерфейсы (или компонент <<specification>>), взаимозаменяемы.

Спецификация подсистем

Если внутреннее состояние и поведение подсистемы сравнительно просты, то может оказаться достаточно указать подсистему посредством ее открытых интерфейсов, диаграмм состояния, описывающих поведение, и текста описания.

Если внутреннее состояние и поведение более сложные, то с помощью классов анализа можно указать подсистему на высоком уровне абстракции. В случае больших систем, состоящих из других систем, спецификация подсистемы может также включать варианты использования.

Полная спецификация определяет все, что необходимо знать клиенту для работы с подсистемой. Это означает уточнение открытых интерфейсов и всех общедоступных видимых элементов, так чтобы они в точности соответствовали коду.

Классы анализа, введенные с целью указать поведение подсистемы, должны оставаться на высоком уровне абстракции, поскольку предполагается, что они независимы от реализаций подсистем.

3 ПРОЕКТИРОВАНИЕ КЛАССОВ

Детальное проектирование
(проектирование элементов системы)

1. **Проектирование прецедентов** (уточнение реализаций вариантов использования).
2. **Проектирование подсистем.**
3. **Проектирование классов.**
4. **Проектирование баз данных.**
5. **Проектирование интерфейса.**

❸ ПРОЕКТИРОВАНИЕ КЛАССОВ

1. **Р**аспределите поведение. Для каждого прецедента:
   - выявите сообщения, которыми обмениваются объекты, и соответствующие им методы. Нарисуйте диаграмму последовательности так, чтобы текст прецедента располагался сверху вниз вдоль левой стороны, а сообщения были справа. Вновь модифицируйте диаграмму классов, добавив новые атрибуты и операции;
   - если хотите, покажите на диаграмме кооперации основные транзакции, выполняемые объектами;
   - по желанию продемонстрируйте поведение в реальном времени на диаграмме состояний.

2. **З**авершите статическую модель, добавив в нее детальную проектную информацию (например, о видимости и примененных паттернах).
3. **В**месте с коллегами проверьте, что проект удовлетворяет всем записанным ранее требованиям.

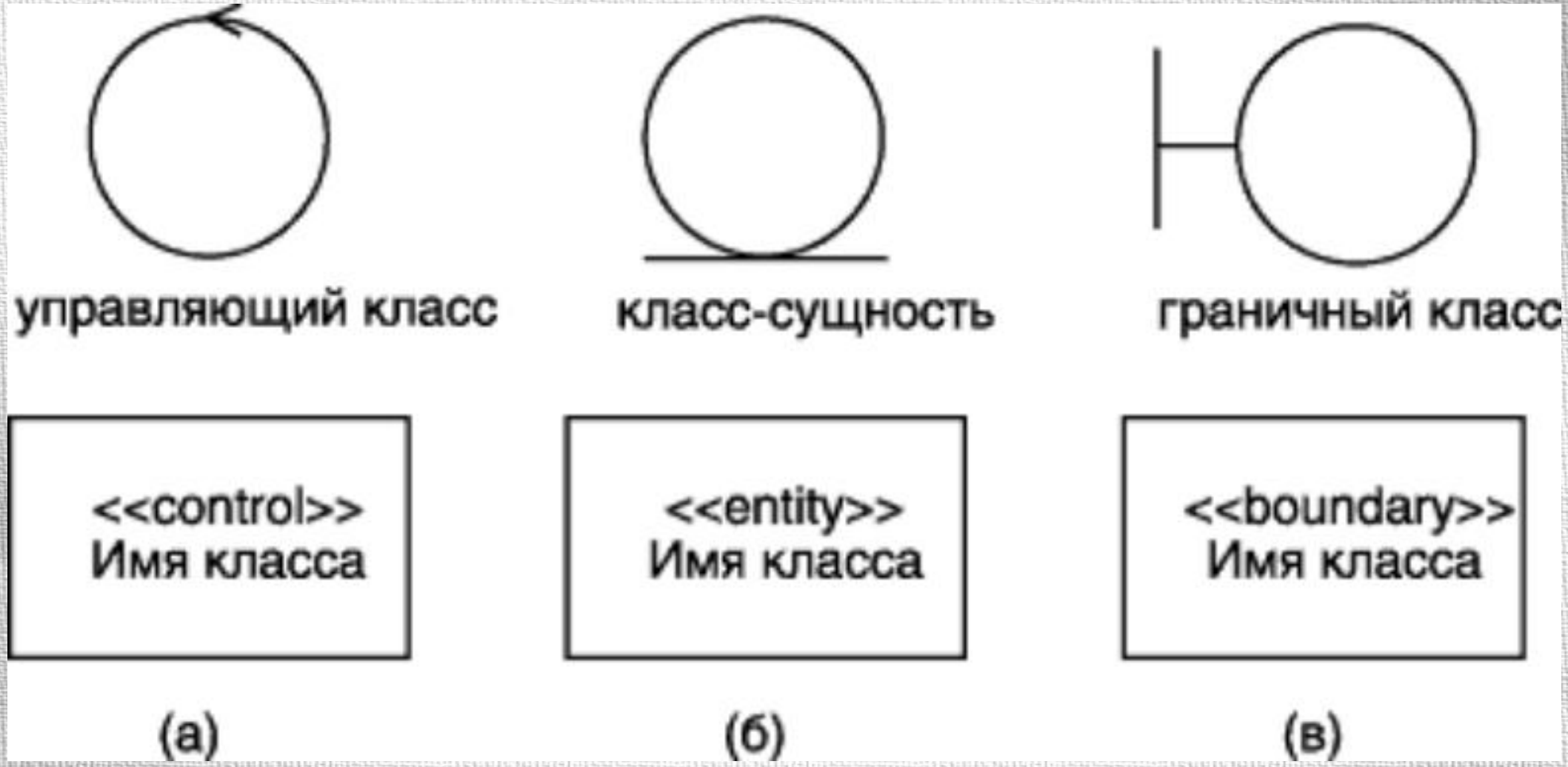
Стереотипы классов

Стереотипы классов – это механизм, позволяющий разделять классы на категории.

В языке UML определены три основных стереотипа классов:

- **Граничный класс (Boundary**). Граничный класс может быть изображен также стандартным образом в форме прямоугольника класса со стереотипом <<boundary>>
- **Класс-сущность** (**Entity**). Класс -сущность может быть изображен стандартным образом в форме прямоугольника класса со стереотипом <<entity>>
- **Управляющий класс** (**Control**). Управляющий класс может быть изображен в форме прямоугольника класса со стереотипом <<control>>

Графическое изображение классов



Граничные классы

Граничными классами (boundary classes) называются такие классы, которые расположены на границе системы и всей окружающей среды. Это экранные формы, отчеты, интерфейсы с аппаратурой и интерфейсы с другими системами.

Чтобы найти граничные классы, надо исследовать диаграммы вариантов использования. Каждому взаимодействию между действующим лицом и вариантом использования должен соответствовать, по крайней мере, один граничный класс. Именно такой класс позволяет действующему лицу взаимодействовать с системой.

Классы-сущности

Классы-сущности (entity classes) содержат хранимую информацию. Они имеют наибольшее значение для пользователя, и потому в их названиях часто используют термины из предметной области. Класс-сущность содержит информацию, которая должна храниться постоянно и не уничтожается с уничтожением объектов данного класса или прекращением работы моделируемой системы, связанные с выключением системы или завершением программы. Обычно для каждого класса-сущности создают таблицу в базе данных.

В этом случае его *атрибуты* являются полями таблицы, а *операции* – присоединенными или хранимыми процедурами. Этот *класс* пассивный и лишь принимает сообщения от других *классов* модели.

Управляющие классы

Управляющие классы (control classes) отвечают за координацию действий других классов. Обычно у каждого варианта использования имеется один управляющий класс, контролирующий последовательность событий этого варианта использования. В дальнейшем они могут объединяться, разделяться или исключаться. Управляющий класс отвечает за координацию, но сам не несет в себе никакой функциональности, так как остальные классы не посылают ему большого количества сообщений. Вместо этого он сам посылает множество сообщений. Управляющий класс просто делегирует ответственность другим классам, по этой причине его часто называют классом-менеджером.

В системе могут быть и другие управляющие классы, общие для нескольких вариантов использования. Например, может быть

- Класс ***SecurityManager*** (менеджер безопасности), отвечающий за контроль событий, связанных с безопасностью.
- Класс ***TransactionManager*** (менеджер транзакций) занимается координацией сообщений, относящихся к транзакциям с базой данных.

Могут быть и другие менеджеры для работы с другими элементами функционирования системы, такими как разделение ресурсов, распределенная обработка данных или обработка ошибок.

Шаблон Controller

Обсуждение. Большинство систем получает внешние события. Обычно они связаны с графическим интерфейсом пользователя. Кроме того, системе могут передаваться внешние сообщения, например, при обработке телекоммуникаци-онных сигналов или сигналов от датчиков в системах управления.

Во всех случаях при использовании объектно-ориентированного подхода для обработки внешних событий применяются контроллеры. Шаблон Controller обеспечивает наиболее типичные проектные решения для этого случая.

Проблема. Кто должен отвечать за обработку входных системных событий?

Системное событие (systemevent) – это событие высокого уровня, генери-руемое внешним исполнителем (событие с внешним входом). Системные события связаны с системными операциями (systemoperation), т.е. операциями, выпол-няемыми системой в ответ на события.

> Например, когда кассир в POS-системе щелкает на кнопке *Оплатить*, он гене-рирует системное событие, свидетельствующее о завершении торговой операции.

Контроллер (controller) – это объект, не относящийся к интерфейсу поль-зователя и отвечающий за обработку системных событий. Контроллер определя-ет методы для выполнения системных операций.

Решение. Делегирование обязанностей по обработке системных сообщений классу, удовлетворяющему одному из следующих условий:

- Класс представляет всю систему в целом, устройство или подсистему (внешний контроллер).
- Класс представляет сценарий некоторого прецедента, в рамках которого выпол-няется обработка всех системных событий, и обычно называется <Прецедент> **Handler**, <Прецедент> **Coordinator** или <Прецедент> **Session** (контроллер прецедента или контроллер сеанса). Для всех системных событий в рамках одного сценария прецедента ис-пользуется один и тот же класс-контроллер.

- Неформально, сеанс – это экземпляр взаимодействия с исполнителем. Сеансы могут иметь произвольную длину, но зачастую организованы в рамках прецедента (сеансы прецедента).

Следствие. Заметим, что в этот перечень не включаются классы, реали-зующие окно, приложение, вид и документ. Такие граничные классы не выполняют задачи, связанные с системными событиями. Они обычно получают сообщения и делегируют их контроллерам.

Чтобы обеспечить возможность поддержки информации о состоянии прецедента, для обработки всех системных событий в рамках одного прецедента дол-жен использоваться один и тот же класс контроллера. Такая информация может понадобиться, например, для идентификации момента нарушения последова-тельности системных событий. Для различных прецедентов можно ис-пользовать разные контроллеры.

Типичной ошибкой при создании контроллеров является возложение на них слишком большого числа обязанностей.

Обычно контроллер должен лишь делегировать функции другим объектам и ко-ординировать их деятельность, а не выполнять эти действия самостоятельно.

Важным следствием применения шаблона Controller является вынесение обя-занностей по выполнению системных событий за пределы уровня представления и внешнего интерфейса объектов (например, объектов окон). Другими словами, системные операции, отражающие процессы в предметной области, долж-ны обрабатываться на уровне логики приложения или реализации объектов, а не на уровне интерфейса.

Объект-контроллер, как правило, относится к клиентской части приложения и функционирует в рамках того же процесса, что и интерфейс пользователя. Поэтому шаблон Controller напрямую неприменим, если в качестве клиента выступает Web-броузер. Для такой архитектуры существуют различные шаблоны обработки системных со-бытий, основанные на использовании серверных технических каркасов. Чаще всего основная идея сводится к использованию кон-троллеров прецедентов в серверной части приложения, когда обработку каждого прецедента осуществляет либо отдельный сервлет, либо компонент-обработчик сеанса EJB (EnterpriseJavaBeans). Серверный объект-обработчик сеанса обслуживает один сеанс взаимодействия с внешним исполнителем.

Типы операций

Следует рассмотреть четыре различных типа операций:

- Операции реализации;
- Операции управления;
- Операции доступа;
- Вспомогательные операции

Операции реализации

Операции реализации (implementor operations) реализуют некоторые бизнес-функции. Такие операции можно найти, исследуя диаграммы взаимодействия. Диаграммы этого типа фокусируются на бизнес-функциях, и каждое сообщение диаграммы, скорее всего, можно соотнести с операцией реализации.

Каждая операция реализации должна быть легко прослеживаема до соответствующего требования. Это достигается на различных этапах моделирования. Операция выводится из сообщения на диаграмме взаимодействия, сообщения исходят из подробного описания потока событий, который создается на основе варианта использования, а последний – на основе требований. Возможность проследить всю эту цепочку позволяет гарантировать, что каждое требование будет реализовано в коде, а каждый фрагмент кода реализует какое-то требование.

Операции управления

Операции управления (manager operations) управляют созданием и уничтожением объектов. В эту категорию попадают конструкторы и деструкторы классов.

Операции доступа

Атрибуты обычно бывают закрытыми или защищенными. Тем не менее, другие классы иногда должны просматривать или изменять их значения. Для этого существуют операции доступа (access operations). Такой подход дает возможность безопасно инкапсулировать атрибуты внутри класса, защитив их от других классов, но все же позволяет осуществить к ним контролируемый доступ. Создание операций Get и Set (получения и изменения значения) для каждого атрибута класса является стандартом.

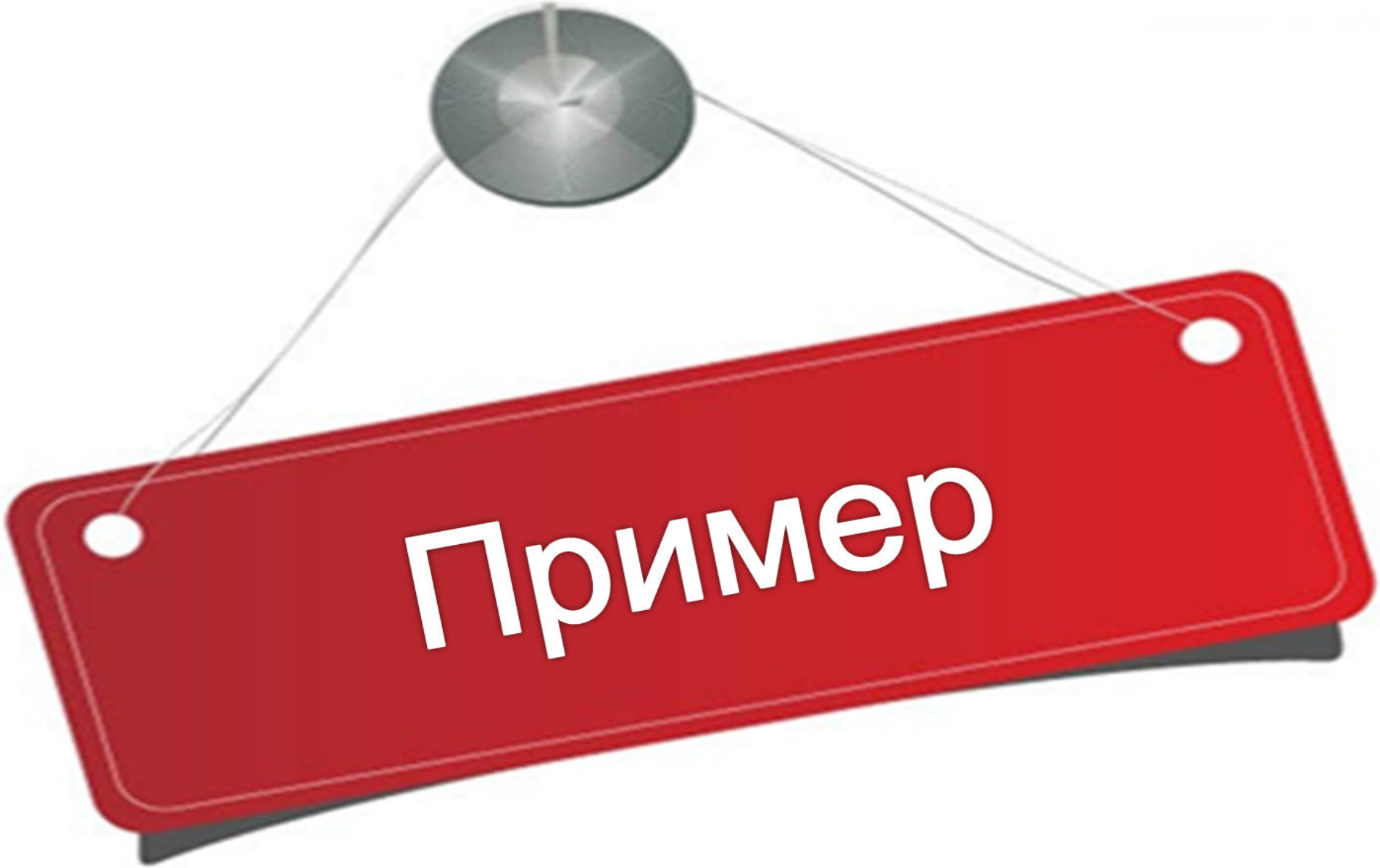
Вспомогательные операции

Вспомогательными (helper operations) называются такие операции класса, которые необходимы ему для выполнения его ответственностей, но о которых другие классы не должны ничего знать. Это закрытые и защищенные операции класса.

Чтобы идентифицировать операции, выполните следующие действия:

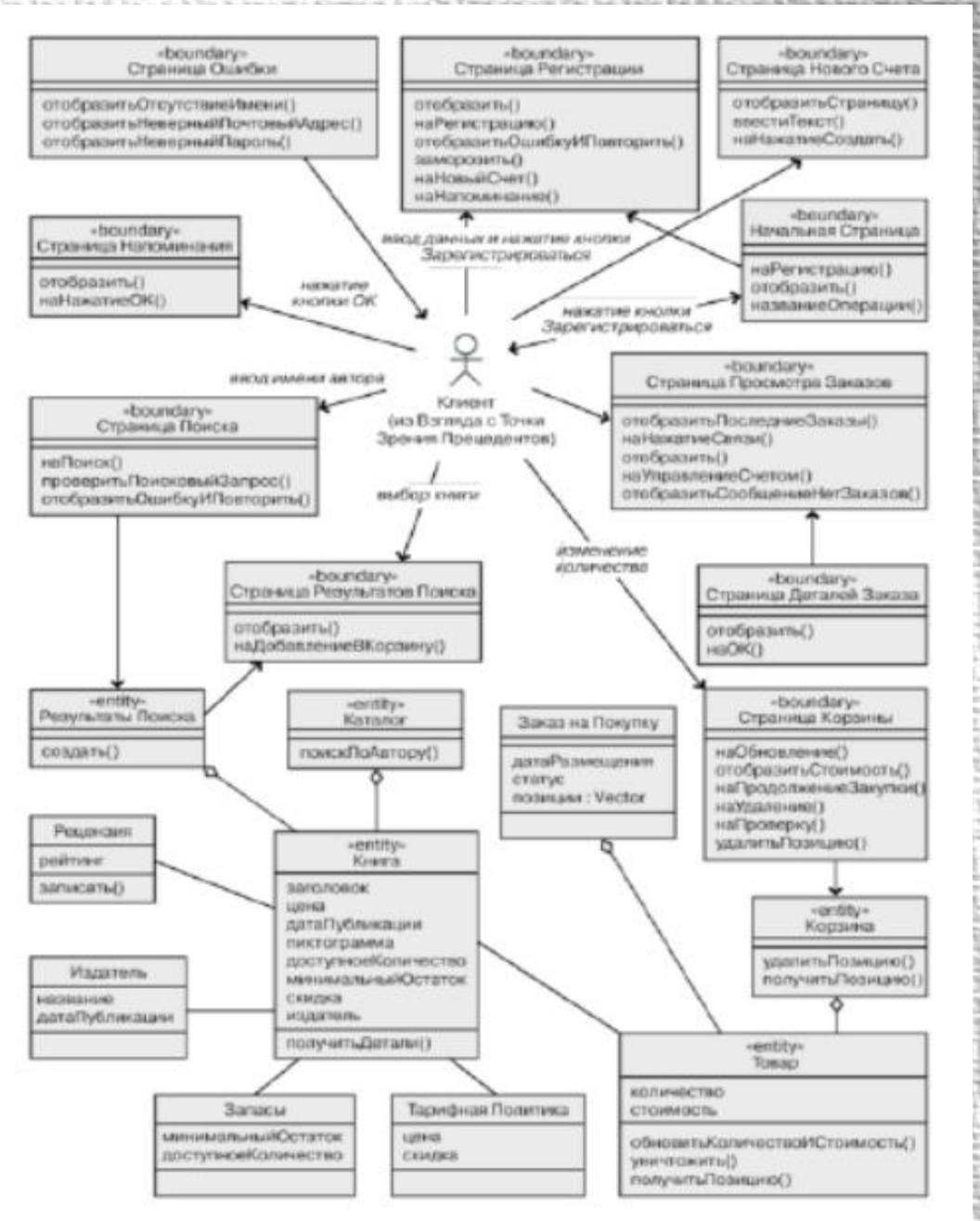
- Изучите диаграммы последовательности и кооперативные диаграммы. Большая часть сообщений на этих диаграммах является операциями реализации. Рефлексивные сообщения будут вспомогательными операциями.

- Рассмотрите управляющие операции. Может потребоваться добавить конструкторы и деструкторы.
- Рассмотрите операции доступа. Для каждого атрибута класса, с которым должны будут работать другие классы, надо создать операции **Get** и **Set**.

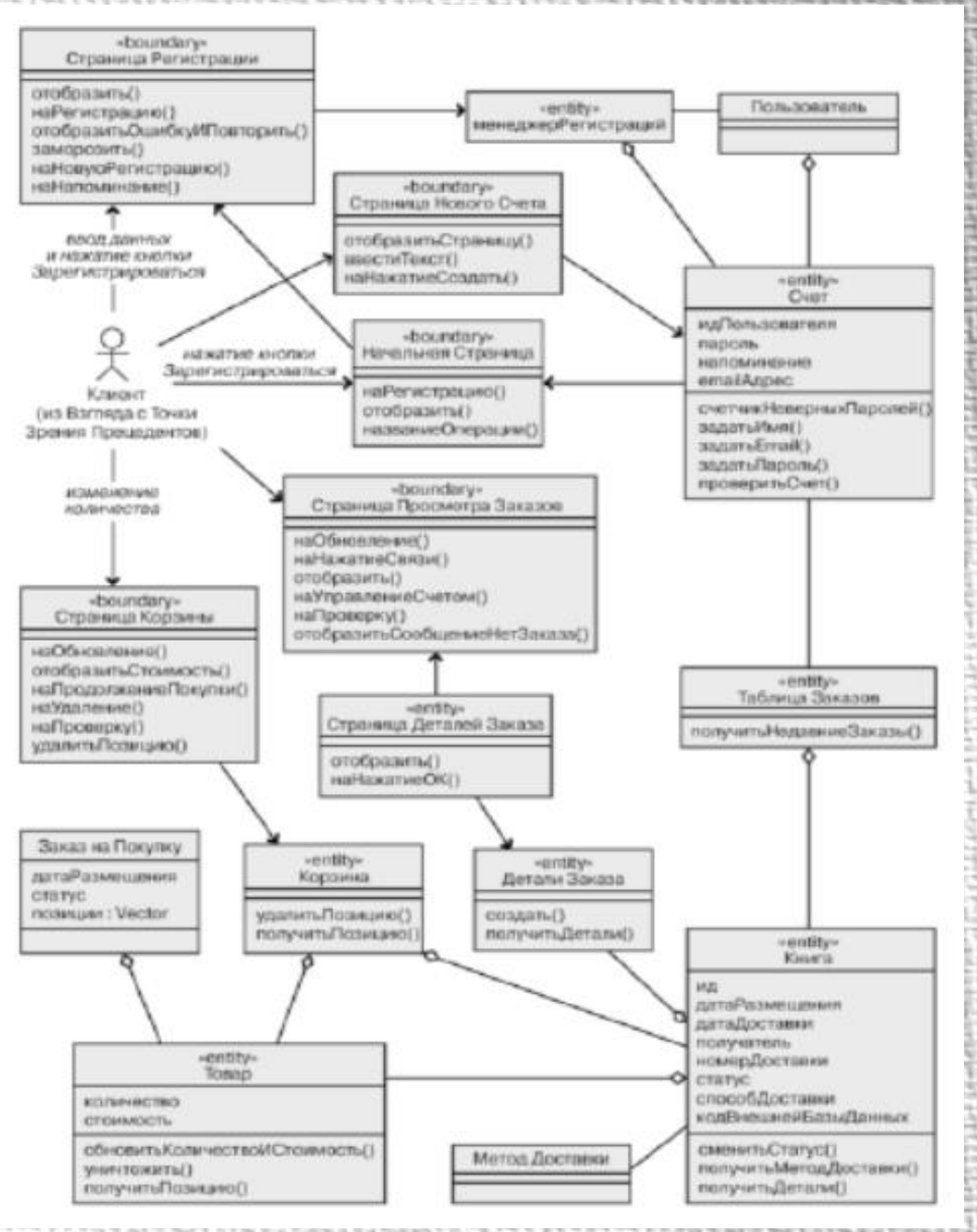

Пример

Диаграммы классов уровня проектирования

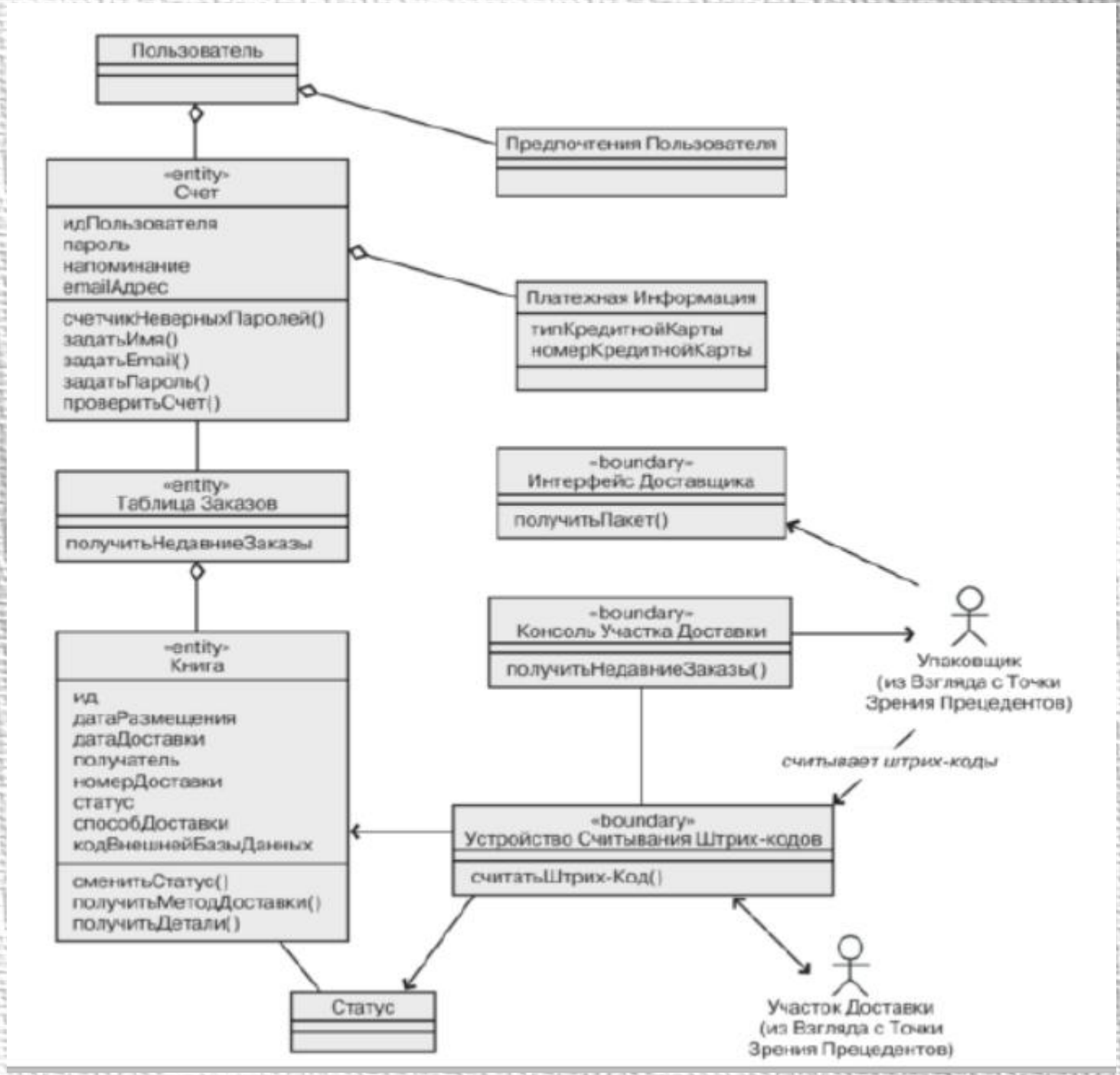
Часть 1



2. Часть 2


«boundary»
Страница Регистрации
отобразить()
наРегистрацию()
отобразитьОшибкуИПовторить()
заморозить()
наНовуюРегистрацию()
наНапоминание()
«entity»
менеджерРегистраций
Пользователь
«boundary»
Страница Нового Счета
отобразитьСтраницу()
ввестиТекст()
наНажатиеСоздать()
«entity»
Счет
идПользователя
пароль
напоминание
емайлАдрес
счетчикНеверныхПаролей()
задатьИмя()
задатьЕмайл()
задатьПароль()
проверитьСчет()
ввод данных
и нажатие кнопки
Зарегистрироваться
нажатие кнопки
Зарегистрироваться
Клиент
(из Взгляда с Точки
Зрения Прецедентов)
«boundary»
Начальная Страница
наРегистрацию()
отобразить()
названиеОперации()
изменение
количества
«boundary»
Страница Просмотра Заказов
наОбновление()
наНажатиеСвязи()
отобразить()
наУправлениеСчетом()
наПроверку()
отобразитьСообщениеНетЗаказа()
«boundary»
Страница Корзины
наОбновление()
отобразитьСтоимость()
наПродолжениеПокупки()
наУдаление()
наПроверку()
удалитьПозицию()
«entity»
Страница Деталей Заказа
отобразить()
наНажатиеОК()
«entity»
Таблица Заказов
получитьНедавниеЗаказы()
Заказ на Покупку
датаРазмещения
статус
позиции : Vector
«entity»
Корзина
удалитьПозицию()
получитьПозицию()
«entity»
Детали Заказа
создать()
получитьДетали()
«entity»
Книга
ид
датаРазмещения
датаДоставки
получатель
номерДоставки
статус
способДоставки
кодВнешнейБазыДанных
сменитьСтатус()
получитьМетодДоставки()
получитьДетали()
«entity»
Товар
количество
стоимость
обновитьКолчествоИСтоимость()
уничтожить()
получитьПозицию()
Метод Доставки

2. Часть 3

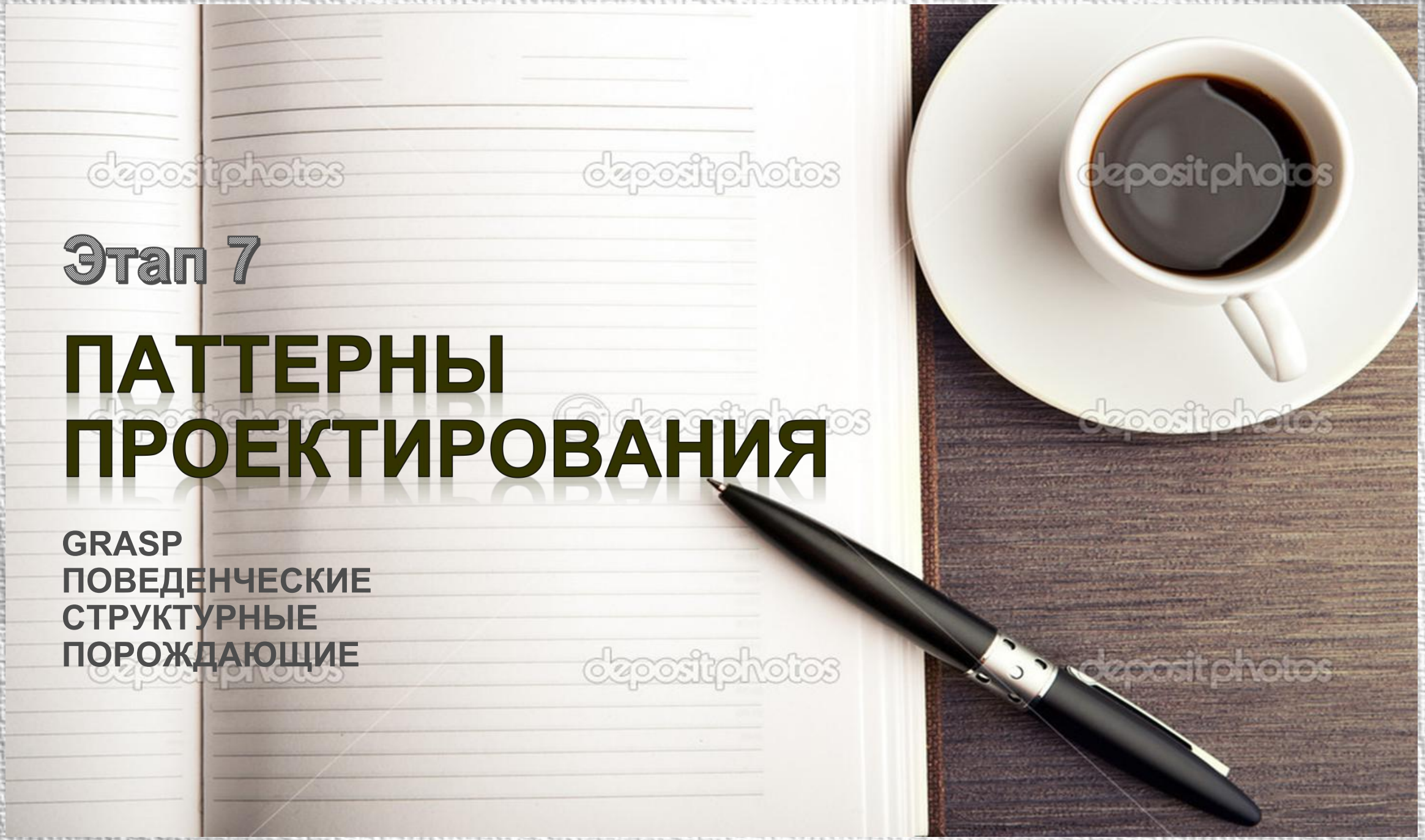

Пользователь
Предпочтения Пользователя
«entity»
Счет
идПользователя
пароль
напоминание
emailАдрес
счетчикНеверныхПаролей()
задатьИмя()
задатьEmail()
задатьПароль()
проверитьСчет()
Платежная Информация
типКредитнойКарты
номерКредитнойКарты
«entity»
Таблица Заказов
получитьНедавниеЗаказы
«boundary»
Интерфейс Доставщика
получитьПакет()
«boundary»
Консоль Участка Доставки
получитьНедавниеЗаказы()
Упаковщик
(из Взгляда с Точки
Зрения Прецедентов)
считывает штрих-коды
«entity»
Книга
ид
датаРазмещения
датаДоставки
получатель
номерДоставки
статус
способДоставки
кодВнешнейБазыДанных
сменитьСтатус()
получитьМетодДоставки()
получитьДетали()
«boundary»
Устройство Считывания Штрих-кодов
считатьШтрих-Код()
Статус
Участок Доставки
(из Взгляда с Точки
Зрения Прецедентов)

4 ПРОЕКТИРОВАНИЕ БАЗ ДАННЫХ

Очень надеюсь, что вы это знаете!

5 ПРОЕКТИРОВАНИЕ ИНТЕРФЕЙСА

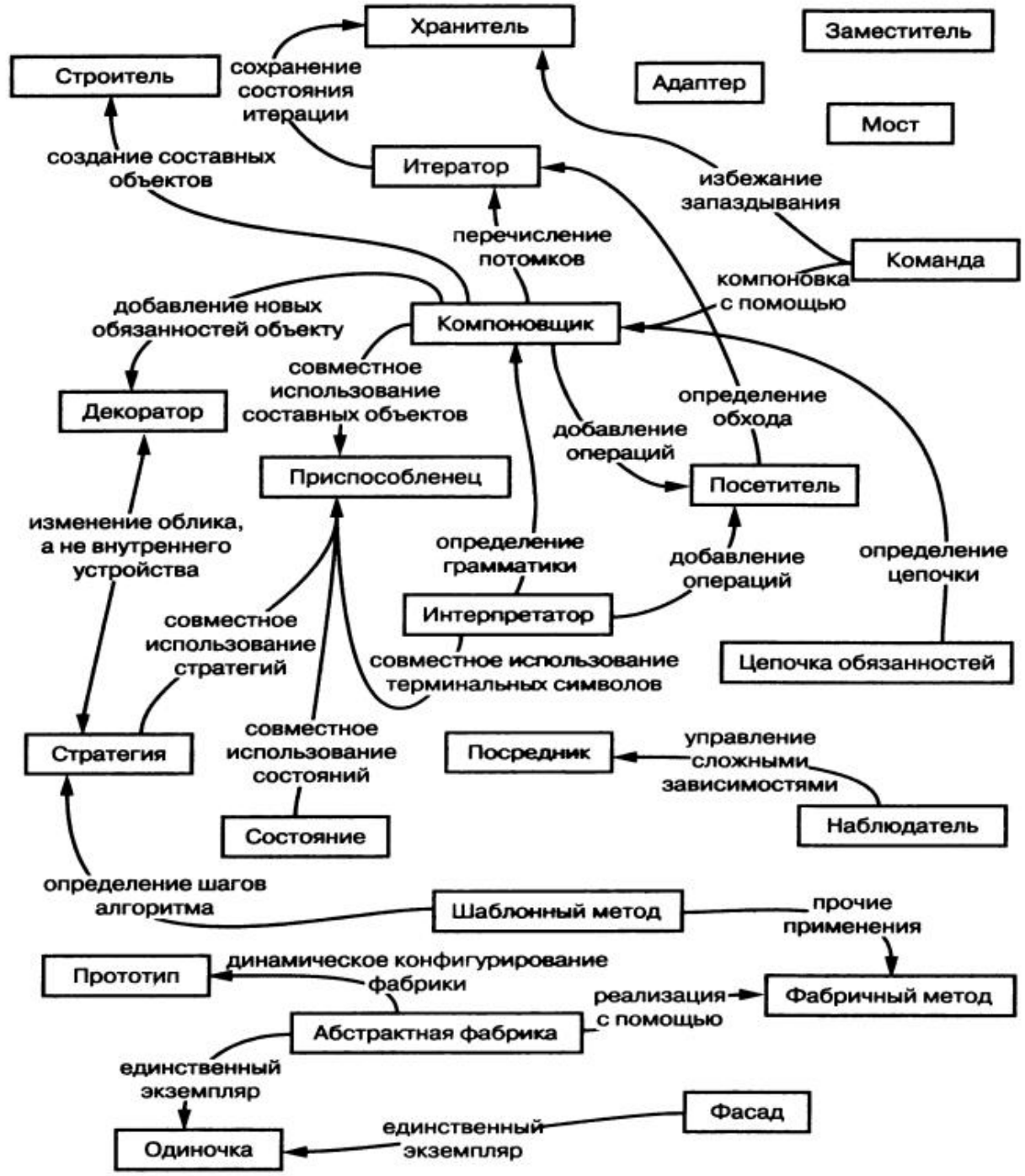
Проектирование интерфейса в другом курсе!



Этап 7

ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ

GRASP
ПОВЕДЕНЧЕСКИЕ
СТРУКТУРНЫЕ
ПОРОЖДАЮЩИЕ


Хранитель
Заместитель
Строитель
сохранение состояния итерации
Адаптер
Мост
создание составных объектов
Итератор
избежание запаздывания
перечисление потомков
Команда
компоновка с помощью
добавление новых обязанностей объекту
Компоновщик
совместное использование составных объектов
определение обхода
Декоратор
добавление операций
Посетитель
Приспособленец
изменение облика, а не внутреннего устройства
определение грамматики
добавление операций
определение цепочки
совместное использование стратегий
Интерпретатор
совместное использование терминальных символов
Цепочка обязанностей
Стратегия
совместное использование состояний
Посредник
управление сложными зависимостями
Состояние
Наблюдатель
определение шагов алгоритма
Шаблонный метод
прочие применения
Прототип
динамическое конфигурирование фабрики
реализация с помощью
Фабричный метод
Абстрактная фабрика
единственный экземпляр
Фасад
Одиночка
единственный экземпляр

Некоторые проблемы слишком малы для выделения в архитектурный подход, но достаточно часто встречаются, чтобы породить стандартные решения. Такие стандартные решения называются паттернами или шаблонами.

Например, сервер присылает нам данные в виде:

```
{ some_data: ["Name", "Lastname"] }
```

А мы хотим их видеть такими:

```
{ someData: "Name Lastname" }
```

Для решения такой проблемы есть паттерн «Адаптер».

Он делает несовместимое стороннее API подходящим для нашего приложения.

```
function serverToClientAdapter(data) { return { someData:
data.some_data.join(" "), } }
```

Таких стандартных решений много. Мы можем разделить их на несколько групп.

ШАБЛОНЫ GRASP

Фундаментальные принципы распределения ответственности между классами

GRASP (General Responsibility Assignment Software Patterns) — шаблоны проектирования, используемые для решения общих задач по назначению обязанностей классам и объектам.

В отличие от привычных паттернов из Банды Четырех, GRAPS паттерны не имеют выраженной структуры, четкой области применения и конкретной решаемой проблемы, а лишь представляют собой обобщенные подходы/рекомендации/принципы, используемые при проектировании дизайна системы.

Шаблоны GRASP

- Information Expert
- Creator
- Controller
- Low Coupling
- High Cohesion
- Polymorphism
- Pure Fabrication
- Indirection
- Protected Variations

1. Information Expert

Суть данного паттерна можно выразить следующим образом: **информация должна обрабатываться там, где она содержится**.

Несмотря на кажущуюся простоту и очевидность, в коде любого проекта можно найти множество нарушений данного принципа.

Рассмотрим простейшую систему классов: `Order (заказ)`, содержащий список `OrderItem'ов (строк заказа)`, элементы которого в свою очередь содержат `Good (товар)` и его количество, а товар может содержать, например, цену, название и т.д.:

3. Шаблон Low Coupling

Low Coupling, слабая связанность, означает что модули, на которые разбивается система, должны быть, по возможности, независимы или слабо связанны друг с другом. Они должны иметь возможность взаимодействовать, но при этом как можно меньше знать друг о друге (принцип минимального знания).

Это значит, что при правильном проектировании, при изменении одного модуля, не придется править другие или эти изменения будут минимальными. Чем слабее связанность, тем легче писать, понимать, расширять, чинить программу.

Самым ярким примером нарушения этого принципа является циклическая зависимость (это то, за что обычно отрывают руки на code review).

На UML — диаграмме классов для такой системы можно будет увидеть как зависимость класса A на класс B, так и зависимость класса B на класс A. Почему это плохо? Дело в том, что мы не можем отдать класс A без класса B, также как и класс B без класса A: их нельзя переиспользовать по отдельности, только вместе. Чем меньше связей между классами — тем лучше, вот о чем говорит нам принцип Low Coupling.

4. High Cohesion

Формулировка

Если возвести Low Coupling в абсолют, то достаточно быстро можно прийти к тому, чтобы разместить всю функциональность в одном единственном классе. В таком случае связей не будет вообще, но все при этом понимают, что что-то тут явно не так, ведь в этот класс попадет совершенно несвязанная между собой бизнес — логика. Принцип High Cohesion говорит нам следующее:

классы должны содержать связанную бизнес — логику.

Пусть класс содержит бизнес — логику по работе как с температурой, так и со временем. Они не имеют ничего общего, за исключением того, что собираются с одного датчика. Если мы захотим переиспользовать бизнес — логику, связанную по работе только с температурой, то осуществить это легко не получится.

Если проводить параллели с принципами SOLID, то класс нарушает SRP: через него проходят две оси изменения.

Вывод

Low Coupling и *High Cohesion* представляют из себя два связанных между собой паттерна, рассматривать которые имеет смысл только вместе. Их суть можно объединить следующим образом: система должна состоять из слабо связанных классов, которые должны содержать связанную бизнес — логику. Соблюдение этих принципов позволяет удобно переиспользовать созданные классы, не теряя понимания об их зоне ответственности.

ПОВЕДЕНЧЕСКИЕ ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ

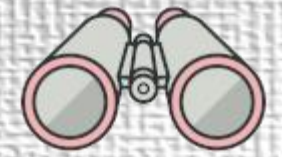
Отвечают за построение удобных в поддержке иерархий классов.

Поведенческие шаблоны проектирования

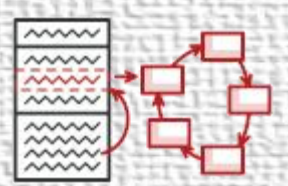
Поведенческие шаблоны (behavioral patterns) — шаблоны проектирования, определяющие алгоритмы и способы реализации взаимодействия различных объектов и классов.

В поведенческих шаблонах уровня класса используется **наследование**, чтобы определить поведение для различных классов. В поведенческих шаблонах уровня объекта используется **композиция**. Некоторые из них описывают, как с помощью кооперации несколько равноправных объектов работают над заданием, которое они не могут выполнить по отдельности.

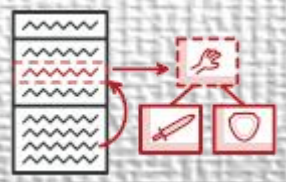
Здесь важно то, как объекты получают информацию о существовании друг друга. Объекты-коллеги могут хранить ссылки друг на друга, но это усиливает степень связанности системы. При высокой связанности каждому объекту пришлось бы иметь информацию обо всех остальных. Некоторые из шаблонов решают эту проблему.



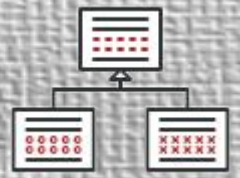
Наблюдатель. Создаёт механизм подписки, позволяющий одним объектам следить и реагировать на события, происходящие в других объектах.



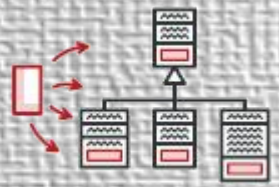
Состояние. Позволяет объектам менять поведение в зависимости от своего состояния. Извне создаётся впечатление, что изменился класс объекта.



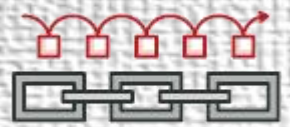
Стратегия. Позволяет объектам менять поведение в зависимости от своего состояния. Извне создаётся впечатление, что изменился класс объекта..



Шаблонный метод. Позволяет уменьшить связанность множества классов между собой, благодаря перемещению этих связей в один класс-посредник.



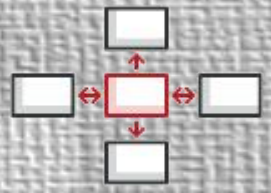
Посетитель. Позволяет добавлять в программу новые операции, не изменяя классы объектов, над которыми эти операции могут выполняться.



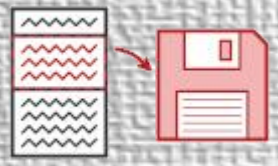
Цепочка обязанностей. Позволяет передавать запросы последовательно по цепочке обработчиков. Каждый последующий обработчик решает, может ли он обработать запрос сам и стоит ли передавать запрос дальше по цепи.

Команда. Превращает запросы в объекты, позволяя передавать их как аргументы при вызове методов, ставить запросы в очередь, логировать их, а также поддерживать отмену операций.

Итератор. Даёт возможность последовательно обходить элементы составных объектов, не раскрывая их внутреннего представления.



Посредник. Позволяет уменьшить связанность множества классов между собой, благодаря перемещению этих связей в один класс-посредник.



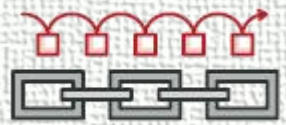
Снимок. Позволяет сохранять и восстанавливать прошлые состояния объектов, не раскрывая подробностей их реализации.

Поведенческие паттерны распределяют ответственность между модулями и определяют, как именно будет происходить общение.

Простыми словами — отвечают на вопрос «Как организовать поведение программного компонента?»

Среди поведенческих паттернов можем выделить:

- **Цепочка обязанностей**
- **Стратегия**
- **Команда**
- **Наблюдатель**

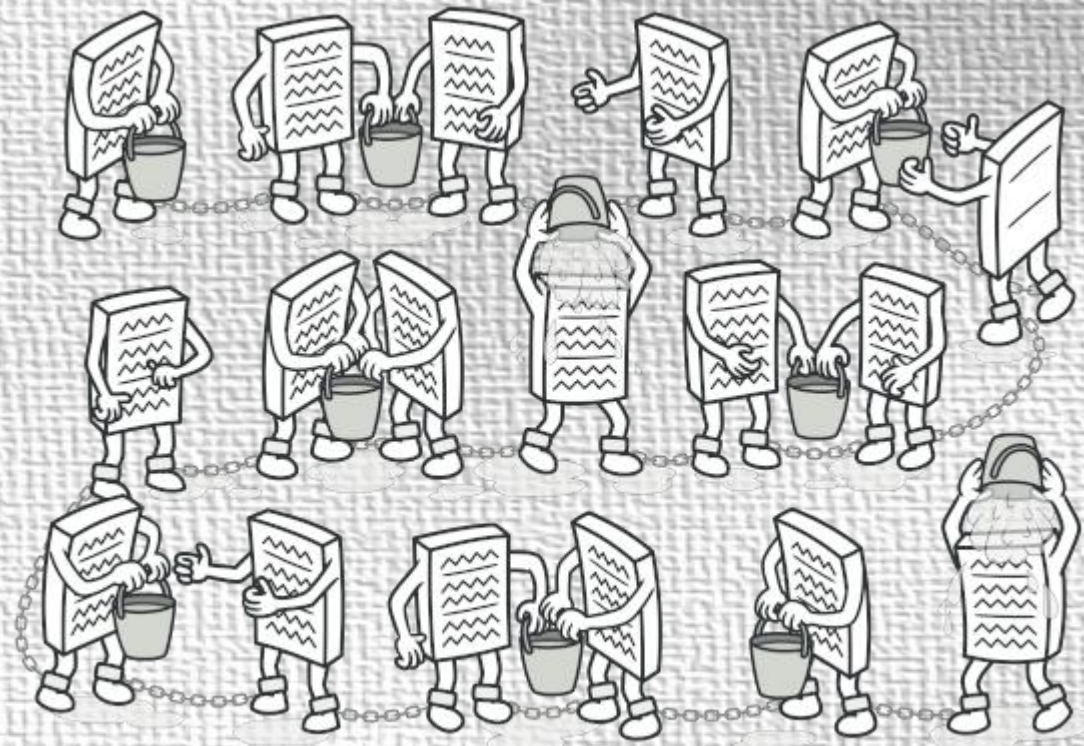


Цепочка обязанностей

Поведенческий паттерн

Суть паттерна

Цепочка обязанностей – это поведенческий паттерн проектирования, который позволяет передавать запросы последовательно по цепочке обработчиков. Каждый последующий обработчик решает, может ли он обработать запрос сам и стоит ли передавать запрос дальше по цепи.



Проблема

Представьте, что вы делаете систему приёма онлайн-заказов. Вы хотите ограничить к ней доступ так, чтобы только авторизованные пользователи могли создавать заказы. Кроме того, определённые пользователи, владеющие правами администратора, должны иметь полный доступ к заказам.

Вы быстро сообразили, что эти проверки нужно выполнять последовательно. Ведь пользователя можно попытаться «залогинить» в систему, если его запрос содержит логин и пароль. Но если такая попытка не удалась, то проверять расширенные права доступа попросту не имеет смысла.

Запрос проходит ряд проверок
перед доступом в систему заказов

На протяжении следующих нескольких месяцев вам пришлось добавить ещё несколько таких последовательных проверок.

Кто-то резонно заметил, что неплохо бы проверять данные, передаваемые в запросе перед тем, как вносить их в систему – вдруг запрос содержит данные о покупке несуществующих продуктов.

Кто-то предложил блокировать массовые отправки формы с одним и тем же логином, чтобы предотвратить подбор паролей ботами.

Кто-то заметил, что форму заказа неплохо бы доставать из кеша, если она уже была однажды показана.

С каждой новой «фичей» код проверок, выглядящий как большой клубок условных операторов, всё больше и больше раздувался. При изменении одного правила приходилось трогать код всех проверок. А для того, чтобы применить проверки к другим ресурсам, пришлось продублировать их код в других классах.

Поддерживать такой код стало не только очень хлопотно, но и затратно. И вот в один прекрасный день вы получаете задачу рефакторинга...

Со временем код проверок становится всё более запутанным

Решение

Как и многие другие поведенческие паттерны, Цепочка обязанностей базируется на том, чтобы превратить отдельные поведения в объекты. **В нашем случае каждая проверка переедет в отдельный класс с единственным методом выполнения.** Данные запроса, над которым происходит проверка, будут передаваться в метод как аргументы.

А теперь по-настоящему важный этап. Паттерн предлагает связать объекты обработчиков в одну цепь. Каждый из них будет иметь ссылку на следующий обработчик в цепи. Таким образом, при получении запроса обработчик сможет не только сам что-то с ним сделать, но и передать обработку следующему объекту в цепочке.

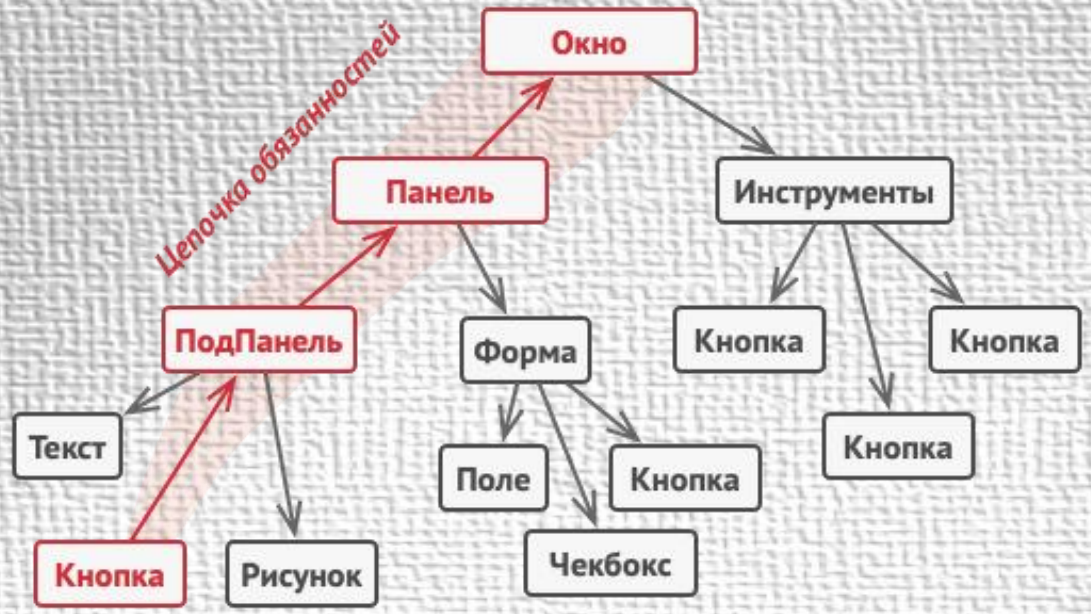
Передавая запросы в первый обработчик цепочки, вы можете быть уверены, что все объекты в цепи смогут его обработать. При этом длина цепочки не имеет никакого значения.

И последний штрих. Обработчик не обязательно должен передавать запрос дальше, причём эта особенность может быть использована по-разному.

В примере с фильтрацией доступа обработчики прерывают дальнейшие проверки, если текущая проверка не прошла. Ведь нет смысла тратить попусту ресурсы, если и так понятно, что с запросом что-то не так.

Но есть и другой подход, при котором обработчики прерывают цепь, только когда они могут обработать запрос. В этом случае запрос движется по цепи, пока не найдётся обработчик, который может его обработать. Очень часто такой подход используется для передачи событий, создаваемых классами графического интерфейса в результате взаимодействия с пользователем.

Например, когда пользователь кликает по кнопке, программа выстраивает цепочку из объекта этой кнопки, всех её родительских элементов и общего окна приложения на конце. Событие клика передаётся по этой цепи до тех пор, пока не найдётся объект, способный его обработать. Этот пример примечателен ещё и тем, что цепочку всегда можно выделить из древовидной структуры объектов, в которую обычно и свёрнуты элементы пользовательского интерфейса.



Очень важно, чтобы все объекты цепочки имели общий интерфейс. Обычно каждому конкретному обработчику достаточно знать только то, что следующий объект в цепи имеет метод выполнить. Благодаря этому связи между объектами цепочки будут более гибкими. Кроме того, вы сможете формировать цепочки на лету из разнообразных объектов, не привязываясь к конкретным классам.

Аналогия из жизни

Вы купили новую видеокарту. Она автоматически определилась и заработала под Windows, но в вашей любимой Ubuntu «завести» её не удалось. Со слабой надеждой вы звоните в службу поддержки.

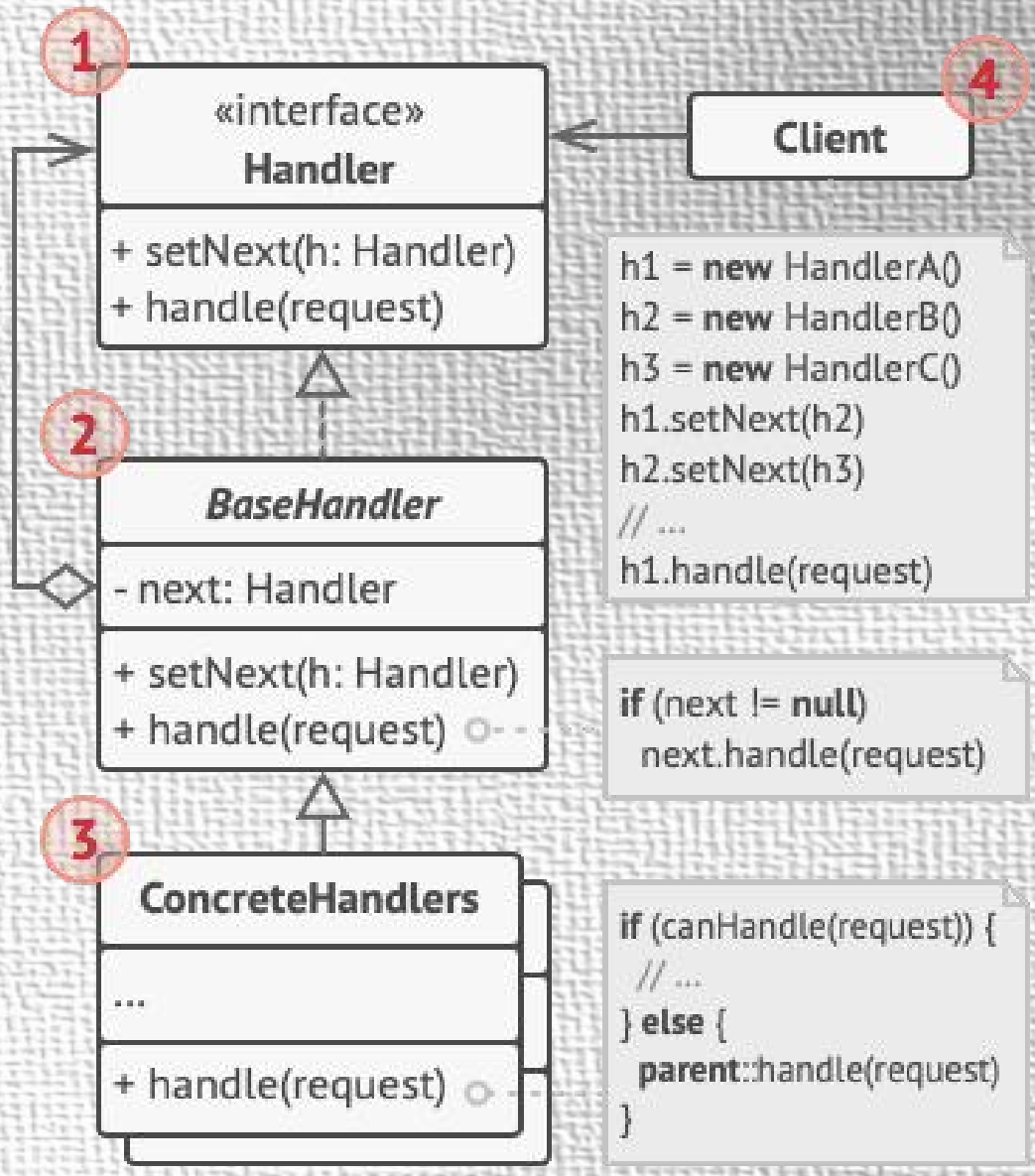
Первым вы слышите голос автоответчика, предлагающий выбор из десятка стандартных решений. Ни один из вариантов не подходит, и робот соединяет вас с живым оператором.

Увы, но рядовой оператор поддержки умеет общаться только заученными фразами и давать шаблонные ответы. После очередного предложения «выключить и включить компьютер» вы просите связать вас с настоящими инженерами.

Оператор перебрасывает звонок дежурному инженеру, изнывающему от скуки в своей каморке. Уж он-то знает, как вам помочь! Инженер рассказывает вам, где скачать подходящие драйвера и как настроить их под Ubuntu. Запрос удовлетворён. Вы кладёте трубку.

Структура

❶. Обработчик определяет общий для всех конкретных обработчиков интерфейс. Обычно достаточно описать единственный метод обработки запросов, но иногда здесь может быть объявлен и метод выставления следующего обработчика.



❷. Базовый обработчик – опциональный класс, который позволяет избавиться от дублирования одного и того же кода во всех конкретных обработчиках.

Обычно этот класс имеет поле для хранения ссылки на следующий обработчик в цепочке. Клиент связывает обработчики в цепь, подавая ссылку на следующий обработчик через конструктор или сеттер поля. Также здесь можно реализовать базовый метод обработки, который бы просто перенаправлял запрос следующему обработчику, проверив его наличие.

❸. Конкретные обработчики содержат код обработки запросов. При получении запроса каждый обработчик решает, может ли он обработать запрос, а также стоит ли передать его следующему объекту.

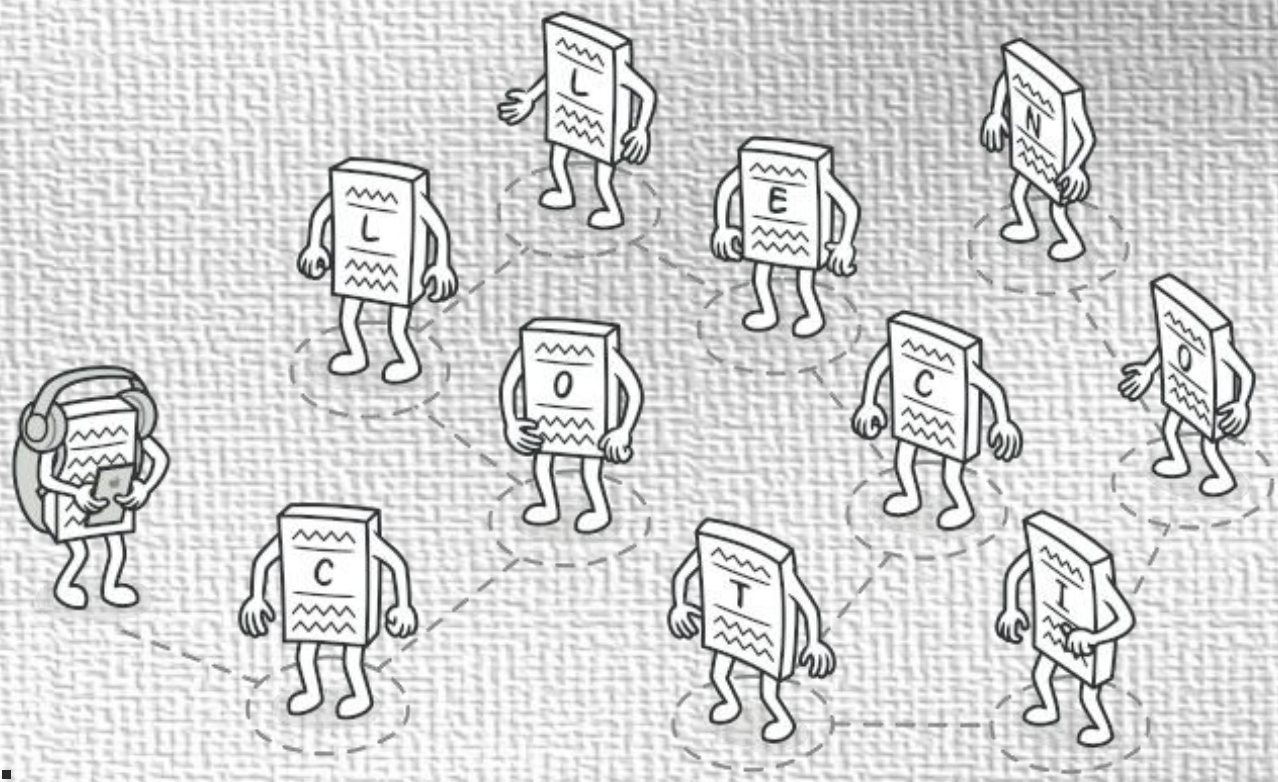
В большинстве случаев обработчики могут работать сами по себе и быть неизменяемыми, получив все нужные детали через параметры конструктора.

❹. Клиент может либо сформировать цепочку обработчиков единожды, либо перестраивать её динамически, в зависимости от логики программы. Клиент может отправлять запросы любому из объектов цепочки, не обязательно первому из них.

ИТЕРАТОР
Поведенческий
паттерн

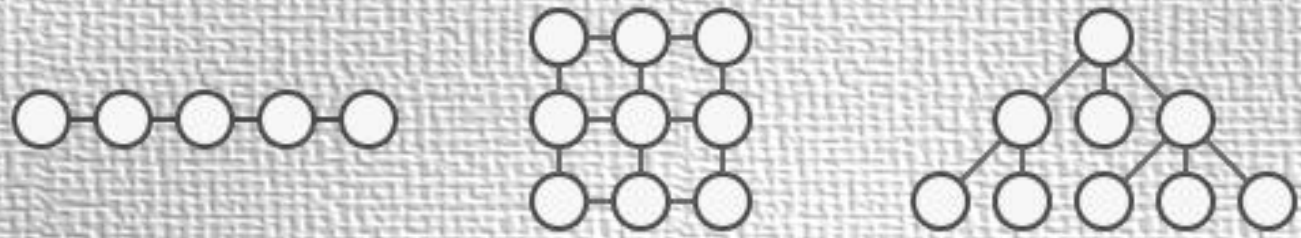
Суть паттерна

Итератор – это поведенческий паттерн проектирования, который даёт возможность последовательно обходить элементы составных объектов, не раскрывая их внутреннего представления.



Проблема

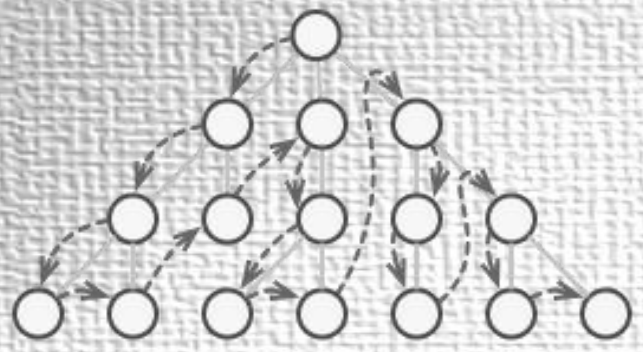
Коллекции – самая распространённая структура данных, которую вы можете встретить в программировании. Это набор объектов, собранный в одну кучу по каким-то критериям.

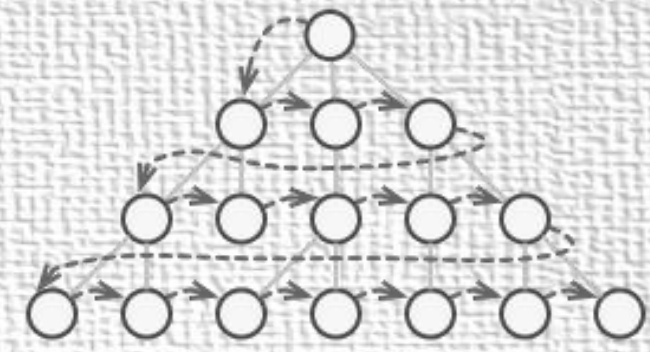


Большинство коллекций выглядят как обычный список элементов. Но есть и экзотические коллекции, построенные на основе деревьев, графов и других сложных структур данных.

Но как бы ни была структурирована коллекция, пользователь должен иметь возможность последовательно обходить её элементы, чтобы проделывать с ними какие-то действия.

Но каким способом следует перемещаться по сложной структуре данных? Например, сегодня может быть достаточным обход дерева в глубину, но завтра потребуется возможность перемещаться по дереву в ширину. А на следующей неделе и того хуже – понадобится обход коллекции в случайном порядке.

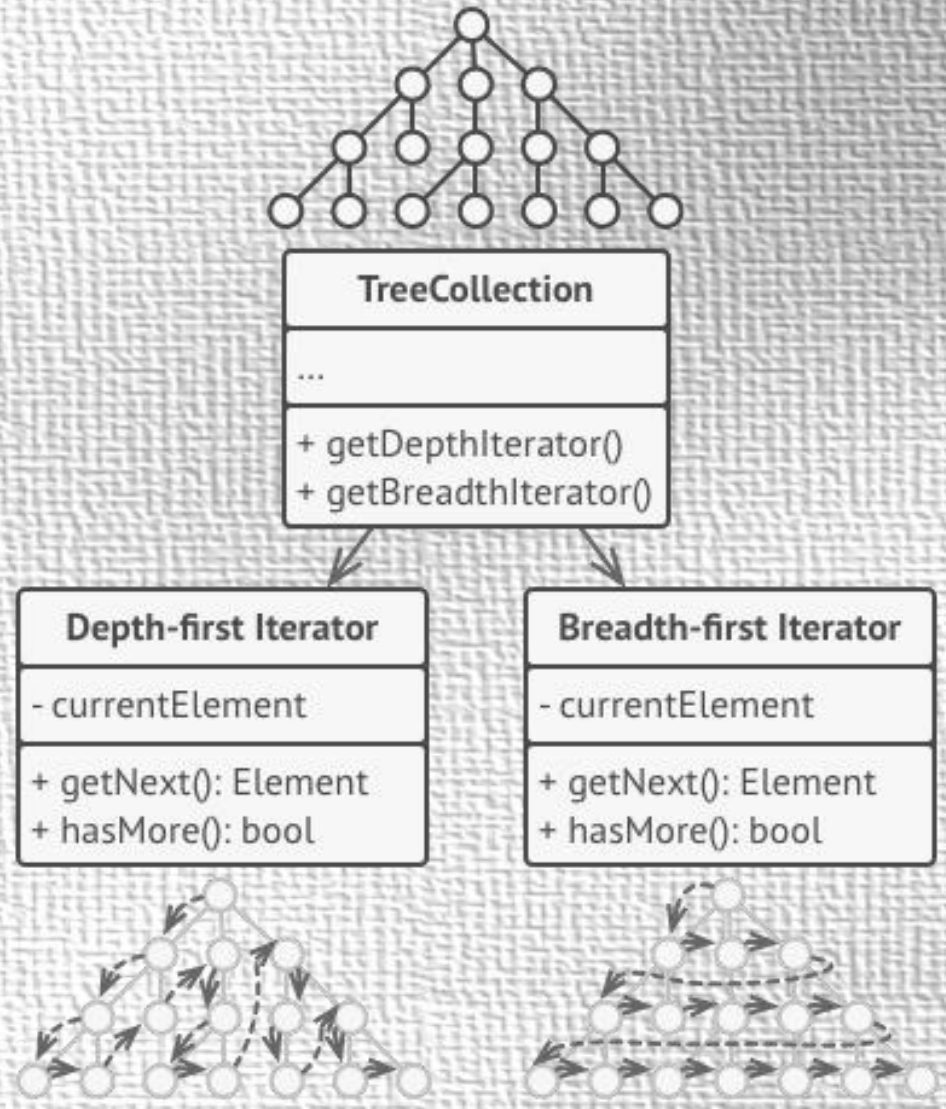




Добавляя всё новые алгоритмы в код коллекции, вы понемногу размываете её основную задачу, которая заключается в эффективном хранении данных. Некоторые алгоритмы могут быть и вовсе слишком «заточены» под определённое приложение и смотреться дико в общем классе коллекции.

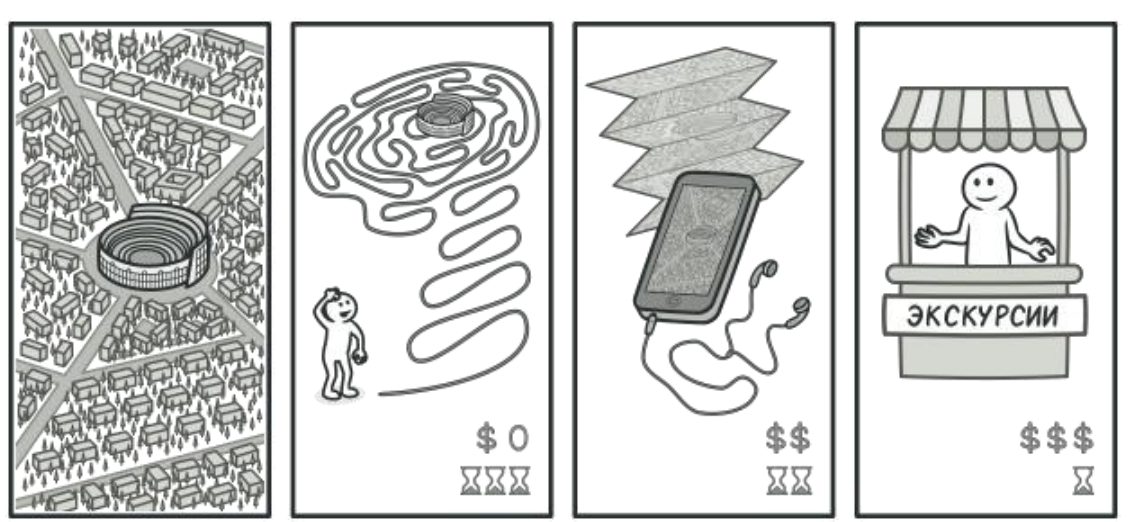
Решение

Идея паттерна Итератор состоит в том, чтобы вынести поведение обхода коллекции из самой коллекции в отдельный класс.



Аналогия из жизни

Вы планируете полететь в Рим и обойти все достопримечательности за пару дней. Но приехав, вы можете долго петлять узкими улочками, пытаясь найти Колизей.
Если у вас ограниченный бюджет – не беда. Вы можете воспользоваться виртуальным гидом, скачанным на телефон, который позволит отфильтровать только интересные вам точки. А можете плюнуть и нанять локального гида, который хоть и обойдётся в копеечку, но знает город как свои пять пальцев, и сможет посвятить вас во все городские легенды.

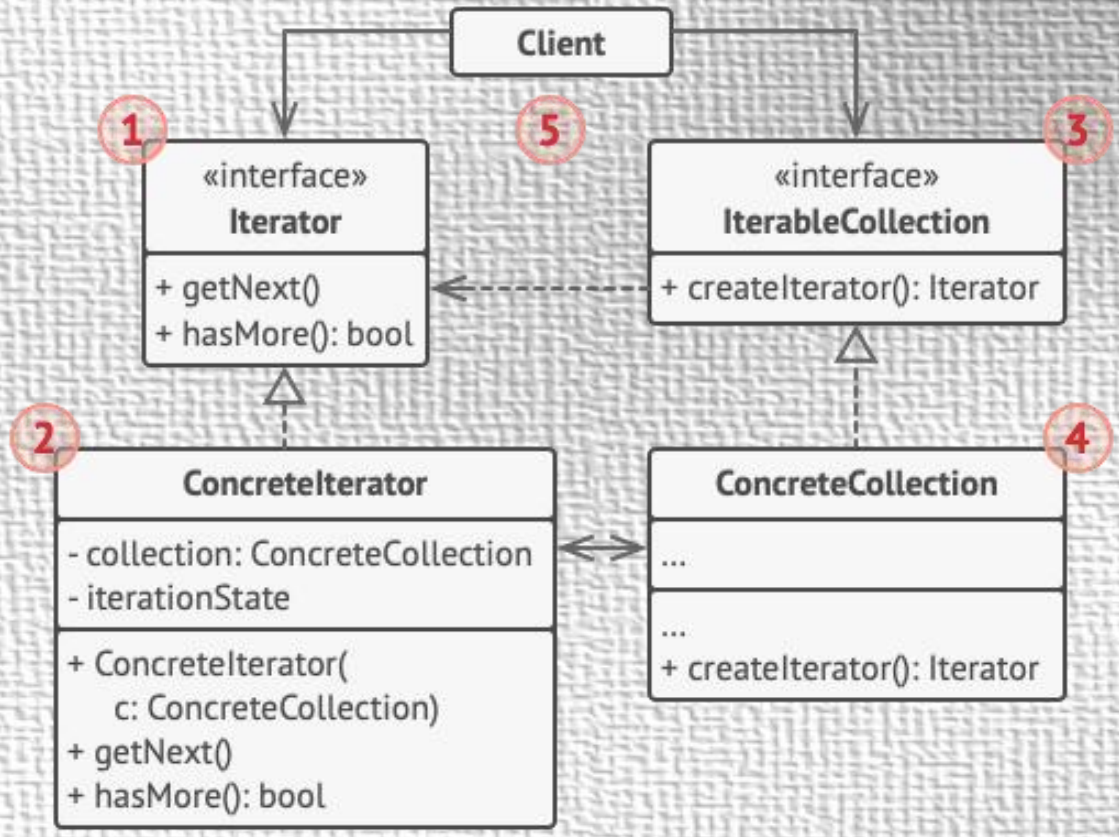


Таким образом, Рим выступает коллекцией достопримечательностей, а ваш мозг, навигатор или гид – итератором по коллекции. Вы, как клиентский код, можете выбрать один из итераторов, отталкиваясь от решаемой задачи и доступных ресурсов.

Структура

❶. Итератор описывает интерфейс для доступа и обхода элементов коллекции.

❷. Конкретный итератор реализует алгоритм обхода какой-то конкретной коллекции. Объект итератора должен сам отслеживать текущую позицию при обходе коллекции, чтобы отдельные итераторы могли обходить одну и ту же коллекцию независимо.



❸. Коллекция описывает интерфейс получения итератора из коллекции. Как мы уже говорили, коллекции не всегда являются списком. Это может быть и база данных, и удалённое API, и даже дерево *Компоновщика*. Поэтому сама коллекция может создавать итераторы, так как она знает, какие именно итераторы способны с ней работать.

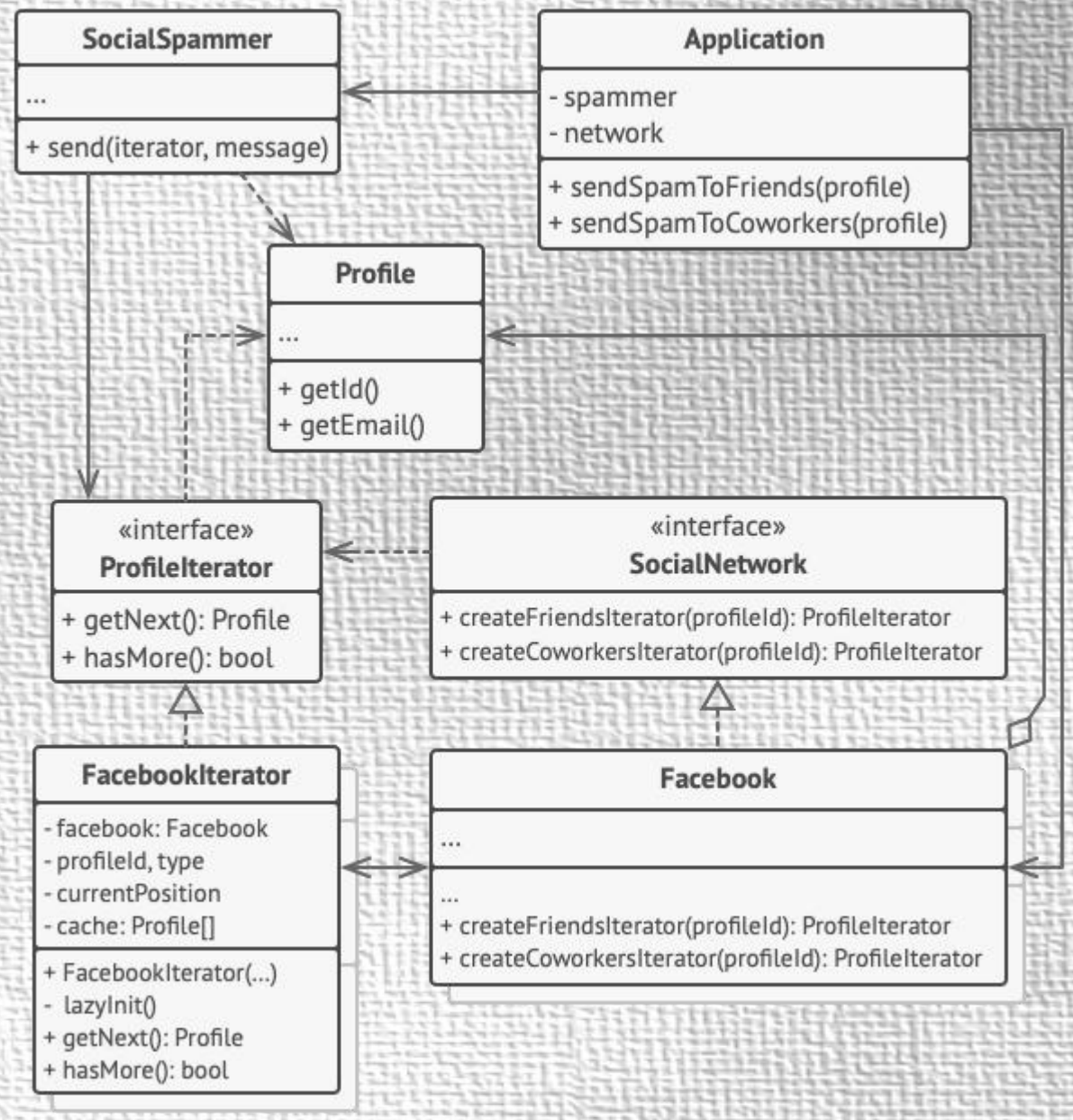
❹. Конкретная коллекция возвращает новый экземпляр определённого конкретного итератора, связав его с текущим объектом коллекции. Обратите внимание, что сигнатура метода возвращает интерфейс итератора. Это позволяет клиенту не зависеть от конкретных классов итераторов.

❺. Клиент работает со всеми объектами через интерфейсы коллекции и итератора. Так клиентский код не зависит от конкретных классов, что позволяет применять различные итераторы, не изменяя существующий код программы.

В общем случае клиенты не создают объекты итераторов, а получают их из коллекций. Тем не менее, если клиенту требуется специальный итератор, он всегда может создать его самостоятельно.

Псевдокод

В этом примере паттерн **Итератор** используется для реализации обхода нестандартной коллекции, которая инкапсулирует доступ к социальному графу *Facebook*. Коллекция предоставляет несколько итераторов, которые могут по-разному обходить профили людей.



Так, итератор друзей перебирает всех друзей профиля, а итератор коллег – фильтрует друзей по принадлежности к компании профиля. Все итераторы реализуют общий интерфейс, который позволяет клиентам работать с профилями, не вникая в детали работы с социальной сетью (например, в авторизацию, отправку REST-запросов и т. д.)

Кроме того, *Итератор* избавляет код от привязки к конкретным классам коллекций. Это позволяет добавить поддержку другого вида коллекций (например, LinkedIn), не меняя клиентский код, который работает с итераторами и коллекциями.

Применимость

Когда у вас есть сложная структура данных, и вы хотите скрыть от клиента детали её реализации (из-за сложности или вопросов безопасности).

Итератор предоставляет клиенту всего несколько простых методов перебора элементов коллекции. Это не только упрощает доступ к коллекции, но и защищает её данные от неосторожных или злоумышленных действий.

Когда вам нужно иметь несколько вариантов обхода одной и той же структуры данных.

Нетривиальные алгоритмы обхода структуры данных могут иметь довольно объёмный код. Этот код будет захламлять всё вокруг – будь то сам класс коллекции или часть бизнес-логики программы. Применив итератор, вы можете выделить код обхода структуры данных в собственный класс, упростив поддержку остального кода.

Когда вам хочется иметь единый интерфейс обхода различных структур данных.

Итератор позволяет вынести реализации различных вариантов обхода в подклассы. Это позволит легко взаимозаменять объекты итераторов, в зависимости от того, с какой структурой данных приходится работать.

Шаги реализации

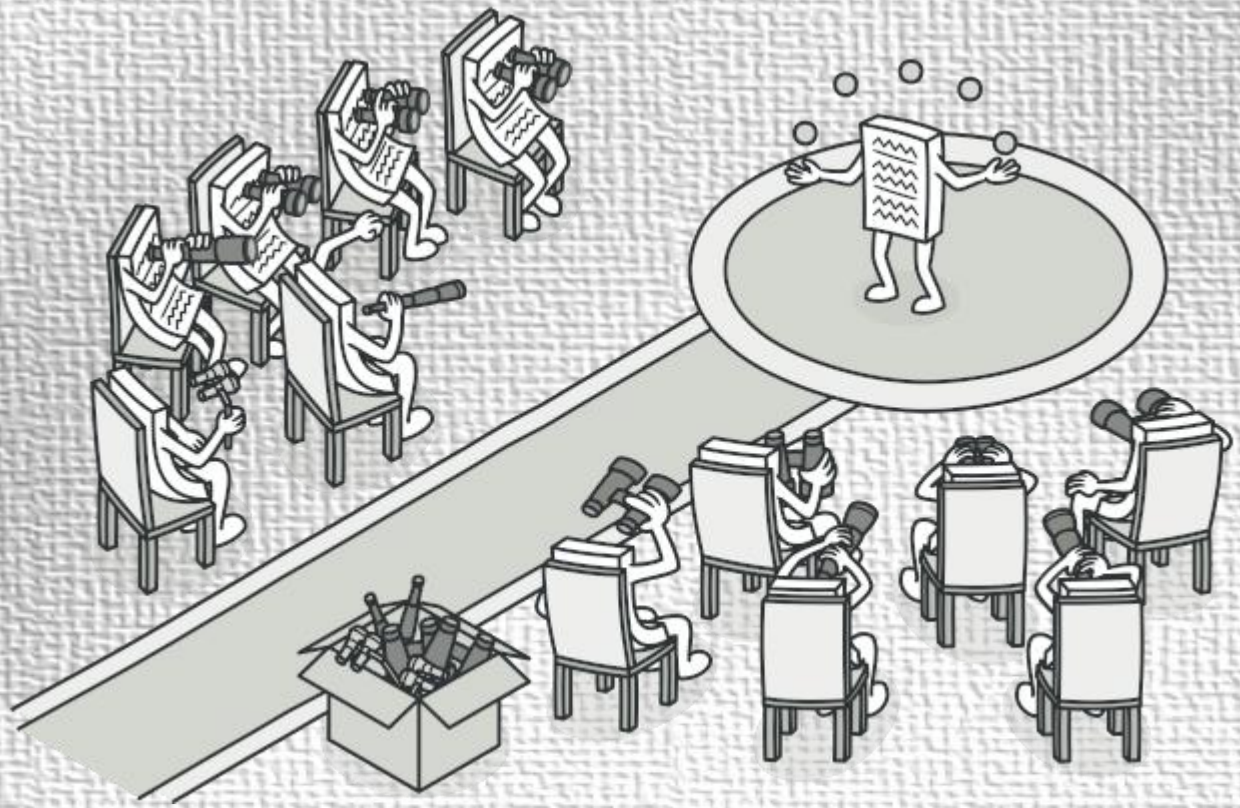
1. **С**оздайте общий интерфейс итераторов. Обязательный минимум – это операция получения следующего элемента коллекции. Но для удобства можно предусмотреть и другое. Например, методы для получения предыдущего элемента, текущей позиции, проверки окончания обхода и прочие.
2. **С**оздайте интерфейс коллекции и опишите в нём метод получения итератора. Важно, чтобы сигнатура метода возвращала общий интерфейс итераторов, а не один из конкретных итераторов.
3. **С**оздайте классы конкретных итераторов для тех коллекций, которые нужно обходить с помощью паттерна. Итератор должен быть привязан только к одному объекту коллекции. Обычно эта связь устанавливается через конструктор.

4. **Р**еализуйте методы получения итератора в конкретных классах коллекций. Они должны создавать новый итератор того класса, который способен работать с данным типом коллекции. Коллекция должна передавать ссылку на собственный объект в конструктор итератора.
5. **В** клиентском коде и в классах коллекций не должно остаться кода обхода элементов. Клиент должен получать новый итератор из объекта коллекции каждый раз, когда ему нужно перебрать её элементы.

Наблюдатель
(Observer)
Поведенческий паттерн

Суть паттерна

Наблюдатель – это поведенческий паттерн проектирования, который создаёт механизм подписки, позволяющий одним объектам следить и реагировать на события, происходящие в других объектах.

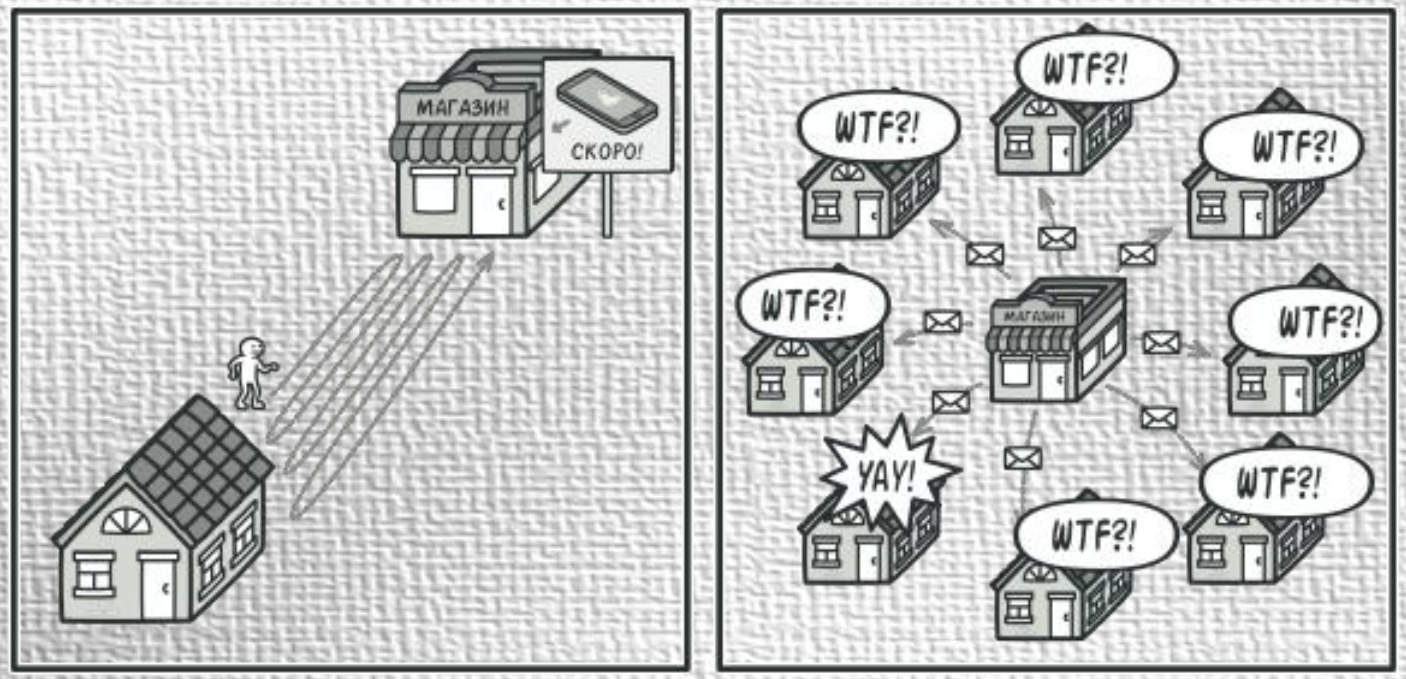


Проблема

Представьте, что вы имеете два объекта: Покупатель и Магазин. В магазин вот-вот должны завезти новый товар, который интересен покупателю.

Покупатель может каждый день ходить в магазин, чтобы проверить наличие товара. Но при этом он будет злиться, без толку тратя своё драгоценное время.

Постоянное посещение магазина или спам?



С другой стороны, магазин может разослать спам каждому своему покупателю. Многих это расстроит, так как товар специфический, и не всем он нужен.
Получается конфликт: либо покупатель тратит время на периодические проверки, либо магазин тратит ресурсы на бесполезные оповещения.

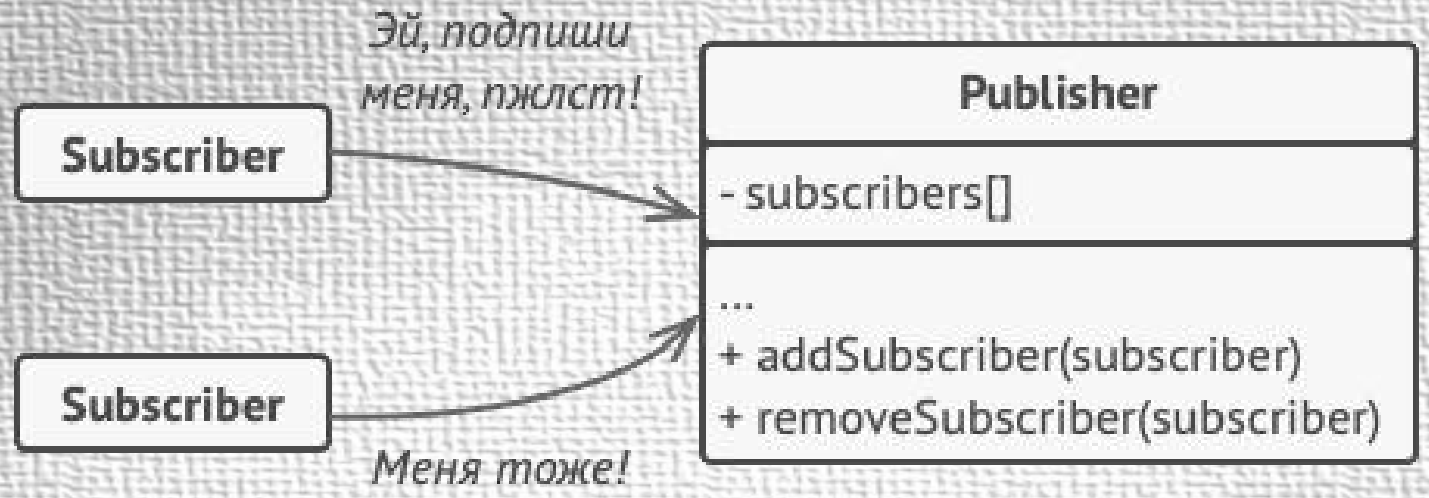
Решение

Давайте называть **Издателями** те объекты, которые содержат важное или интересное для других состояние. Остальные объекты, которые хотят отслеживать изменения этого состояния, назовём **Подписчиками**.

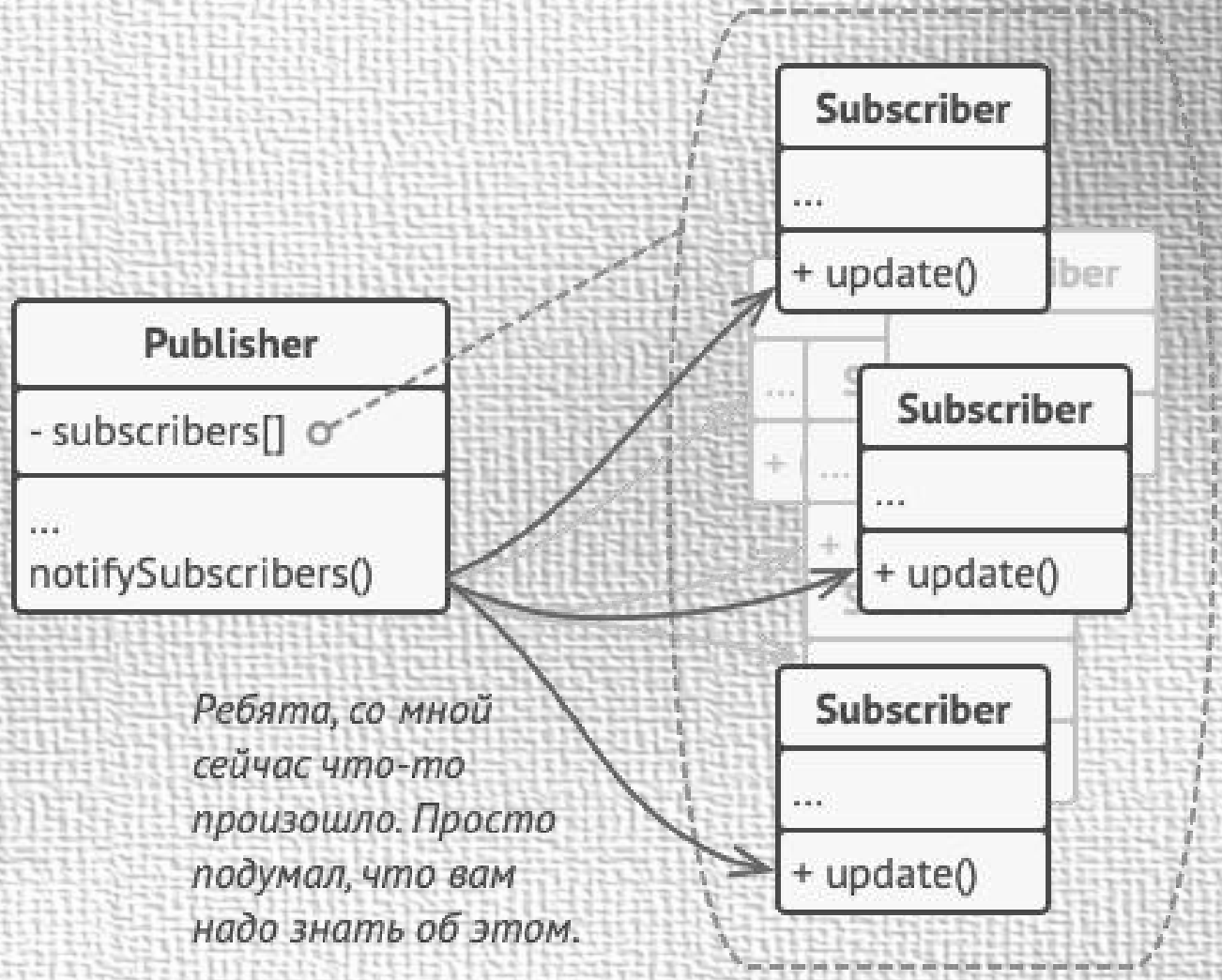
Паттерн **Наблюдатель** предлагает хранить внутри объекта издателя список ссылок на объекты подписчиков, причём издатель не должен вести список подписки самостоятельно. Он предоставит методы, с помощью которых подписчики самостоятельно могли бы добавлять или убирать себя из списка.

Теперь самое интересное. Когда в издателе будет происходить важное событие, он будет проходиться по списку подписчиков и оповещать их об этом, вызывая определённый метод объектов-подписчиков.

Издателю безразлично, какой класс будет иметь тот или иной подписчик, так как все они должны следовать общему интерфейсу и иметь единый метод оповещения.



Увидев, как складно всё работает, вы можете выделить общий интерфейс, описывающий методы подписки и отписки, и для всех издателей. После этого подписчики смогут работать с разными типами издателей, а также получать оповещения от них через один и тот же метод.



Оповещения о событиях

Аналогия из жизни

После того как вы оформили подписку на газету или журнал, вам больше не нужно ездить в супермаркет и проверять, не вышел ли очередной номер. Вместо этого издательство будет присылать новые номера по почте прямо к вам домой сразу после их выхода.

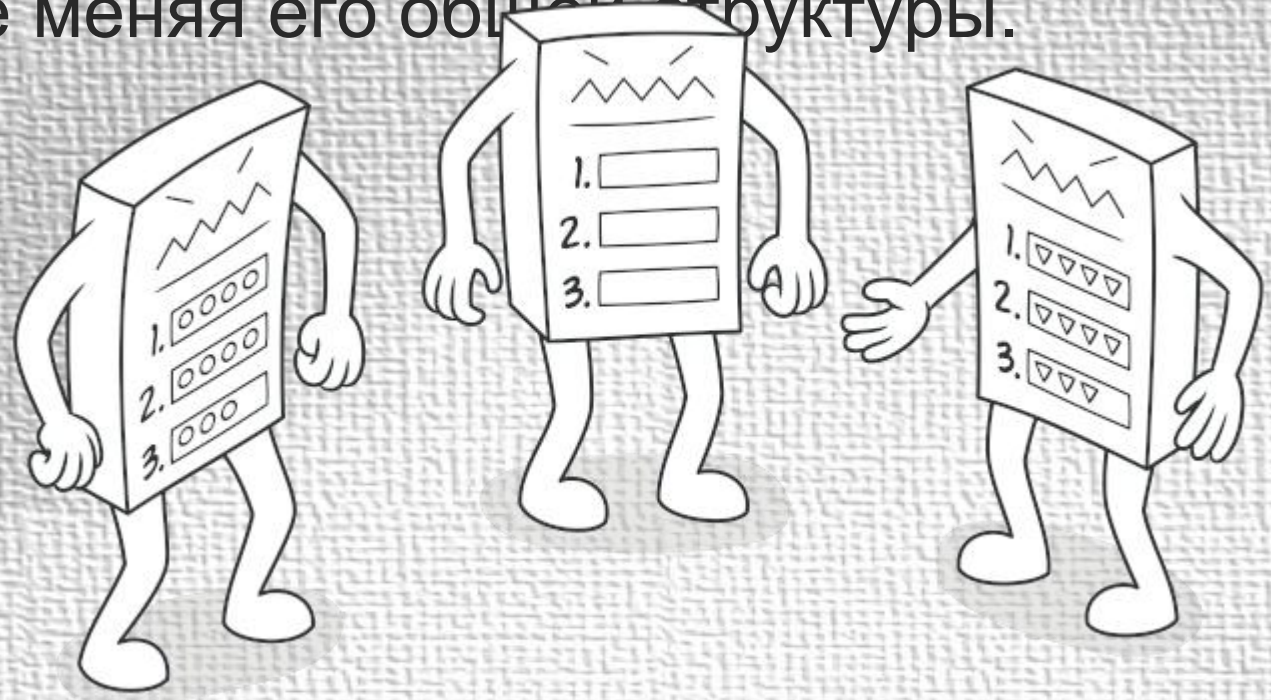
Издательство ведёт список подписчиков и знает, кому какой журнал высылать. Вы можете в любой момент отказаться от подписки, и журнал перестанет вам приходить.

Шаблонный метод

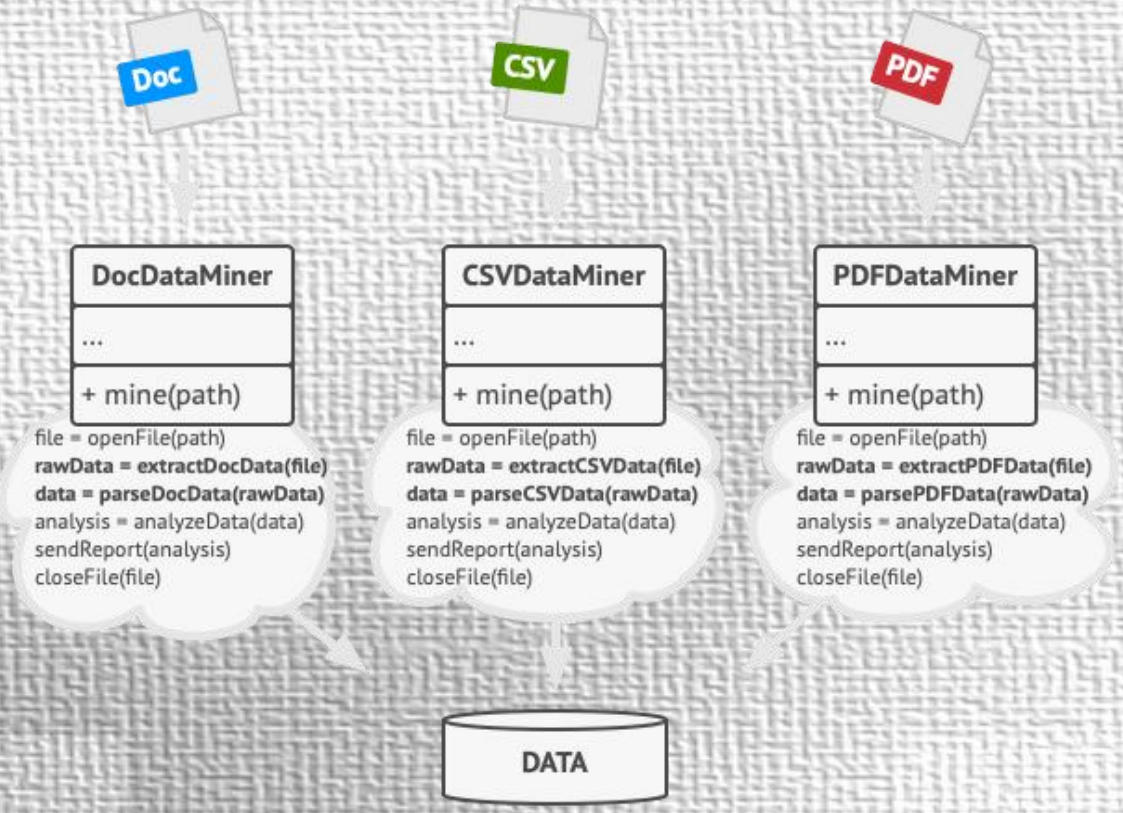
Поведенческий паттерн

Суть паттерна

Шаблонный метод – это поведенческий паттерн проектирования, который определяет скелет алгоритма, перекладывая ответственность за некоторые его шаги на подклассы. Паттерн позволяет подклассам переопределять шаги алгоритма, не меняя его общей структуры.



Проблема



Вы пишете программу для дата-майнинга в офисных документах. Пользователи будут загружать в неё документы в разных форматах (PDF, DOC, CSV), а программа должна извлекать из них полезную информацию.

В первой версии вы ограничились только обработкой DOC-файлов. В следующей версии добавили поддержку CSV. А через месяц прикрутили работу с PDF-документами.

В какой-то момент вы заметили, что код всех трёх классов обработки документов хоть и отличается в части работы с файлами, но содержит довольно много общего в части самого извлечения данных. Было бы здорово избавится от повторной реализации алгоритма извлечения данных в каждом из классов.

К тому же остальной код, работающий с объектами этих классов, наполнен условиями, проверяющими тип обработчика перед началом работы. Весь этот код можно упростить, если слить все три класса воедино либо свести их к общему интерфейсу.

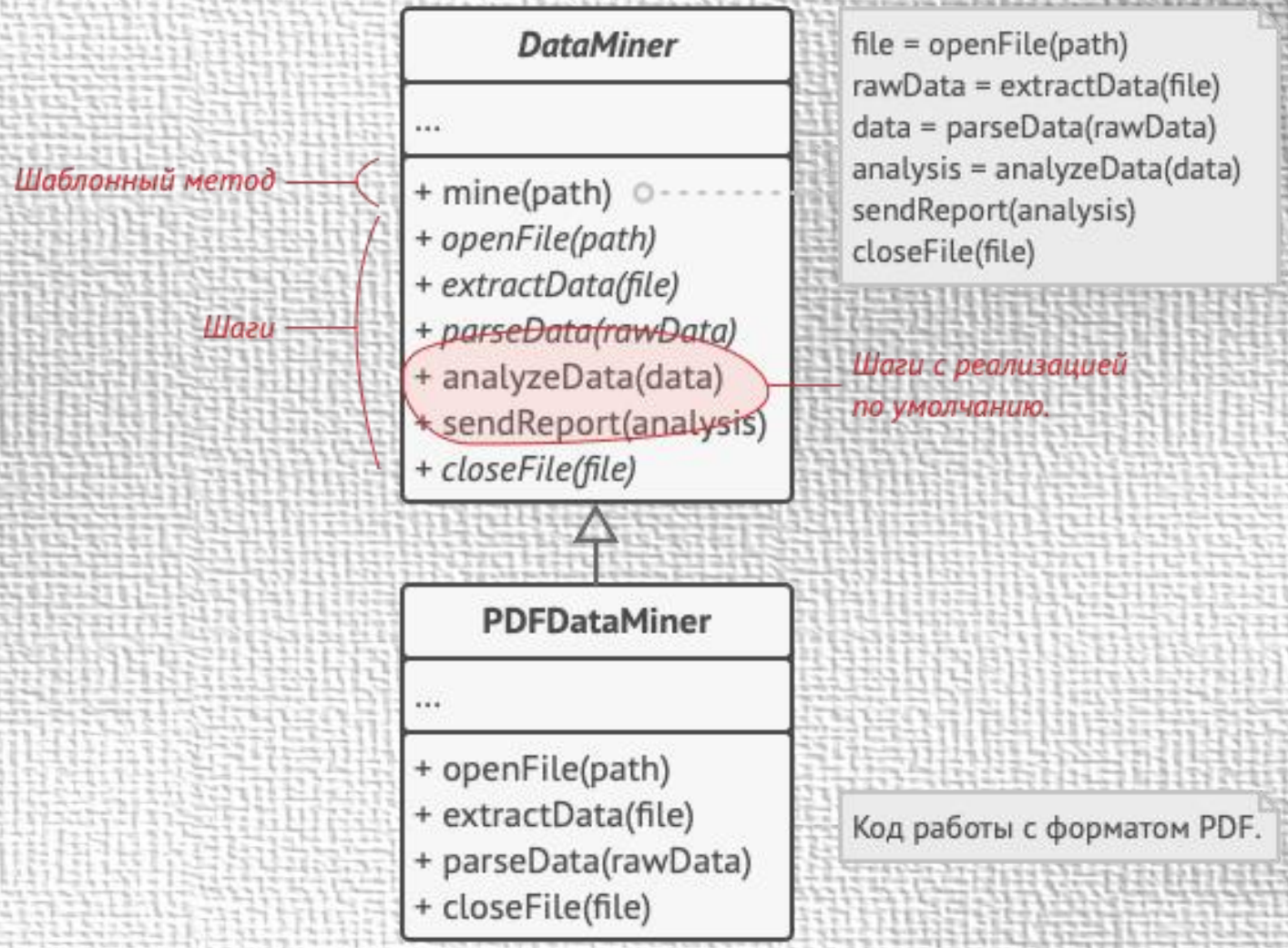
Решение

Паттерн Шаблонный метод предлагает разбить алгоритм на последовательность шагов, описать эти шаги в отдельных методах и вызывать их в одном шаблонном методе друг за другом.

Это позволит подклассам переопределять некоторые шаги алгоритма, оставляя без изменений его структуру и остальные шаги, которые для этого подкласса не так важны.

В нашем примере с дата-майнингом мы можем создать общий базовый класс для всех трёх алгоритмов. Этот класс будет состоять из шаблонного метода, который последовательно вызывает шаги разбора документов.

Шаблонный метод разбивает алгоритм на шаги, позволяя подклассам переопределить некоторые из них



Для начала шаги шаблонного метода можно сделать абстрактными. Из-за этого все подклассы должны будут реализовать каждый из шагов по-своему. В нашем случае все подклассы и так содержат реализацию каждого из шагов, поэтому ничего дополнительно делать не нужно.

По-настоящему важным является следующий этап. Теперь мы можем определить общее для всех классов поведение и вынести его в суперкласс. В нашем примере шаги открытия, считывания и закрытия могут отличаться для разных типов документов, поэтому останутся абстрактными. А вот одинаковый для всех типов документов код обработки данных переедет в базовый класс.

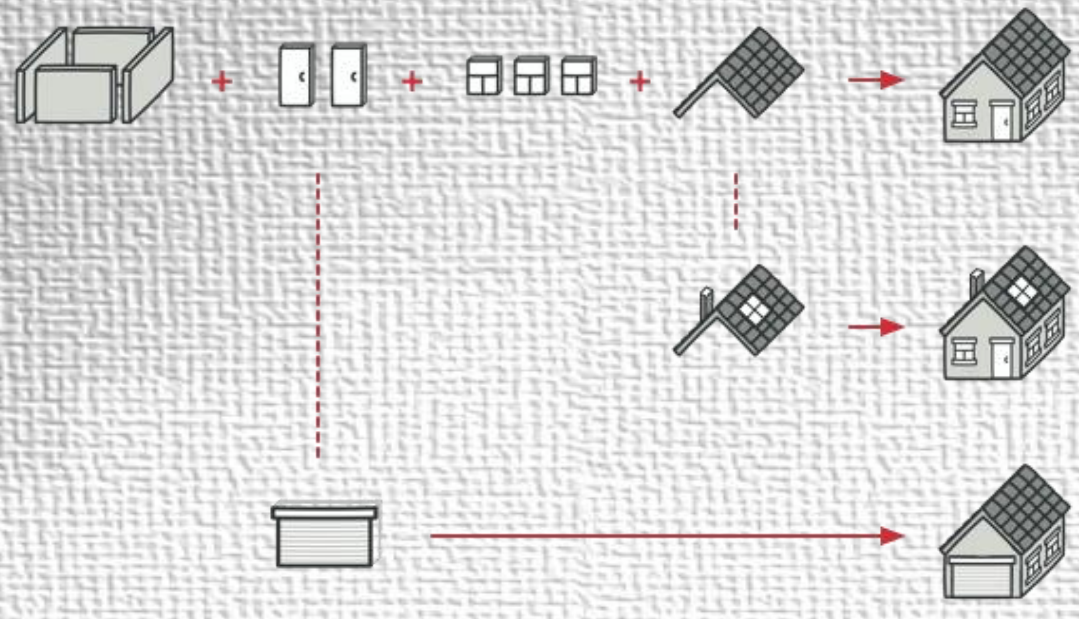
Как видите, у нас получилось два вида шагов: абстрактные, которые каждый подкласс обязательно должен реализовать, а также шаги с реализацией по умолчанию, которые можно переопределять в подклассах, но не обязательно.

Но есть и третий тип шагов – хуки: их не обязательно переопределять, но они не содержат никакого кода, выглядя как обычные методы. Шаблонный метод останется рабочим, даже если ни один подкласс не переопределит такой хук. Однако, хук даёт подклассам дополнительные точки «вклинивания» в шаблонный метод.

Аналогия из жизни

Строители используют подход, похожий на шаблонный метод при строительстве типовых домов. У них есть основной архитектурный проект, в котором расписаны шаги строительства: заливка фундамента, постройка стен, перекрытие крыши, установка окон и так далее.

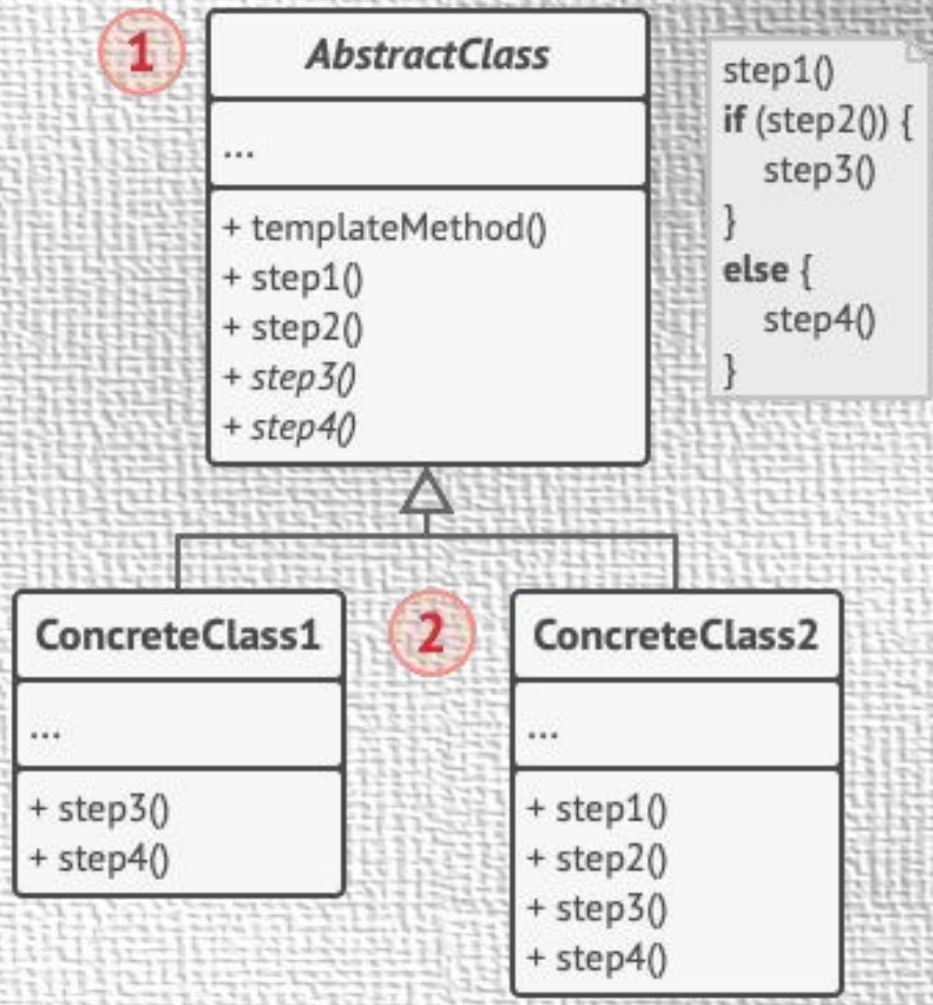
Но, несмотря на стандартизацию каждого этапа, строители могут вносить небольшие изменения на любом из этапов, чтобы сделать дом чуточку непохожим на другие.



Структура

❶. Абстрактный класс определяет шаги алгоритма и содержит шаблонный метод, состоящий из вызовов этих шагов. Шаги могут быть как абстрактными, так и содержать реализацию по умолчанию.

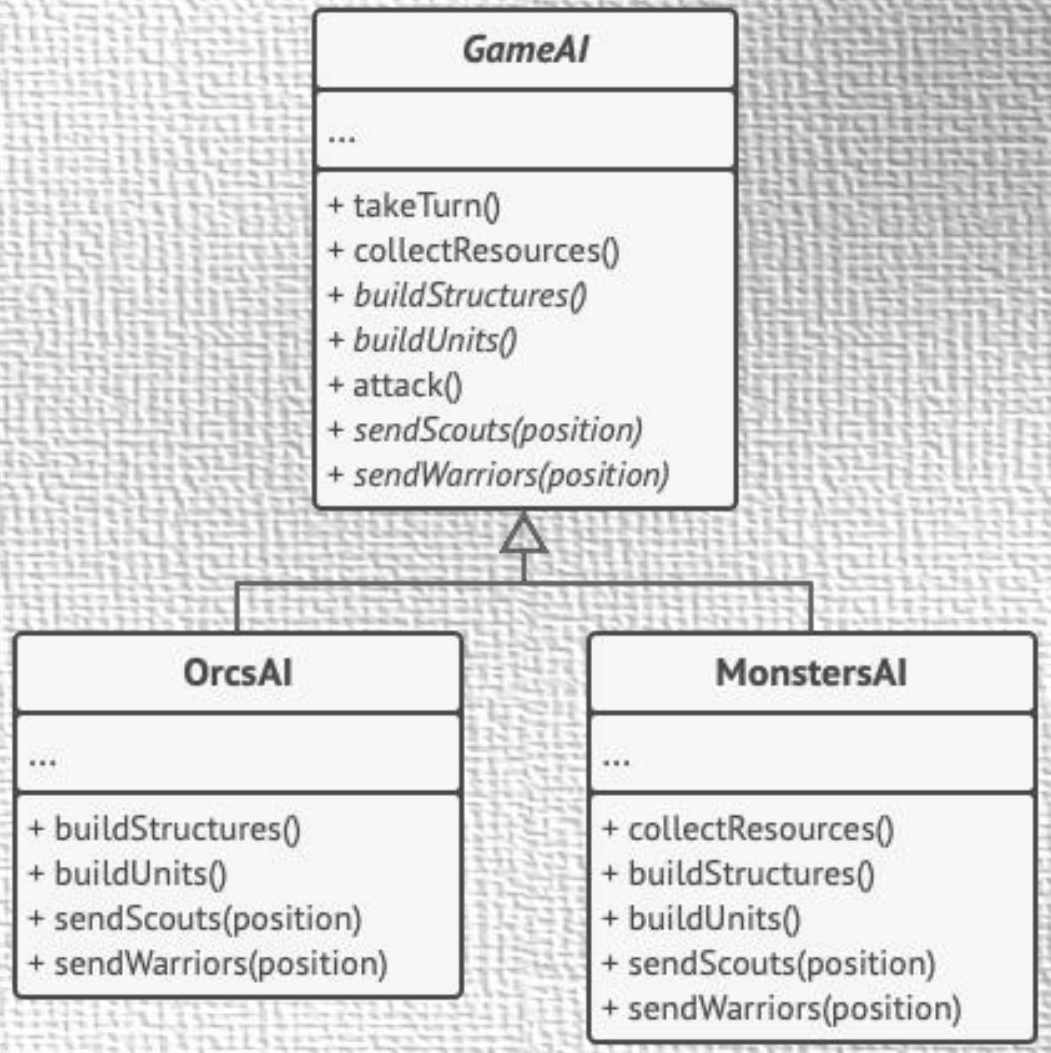
❷. Конкретный класс переопределяет некоторые (или все) шаги алгоритма. Конкретные классы не переопределяют сам шаблонный метод.



Псевдокод

В этом примере Шаблонный метод используется как заготовка для стандартного искусственного интеллекта в простой игре-стратегии. Для введения в игру новой расы достаточно создать подкласс и реализовать в нём недостающие методы.

Все расы игры будут содержать примерно такие же типы юнитов и строений, поэтому структура ИИ будет одинаковой. Но разные расы могут по-разному реализовать эти шаги. Так, например, орки будут



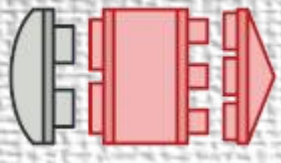
СТРУКТУРНЫЕ ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ

Отвечают за построение удобных в поддержке иерархий классов.

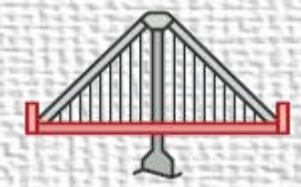
Структурные шаблоны проектирования

Структурные шаблоны - шаблоны проектирования, в которых рассматривается вопрос о том, как из классов и объектов образуются более крупные структуры.

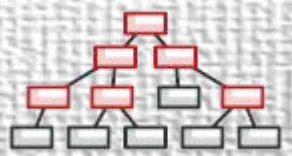
Структурные шаблоны уровня класса используют наследование для составления композиций из интерфейсов и реализаций. Простой пример - использование множественного наследования для объединения нескольких классов в один. В результате получается класс, обладающий свойствами всех своих родителей. Особенно полезен этот шаблон, когда нужно организовать совместную работу нескольких независимо разработанных библиотек.



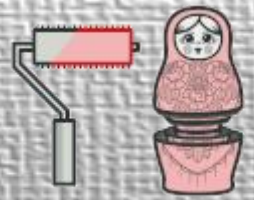
Адаптер. Позволяет объектам с несовместимыми интерфейсами работать вместе.



Мост. Разделяет один или несколько классов на две отдельные иерархии — абстракцию и реализацию, позволяя изменять их независимо друг от друга.

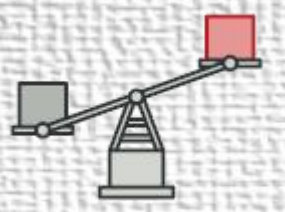


Компоновщик. Позволяет сгруппировать множество объектов в древовидную структуру, а затем работать с ней так, как будто это единичный объект.

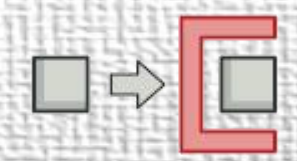


Декоратор. Позволяет динамически добавлять объектам новую функциональность, оборачивая их в полезные «обёртки».

Фасад. Предоставляет простой интерфейс к сложной системе классов, библиотеке или фреймворку.



Легковес. Позволяет вместить большее количество объектов в отведённую оперативную память. Легковес экономит память, разделяя общее состояние объектов между собой, вместо хранения одинаковых данных в каждом объекте.



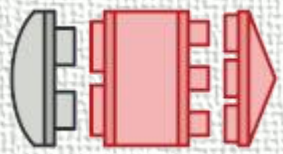
Заместитель. Позволяет подставлять вместо реальных объектов специальные объекты-заменители. Эти объекты перехватывают вызовы к оригинальному объекту, позволяя сделать что-то до или после передачи вызова оригиналу.

Структурные паттерны помогают решать задачи по совмещению и сочетанию сущностей. Они заботятся о том, как сущности могут использовать друг друга.

✌ Простыми словами — отвечают на вопрос «Как составить программный компонент?»

Среди структурных паттернов можем выделить:

- **Адаптер**
- **Декоратор**
- **Фасад**
- **Заместитель (Прокси)**

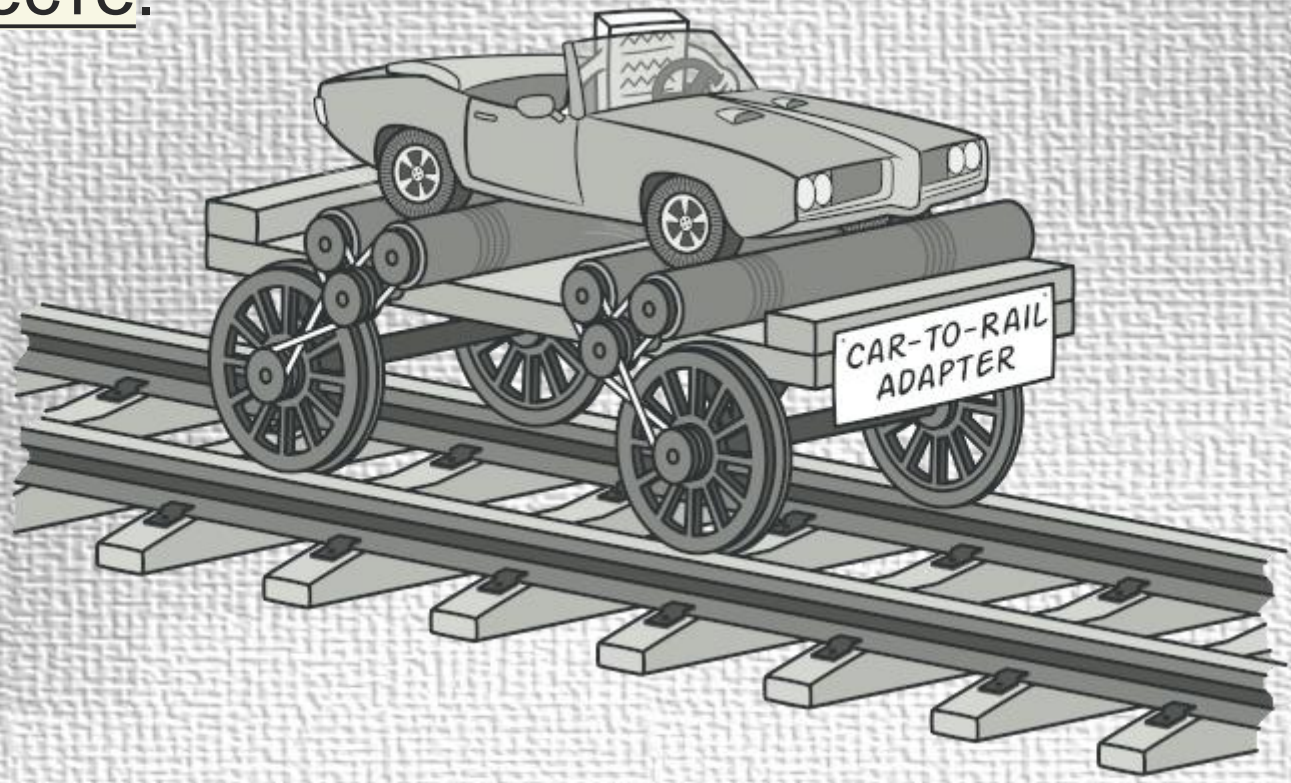


Адаптер
(Adapter)

Структурный паттерн

Суть паттерна

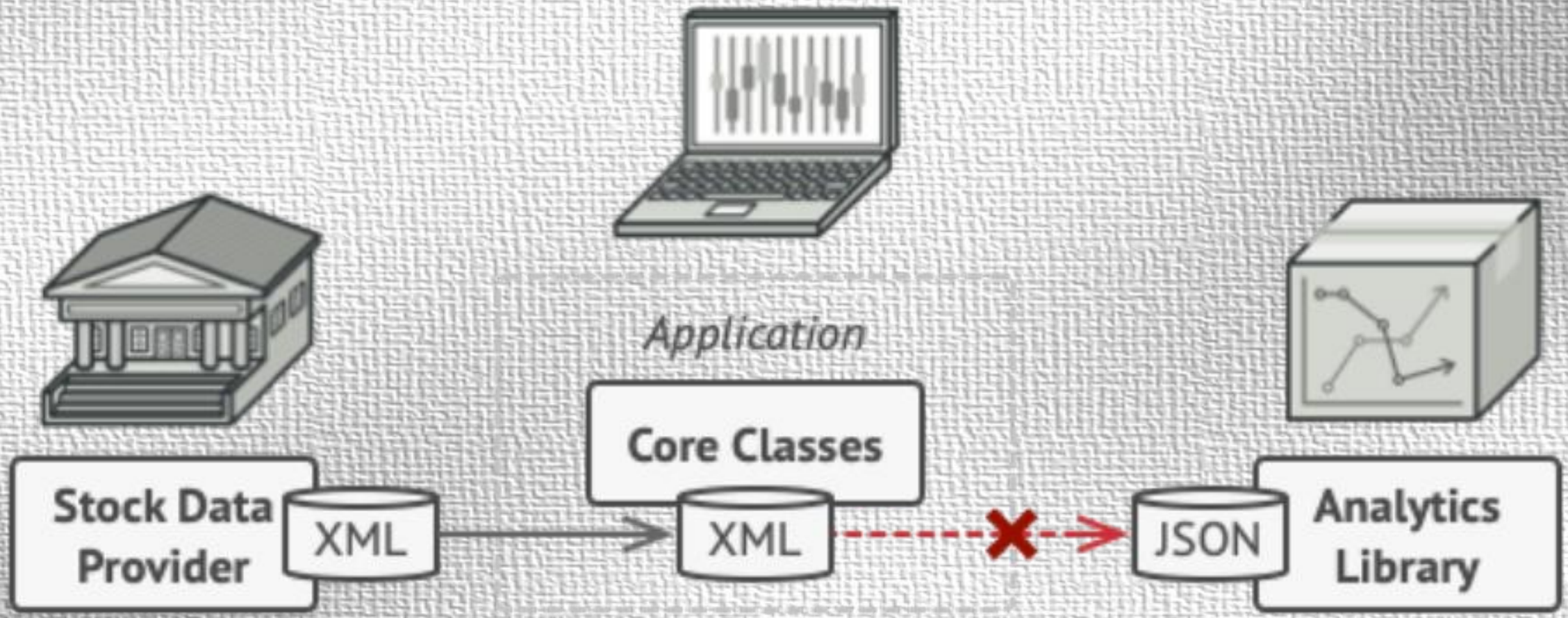
Адаптер – это структурный паттерн проектирования, который позволяет объектам с несовместимыми интерфейсами работать вместе.



Проблема

Представьте, что вы делаете приложение для торговли на бирже. Ваше приложение скачивает биржевые котировки из нескольких источников в XML, а затем рисует красивые графики.

В какой-то момент вы решаете улучшить приложение, применив стороннюю библиотеку аналитики. Но вот беда – библиотека поддерживает только формат данных JSON, несовместимый с вашим приложением.



Вы смогли бы переписать библиотеку, чтобы та поддерживала формат XML. Но, во-первых, это может нарушить работу существующего кода, который уже зависит от библиотеки. А во-вторых, у вас может просто не быть доступа к её исходному коду.

Решение

Вы можете создать адаптер. Это объект-переводчик, который трансформирует интерфейс или данные одного объекта в такой вид, чтобы он стал понятен другому объекту.

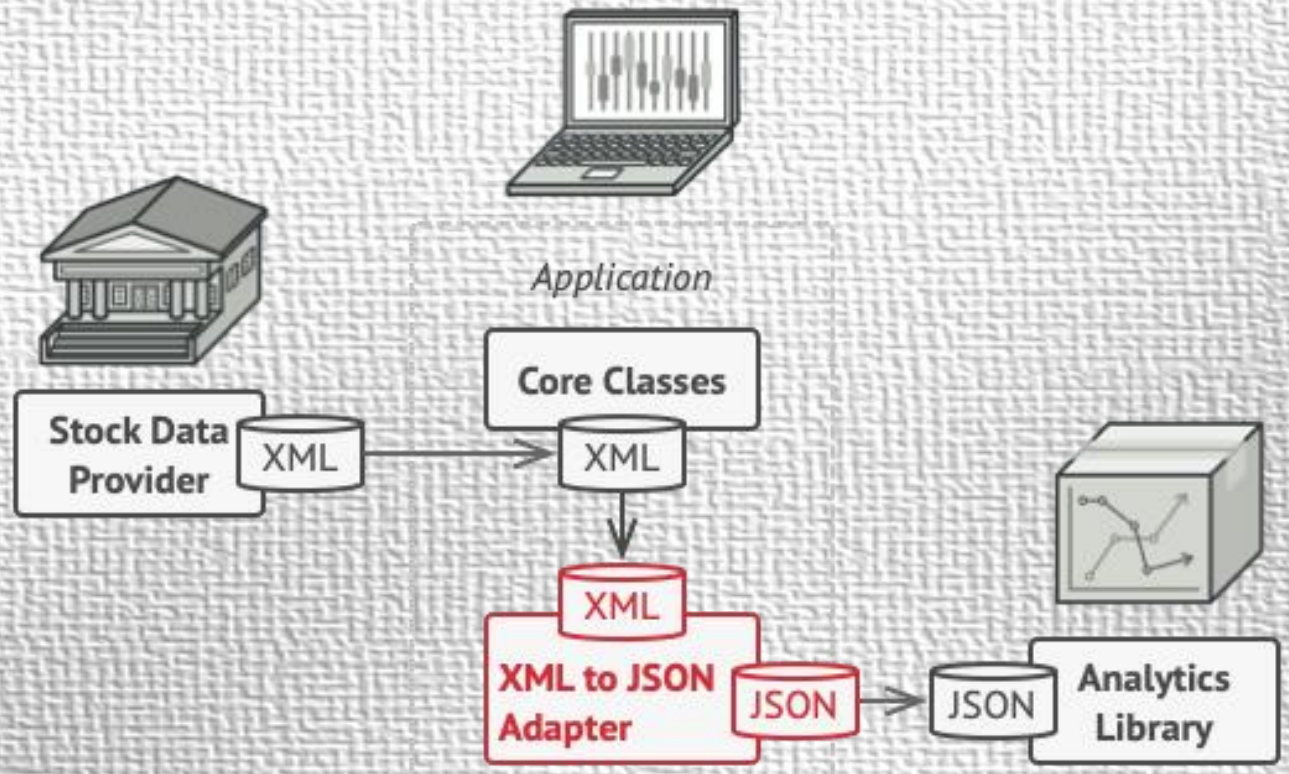
При этом адаптер оборачивает один из объектов так, что другой объект даже не знает о наличии первого. Например, вы можете обернуть объект, работающий в метрах, адаптером, который бы конвертировал данные в футы.

Адаптеры могут не только переводить данные из одного формата в другой, но и помогать объектам с разными интерфейсами работать сообща.

Это работает так:

1. Адаптер имеет интерфейс, который совместим с одним из объектов. Поэтому этот объект может свободно вызывать методы адаптера.
2. Адаптер получает эти вызовы и перенаправляет их второму объекту, но уже в том формате и последовательности, которые понятны второму объекту.

Иногда возможно создать даже двухсторонний адаптер, который работал бы в обе стороны.



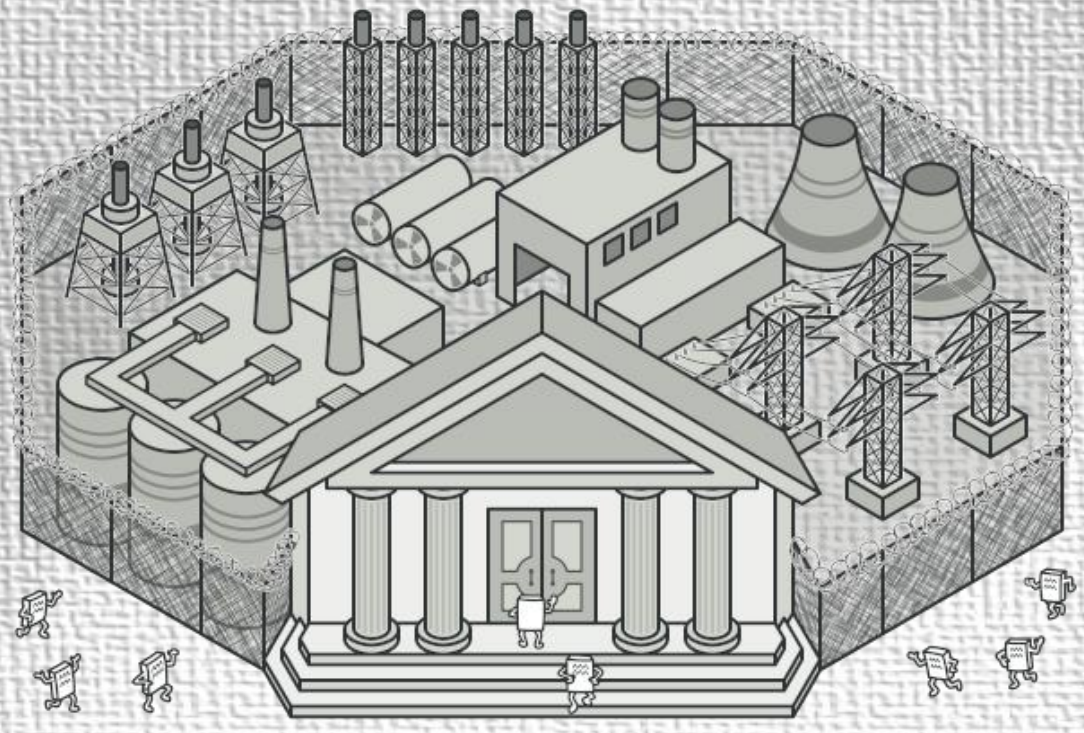
Таким образом, в приложении биржевых котировок вы могли бы создать класс XML_To_JSON_Adapter, который бы оборачивал объект того или иного класса библиотеки аналитики. Ваш код посылал бы адаптеру запросы в формате XML, а адаптер сначала транслировал входящие данные в формат JSON, а затем передавал бы их методам обёрнутого объекта аналитики.

Фасад
(Facade)

Структурный паттерн

Суть паттерна

Фасад – это структурный паттерн проектирования, который предоставляет простой интерфейс к сложной системе классов, библиотеке или фреймворку.



Проблема

Вашему коду приходится работать с большим количеством объектов некой сложной библиотеки или фреймворка. Вы должны самостоятельно инициализировать эти объекты, следить за правильным порядком зависимостей и так далее.

В результате бизнес-логика ваших классов тесно переплетается с деталями реализации сторонних классов. Такой код довольно сложно понимать и поддерживать.

Решение

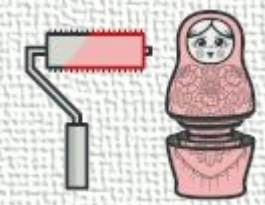
Фасад – это простой интерфейс для работы со сложной подсистемой, содержащей множество классов. Фасад может иметь урезанный интерфейс, не имеющий 100% функциональности, которой можно достичь, используя сложную подсистему напрямую. Но он предоставляет именно те фичи, которые нужны клиенту, и скрывает все остальные.

Фасад полезен, если вы используете какую-то сложную библиотеку со множеством подвижных частей, но вам нужна только часть её возможностей.

К примеру, программа, заливающая видео котиков в социальные сети, может использовать профессиональную библиотеку сжатия видео. Но всё, что нужно клиентскому коду этой программы – простой метод encode(filename, format). Создав класс с таким методом, вы реализуете свой первый фасад.

Аналогия из жизни

Когда вы звоните в магазин и делаете заказ по телефону, сотрудник службы поддержки является вашим фасадом ко всем службам и отделам магазина. Он предоставляет вам упрощённый интерфейс к системе создания заказа, платёжной системе и отделу доставки.

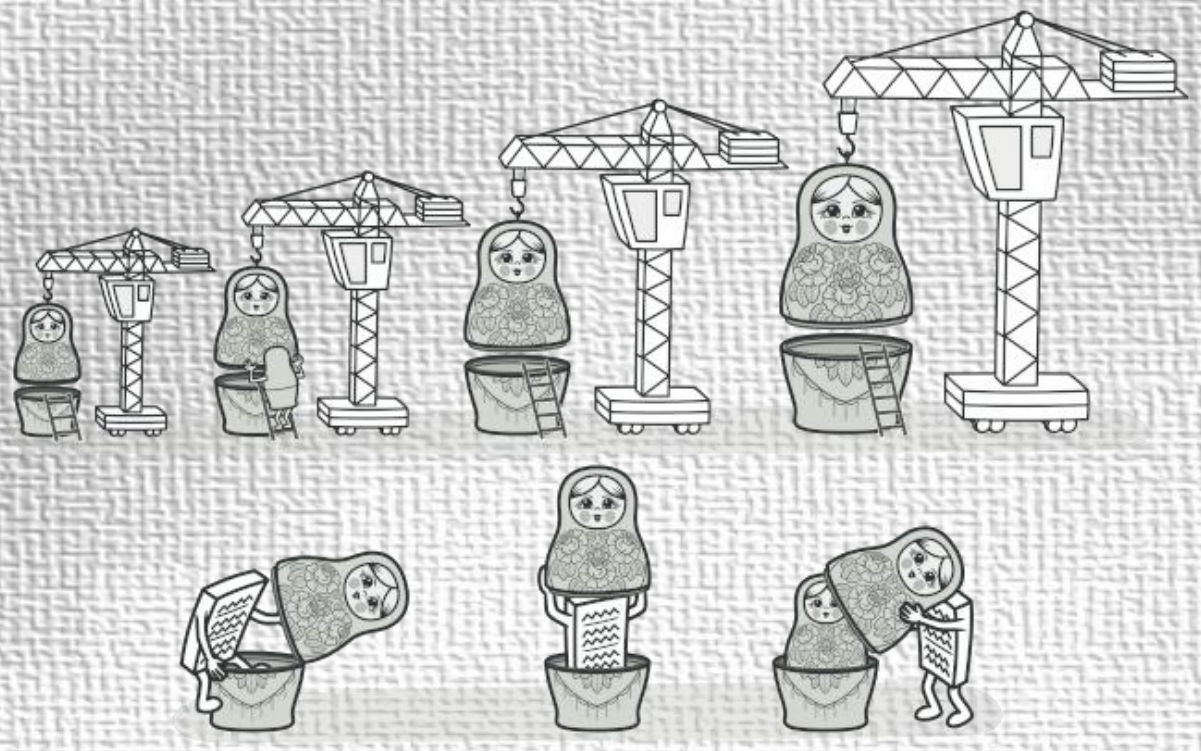


Декоратор
(Decorator)

Структурный паттерн

Суть паттерна

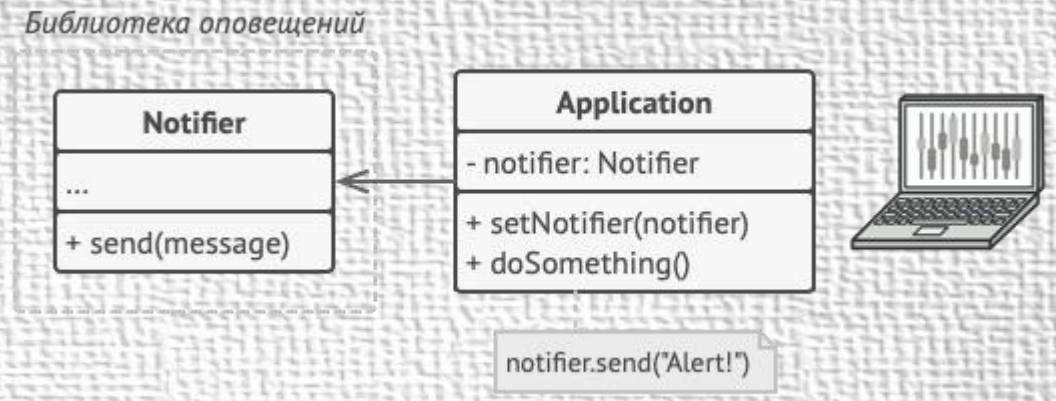
Декоратор — это структурный паттерн проектирования, который позволяет динамически добавлять объектам новую функциональность, оборачивая их в полезные «обёртки».



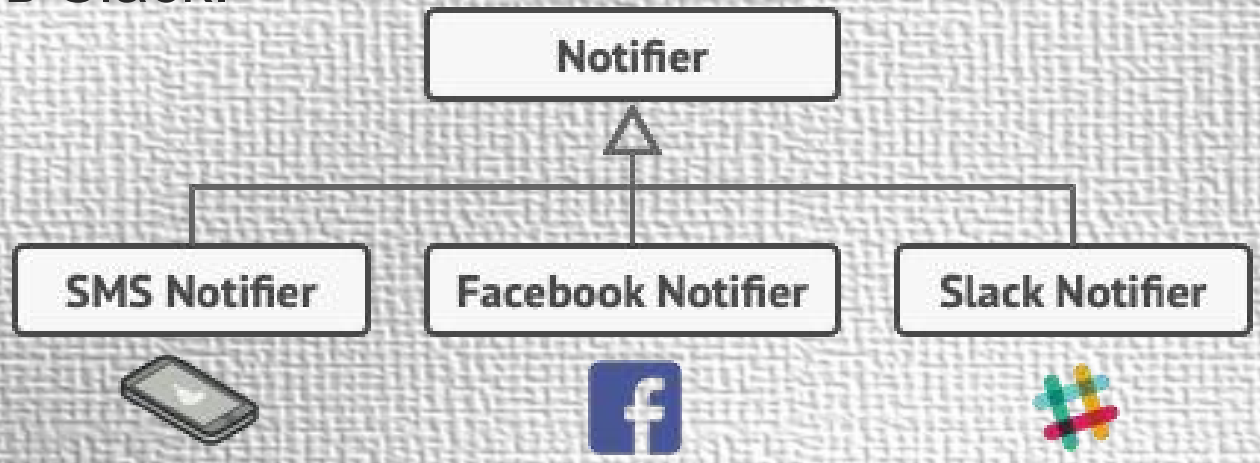
Проблема

Вы работаете над библиотекой оповещений, которую можно подключать к разнообразным программам, чтобы получать уведомления о важных событиях. Основой библиотеки является класс `Notifier` с методом `send`, который принимает на вход строку-сообщение и высылает её всем администраторам по электронной почте. Сторонняя программа должна создать и настроить этот объект, указав кому отправлять оповещения, а затем использовать его каждый раз, когда что-то случается.

Сторонние программы используют главный класс оповещений



В какой-то момент стало понятно, что одних email-оповещений пользователям мало. Некоторые из них хотели бы получать извещения о критических проблемах через SMS. Другие хотели бы получать их в виде сообщений Facebook. Корпоративные пользователи хотели бы видеть сообщения в Slack.

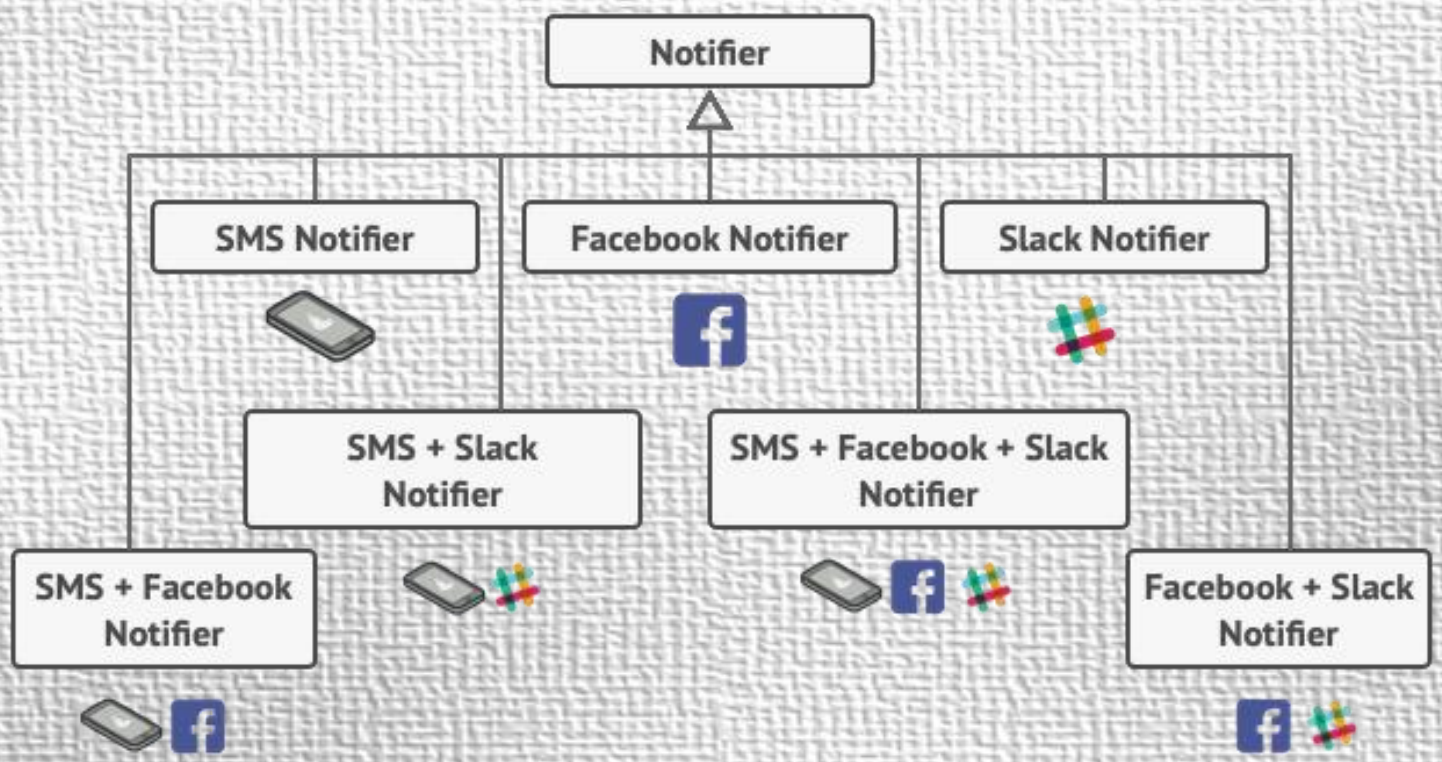


Каждый тип оповещения живёт в собственном подклассе

Сначала вы добавили каждый из этих типов оповещений в программу, унаследовав их от базового класса Notifier. Теперь пользователь выбирал один из типов оповещений, который и использовался в дальнейшем.

Но затем кто-то резонно спросил, почему нельзя выбрать несколько типов оповещений сразу? Ведь если вдруг в вашем доме начался пожар, вы бы хотели получить оповещения по всем каналам, не так ли?

Вы попытались реализовать все возможные комбинации подклассов оповещений. Но после того как вы добавили первый десяток классов, стало ясно, что такой подход невероятно раздувает код программы.



Комбинаторный взрыв подклассов при совмещении типов оповещений

Итак, нужен какой-то другой способ комбинирования поведения объектов, который не приводит к взрыву количества подклассов.

Решение

Наследование – это первое, что приходит в голову многим программистам, когда нужно расширить какое-то существующее поведение. Но механизм наследования имеет несколько досадных проблем.

- Он статичен. Вы не можете изменить поведение существующего объекта. Для этого вам надо создать новый объект, выбрав другой подкласс.
- Он не разрешает наследовать поведение нескольких классов одновременно. Из-за этого вам приходится создавать множество подклассов-комбинаций для получения совмещённого поведения.

Одним из способов обойти эти проблемы является замена наследования агрегацией либо композицией . **Это когда один объект содержит ссылку на другой и делегирует ему работу, вместо того чтобы самому наследовать его поведение. Как раз на этом принципе построен паттерн Декоратор**.

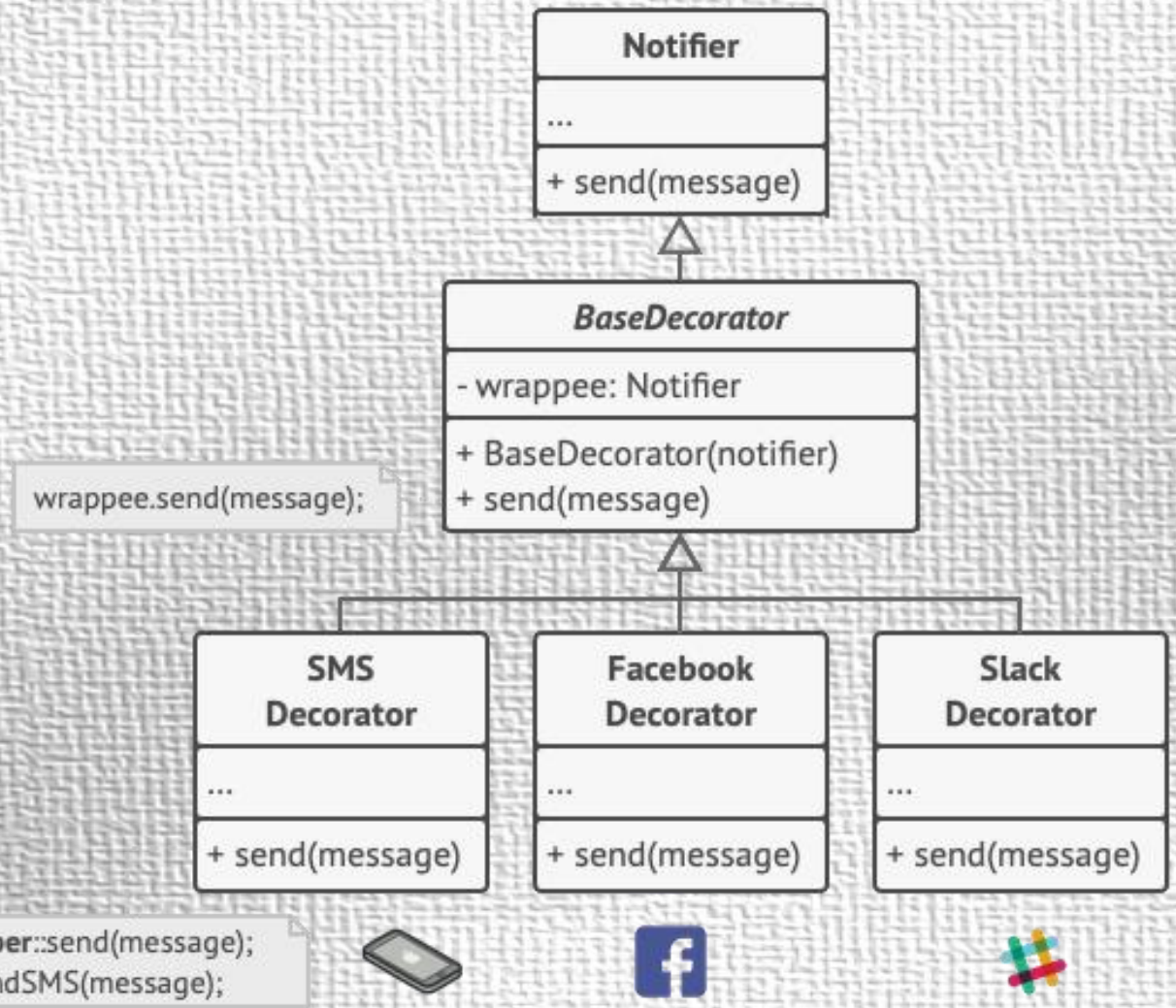
Наследование

Агрегация

Декоратор имеет альтернативное название – ***обёртка***. Оно более точно описывает суть паттерна: вы помещаете целевой объект в другой объект-обёртку, который запускает базовое поведение объекта, а затем добавляет к результату что-то своё.

Оба объекта имеют общий интерфейс, поэтому для пользователя нет никакой разницы, с каким объектом работать – чистым или обёрнутым. Вы можете использовать несколько разных обёрток одновременно – результат будет иметь объединённое поведение всех обёрток сразу.

В примере с оповещениями мы оставим в базовом классе простую отправку по электронной почте, а расширенные способы отправки сделаем декораторами.



Расширенные способы оповещения становятся декораторами

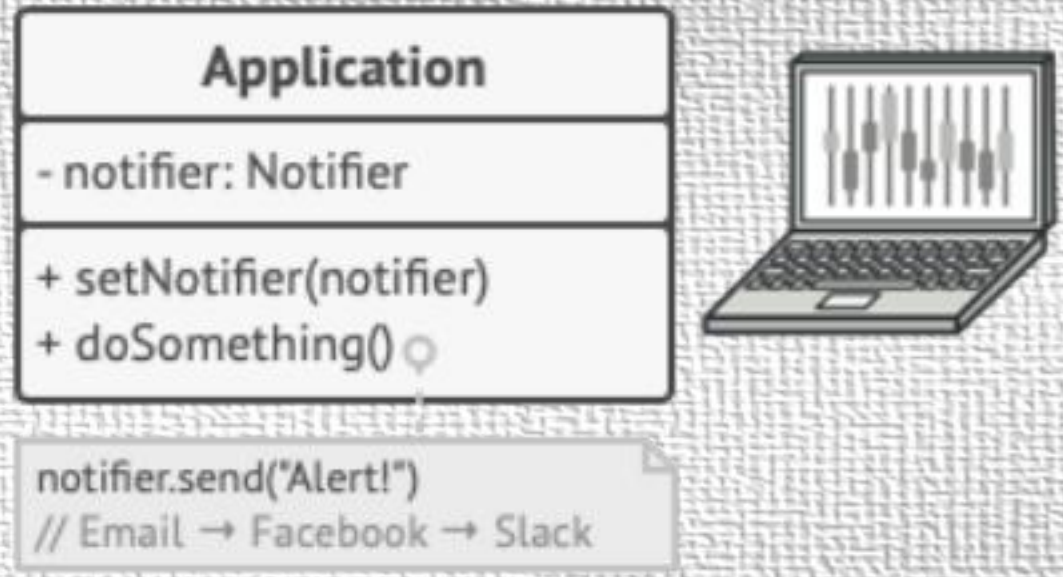
Сторонняя программа, выступающая клиентом, во время первичной настройки будет заворачивать объект оповещений в те обёртки, которые соответствуют желаемому способу оповещения.

Последняя обёртка в списке и будет тем объектом, с которым клиент будет работать в остальное время. Для остального клиентского кода, по сути, ничего не изменится, ведь все обёртки имеют точно такой же интерфейс, что и базовый класс оповещений.

- Таким же образом можно изменять не только способ доставки оповещений, но и форматирование, список адресатов и так далее. К тому же клиент может «дообернуть» объект любыми другими обёртками, когда ему захочется.

```
stack = new Notifier()
if (facebookEnabled)
    stack = new FacebookDecorator(stack)
if (slackEnabled)
    stack = new SlackDecorator(stack)

app.setNotifier(stack)
```




```
notifier.send("Alert!")
// Email → Facebook → Slack
```

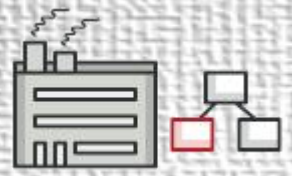
ПОРОЖДАЮЩИЕ ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ

Отвечают за удобное и безопасное создание новых объектов или даже целых семейств объектов.

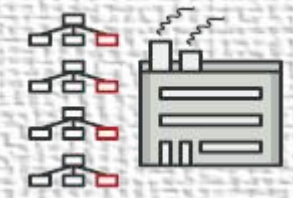
Порождающие паттерны помогают решать задачи с созданием сущностей или групп похожих сущностей. Они убирают лишнее дублирование, делают процесс создания объектов короче и прямолинейнее.

Среди порождающих паттернов мы можем выделить:

- **Фабрика**
- **Фабричный метод**
- **Абстрактная фабрика**
- **Строитель**

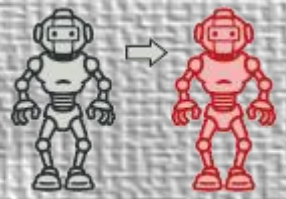


Фабричный метод. Определяет общий интерфейс для создания объектов в суперклассе, позволяя подклассам изменять тип создаваемых объектов.

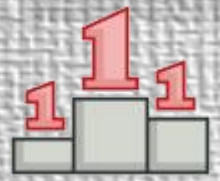


Абстрактная фабрика. Позволяет создавать семейства связанных объектов, не привязываясь к конкретным классам создаваемых объектов.

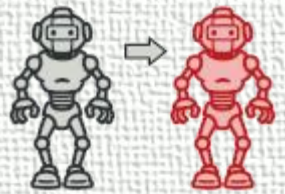
Строитель. Позволяет создавать сложные объекты пошагово. Строитель даёт возможность использовать один и тот же код строительства для получения разных представлений объектов.



Прототип. Позволяет копировать объекты, не вдаваясь в подробности их реализации.



Одиночка. Гарантирует, что у класса есть только один экземпляр, и предоставляет к нему глобальную точку доступа.

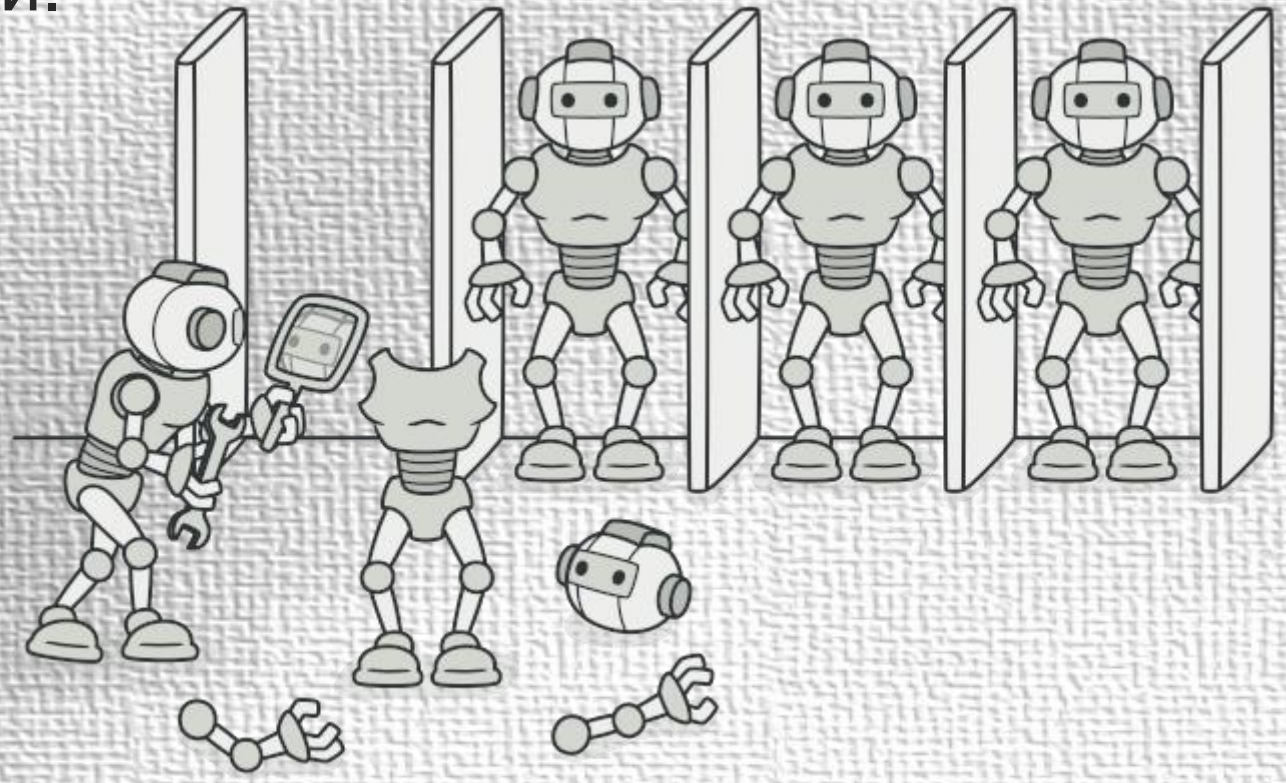


Прототип
(Prototype)

Порождающие паттерны

Суть паттерна

Прототип – это порождающий паттерн проектирования, который позволяет копировать объекты, не вдаваясь в подробности их реализации.

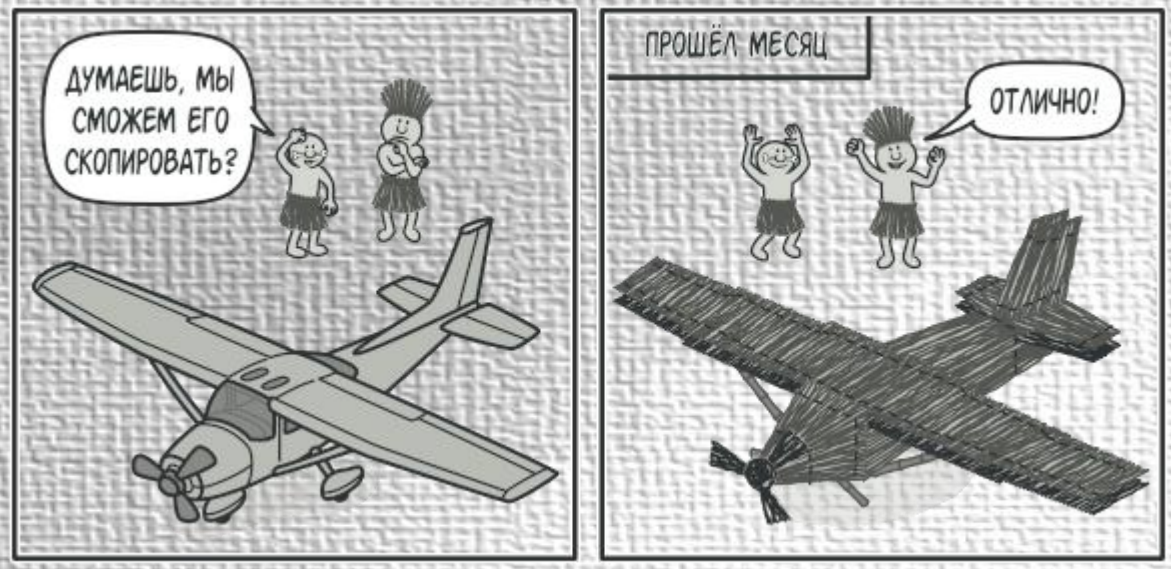


Проблема

У вас есть объект, который нужно скопировать. Как это сделать? Нужно создать пустой объект такого же класса, а затем поочерёдно скопировать значения всех полей из старого объекта в новый.

Прекрасно! Но есть нюанс. Не каждый объект удастся скопировать таким образом, ведь часть его состояния может быть приватной, а значит – недоступной для остального кода программы.

Но есть и другая проблема. Копирующий код станет зависим от классов копируемых объектов. Ведь, чтобы перебрать все поля объекта, нужно привязаться к его классу. Из-за этого вы не сможете копировать объекты, зная только их интерфейсы, а не конкретные классы.



*Копирование «извне» **не всегда** возможно в реальности*

Решение

Паттерн Прототип поручает создание копий самим копируемым объектам. Он вводит общий интерфейс для всех объектов, поддерживающих клонирование. Это позволяет копировать объекты, не привязываясь к их конкретным классам. Обычно такой интерфейс имеет всего один метод clone.

Реализация этого метода в разных классах очень схожа. Метод создаёт новый объект текущего класса и копирует в него значения всех полей собственного объекта. Так получится скопировать даже приватные поля, так как большинство языков программирования разрешает доступ к приватным полям любого объекта текущего класса.

Объект, который копируют, называется *прототипом* (откуда и название паттерна). Когда объекты программы содержат сотни полей и тысячи возможных конфигураций, прототипы могут служить своеобразной альтернативой созданию подклассов.

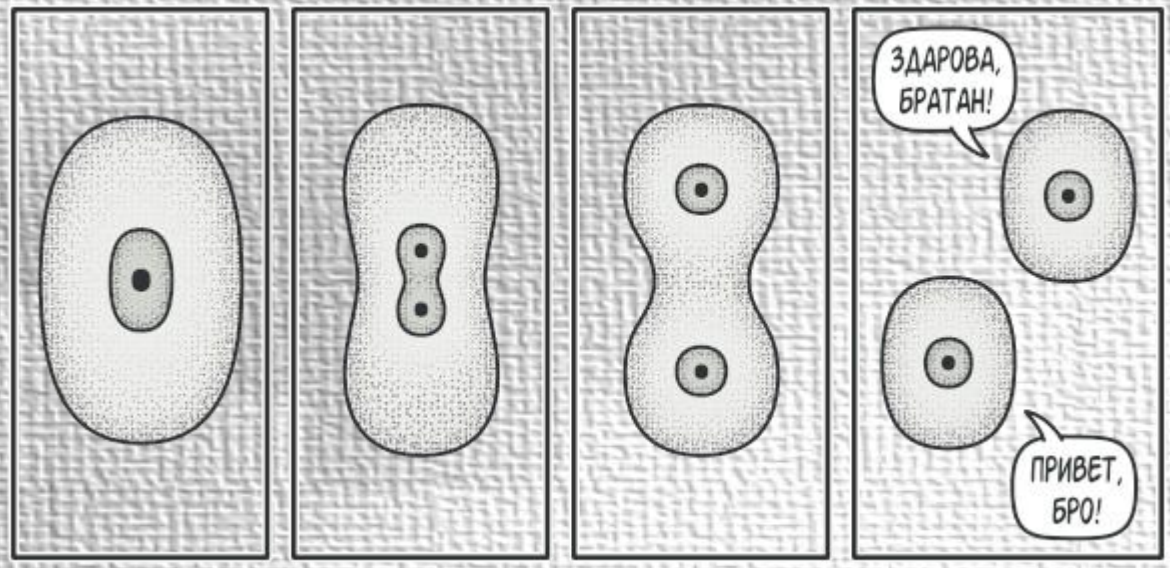
В этом случае все возможные прототипы заготавливаются и настраиваются на этапе инициализации программы. Потом, когда программе нужен новый объект, она создаёт копию из приготовленного прототипа.

Предварительно заготовленные прототипы могут стать заменой подклассам

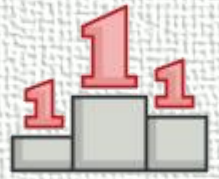
Аналогия из жизни

В промышленном производстве прототипы создаются перед основной партией продуктов для проведения всевозможных испытаний. При этом прототип не участвует в последующем производстве, отыгрывая пассивную роль.

Пример деления клетки



Прототип на производстве не делает копию самого себя, поэтому более близкий пример паттерна – деление клеток. После митозного деления клеток образуются две совершенно идентичные клетки. Оригинальная клетка отыгрывает роль прототипа, принимая активное участие в создании нового объекта.

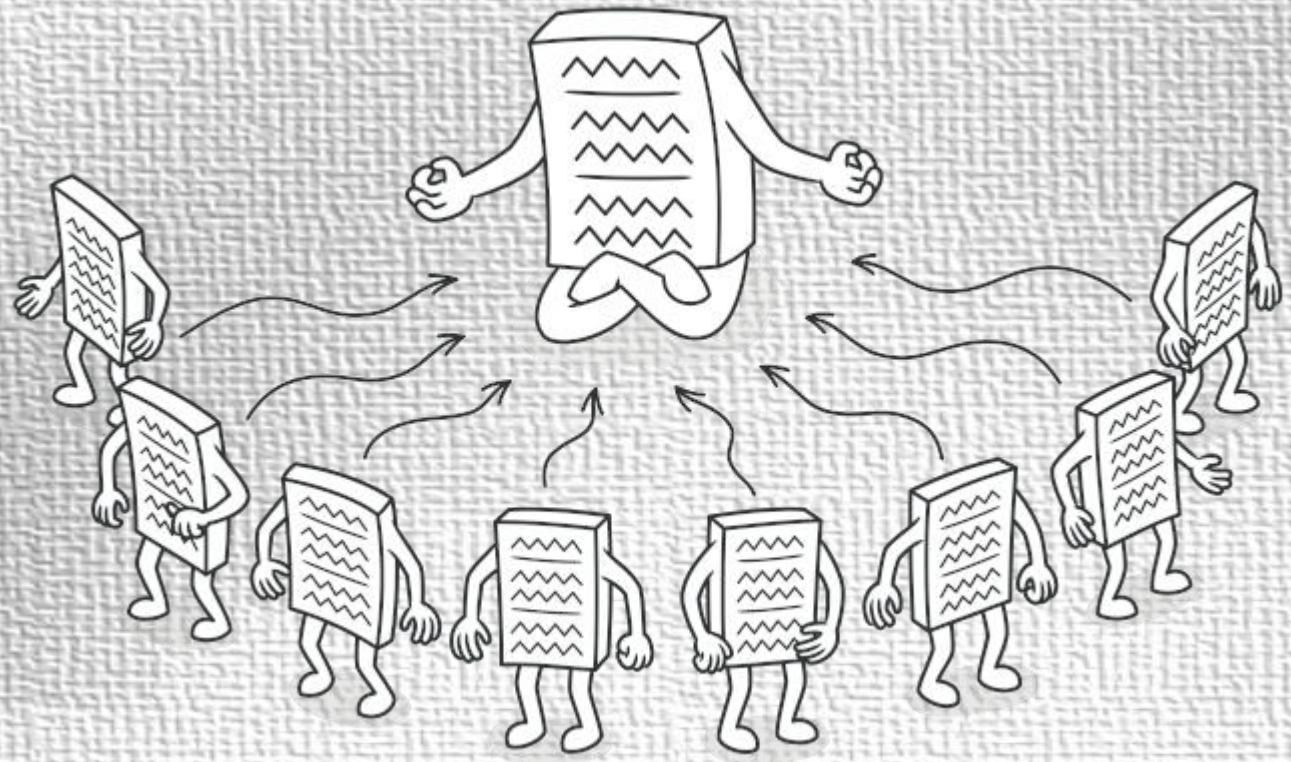


Одиночка
(Singleton)

Порождающие паттерны

Суть паттерна

Одиночка – это порождающий паттерн проектирования, который гарантирует, что у класса есть только один экземпляр, и предоставляет к нему глобальную точку доступа.



Проблема

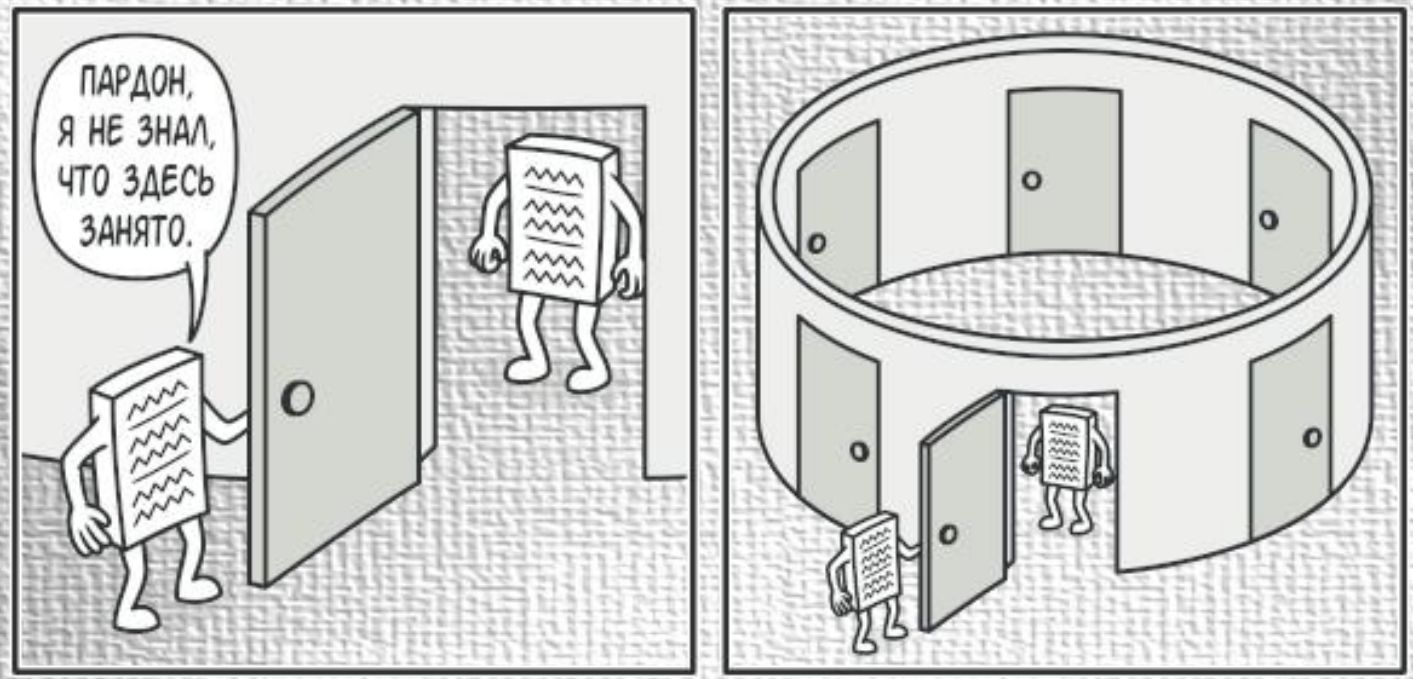
Одиночка решает сразу две проблемы, нарушая *принцип единственной ответственности* класса.

1. Гарантирует наличие единственного экземпляра класса. Чаще всего это полезно для доступа к какому-то общему ресурсу, например, базе данных.

Представьте, что вы создали объект, а через некоторое время пробуете создать ещё один. В этом случае хотелось бы получить старый объект, вместо создания нового.

Такое поведение невозможно реализовать с помощью обычного конструктора, так как конструктор класса **всегда** возвращает новый объект.

Клиенты могут не подозревать, что работают с одним и тем же объектом.



2. Предоставляет глобальную точку доступа. Это не просто глобальная переменная, через которую можно достучаться к определённому объекту. Глобальные переменные не защищены от записи, поэтому любой код может подменять их значения без вашего ведома.

Но есть и другой нюанс. Неплохо бы хранить в одном месте и код, который решает проблему №1, а также иметь к нему простой и доступный интерфейс.

Интересно, что в наше время паттерн стал настолько известен, что теперь люди называют «одиночками» даже те классы, которые решают лишь одну из проблем, перечисленных выше.

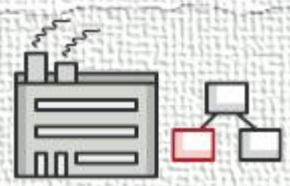
Решение

Все реализации одиночки сводятся к тому, чтобы скрыть конструктор по умолчанию и создать публичный статический метод, который и будет контролировать жизненный цикл объекта-одиночки.

Если у вас есть доступ к классу одиночки, значит, будет доступ и к этому статическому методу. Из какой точки кода вы бы его ни вызвали, он всегда будет отдавать один и тот же объект.

Аналогия из жизни

Правительство государства – хороший пример одиночки. В государстве может быть только одно официальное правительство. Вне зависимости от того, кто конкретно заседает в правительстве, оно имеет глобальную точку доступа «Правительство страны N».



Фабричный метод

(Factory Method)

Порождающие паттерны

Суть паттерна

Фабричный метод – это порождающий паттерн проектирования, который определяет общий интерфейс для создания объектов в суперклассе, позволяя подклассам изменять тип создаваемых объектов.

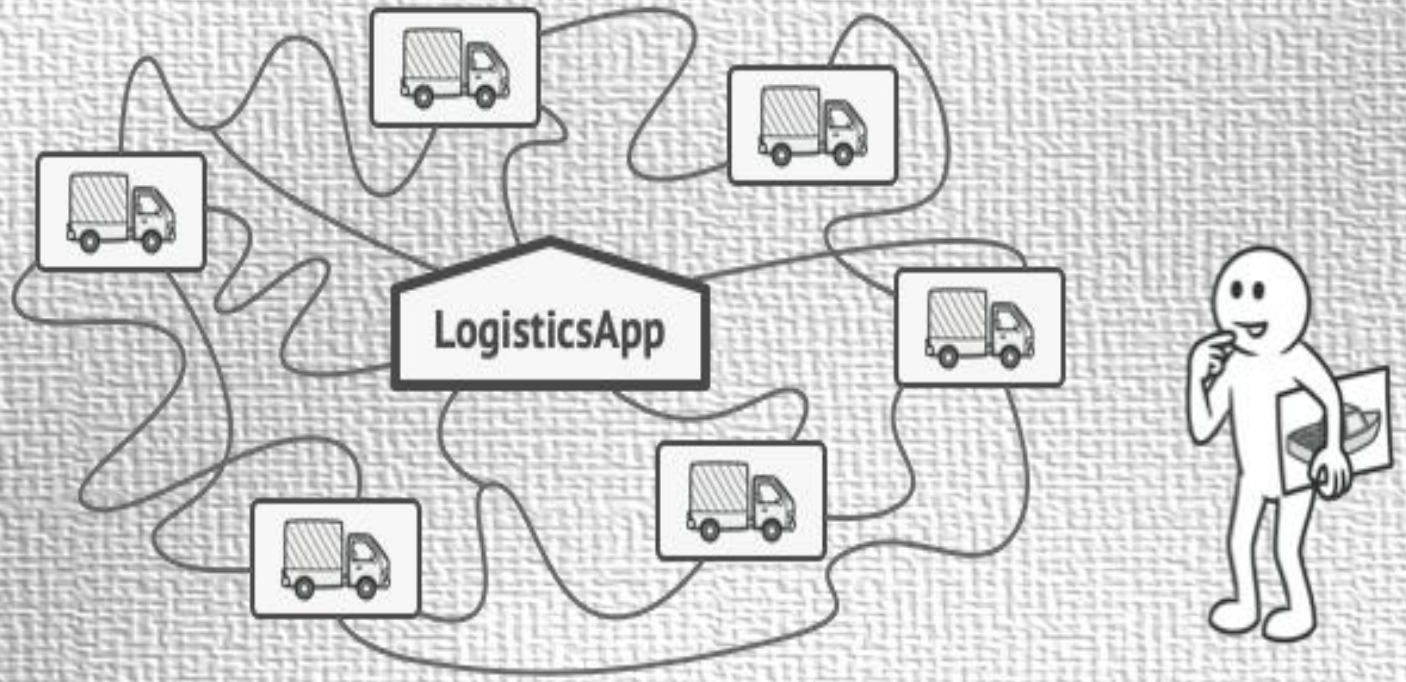
Проблема

Представьте, что вы создаёте программу управления грузовыми перевозками. Сперва вы рассчитываете перевозить товары только на автомобилях. Поэтому весь ваш код работает с объектами класса **Truck** (грузовик).

В какой-то момент ваша программа становится настолько известной, что морские перевозчики выстраиваются в очередь и просят добавить поддержку морской логистики в программу.

Отличные новости, правда?! Но как насчёт кода? Большая часть существующего кода жёстко привязана к классу Truck. Чтобы добавить в программу классы морских судов, понадобится перелопатить всю программу. Более того, если вы потом решите добавить в программу ещё один вид транспорта, то всю эту работу придётся повторить.

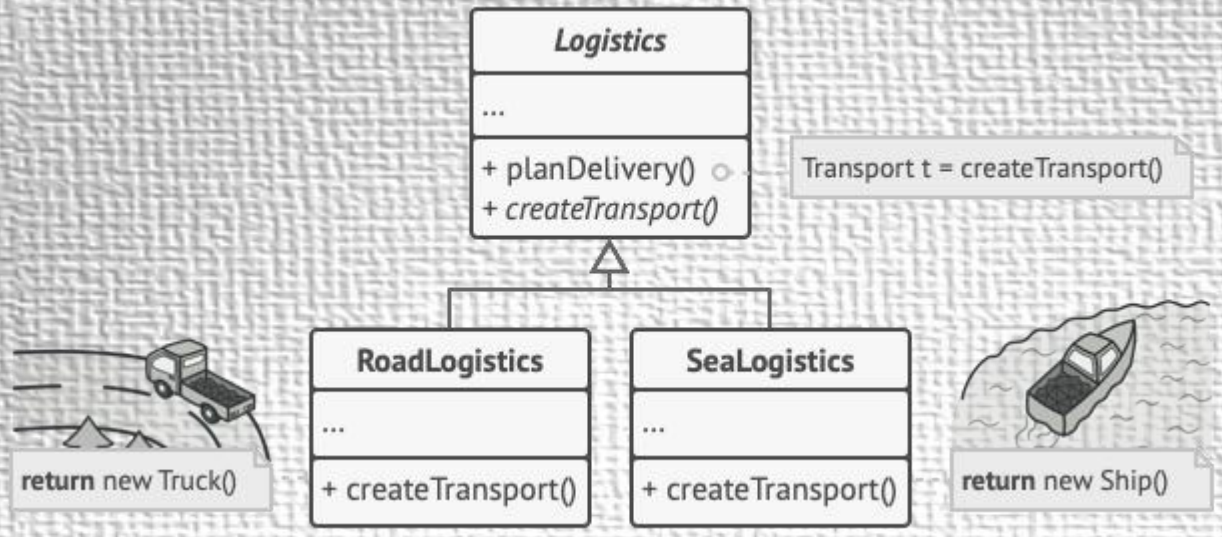
В итоге вы получите ужасающий код, наполненный условными операторами, которые выполняют то или иное действие, в зависимости от класса транспорта.



Добавить новый класс не так-то просто, если весь код уже завязан на конкретные классы.

Решение

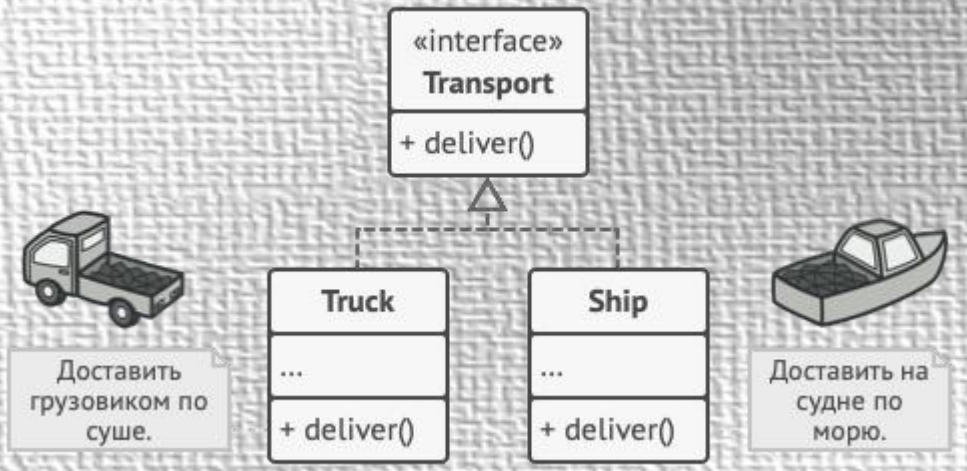
Паттерн Фабричный метод предлагает создавать объекты не напрямую, используя оператор *new*, а через вызов специального фабричного метода. Не пугайтесь, объекты всё равно будут создаваться при помощи *new*, но делать это будет фабричный метод. Объекты, возвращаемые фабричным методом, часто называют продуктами.



Подклассы могут изменять класс создаваемых объектов.

На первый взгляд, это может показаться бессмысленным: мы просто переместили вызов конструктора из одного конца программы в другой. Но теперь вы сможете переопределить в подклассе фабричный метод, чтобы изменить тип создаваемого продукта.

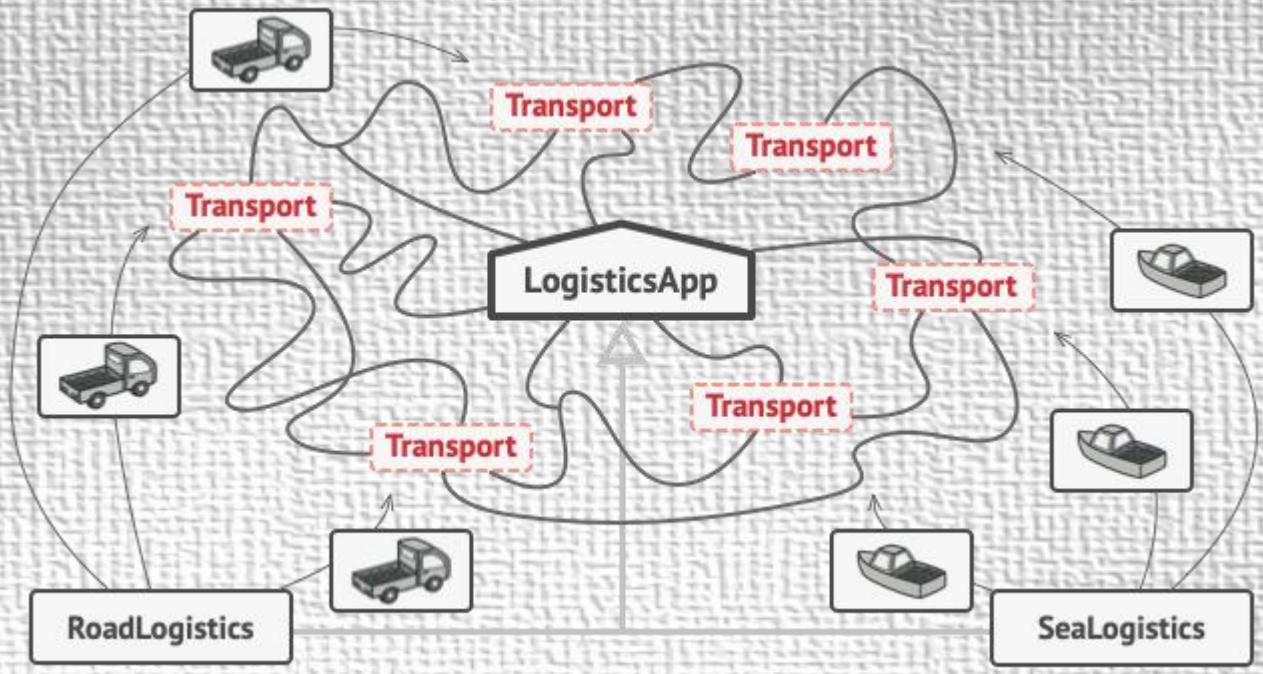
Однако здесь есть небольшое ограничение: подклассы могут возвращать различные типы объектов-продуктов, только если эти объекты имеют общий базовый класс или интерфейс. Кроме того, тип возвращаемого значения для фабричного метода в базовом классе должен быть объявлен как этот интерфейс.



Все объекты-продукты должны иметь общий интерфейс.

Например, классы ***Truck*** (грузовик) и ***Ship*** (судно) реализуют интерфейс ***Transport*** (транспорт) с методом ***deliver()*** (доставить). Каждый из этих классов реализует этот метод по-своему: грузовики везут грузы по земле, а суда – по морю. Фабричный метод в классе ***RoadLogistic*** (дорожная логистика) возвращает объекты-грузовики, а класс ***SeaLogistics*** (морская логистика) – объекты-суда.

Пока все продукты реализуют общий интерфейс, их объекты можно взаимозаменять в клиентском коде.



Код, использующий фабричный метод (часто называемый клиентским кодом), не видит разницы между фактическими продуктами, возвращаемыми различными подклассами. Клиент рассматривает все продукты как объекты абстрактного класса ***Transport*** (транспорт). Клиент знает, что все объекты ***Transport*** должны иметь метод deliver(), но то, как именно он работает, для клиента не имеет значения.

Применимость

Используйте фабричный метод, когда заранее неизвестны типы и зависимости объектов, с которыми должен работать ваш код.

Фабричный метод отделяет код создания продуктов от остального кода, который эти продукты использует.

Благодаря этому, код создания можно расширять, не трогая основной код. Так, чтобы добавить поддержку нового продукта, вам нужно создать новый подкласс создателя и определить в нём фабричный метод, возвращающий экземпляр нового продукта.

Используйте фабричный метод, когда хотите дать возможность пользователям вашего фреймворка или библиотеки расширять его внутренние компоненты.

Наследование, возможно, самый простой способ, чтобы расширить поведение по умолчанию библиотеки или фреймворка. Но как сделать так, чтобы фреймворк понимал, что вместо стандартного компонента нужно использовать ваш подкласс?

Решение состоит в том, чтобы сократить код, который создает компоненты в рамках фреймворка, до единственного фабричного метода и позволить пользователю переопределить этот метод в дополнение к расширению самого компонента.

АНТИ-ШАБЛОНЫ

- в управлении разработкой ПО
- в проектировании ПО
- в программировании
- в объектно-ориентированном программировании

Анти-паттерны (anti-patterns), также известные как **ловушки** (pitfalls) — это классы наиболее часто внедряемых плохих решений проблем. Они изучаются, как категория, в случае когда их хотят избежать в будущем, и некоторые отдельные случаи их могут быть распознаны при изучении неработающих систем.

Были предложены после появления шаблонов проектирования. Анти-паттерны описывают, как не следует поступать при разработке программ, показывая характерные ошибки в дизайне и в реализации.

Анти-паттерн — неудачный, но тем не менее широко распространенный паттерн проектирования программного обеспечения.

Анти-паттерны концептуально похожи на паттерны в том, что они документируют повторяющиеся решения общих проблем. Они известны как анти-паттерны, потому что их использование (или злоупотребления) дает негативные последствия.

Термин, введенный в 1995 году программистом Эндрю Кенигом, был вдохновлен книгой Design Patterns (в которой освещается ряд шаблонов проектирования при разработке программного обеспечения, которые ее авторы считают высоконадежными и эффективными). В другой статье, представленной Майклом Экройдом в 1996 году на конференции Object World West, также были задокументированы анти-паттерны.

Организационные антипаттерны

Проблемы, с которыми встречаются менеджеры (или группы менеджеров):

- **Аналитический паралич** (Analysis paralysis): неоправданно большие затраты на анализ и проектирование. Часто приводит к закрытию проекта до начала его реализации.
- **Дойная корова** (Cash cow): при наличии продукта, приносящего выгоду без существенных вложений, не вкладываются средства в развитие и разработку новых продуктов.
- **Продолжительное устаревание** (Continuous obsolescence): выделение непропорционально больших усилий на портирование системы в новые окружения.
- **Сваливание расходов** (Cost migration): перенос расходов на проект к слабому отделу или бизнес-партнёру.
- **Раздутый улучшизм** (Creeping featurism): добавление новых улучшений в ущерб суммарному качеству системы.
- **Раздутый элегантизм** (Creeping elegance): непропорциональное улучшение красивости кода в ущерб функциональности и суммарному качеству системы.
- …

Анти-паттерны в управлении разработкой ПО

1. Дым и зеркала (Smoke and mirrors): Демонстрация того, как будут выглядеть ненаписанные функции (название происходит от двух излюбленных способов, которыми фокусники скрывают свои секреты)
2. Раздувание ПО (Software bloat): Разрешение последующим версиям системы требовать всё больше и больше ресурсов
3. Функции для галочки: Превращение программы в конгломерат плохо реализованных и не связанных между собой функций (как правило, для того, чтобы заявить в рекламе, что функция есть)

Анти-паттерны в проектировании ПО

1. Инверсия абстракции (Abstraction inversion): Сокрытие части функциональности от внешнего использования, в надежде на то, что никто не будет его использовать
2. Неопределённая точка зрения (Ambiguous viewpoint): Представление модели без спецификации её точки рассмотрения
3. Большой комок грязи (Big ball of mud): Система с нераспознаваемой структурой
4. Блоб (Blob): см. Божественный объект (God object)
5. Бензиновая фабрика (Gas factory): Необязательная сложность дизайна
6. Затычка на ввод данных (Input kludge): Забывчивость в спецификации и выполнении поддержки возможного неверного ввода

- Раздувание интерфейса (Interface bloat): Изготовление интерфейса очень мощным и очень трудным для осуществления
- Волшебная кнопка (Magic pushbutton): Выполнение результатов действий пользователя в виде неподходящего (недостаточно абстрактного) интерфейса. Например, в системах типа Delphi это написание прикладной логики в обработчиках нажатий на кнопку
- Перестыковка (компьютер) (Re-Coupling): Процесс внедрения ненужной зависимости
- Дымоход (Stovepipe system): Редко поддерживаемая сборка плохо связанных компонентов
- Состояние гонки (Race hazard, Race condition): непредвидение возможности наступления событий в порядке, отличном от ожидаемого

- Мышиная возня: Неоправданное создание множества мелких и абстрактных классов для решения одной конкретной задачи более высокого уровня
- Висящие концы: Интерфейс, большинство методов которого бессмысленны и реализуются "пустышками"
- Золушкина туфелька: Попытка "натянуть" на объект уже имеющийся малоподходящий по смыслу интерфейс, вместо создания нового.
- Членовредительство (Mutilation): Излишнее "затачивание" объекта под определенную очень узкую задачу таким образом, что он не способен будет работать с никакими иными, пусть и очень схожими задачами.

Анти-паттерны в программировании

Ненужная сложность (Accidental complexity): Внесение ненужной сложности в решение

Действие на расстоянии (Action at a distance): Неожиданное взаимодействие между широко разделёнными частями системы

Накопить и запустить (Accumulate and fire): Установка параметров подпрограмм в наборе глобальных переменных

Слепая вера (Blind faith): Недостаточная проверка (a) корректности исправления ошибки или (b) результата работы подпрограммы

Лодочный якорь (Boat anchor): Сохранение более не используемой части системы

Активное ожидание (Busy spin): Потребление ресурсов ЦПУ (процессорного времени) во время ожидания события, обычно при помощи постоянно повторяемой проверки, вместо того, чтобы использовать асинхронное программирование (к примеру, систему сообщений или событий)

Кэширование ошибки (Caching failure): Забывать сбросить флаг ошибки после её обработки

Воняющий подгузник (The Diaper Pattern Stinks): Сброс флага ошибки без её обработки или передачи вышестоящему обработчику

Проверка типа вместо интерфейса (Checking type instead of membership, Checking type instead of interface): Проверка того, что объект имеет специфический тип в то время, когда требуется только определённый интерфейс

Инерция кода (Code momentum): Сверхограничение части системы путём постоянного подразумевания её поведения в других частях системы

Кодирование путём исключения (Coding by exception): Добавление нового кода для поддержки каждого специального распознанного случая

Таинственный код (Cryptic code): Использование аббревиатур вместо мнемоничных имён

Жёсткое кодирование (Hard code): Внедрение предположений об окружении системы в слишком большом количестве точек её реализации

Мягкое кодирование (Soft code): Патологическая боязнь жёсткого кодирования, приводящая к тому, что настраивается всё что угодно, при этом конфигурирование системы само по себе превращается в программирование

Поток лавы (Lava flow): Сохранение нежелательного (излишнего или низкокачественного) кода по причине того, что его удаление слишком дорого или будет иметь непредсказуемые последствия

Волшебные числа (Magic numbers): Включение в алгоритмы чисел без объяснений их смысла

Процедурный код (Procedural code): Когда другая парадигма является более подходящей

Спагетти-код (Spaghetti code, иногда «макароны»): Код с чрезмерно запутанным порядком выполнения

Мыльный пузырь (Soap bubbel): Класс, инициализированый мусором, максимально долго притворяется, что содержит какие-то данные

Мютексный ад (Mutex hell): Внедрение слишком большого количества объектов синхронизации между потоками

Анти-паттерны в объектно-ориентированном программировании

Базовый класс-утилита (BaseBean): Наследование функциональности из класса-утилиты вместо делегирования к нему

Вызов предка (CallSuper): Для реализации прикладной функциональности методу класса-потомка требуется в обязательном порядке вызывать те же методы класса-предка

Ошибка пустого подкласса (Empty subclass failure): Создание класса (в Perl), который не проходит «проверку пустоты подкласса» («Empty Subclass Test») из-за различного поведения по сравнению с классом, который наследуется от него без изменений

Божественный объект (God object): Концентрация слишком большого количества функций в одной части системы (классе)

Объектная клоака (Object cesspool): Переиспользование объектов, находящихся в непригодном для переиспользования состоянии

Полтергейст (Poltergeist): Объекты, чьё единственное предназначение — передавать информацию другим объектам

Проблема йо-йо (Yo-yo problem): Чрезмерная размытость сильно связанного кода (например, выполняемого по порядку) по иерархии классов

Одиночество (Singletonitis): Неуместное использование паттерна одиночка

Приватизация (Privatisation): Сокрытие функциональности в приватной секции (private), что затрудняет его расширение в классах-потомках

Френд-зона (Friend zone): Неуместное использование дружественных классов и дружественных функций в языке c++

Методологические анти-паттерны

Программирование методом копирования-вставки (Copy and paste programming): Копирование (и лёгкая модификация) существующего кода вместо создания общих решений

Дефакторинг (De-Factoring): Процесс уничтожения функциональности и замены её документацией

Золотой молоток (Golden hammer): Сильная уверенность в том, что любимое решение универсально применимо. Название происходит от поговорки «когда в руках молоток, все проблемы кажутся гвоздями»

Фактор невероятности (Improbability factor): Предположение о невозможности того, что сработает известная ошибка

Преждевременная оптимизация (Premature optimization): Оптимизация на основе недостаточной информации

Изобретение колеса (Reinventing the wheel): Создание с нуля, вместо использования готового решения

Изобретение квадратного колеса (Reinventing the square wheel): Создание плохого решения, когда существует хорошее известное решение

Самоуничтожение (Self-destruction): Фатальная ошибка либо нестандартное поведение программы, приводящая к отказу в обслуживании, возникшая вследствие другой менее серьёзной ошибки. Например, при возникновении ошибки, приложение начинает очень быстро и много писать в лог, вследствие чего заканчивается место на жёстком диске быстрее, чем это обнаружит мониторинг.

Два тоннеля: Вынесение новой функциональности в отдельное приложение вместо расширения уже имеющегося. Чаще всего применяется, когда по каким-либо причинам (в основном, при нехватке времени либо нежелании менеджмента) внесение изменений в уже имеющийся код требует больших затрат, чем создание нового. При этом у клиента в конечном итоге работают два приложения, запускаясь одновременно либо попеременно друг из друга.

Анти-паттерн «Golden Hammer»

Золотой молоток — уверенность в полной универсальности какого-либо решения. На практике, это применение одного решения (например одного из паттернов проектирования) для всех возможных и невозможных задач. Анти-паттерн «золотой молоток» может появляться как на управленческом уровне, так и на уровне разработчиков, суть анти-паттерна от этого не меняется.

Программист «влюбляется» в какой-то из паттернов и начинает использовать его везде, где это возможно, как универсальное решение любых проблем.

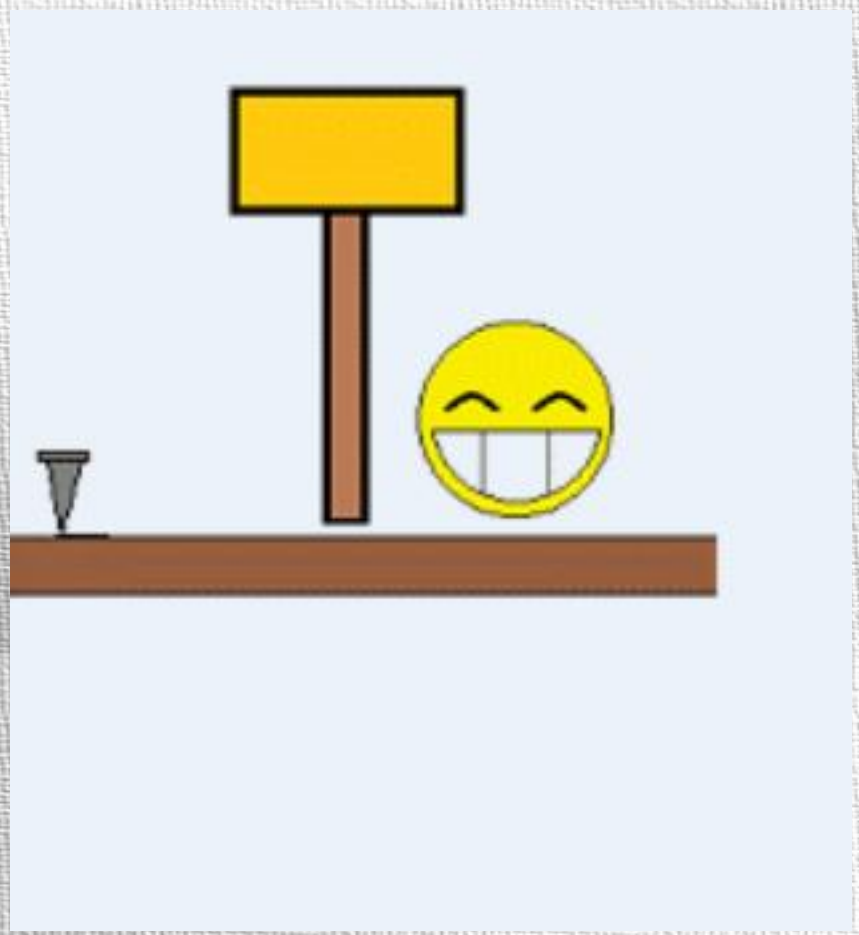
Золотой молоток также встречается и среди новичков. В данном случае по другим причинам — новичку лень изучать что-то новое, и он применяет единственный освоенный метод для всех встречающихся ему задач.

Можно выделить ряд причины возникновения:

- Незнание альтернативных способов решения.
- Нежелание тратить время на анализ альтернатив.
- Привычка и «автоматизм» среди профессионалов.

Следствиями являются: неэффективное и неоптимальное решение, которое снаружи выглядит «красиво»; ненужное усложнение (или упрощение) системы.

Если у вас в руках молоток, ты вы напрасно думаете, что вокруг одни гвозди



Анимация идеи «Золотого молотка». Молоток успешно применяется для решения задачи — забивания гвоздей. Чтобы предотвратить его повсеместное использование применяются техники:

- прототипирование,
- просмотр (анализ) исходного кода,
- рефакторинг,
- анализ альтернативных решений.

Способы борьбы с золотым молотком

Способы предотвращения:

- ***Анализ наличия альтернативных решений***, поиск и сравнение других способов решения поставленной задачи. Например разработчики программного обеспечения должны следить за современными технологическими тенденциями. Это касается как и непосредственно разработчиков ПО, так и управляющего менеджмента. Хорошим способом поддержания обмена знаниями о технических новинках, является практика организации дискуссионных совещаний, на которых будут обсуждаться новые технологии (шаблоны, появляющиеся стандарты, новые продукты).
- **Прототипирование**, сравнение различных способов решения поставленной задачи, путем создания прототипов.